

第1章 单片机概述



第1章 单片机概述

1.1 单片机简介

1.2 单片机的发展历史

1.3 单片机的特点

1.4 单片机的应用

1.5 单片机的发展趋势

1.6 MCS-51系列与AT89S5x系列单片机

1.6.1 MCS-51系列单片机

1.6.2 AT89S5x系列单片机简介

1.7 各种衍生品种的8051单片机

1.7.1 STC系列单片机

1.7.2 C8051F×××单片机

1.7.3 AD μ C812单片机11

1.7.4 华邦W77系列、W78系列单片机

1.8 PIC系列单片机与AVR系列单片机

1.8.1 PIC系列单片机

1.8.2 AVR系列单片机

1.9 其他的嵌入式处理器简介

1.9.1 嵌入式DSP处理器

1.9.2 嵌入式微处理器

内容概要

本章介绍单片机基础知识。Intel公司的8051单片机被世界许多厂商作为基核，先后推出多种兼容机型，在世界范围内得到广泛应用，已成为国内外公认的标准体系结构。

在众多的兼容机型中，美国ATMEL公司的AT89S5x系列中的增强型AT89S52单片机非常适合作为单片机初学者的入门机型。

本章除了对AT89S51/52单片机作以简单介绍外，还对嵌入式处理器家族中的DSP、嵌入式微处理器进行概括性介绍，使读者初步了解，为后续学习打下基础。

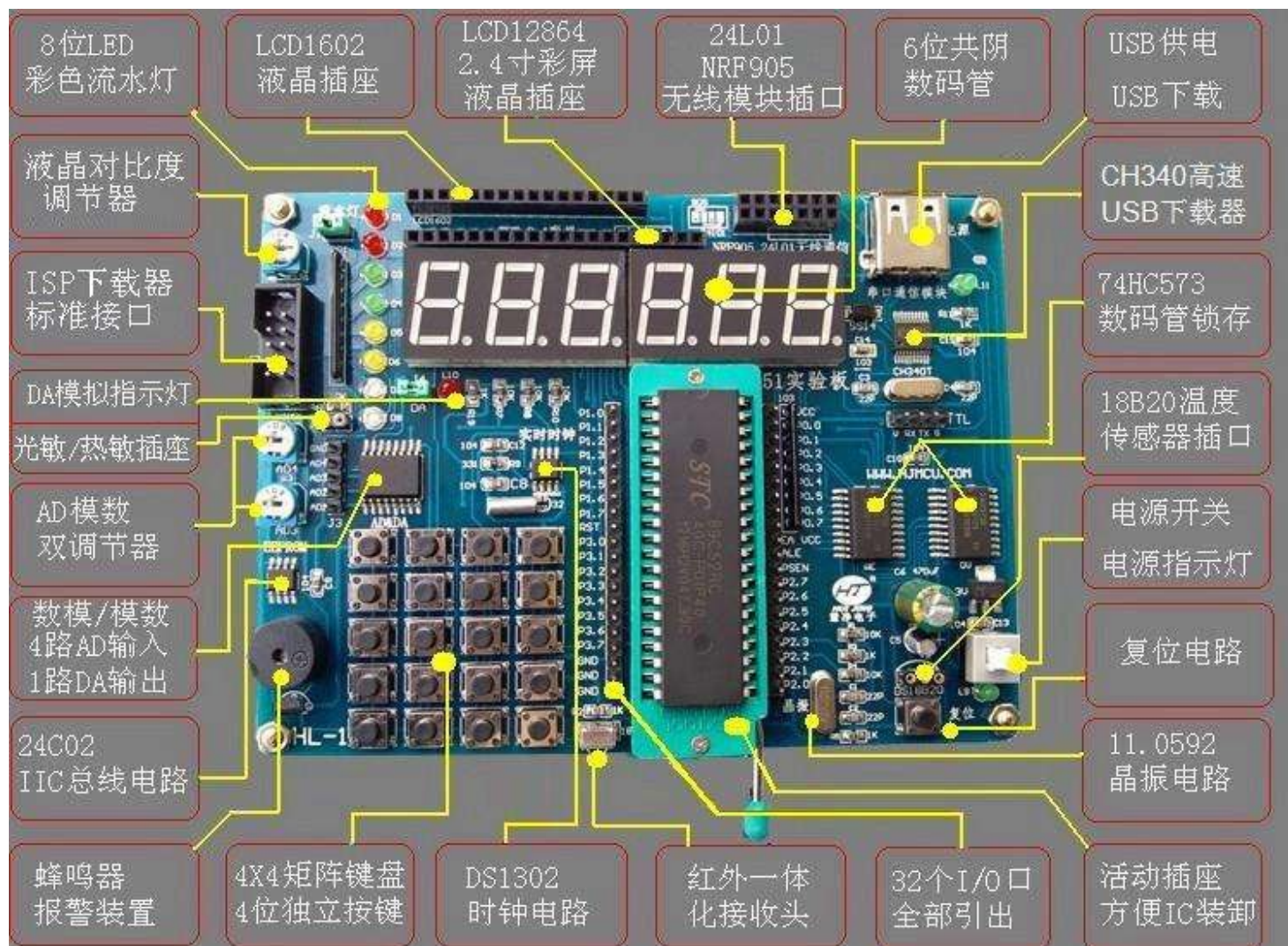
20世纪70年代问世，广泛地应用在工业自动化、自动检测与控制、智能仪器仪表、机电一体化设备、汽车电子、家用电器等各个方面。什么是单片机？

1.1 单片机简介

一片半导体硅片集成：中央处理单元（CPU）、存储器（RAM、ROM）、并行I/O、串行I/O、定时器/计数器、中断系统、系统时钟电路及系统总线。

用于测控领域的单片微型计算机，简称**单片机**。

单片机示例



单片机使用时，通常是处于测控系统的核心地位并嵌入其中，所以国际上通常把单片机称为嵌入式控制器（EMCU, Embedded MicroController Unit），或微控制器（MCU, MicroController Unit）。

我国习惯于使用“单片机”这一名称。

单片机的问世是计算机技术发展史上的一个重要里程碑，标志着计算机正式形成了通用计算机系统和嵌入式计算机系统两大分支。

单片机体积小、成本低，嵌入到工业控制单元、机器人、智能仪器仪表、汽车电子系统、武器系统、家用电器、办公自动化设备、金融电子系统、玩具、个人信息终端及通讯产品中。

按照其用途可分为通用型和专用型两大类。

(1) 通用型单片机就是其内部可开发的资源（如存储器、I/O等各种外围功能部件等）可以全部提供给用户。

用户根据需要，设计一个以通用单片机芯片为核心，再配以外围接口电路及其它外围设备，并编写相应的软件来满足各种不同需要的测控系统。通常所说的和本书介绍的是指通用型单片机。

(2) 专用型单片机是专门针对某些产品的特定用途而制作的单片机。

例如，各种家用电器中的控制器等。由于用于特定用途，单片机芯片制造商常与产品厂家合作，设计和生产“专用”的单片机芯片。

由于在设计中，已经对“专用”单片机的系统结构最简化、可靠性和成本的最佳化等方面都做了全面的综合考虑，所以“专用”单片机具有十分明显的综合优势。

无论“专用”单片机在用途上有多么“专”，其基本结构和工作原理都是以通用单片机为基础的。

1.2 单片机的发展历史

单片机按其处理的**二进制位数**主要分为：**4位**单片机、**8位**单片机、**16位**单片机和**32位**单片机。

发展大致分为**4个阶段**。

第一阶段（1974年～1976年）：单片机**初级阶段**。因工艺限制，单片机采用双片的形式而且功能比较简单。1974年12月，仙童公司推出了8位的F8单片机，实际上只包括了8位CPU、64B RAM和2个并行口。

第二阶段（1976年～1978年）：**低性能单片机阶段**。1976年Intel的MCS-48单片机（8位）极大地促进了单片机的变革和发展，**1977年GI公司**

推出了PIC1650，但这个阶段仍处于低性能阶段。

第三阶段（1978年～1983年）：**高性能单片机阶段**。1978年，Zilog公司推出Z8单片机，1980年，Intel公司在MCS-48系列基础上推出MCS-51系列，Mortorola推出6801单片机。使单片机的性能及应用跃上新的台阶。

此后，各公司的8位单片机迅速发展。推出的单片机普遍带有串行I/O口、多级中断系统、16位定时器/计数器，片内ROM、RAM容量加大，且寻址范围可达64KB，有的片内还带有A/D转换器。由于这类单片机的性能价格比高，所以被广泛应用，是目前应用数量最多的单片机。

第四阶段（1983年～现在）：**8位单片机巩固发展及16位单片机、32位单片机推出阶段。**

20世纪90年代单片机制造业大发展时期，Mortorola、Intel、ATMEL、德州仪器（TI）、三菱、日立、飞利浦、LG等公司开发一大批性能优越的单片机，极大推动单片机的应用。近年，又有不少**新型的高集成度的单片机产品**涌现出来，出现了产品丰富多彩的局面。

目前，除**8位**单片机得到广泛应用外，**16位**单片机、**32位**单片机也得到广大用户青睐。

1.3 单片机的特点

是集成电路技术与微型计算机技术高速发展的产物。体积小、价格低、应用方便、稳定可靠，因此，给工业自动化等领域带来了一场重大革命和技术进步。

由于体积小，很容易地嵌入到系统之中，以实现各种方式的检测、计算或控制，这一点，一般微机根本做不到。

由于单片机本身就是一个微型计算机，因此只要在单片机的外部适当增加一些必要的外围扩展电路，就可以灵活地构成各种应用系统，如工业自动检测监视系统、数据采集系统、自动控制系统、智能仪器仪表等。

为什么**应用广泛**？具有**以下优点**：

- (1) **简单方便，易掌握和普及**。单片机应用系统设计、组装、调试已经是一件容易的事情，广大工程技术人员通过学习可很快地掌握其应用设计与调试技术。
- (2) **功能齐全，应用可靠，抗干扰能力强**。
- (3) **发展迅速，前景广阔**。短短几十年，单片机经过**4位机、8位机、16位机、32位机**等几大发展阶段。尤其是**集成度高、功能日臻完善的单片机**不断问世，使单片机在工业控制及工业自动化领域获得长足发展和大量应用。目前，单片机内部结构愈加完美，片内外围功能部件越来越完善，向**更高层次和更大规模**的发展奠定坚实的基础。

(4) 嵌入容易，用途广泛，体积小、性能价格比高，应用灵活性强等特点在嵌入式微控制系统中具有十分重要的地位。

单片机出现前，制作一套测控系统，大量的模拟电路、数字电路、分立元件完成，以实现计算、判断和控制功能。系统的体积庞大，线路复杂，连接点多，易出现故障。

单片机出现后，测控功能的绝大部分由单片机的软件程序实现，其它电子线路则由片内的外围功能部件来替代。

1.4 单片机的应用

软硬件结合、体积小，容易嵌入到各种应用系统中。得到广泛应用。

1. 工业检测与控制

主要应用：工业过程控制、智能控制、设备控制、数据采集和传输、测试、测量、监控等。在工业自动化的领域中，**机电一体化技术**将发挥愈来愈重要的作用，在这种集机械、微电子和计算机技术为一体的综合技术（如**机器人技术**）中，单片机发挥着非常重要的作用。

2. 仪器仪表

目前对仪器仪表的**自动化**和**智能化**要求越来越高。单片机的使用有助于提高仪器仪表的精度和准确度，简化结构，减小体积而易于携带和使用，**加速仪器仪表向数字化、智能化、多功能化方向发展**。

3. 消费类电子产品

例如，洗衣机、电冰箱、空调机、电风扇、电视机、微波炉、加湿机、消毒柜等。嵌入了单片机后，功能和性能大大提高，并实现智能化、最优化控制。

4. 通讯

在调制解调器、各类手机、传真机、程控电话交换机、信息网络及各种通讯设备中，单片机也已经得到广泛应用。

5. 武器装备

在**现代化的武器装备**中，如飞机、军舰、坦克、导弹、鱼雷制导、智能武器装备、航天飞机导航系统，都有单片机嵌入其中。

6. 各种终端及计算机外部设备

计算机网络终端（如银行终端）以及计算机外部设备（如打印机、硬盘驱动器、绘图机、传真机、复印机等）中都使用了单片机作为控制器。

7. 汽车电子设备

已经广泛地应用在各种汽车电子设备中，如汽车安全系统、汽车信息系统、智能自动驾驶系统、卫星汽车导航系统、汽车紧急请求服务系统、汽车防撞监控系统、汽车自动诊断系统以及汽车黑匣子等。

8. 分布式多机系统

在较复杂多节点的测控系统中，常采用分布式多机系统。

一般由若干台功能各异的单片机组成，各自完成特定的任务，它们通过串行通信相互联系、协调工作。在这种系统中，单片机往往作为一个终端机，安装在系统的某些节点上，对现场信息进行实时的测量和控制。

从工业自动化、自动控制、智能仪器仪表、消费类电子产品等方面，直到国防尖端技术领域，单片机都发挥着十分重要的作用。

单片机市场每年销售量超过50亿片，其中8位单片机的年销售量达到30亿片

研究表明：一个消费者一天接触到的物品中，包括至少100个嵌入式单片机系统：从微波炉、吹风机、无绳电话、冰箱、洗衣机到汽车、微机、中英空调系统、智能门禁、安防系统等众多产品，都加入了嵌入式单片机来增强可靠性、改善性能、保证安全、提高产品灵活性。

应用举例1

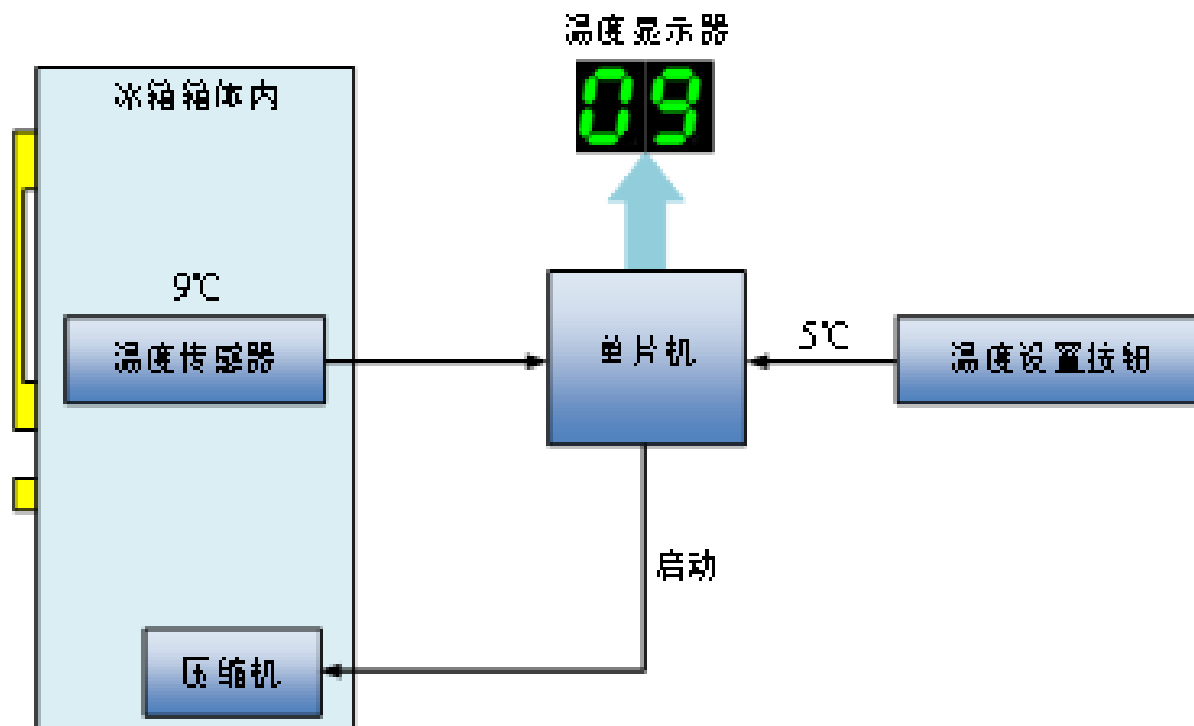


图 1-1 电冰箱与单片机

应用举例2

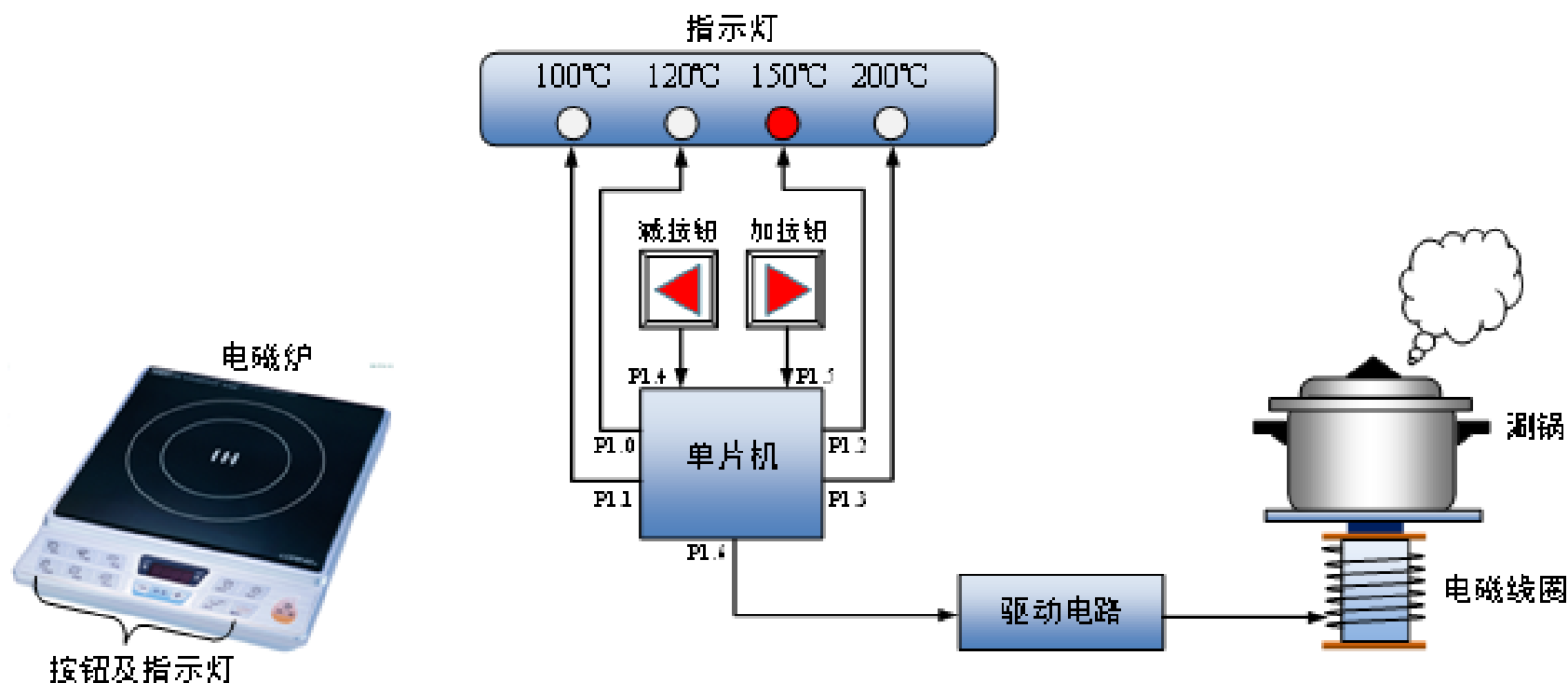


图 1-2 电磁炉与单片机

应用举例3

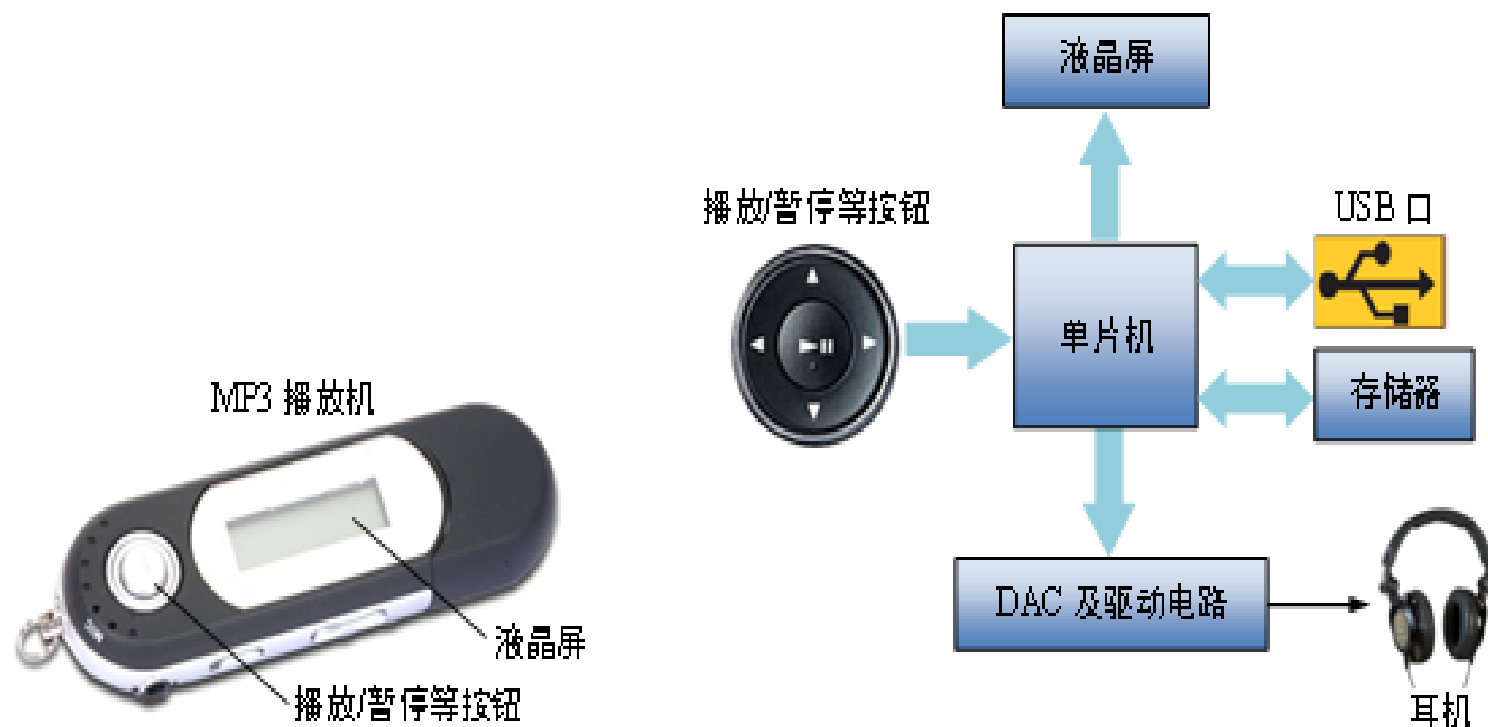


图 1-3 MP3 播放机与单片机

1.5 单片机的发展趋势

单片机的发展趋势将是向大容量、高性能化，外围电路内装化等方面发展。

1. CPU的改进

- (1) 增加CPU数据总线宽度。例如，各种16位单片机和32位单片机，数据处理能力要优于8位单片机。
- (2) 采用双CPU结构，以提高数据处理能力。

2. 存储器的发展

- (1) 片内程序存储器普遍采用闪烁（Flash）存储器。可不用外扩展程序存储器，简化系统结构。

(2) **加大存储容量**。目前有的单片机片内程序存储器容量可达128KB甚至更多。例如8位单片机PIC18F452片内集成了4KB的RAM，以满足动态数据存储的需要。

3. 片内I/O的改进

- (1) 增加并行口**驱动能力**，以减少外部驱动芯片。有的单片机可以直接输出大电流和高电压，以便能直接驱动LED和VFD（荧光显示器）。
- (2) 有些单片机设置了一些**特殊的串行I/O功能**，为**构成分布式、网络化系统**提供方便条件。
- (3) 引入了数字交叉开关，改变了以往片内外设与外部I/O引脚的固定对应关系。

4. 低功耗

CMOS化，功耗小，配置有等待状态、睡眠状态、关闭状态等工作方式。

消耗电流仅在 μA 或 nA 量级，适于电池供电的便携式、手持式的仪器仪表以及其它消费类电子产品。

5. 外围电路内装化

众多外围电路全部装入片内，即系统的单片化是目前发展趋势之一。一片芯片就是一个“测控”系统。

6. 编程及仿真的简单化

目前大多数的单片机都支持程序在系统编程ISP（In System Program），也称在线编程，只需一条与PC机USB口（或串行口）相连的ISP

下载线，就可把仿真调试通过的程序代码从PC机在线写入单片机的Flash存储器内，省去编程器与仿真器。某些机型还支持在线应用编程IAP，即可在线升级或销毁单片机的应用程序。

7. 实时操作系统的使用

单片机可配置实时操作系统**RTX51**，简化了对实时事件反应速度要求较高的复杂系统设计、编程和调试，已完全集成到C51编译器中，使用简单方便。

综上所述，单片机正在向多功能、高性能、高速度（时钟达40MHz）、低电压（2.7V即可工作）、低功耗、低价格（几元钱）、外围电路内装化以及片内程序存储器和数据存储器容量不断增大的方向发展。

1.6 MCS-51系列与AT89C5x系列单片机

20世纪80年代以来，单片机的发展非常迅速，其中Intel公司推出的MCS-51系列单片机是一款设计成功、易掌握并在世界范围得到广泛普及应用的机型。

1.6.1 MCS-51系列单片机

MCS是Intel公司单片机的系列符号。MCS-51系列是在MCS-48系列基础上于20世纪80年代初发展起来的，在我国得到广泛应用的单片机主流品种。

基本型产品主要包括8031、8051、8751（对应的CMOS工艺的低功耗型为80C31、80C51、87C51）和**增强型**产品8032、8052、8752。

1. 基本型

典型产品：8031/8051/8751。

8031内部包括1个8位CPU、128B RAM，21个特殊功能寄存器（SFR）、4个8位并行I/O口、1个全双工串行口，2个16位定时器/计数器，5个中断源，但片内无程序存储器，需外扩程序存储器芯片。

8051是在8031的基础上，片内又集成有4KB ROM作为程序存储器。是一个程序不超过4KB的小系统。ROM内的程序是制作芯片时，代为用户烧制的。主要用在程序已定且批量大的产品中。

8751与8051相比，片内的4KB EPROM取代了8051的4KB ROM来作为程序存储器。

2. 增强型

典型产品：8032/8052/8752。**内部RAM**增到256B，8052、**片内程序存储器**扩展到8KB，**16位定时器/计数器**增至3个，6个中断源，串行口通信速率提高5倍。

表1-1列出了基本型和增强型的MCS-51系列单片机片内的基本硬件资源。

1. 6. 2 AT89C5x (AT89S5x) 系列单片机

1. 8051内核单片机

20世纪80年代中期以后，**Intel**精力集中在高档CPU芯片的开发、研制上，淡出单片机芯片的开发和生产。

表 1-1 MCS-51 系列单片机的片内硬件资源

	型号	片内 程序存储器	片内数据存储器 (B)	I/O 口线 (位)	定时器/计数器 (个)	中断源个数 (个)
基本型	8031	无	128	32	2	5
	8051	4KB ROM	128	32	2	5
	8751	4KB EPROM	128	32	2	5
增强型	8032	无	256	32	3	6
	8052	8KB ROM	256	32	3	6
	8752	8KB EPROM	256	32	3	6

MCS-51系列**设计上的成功**，以及**较高的市场占有率**，已成为许多厂家、电气公司竞相选用的对象。

Intel公司以**专利**形式把**8051内核技术转让给**ATMEL、Philips、CygnaI、ANALOG、LG、ADI、Maxim、DALLAS等公司。

目前世界其它公司推出的各种与8051兼容、扩展型单片机都是在8051内核的基础上进行了功能模块的增加与扩展，使其集成度更高，功能和市场竞争力更强。人们常用8051（或80C51，“C”表示采用CMOS工艺）来称呼所有这些具有8051内核，且使用8051指令系统的单片机，统称为8051单片机或简称为51单片机。

2. AT89S5x系列单片机

在众多的**衍生机型**中，ATMEL公司的AT89C5x/AT89S5x系列，尤其是AT89C51/AT89S51和AT89C52/AT89S52在8位单片机市场中**占有较大的市场份额**。

ATMEL公司的AT89C5x/AT89S5x系列与MCS-51系列**在原有功能、引脚以及指令系统方面完全兼容**。

系列中某些品种**又增加了一些新的功能**，如**看门狗定时器WDT、ISP**（在系统编程也称在线编程）。片内Flash存储器可直接在线（ISP）重复编程。此外，还支持两种节电工作方式，非常适合于电池供电或其他低功耗场合。

AT89S5x的“S”表示含有串行下载的Flash存储器。**AT89C51**单片机已不再生产，可用**AT89S51直接代换**。AT89S51片内集成有双数据指针DPTR，看门狗定时器，具有低功耗的空闲工作方式和掉电工作方式，还增加了5个特殊功能寄存器。

AT89S51与AT89S52差别在于AT89S51片内有4KB Flash存储器、128B的RAM、5个中断源以及2个定时器/计数器；而AT89S52片内有8KB的Flash程序存储器、256B的RAM、6个中断源、3个定时器（比AT89S51多出1个定时器，且具有捕捉功能）。AT89S52在片内硬件资源方面对AT89S51做了“增强”。AT89S51/52单片机是目前广泛应用的8051单片机的代表性机型。

本书中经常用到“8051”，它是泛指世界各芯片厂商生产的具有8051内核的各种增强型、扩展型的单片机。而“AT89S51”或“AT89S52”仅指ATMEL公司的单片机产品。

除了8位单片机得到广泛应用外，一些厂家的16位单片机也得到了用户的青睐。例如，美国TI公司的16位的MSP430系列，Microchip公司的PIC24xx系列单片机。这些单片机本身带有A/D转换器，各种串行口以及各种数字控制与检测部件，一片芯片就构成了一个小的测控系统，使用非常方便。

除了16位单片机外，各公司还推出了32位单片机，例如，Microchip公司的PIC32xx系列单片机，意法半导体的STM32xx单片机。尽管如此，8位单片机的性能基本能满足中低端应用的大部分实际需求，况且8位单片机价格十分低廉，性价比高，在中、低端应用场合还是得到了广泛应用。

1.7 各种衍生品种的8051单片机

除了AT89S5x系列单片机外，世界各半导体器件厂家也推出8051内核、各种集成度高、功能强的增强扩展型单片机，已得到广泛应用。

1.7.1 STC系列单片机

STC系列单片机是我国具有自主知识产权，功能与抗干扰性强的增强型8051单片机，多种子系列，几百个品种。其中STC12C5410/STC12C2052系列机型主要性能及特点如下。

(1) 高速：普通的8051单片机为每个机器周期为12个时钟，

而STC单片机可以为每机器周期1个时钟，指令执行速度大大提高，速度比普通的8051快8~12倍。

(2) 宽工作电压：5.5~3.8V，2.4~3.8V（STC12LE5410AD系列）。

(3) 12KB/10KB/8KB/6KB/4KB片内Flash程序存储器，擦写次数10万次以上。

(4) 512B片内的RAM数据存储器。

(5) 可在线编程（ISP）/在应用可编程（IAP），无需编程器/仿真器，可远程升级。

(6) 8通道的10位ADC，4路PWM输出。

(7) 4通道捕捉/比较单元，也可用来再实现4个定时器或4个外部中断（支持上升沿/下降沿中断）。

- (8) 2个硬件16位定时器，兼容普通8051的定时器。4路可编程计数/定时器阵列（PCA）还可再实现4个定时器。
- (9) 硬件看门狗（WDT）。
- (10) 高速SPI串口。
- (11) 全双工异步串行口(UART)，兼容普通8051的串口。
- (12) 通用I/O口（27/23/15个）中的每个I/O口驱动能力均可达到20mA，但整个芯片最大不可超过55mA。
- (13) 超强抗干扰能力与高可靠性：

(14) 采取降低单片机时钟对外部电磁辐射的措施：如选每个机器周期为6个时钟，外部时钟频率可降一半；

(15) 超低功耗设计：

STC单片机可直接替换世界各公司的8051/52的兼容机型产品，是一款高性能、高可靠性且价格低廉的机型。

1.7.2 C8051Fxxx单片机

美国Cygnal公司产品，集成度高，8051内核。代表性产品为C8051F020。

内部采用流水线结构，大部分指令的完成时间为1或2个时钟周期，峰值处理能力为25MIPS。与经典的8051单片机相比，可靠性和速度有很大提高。

片内集成了：1个8位ADC、1个12位ADC、1个双12位DAC；64kB片内Flash程序存储器，256B RAM，128B SFR；8个I / O端口共64根I / O口线；5个16位通用定时器；5个捕捉 / 比较模块的可编程计数 / 定时器阵列

(PCA)，1个UART串行口、1个SMBus/ I²C串口、1个SPI串行口；2路电压比较器，VDD监视器(电源监测)，**内置温度传感器**。

C8051Fxxx单片机**最突出的改进是引入了数字交叉开关**。

用户可通过可编程的交叉开关控制寄存器将片内的计数器/定时器、串行总线、硬件中断、ADC转换器输入、比较器输出以及单片机内部的其他硬件外设配置出现在端口I/O引脚。用户可根据特定应用，选择通用I/O端口与片内硬件资源的灵活组合。

1.7.3 AD μ C812单片机

AD μ C812是美国ADI（Analog Device Inc）公司生产的高性能8051内核单片机，其内部包含高精度自校准的8通道12位模数转换器（ADC），2通道的12位数模转换器（DAC），指令系统与8051单片机兼容。片内有8KB Flash程序存储器、640B Flash数据存储器、256B数据SRAM（支持可编程）。

AD μ C812片内集成有看门狗定时器、电源监视器以及ADC DMA功能。为多处理器接口和I/O扩展提供了32条可编程的I/O线、包含有与I2C兼容的串行口以及SPI串行口和标准的UART串行口。

1.7.4 华邦W77系列、W78系列单片机

台湾华邦公司（Winbond）的W77系列、W78系列单片机与8051单片机完全兼容。

华邦单片机对8051的时序作了改进：每个指令周期只需要4个时钟周期，速度提高了3倍，工作频率最高可达40MHz。

W77系列为增强型，片内增加了看门狗WatchDog、两组UART串口、两组DPTR数据指针（编写应用程序非常便利）、ISP（在线编程）等功能。片内集成有USB接口，具有语音处理等功能，有6组外部中断源。

1.8 PIC系列单片机与AVR系列单片机

除8051单片机外，各种非8051机型的8位单片机也得到广泛应用。其中使用较为较多的是PIC系列与AVR系列单片机，这两种单片机博采众长，又具独特技术，已占有较大的市场份额。

1.8.1 PIC系列单片机

美国Microchip公司的产品。特性如下：

- (1) 最大的特点是**从实际出发**，重视性能价格比，已经开发出多种型号来满足应用需求。**例如**，一个摩托车的点火器需要一个I/O较少、RAM及程序存储空间不大、可靠性较高的小型单片机，此时如果采用40引脚的机型就可能是“大马拉小车”。

PIC10F322 6脚单片机 被认为是世界上最小的单片机。该单片机带有4个I/O。最大特色是外设增加了可配置逻辑单元CLC，数控振荡器NCO，互补波形发生器CWG，另外还有3个通道的8位ADC，2个10位的PWM，2个8位定时器；64字节的静态RAM，512字的程序空间，支持高性能的精简指令集（RISC）的CPU。

PIC的**40只引脚**的单片机**PIC16C74**，其内部有4KB 的ROM、192字节RAM、8路A/D、3个8位定时器、2个CCP模块、3个串行口、1个并行口、11个中断源、33个I/O脚，可以与其他品牌的高档型号媲美。

(2) **精简指令集**使执行效率大为提高。PIC系列8位单片机采用精简指令集（RISC），数据总线和指令总线分离的哈佛总线（Harvard）结构，指令单字长，且允许指令代码的位数可多于8位的数据位数，这传统的采用复杂指令结构（CISC）结构的8位单片机相比，可以达到2:1的代码压缩，速度提高4倍。

(3) 引脚通过限流电阻可以接至220V交流电源，可直接与继电器控制电路相连，无须光电耦合器隔离，给使用带来极大方便。

PIC单片机分低档型、中档型和高档型：

中档产品是Microchip公司重点发展的系列产品，品种最为丰富。尤其是PIC18系列，其程序存储器最大可达64KB，通用数据存储器为3968B；有8位和16位定时器，比较器；8级硬件堆栈，10位A/D转换器，捕捉输入，PWM输出；配置了I²C、SPI，UART串口，CAN、USB接口，模拟电压比较器及LCD驱动电路等，其封装从14脚到64脚，价格适中，性价比高。已广泛应用在高、中、低档的各类电子产品中。

高档型产品PIC17Cxx是在中档型产品的基础上增加了硬件乘法器，指令周期可达160ns。它是目前世界上8位单片机中性价比最高的机型，可用于电机控制等。

此外，Microchip公司还推出了高性能的16位的PIC24xx系列和32位的PIC32xx系列单片机，受到用户欢迎，已得到较为广泛的应用。

1.8.2 AVR系列单片机

AVR系列是1997年ATMEL公司研发出的**精简指令集 (RISC—Reduced Instruction Set Computer)**的**高速**8位单片机。**特点如下：**

- (1) 废除了机器周期，抛弃复杂指令追求指令完备的做法。采用**精简指令集**，以字作为指令长度单位，将操作数与操作码安排在一字之中，指令长度固定、指令格式与种类相对较少、寻址方式也相对较少，绝大部分指令都为单周期指令。取指周期短，又可预取指令，实现流水作业，故可高速执行指令。

(2) 新工艺AVR器件的Flash程序存储器擦写可达10 000次以上。片内还集成有较大容量的RAM，不仅能满足一般场合的使用，同时也更有效的支持使用高级语言开发系统程序，并且很容易地扩展外部RAM。

(3) 丰富的片内外设。定时器/计数器、看门狗电路、低电压检测电路BOD，多个复位源(自动上下电复位、外部复位、看门狗复位、BOD复位)，可设置的启动后延时运行程序，增强了单片机应用系统的可靠性。

多种串口：如通用的异步串行口（UART），面向字节的高速硬件串行接口TWI（与I²C接口兼容）、SPI。此外还有ADC、PWM等部件。

此外，还有ADC、PWM等片内外设。

(4) I/O口功能强、驱动能力大。工业级产品，具有大电流（最大可达40mA），驱动能力强，可省去功率驱动器件，直接驱动可控硅SSR或继电器。

I/O口的输入可设定为三态高阻抗输入或带上拉电阻输入，以便于满足各种多功能I/O口应用的需要，具备10mA-20mA灌电流的能力。

(5) 低功耗。具有省电功能(Power Down)及休眠功能(Idle)的低功耗的工作方式。一般耗电在1~2.5 mA；典型功耗，WDT关闭时为100nA，更适用于电池供电。有的器件最低1.8V即可工作。

(6) 支持程序的在线编程（ISP—In System Program），开发门槛较低。只需一条ISP并口下载线，就可以把程序写入AVR单片机，所以使用AVR门槛低、花钱少。其中MEGA系列还支持在线应用编程（IAP，可在线升级或销毁应用程序）。

AVR单片机系列全，3个档次，适于各种不同要求：

- (1) 低档Tiny系列：Tiny11/12/13/15/26/28等；
- (2) 中档AT90S系列：AT90S1200/2313/8515/8535等；
- (3) 高档ATmega系列：有ATmega8/16/32/64/128（存储容量为8/16/32/64/128 KB）以及ATmega8515/8535等。

1.9 其他的嵌入式处理器简介

目前各类嵌入式处理器为核心的嵌入式系统的应用，是当今电子信息技术应用的一大热点。

具有各种不同体系结构的嵌入式处理器是嵌入式系统的核心部件，除单片机外还有嵌入式数字信号处理器（简称DSP）以及嵌入式微处理器。

1.9.1 嵌入式DSP处理器 (DSP)

DSP (Digital Signal Processor)，非常擅长于高速实现各种数字信号处理运算（如数字滤波、FFT、频谱分析等）。由于硬件结构和指令的特殊设计，使其能够高速完成各种数字信号处理算法。

与单片机相比，DSP所具有的实现高速运算的硬件结构与指令系统以及多总线结构，尤其是DSP处理的是数字信号处理算法的复杂度和大的数据处理流量更是单片机不可企及的。

1981年，TI公司研制出TMS320系列的首片低成本、高性能DSP处理器芯片，使DSP技术向前跨出意义重大的一步。

上世纪90年代，由于无线通信、各种网络通信、多媒体技术的普及和应用，高清晰度数字电视的研究，极大地刺激了DSP在工程上的推广应用。

DSP处理器大量进入嵌入式领域。推动DSP快速发展的是嵌入式系统的智能化，例如各种带有智能逻辑的消费类产品，生物信息识别终端，实时语

音解压系统、数字图像处理等。这类**智能化算法**一般都是运算量较大，特别是向量运算、指针线性寻址等较多，而这些正是DSP的长处所在。但在一些实时性要求很高的场合，单片DSP的处理能力有时还不能满足要求。因此，各大公司又研制出多总线、多流水线和并行处理的包含多个DSP处理器的芯片，大大提高了系统性能。

DSP的主要厂商有美国**TI、ADI、Motorola、Zilog**等公司。TI公司位居榜首，占全球DSP市场约60%。DSP代表性的产品是TI公司的TMS320系列，其中包括用于控制领域的2000系列，用于移动通信的5000系列以及应用在网络、多媒体以及图像处理领域的6000系列等。

今天，随着全球信息化和Internet的普及，多媒体技术的广泛应用，尖端技术向民用领域的迅速转移，DSP也大范围地进入消费类电子产品。DSP的不断更新换代，性能指标不断提高，价格不断下降，已成为新兴科技：通信、多媒体系统、消费电子、医用电子等飞速发展的主要推动力。

据国际著名市场调查研究公司Forward Concepts 发布的一份统计和预测报告显示，目前世界DSP产品市场每年正以30%的增幅增长，是目前最有发展和应用前景的嵌入式处理器之一。

1.9.2 嵌入式微处理器

嵌入式微处理器（Embedded MicroProcessor Unit, EMPU）的基础是通用计算机中的CPU。虽然在功能上和标准微处理器基本是一样的，但由于只保留和嵌入式应用有关的功能，这样可大幅度减小系统体积和功耗，

同时在工作温度、抗电磁干扰、可靠性等方面一般都做了各种增强处理。

嵌入式微处理器中比较有代表性的产品为ARM系列，主要的产品系列：ARM7/9/11等。

以ARM7为例，它的地址线为32条，所扩展的存储器空间要比单片机存储器空间大得多，可配置实时多任务操作系统(RTOS)，它是嵌入式应用软件的基础和开发平台。

常用的RTOS为Linux（数百KB）和VxWorks（数MB）以及 μ C-OS II。由于嵌入式实时多任务操作系统具有高度灵活性，可很容易地对它进行定制或适当开发，即对它进行“裁剪”、“移植”和“编写”，从而设计出用户所需的程序，满足实际应用需要。

由于嵌入式微处理器能运行实时多任务操作系统，所以能够胜任复杂的系统管理任务和数据处理工作。因此，在移动计算平台、媒体手机、工业控制和商业领域（例如，智能工控设备、ATM机等）、电子商务平台、信息家电（机顶盒、数字电视）等方面，甚至军事上的应用，具有巨大的吸引力。因此，以嵌入式微处理器为核心的嵌入式系统的应用，已经成为继单片机、DSP之后的电子信息技术应用的又一大热点。

广义上讲，凡是系统中嵌入了“嵌入式处理器”，如单片机、DSP、嵌入式微处理器，都称为“嵌入式系统”。也有仅把“嵌入”嵌入式微处理器的系统，称为“嵌入式系统”。还没有严格的定义。目前的“嵌入式系统”，多指后者。

第2章 AT89S52单片机的 片内硬件结构



第2章 AT89S52单片机的片内硬件结构

2.1 AT89S52单片机的硬件组成

2.2 AT89S52的引脚功能

2.2.1 电源及时钟引脚

2.2.2 控制引脚

2.2.3 并行I/O口引脚

2.3 AT89S52的CPU

2.3.1 运算器

2.3.2 控制器

2.4 AT89S52的存储器结构

2.4.1 程序存储器空间

2.4.2 数据存储器空间

2.4.3 特殊功能寄存器

2.4.4 位地址空间

2.4.5 存储器结构总结

2.5 AT89S52的并行I/O端口

2.5.1 P0口

2.5.2 P1口

2.5.3 P2口

2.5.4 P3口

2.6 时钟电路与时序

2.6.1 时钟电路设计

2.6.2 时钟周期、机器周期、指令周期与指令时序

2.7 复位操作和复位电路

2.7.1 复位操作

2.7.2 复位电路设计

2.8 AT89S52单片机的最小应用系统

2.9 看门狗定时器（WDT）功能及应用

2.10 低功耗节电模式

2.10.1 空闲模式

2.10.2 掉电运行模式

内容概要

本章介绍 AT89S52 单片机的 **片内硬件结构**。读者应了解并熟知 AT89S52 单片机的片内硬件结构，以及片内外设资源的工作原理与基本功能，**重点掌握** AT89S52 单片机的存储器结构、常见的特殊功能寄存器的基本功能以及复位电路与时钟电路的设计，掌握单片机 **最小系统** 的概念。此外还介绍了 **低功耗节电模式**。

2.1 AT89S52单片机的硬件组成

片内硬件组成结构如图2-1所示。把作为控制应用所必需的基本外围功能部件都集成在一个尺寸有限的集成电路芯片上。

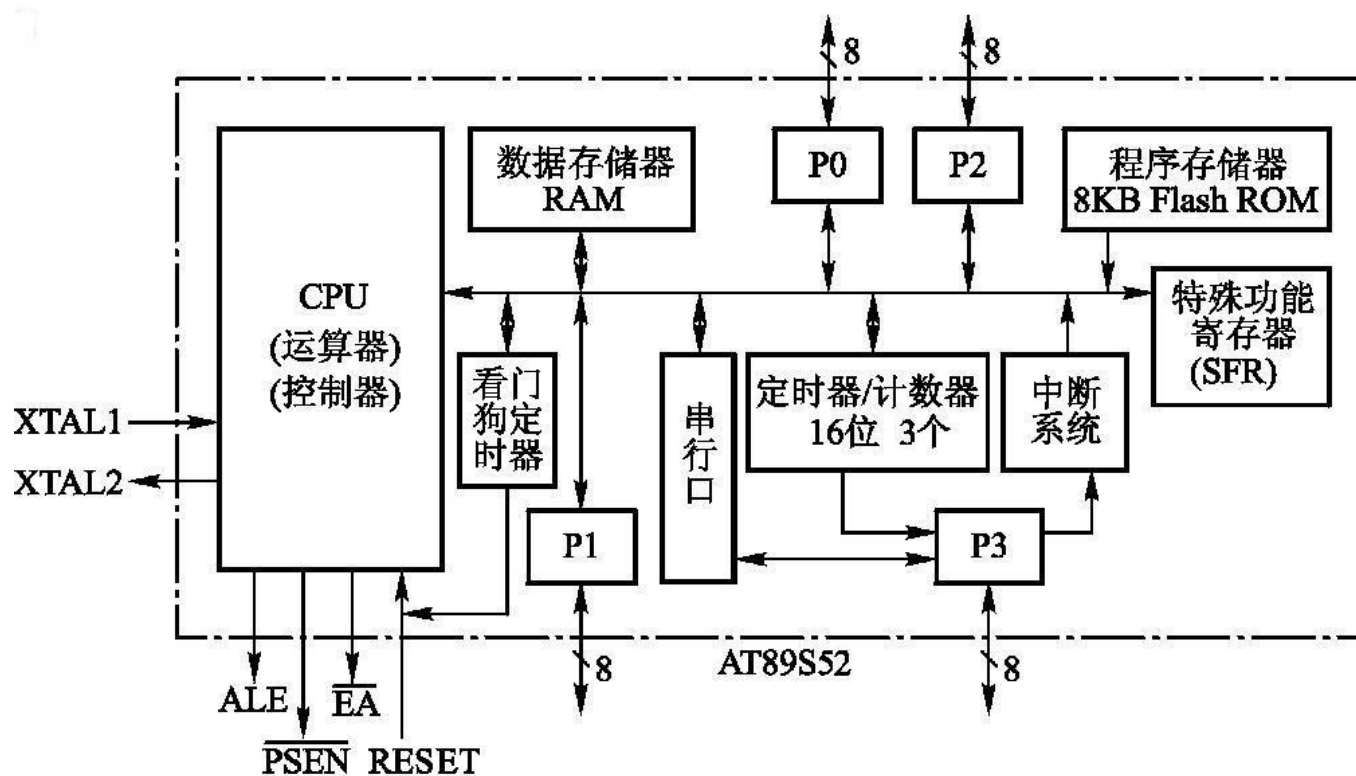
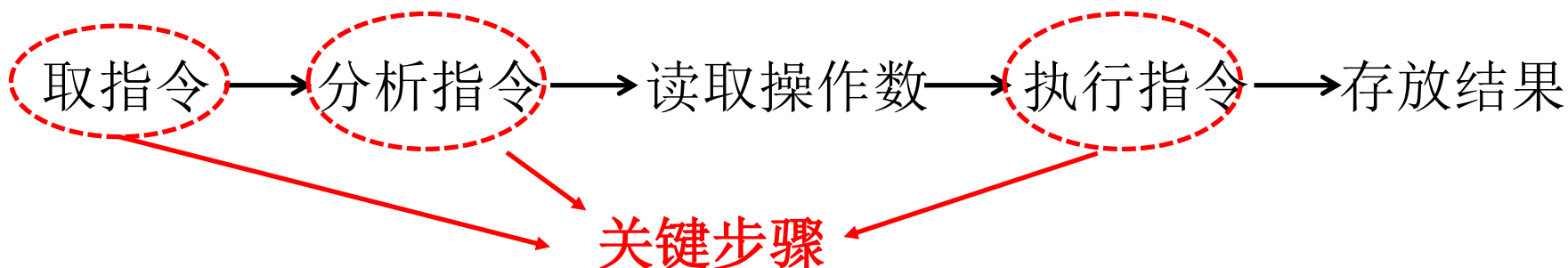
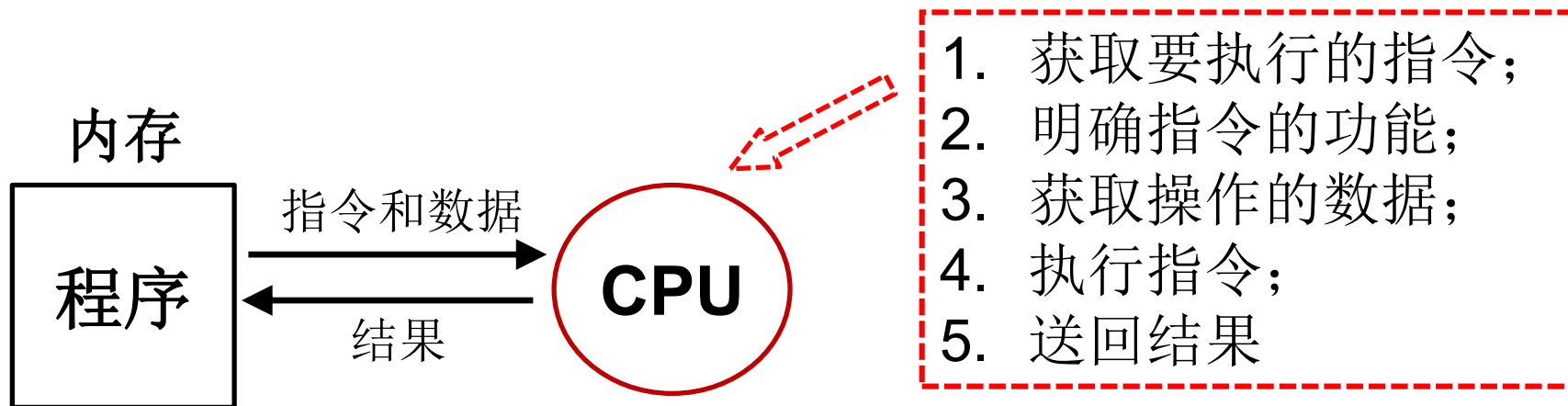


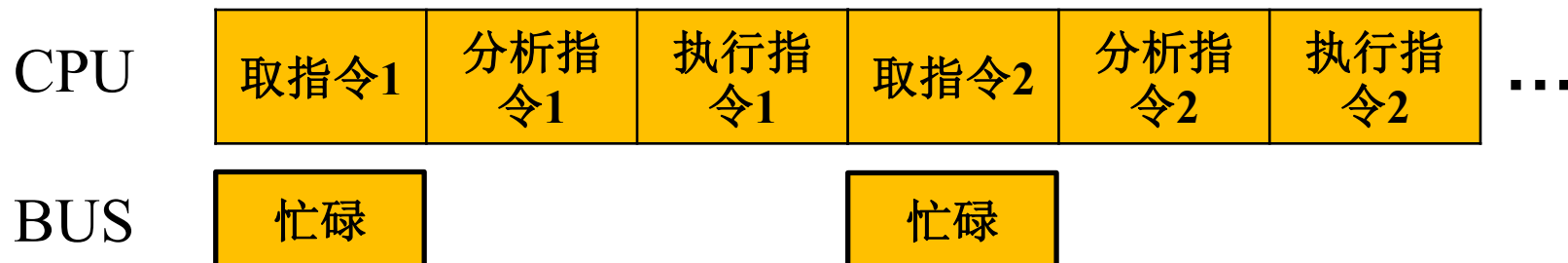
图2-1 AT89S52单片机片内结构

单片机的一般工作过程

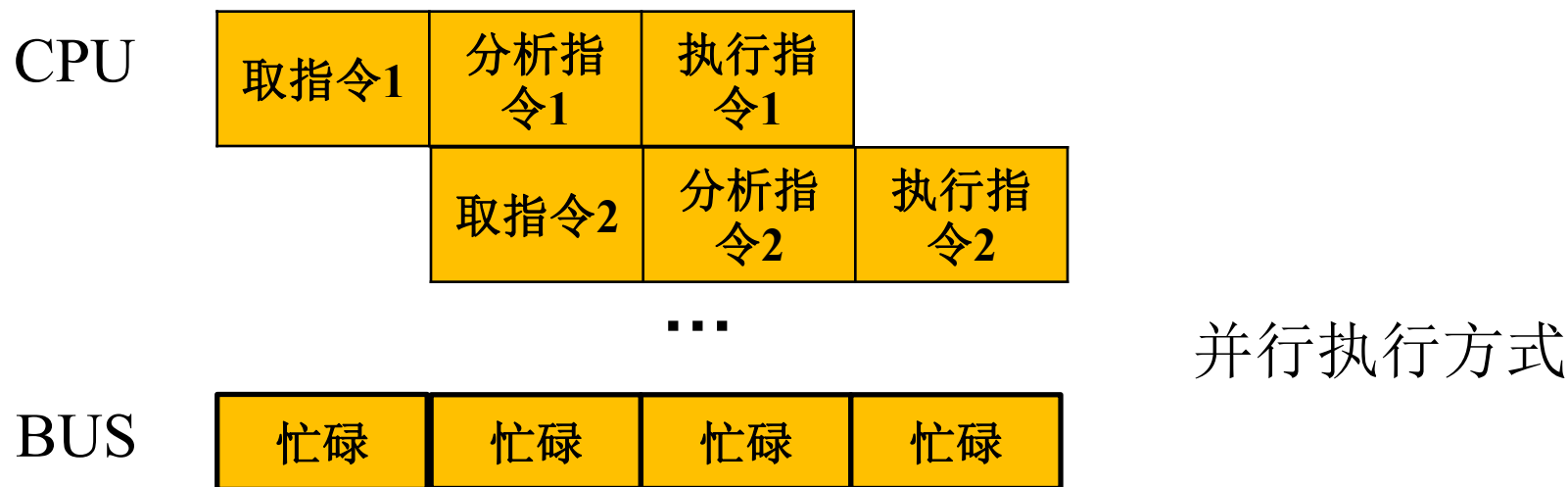
计算机执行程序过程：



顺序执行方式和并行执行方式

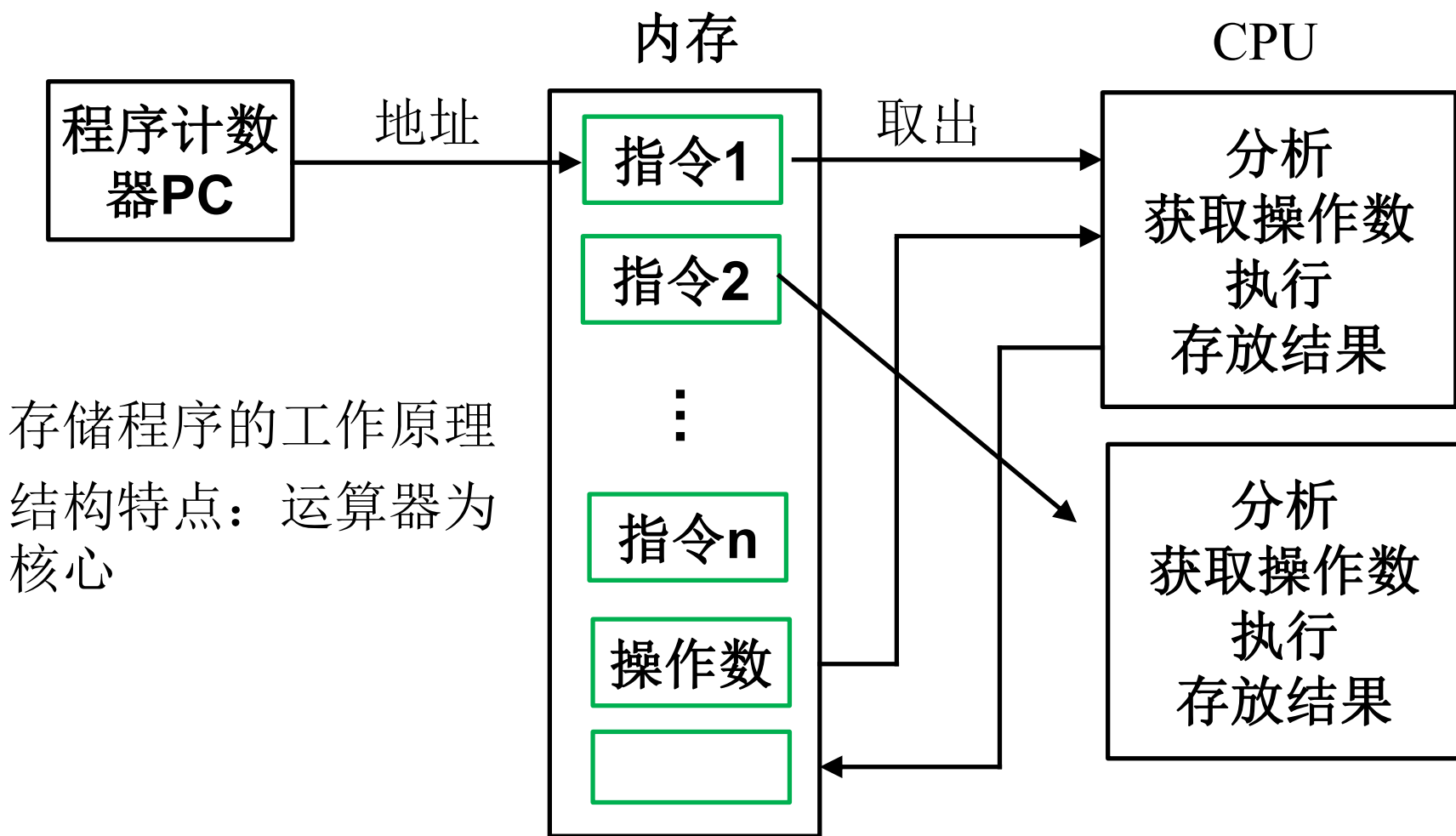


顺序执行方式



并行执行方式

冯-诺依曼机的工作过程



片内各外围功能部件通过片内单一总线连接而成（见[图2-1](#)），**基本结构**依旧是CPU 加上外围芯片的**传统微机结构**。

CPU对各种功能部件的控制是采用**特殊功能寄存器**（SFR, Special Function Register）的集中控制方式。

下面介绍[图2-1](#)中**片内各功能部件**。

（1）CPU（微处理器）

8位的CPU，与通用CPU基本相同，同样包括了**运算器**和**控制器**两大部分，还有面向控制的**位处理功能**和**位控功能**。

（2）数据存储器（RAM）

片内为**256B**，片外最多可扩64KB。

(3) 程序存储器 (Flash ROM)

用来存储程序。AT89S52 片内有8KB的Flash存储器；AT89S53/AT89S54/AT89S55片内集成了12KB/16KB/20KB的Flash存储器，如果片内程序存储器容量不够，片外最多可外扩至64KB程序存储器，即“片内+片外”的程序存储器总容量不超过64KB。

(4) 定时器/计数器

片内有3个16位的定时器/计数器，具有4种工作方式。

(5) 中断系统

具有6个中断源，2级中断优先权。

(6) 串行口

1个全双工的异步串行口（UART），4种工作方式。

(7) P0口、P1口、P2口和P3口

4个8位并行I/O口。

(8) 特殊功能寄存器（SFR）

共有32个特殊功能寄存器，用于CPU对片内各外围部件进行管理、控制和监视。特殊功能寄存器实际上是片内各外围部件的控制寄存器和状态寄存器，这些特殊功能寄存器映射在片内RAM区的80H~FFH的地址区间内。

(9) 1个看门狗定时器WDT

当单片机由于干扰而使程序陷入死循环或跑飞状态时，可引起单片机复位，使程序恢复正常运行。

AT89S52完全兼容AT89C51/AT89S51单片机，使用AT89C51/AT89S51单片机的系统，在保留原来软硬件的基础上，可用AT89S52直接代换。

2.2 AT89S52的引脚功能

AT89S52与各种8051单片机的引脚是兼容的。目前，AT89S52多采用40引脚的DIP封装（双列直插），以及44引脚的PLCC和TQFP封装方式的芯片，外形见[图2-2（a）](#)和[图2-2（b）](#)。



(a) 40引脚的DIP封装

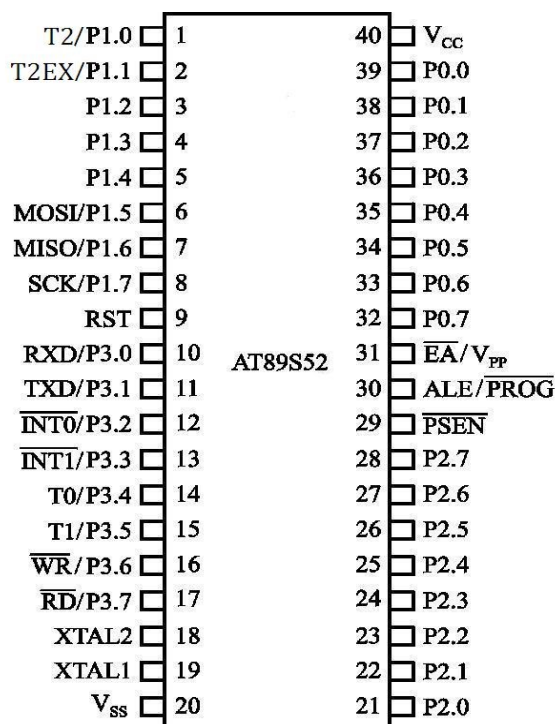


(b) 44引脚的PLCC封装或TQFP封装

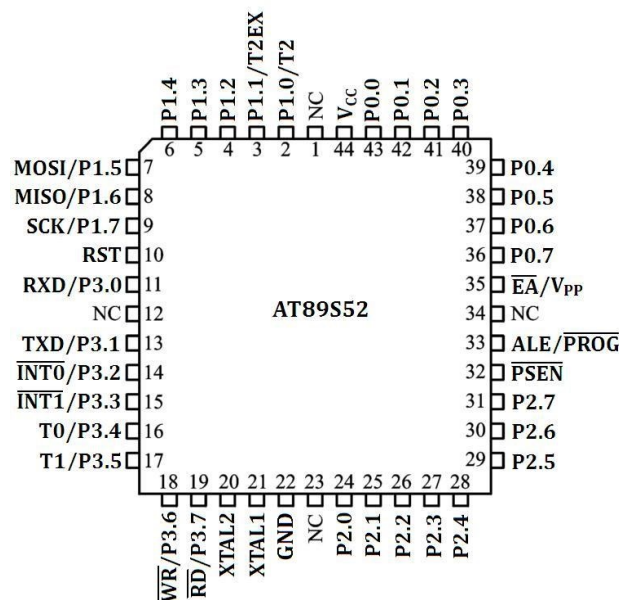
图2-2 AT89S52单片机的外形



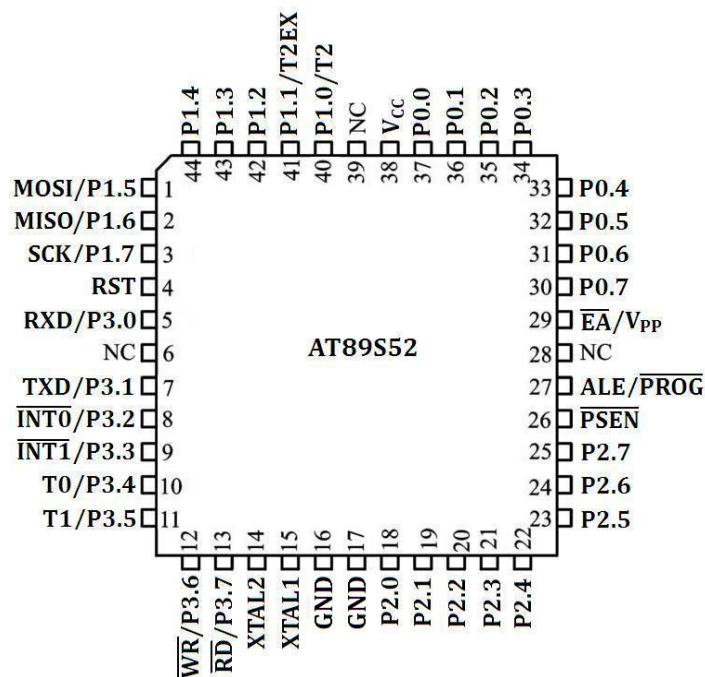
AT89S52单片机的DIP封装的引脚名称见图2-3（a），44引脚的PLCC和TQFP封装方式的引脚名称，见图2-3（b）和图2-3（c）。44引脚的PLCC和TQFP封装方式的芯片，有4只引脚是无用的，标为“NC”。



(a) DIP封装的引脚分布



(b) PLCC封装的引脚分布



(c) TQFP封装的引脚分布

图2-3 AT89S52各种封装方式的引脚

引脚按其功能可分为如下3类：

- (1) 电源及时钟引脚— V_{CC} 、 V_{SS} ；XTAL1、XTAL2。
- (2) 控制引脚— PSEN*、ALE/PROG*、EA* / V_{PP} 、RST
- (3) I/O口引脚——P0、P1、P2、P3，为4个8位I/O口

下面结合图2-3(a)介绍各引脚的功能。

2.2.1 电源及时钟引脚

1. 电源引脚

- (1) V_{CC} (40脚) : +5V电源。
- (2) V_{SS} (20脚) : 数字地。

2. 时钟引脚

(1) XTAL1 (19脚) : 片内振荡器反相放大器和时钟发生器电路输入端。

用片内振荡器时, 该脚接外部石英晶体和微调电容。外接时钟源时, 该脚接外部时钟振荡器的信号。

(2) XTAL2 (18脚) : 片内振荡器反相放大器的输出端。当使用片内振荡器, 该脚连接外部石英晶体和微调电容。当使用外部时钟源时, 本脚悬空。

2.2.2 控制引脚

(1) RST (RESET, 9脚)

复位信号输入, 在引脚加上持续时间大于2个机器周期的高电平, 可使单片机复位。正常工作, 此脚电平应 $\leq 0.5V$ 。

当看门狗定时器溢出输出时，该脚将输出长达96个时钟振荡周期的高电平。

(2) EA*/VPP (Enable Address/Voltage Pulse of Programing, 31脚)

EA*：引脚**第一功能**：外部程序存储器访问允许控制端。

EA*=1，在**PC值不超出1FFFH**（即不超出片内8KB Flash存储器的地址范围）时，单片机读**片内程序存储器**（8KB）中的程序，但**PC值超出1FFFH**（即超出片内8KB Flash地址范围）时，将**自动转向读取片外**（2000H-FFFFH）程序存储器空间中的程序。

EA*=0，只读取外部的程序存储器中的内容，读取的地址范围为0000H～FFFFH，片内的8KB Flash 程序存储器不起作用。

V_{pp}：引脚第二功能，对片内Flash编程，接编程电压。

(3) **ALE/PROG*** (Address Latch Enable/PROGramming, 30脚)

ALE为地址锁存控制信号端，为第一功能。由于引脚数目有限，P0口是作为低8位地址总线与8位数据总线分时复用的。当单片机访问外部程序存储器或外部数据存储器时，ALE的负跳变将单片机P0口先发出的低8位地址锁存在P0口外接的地址锁存器中，然后P0口再作为8位数据总线使用，如图2-4所示。

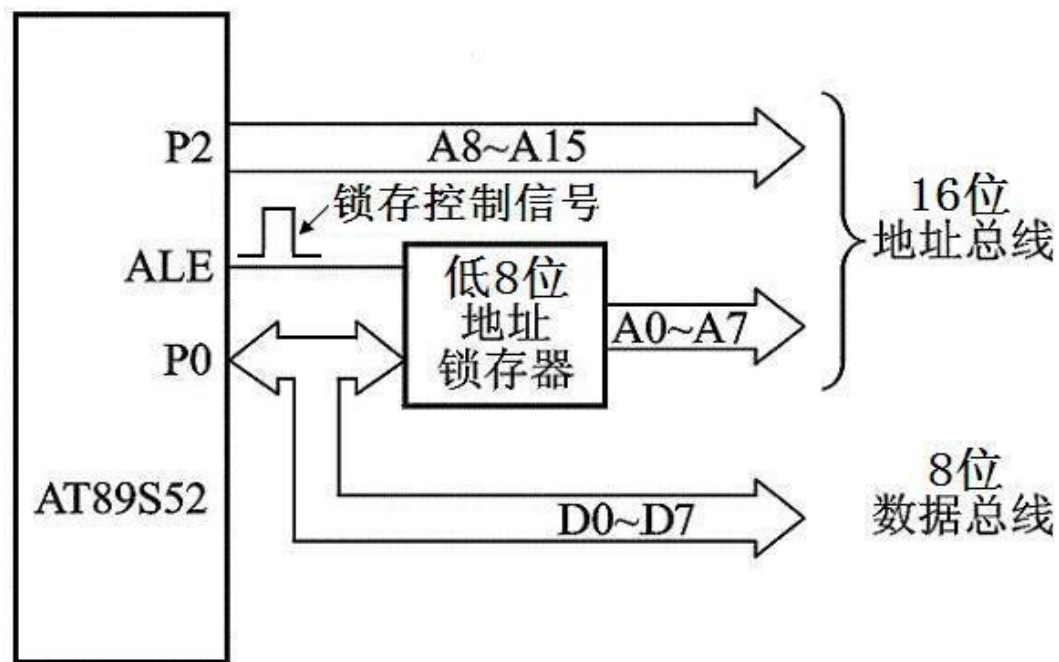


图2-4 ALE引脚输出地址锁存控制信号

此外，单片机正常运行时，ALE端一直有正脉冲信号输出，此频率为时钟振荡器频率 f_{osc} 的1/6。可用作外部定时或触发信号。

注意，每当AT89S52访问外部RAM时（执行MOVX类指令），要丢失一个ALE脉冲。

如不需要ALE端输出脉冲信号，可将特殊功能寄存器AUXR（地址为8EH，将在后面介绍）的第0位（ALE禁止位）置1，来禁止ALE操作，但执行访问外部程序存储器或外部数据存储器指令“MOVC”或“MOVX”时，ALE仍然有效。即ALE禁止位不影响对外部存储器的访问。

PROG*：引脚第二功能，对片内Flash编程，为编程脉冲输入脚。

(4) PSEN* (Program Strobe ENable, 29脚)

片外程序存储器读选通信号，低电平有效。

2.2.3 并行I/O口引脚

(1) P0口：P0.7~P0.0引脚

为漏极开路的8位并行双向I/O口。作为输出口时，每个引脚可驱动8个LS型TTL负载。当AT89S52扩展外部存储器及I/O接口芯片时，P0口为分时复用的低8位地址/数据总线。在向P0口写入“1”后就成为高阻态的输入口。

当P0口作为通用I/O口使用时，需外加上拉电阻，这时为准双向口。

在对Flash程序存储器编程时，P0口接收字节代码，程序校验时P0口输出字节代码，程序校验期间应外接上拉电阻。



(2) P1口: P1.7~P1.0引脚

准双向I/O口，具有内部上拉电阻，可驱动4个LS型TTL负载。

在对片内Flash编程和校验时定义为低8位地址线。

P1口某些引脚的**第二功能**如下。

- P1.0/T2: T2脚为定时器T2的外部计数信号输入端T2。
- P1.1/T2EX: T2EX为T2的捕捉/重新装载触发及方向控制T2EX。
- P1.5/MOSI: MOSI用于对片内Flash存储器串行编程和校验。
- P1.6/MISO: MISO用于对片内Flash存储器串行编程和校验。
- P1.7/SCK: SCK用于对片内Flash存储器的串行编程和校验的移位脉冲输入引脚。

注意： AT89S51与AT89S52引脚的差别仅仅是在1脚（P1.0）与2脚（P1.1）上，AT89S52的1脚（P1.0）与2脚（P1.1）分别增加了定时器/计数器T2的两个外部引脚**T2**和**T2EX**的复用功能。

当AT89S52单片机**不使用**片内的T2的**两个引脚T2**（P1.0）和**T2EX**（P1.1）的复用功能时，AT89S51以及各种8051兼容机与AT89S52的引脚功能则完全相同，它们的外设硬件接口电路是完全相互通用的。

但是如果使用定时器T2的外部计数输入T2（P1.0）和“捕捉”输入T2EX（P1.1）的功能时，则**AT89S52的P1.0脚和P1.1脚**就不能作为通用I/O使用，这是**AT89S52与AT89S51**（或AT89C51）在外围接口电路设计上的**微小差别**。

(3) P2口：P2.7~P2.0引脚

准双向I/O口，引脚内部接有上拉电阻，可驱动4个LS型TTL负载。

当AT89S52访问外部存储器及I/O口时，P2口作为高8位地址总线使用，输出高8位地址。

当P2口不作为高8位地址总线时，可作为通用的I/O口使用。

(4) P3口：P3.7～P3.0

准双向I/O口，具有内部上拉电阻。

P3口的第一功能是作为通用的I/O口使用，可驱动4个LS型TTL负载。

P3口还可提供第二功能。第二功能定义见表2-1，应熟记。

综上所述，P0口可作为总线口，为双向口。作为通用的I/O口使用时，为准双向口，这时需加上拉电阻。P1口、P2口、P3口均为准双向口。

表 2-1 P3 口的第二功能定义

引 脚	第 二 功 能	说 明
P3.0	RXD	串行数据输入口
P3.1	TXD	串行数据输出口
P3.2	$\overline{\text{INT0}}$	外部中断 0 输入
P3.3	$\overline{\text{INT1}}$	外部中断 1 输入
P3.4	T0	定时器 0 外部计数输入
P3.5	T1	定时器 1 外部计数输入
P3.6	$\overline{\text{WR}}$	外部数据存储器写选通输出
P3.7	$\overline{\text{RD}}$	外部数据存储器读选通输出

注意：准双向口与双向口的差别。准双向口仅有两个状态。而P0口作为总线使用，口线内无上拉电阻，处于高阻“悬浮”态。故P0口为双向三态I/O口。

为什么P0口要有高阻“悬浮”态？准双向I/O口则无高阻的“悬浮”状态。

这是由于P0口作为数据总线使用时，多个数据源都挂在数据总线上，当P0口不需与其它数据源打交道时，需要P0口与数据总线上的其它数据源高阻“悬浮”隔离。而准双向I/O口无高阻“悬浮”状态。另外，准双向口作通用I/O的输入口使用时，一定要向该口先写入“1”。以上的准双向口与双向口的差别，读者在学习本章2.5节的P0~P3口的内部结构后，将会有更深入的理解。

至此，40个引脚已介绍完毕，应熟记每一引脚功能对应用系统硬件电路设计十分重要。

2.3 AT89S52的CPU

CPU由**运算器**和**控制器**构成。

2.3.1 运算器

对操作数进行算术、逻辑和位操作运算。主要包括算术逻辑运算单元ALU、累加器A、位处理器、程序状态字寄存器PSW及两个暂存器等。

1. 算术逻辑运算单元ALU

可对**8位变量逻辑运算**（与、或、异或、循环、求补和清零），还可**算术运算**（加、减、乘、除）

ALU还有位操作功能，对位变量进行位处理，如置“1”、清“0”、求补、测试转移及逻辑“与”、“或”等。

2. 累加器A

使用最频繁的寄存器，可写为Acc。位于片内的特殊功能寄存器区。

作用如下：

(1) ALU单元的输入数据源之一，又是ALU运算结果存放单元。

(2) 数据传送大多都通过累加器A，相当于数据的中转站。为解决“瓶颈堵塞”问题，AT89S52增加了一部分可以不经过累加器的传送指令（指的是汇编语言指令）。

A的进位标志Cy是特殊的，因为它同时又是位处理机的位累加器。

3. 程序状态字寄存器PSW

PSW (Program Status Word) 位于片内特殊功能寄存器区，字节地址为D0H。

包含了程序运行状态的信息，其中4位保存当前指令执行后的状态，供程序查询和判断。格式如图2-5所示。

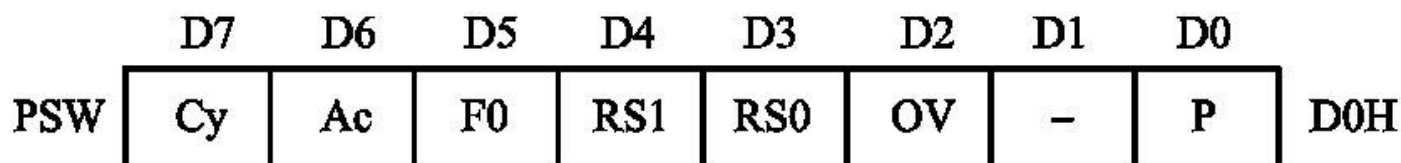


图2-5 PSW的格式

PSW中各个位的功能：

(1) Cy (PSW. 7) 进位标志位

可写为C。在算术和逻辑运算时，若有进位/借位， $Cy=1$ ；否则， $Cy=0$ 。在位处理器中，它是位累加器。

(2) Ac (PSW. 6) 辅助进位标志位

在BCD码运算时，用作十进位调整。即当D3位向D4位产生进位或借位时， $Ac=1$ ；否则， $Ac=0$ 。

(3) F0 (PSW. 5) 用户设定标志位

由用户使用的一个状态标志位，可用指令来使它置1或清0，控制程序的流向。用户应充分利用。

(4) RS1、RS0 (PSW. 4、PSW. 3) 4组工作寄存器区选择

选择片内RAM区中的4组工作寄存器区中的某一组为当前工作寄存器区见
表2-2。

表 2-2 RS1、RS0 与 4 组工作寄存器区的对应关系

RS1	RS0	所选的4组寄存器
0	0	0区 (内部RAM地址00H~07H)
0	1	1区 (内部RAM地址08H~0FH)
1	0	2区 (内部RAM地址10H~17H)
1	1	3区 (内部RAM地址18H~1FH)

(5) OV (PSW. 2) 溢出标志位

当执行算术指令时，用来指示运算结果是否产生溢出。如果结果产生溢出，OV=1；否则，OV=0。

(6) PSW. 1位：保留位

(7) P (PSW. 0) 奇偶标志位

指令执行完，累加器A中“1”的个数是奇数还是偶数。

P=1，表示A中“1”的个数为奇数。

P=0，表示A中“1”的个数为偶数。

此标志位对串行通信有重要的意义，常用奇偶检验的方法来检验数据串行传输的可靠性。

2.3.2 控制器

任务识别指令，并根据指令的性质控制单片机各功能部件，从而保证单片机各部分能自动协调地工作。

控制器包括：程序计数器、指令寄存器、指令译码器、定时及控制逻辑电路等。功能是控制指令的读入、译码和执行，从而对各功能部件进行定时和逻辑控制。

程序计数器PC是一个独立的16位计数器，不可访问。单片机复位时，PC中内容为0000H，从程序存储器0000H单元取指令，开始执行程序。

PC的基本工作过程：CPU读指令时，PC的内容作为所取指令的地址，程序存储器按此地址输出指令字节，同时PC自动加1。这也是PC被称为程序计数器的原因。由于PC实质上是作为程序寄存器的地址指针，所以也称其为程序指针。

PC中内容变化轨迹决定程序流程。当顺序执行程序时自动加1；执行转移程序或子程序、中断子程序调用时，自动将其内容更改成所要转移的目的地址。

PC的计数宽度决定了程序存储器的地址范围。PC为16位，故可对64KB ($=2^{16}\text{B}$) 寻址。

2.4 AT89S52存储器的结构

存储器的结构特点是将程序存储器和数据存储器分开（哈佛结构），并有各自的访问指令。

存储器空间可分为4类。

1. 程序存储器空间

片内和片外两部分。

片内4KB Flash，编程和擦除完全是电气实现。可用通用编程器对其编程，也可在线编程。

当片内4KB Flash 存储器不够用时，可片外扩展，最多可扩展至64KB 程序存储器。

2. 数据存储器空间

片内与片外两部分。

片内有256B RAM。

片内RAM 不够用时，在片外可扩展至64KB RAM。

3. 特殊功能寄存器SFR (Special Function Register)

片内各功能部件的控制寄存器及状态寄存器。综合反映了整个单片机基本系统内部实际的工作状态及工作方式。

4. 位地址空间

共有219个可寻址位，构成了位地址空间。位于片RAM区字节地址20H~2FH（共128位）和特殊功能寄存器区（片内RAM区字节地址80H~FFH区间内，共定义了91个可寻址位）。

2.4.1 程序存储器空间

存放程序和表格之类的固定常数。片内为8KB的 Flash，地址为0000H~1FFFH。16位地址线，可外扩的程序存储器空间最大为64KB，地址为0000H~FFFFH。使用时应注意以下问题：

(1) 分为片内和片外两部分，访问片内的还是片外的程序存储器，由EA*引脚电平确定。

EA*=1时，CPU从片内0000H开始取指令，当PC值没有超出1FFFH时，只访问片内Flash 存储器，当PC值超出1FFFH自动转向读片外程序存储器空间2000H~FFFFH 内的程序。

EA*=0时，只能执行片外程序存储器（0000H~FFFFH）中的程序。不理睬片内8KB Flash 存储器。

(2) 程序存储器某些固定单元用于各中断源中断服务程序入口。

64KB程序存储器空间中有6个特殊单元分别对应于6个中断源的中断入口地址，见[表2-3](#)。

通常这6个中断入口地址处都放一条跳转指令跳向对应的中断服务子程序，而不是直接存放中断服务子程序。

表 2-3 AT89S52 各中断源的中断入口地址

中 断 源	入 口 地 址
外部中断 0	0003H
定时器/计数器 T0	000BH
外部中断 1	0013H
定时器/计数器 T1	001BH
串行口	0023H
定时器/计数器 T2	002BH

表2-3中最后一行,即第6个中断入口地址**002BH**, 是AT89S52在AT89S51单片机基础上新增加的T2对应的中断入口地址。

2.4.2 数据存储器空间

片内与片外两部分。

1. 片内数据存储器

AT89S52的片内数据存储器（RAM）共有256个单元，字节地址为**00H～FFH**。[图2-6](#)为AT89S52片内RAM的结构。地址为80H～FFH为**特殊功能寄存器区**，与片内的**高128B**的RAM单元统一编址，但它是另一专用空间区域，将在后面介绍。

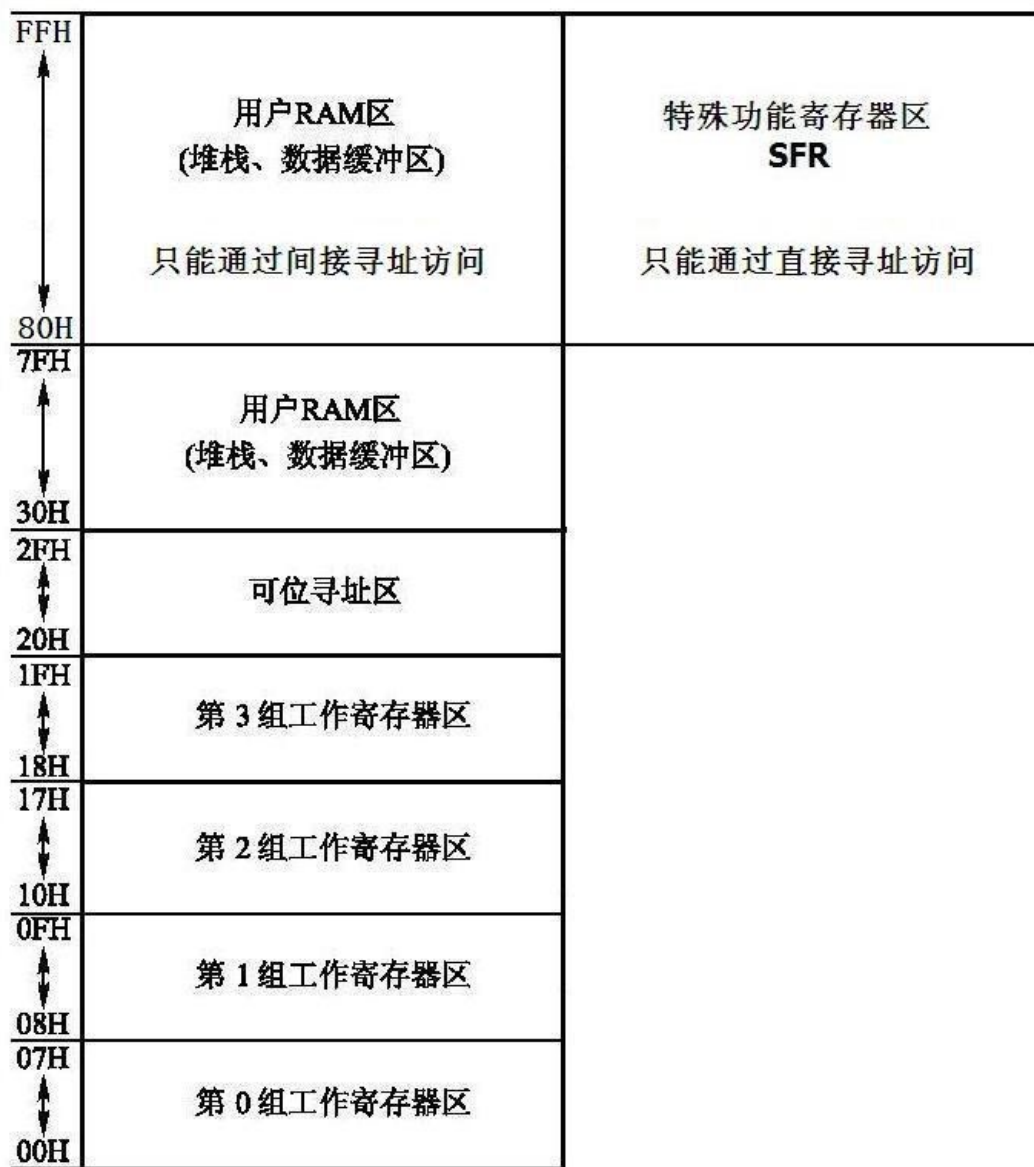


图2-6 片内数据存储器RAM的结构

00H~1FH 的32个单元是4组通用工作寄存器区，每区包含8B，为R7~R0。可通过指令改变RS1、RS0两位来选择。

20H~2FH的16个单元的128位可位寻址，也可字节寻址。

30H~FFH的单元只能字节寻址，用作存数据以及作为堆栈区。

AT89S52与AT89S51片内数据存储器相比，片内数据存储器增加了128B，对应的字节地址为80H~FFH。这高128B的RAM单元地址与特殊功能寄存器区的字节地址重合，但它们是两个不同的物理区域。

如何在C51编程中来访问这两个具有相同地址的两个不同区域，是通过不同关键字来区分的。例如，为了能直接访问特殊功能寄存器SFR，C51语言可使用C51语言的关键字sfr来定义特殊功能寄存器，这将在第3章介绍。

2. 片外数据存储器

当AT89S52片内256B的RAM不够用时，需要外扩数据存储器，片外最多可扩展64KB的RAM。注意，虽然片内RAM与片外扩展的RAM的低256B的地址是相同的，但这是两个不同的数据存储区。

2.4.3 特殊功能寄存器（SFR）

特殊功能寄存器映射在片内RAM的 80H~FFH 区域中，共32个。[表2-4](#) SFR的名称及其分布。有些还可位寻址，位地址见[表2-4](#)。

与AT89S51相比，新增加了6个特殊功能寄存器，已在[表2-4](#)中标出。

凡是可位寻址的SFR，字节地址末位只能是0H或8H。另外，若读/写未定义单元，将得到一个不确定的随机数。

下面介绍某些SFR，余下的SFR将在后面介绍。

表 2-4 AT89S52 单片机片内 SFR 的名称及其分布

序号	特殊功能 寄存器符号	名 称	字节 地址	位地址	复位值
1	P0	P0 口	80H	87H~80H	FFH
2	SP	堆栈指针	81H	—	07H
3	DP0L	数据指针 DPTR0 低字节	82H	—	00H
4	DP0H	数据指针 DPTR0 高字节	83H	—	00H
5	DP1L	数据指针 DPTR1 低字节	84H	—	00H
6	DP1H	数据指针 DPTR1 高字节	85H	—	00H
7	PCON	电源控制寄存器	87H	—	0×××0000B
8	TCON	定时器/计数器 0、1 的控制 寄存器	88H	8FH~88H	00H
9	TMOD	定时器/计数器 0、1 的方式 控制寄存器	89H	—	00H
10	TL0	定时器/计数器 0 (低字节)	8AH	—	00H
11	TL1	定时器/计数器 1 (低字节)	8BH	—	00H
12	TH0	定时器/计数器 0 (高字节)	8CH	—	00H
13	TH1	定时器/计数器 1 (高字节)	8DH	—	00H
14	AUXR	辅助寄存器	8EH	—	×××0 0××0B

15	P1	P1 口寄存器	90H	97H~90H	FFH
16	SCON	串行控制寄存器	98H	9FH~98H	00H
17	SBUF	串行发送数据缓冲器	99H	—	×××× ×××B
18	P2	P2 口寄存器	A0H	A7H~A0H	FFH
19	AUXR1	辅助寄存器	A2H	—	×××× ×××0B
20	WDTRST	看门狗复位寄存器	A6H	—	×××× ×××B
21	IE	中断允许控制寄存器	A8H	AFH~A8H	0××0 0000B
22	P3	P3 口寄存器	B0H	B7H~B0H	FFH
23	IP	中断优先级控制寄存器	B8H	BFH~B8H	××00 0000B
24	PSW	程序状态字寄存器	D0H	D7H~D0H	00H
25	A (或 Acc)	累加器	E0H	E7H~E0H	00H
26	B	B 寄存器	F0H	F7H~F0H	00H
27	T2CON	定时器/计数器 2 控制寄存器	C8H	CFH~C8H	00H
28	T2MOD	定时器/计数器 2 方式控制	C9H		×××× ××00 B
29	RCAP2L	定时器/计数器 2 陷阱寄存器 (低字节)	CAH		00H
30	RCAP2H	定时器/计数器 2 陷阱寄存器 (高字节)	CBH		00H
31	TL2	定时器/计数器 2 (低字节)	CCH		00H
32	TH2	定时器/计数器 2 (高字节)	CDH		00H

1. 堆栈指针SP

堆栈只能设在片内的RAM区，SP指示堆栈顶部在内部RAM块中的位置。

堆栈结构是向上生长型。单片机复位后，SP为07H，使得堆栈实际上从08H单元开始，由于08H~1FH单元分别是属于1~3组的工作寄存器区，最好在复位后把SP值改置为60H或更大的值，避免堆栈与工作寄存器冲突。

堆栈是为子程序调用和中断操作而设，用来保护断点和现场。

(1) 保护断点。无论是子程序调用操作还是中断服务子程序调用，最终都要返回主程序。应预先把主程序的断点在堆栈中保护起来，为程序正确返回做准备。

(2) 现场保护。执行子程序或中断服务子程序时，要用到一些寄存器单元，会破坏原有内容。要把有关寄存器单元的内容保存起来，送入堆栈，这就是所谓的“现场保护”。

两种操作：**数据压入 (PUSH)** 堆栈，**数据弹出 (POP)** 堆栈。数据压入堆栈，SP自动加1；数据弹出堆栈，SP自动减1。

2. 寄存器B

为执行乘法和除法而设。在不执行乘、除法操作的情况下，可把它当作一个普通寄存器来使用。

乘法：两乘数分别在A、B中，执行乘法指令后，乘积在BA中。

除法：被除数取自A，除数取自B，商存放在A中，余数存B中。

3. AUXR寄存器

AUXR是辅助寄存器，其格式如图2-7所示：

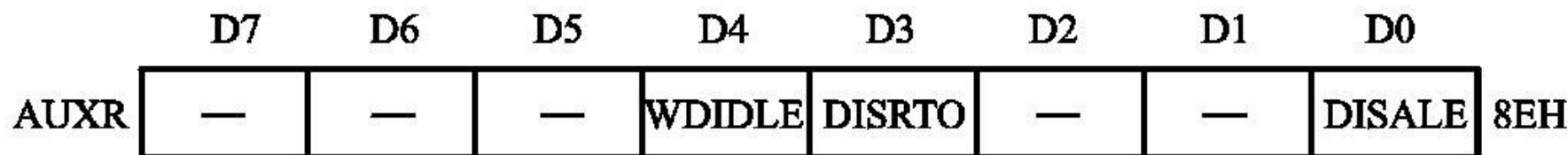


图2-7 AUXR寄存器的格式

其中：

DISALE： ALE的禁止/允许位。

0： ALE有效，发出脉冲；

1： ALE仅在执行MOVC和MOVX类指令时有效，不访问外部存储器时，ALE不输出脉冲信号。

DISRTO： 禁止/允许WDT溢出时的复位输出。

0： WDT溢出时，在RST引脚输出一个高电平脉冲；

1： RST引脚仅为输入脚。

WDIDLE： WDT在空闲模式下的禁止/允许位。

0： WDT在空闲模式下继续计数；

1： WDT在空闲模式下暂停计数。

4. 数据指针DPTR0和DPTR1

双数据指针寄存器，便于访问数据存储器。

AUXR1的DPS位用于选择两个数据指针。当DPS=0时，选用DPTR0；当DPS=1时，选用DPTR1。

数据指针可作为一个16位寄存器来用，也可作为两个独立的8位寄存器DP0H（或DP1H）和DP0L（或DP1L）来用。

5. AUXR1寄存器

AUXR1是辅助寄存器，格式如图2-8所示：

DPS：数据指针寄存器选择位。

0：选择数据指针寄存器DPTR0；

1：选择数据指针寄存器DPTR1。

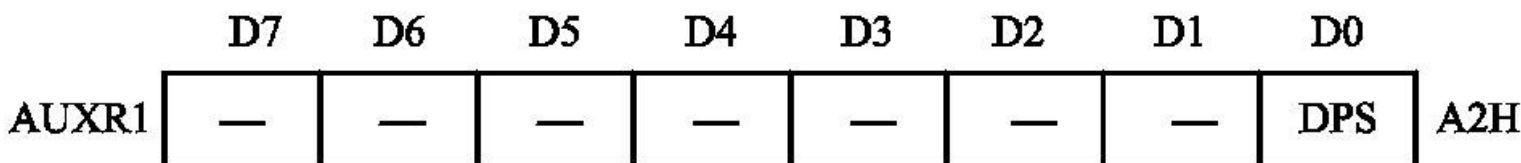


图2-8 AUXR1寄存器的格式

数据指针可作为一个16位寄存器来用，也可作为两个独立的8位寄存器DP0H（或DP1H）和DP0L（或DP1L）来用。

6. 看门狗定时器WDT

WDT包含一个14位计数器和看门狗定时器复位寄存器——WDTRST）。

当CPU由于干扰，程序陷入死循环或跑飞状态时，WDT提供了一种使程序恢复正常运行的有效手段。

2.4.4 位地址空间

219个寻址位的位地址，位地址范围为 $00H \sim FFH$ ，其中 $00H \sim 7FH$ 这128位处于片内RAM 字节地址 $20H \sim 2FH$ 单元中，如表2-5所示。

其余的91个可寻址位分布在特殊功能寄存器SFR中，见表2-6。

从表2-6中可发现一个规律，凡是可位寻址的特殊功能寄存器，其最低的位地址等于其字节地址，且其字节地址的末位都为0H或8H。

表 2-5 片内 RAM 的位地址

字 节 地 址	位 地 址							
	D7	D6	D5	D4	D3	D2	D1	D0
2FH	7FH	7EH	7DH	7CH	7BH	7AH	79H	78H
2EH	77H	76H	75H	74H	73H	72H	71H	70H
2DH	6FH	6EH	6DH	6CH	6BH	6AH	69H	68H
2CH	67H	66H	65H	64H	63H	62H	61H	60H
2BH	5FH	5EH	5DH	5CH	5BH	5AH	59H	58H
2AH	57H	56H	55H	54H	53H	52H	51H	50H
29H	4FH	4EH	4DH	4CH	4BH	4AH	49H	48H
28H	47H	46H	45H	44H	43H	42H	41H	40H
27H	3FH	3EH	3DH	3CH	3BH	3AH	39H	38H
26H	37H	36H	35H	34H	33H	32H	31H	30H
25H	2FH	2EH	2DH	2CH	2BH	2AH	29H	28H
24H	27H	26H	25H	24H	23H	22H	21H	20H
23H	1FH	1EH	1DH	1CH	1BH	1AH	19H	18H
22H	17H	16H	15H	14H	13H	12H	11H	10H
21H	0FH	0EH	0DH	0CH	0BH	0AH	09H	08H
20H	07H	06H	05H	04H	03H	02H	01H	00H



表 2-6 AT89S52 单片机 SFR 中的位地址分布

特殊功能 寄存器	位 地 址								字节地址
	D7	D6	D5	D4	D3	D2	D1	D0	
B	F7H	F6H	F5H	F4H	F3H	F2H	F1H	FOH	FOH
Acc	E7H	E6H	E5H	E4H	E3H	E2H	E1H	EOH	EOH
PSW	D7H	D6H	D5H	D4H	D3H	D2H	D1H	DOH	DOH
T2CON	—	—	CDH	CCH	CBH	CAH	C9H	C8H	C8H
IP	—	—	BDH	BCH	BBH	BAH	B9H	B8H	B8H
P3	B7H	B6H	B5H	B4H	B3H	B2H	B1H	BOH	BOH
IE	AFH	—	ADH	ACH	ABH	AAH	A9H	A8H	A8H
P2	A7H	A6H	A5H	A4H	A3H	A2H	A1H	AOH	AOH
SCON	9FH	9EH	9DH	9CH	9BH	9AH	99H	98H	98H
P1	97H	96H	95H	94H	93H	92H	91H	90H	90H
TCON	8FH	8EH	8DH	8CH	8BH	8AH	89H	88H	88H
P0	87H	86H	85H	84H	83H	82H	81H	80H	80H



作为对AT89S52存储器结构的总结，**图2-9为各类存储器的结构图**。从图中可清楚看出各类存储器在存储器空间的位置。

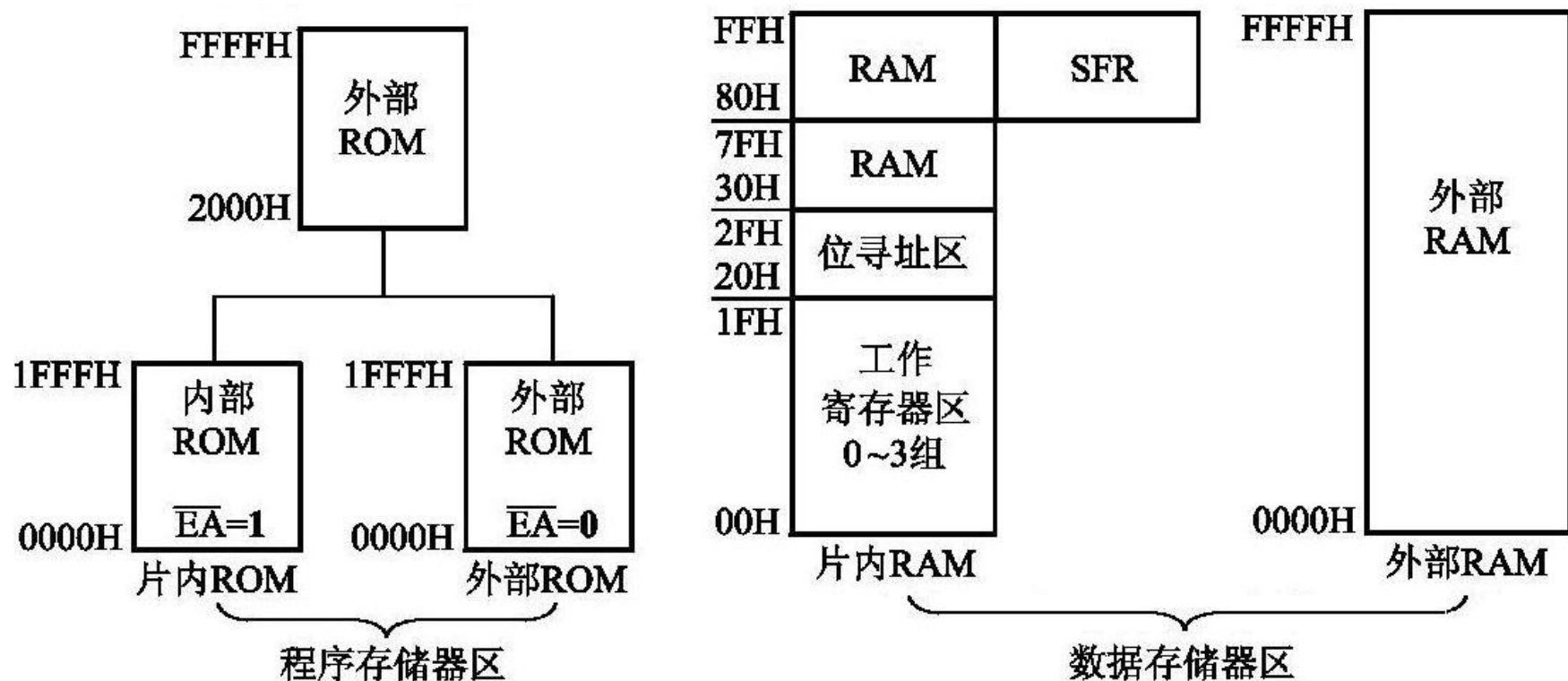


图2-9 AT89S52单片机的存储器结构

2.5 AT89S52的并行I/O端口

4个双向的8位并行I/O端口，分别记为P0、P1、P2和P3，其中**输出锁存器**属于**特殊功能寄存器**。端口的每一位均由输出锁存器、输出驱动器和输入缓冲器组成，4个端口按**字节输入/输出**外，也可**位寻址**，便于实现位控功能。

2.5.1 P0口

P0口是一个**双功能**的8位并行端口，**字节地址**为80H，位地址为80H~87H。

端口的各位具有完全相同但又相互独立的电路结构，**P0口**某一位的**位电路结构**如**图2-10**所示。

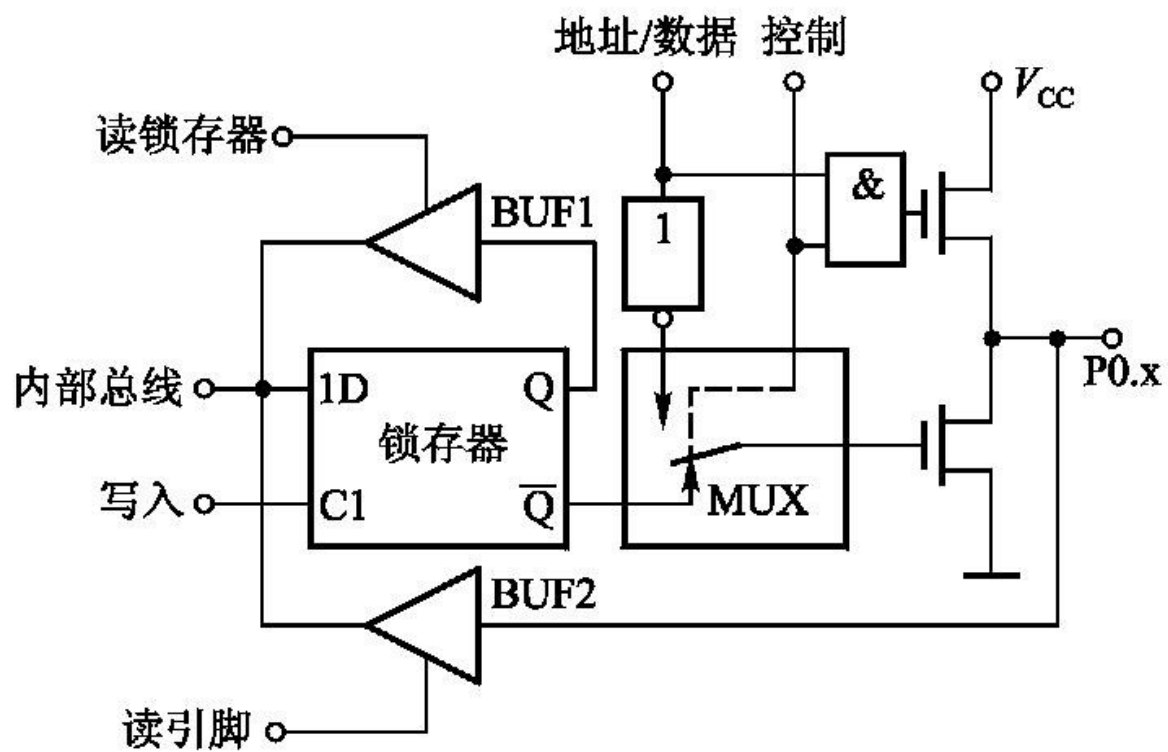


图2-10 P0口某一位的位电路结构

1. P0口的工作原理

(1) P0口用作系统的地址/数据总线用

当AT89S52外部扩展存储器或I/O时，P0口作为单片机系统复用的地址/数据总线使用。此时，图2-10中的“控制”信号使转接开关MUX打向上面，接通反相器的输出，同时使与上方场效应管栅极相连的“与门”处于开启状态。

当输出的“地址/数据”信息为1时，“与门”输出为1，上方的场效应管导通，下方的场效应管截止，P0.x引脚输出为1。

当输出的“地址/数据”信息为0时，上方的场效应管截止，下方的场效应管导通，P0.x引脚输出为0。可见P0.x引脚的输出状态随“地址/数据”信号

的状态变化而变化。上方的场效应管起到内部上拉电阻的作用。

当P0口作为数据总线输入时，仅从外部存储器（或外部I/O）读入信息，对应的“控制”信号使MUX接通锁存器的Q'端。由于P0口作为地址/数据复用方式访问外部存储器时，CPU自动向P0口写入FFH，使下方的场效应管截止，由于“控制”信号为0，上方的场效应管也截止，从而保证数据信息的高阻悬浮输入，从外部P0.x引脚输入的数据信息直接通过输入缓冲器BUF2进入内部总线。

由以上分析可见，P0口作为总线端口使用时，具有高电平、低电平和高阻悬浮输入3种状态的端口，此时P0口是一**真正的双向端口**，简称双向口。

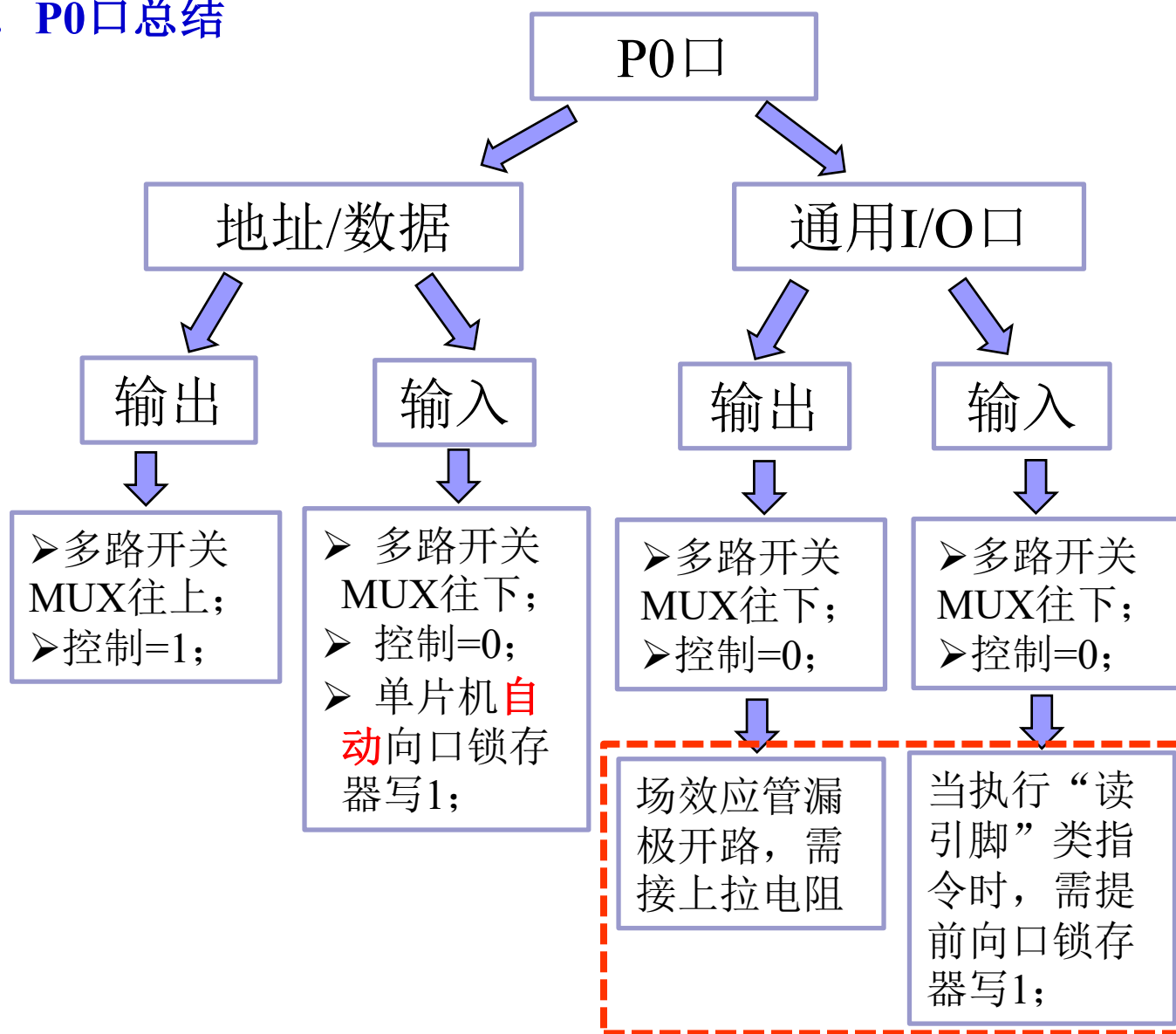
(2) P0口用作通用I/O口使用

当P0口不作地址/数据总线使用时，也可作为通用的I/O口使用，各引脚需要在片外接上拉电阻，此时端口不存在高阻抗的悬浮状态，对应的“控制”端信号为0，MUX打向下面，接通锁存器的Q*端，“与门”输出为0，上方的场效应管截止。

P0口用作通用I/O口输出时，来自CPU的“写”脉冲加在D锁存器的CP端，内部总线上的数据写入D锁存器，并由引脚P0.x输出。当D锁存器为1时，Q*端为0，下方场效应管截止，输出为漏极开路，此时，引脚P0.x必须外接上拉电阻才能有高电平输出；当D锁存器为0时，下方场效应管导通，P0口输出为低电平。

P0口作为通用I/O口输入时，有两种读入方式：“读锁存器”和“读引脚”。当CPU发出“读锁存器”类指令时，锁存器的状态由Q端经上方的三态缓冲器BUF1进入内部总线；当CPU发出“读引脚”类指令时，锁存器的输出状态=1（即Q*端为0），从而使下方场效应管截止，引脚的状态经下方的三态缓冲器BUF2进入内部总线。

2. P0口总结



额外的
操作

2. P0口总结

综上所述，P0口具有如下特点：

- (1) 当P0口用作地址/数据总线口使用时，是一个真正的双向口，用作与外部扩展的存储器或I/O连接，输出低8位地址和输出/输入8位数据。
- (2) 当P0口用作通用I/O口使用时，各引脚需要在片外接上拉电阻，此时端口不存在高阻抗的悬浮状态，因此是一个准双向口。

大多数情况下，单片机片外都要扩展RAM或I/O接口芯片，此时P0口只能作为复用的地址/数据总线使用。如果单片机片外没有扩展外部RAM和I/O接口芯片，不作地址/数据总线口使用时，P0口才能作为通用I/O口使用，P0口的引脚需在片外接一个上拉电阻。

2.5.2 P1口

P1口为通用I/O端口，字节地址为90H，位地址为90H~97H。位电路结构见[图2-11](#)。

1. P1口的工作原理

P1口只作为通用I/O口使用。

- (1) P1口作为输出口时，若CPU输出1， $Q=1$ ， $Q^*=0$ ，场效应管截止，P1.x引脚的输出为1；若CPU输出0， $Q=0$ ， $Q^*=1$ ，场效应管导通，P1.x引脚的输出为0。

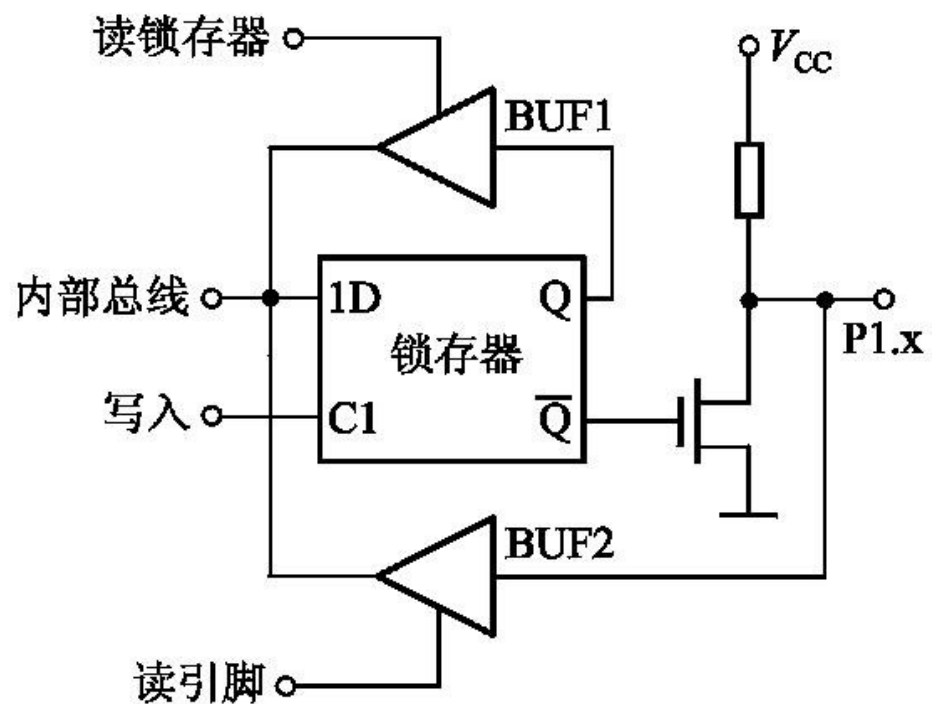


图2-11 P1口某一位的位电路结构

(2) P1口作为输入口时，分为“读锁存器”和“读引脚”两种方式。

“读锁存器”时，锁存器的输出端Q的状态经输入缓冲器BUF1进入内部总线；“读引脚”时，先向锁存器写1，使场效应管截止，P1.x引脚上的电平经输入缓冲器BUF2进入内部总线。

2. P1口总结

P1口由于有内部上拉电阻，没有高阻抗输入状态，故为准双向口。作为输出口时，不需要在片外接上拉电阻。

P1口“读引脚”输入时，必须先向P1口的锁存器写入1。

2.5.3 P2口

双功能口，字节地址为A0H，位地址为A0H~A7H。P2口某一位的位电路结构如图2-12所示。

1. P2口的工作原理

(1) P2口用作高8位地址总线口

在内部控制信号作用下，MUX与“地址”接通。当“地址”线为0时，场效应管导通，P2.x引脚输出0；当“地址”线为1时，场效应管截止，P2.x引脚输出1。

(2) P2口用作通用I/O口

在内部控制信号作用下，MUX与锁存器的Q端接通。

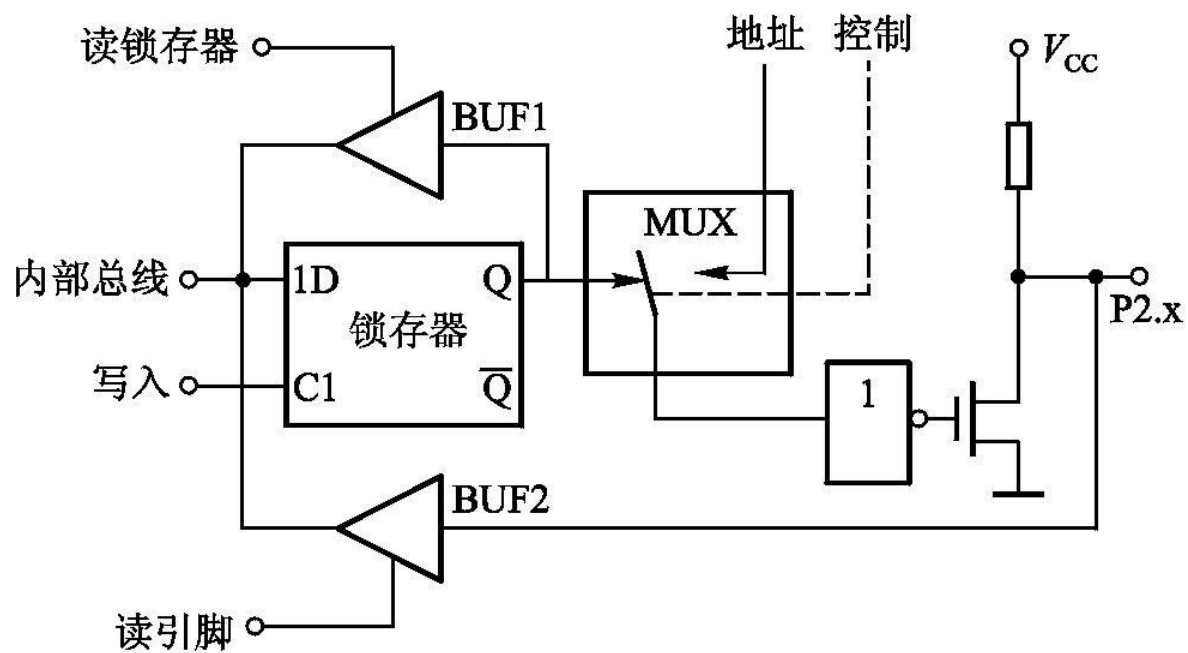


图2-12 P2口某一位的位电路结构

CPU输出1时， $Q=1$ ，场效应管截止，P2.x引脚输出1；**CPU输出0**时， $Q=0$ ，场效应管导通，P2.x引脚输出0。

输入时，分为“**读锁存器**”和“**读引脚**”两种方式。“读锁存器”时，Q端信号经输入缓冲器BUF1进入内部总线；“读引脚”时，先向锁存器写1，使场效应管截止，P2.x引脚上的电平经输入缓冲器BUF2进入内部总线。

2. P2口总结

P2口作为高8位地址总线使用时，可输出外部存储器或I/O的高8位地址，与P0口输出并经锁存器的锁存的低8位地址一起构成16位地址，共可寻址64KB的片外地址空间。当P2口作为高8位地址输出口时，输出锁存器的内容保持不变。

P2口作为通用I/O口使用时，为准双向口，功能与P1口一样。

一般情况下，P2口大多作为高8位地址总线口使用，这时就不能再作为通用I/O口。如果不作为地址总线口使用，可作为通用I/O口使用。

2.5.4 P3口

由于引脚数目有限，在P3口增加了第二功能。每1位都可以分别定义为第二输入功能或第二输出功能。P3口字节地址为B0H，位地址B0H~B7H。P3口某一位的位电路结构见图2-13。

1. P3口的工作原理

(1) P3口用作第二输入/输出功能。

当选择第二输出功能时，该位的锁存器需要置“1”，使“与非门”为开启状态。当第二输出为1时，场效应管截止，P3.x引脚输出为1；当第二输出为0时，场效应管导通，P3.x引脚输出为0。

当选择第二输入功能时，该位的锁存器和第二输出功能端均应置1，保证场效应管截止，P3.x引脚的信息由输入缓冲器BUF3的输出获得。

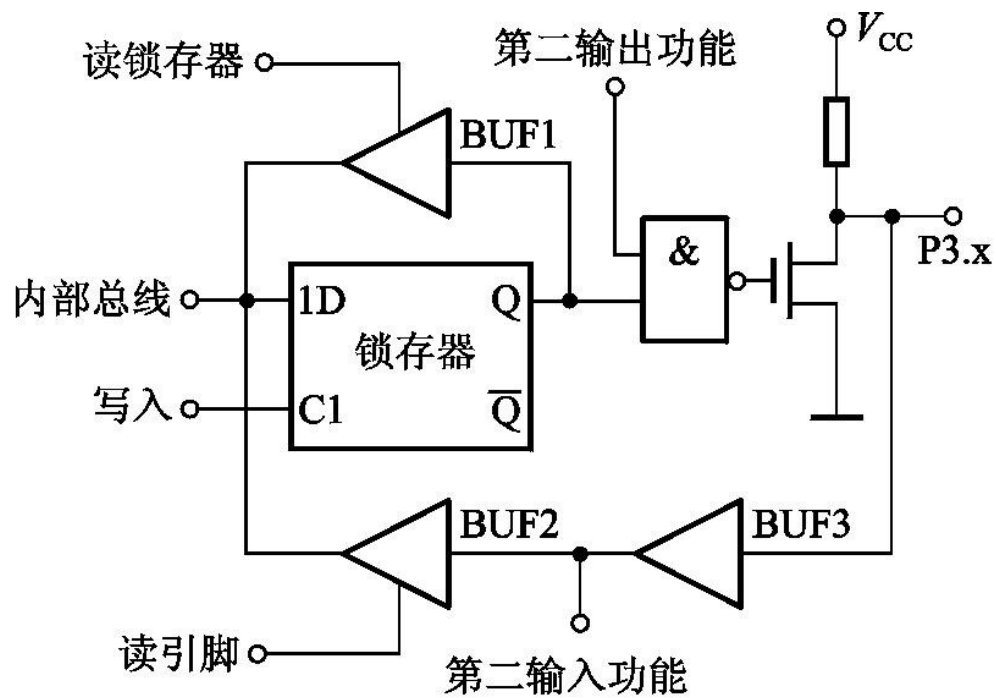


图2-13 P3口某一位的位电路结构

(2) P3口用作第一功能—通用I/O口。

当P3口用作**通用I/O的输出**时，“第二输出功能”端应保持高电平，“与非门”为开启状态。CPU输出1时， $Q=1$ ，场效应管截止，P3.x引脚输出为1；CPU输出0时， $Q=0$ ，场效应管导通，P3.x引脚输出为0。

当P3口用作**通用I/O的输入**时，P3.x位的输出锁存器和“第二输出功能”端均应置1，场效应管截止，P3.x引脚信息通过输入BUF3和BUF2进入内部总线，完成“读引脚”操作。

当P3口用作**通用I/O的输入**时，也可执行“读锁存器”操作，此时Q端信息经过缓冲器BUF1进入内部总线。

2. P3口总结

P3口内部有上拉电阻，不存在高阻抗输入状态，故为**准双向口**。

由于P3口每一引脚有**第一功能**与**第二功能**，究竟是使用哪个功能，完全由单片机执行的指令控制来**自动切换**的，用户不需要进行任何设置。

引脚输入部分有两个缓冲器，第二功能的输入信号取自缓冲器BUF3的输出端，第一功能的输入信号取自缓冲器BUF2的输出端。

2.6 时钟电路与时序

时钟电路产生AT89S52工作时所必需的控制信号，在时钟信号的控制下，严格按时序执行指令。

执行指令时，CPU首先到程序存储器中取出需要执行的指令操作码，然后译码，并由时序电路产生一系列控制信号完成指令所规定的操作。

CPU发的时序信号两类，一类用对片内各个功能部件控制，用户无须了解；另一类用于对片外存储器或I/O口的控制，这部分时序对于分析、设计硬件接口电路至关重要。

2.6.1 时钟电路设计

时钟频率直接影响单片机的速度，时钟电路的质量也直接影响单片机系统的稳定性。常用的时钟电路有两种方式，一种是内部时钟方式，另一种是外部时钟方式。

AT89S52单片机的最高时钟频率为33MHz。

1. 内部时钟方式

AT89S52内部有一个用于构成振荡器的高增益反相放大器，输入端为芯片引脚XTAL1，输出端为引脚XTAL2。这两个引脚跨接石英晶体振荡器和微调电容，构成一个稳定的自激振荡器，图2-14是AT89S52内部时钟方式的电路。

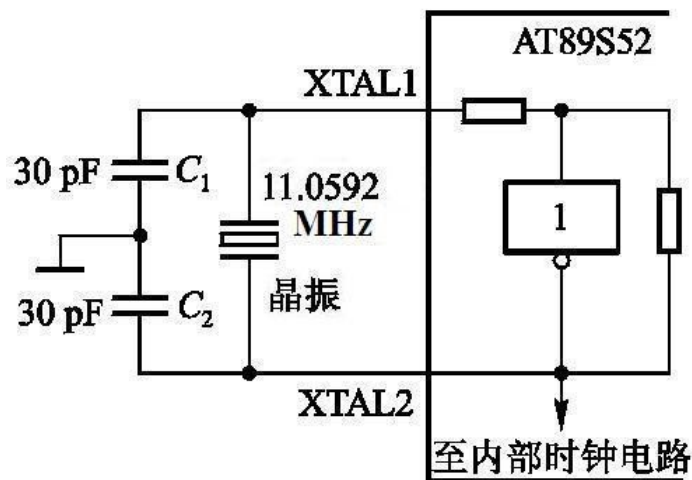


图2-14 内部时钟方式电路

电路中的电容 C_1 和 C_2 的典型值通常选择为 30pF。晶体的振荡频率通常选择 6MHz、12MHz（可得到准确的定时）或 11.0592MHz（在串行通信时用，可得到准确的串行通信波特率）。

2. 外部时钟方式

用现成的外部振荡器产生脉冲信号，**常用于多片**AT89S52同时工作，以便于多片AT89S52单片机之间的同步，一般为低于12MHz的方波。

外部时钟源直接接到XTAL1端，XTAL2端悬空，见图2-15。

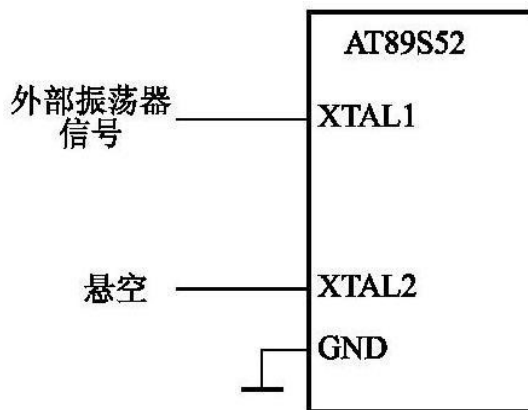


图2-15 AT89S52的外部时钟方式电路

3. 时钟信号的输出

当使用片内振荡器，XTAL1、XTAL2引脚还能为应用系统中的其他芯片提供时钟，但需增加驱动能力。引出的方式有两种，如图2-16所示。

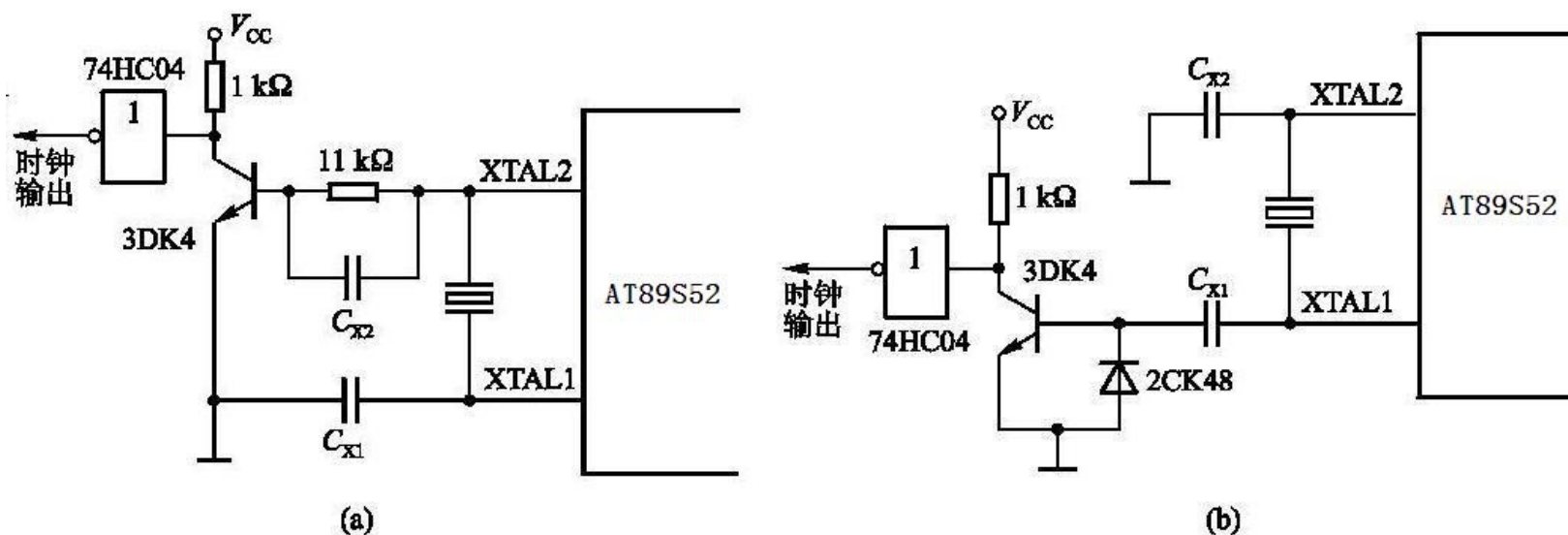


图2-16 时钟信号的两引出方式

2.6.2 时钟周期、机器周期、指令周期与指令时序

各种指令时序与时钟周期相关。

1. 时钟周期

时钟控制信号的基本时间单位。若晶振频率为 f_{osc} ，则时钟周期 $T_{\text{osc}} = 1 / f_{\text{osc}}$ 。如 $f_{\text{osc}} = 6\text{MHz}$ ， $T_{\text{osc}} = 166.7\text{ns}$ 。

2. 机器周期

CPU完成一个基本操作所需时间为机器周期。执行一条指令分为几个机器周期。每个机器周期完成一个基本操作，如取指令、读或写数据等。每12个时钟周期为1个机器周期。

1个机器周期包括12个时钟周期，分6个状态：S1~S6。每个状态又分两拍：P1和P2。因此，一个机器周期中的12个时钟周期表示为S1P1、S1P2、S2P1、S2P2、…、S6P2，如图2-17所示。

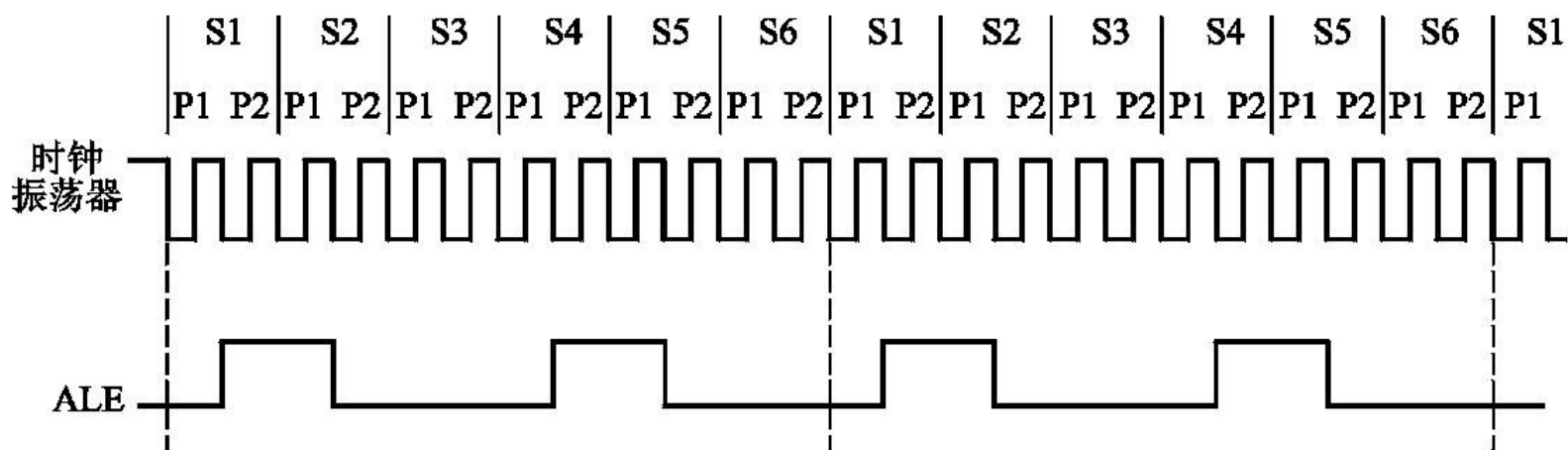


图2-17 AT89S52的机器周期

3. 指令周期

执行一条指令所需的时间。简单的单字节指令，取出指令立即执行，只需一个机器周期的时间。而有些复杂的指令，如转移、乘、除指令则需两个或多个机器周期。

从指令执行时间看：

- 单字节和双字节指令一般为单机器周期和双机器周期；
- 三字节指令都是双机器周期；
- 乘、除指令占用4个机器周期。

2.7 复位操作和复位电路

单片机的初始化操作，给复位脚RST加上大于2个机器周期（即24个时钟振荡周期）的高电平就使AT89S52复位。

2.7.1 复位操作

复位时，PC初始化为0000H，程序从0000H单元开始执行。

除系统的正常初始化外，当程序出错（如程序跑飞）或操作错误使系统处于死锁状态时，需按复位键使RST脚为高电平，使AT89S52摆脱“跑飞”或“死锁”状态而重新启动程序。

复位操作还对**其他一些寄存器**有影响，这些寄存器复位时的状态见**表2-7**。

由**表2-7**可看出，复位时，**SP=07H**，而**P0~P3**引脚均为高电平。
在某些控制应用中，要**注意考虑P0~P3引脚的高电平**对接在这些引脚上的**外部电路的影响**。

例如，当P1口某个引脚外接一个继电器绕组，当复位时，该引脚为高电平，继电器绕组就会有电流通过，就会吸合继电器开关，使开关接通，可能会引起意想不到的后果。

表 2-7 复位时片内各寄存器的状态

寄 存 器	复 位 状 态	寄 存 器	复 位 状 态
PC	0000H	DP1H	00H
Acc	00H	DP1L	00H
PSW	00H	TMOD	00H
B	00H	TCON	00H
SP	07H	TH0、TH1	00H
DPTR	0000H	TLO、TL1	00H
P0~P3	FFH	SCON	00H
IP	××00 0000 B	SBUF	×××××××× B
IE	0×00 0000 B	PCON	0×××0 0 0 0 B
DPOH	00H	AUXR	×××0 0××0 B
DPOL	00H	AUXR1	××××××××0 B
WDTRST	×××××××× B		

2.7.2 复位电路设计

AT89S52单片机的复位是由外部的复位电路实现的。复位电路应兼有上电复位和人工按键复位两种功能。典型的复位电路如图2-18所示。

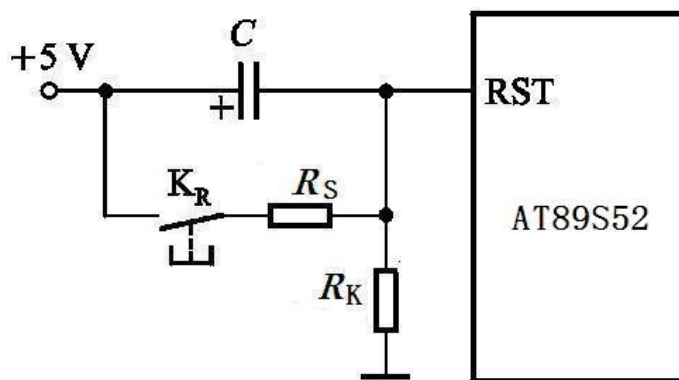


图2-18 复位电路

上电自动复位是给电容 C 充电加给RST引脚一个短的高电平信号，此信号随着 V_{CC} 对电容 C 的充电过程而逐渐回落，即RST引脚上的高电平持续时间取决于电容 C 充电时间。为保证系统可靠复位，RST引脚上的高电平必须维持足够长的时间。

除了上电复位外，有时还需要人工按键复位。按下按键后，通过两个电阻 R_s 和 R_k 的分压，在RST端产生高电平，按键按下的时间决定了复位的时间。

当时钟频率选用6MHz时，电容 C 的参考取值为 $22\mu\text{F}$ ，两个电阻 R_s 和 R_k 的参考阻值分别为 220Ω 和 $1\text{k}\Omega$ 。

一般来说，单片机的复位速度比外部扩展的I/O接口电路快些。因此在实际应用设计中，为保证系统可靠复位，在单片机的初始化程序段应安排一定的复位延迟时间，以保证单片机与外部扩展的I/O接口电路都能可靠地复位。

2.8 AT89S52单片机的最小应用系统

AT89S52本身片内有8KB闪烁存储器，256B的RAM单元，4个I/O口，外接时钟电路和复位电路即构成了一个AT89S52单片机最小应用系统，如图2-19所示。

该最小应用系统只能作为小型的数字量的测控单元。

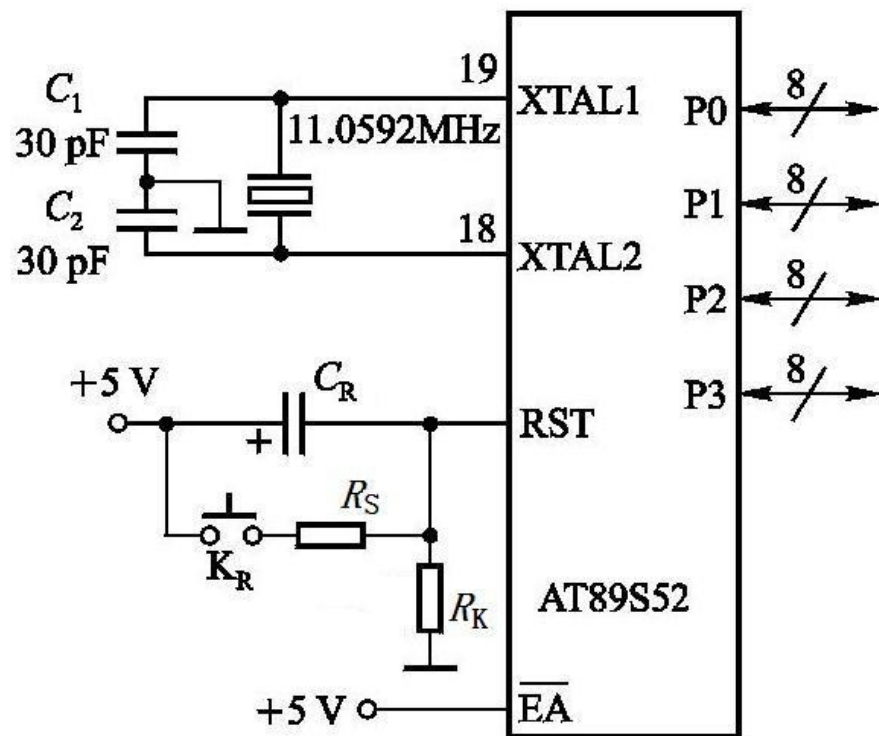


图2-19 AT89S52单片机的最小应用系统

2.9 看门狗定时器（WDT）功能及应用

单片机应用系统受到干扰可能会引起程序“跑飞”或“死循环”，会使系统失控。如果操作人员在场，可按人工复位按钮，强制系统复位。但操作人员不可能一直监视着系统，即使监视着系统，也往往是在引起不良后果之后才进行人工复位。能不能不要人来监视，使系统摆脱失控状态，重新从复位入口地址0000H处执行程序呢？这时可采用“看门狗”技术。

“看门狗”技术就是使用一个“看门狗”定时器来对系统时钟不断计数，监视程序的运行。当看门狗定时器启动运行后，为防止看门狗定时器的不必要溢出而引起单片机的非正常复位，应定期地把看门狗定时器清0，我们称为“喂狗”，以保证看门狗定时器不溢出。

“看门狗”部件包含1个14位看门狗定时器和看门狗复位寄存器（表2-4中的特殊功能寄存器WDTRST，地址A6H）。开启看门狗定时器后，14位定时器会自动对系统时钟12分频后的信号计数，即每16384（ 2^{14} ）个机器周期溢出一次，并产生一个高电平复位信号，使单片机复位。采用12MHz的系统时钟时，则每16384 μ s产生一个复位信号。

当由于干扰，使单片机程序“跑飞”或陷入“死循环”时，单片机也就不能正常运行程序来定时地把看门狗定时器清0，使得看门狗定时器计满溢出，在AT89S52的RST引脚上输出一个正脉冲（宽度为98个时钟周期），使单片机复位，然后从单片机的复位入口0000H处重新开始执行主程序，从而使程序摆脱“跑飞”或“死循环”状态，让单片机归复于正常的工作状态。

看门狗定时器的启动和清0的方法是一样的。实际应用中，用户只要向寄存器**WDTRST**（地址为A6H）**先写入1EH，接着写入E1H**，看门狗定时器便启动计数。为防止看门狗定时器启动后产生不必要的溢出，在执行程序的过程中，应在看门狗定时器计数未溢出之前，即16384 μ s（时钟为12MHz时）内，不断地复位清“0”看门狗（“喂狗”）。下面介绍看门狗抗干扰的C51语言应用编程。

由于在头文件reg51.h中，并没有声明WDTRST寄存器，若使用看门狗，**必须事先声明看门狗的WDTRST寄存器**，如：

```
sfr WDTRST=0xa6
```

声明后，用户只要向寄存器WDTRST（地址为A6H）先写入**1EH**，然后再写入**E1H**：

```
WDTRST=0x1e;  
WDTRST=0xe1;
```

看门狗便在未启动时启动计数运行或计数运行中清0。

下面通过一个例子来说明看门狗的使用。

【例】看门狗的应用编程。

```
#include<reg51.h>
sfr WDTRST=0xa6
.....
main()
{
    .....;
    WDTRST=0x1e;           //启动看门狗运行
    WDTRST=0xe1;
    .....;
    while(1)               //无限循环
    {
        WDTRST=0x1e;       //清0看门狗并启动运行
        WDTRST=0xe1;
```

```
.....;          //执行时间必须小于16384μs（系统时钟为12MHz时）
```

```
}
```

```
}
```

上述程序在做无限循环的运行，通过看门狗可防止程序在执行过程中“跑飞”或“死循环”，因为只要程序一跑出while()循环，不执行清0看门狗的两条命令，使得看门狗不能及时清0，看门狗就会溢出使单片机复位，使程序从main()处开始重新运行。所以使用看门狗时一定要注意，一定要在看门狗启动后的16384μs（系统时钟12MHz）之内将看门狗定时器及时清0，以防其溢出导致单片机复位。

为防止看门狗启动后产生不必要的溢出，应在执行程序的过程中，及时地把WDTRST清0，即向WDTRST寄存器写入数据1EH和E1H。

在程序编写中，一般把WDTRST清0的这两条指令，设计为一个函数WDTClear():

```
void WDTClear(void)                                //看门狗清0函数
{
    WDTRST=0x1e;                                    //看门狗清0
    WDTRST=0xe1;
}
```

只要在程序的正常运行中，不断调用该函数，把看门狗计数器清0，使其不溢出即可。

2.10 低功耗节电模式

两种低功耗节电工作模式：**空闲模式**（idle mode）和**掉电保持模式**（power down mode）。

掉电保持模式下， V_{CC} 可由**后备电源**供电。**图2-20**为两种节电模式的**内部控制电路**。

两种节电模式可通过PCON的**位IDL**和**位PD**的设置来实现。格式如**图2-21**所示。

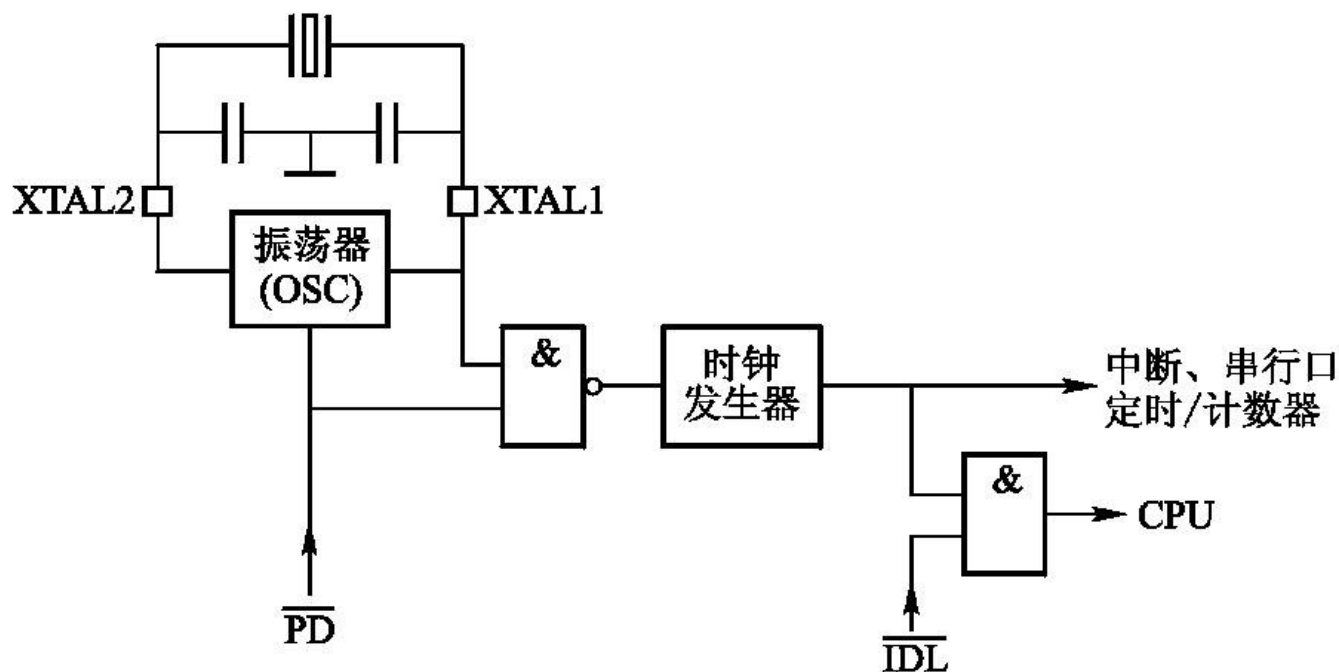


图2-20 低功耗节电模式的控制电路

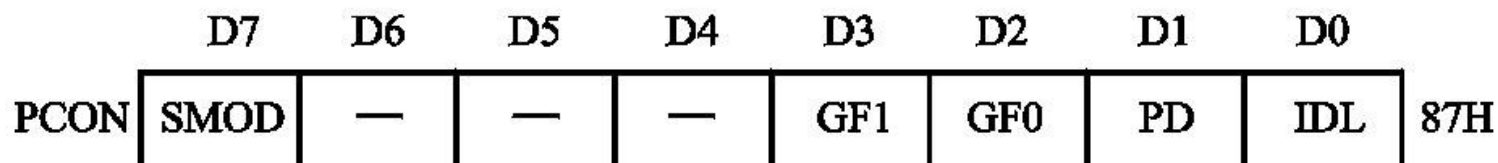


图2-21 特殊功能寄存器PCON的格式

PCON寄存器各位定义：

- **SMOD**：串行通信波特率选择（见串行口一章的介绍）。
- **—**：保留位。
- **GF1、GF0**：通用标志位，两个标志位用户使用。
- **PD**：掉电保持模式控制位，**PD=1**，则**进入掉电保持模式**。
- **IDL**：空闲模式控制位，若**IDL=1**，则**进入空闲运行模式**。

2. 10. 1 空闲模式

1. 空闲模式进入

如果用指令把寄存器PCON中的IDL位置“1”，由图2-20可见，则把通往CPU的时钟信号关断，单片机便进入空闲模式，虽然振荡器仍然运行，但是CPU进入空闲状态。此时，片内所有外围电路（中断系统、串行口和定时器）仍继续工作，SP、PC、PSW、A、P0~P3端口等所有其他寄存器，以及内部RAM和SFR中的内容均保持进入空闲模式前的状态。因为CPU耗电量通常要占芯片耗电的80%~90%，因此CPU停止工作则会大大降低功耗。

2. 空闲模式的退出

系统进入空闲模式后有**两种方法可退出**：

第一种是中断退出。由于在空闲模式下，中断系统还在工作，所以任意的中断请求被响应时，都可使IDL位由片内硬件自动清“0”，从而退出空闲模式，进入中断服务程序。

第二种是硬件复位退出。复位时，各个特殊功能寄存器都恢复默认状态，特殊功能寄存器寄存器PCON也不例外，复位使IDL位清“0”，退出空闲模式，单片机将从头开始执行程序。

2.10.2 掉电运行模式

1. 掉电模式的进入

用指令把PCON寄存器的PD位置1，便进入掉电模式。由图2-20，在掉电模式下，进入时钟振荡器的信号被封锁，振荡器停止工作。

由于没有时钟信号，内部的所有功能部件均停止工作，但片内RAM和SFR的原来的内容都被保留，有关端口的输出状态值都保存在对应的特殊功能寄存器中。

2. 掉电模式的退出

退出的方法是由**中断外部中断唤醒**（低电平触发或由下跳沿触发）或者硬件复位来唤醒单片机。需注意的是，使用中断唤醒单片机时，程序从原来停止处继续运行。当使用硬件复位来唤醒单片机时，程序将从头开始执行。

在掉电模式下， V_{cc} 可以降到2V，但是在进入掉电模式之前， V_{cc} 不能降低，而在准备退出掉电模式之前， V_{cc} 必须恢复正常的工作电压值，并维持约10ms的时间，使振荡器重新启动并稳定后方可退出掉电模式。

- 
- 课后作业
 - P41 填空题： 1-12
 - P42 简答题： 1、 2、 5

第3章 C51语言编程基础

第3章 C51编程语言基础

3.1 C51编程语言简介

3.1.1 C51语言与8051汇编语言的比较

3.1.2 C51语言与标准C语言的比较

3.2 C51语言程序设计基础

3.2.1 C51语言中的数据类型与存储类型

3.2.2 C51语言的特殊功能寄存器及位变量定义

3.2.3 C51语言的绝对地址访问

3.2.4 C51的基本运算

3.2.5 C51的分支与循环程序结构

3.2.6 C51的数组

3.2.7 C51的指针



3.3 C51语言的函数

3.3.1 函数的分类

3.3.2 中断服务函数

3.3.3 变量及存储方式

3.3.4 宏定义与文件包含

3.3.5 库函数



单片机应用系统日趋复杂，对程序的可读性、升级与维护以及模块化要求越来越高，对软件编程要求也越来越高，要求编程人员在短时间内编写出执行效率高、运行可靠的程序代码。同时，也要方便多个编程人员来进行协同开发。

C51语言是近年来在8051单片机开发中，普遍使用的程序设计语言，能直接对8051单片机硬件进行操作，既有高级语言特点，又有汇编语言特点，因此在8051单片机程序设计中，得到广泛使用。

3.1 C51编程语言简介

C51语言在标准C基础上针对8051硬件特点进行扩展，并向8051上移植，经多年努力，C51已成为公认的高效、简洁的8051单片机的实用高级编程语言。与8051汇编语言相比，C51语言在功能上、结构性、可读性、可维护性上有明显优势，易学易用。

3.1.1 C51语言与8051汇编语言比较

与8051汇编语言相比，C51有如下优点。

(1) **可读性好**。C51语言程序比汇编语言程序的可读性好，编程效率高，程序**便于修改、维护以及升级**。

(2) **模块化开发与资源共享**。用C51开发的程序模块可不经修改，直接被其他工程所用，使得开发者能够很好地**利用已有的大量标准C程序资源与丰富的库函数，减少重复劳动**，同时也有利于多个工程师进行协同开发。

(3) **可移植性好**。为某种型号单片机开发的C语言程序，只需把与硬件相关的**头文件和编译链接的参数**进行适当修改，就可方便地移植到其他型号的单片机上。

(4) **生成的代码效率高**。当前较好的C51语言编译系统编译出来的代码效率只比直接使用汇编语言**低20%左右**，如果使用**优化编译选项**，最高可达到**90%左右**。

3.1.2 C51语言与标准C语言的比较

C51语言与标准C语言间有许多相同地方，但也有自身特点。不同的嵌入式C语言编译系统之所以与标准C语言有不同的地方，主要是由于它们所针对的是不同的硬件系统。

C51语言基本语法与标准C相同，是在标准C的基础上进行适合8051内核单片机硬件的扩展。深入理解C51语言对标准C语言的扩展部分以及它们的不同之处，是掌握C51语言的关键之一。

C51语言与标准C语言一些差别如下。

(1) 库函数不同。标准C中不适合于嵌入式控制器系统的库函数，被排除在C51语言之外，如字符屏幕和图形函数。有些库函数必须针对8051的硬件特点来做出相应的开发。

例如，在标准C中，库函数`printf`和`scanf`，常用于屏幕打印和接收字符，而在C51语言中，主要用于串行口数据的收发。

(2) 数据类型有一定区别。在C51中增加几种8051单片机的数据类型，在标准C的基础上又扩展了4种类型。例如，8051单片机包含位操作空间和丰富的位操作指令，因此，C51语言与标准C语言相比增加了位类型。

(3) C51语言变量存储模式与标准C语言中变量存储模式数据不一样。

标准C最初是为通用计算机设计的，在通用计算机中只有一个程序和数据统一寻址的内存空间，而C51语言中变量的存储模式与8051单片机的各种存储区紧密相关。

(4) 数据存储类型不同。8051存储区可分为内部数据存储区、外部数据存储区以及程序存储区。

内部数据存储区可分为3个不同的C51存储类型：`data`、`idata`和`bdata`。

外部数据存储区分为2个不同的C51存储类型：`xdata`和`pdata`。

程序存储区只能读不能写，可能在8051内部或者在外部，C51语言提供了`code`存储类型用来访问程序存储区。

(5) 标准C语言没有处理单片机中断的定义，而C51语言中有专门的中断函数。

(6) 头文件不同。C51语言头文件必须把8051单片机内部的外设硬件资源（如定时器、中断、I/O等）相应的特殊功能寄存器写入到头文件内，而标准C不用。

(7) 程序结构的差异。由于8051单片机的硬件资源有限，它的编译系统不允许太多的程序嵌套。

但从数据运算操作、程序控制语句以及函数的使用上来说，C51与标准C几乎没有什么明显差别。如果程序设计者具备了有关标准C语言的编程基础，只要注意C51与标准C不同之处，并熟悉8051单片机的硬件结构，就能较快掌握C51编程。

3.2 C51语言程序设计基础

本节在标准C基础上，了解掌握C51的数据类型和存储类型、C51的基本运算与流程控制语句、C51函数以及C51程序设计的其他一些问题，为C51的程序开发打下基础。

3.2.1 C51语言中的数据类型与存储类型

1. 数据类型

Keil C51支持的基本数据类型见表3-1。

针对8051的硬件特点，C51在标准C基础上，扩展了4种数据类型（见表3-1中最后4行）。

注意，扩展的4种数据类型，不能使用指针来对它们存取。

表 3-1 Keil C51 支持的数据类型

数据类型	位数	字节数	值 域
signed char	8	1	-128~+127, 有符号字符变量
unsigned char	8	1	0~255, 无符号字符变量
signed int	16	2	-32 768~+32 767, 有符号整型数
unsigned int	16	2	0~65 535, 无符号整型数
signed long	32	4	-2 147 483 648~+2 147 483 647, 有符号长整型数
unsigned long	32	4	0~+4 294 967 295, 无符号长整型数
float	32	4	$\pm 1.175494\text{E}-38 \sim \pm 3.402823\text{E}+38$
double	64	8	$\pm 1.175494\text{E}-38 \sim \pm 3.402823\text{E}+38$
*	8~24	1~3	对象指针
bit	1		0 或 1
sfr	8	1	0~255
sfr16	16	2	0~65 535
sbit	1		可进行位寻址的特殊功能寄存器的某位的绝对地址

2. C51的扩展数据类型

下面对扩展的4种数据类型说明。

(1) 位变量bit的值可以是1 (true)，也可能是0 (false)。

(2) 特殊功能寄存器sfr。8051单片机的特殊功能寄存器分布在片内数据存储区的地址单元80H~FFH之间，“sfr”数据类型占用一个内存单元。利用它可访问8051单片机内部的所有特殊功能寄存器。

例如：sfr P1=0x90这一语句定义了P1端口在片内的寄存器，在程序后续的语句中可以用“P1=0xff”，使P1的所有引脚输出为高电平的语句来操作特殊功能寄存器。

(3) 特殊功能寄存器sfr16。

“sfr16”数据类型**占用两个内存单元**，用于操作占两个字节的特殊功能寄存器。**例如：**“sfr16 DPTR=0x82”语句定义了片内16位数据指针寄存器DPTR，其低8位字节地址为82H，高8位字节地址为83H。在程序的后续语句中就可对DPTR进行操作。

(4) 特殊功能位sbit。

sbit是指AT89S51片内特殊功能寄存器的可寻址位。例如：

```
sfr    PSW=0xd0;           //定义PSW寄存器地址为0xd0
sbit   OV=PSW^2;           //定义OV位为PSW.2
```

符号“^”前是特殊功能寄存器名字，“^”后的数字定义特殊功能寄存器可寻址位在寄存器中的位置，取值必须是0~7。

注意，不要把bit与sbit相混淆。bit定义普通的位变量，只能是0或1。sbit是定义特殊功能寄存器的可寻址位，值是可以进行位寻址的特殊功能寄存器的某位的绝对地址，例如，PSW寄存器0V位的绝对地址0xd2。

上面的例子还涉及到C51注释的写法问题，注释写法有2种：

(1) //.....，两个斜杠后面跟着的为注释语句，本写法只能注释一行，当换行时，必须在新行上重新写两个斜杠。

(2) /*.....*/，本写法可注释任一行，即斜杠星号与星号斜杠之间的所有文字都作为注释，即注释有多行时，只需在注释的开始处，加斜杠星号，在注释的结尾处，加上星号斜杠即可。

加注释的目的是为了便于读懂程序，所有注释都不参与程序编译，编译器在编译过程中会自动删去注释。

3. 数据存储类型

在讨论C51数据类型时，须同时提及它的存储类型，以及它与8051单片机存储器结构的关系，因为C51定义的任何数据类型必须以一定的方式，定位在8051单片机的某一存储区中，否则没有任何实际意义。

8051有片内、片外数据存储区，还有程序存储区。

片内的数据存储区是可读写的，8051的衍生系列最多可有256字节的内部数据存储区（例如AT89S52单片机），其中低128字节可直接寻址，高128字节（80H～FFH）只能间接寻址，从地址20H开始的16字节可位寻址。

内部数据存储区可分为3个不同的数据存储类型：`data`、`idata`和`bdata`。

访问片外数据存储区比访问片内数据存储区慢，因为访问片外数据存储区要通过对数据指针加载地址来间接寻址访问。

C51提供两种不同的数据存储类型`xdata`和`pdata`来访问片外数据存储区。

程序存储区只能读不能写，可能在8051单片机内部或者外部，或外部和内部都有，由8051单片机硬件连线决定，C51提供了`code`存储类型来访问程序存储区。

C51存储类型与8051实际的存储空间的对应关系见表3-2。

表 3-2 C51 语言存储类型与 8051 存储空间的对应关系

存储区	存储类型	与存储空间的对应关系
DATA	data	片内 RAM 直接寻址区，位于片内 RAM 的低 128 字节
BDATA	bdata	片内 RAM 位寻址区，位于 20H~2FH 空间
IDATA	idata	片内 RAM 的 256 字节，必须间接寻址的存储区
XDATA	xdata	片外 64KB 的 RAM 空间，使用 @DPTR 间接寻址
PDATA	pdata	片外 RAM 的 256 字节，使用 @Ri 间接寻址
CODE	code	程序存储区，使用 DPTR 寻址

下面对表3-2各种存储区作以说明。

(1) DATA区。寻址最快，应把常使用的变量放在该区，但该区存储空间有限，DATA区除了包含程序变量外，还包含了堆栈和寄存器组。

DATA区声明中的存储类型标识符为data，通常指片内RAM128字节的内部

数据存储的变量，可直接寻址。

声明举例：

```
unsigned char data system_status=0;  
unsigned int data unit_id[8];  
char data inp_string[20];
```

标准变量和用户自声明变量都可存储在DATA区中，只要不超过DATA区的范围即可，由于C51用默认的寄存器组来传递参数，这样DATA区至少失去8字节空间。

另外，当内部堆栈溢出的时候，程序会莫名其妙地复位。这是因为8051没有报错机制，堆栈溢出只能以这种方式表示，因此要留有较大的堆栈空间来防止堆栈溢出。

(2) BDATA区。DATA中的位寻址区，在该区中声明变量就可进行位寻址。BDATA区声明中的存储类型标识符为**bdata**，指的是片内RAM可位寻址

的16字节存储区（字节地址为20H~2FH）中的128个位。

下面是在BDATA区中声明的位变量和使用位变量的例子：

```
unsigned char bdata status_byte;  
unsigned int bdata status_word;  
sbit stat_flag=status_byte^4;  
if(status_word^15)  
{ ..... }  
stat_flag=1;
```

C51编译器不允许在BDATA区中声明float和double型变量。

(3) IDATA区。该区使用寄存器作为指针来对片内RAM进行间接寻址，常用来存放使用比较频繁的变量。与外部存储器寻址相比，它的指令执行周期和代码长度相对较短。

IDATA区声明中的存储类型标识符为idata，指的是片内RAM的256字节的存储区，只能间接寻址，速度比直接寻址慢。

声明举例如下：

```
unsigned char idata system_status=0;  
unsigned int idata unit_id[8];  
char idata inp_string[16];  
float idata out_value;
```

(4) **PDATA区**和**XDATA区** 位于片外存储区，**PDATA区**和**XDATA区**声明中的存储类型标识符分别为**pdata**和**xdata**。

PDATA区只有256字节，仅指定256字节的外部数据存储区。

但**XDATA区**最多可达64KB，对应的**xdata**存储类型标识符可指定外部数据区64KB内的任何地址。

对PDATA区的寻址要比对XDATA区寻址快，因为对PDATA区寻址，只需装入8位地址，而对XDATA区寻址要装入16位地址，所以尽量把外部数据存储在PDATA区中。

对PDATA区和XDATA区的声明举例如下：

```
unsigned char xdata system_status=0;  
unsigned int pdata unit_id[8];  
char xdata inp_string[16];  
float pdata out_value;
```

由于外部数据存储器与外部I/O口是统一编址的，外部数据存储器地址段中除了包含数据存储器地址外，还包含外部I/O口的地址。对外部数据存储器及外部I/O口的寻址将在本章的绝对地址寻址中介绍。

(5) 程序存储区CODE。程序存储区CODE声明的标识符为code，储存的数据不可改变。在C51编译器中可用存储区类型标识符code来访问程序存储区。

声明举例如下：

```
unsigned char code a[ ]={0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06,  
                          0x07, 0x08};
```

上面介绍了C51的数据存储类型，其大小和值域见表3-3。

单片机访问片内RAM比访问片外RAM相对快一些，所以应尽量把频繁使用的变量置于片内RAM。即采用data、bdata或idata存储类型，而将容量较大或使用不太频繁的那些变量置于片外RAM，即采用pdata或xdata存储类型。常量只能采用code存储类型。

表 3-3 C51 存储类型及其大小和值域

存储类型	长度/bit	长度/byte	值域
data	8	1	0~255
idata	8	1	0~255
bdata	1		0 或 1
pdata	8	1	0~255
xdata	16	2	0~65 535
code	16	2	0~65 535

变量存储类型定义举例：

- (1) `char data a1;` `/*字符变量a1被定义为data型，分配在片内RAM低128字节中*/`
- (2) `float idata x,y;` `/*浮点变量x和y被定义为idata型，定位在片内RAM中，只能用间接寻址方式寻址*/`
- (3) `bit bdata p;` `/*位变量p被定义为bdata型，定位在片内RAM中的位寻址区*/`
- (4) `unsigned int pdata var1;` `/*无符号整型变量var1定义为pdata型，定位在片外RAM中，相当于@Ri间接寻址*/`
- (5) `unsigned char xdata a[2][4];` `/*无符号字符型二维数组变量 a[2][4]被定义为xdata存储类型，定位在片外RAM中，占据2×4=8字节，相当于 @DPTR间接寻址*/`

4. 数据存储模式

如果在变量定义时略去存储类型标识符，编译器会自动默认存储类型。

进一步由SMALL、COMPACT和LARGE存储模式指令限制。例如，若声明char var1，则在使用SMALL存储模式下，var1被定位在data存储区，在使用COMPACT模式下，var1被定位在idata存储区；在LARGE模式下，var1被定位在xdata存储区中。

下面对存储模式作进一步说明。

(1) SMALL模式。该模式下，所有变量都默认位于8051单片机内部的数据存储器，与使用data指定存储器类型的方式一样。在此模式下，变量访问的效率高，但是所有数据对象和堆栈必须使用内部RAM。

(2) COMPACT模式

本模式下所有变量都默认在外部数据存储器的1页（256字节）内，这与

使用pdata指定存储器类型是一样的。该类型适用于变量不超过256字节的情况，此限制是由寻址方式决定的，相当于使用数据指针@Ri寻址。与SMALL模式相比，该存储模式的效率比较低，对变量访问的速度也慢一些，但比LARGE模式快。

(3) LARGE模式

本模式下所有变量都默认位于外部数据存储器，相当于用@DPTR寻址。通过数据指针访问外部数据存储器的效率较低，特别是当变量为2字节或更多字节时，该模式要比SMALL和COMPACT产生更多的代码。

3.2.2 C51语言的特殊功能寄存器及位变量定义

介绍C51如何对特殊功能寄存器及位变量进行定义并访问。

1. 特殊功能寄存器的C51定义

C51语言允许通过使用**关键字sfr**、**sbit**或直接引用编译器提供的头文件来对特殊功能寄存器（SFR）进行访问，特殊功能寄存器分布在片内RAM高128字节中，只能采用直接寻址方式。

(1) 使用关键字定义sfr。为能直接访问特殊功能寄存器SFR，C51提供了一种定义方法，即引入关键字sfr，语法如下：

sfr 特殊功能寄存器名字=特殊功能寄存器地址；

例如：

```
sfr  IE=0xA8;           //中断允许寄存器IE地址A8H
sfr  TCON=0x88;          //定时器/计数器控制寄存器地址88H
sfr  SCON=0x98;          //串行口控制寄存器地址98H
```

在8051中，要访问16位SFR，要用关键字sfr16。16位SFR的低字节地址须作为“sfr16”的定义地址，例如：

sfr16 DPTR=0x82 //DPTR 的低8位地址为82H, 高8位地址为83H

(2) 通过头文件访问SFR。各种**衍生型**的8051单片机的特殊功能寄存器的数量与类型有时是不相同的, 对其访问可通过头文件访问来进行。

为用户处理方便, C51把8051 (或8052单片机) 常用的特殊功能寄存器和其中的可寻址位进行了定义, 放在一个**reg51.h** (或**reg52.h**) 的头文件中。

当用户要使用时, 只需在使用之前用一条**预处理命令**

include<reg51.h>把这个**头文件包含到程序中**, 就可使用特殊功能寄存器

名和其中的可寻址位名称了。用户可对头文件进行增减。

头文件引用举例如下：

```
#include<reg51.h>    // 包含8051单片机的头文件
void main(void)
{
    TL0=0xf0;         //给T0低字节TL0设置时间常数，已在reg51.h中定义
    TH0=0x3f;         //给T0高字节TH0设置时间常数，已在reg51.h中定义
    TR0=1;            //启动定时器0
    .....
}
```

(3) 特殊功能寄存器中的位定义。对SFR中的可寻址位的访问，要使用关键字来定义可寻址位，共**3种方法**。

① **sbit** 位名=特殊功能寄存器^位置;

例如:

```
sfr    PSW=0xd0;           //定义PSW 寄存器的字节地址0xd0
sbit   CY=PSW^7;           //定义CY位为PSW. 7, 地址为0xd0
sbit   OV=PSW^2;           //定义OV位为PSW. 2, 地址为0xd2
```

② **sbit** 位名=字节地址^位置;

例如:

```
sbit   CY=0xd0^7;          // CY位地址为0xd7
sbit   OV=0xd0^2;          // OV位地址为0xd2
```

③ **sbit** 位名=位地址;

将位的绝对地址赋给变量, 位地址必须在0x80~0xff。

例如:

```
sbit   CY=0xd7;            // CY位地址为0xd7
sbit   OV=0xd2;            // OV位地址为0xd2
```

【例】 AT89S51单片机片内P1口的各寻址位的定义如下：

```
sfr    P1=0x90;  
sbit   P1_7= P1^7;  
sbit   P1_6= P1^6;  
sbit   P1_5= P1^5;  
sbit   P1_4= P1^4;  
sbit   P1_3= P1^3;  
sbit   P1_2= P1^2;  
sbit   P1_1= P1^1;  
sbit   P1_0= P1^0;
```

2. 位变量的C51定义

(1) 由于8051可位操作，C51扩展的“bit”数据类型用来定义位变量，这是与标准C的不同之处。

C51采用关键字“bit”来定义位变量，一般格式为：

```
bit bit_name;
```

例如：

```
bit ov_flag;           //将ov_flag定义为位变量  
bit lock_pointer;      //将lock_pointer定义为位变量
```

(2) 函数可以包含类型为bit的参数，也可将其作为返回值。例如：

```
bit func(bit b0, bit b1); // 位变量b0与b1作为函数func的参数  
{  
    .....  
    return(b1);           // 位变量b1作为return函数的返回值  
}
```

(3) 位变量定义的限制。位变量不能用来定义指针和数组。例如：

```
bit *ptr;           // 错误，不能用位变量来定义指针
bit array[ ];       // 错误，不能用位变量来定义数组array[ ]
```

定义位变量时，允许定义存储类型，位变量都被放入一个位段，此段总是位于8051的片内RAM中，因此其存储类型限制为DATA或IDATA，如果将位变量定义成其他类型，将会导致编译时出错。

3.2.3 C51语言的绝对地址访问

如何对8051片内RAM、片外RAM及I/O空间进行访问，C51提供两种常用的访问绝对地址的方法。

1. 绝对宏

编译器提供了一组宏定义对code、data、pdata和xdata空间进行绝对寻址。

程序中用“`#include<absacc.h>`”来对absacc.h中声明的宏来访问绝对地址，包括CBYTE、CWORD、DBYTE、DWORD、XBYTE、XWORD、PBYTE、PWORD，具体使用参见absacc.h头文件。其中：

- CBYTE以字节形式对code区寻址；
- CWORD以字形式对code区寻址；
- DBYTE以字节形式对data区寻址；
- DWORD以字形式对data区寻址；
- XBYTE以字节形式对xdata区寻址；
- XWORD以字形式对xdata区寻址；
- PBYTE以字节形式对pdata区寻址；
- PWORD以字形式对pdata区寻址。

【例】片内RAM、片外RAM及I/O定义的程序如下：

```
#include<absacc.h>
```

```
#define PORTA XBYTE[0xFFC0]
```

 //将PORTA定义为外部I/O口，地址为0xFFC0，长度8位

```
#define NRAM DBYTE[0x50]
```

 //将NRAM定义为片内RAM，地址为0x50，长度8位

```
main( )
```

```
{
```

```
    PORTA=0x3d;   //将数据3DH写入地址为0xffc0的外部I/O端口PORTA中
```

```
    NRAM=0x01;    //将数据01H写入片内RAM的0x50单元
```

```
}
```

2. `_at_` 关键字

关键字 `_at_` 可对指定的存储器空间的绝对地址访问，格式如下：

`[存储器类型] 数据类型说明符 变量名 _at_ 地址常数`

其中，存储器类型为C51能识别的数据类型；数据类型为C51支持的数据类型；**地址常数**用于指定变量的绝对地址，必须位于有效的存储器空间之内；使用 `_at_` 定义的变量必须为**全局变量**。

【例】 使用关键字 `_at_` 实现绝对地址的访问，程序如下：

```
void main(void)
{
    data unsigned char y1 _at_ 0x50; //在data区定义字节变量y1，地址
                                     //为50H
    xdata unsigned int y2 _at_ 0x4000;
                                     //在xdata区定义字变量y2，地址为4000H
    y1=0xff;
    y2=0x1234;
    .....
    while(1);
}
```

【例】 将片外RAM 2000H开始的连续20字节清0，程序如下：

```
xdata unsigned char buffer[20] _at_ 0x2000;
void main(void)
{
    unsigned char i;
    for(i=0; i<20; i++)
    {
        buffer[i]=0
    }
}
```

如把片内RAM 40H单元开始的8个单元内容清0，程序如下：

```
data unsigned char buffer[8] _at_ 0x40;
void main(void)
{
```

```
unsigned char j ;  
    for (j=0;j<8;j++)  
    {  
        buffer[j]=0  
    }  
}
```

3.2.4 C51的基本运算

与标准C类似，主要包括算术运算、关系运算、逻辑运算、位运算和赋值运算及其表达式等。

1. 算术运算符

算术运算符及说明见表3-4。

表 3-4 算术运算符及其说明

符 号	说 明	举例（设 $x=10, y=3$ ）
+	加法运算	$z=x+y;$ $//z=13$
-	减法运算	$z=x-y;$ $//z=7$
*	乘法运算	$z=x*y;$ $//z=30$
/	除法运算	$z=x/y;$ $//z=3$
%	取余数运算	$z=x\%y;$ $//z=1$
++	自增 1	
--	自减 1	

C51中表示加1和减1时可以采用**自增运算符**和**自减运算符**，自增和自减运算符是使变量自动加1或减1，自增和自减运算符放在变量前和变量之后是不同的，见表3-5。

表 3-5 自增运算符与自减运算符

运算符	说 明	举例（设 x 初值为 4）
x++	先用 x 的值，再让 x 加 1	y=x++; // y 为 4，x 为 5
++x	先让 x 加 1，再用 x 的值	y=++x; // y 为 5，x 为 5
x--	先用 x 的值，再让 x 减 1	y=x--; // y 为 4，x 为 3
--x	先让 x 减 1，再用 x 的值	y=--x; // y 为 3，x 为 3

2. 逻辑运算符

逻辑运算的结果只有“真”和“假”两种，“1”表示真，“0”表示假。

表3-6列出了逻辑运算符及其说明。

表 3-6 逻辑运算符及其说明

运算符	说 明	举例（设 a=2,b=3）
&&	逻辑与	a&&b; //返回值为 1
	逻辑或	a b; //返回值为 1
!	逻辑非	! a; //返回值为 0

例如条件“10>20”为假，“2<6”为真，则逻辑与运算为：

$(10>20) \&\& (2<6) = 0 \&\& 1 = 0$ 。

3. 关系运算符

关系运算符是判断两个数之间的关系。说明如表3-7所示。

表 3-7 关系运算符及其说明

符 号	说 明	举例（设 $a=2, b=3$ ）
$>$	大于	$a > b$; //返回值为 0
$<$	小于	$a < b$; //返回值为 1
$>=$	大于等于	$a >= b$; //返回值为 0
$<=$	小于等于	$a <= b$; //返回值为 1
$==$	等于	$a == b$; //返回值为 0
$!=$	不等于	$a != b$; //返回值为 1

4. 位运算

位运算符及其说明见表3-8。

表 3-8 位运算及其说明

符 号	说 明	举例
&	按位逻辑与	$0x19 \& 0x4d = 0x09$
	按位逻辑或	$0x19 \mid 0x4d = 0x5d$
^	按位异或	$0x19 \wedge 0x4d = 0x54$
~	按位取反	$x = 0x0f$, 则 $\sim x = 0xf0$
<<	按位左移(高位丢弃, 低位补 0)	$y = 0x3a$, 若 $y \ll 2$, 则 $y = 0xe8$
>>	按位右移(高位补 0, 低位丢弃)	$w = 0x0f$, 若 $w \gg 2$, 则 $w = 0x03$

在实际应用中，常想改变I/O口中某一位的值，而不影响其他位，如果I/O口可位寻址的，这个问题很简单。但有时外扩的I/O口只能进行字节操作，要想实现单独位控，就要采用位操作。

【例】 编程将扩展的某I/O 口 PORTA（只能字节操作）的PORTA.5清0，PORTA.1置1，程序如下：

```
#define <absacc.h> //定义片外 I/O 口变量PORTA要用该头文件<absacc.h>
PORTA XBYTE[0xffc0] //定义一个片外 I/O 口变量PORTA
void main( )
{
    .....

    PORTA=( PORTA&0xdf) | 0x02;

    .....
}
```

程序中，第2行定义一个片外 I/O 口变量PORTA，地址为片外数据存储区的0xffc0。在main()函数中，“PORTA=(PORTA&0xdf) | 0x02”的作用是先运算符“&”将PORTA. 5置成0，然后再用“ | 0x02”运算将PORTA. 1置为1。

5. 指针和取地址运算符

指针是C51语言中一个十分重要的概念，指针变量用于存储某个变量的地址，C51用“*”和“&”运算符来提取变量内容和变量地址，见表3-9。

表 3-9 赋值、指针和取值运算及其说明

符 号	说 明
*	提取变量的内容
&	提取变量的地址

提取变量的内容和变量的地址的一般形式分别为：

目标变量=*指针变量 //将指针变量所指的存储单元内容赋值给目标变量

指针变量=&目标变量 //将目标变量的地址赋值给指针变量

例如：

a=&b; //取b变量的地址送至变量a

c=*b; //把以指针变量b为地址的单元内容送至变量c

指针变量中只能存放地址（即指针型数据），不能将非指针类型的数据赋值给指针变量。例如：

int i ; //定义整型变量i

int *b; //定义指向整数的指针变量b

b=& i; //将变量i的地址赋给指针变量b

b= i; //错，指针变量b只能存放变量指针（变量地址），不能存放变量i的值

3.2.5 C51的分支与循环程序结构

顺序、分支和循环结构按结构可分为3类。顺序结构是基本结构，程序自上而下，从main()的函数开始一直到程序结束，只有一条路可走，无其他路径可选，结构较简单便于理解，这里仅介绍分支结构和循环结构。

1. 分支控制语句

分支控制语句有：if语句和switch语句。

(1) if语句用来判定所给定的条件是否满足，根据判定结果决定执行两种操作之一。

if语句的基本结构如下：

if (表达式) {语句}

括号中的表达式成立时，程序执行大括号内的语句，否则程序跳过

大括号中的语句部分，而直接执行下面的其他语句。

C51提供**3种形式**的if语句：

形式1

if (表达式) {语句}

例如：

if (x>y) {max=x; min=y;}

即如果 $x>y$ ，则 x 赋给 $\max$ ， y 赋给 $\min$ 。如果 $x>y$ 不成立，则不执行大括号中的赋值运算。

形式2

if (表达式) {语句1;} else {语句2;}

例如:

```
if (x>y)
{max=x; }
else {max=y;}
```

本形式相当于双分支选择结构。

形式3

```
if (表达式1) {语句1;}
else if (表达式2) {语句2;}
else if (表达式3) {语句3;}
.....
else {语句n;}
```

例如：

```
if (x>100) {y=1;}  
else if (x>50) {y=2;}  
else if (x>30) {y=3;}  
else if (x>20) {y=4;}  
else {y=5;}
```

本形式相当于**串行多分支选择**结构。

在if语句中又含有一个或多个if语句，这称为**if语句的嵌套**。应当注意if与else的对应关系，**else总是与它前面最近的一个if语句相对应**。

(2) **switch**语句。if语句只有两个分支可选择，而**switch**语句是多分支选择语句。switch语句的一般形式如下：

```
switch (表达式1)
{
    case 常量表达式1:{语句1;}break;
    case 常量表达式2:{语句2;}break;
    .....
    case 常量表达式n:{语句n;}break;
    default:{语句n+1;}
}
```

上述switch语句说明如下。

(1) 每一**case**常量表达式**须互不相同**，否则将混乱。

- (2) 各个**case**和**default**出现次序，不影响程序执行的结果。
- (3) **switch**括号内表达式的值与某**case**后面的常量表达式的值相同时，就执行它后面的语句，遇到**break**语句则退出**switch**语句。若所有的**case**中的常量表达式的值都没有与**switch**语句表达式的值相匹配时，就执行**default**后面的语句。
- (4) 如果在**case**语句中遗忘了**break**语句，则程序执行了本行之后，不会按规定退出**switch**语句，而是将执行后续的**case**语句。在执行1个**case**分支后，使流程跳出**switch**结构，即中止**switch**语句的执行，可以用1条**break**语句完成。**switch**语句的最后一个分支可以不加**break**语句，结束后直接退出**switch**结构。

【例】在单片机程序设计中，常用**switch**语句作为**键盘中按键按下的判别**，并根据按下键的键号跳向各自的**分支处理程序**。

```
input: keynum=keyscan( )
switch(keynum)
{
    case 1: key1( ); break; //如果按下键为1键, 则执行函数key1( )
    case 2: key2( ); break; //如果按下键为2键, 则执行函数key2( )
    case 3: key3( ); break; //如果按下键为3键, 则执行函数key3( )
    case 4: key4( ); break; //如果按下键为4键, 则执行函数key4( )
    .....
    default:goto input
}
```

例子中的keyscan()是另行编写的一个键盘扫描函数, 如有键按下, 该函数就会得到按下键的键值, 将键值赋予变量keynum。如果键值为2, 则执行键值处理函数key2()后返回; 如键值为4, 则执行key4()函数后返回。



执行完1个键值处理函数后，则跳出**switch**语句，从而达到按下不同的按键来进行不同的键值处理的目的。

2. 循环控制语句

许多实用程序都包含循环结构，熟练掌握和运用循环结构的程序设计是**C51**语言程序设计的基本要求。

实现循环结构的语句有以下**3种**：**while**语句、**do-while**语句和**for**语句。

(1) **while**语句。语法形式为：

```
while(表达式)
```

```
{
```

```
    循环体语句;
```

```
}
```

表达式是while循环能否继续的**条件**，如果表达式为真，就重复执行循环体语句；反之，则终止循环体内的语句。

while循环结构特点：循环条件测试在**循环体开头**，要想执行重复操作，首先必须进行循环条件的测试，如**条件不成立**，则循环体内的重复操作一次也不能执行。

例如：

```
while((P1&0x80) == 0)
{ }
```

while中的条件语句对AT89S8052单片机的P1口的P1.7位进行测试，如果P1.7为低（0），则由于循环体无实际操作语句，故继续测试下去（等待），一旦P1.7的电平变高（1），则循环终止。

(2) do-while语句。语法形式为：

```
do
{
    循环体语句;
}
while(表达式);
```

do-while语句特点是先执行内嵌的循环体语句，再计算表达式，如表达式的值为非0，则继续执行循环体语句，**直到表达式的值为0时结束循环。**

由**do-while**构成的循环与**while**循环的**重要区别**是：**while**循环的控制出现在循环体之前，只有当**while**后面表达式的值非0时，才可能执行循环体；在**do-while**构成的循环中，总是先执行一次循环体，然后再求表达式的值，因此无论表达式的值是0还是非0，**循环体至少要被执行一次。**

在**do-while**循环体中，要有能使**while**后表达式的值**变为0**的操作，否则，循环会无限制地进行下去。根据经验，**do-while**循环用的并不多，大多数的循环用**while**来实现会直观。

【例】实型数组sample存有10个采样值，编写程序段，要求返回其平均值（平均值滤波）。程序如下：

```
float avg(float *sample)
{
    float sum=0;
    char n=0;
    do
    {
        sum+=sample[n];
        n++;
    } while(n<10);
    return(sum/10);
}
```

(3) 基于for语句的循环。3种循环常用的是for循环。不仅可用于循环

次数已知的情况，也可用于循环次数不确定而只给出循环条件情况，完全可替代while语句。

for循环的一般格式为：

```
for(表达式1;表达式2;表达式3)
{
    循环体语句;
}
```

for是关键字，括号中常含有3个表达式，各表达式间用“；”隔开。这3个表达式可以是任意形式的表达式，通常主要用于for循环控制。紧跟在for()之后的循环体，在语法上要求是1条语句；若在循环体内需要多条语句，应用大括号括起来组成复合语句。

for执行过程如下：

- ① 计算“**表达式1**”，表达式1通常称为“**初值设定表达式**”。
- ② 计算“**表达式2**”，表达式2通常称为“**终值条件表达式**”，若满足条件，转下一步，若不满足条件，则转步骤⑤。
- ③ 执行**1次for**循环体。
- ④ 计算“**表达式3**”，“表达式3”通常称为“**更新表达式**”转向步骤②。
- ⑤ 结束循环，执行**for**循环之后的语句。

下面对**for**语句的几个特例进行说明。

① **for**语句中的小括号内的**3个表达式全部为空**。

例如：

```
for(;;)  
{  
    循环体语句;  
}
```

在小括号内**只有两分号**，无表达式，这意味着**没有设初值，无判断条件，循环变量为增值**，它的作用相当于**while(1)**，这将导致一个**无限循环**。一般在编程时，需要无限循环时，可采用这种形式的**for**循环语句。

② **for**语句的3个表达式中，**表达式1缺省**。

例如：

```
for (; i <= 100; i++) sum = sum + i;
```

即不对*i*设初值。

③ for语句的3个表达式中，**表达式2缺省**。

例如：

```
for (i = 1; ; i++) sum = sum + i;
```

即不判断循环条件，认为表达式始终为真，循环将无休止地进行下去。

④ for语句的3个表达式中，**表达式1、表达式3省略**。

例如：

```
for (; i <= 100; )  
{  
    sum = sum + i;  
    i++;  
}
```

⑤ 没有循环体的for语句。

例如：

```
int a=1000;  
for(t=0;t<a;t++)  
{;}
```

本例**典型应用**就是**软件延时**。可用循环结构来实现，即循环执行指令，消磨一段已知的时间。指令的执行时间是靠一定数量的时钟周期来计时的，如果使用12MHz晶振，则12个时钟周期花费的时间为1 μ s。

【例】 编写一个延时1ms程序。

```
void delays( unsigned char int j)
{
    unsigned char i;
    while(j-->0)
    {
        for(i=0;i<125;i++)
            {;}
    }
}
```

如把上述程序段编译成汇编代码分析，用for的内部循环大约延时8 μ s，但不是特别精确。不同编译器会产生不同延时，因此i的上限值125应根据实际情况进行补偿调整。

【例】 求 $1+2+3+\cdots+100$ 的累加和。

用for语句编写的程序如下：

```
#include <reg51.h>
#include <stdio.h>
main( )
{
    int  nvar1, nsum;
    for(nvar1=0,nsum=1;nsum<=100;nsum++)
        nVar1+=ncount;           //累加求和
    while(1);
}
```

【例】 无限循环的结构实现。

编写无限循环程序段，可用以下3种结构。

① 使用**while(1)**的结构:

```
while(1)
{
    代码段;
}
```

② 使用**for (; ;)**的结构:

```
for(;;)
{
    代码段;
}
```

③ 使用**do-while(1)**的结构:

```
do
{
    代码段;
} while(1);
```

3. **break**语句、**continue**语句和**goto**语句

在循环体执行中，如满足循环判定条件的情况下跳出代码段，可使用 **break**语句或**continue**语句；如要从任意地方跳转到代码某地方，可使用**goto**语句。

(1) **break**语句

循环结构中，可使用**break**语句跳出本层循环体，马上结束本层循环。

【例】 执行如下程序段。


```
void main(void )
{
    int i, sum;
    sum=0;
    for(i=1;i<=10;i++)
    {
        sum=sum+i;
        if(sum>5) break;
        print( "sum=%d\n", sum); /*通过串口向计算机屏幕输出显示sum值*/
    }
}
```

本例如没有**break**语句，程序将进行**10**次循环；当**i=3** 时，**sum**的值为**6**，此时，**if**语句的表达式 “**sum>5**”的值为**1**，于是执行**break**语句，跳出**for**循环，从而提前终止循环。

因此在一个循环程序中，既可通过循环语句中的表达式来控制循环是否结束，还可直接通过break语句强行退出循环结构。

(2) continue语句

作用及用法与break语句类似，区别：当前循环遇到break，是直接结束循环，若遇上continue，则是停止当前这一层循环，然后直接尝试下一层循环。可见，continue并不结束整个循环，而仅仅是中断这一层循环，然后跳到循环条件处，继续下一层的循环。当然，如果跳到循环条件处，发现条件已不成立，那么循环也会结束。

【例】 输出整数1~100的累加值，但**要求跳过所有个位为3**的数。

为完成题目要求，在循环中加一个判断，如果该数各位是3，就跳过该数不加。**如何来判断1~100的数中那些数个位是3呢？**用求余数的运算符“%”，**将一个两位以内的正整数，除以10后，余数是3**，就说明这个数的个位为3。例如对于数73，除以10后，余数是3。

根据以上分析，参考程序如下：

```
void main(void )
{
    int i, sum=0;
    sum=0;
    for(i=1;i<=100;i++)
    {
        if(i%10==3)
            continue;
        sum=sum+i;
    }
}
```

```
print(“sum=%d\n”, sum);    /*在计算机屏幕显示sum值，了解本语句的  
                             功能即可*/  
}
```

(3) goto语句

无条件转移语句，当执行**goto**语句时，将程序指针跳转到**goto**给出的下一条代码。基本格式如下：

goto 标号

【例】计算整数1~100的累加值，存放到sum中。

```
void main(void )
{
    unsigned char i
    int sum;
    sumadd:
    sum=sum+i;
    i++;
    if(i<101)
    {
        goto sumadd;
    }
}
```

goto语句在C51中经常用于无条件跳转某条必须执行的语句以及在死循环程序中退出循环。为方便阅读，也为了避免跳转时引发错误，在程序设计中要慎重使用goto语句。

3.2.6 C51的数组

在C51程序设计中，数组使用的较为广泛。

1. 数组简介

数组是同类数据的一个有序结合，用**数组名**来标识。整型变量的有序结合称为**整型数组**，字符型变量的有序结合称为**字符型数组**。数组中的数据，称为**数组元素**。

数组中**各元素的顺序**用**下标**表示，下标为n的元素可以表示为数组名[n]。改变 [] 中的下标就可以访问数组中的所有的元素。

数组有一维、二维、三维和多维数组之分。C51语言中常用的一维、二维数组和字符数组。

(1) 一维数组

具有一个下标的数组元素组成的数组成为一维数组，一维数组形式如下：

类型说明符 数组名[元素个数];

其中，数组名是一个标识符，元素个数是一个常量表达式，不能是含有变量的表达式：

例如：

```
int array1[8]
```

定义名为array1的数组，包含8个整型元素，在定义数组时，可对数组进行整体初始化，若定义后对数组赋值，则只能对每个元素分别赋值。例如：

```
int a[3]={2, 4, 6};    /*给全部元素赋值, a[0]=2, a[1]=4, a[2]=6 * /  
int b[4]={5, 4, 3, 2}; /*给全部元素赋值, b[0]=5, b[1]=4, b[2]=3, b[3]=2* /
```

(2) 二维数组或多维数组

具有两个或两个以上下标的数组, 称为二维数组或多维数组。定义二维数组的一般形式如下:

类型说明符 数组名[行数] [列数];

其中, 数组名是一个标识符, 行数和列数都是常量表达式。例如:

```
float array2 [4] [3] /* array2数组, 4行3列共12个浮点型元素* /
```

二维数组可以在定义时进行整体初始化, 也可在定义后单个地进行赋值。
例如:


```
int a[3][4]={1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}; /*a数组全部初始化*/  
int b[3][4]={1, 3, 5, 7}, {2, 4, 6, 8}, { }; /* b数组部分初始化, 未初始化的元素  
为0* /
```

(3) 字符数组

若一个数组的元素是字符型的, 则该数组就是一个字符数组。例如:

```
char a[10]= { 'B' , 'E' , 'I' , ' ' , 'J' , 'I' , 'N' , 'G' , '\0' };  
//字符串数组
```

定义了一个字符型数组a[], 有10个数组元素, 并且将9个字符 (其中包括一个字符串结束标志 '\0') 分别赋给了a[0]~a[8], 剩余的a[9]被系统自动赋予空格字符。

C51还允许用字符串直接给字符数组置初值，例如：

```
char a[10]= { “BEI JING” };
```

用**双引号括起来**的一串字符，成为字符串常量，C51编译器会自动地在字符串末尾加上结束符 ‘\0’ 。

用**单引号括起来**的字符为字符的**ASCII码值**，而不是字符串。例如 ‘a’ 表示a的ASCII码值61H，而 “a” 表示一个字符串，由两个字符a和\0组成。

一个字符串可以用一维数组来装入，但数组的元素数目一定要比字符多一个，以便C51编译器自动在其后面加入结束符 ‘\0’ 。

2. 数组的应用

在C51的编程中，数组一个非常有用的功能是查表。例如数学运算，编程者更愿意采用查表计算而不是公式计算。例如，对于传感器的非线性转换需要进行补偿，使用查表法就要有效的多。

再如，LED显示程序中根据要显示的数值，找到对应的显示段码送到LED显示器显示。表可以事先计算好后装入程序存储器中。

【例】使用查表法，计算数0~9的平方。

```
#define uchar unsigned char
uchar code square[0, 1, 4, 9, 16, 25, 36, 49, 64, 81] ; /*0~9的平方表*/
uchar fuction(uchar number)
{
    return square[number];          /*返回要求得其平方的数 */
}
main( )
{
    result= fuction(7); //函数fuction( )返回值为7，其平方49存入result
                        //单元
}
```

程序开始处，“uchar code square[0, 1, 4, 9, 16, 25, 36, 49, 64, 81] ; ”定义了一个无符号字符型的数组square[]，并对其进行了初始化，将数0~9的平方值赋予了数组square[]，数据类型代码code指定编译器将平方表定位

在程序存储器中。

主函数调用函数fuction()，假设得到返回值number=7；从square数组中查表获得相应的求得其平方的数为49。执行result= fuction(7)后，result的结果为相应的平方数49

3. 数组与存储空间

当程序中设定了一个数组时，C51编译器就会在系统的存储空间中开辟一个区域，用于存放数组的内容。数组就包含在这个由连续存储单元组成的模块的存储体内。对**字符数组**而言，占据了**内存中一连串的字节**位置。对**整型（int）数组**而言，将在存储区中占据一连串连续的**字节对**的位置。对**长整型（long）数组**或**浮点型（float）数组**，一个数组成员将占有**4字节**的存储空间。

当**一维数组**被创建时，C51编译器就会根据数组的类型在内存中开辟一块大小等于数组长度乘以数据类型长度（即类型占有的字节数）的区域。

对于**二维数组** $a[m][n]$ 而言，其**存储顺序是按行存储**，先存第0行元素的第0列、第1列、第2列，直至第 $n-1$ 列，然后返回到存第1行元素的第0列、第1列、第2列，直至第 $n-1$ 列，……，如此顺序存储，直到第 $m-1$ 行的第 $n-1$ 列。

当数组特别是多维数组中大多数元素没有被有效利用地利用时，就会浪费大量的存储空间。对于51单片机，其存储资源极为有限，因此在进行C51语言编程开发时，**要仔细地根据需要来选择数组的大小**。

3.2.7 C51的指针

C51支持两种不同类型的指针：**通用指针**和**存储器指针**。

1. 通用指针

C51提供一个3字节的通用指针，通用指针声明和使用与标准C语言完全一样。通用指针的形式如下：

数据类型 *指针变量；

例如：

uchar *pz

例中pz就是通用指针，用3字节来存储指针，第一字节表示存储器类型，第二、三字节分别是指针所指向数据地址的高字节和低字节，这种定义**很方便但速度较慢**，在所指向的目标存储器空间不明确时普遍使用。

2. 存储器的指针

存储器指针在定义时指明了存储器类型，并且指针总是指向特定的存储器空间（片内数据**RAM**、片外数据**RAM**或程序**ROM**）。例如：

```
char xdata *str;           // str指向xdata区中的char型数据  
int xdata *pd;            // pd指向外部RAM区中的int型整数
```

由于定义中已经指明了存储器类型，因此，相对于通用指针而言，指针第一个字节省略，对于**data**、**bdata**、**idata**与**pdata**存储器类型，指针仅需要**1B**，因为它们的寻址空间都在**256B**以内，而**code**和**xdata**存储器类型则需要**2B**指针，因为它们的寻址空间最大为**64KB**。



使用**存储器指针好处**是节省了存储空间，编译器不用为存储器选择和决定正确的存储器操作指令来产生代码，使代码更加简短，但必须保证指针不指向所声明的存储区以外的地方，否则会产生错误。

通用指针产生的代码执行速度比指定存储区的指针要慢，因为存储区在运行前是未知的，编译器不能优化存储区访问，必须产生可以访问任何存储区的通用代码。

由上所述，使用存储器指针比使用通用指针效率高，存储器指针所占空间小，速度更快，**在存储器空间明确时，建议使用存储器指针**，如果存储器空间不明确则使用通用指针。

3.3 C51语言的函数

函数是一个完成一定相关功能的执行代码段。在高级语言中，函数与另外两个名词“子程序”和“过程”用来描述同样的事情。在C51语言中使用的是函数这个术语。

C51语言中函数的数目是不限制的，但是一个C51程序必须至少有一个函数，以main为名，称为主函数，主函数是唯一的，整个程序从这个主函数开始执行。

C51语言还可建立和使用库函数，可由用户根据需求调用。

3.3.1 函数的分类

从结构上分，**C51**语言函数可分为**主函数main()**和普通函数两种。而**普通函数**又划分为**两种**：**标准库函数**和**用户自定义函数**。

1. 标准库函数

标准库函数是由**C51**编译器提供的。编程者在进行程序设计时，应该善于充分利用这些功能强大、资源丰富的标准库函数资源，以提高编程效率。

用户可直接调用**C51**库函数而不需为这个函数写任何代码，只需要包含具有该函数说明的头文件即可。**例如**调用输出函数**printf**时，要求程序在调用输出库函数前包含以下的**include** 命令：

```
#include <stdio.h>
```

2. 用户自定义函数

用户自定义函数是用户根据需要所编写的函数。从函数定义的形式分为：
无参函数、有参函数和空函数。

(1) 无参函数

此种函数在被调用时，既无参数输入，也不返回结果给调用函数，只是为完成某种操作而编写的函数。

无参函数的定义形式为：

返回值类型标识符 函数名 ()

{

函数体;

}

无参函数一般不带返回值，因此函数的返回值类型的标识符可省略。

例如函数：`main()`，为无参函数，返回值类型的标识符可省略，默认值是int类型。

(2) 有参函数

调用此种函数时，必须提供实际的输入函数。有参函数的定义形式为：

返回值类型标识符 函数名（形式参数列表）

形式参数说明

```
{  
    函数体;  
}
```

【例】定义一个函数max()，用于求两个数中的大数。

```
int a,b
int max(a, b)
{
    if (a>b) return (a) ;
    else return (b) ;
}
```

程序段中，a、b为形式参数。return () 为返回语句。

(3) 空函数

此种函数体内是空白的。调用空函数时，什么工作也不做，不起任何作用。定义空函数的目的，并不是为了执行某种操作，而是为了以后程序功能的扩充。先将一些基本模块的功能函数定义成空函数，占好位

置，并写好注释，以后再用一个编好的函数代替它。这样整个程序的结构清晰，可读性好，以后扩充新功能方便。

空函数的定义形式为：

返回值类型标识符 函数名 ()

{ }

例如：

float min()

{ }

//空函数，占好位置

3.3.2 中断服务函数

由于标准C没有处理单片机中断的定义，为能进行8051的中断处理，C51编译器对函数定义进行了扩展，增加了一个扩展关键字**interrupt**。

使用**interrupt**可将一个函数定义成**中断服务函数**。由于C51编译器在编译时对声明为中断服务程序的函数自动添加了相应的现场保护、阻断其他中断、返回时自动恢复现场等处理的程序段，因而在编写中断服务函数时可不考虑这些问题，减小了用户编写中断服务程序的繁琐程度。

中断服务函数的一般形式为：

函数类型 函数名（形式参数表） **interrupt n using n**

关键字**interrupt**是中断号，对于51单片机，n取值为0~4。

关键字**using**后的 n是所选择的寄存器组，**using**是一个选项，可省略。

如果不用关键字**using**指明寄存器组，中断函数中的所有工作寄存器的内容将被保存到堆栈中。

有关中断服务函数的具体使用注意事项，将在中断系统一章中详细介绍。

3.3.3 变量及存储方式

1. 变量

(1) 局部变量

是某一个函数中存在的变量，它只在该函数内部有效。

(2) 全局变量

在**整个源文件**中都存在的变量。**有效区间**是从**定义点开始到源文件结束**，其中的所有函数都可直接访问该变量。如果定义前的函数需要访问该变量，则需要使用**extern**关键词对该变量进行说明，如果全局变量声明文件之外的源文件需要访问该变量，也需要使用**extern**关键词进行说明。

由于全局变量一直存在，占用了大量的内存单元，且加大了程序的耦合性，不利于程序的移植或复用。

全局变量可以使用**static**关键词进行定义，该变量只能在变量定义的源文件内使用，不能被其他源文件引用，这种全局变量称为**静态全局变量**。如果一个其他文件的非静态全局变量需要被某文件引用，则需要在该文件调用前使用**extern**关键词对该变量声明。

2. 变量的存储方式

单片机的存储区间，可以分为**程序存储区**、**静态存储区**和**动态存储区**3个部分。数据存放在静态存储区或动态存储区。其中**全局变量**存放在**静态存储区**，在程序开始运行时，给全局变量分配存储空间；**局部变量**存放在**动态存储区**，在进入拥有该变量的函数时，给这些变量分配存储间。

3.3.4 宏定义与文件包含

在C51程序设计中要经常用到宏定义、文件包含与条件编译。

1. 宏定义

宏定义语句属于C51语言的预处理指令，使用宏可以使变量书写简化，增加程序的可读性、可维护性和可移植性。宏定义分为简单的宏定义和带参数的宏定义。

在C51的程序编写中，经常使用简单的宏定义。简单的宏定义格式如下：

#define 宏替换名 宏替换体

#define是宏定义指令的关键词，**宏替换名**一般用大写字母来表示，而**宏替换体**可以是数值常数、算术表达式、字符和字符串等。

宏定义可以出现在程序的任何地方，**例如宏定义：**

```
#define uchar unsigned char
```

在编译时可由C51编译器把“unsigned char”用“uchar”来替代。

例如，在某程序的开头处，进行了3个宏定义：

```
#define uchar unsigned char    /*宏定义无符号字符型变量方便书写*/  
#define uint unsigned int      /*宏定义无符号整型变量方便书写*/  
#define gain 4                 /*宏定义增益*/  
.....
```

由上见，宏定义不仅可以方便无符号字符型和无符号整型变量的书写，而且当增益需要变化时，只需要修改增益`gain`的宏替换体4即可，而不必在程序的每处修改，大大增加了程序的可读性和可维护性。

2. 文件包含

是指一个程序文件将另一个指定的文件的内容包含进去。文件包含的一般格式为：

```
#include <文件名>
```

或

```
#include “文件名”
```

上述两种格式的差别：**采用<文件名>格式**时，在头文件目录中查找指定文件。**采用“文件名”格式**时，应在当前的目录中查找指定文件。

例如：

```
#include<reg51.h>    /*将特殊功能寄存器包含文件包含到程序中来*/  
#include<stdio.h>    /*将标准的输入、输出头文件stdio.h（在函数库中）  
                      包含到程序中来*/  
#include "stdio.h"    /*同上，在当前的目录中查找指定文件stdio.h*/
```

当程序中需**调用编译器提供的各种库函数**时，须在文件的开头使用**#include**命令将相应函数的说明文件包含进来。

3.3.5 库函数

C51语言的强大功能及其高效率在于**提供了丰富的可直接调用的库函数**。库函数可以使程序代码简单、结构清晰、易于调试和维护。

下面介绍几类重要的库函数。

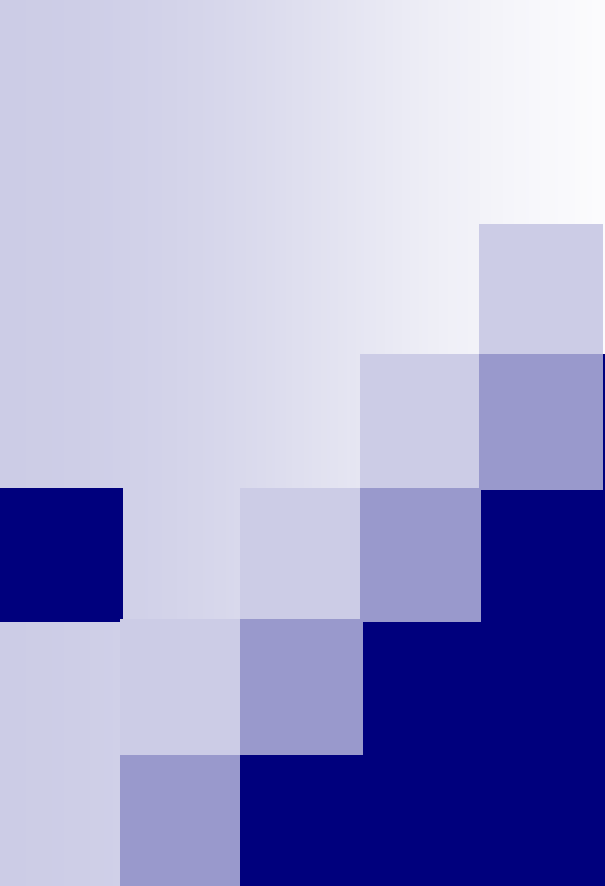
(1) 特殊功能寄存器包含文件**reg51.h**或**reg52.h**。**reg51.h**中包含所有的**8051**的**sfr**及其位定义。**reg52.h**中包含所有**8052**的**sfr**及其位定义，一般系统都包含**reg51.h**或**reg52.h**。

(2) 绝对地址包含文件**absacc.h**：该文件定义了几个宏，以**确定各类存储空间的绝对地址**。

(3) 输入/输出流函数位于**stdio.h**文件中。流函数默认**8051**的串口来作为数据的输入/输出。如果要修改为用户定义的I/O口读写数据，例如，改为**LCD**显示，可以修改**lib**目录中的**getkey.c**及**putchar.c**源文件，然后在库中替换它们既可。

(4) 动态内存分配函数，位于**stdlib.h**中。

(5) 能够对方便地对缓冲区进行处理的字符串处理函数位于**string.h**中。其中包括复制、移动、比较等函数。



第4章 软件开发工具Keil与虚拟仿真平台Proteus的使用

【内容概要】

本章介绍**C51**软件开发平台**Keil C51**与虚拟仿真平台**Proteus**的基本特性与使用。

通过本章学习，读者应初步了解如何运用**Keil**工具进行软件编程与调试，掌握使用**Proteus**平台来进行硬件的设计，以及使用**Keil**工具和**Proteus**平台进行单片机应用系统的设计与虚拟仿真的基本方法与步骤。

4.1 Keil C51的使用

4.1.1 Keil C51简介

Keil C51是用于**8051**单片机的**C51**语言编程的集成开发环境，由德国**Keil software**公司(已被**ARM**公司收购)开发，是**C51**语言开发编程所必须掌握的软件开发工具。

Keil C51集编辑、编译、仿真等功能于一体，具有强大的软件调试功能，生成的程序代码运行速度快，所需的存储器空间小，完全可与汇编语言相媲美，是目前**8051**单片机的最优秀软件开发工具之一。**Keil C51**集成了文件编辑处理、编译、链接、项目（**Project**）管理窗口、工具引用、仿真软件模拟器以及**Monitor51**硬件目标调试器等多种功能。

4.1.2 基本操作

1. 软件安装与启动

Keil C51软件安装完毕后，在桌面上出现Keil C51软件的快捷图标。单击该快捷图标，则启动该软件，出现如图4-1所示的Keil C51界面，图中标出了Keil C51界面各窗口的名称。

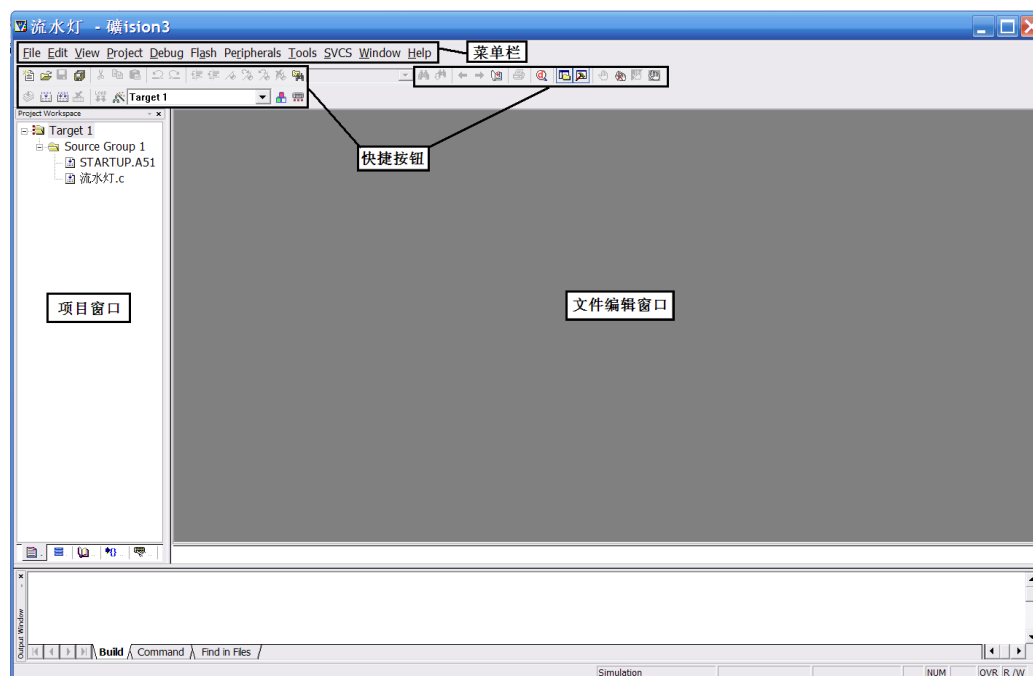


图4-1 Keil C51软件开发环境界面

2. 创建项目

编写一个新的应用程序前，首先要建立**项目（Project）**。**Keil C51**用项目管理的方法把一个程序设计中所需要用到的、互相关联的程序链接在同一项目中。这样，打开一个项目时，所需要的关联程序也都跟着进入了调试窗口，方便用户对项目中各个程序的编写、调试和存储。具体操作如下。

（1）在**图4-1**所示的编辑界面下，单击菜单栏中的**【Project】**，出现下拉菜单，再点击选择中的**“New Project”**，如**图4-2**所示。

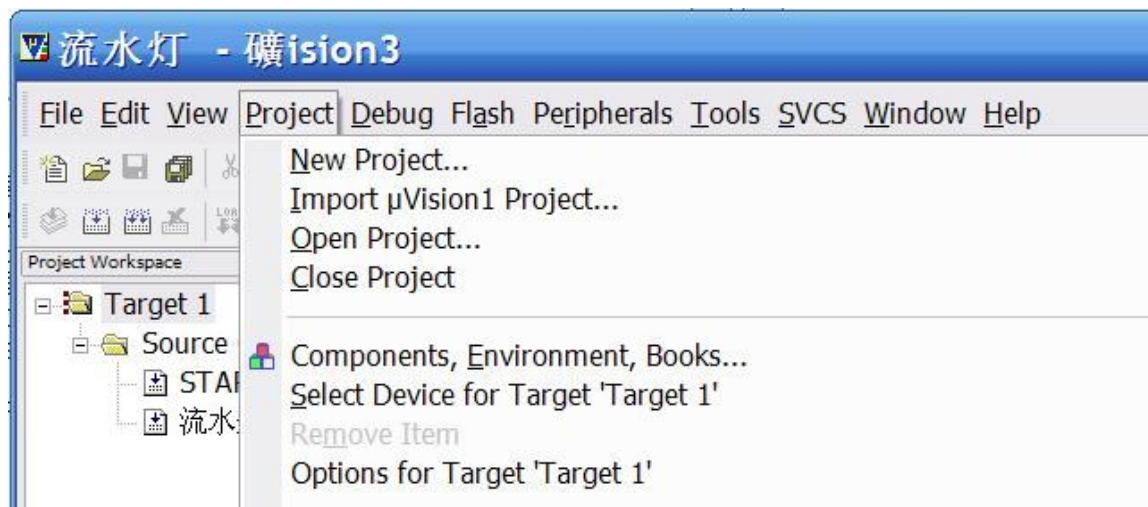


图4-2 新建项目菜单

(2) 单击“**New Project...**”选项后，就会弹出“**Create New Project**”窗口，见图4-3。在“文件名(N)”中输入一个项目的名称，保存后的文件扩展名为“**.uv2**”，即项目文件的扩展名，以后可直接单击此文件就可打开先前建立的项目。

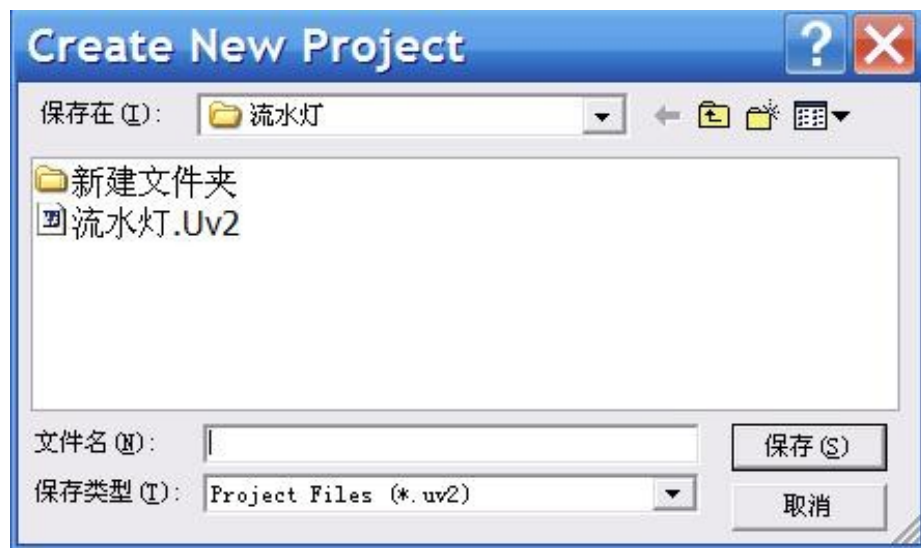


图4-3 “Create New Project” 窗口

在“文件名 (N)”窗口中输入新建项目文件的名称后，在“保存在 (I)”下拉框中选择项目的保存目录，单击“保存 (S)”即可。

(3) 选择单片机，单击“保存 (S)”后，会弹出如图4-4所示“**Select Device for Target**”（选择单片机）窗口，按照提示选择相应的单片机。选择“**Atmel**”目录下的“**AT89C51**”(对于**AT89S51**，也是选择**AT89C51**)。

(4) 点击“确定”按钮后，会出现图4-5所示的对话框。如果需要复制启动代码到新建的项目，如选择单击“是”，会出现图4-6的窗口，如选择单击“否”，图4-6中的启动代码项“**SARTUP. A51**”，不会出现，这时新的项目已经创建完毕。



图4-4 “Select Device for Target” 窗口

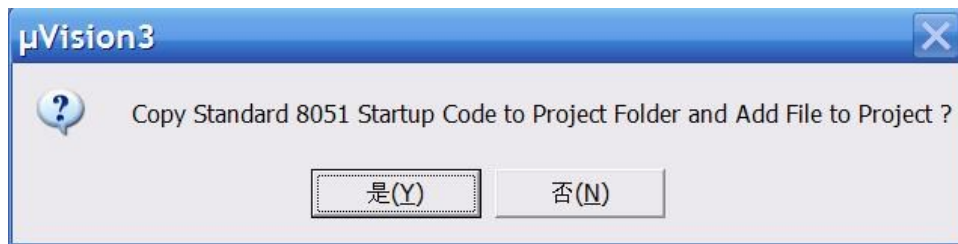


图4-5 是否复制启动代码到项目对话框

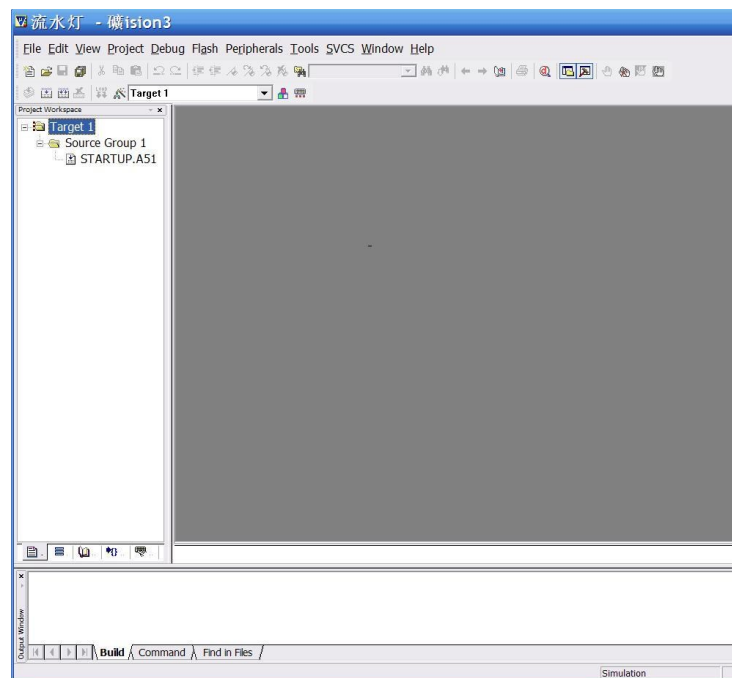


图4-6 完成项目的创建

4.1.3 添加用户源程序文件

新的项目文件创建完成后，就需要将用户源程序文件添加到这个项目中，添加用户程序文件通常有两种方式：一种是新建文件，另一种是添加已创建的文件。

1. 新建文件

（1）单击图4-1中快捷按钮，这时会出现如图4-7所示窗口。在这个窗口会出现一个空白的文件编辑画面，用户可在这里输入编写的程序源代码。

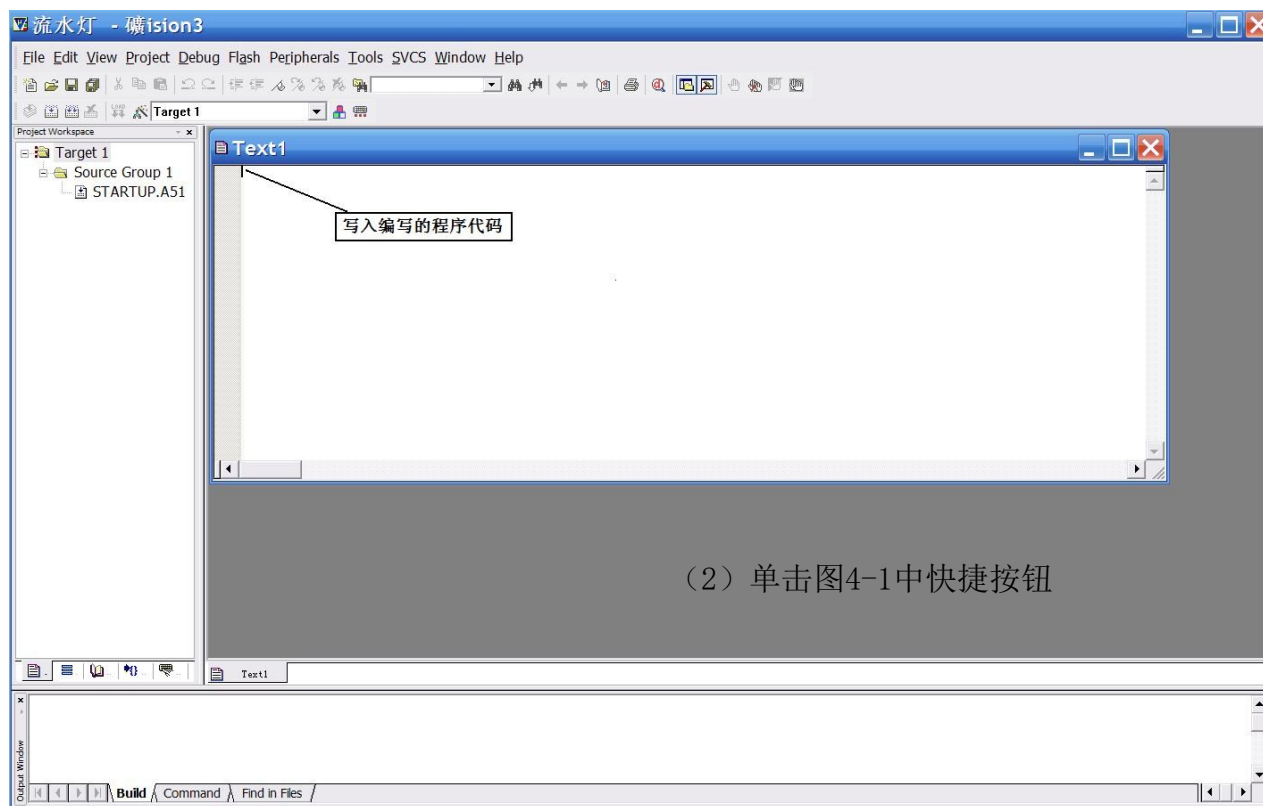


图4-7 建立新文件

(2) 单击图4-1中快捷按钮 , 保存用户程序文件, 这时会弹出如图4-8所示窗口。 , 保存用户程序文件, 这时会弹出如图4-8所示窗口。



图4-8 “Save As”对话框

(3) 在图4-8 “Save As”的对话框中，在“保存（S）”下拉框中选择新文件的保存目录，这样就将这个新文件与刚才建立的项目保存在同一个文件夹下，然后在“文件名（N）”窗口中输入新建文件的名称“流水灯”，如果使用C51语言编程，则文件名的扩展名应为“.c”。

如果用汇编语言编程，文件扩展名应为“.asm”。完成上述步骤后单击“保存”，此时新文件已创建完成。

这个新文件还需添加到刚才创建的项目中，操作步骤与下面的“添加已创建文件”步骤相同。

2. 添加已创建文件

(1) 在项目窗口（见图4-1）中，右键单击“**Source Group1**”，选择“**Add File to‘Source Group1’**”选项，如图4-9所示。

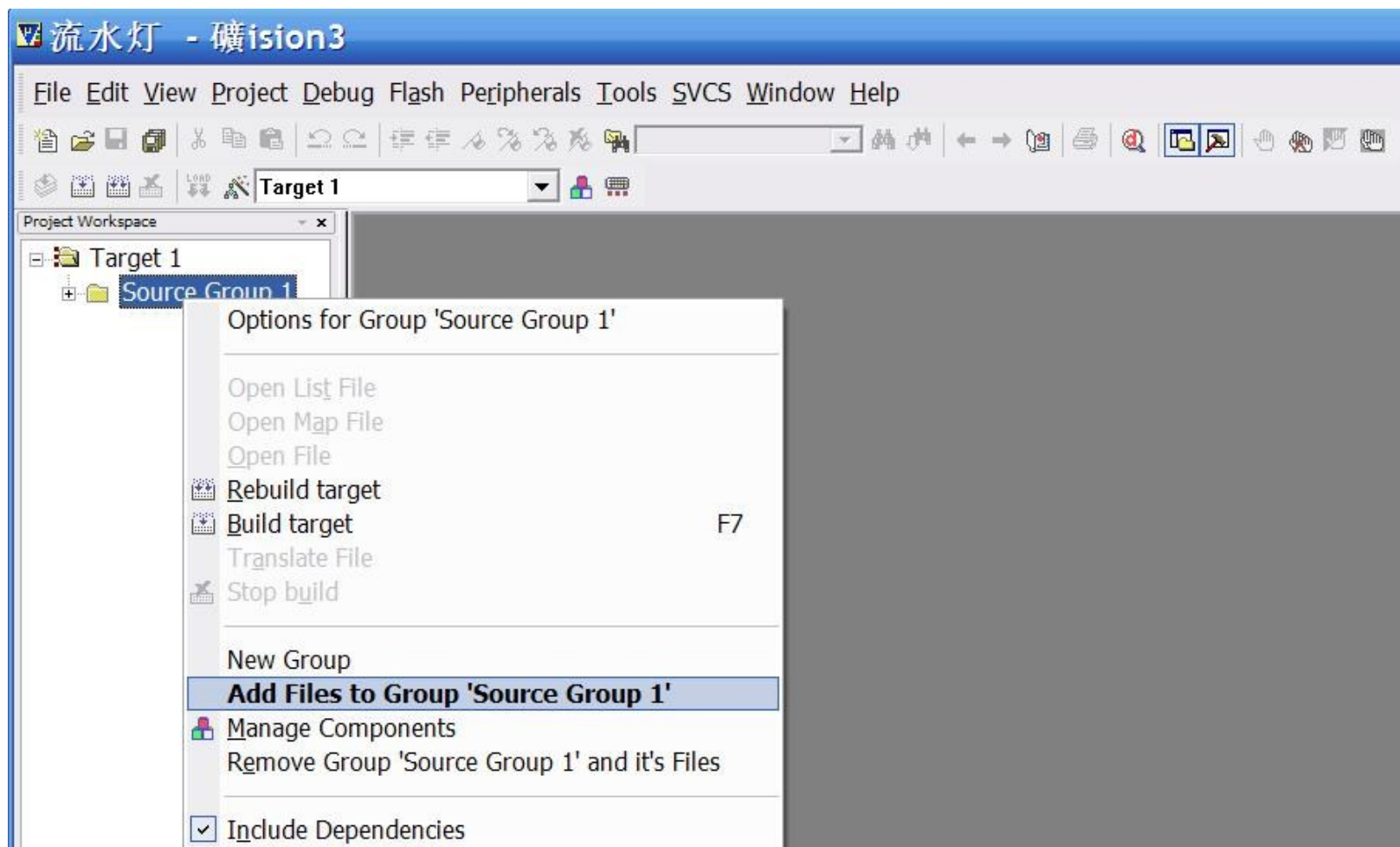


图4-9 添加文件

(2) 完成上述操作后会出现如图4-10 “Add File to ‘Source Group1’ ”所示的对话框。在该窗口中选择要添加的文件，这里只有刚刚建立的文件“流水灯.c”，点击这个文件后，单击“Add”按钮，再单击



图4-10 “Add File to ‘Source Group1’ ”对话框

“Close”按钮，文件添加已经完成了，这时的项目窗口如图4-11所示，用户程序文件“流水灯.c”已经出现在“Source Group1”目录下了。

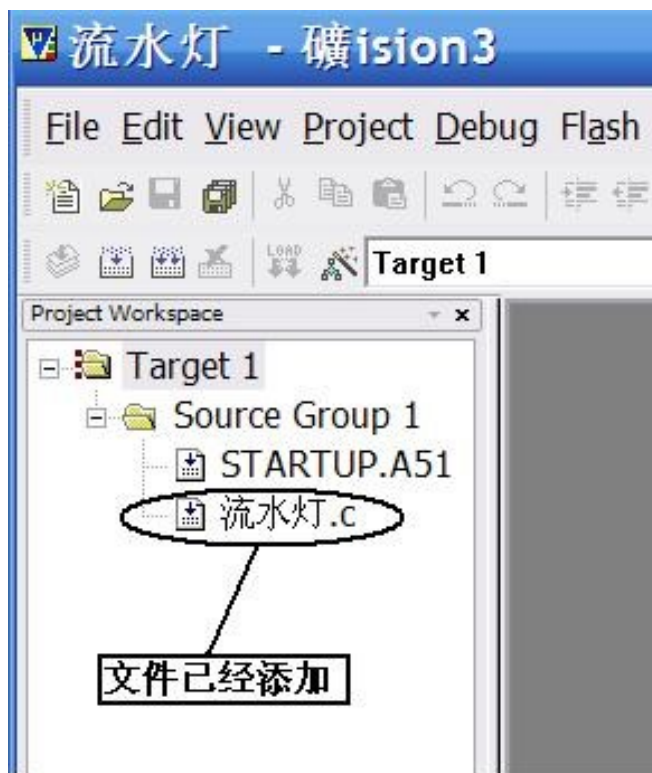


图4-11 文件已添加到项目中

4.1.4 程序的编译与调试

上面在文件编辑窗口建立了文件“流水灯.c”（或“流水灯.asm”），并且将文件添加到项目中，还需将文件进行编译和调试，最终生成可执行的.hex文件，具体步骤如下。

1. 程序编译

单击快捷按钮中的  对当前文件进行编译，这里是以“流水灯.c”文件为例，在图4-12中的输出窗口会出现编译结果的提示信息。

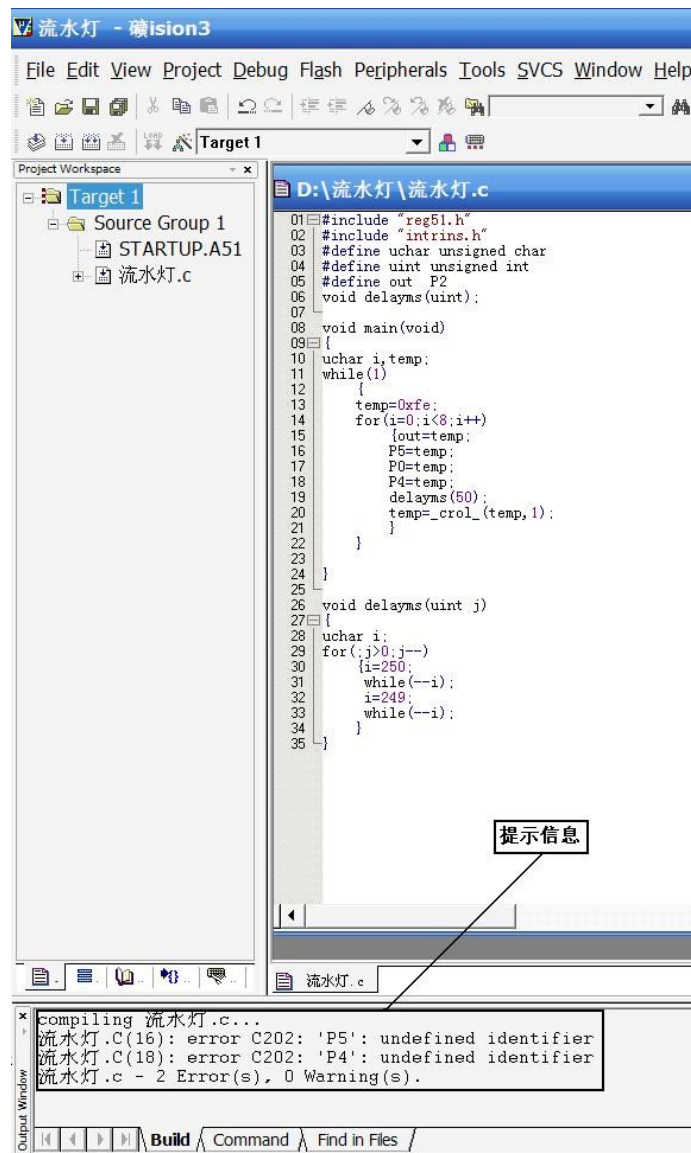


图4-12 文件编译信息

从输出窗口中的提示信息可以看到，程序中有2个错误，认真检查程序找到错误并改正，改正后再次单击  进行编译，直至提示信息显示没有错误为止，如图4-13所示。

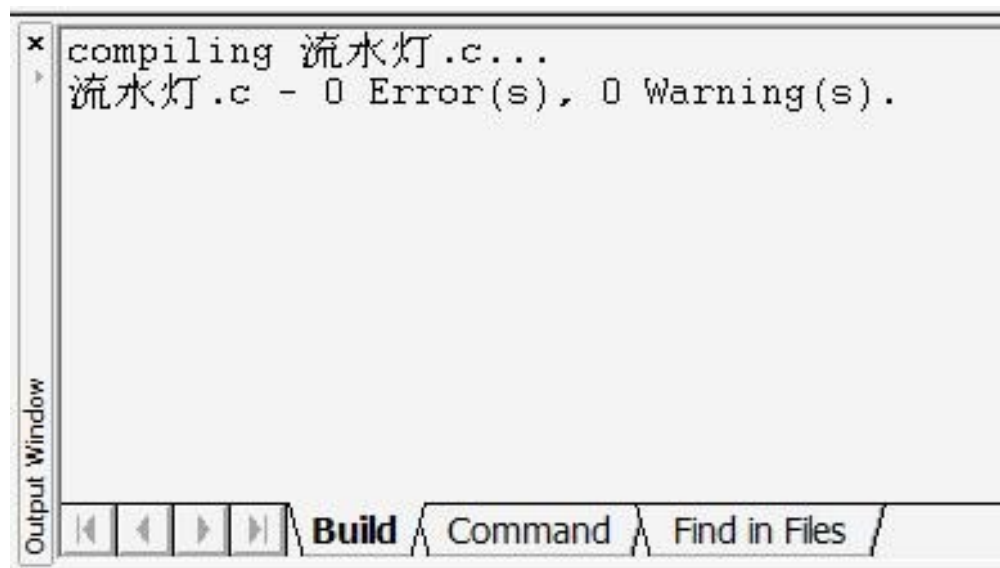


图4-13 提示信息显示没有错误

2. 程序调试

程序编译没有错误后，就可以进行调试与仿真。单击开始/停止调试的快捷按钮 （或在主界面点击【Debug】菜单中的“Start/Stop Debug Session”选项），进入程序调试状态，如图4-14所示。

图4-14左面的工程窗口给出了常用的寄存器R0~R7以及A、B、SP、DPTR、PC、PSW等特殊功能寄存器的值，这些值会随着程序的执行发生相应的变化。

在图4-14存储器窗口的地址栏处输入0000H后回车，则可查看单片机片内程序存储器的内容，单元地址前有“C:”，表示程序存储器。如要查看单片机片内数据存储器的内容，在存储器窗口的地址栏处输入D: 00H后回车，则可以看到数据存储器的内容。单元地址前有“D:”，表示数据存储器。

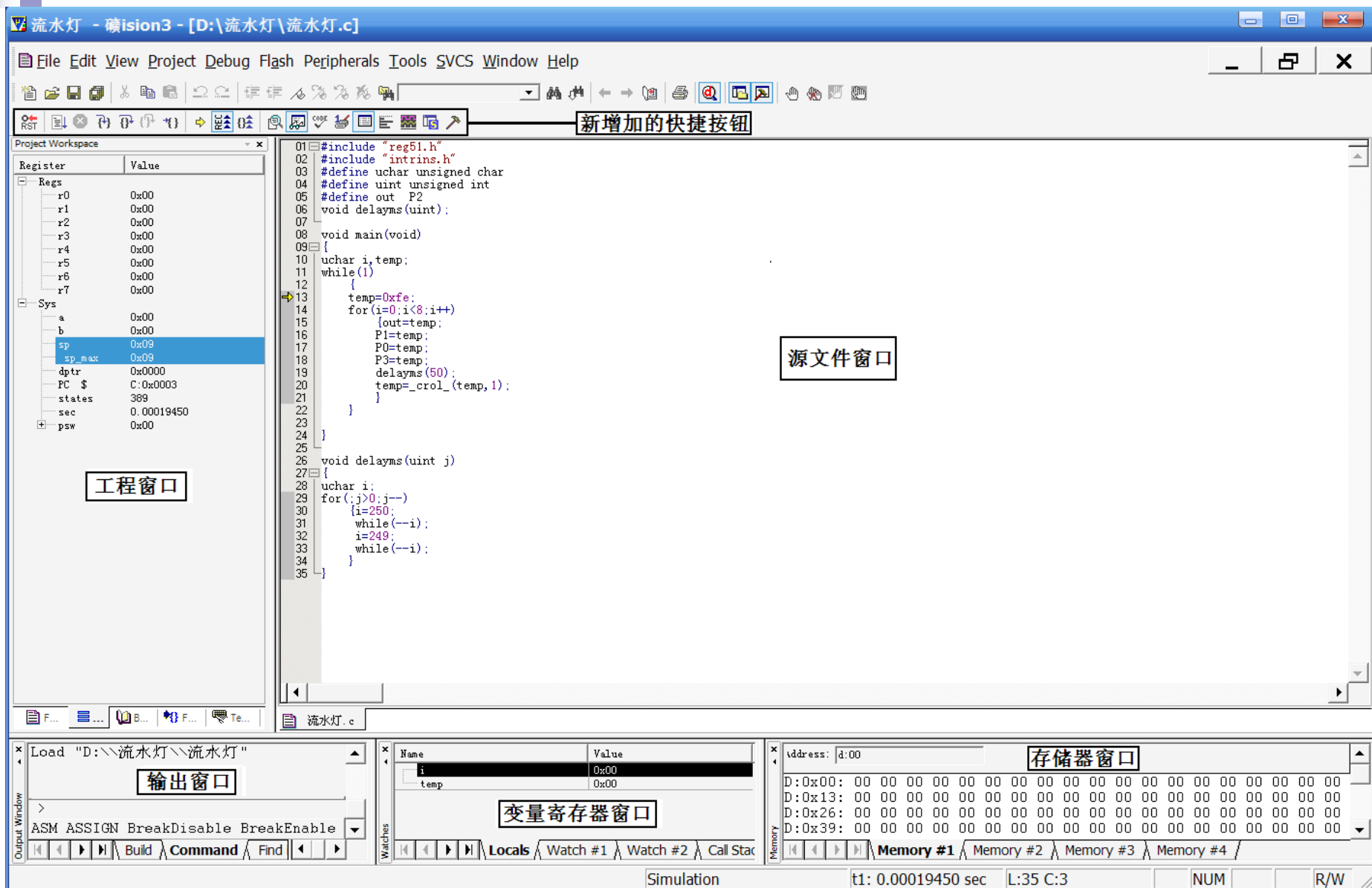


图4-14 程序调试界面

在图4-14中出现了一行新增加的用于调试的快捷命令图标，见图4-15。还有几个原来就有的用于调试的快捷图标，见图4-16。



图4-15 调试状态下的新增加的快捷命令按钮图标



图4-16 用于调试的其他几个快捷命令按钮图标



在程序调试状态下，可运用快捷按钮进行单步、跟踪、断点、全速运行等方式进行调试，也可观察单片机资源的状态，例如程序存储器、数据存储器、特殊功能寄存器、变量寄存器及I/O端口的状态。

这些图标大多数是与菜单栏命令 **【Debug】** 下拉菜单中的各项子命令是相对应的，只是快捷按钮图标要比下拉菜单使用起来更加方便快捷。

图4-15与图4-16中常用的快捷按钮图标的功能介绍如下。

(1) 各调试窗口显示的开关按钮

下面的图标控制图4-14中各个窗口的开与关。



: 工程窗口的开与关。



: 特殊功能寄存器显示窗口的开与关。



: 输出窗口的开与关。



: 存储器窗口的开与关。



: 变量寄存器窗口的开与关。

(2) 各调试功能的快捷按钮

: 调试状态的进入/退出。

: 复位 CPU。在程序不改变的情况下，若想使程序重新开始运行，单击本图标命令即可。执行此命令后程序指针返回到 0000H 地址单元。另外，一些内部特殊功能寄存器在复位期间也将重新赋值。例如，A 将变为 00H，SP 变为 07H，DPTR 变为 0000H，P3~P0 口变为 FFH。

: 全速运行。单击本图标命令，即可实现全速运行程序。当然若程序中已经设置断点，程序将执行到断点处，并等待调试指令。在全速运行期间，不允许对任何资源进行查看，也不接受其他命令。

：单步跟踪。可以单步跟踪程序。每执行一次此命令，程序将运行一条指令。当前的指令用黄色箭头标出，每执行一步箭头都会移动，已执行过的语句呈绿色。

：单步运行。本命令实现单步运行程序，此时单步运行命令将把函数和函数调用当作一个实体来看待，因此单步运行是以语句(该语句不管是单一命令行还是函数调用)为基本执行单元。

：运行到光标行。

：停止程序运行。

程序调试中，上述的几种运行方式都要用到，灵活地运用这些手段，可大大提高查找差错的效率。

(3) 断点操作的快捷按钮

在程序调试中常常要设置断点，一旦执行到该程序行即停止，可在断点处观察有关变量值，以确定问题所在。图 4-17 中有关断点操作的命令快捷按钮的功能如下：

：插入/清除断点。

：清除所有的断点设置。

：使能/禁止断点，是开启或暂停光标所在行的断点功能。

：禁止所有断点，是暂停所有断点。

此外，插入或清除断点最简单的方法，即将鼠标移至需要插入或清除断点的行首双击鼠标即可。

上述的 4 个快捷图标命令，也可从菜单命令【Debug】的下拉子菜单找到。

4.1.5 项目的设置

项目创建后，还需对项目进一步设置。右键单击工程窗口的“**Target 1**”，选择“**Options for Target‘Target1’**”，见图4-17，即出现工程设置对话框，见图4-18。该对话框下有多个页面，通常需要设置的有两个，一个是**Target**页面，另一个是**Output**页面，其余设置取默认值就可。

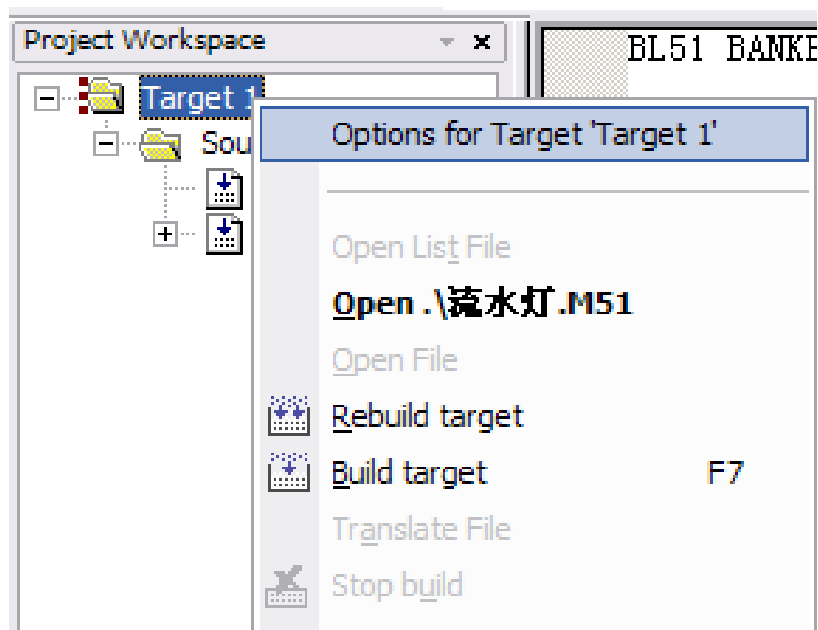


图4-17 项目调试的选择

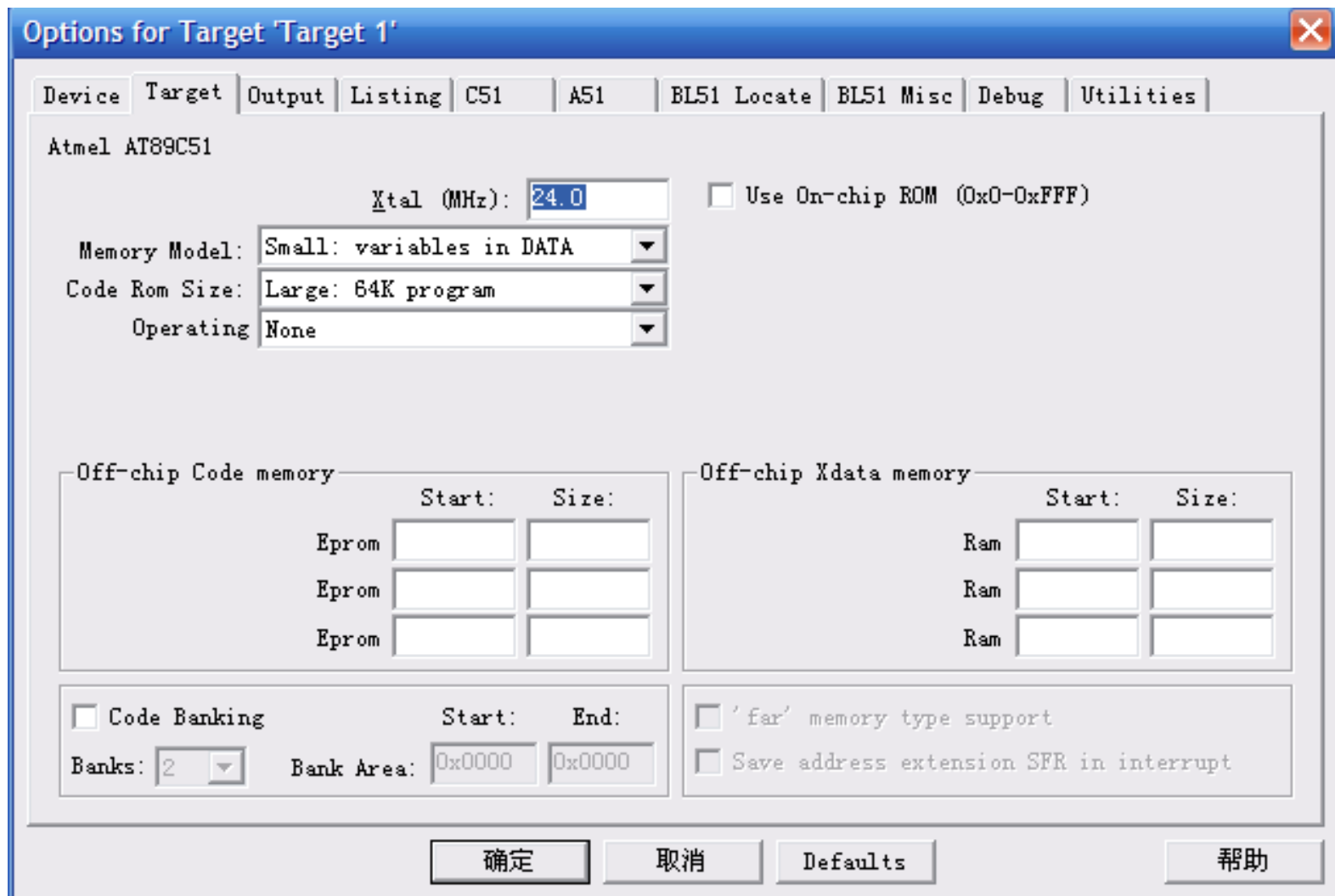


图4-18 “Options for Target ‘Target1’ ” 窗口

1. Target页面

(1) **Xtal(MHz)**—设置晶振频率值，默认值是目标**CPU**的最高可用频率值，可根据需要重新设置。该设置与最终产生的目标代码无关，仅用于软件模拟调试时显示程序执行时间。正确设置该数值可使显示时间与实际所用时间一致，一般将其设置成与硬件目标样机所用的频率相同，如果没必要了解程序执行的时间，也可以不设置。

(2) **Memory Model**—设置**RAM**的存储器模式，有3个选项。

① **Small**—所有变量都在单片机的内部**RAM**中。

② **Compact**—可以使用1页外部**RAM**。

③ **Large**—可以使用全部外部的扩展**RAM**。

(3) **Code Rom Size**—设置**ROM**空间的使用，即程序的代码存储器模式，有3个选项。

① **Small**—只使用低于**2K**的程序空间。

② **Compact**—单个函数的代码量不超过**2K**，整个程序可以使用**64K**程序空间。

③ **Large**—可以使用全部**64K**程序空间。

(4) **Use on-chip ROM**—是否仅使用片内**ROM**选项。注意，选中该项并不会影响最终生成的目标代码量。

(5) **Operation**—操作系统选项。**Keil**提供了两种操作系统：**Rtx tiny**和**Rtx full**。通常不选操作系统，所以选用默认项**None**。

(6) **off-chip Cod Memory**—用以确定系统扩展的程序存储器的地址范围。

(7) **off-chip Xdata Memory**—用以确定系统扩展的数据存储器的地址范围。

上述3个选项必须根据所用硬件来决定，如果是最小应用系统，不进行任何扩展，则按默认值设置。

2. Output页面

点击“Options for Target‘Target1’”窗口的“Output”选项，会出现Output页面，如图4-19所示。

- (1) **Create HEX File**—生成可执行文件代码文件。选择此项后即可生成单片机可以运行的二进制文件（.hex格式文件），文件的扩展名为.hex。
- (2) **Select Folder for objects**—选择最终的目标文件所在的文件夹，默认与项目文件在同一文件夹中，通常选默认。
- (3) **Name of Executable**—用于指定最终生成的目标文件的名称，默认与项目文件相同，通常选默认。

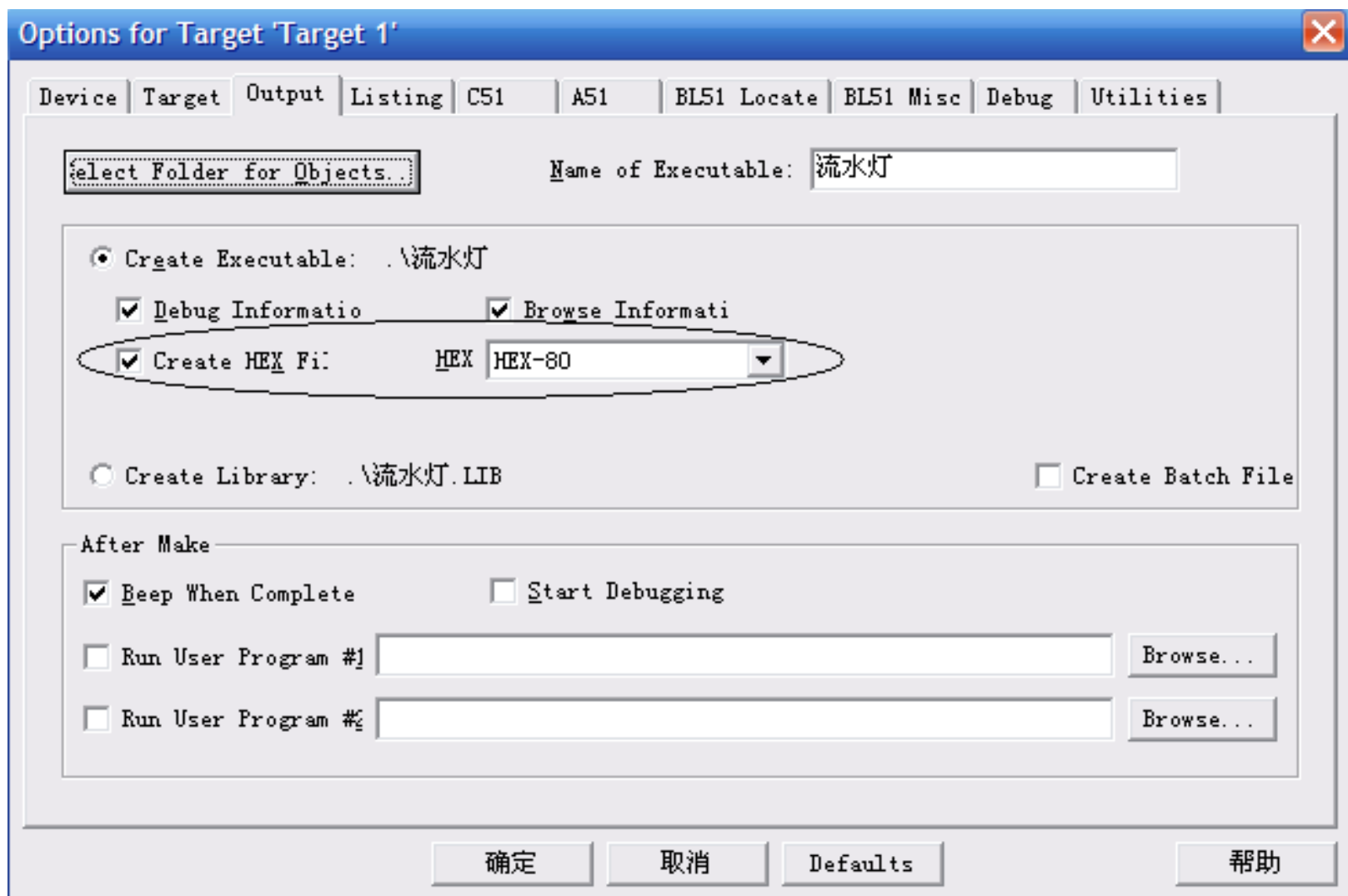


图4-19 Output页面

(4) **Debug information**—将会产生调试信息，这些信息用于调试，如果需要对程序进行调试，应选中该项。

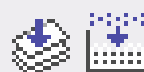
其他选项选默认即可。

完成设置后，就可在程序编译时，单击快捷按钮 ，此时会产生如图4-20的提示信息。该信息中说明程序占用片内**RAM**共**11**字节，片外**RAM**共**0**字节，



图4-20 hex文件生成的提示信息

占用程序存储器共**89**字节。最后生成的**.hex**文件名为“流水灯**.hex**”，至此，整个程序编译过程就结束了，生成的**.hex**文件就可在后面介绍的**Proteus**环境下进行虚拟仿真时，装入单片机运行。

下面对用于**编译、连接时的快捷按钮**   与  作简要说明：

- (1)  用于编译正在操作的文件。。
- (2)  按钮—用于编译修改过的文件，并生成相应的目标程序（.hex文件），供单片机直接下载。
- (3)  按钮—用于重新编译当前项目中的所有文件，并生成相应的目标程序（.hex文件），供单片机直接下载。主要用在当项目文件有改动时，来全部重建整个项目。



因为一个项目不止一个文件，当有多个文件时，可用本按钮进行编译。

用**C51**编写的源代码程序不能直接使用，一定要对该源代码程序编译，生成可执行的目标代码**.hex**文件，并加载到**Proteus**环境下的虚拟单片机中，才能进行虚拟仿真。

4.2 Proteus虚拟仿真平台简介

Proteus是英国**Lab center Electronics**公司**1989**年推出的完全使用软件手段来对单片机应用系统进行虚拟仿真的软件开发平台。

4.2.1 Proteus功能简介

Proteus是目前世界上唯一的支持嵌入式处理器的虚拟仿真平台，除可仿真模拟电路、数字电路外，还可仿真**8051**、**PIC12/16/18**系列、**AVR**系列、**MSP430**等各主流系列单片机，以及各种外围可编程接口芯片。此外还支持**ARM7**、**ARM9**等型号的嵌入式微处理器的仿真。



由于**Proteus**的虚拟仿真，不需要用户硬件样机，就可直接在**PC**机上对单片机系统进行虚拟仿真，将系统的功能及运行过程形象化，可以像焊接好的电路板一样看到单片机系统的执行效果。

Proteus元件库中具有**几万种**元件模型，可直接对单片机的各种外围元件及电路进行仿真，如**RAM**、**ROM**、总线驱动器、各种可编程外围接口芯片、**LED**数码管显示器、**LCD** 显示模块、矩阵式键盘以及多种**D/A**和**A/D** 转换器等。此外还可对**RS232**总线、**I²C**总线、**SPI**总线进行动态仿真。



Proteus提供了各种信号源、虚拟仿真仪器，并能对电路原理图的关键点进行虚拟测试。

Proteus提供了丰富的调试功能。在虚拟仿真中具有全速、单步、设置断点等调试功能，同时可观察各变量、寄存器的当前状态。

目前，**Proteus**已在包括剑桥、斯坦福、牛津大学、加州大学在内的全球数千所高校以及世界各大研发公司得到广泛应用。



尽管**Proteus**具有开发效率高，不需要附加的硬件开发装置成本等优点，但是不能进行用户样机硬件的诊断。

所以在单片机系统的设计开发中，一般是先在**Proteus**环境下绘出系统的硬件原理电路图，在**Keil C51**环境下书写并编译程序，然后在**Proteus**环境下仿真调试通过。

依照仿真结果，来完成实际的硬件设计，并把仿真通过的程序代码通过编程器或在线烧录到单片机的程序存储器中，然后运行程序观察用户样机的运行结果，如有问题，再连接硬件仿真器或直接在线修改程序去分析、调试。

4.2.2 Proteus ISIS的虚拟仿真

Proteus ISIS（智能原理图输入）界面是用来绘制单片机系统的电路原理图，还可直接实现单片机系统的虚拟仿真，可产生声、光及各种动作等逼真的效果。

当电路连接无误后，单击单片机芯片载入经**Keil C51**调试编译后生成的**.hex**文件，点击仿真运行按钮，即可检验电路硬件及软件的设计正确与否。

按要求把Proteus安装在PC机上。安装完后，单击桌面上的ISIS运行界面图标，即可出现如图4-21所示的Proteus ISIS 原理电路图绘制界面。

整个屏幕界面分为若干个区域，由原理图编辑窗口、预览窗口、工具箱、主菜单栏、主工具栏等组成。

一、ISIS各窗口简介

ISIS界面主要有3个窗口：原理图编辑窗口、预览窗口和对象选择窗口。

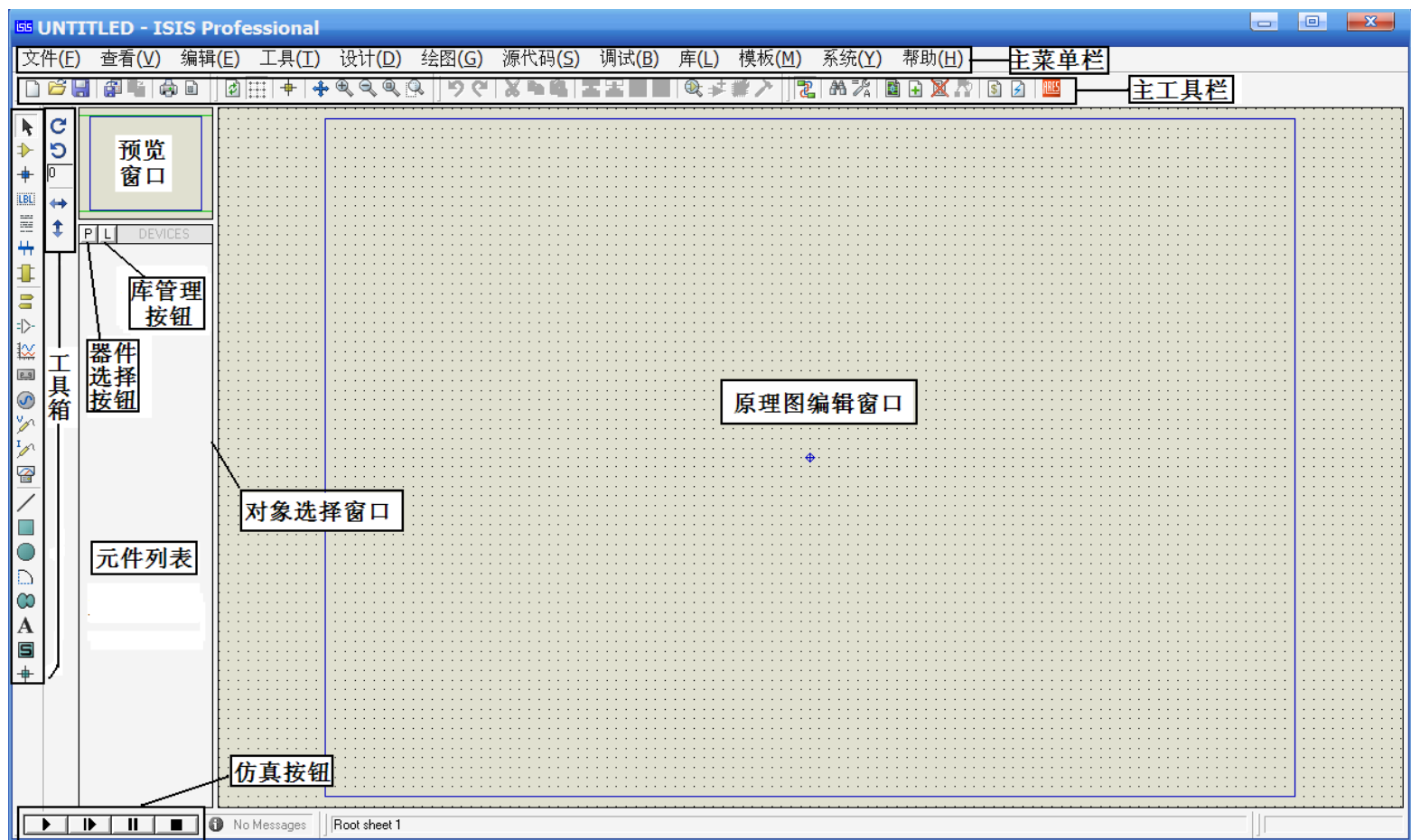


图4-2 Proteus 的ISIS的界面

1.原理图编辑窗口

用来绘制电路原理图、电路设计、设计各种符号模型的区域，蓝色方框内为可编辑区，元件放置、电路设置都在此框中完成。

2. 预览窗口

可对选中的元器件进行预览，也可对原理图编辑窗口预览。如图4-22所示。可显示两个内容：

(1) 如单击元件列表中的元件时，预览窗口会显示该元件符号。

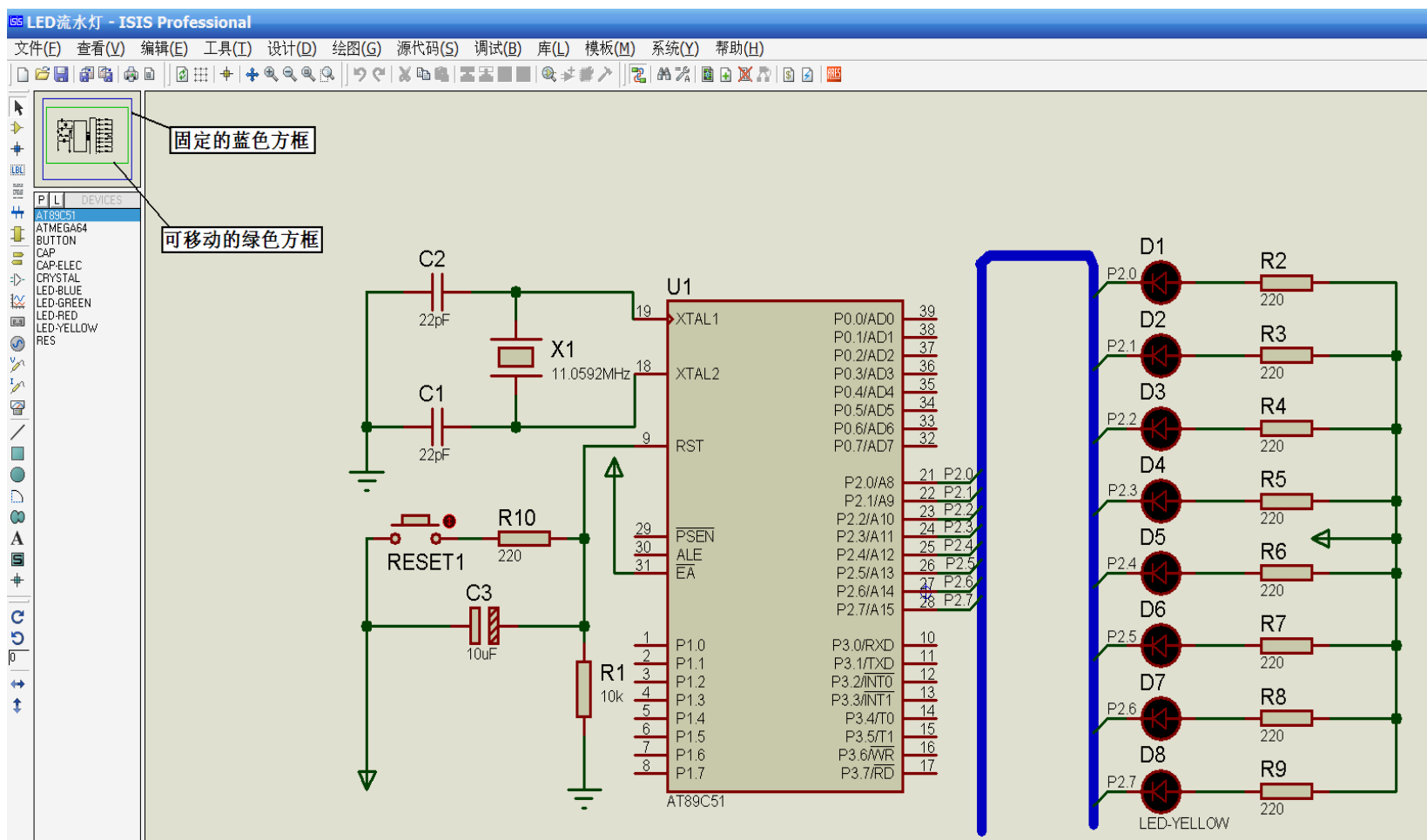


图4-22 预览窗口调整原理图的可视范围

(2) 当鼠标焦点落在原理图窗口时（即放置元件到原理图编辑窗口后或在原理图编辑窗口中点击鼠标后），它会显示整张原理图的缩略图，并会显示一个绿色方框，框里的内容就是当前原理图窗口中显示的内容。

单击绿色方框中的某一点，就可拖动鼠标来改变绿色方框的位置，从而改变原理图的可视范围，最后在绿色方框内单击鼠标，绿色方框就不再移动，使得原理图的可视范围固定。

3.对象选择窗口

用来选择元器件、终端等对象。该窗口中的元件列表区用来表明当前所处模式以及其中的对象列表，如图4-23所示。窗口中两个按钮：“P”为器件选择按钮，“L”为库管理按钮。图中，可以看到元件列表，即已经选择的AT89C51单片机、电容电阻、晶振、发光二极管等各种元器件列表。



P	L	DEVICES
		AT89C51
		ATMEGA64
		BUTTON
		CAP
		CAP-ELEC
		CRYSTAL
		LED-BLUE
		LED-GREEN
		LED-RED
		LED-YELLOW
		RES

图4-23 元件列表

二、主菜单栏

图4-21最上面一行为主菜单栏，包含如下命令：文件、查看、编辑、工具、设计、绘图、源代码、调试、库、模板、系统和帮助。单击任意菜单命令后，都将弹出其下拉的子菜单命令列表。下面简要介绍主菜单栏中的几个常用命令。

1. 文件（File）菜单

包括项目的新建设计、打开设计、导入位图、导入区域、导出区域和打印等操作，如**图4-24**所示。

ISIS下的文件主要是设计文件（**Design Files**），文件扩展名为“**.DSN**”。它包括一个单片机硬件系统的原理电路图及其所有信息，用于虚拟仿真。



图4-24 文件菜单

下面介绍文件菜单下的**新建设计**命令。

点击【文件】→“新建设计，出现一个空的A4纸。新设计的默认名为“UNTITLED.DSN”。本命令会把该设计以这个名字存入磁盘文件中，文件的其他选项也会使用它作为默认名。

如想进行新的设计，需给该设计命名，然后点击【文件】→“保存设计”（也可直接点击主工具栏中的快捷图标），输入新的文件名保存即可。

2. 工具（Tools）菜单

工具菜单见图4-25。菜单中的“自动连线（W）”命令图标，在绘制电路原理图中用到，命令文字前的快捷图标，在绘制电路原理图时出现，按下快捷图标即进入自动连线状态。

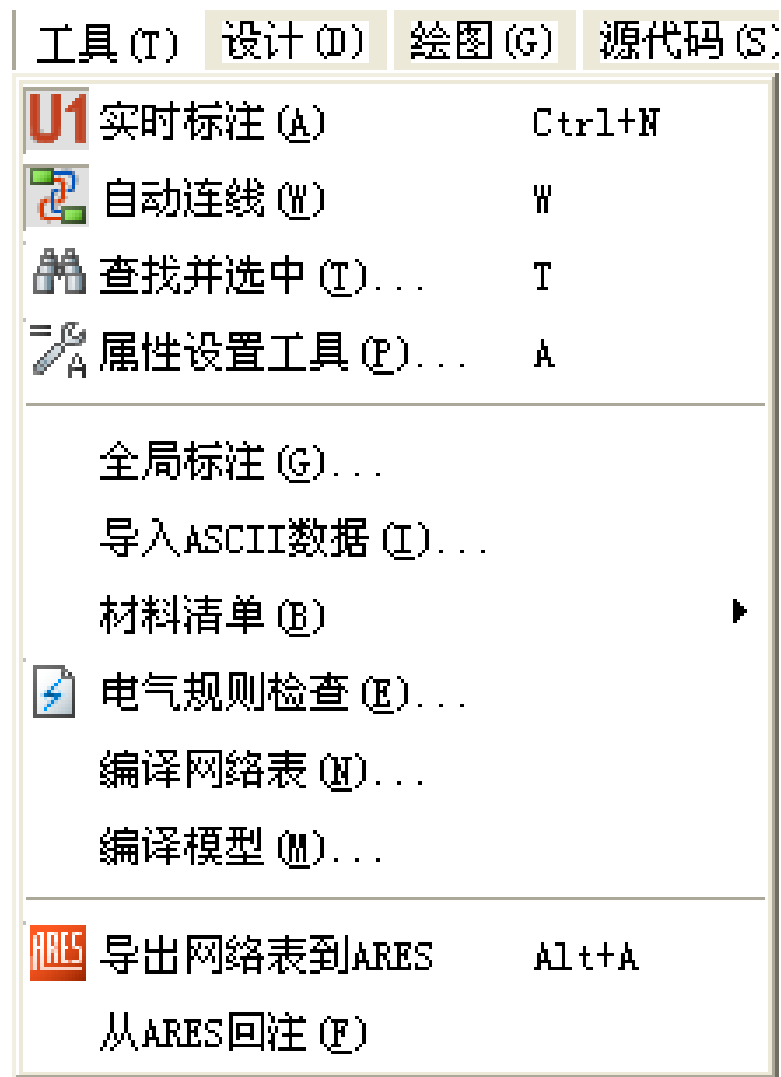


图4-25 工具菜单

3. 调试 (Debug) 菜单

调试菜单见图4-26。主要完成单步运行、断点设置等功能。



图4-26 调试菜单

三、主工具栏

主工具栏位于主菜单下面，以图标形式给出，栏中共有**38个**快捷图标按钮：



每一个图标按钮都对应一个具体的菜单命令，主要目的是为了快捷方便地使用这些命令。

下面把**38个图标**分为**4组**，简要介绍快捷图标命令的功能。

图标    |   |   的功能如下:

	新建一个设计文件
	打开一个已存在的设计文件
	保存当前的电路图设计
	将一个局部文件导入 ISIS 中
	将当前选中的对象导出为一个局部文件
	打印当前设计文件
	选择打印的区域

图标



的功能如下:

	刷新显示
	原理图是否显示网格的开关
	是否显示手动原点
	以鼠标所在点为中心居中
	放大
	缩小
	查看整张图
	查看局部图

图标  的功能如下:

	撤销最后一步操作
	恢复最后一步操作
	剪切选中对象
	复制选中对象至剪切板
	从剪切板粘贴
	复制选中的块对象
	移动选中的块对象
	旋转选中的块对象
	删除选中的块对象
	从库中选取器件
	创建器件
	封装工具
	释放元件

图标           的功能如下：

	自动连线
	查找并连接
	属性分配工具
	设计浏览器
	新建图纸
	移动页面/删除页面
	退出到父页面
	生成元件列表
	生成电气规则检查报告
	生成网表并传输到 ARES

四、工具箱

图4-21最左侧为工具箱，选择相应的工具箱图标按钮，系统将提供不同的操作工具。对象选择器根据不同的工具箱图标决定当前状态显示的内容。显示对象的类型包括：元器件、终端、引脚、图形符号、标注和图表等。

下面介绍工具箱中各图标按钮对应的功能。

(1) 模型工具栏各图标的功能

：选择模式。

：元件模式，用来拾取元器件。

设计者可根据需要，从丰富的元件库中拾取元器件并添加元件到列表中。单击此图标可在列表中选择元件。

：放置电路的连接点

此按钮适用于节点的连线，在不用连线工具的条件下，可方便地在节点之间或节点到电路中任意点或线之间连线。

：标注线标签或网络标号

本图标按钮在绘制电路图时，具有非常重要的意义，它可使连线简单化。例如，从 8051 单片机的 P1.7 脚和二极管的阳极各画出一条短线，并标注网络标号为 1，那么就说明 P1.7 脚和二极管的阳极已经在电路上连接在一起了，而不用真的画一条线把它们连起来。

：输入文本

使用本图标按钮命令，可在绘制的电路上添加说明文本。

：绘制总线

总线在电路图上表现出来的是一条粗线，它代表的是一组总线。当连接到总线上时，要注意标好网络标号。

：绘制子电路块

：选择端子。

点击此图标按钮，在对象选择器中列出可供选择的各种常用端子：

- DEFAULT 默认~~的~~无定义端子
- INPUT 输入端子
- OUTPUT 输出端子
- BIDIR 双向端子
- POWER 电源端子
- GROUND 接地端子
- BUS 总线端子

：选择元件引脚

点击此图标，在对象选择器中列出可供选择的各种引脚（例如，普通引脚、时钟引脚、反电压引脚和短接引脚）。：

：在对象选择器中列出可供选择的各种仿真分析所需的图表（如模拟图表、数字图表、混合图表和噪声图表等）。

：当需要对设计电路分割仿真时，采用此模式。

：在对象选择器中列出各种信号源（如正弦、脉冲和 FILE 信号源等）模式。

：在电路原理图中添加电压探针。电路仿真时可显示探针处的电压值。

：在电路原理图中添加电流探针。电路仿真时可显示探针处的电流值。

：在对象选择器中列出可供选择的虚拟仪器。

(2) 2D 图形模式各图标按钮功能

：画线，点击本图标，右侧的窗口中提供了各种专用的画线工具，具体如下：

：画一个方框

：画一个圆

：画一段弧线

：图形弧线模式。

：图形文本模式。

：图形符号模式。

(3) 旋转或翻转的图标按钮：对元件预览窗口内的元件，可进行旋转或翻转。

：元件顺时针方向旋转，只能是 90° 的整数倍。

：元件逆时针方向旋转，角度只能是 90° 的整数倍。

：元件水平镜像旋转。

：元件垂直镜像旋转。

五、元件列表

如**图4-27**所示，元件列表用于挑选元件、终端接口、信号发生器、仿真图表等。

挑选元件时，单击“**P**”快捷图标，这时会打开挑选元件的对话框，在对话框中的“关键字”里面输入要检索的元器件的关键词，例如要选择使用**AT89C51**，就可以直接输入。输入后能够在中间的“结果”栏里面看到搜索的元器件的结果。

在对话框的右侧，还能够看到选择的元器件的仿真模型以及**PCB**参数，选择了元件**AT89C51**后，并双击**AT89C51**，该元件就会在左侧的元件列表中显示，以后用到该元件时，只需在元件列表中选择即可。

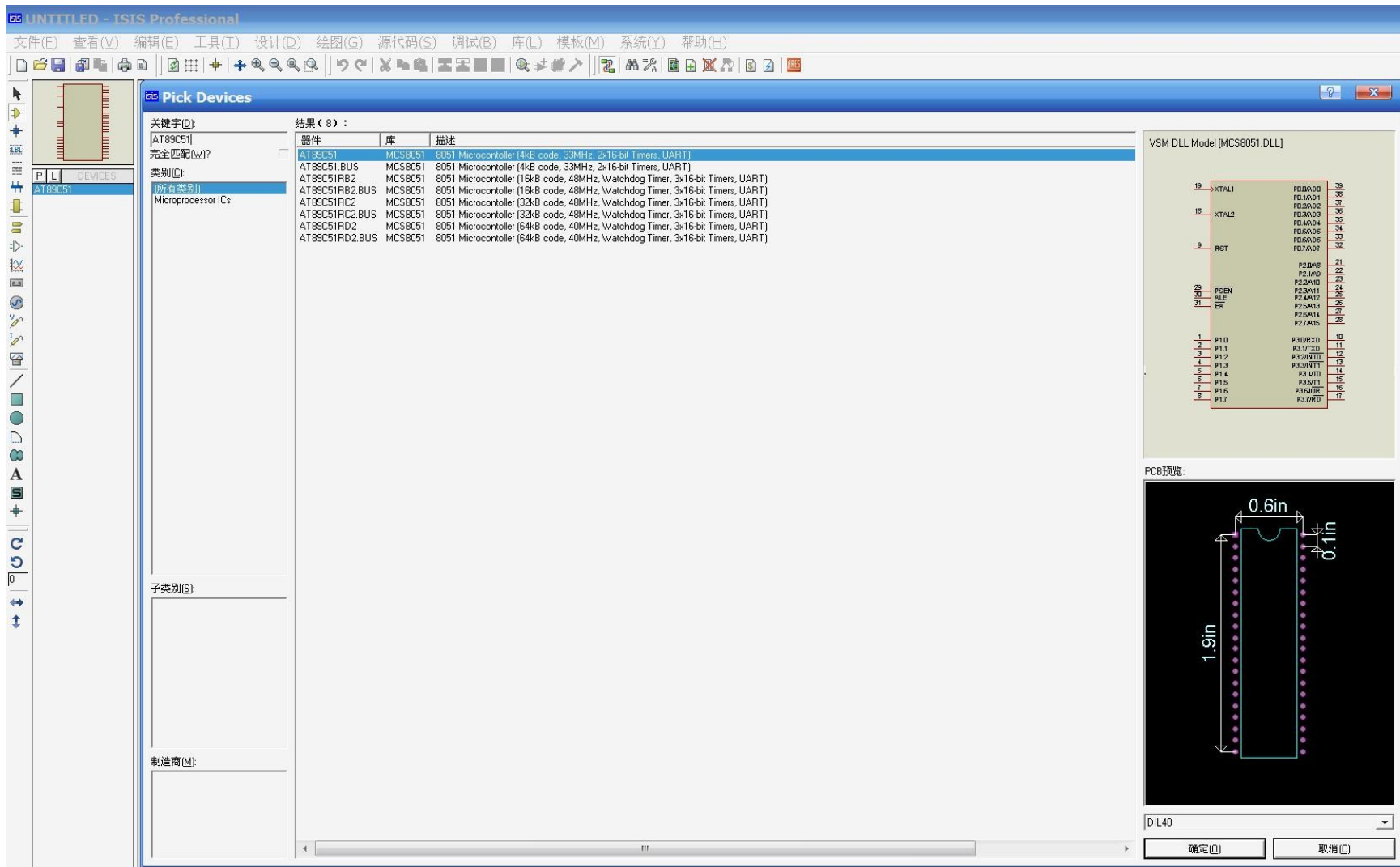


图4-27 元件列表

4.2.3 Proteus的各种虚拟仿真调试工具

Proteus提供了多种虚拟仿真工具，以检查设计的正确性，为单片机系统的电路设计、分析以及软硬件联调测试带来极大的方便。

一、虚拟信号源

Proteus ISIS为用户提供了各种类型的虚拟激励信号源，并允许对其参数进行设置。单击工具箱中的快捷图标 ，就会出现如**图4-28**所示的各种类型的激励信号源的名称列表及对应的符号。**图4-28**中选择的是正弦波信号源，在预览窗口中显示的是正弦波信号源符号。

名称列表中各符号所对应的激励信号源见**表4-1**。

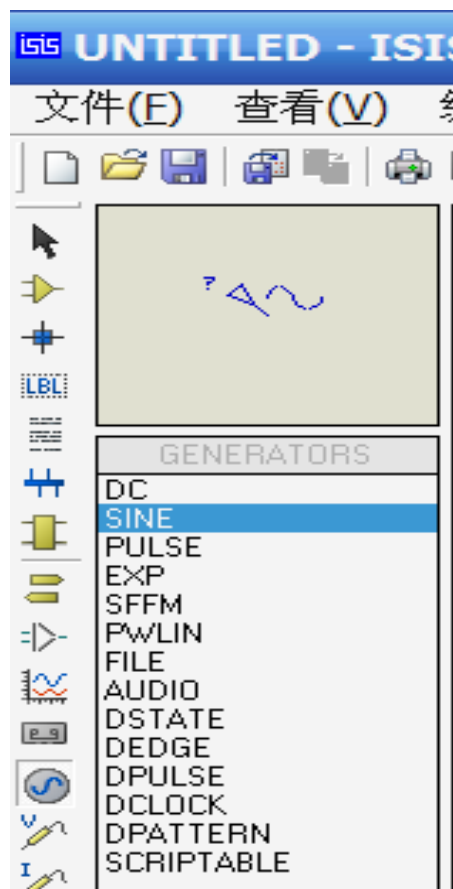


图4-28 各种激励信号源及对应的符号

表4-1 各种激励信号源及对应的符号

符号	激励信号源
DC	直流信号源
SINE	正弦波信号源
PULSE	脉冲发生器
AUDIO	音频信号发生器
DSTATE	单稳态逻辑电平发生器
DEDGE	跳沿信号发生器
DPULSE	单周期数字脉冲发生器
DCLOCK	数字时钟信号发生器
.....	

4.2.3.2 虚拟仪器

单击工具箱中的快捷按钮，可列出所有的虚拟仪器名称，如图4-29所示。

图4-29中的名称列表中所对应的虚拟仪器名称见表4-2。

表 4-2 各种符号对应的虚拟仪器

符号	激励信号源
OSCILLOSCOPE	示波器
LOGIC ANALYSER	逻辑分析仪
COUNTER TIMER	计数器/定时器
VIRTUAL TERMINAL	虚拟终端
SPI DEBUGGER	SPI 调试器
I2C DEBUGGER	I2C 调试器
SIGNAL GENERATOR	信号发生器
PATTEN GENERATOR	图形发生器
DC VOLTMETER	直流电压表
DC AMMETER	直流电流表
AC VOLTMETER	交流电压表
AC AMMETER	交流电流表

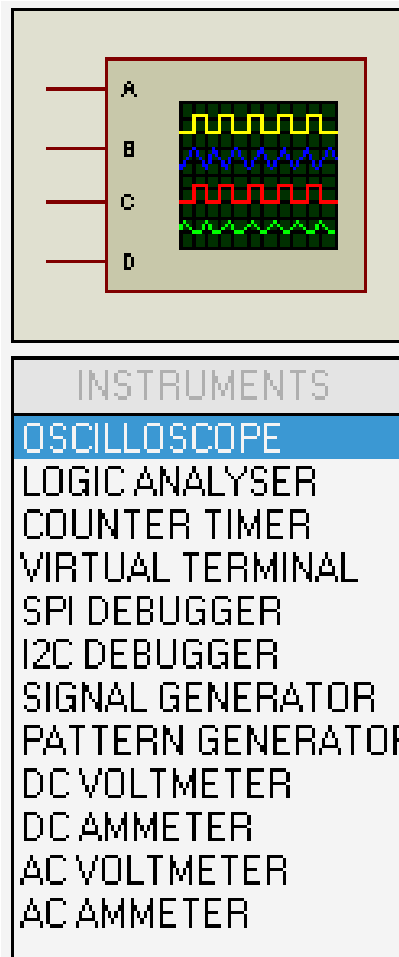


图4-29 虚拟仪器名称列表

下面简要介绍在单片机应用系统调试中几种虚拟仪器。

1. 虚拟终端

虚拟终端的原理图符号如图4-30所示。在调试异步串行通信使用。虚拟终端共有4个接线端，其中**RXD**为数据接收端，**TXD**为数据发送端，**RTS**为请求发送信号，**CTS**为清除传送，是对**RTS**的响应信号。

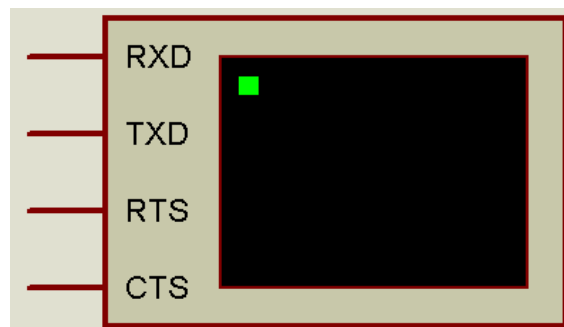


图4-30 虚拟终端的原理图符号



图4-31为单片机与上位机（**PC机**）之间进行串行通信，使用虚拟终端就可免去**PC机**的仿真模型，直接由虚拟终端**VT1**、**VT2**显示出经**RS232**串行接口模型与单片机之间异步发送或接收数据的情况。**VT1**显示的数据表示了单片机经串口发给**PC机**的数据，**VT2**显示的数据表示了**PC机**经**RS232**接口模型接收到的数据，从而省去了**PC机**的串口模型。

虚拟终端在运行仿真时会弹出一个仿真界面，当**PC机**向单片机发送数据时，可以和虚拟键盘关联，用户可从虚拟键盘经虚拟终端输入数据；当**PC机**接收到单片机发送来的数据后，虚拟终端相当于一个显示屏，会显示相应信息。

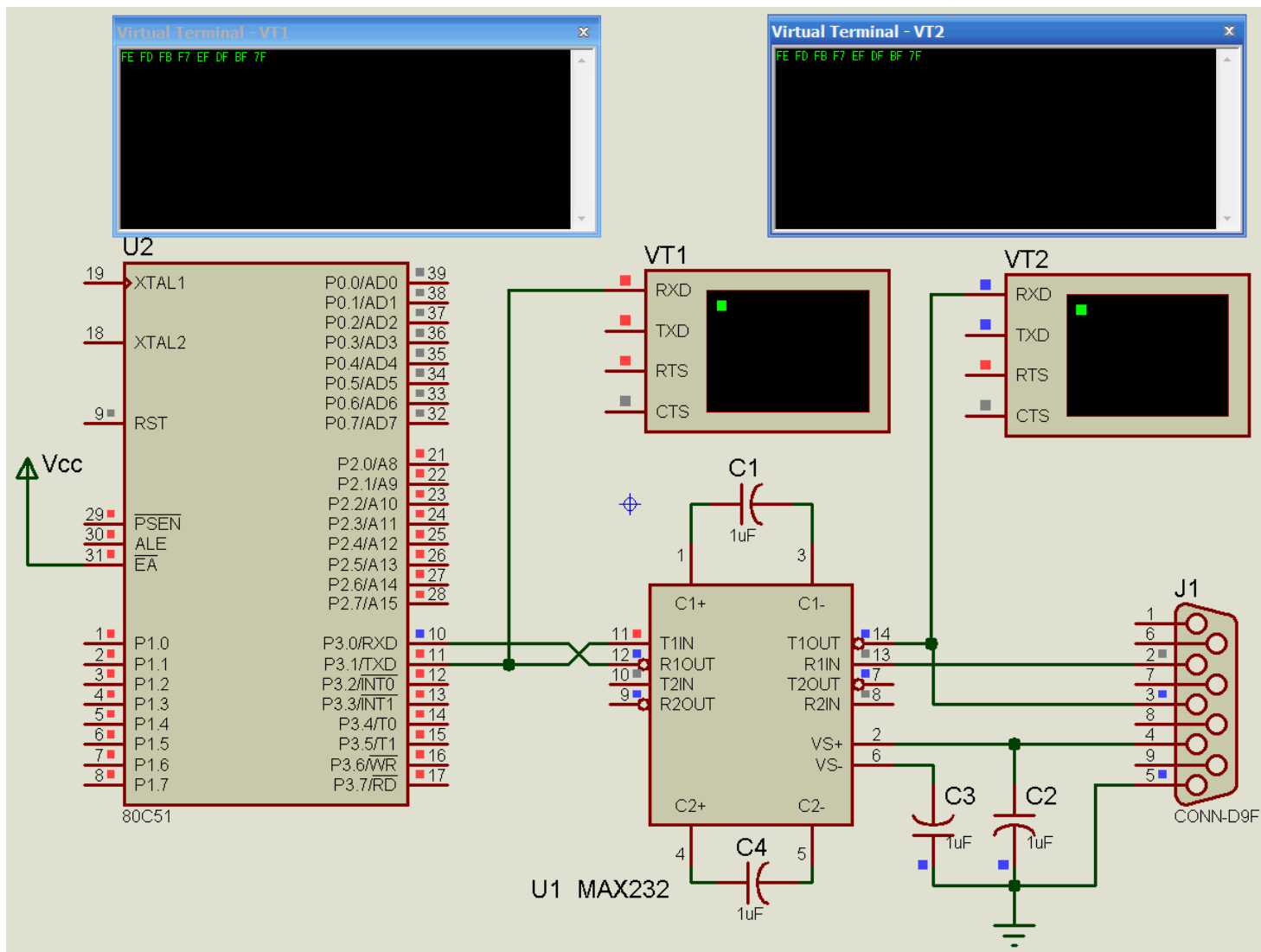


图4-31 单片机与PC机之间串行通信的虚拟终端

2. I2C调试器

图4-32中的虚拟仪器名称列表中的“I2C DEBUGGER”就是I2C调试器，允许用户监测I2C接口总线，可以查看I2C总线发送的数据，同时也可作为从器件向I2C总线发送数据。

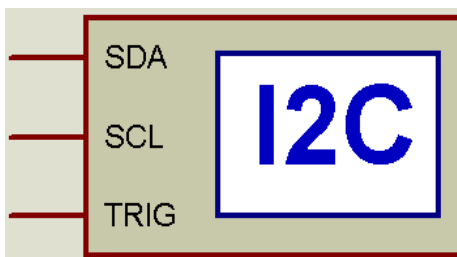


图4-32 I2C调试器的原理图符号

I2C调试器有3个接线端。

- **SDA**: 双向数据线。
- **SCL**: 时钟线，双向。
- **TRIG**: 触发输入，能使存储序列被连续地放置到输出队列中。

图4-33所示为单片机通过控制**I2C**总线向带有**I2C**接口的存储器芯片**AT24C02**进行读写，可利用**I2C**调试器来观察**I2C**总线数据传送的过程。

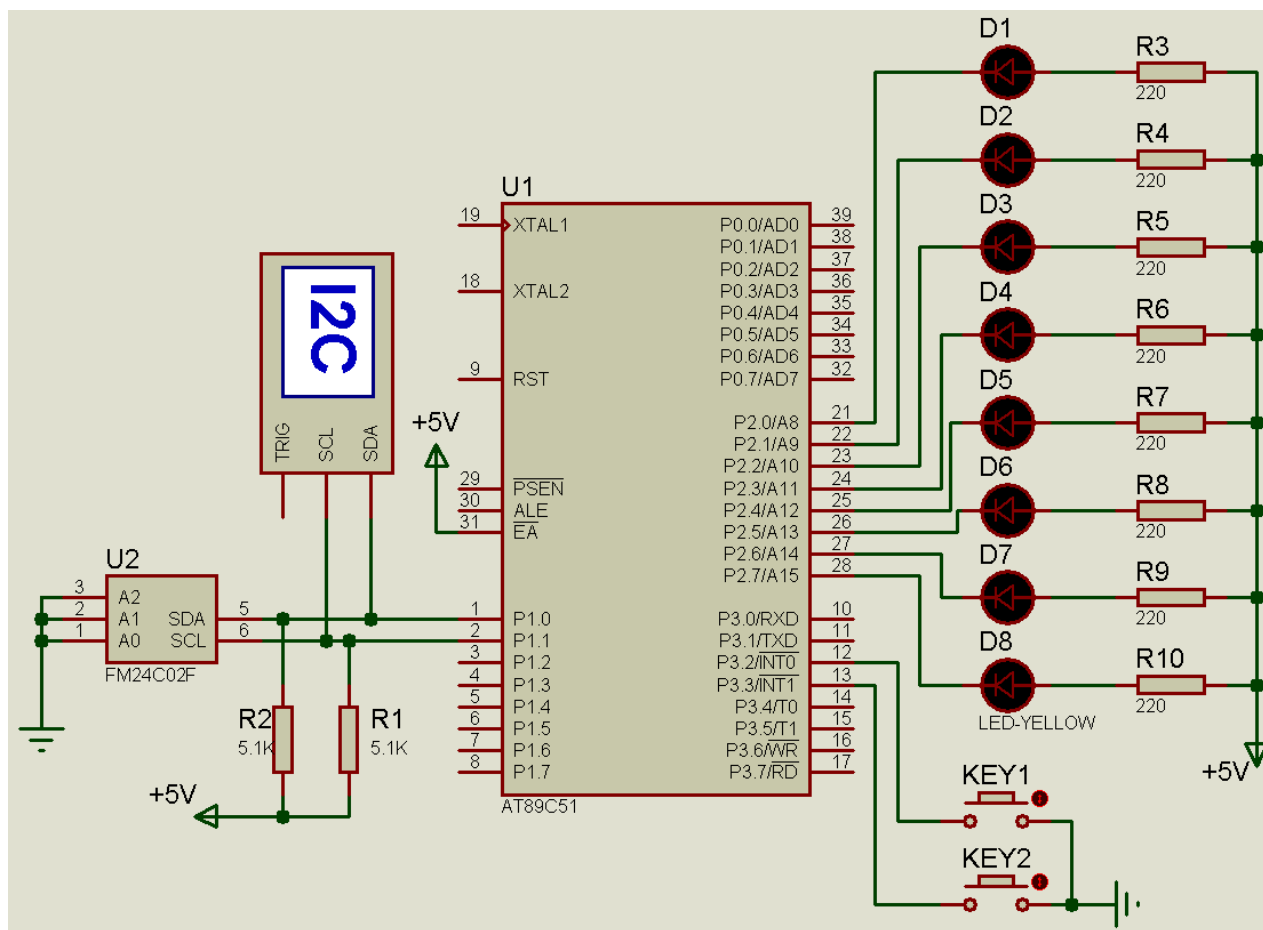


图4-33 单片机读写带有I2C接口的存储器AT24C02的电路原理图

启动仿真，鼠标右键单击**I2C**调试器，出现**I2C**调试窗口，如图4-34所示。

单击其中的“+”符号，还能把**I2C**总线传送数据的细节展现出来。**I2C**总线传送数据时，采用了特别的序列语句，出现在数据监测窗口中。

由图4-34可见，使用**I2C**调试器可非常方便地观察**I2C**总线上传输的数据，非常容易地手动控制**I2C**总线发送的数据，为**I2C**总线的单片机系统提供了十分有效的仿真调试手段。

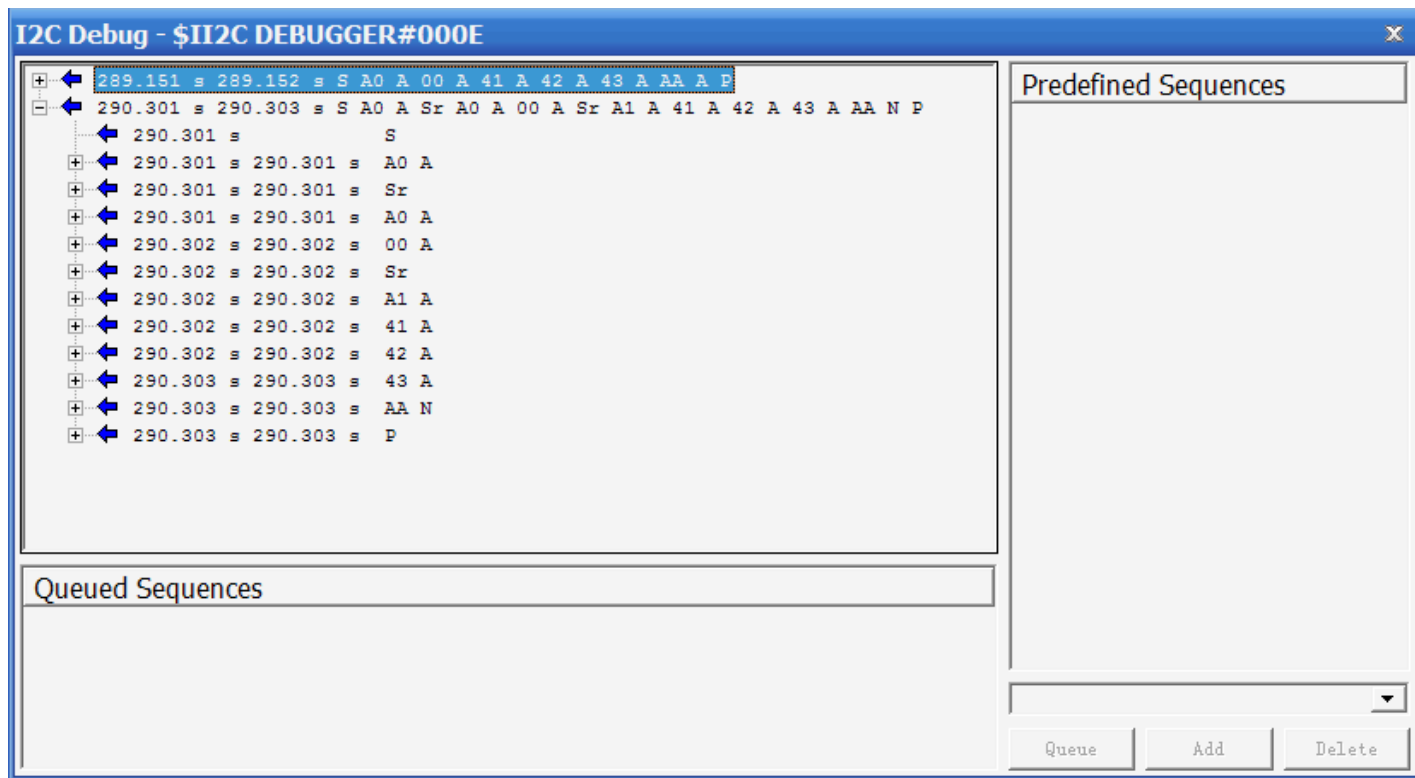


图4-34 I2C调试窗口及单片机向AT24C02写入和读出的数据

3. SPI调试器

SPI调试器允许用户查看沿**SPI**总线发送和接收的数据。

图4-35所示为**SPI**调试器的原理图符号。

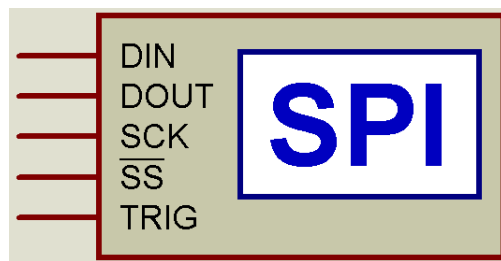


图4-35 SPI调试器的原理图符号

SPI调试器共有**5个接线端**。分别介绍如下。

- **DIN**: 接收数据端。
- **DOUT**: 输出数据端。
- **SCK**: 时钟端。
- **SS***: 从模式选择端，从模式时此端必须为低电平才能使终端响应；当工作在主模式下，而且数据正在传输时此端才为低电平。
- **TRIG**: 输入端，能把下一个存储序列放到**SPI**的输出序列中。

SPI调试器的窗口如**图4-36**所示，它与I2C调试窗口是相似的。



图4-36 SPI调试器的窗口

4. 电压表和电流表

Proteus VSM提供了4种电表，如图4-37所示，分别是DC Voltmeter（直流电压表）、DC Ammeter（直流电流表）、AC Voltmeter（交流电压表）和AC Ammeter（交流电流表）。

可分别把4种电表放置到原理图编辑窗口中。

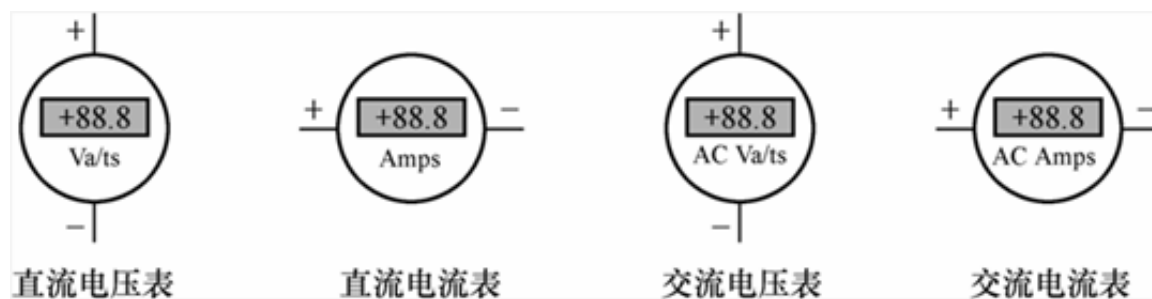


图4-37 4种电表的原理图符号

4.2.4 虚拟设计仿真举例

在**Proteus**开发环境下的一个单片机系统的设计与虚拟仿真应分为**3个**步骤。

(1) **Proteus ISIS**环境下的电路原理图设计。

(2) 在**Keil C51**平台上进行源程序的输入、编译与调试，并最终生成目标代码文件 (*.hex文件)。

(3) 调试与仿真,在**Proteus**环境下将目标代码文件 (*.hex文件) 加载到单片机中，并对系统进行虚拟仿真。

下面以“流水灯”的设计为例，介绍如何使用**Proteus**。

一、新建或打开一个设计文件

1. 建立新设计文件

点击菜单的【文件】→“新建设计”选项（或点击主工具栏的快捷按钮）来新建一个文件。如果选择前者新建设计文件，会弹出如图4-38的“新建设计”窗口。

窗口中提供多种模板。单击要选的模板图标，再单击“确定”按钮，即建立一个该模板的空白文件。如果直接单击“确定”按钮，即选用系统默认的“DEFAULT”模板。

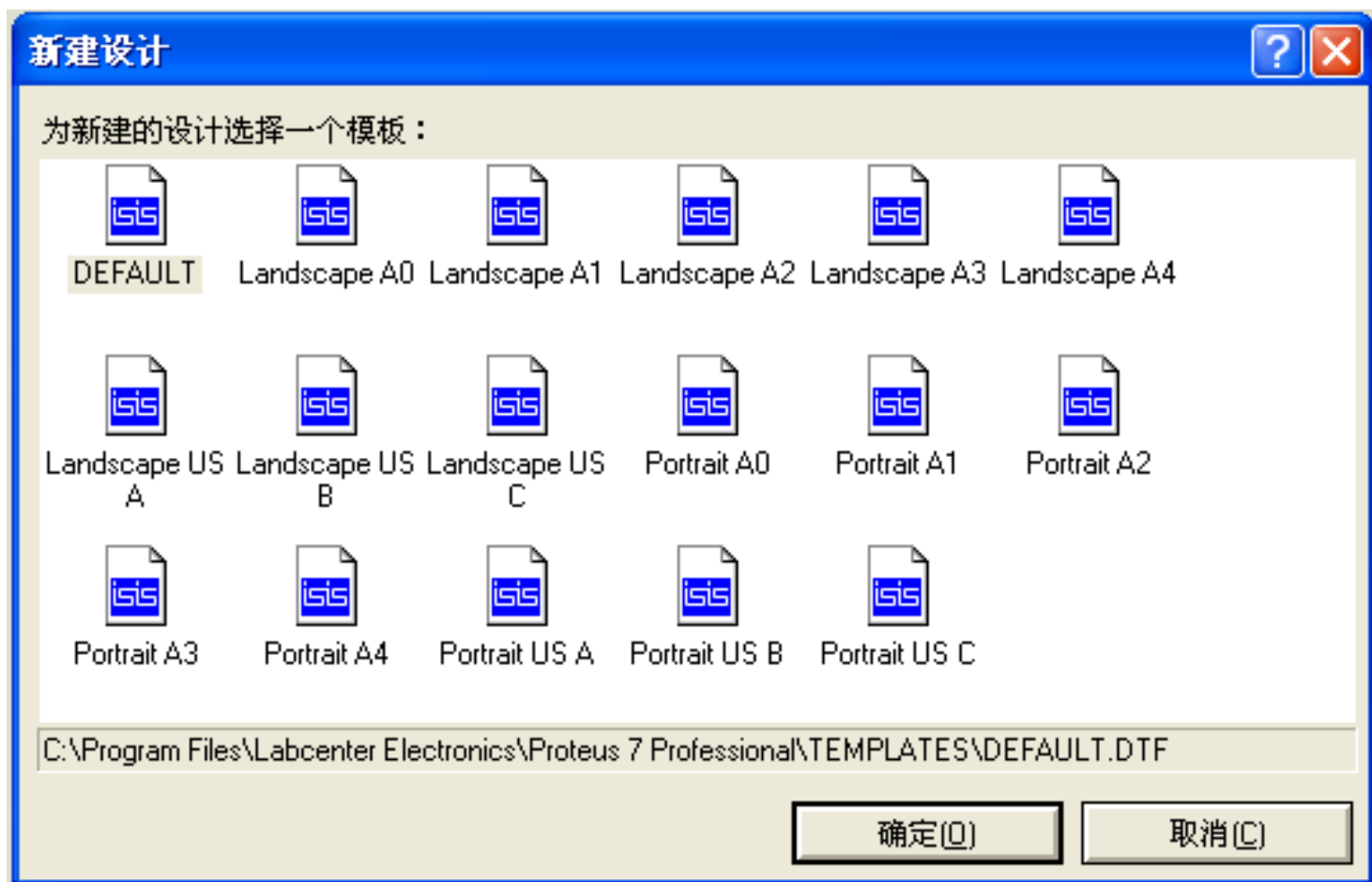


图4-38 “新建设计”窗口

如果用工具栏的快捷图标按钮  来新建文件，就不会出现图4-38所示的窗口，而直接选择系统默认模板。

2. 保存文件

在建立了一个新的文件，第一次保存该文件时，选择菜单栏【文件】→“另存为（A）”选项，即弹出图4-39所示的“保存ISIS设计文件”窗口，在该窗口选择文件的保存路径和文件名“流水灯”，后，单击“保存”，则完成了设计文件的保存。这样就在“实验1（流水灯）”子目录下建立了一个文件名为“流水灯”的新的设计文件。

如不是第一次保存，可选择菜单栏【文件】→“保存设计（S）”选项，或直接单击快捷图标按钮  即可。



图4-39 “保存ISIS设计文件”窗口

3. 打开已保存的文件

选择菜单栏【文件】→“打开设计（O）”，将弹出图4-40的“加载ISIS设计文件”窗口。单击需打开的文件名，再单击“打开”按钮即可。



图4-40 “加载ISIS设计文件” 窗口

二、选择需要的元件到元件列表

在电路设计前，要把设计“流水灯”电路原理图中需要的元器件列出，见表4-3。然后根据表4-3选择元件到元件列表中。

表4-3 流水灯所需元件列表

元件名称	型号	数量	Proteus的关键字
单片机	AT89C51	1	AT89C51
晶振	12MHz	1	CRYSTAL
二极管	蓝色	8	LED-BLUE
二极管	绿色	8	LED-GREEN
二极管	红色	8	LED-RED
二极管	黄色	8	LED-YELLOW
电容	24pF	4	CAP
电解电容	10 μ F	1	CAP-ELEC
电阻	240 Ω	10	RES
电阻	10k	1	RES
复位按钮		1	BUTTON

观察图4-21，左侧的元件列表中没有一个元件，单击左侧工具栏中的按钮  再单击器件选择按钮  就会出现“Pick Devices”窗口，在窗口的“关键字”栏中，输入AT89C51，此时在“结果”栏中出现“元件搜索结果列表”，并在右侧出现“元件预览”和“元件PCB预览”，见图4-41。在“元件搜索结果列表”中双击所需要的元件AT89C51，这时在主窗口的元件列表中就会添加该元件。用同样方法可将表4-3中所需要选择的其他元件也添加到元件列表中即可。

所有元件选取完毕后，单击图4-41中“确定”按钮，即可关闭“Pick Devices”窗口，回到主界面进行原理图绘制。此时“流水灯”的元件列表见图4-23。

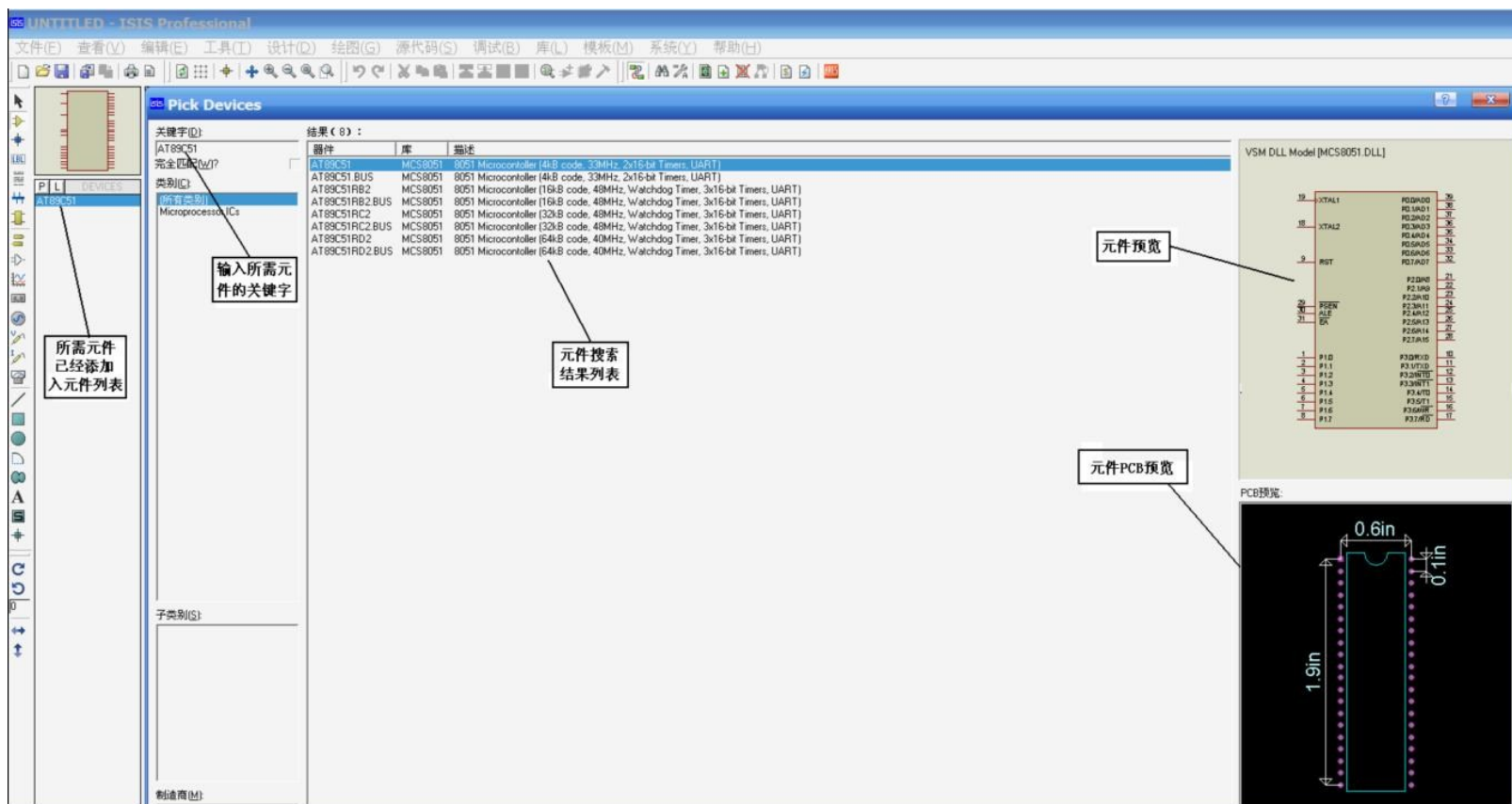


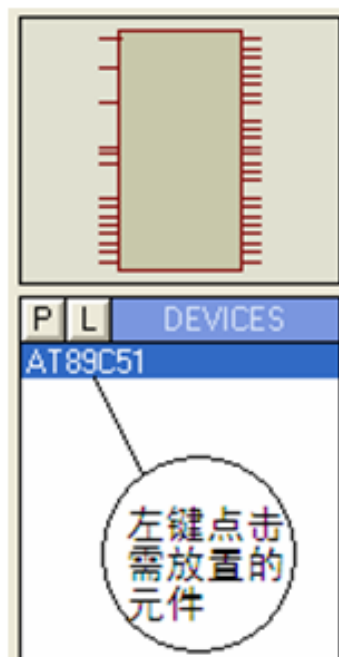
图4-41 “Pick Devices” 窗口

三、放置元件并连接电路

1. 元件的放置、调整与编辑

(1) 元件的放置

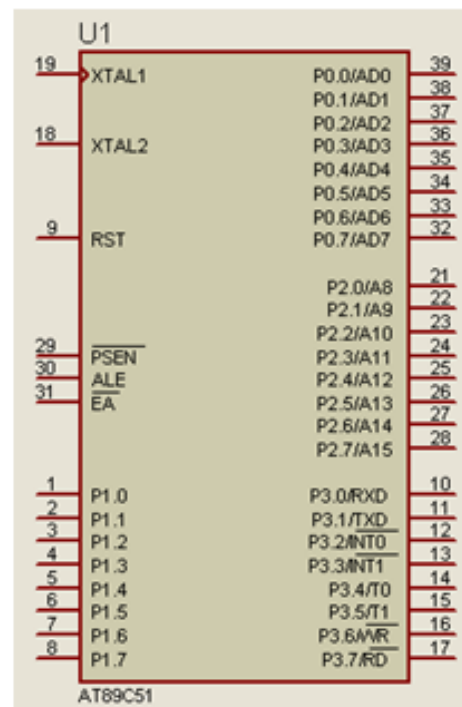
单击元件列表中所需要放置的元器件，然后将鼠标移至原理图编辑窗口中单击一下，此时就会在鼠标处有一个粉红色的元器件，移动鼠标选择合适的位置，单击一下左键，此时该元件就被放置在原理图窗口了。例如选择放置单片机**AT89C51**到原理图编辑窗口，具体步骤见**图4-42**。



(a)



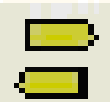
(b)

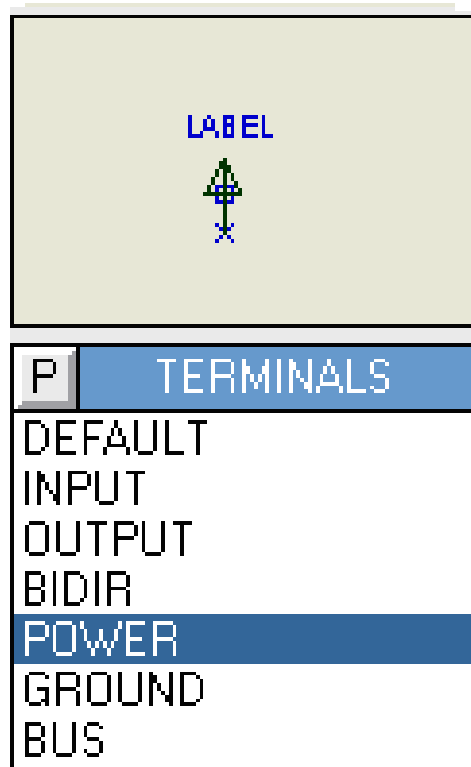


(c)

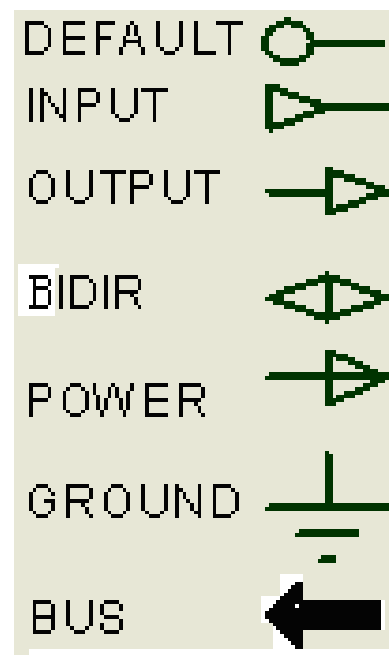
图4-42 元件放置的操作步骤

若要删除已放置的元件，用鼠标左键单击该元件，然后按**Delete**键删除元件，如误删除，可以点击快捷按钮  恢复。

一个电路原理图设计，除元器件还需要电源和地等终端，单击工具栏中的快捷按钮  ，就会出现各种终端列表，点击元件终端中的某一项，上方的窗口中就会出现该终端的符号，如**图4-43 (a)**所示。此时可选择合适的终端放置到电路原理图，放置的方法与元件放置相同。



(a)



(b)

图4-43 终端列表及终端符号

图4-43 (b) 为**图4-43 (a)** 列表中的终端符号。当再次单击  按钮时，即可切换回到用户自己选择的元件列表，见**图4-23**。根据上述介绍，可将所有的元器件及终端放置到原理图编辑窗口中去。

(2) 元件位置的调整

a. 改变元件在原理图中的位置，用鼠标左键点击需调整位置的元件，元件变为红颜色，移动鼠标到合适的位置，再释放鼠标即可。

b. 调整元件角度，用右键单击需调整的元件，会出现**图4-44**的菜单，操作菜单中的命令选项即可。

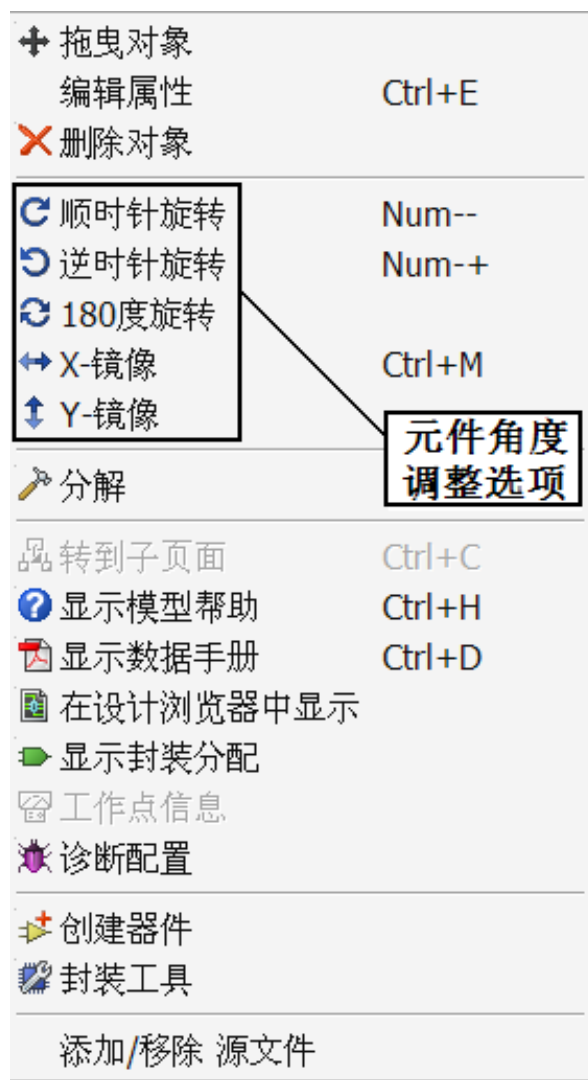


图4-44 调整元件角度的命令选项

（3）元件参数设置

用鼠标双击需要设置参数的元件，就会出现“编辑元件”窗口。下面以单片机**AT89C51**为例，双击**AT89C51**，出现“编辑元件”窗口，其中的基本信息如下：

设计者可根据设计的需要，双击需要设置参数的元件，进入“编辑元件”窗口自行完成原理图中各元件的参数设置。

2. 电路元件的连接

(1) 两元件间绘制导线

在元件模式快捷按钮  与自动布线器快捷按钮  按下时，两个元件导线的连接方法是：先单击第一个元件的连接点，移动鼠标，此时会在连接点引出一根导线。如想要自动绘出直线路径，只需点击另一个连接点。

如果设计者想自己决定走线路径，只需在希望的拐点处单击鼠标左键。需注意的是，拐点处导线的走线只能是直角。在自动布线器快捷按钮  松开时，导线可按任意角度走线，只需要在希望的拐点处单击鼠标左键，把鼠标指针拉向目标点，拐点处导线的走向只取决于鼠标指针的拖动。

(2) 连接导线连接的圆点

单击连接点按钮 ，会在两根导线连接处或两根导线交叉处添加一个圆点，表示它们是连接的。

(3) 导线位置的调整

对某一绘制的导线，要想进行位置的调整，可用鼠标左键点击导线，导线两端各有一个小黑方块，单击右键出现菜单，如**图4-45**所示。点击“拖拽对象”，即可拖拽导线到指定的位置，也可进行旋转，然后单击导线，这就完成了导线位置的调整。

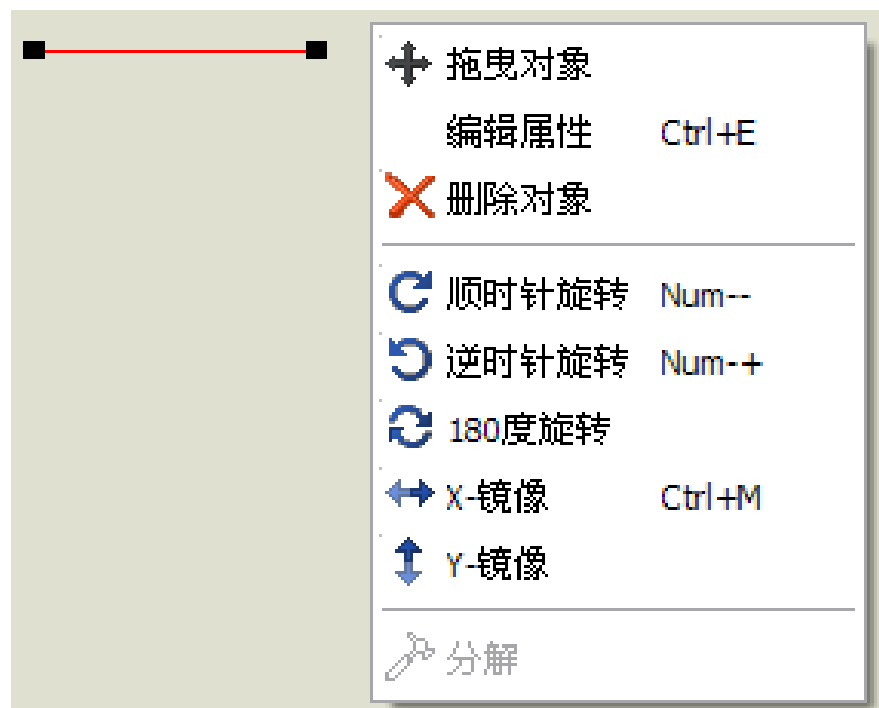


图4-45 改变导线位置的菜单

(4) 绘制总线与总线分支

总线绘制：单击工具栏的图标按钮 ，移动鼠标到绘制总线的起始位置，单击鼠标左键，便可绘制出一条总线。如想要总线出现不是 90° 角的转折，此时自动布线器快捷按钮  应当松开，总线即可按任意角度走线，只需要在希望的拐点处单击鼠标左键，把鼠标指针拉向目标点，拐点处导线的走向只取决于鼠标指针的拖动。在总线的终点处双击鼠标左键，即结束总线的绘制。

总线分支绘制：总线绘制完以后，有时还需绘制总线分支。为了使电路图显得专业和美观，通常把总线分支画成与总线成 45° 角的相互平行的斜线，如图4-46所示。注意此时一定要把自动布线器快捷按钮  松开，总线分支的走向只取决于鼠标指针的拖动。

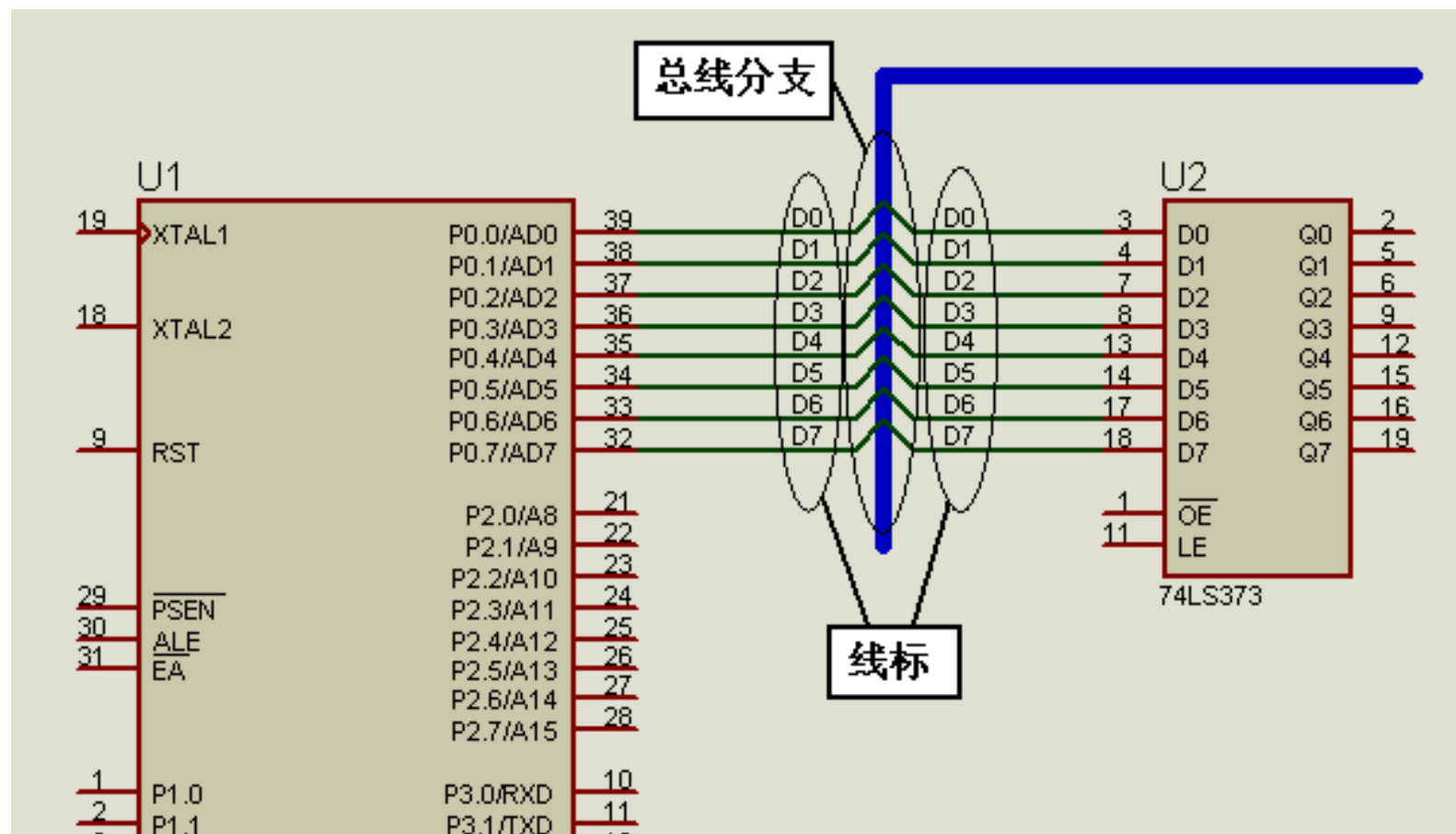


图4-46 总线与总线分支及线标

对于图4-46的总线分支的绘制，先在AT89C51的P0口右侧先画一条总线，然后再画总线分支。在元件模式快捷按钮  按下且自动布线器快捷按钮  松开时，导线可按任意角度走线。

先单击第一个元件的连接点，移动鼠标，在希望的拐点处单击鼠标左键，然后向上移动鼠标，在与总线成 45° 角相交时单击鼠标左键确认，这就完成了一条总线分支的绘制。而其他总线分支的绘制只需在其他总线的起始点双击鼠标左键，不断复制即可。

例如，绘制P0.1引脚至总线的分支，只要把鼠标指针放置在P0.1脚的位置，出现一个红色小方框，双击鼠标左键，自动完成像P0.0引脚到总线的那样连线，这样可依次完成所有总线分支的绘制。在绘制多条平行线时也可采用这种画法。

(5) 放置线标签

从图4-46可看到与总线相连的导线都有线标D0、D1.....D7。

放置线标方法如下：单击工具栏的图 ，再将鼠标移至需要放置线标的导线上单击，即会出现如图4-47所示的“Edit Wire Label”对话框，将线标填入“标号”栏（例如填写“D0”等），点击“确定”按钮即可。与总线相连的导线必须要放置线标，这样连接着相同线标的导线才能够导通。

“Edit Wire Label”对话框除了填入线标外，还有几个选项，设计者根据需要选择即可。

经上述步骤操作，最终画出的“流水灯”电路见图4-48。

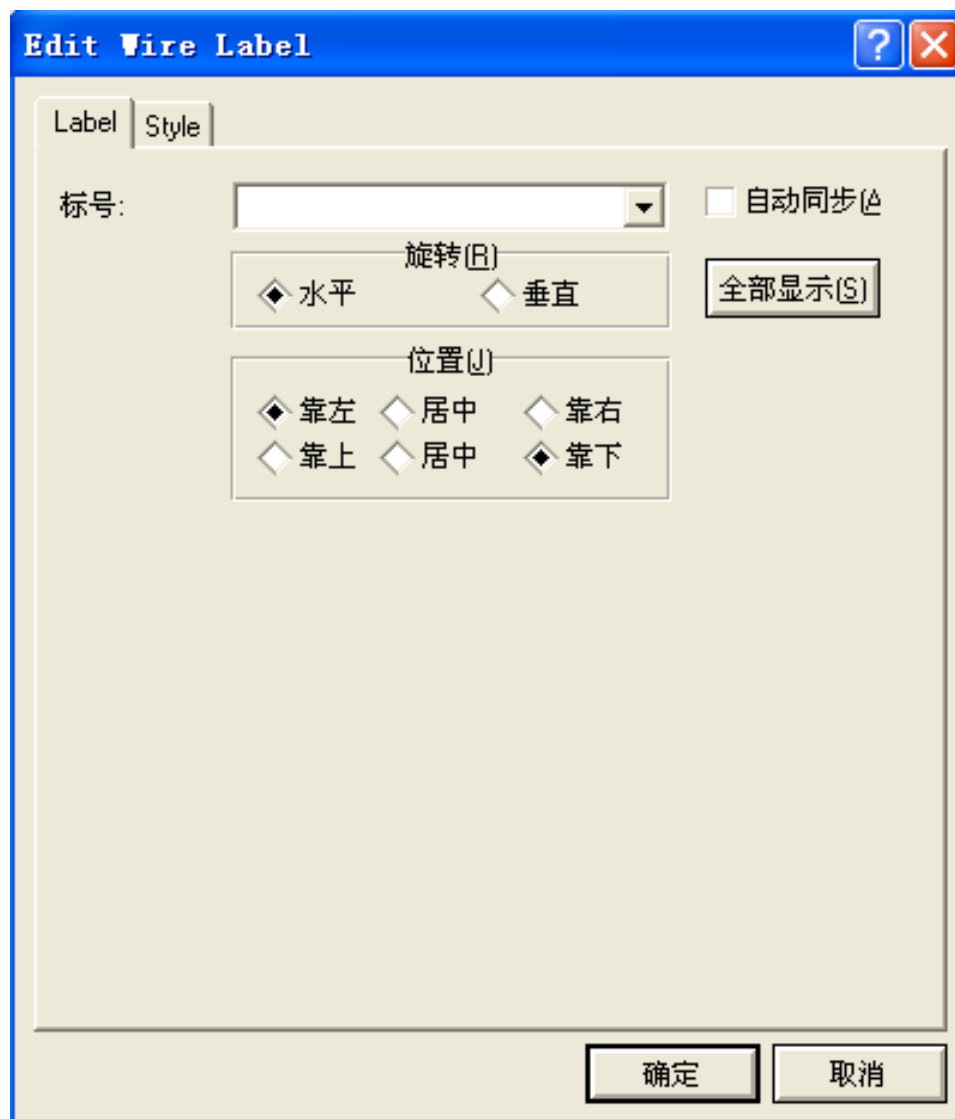


图4-47 “Edit Wire Label” 对话框

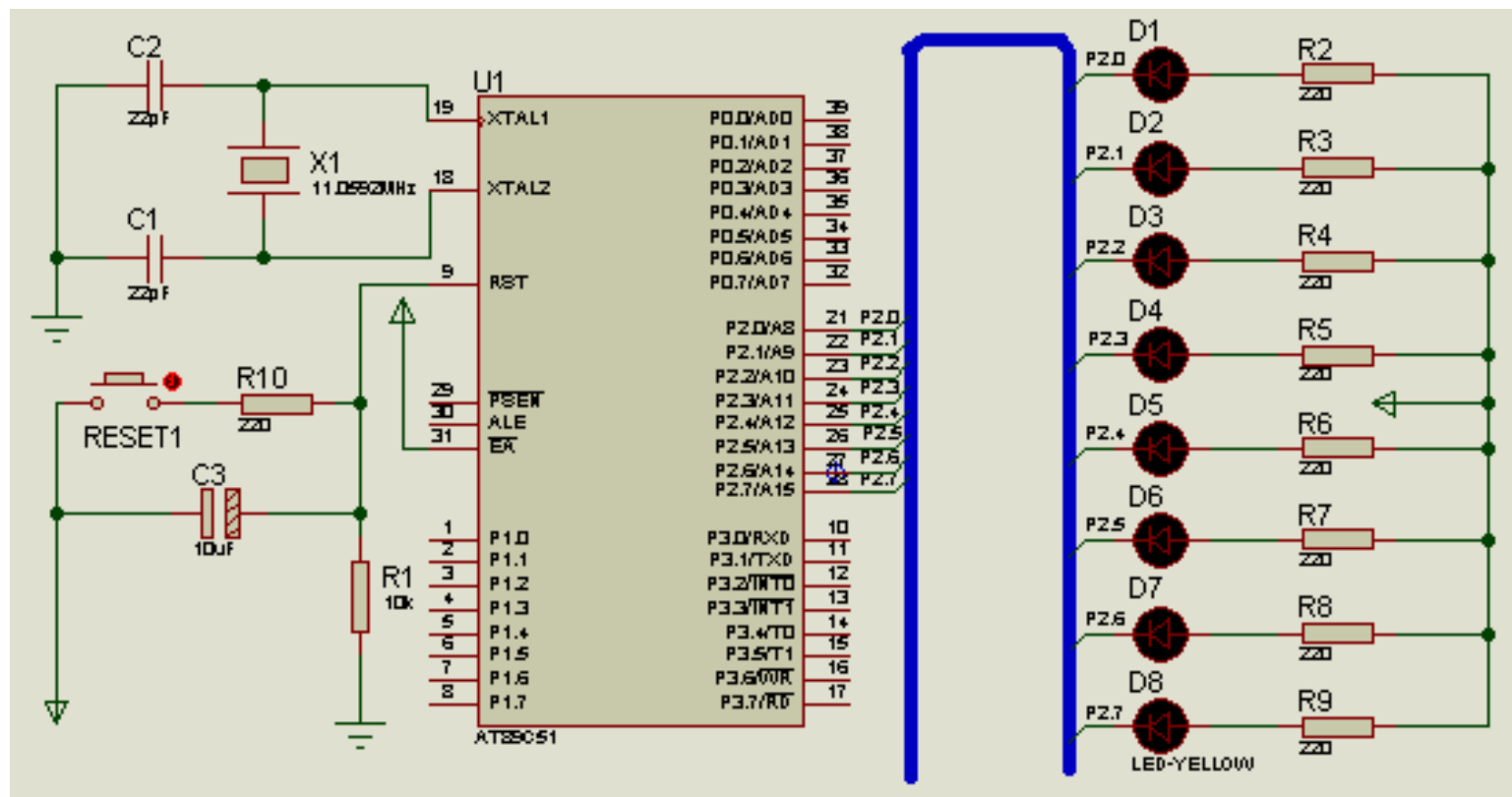


图4-48 “流水灯” 的电路原理图

四、加载目标代码文件、设置时钟频率及仿真运行

1. 加载目标代码文件、设置时钟频率

电路图绘制完成后，在**Proteus**的**ISIS**中双击编辑区中双击原理图中的单片机**AT89C51**，出现图4-49的“编辑元件”窗口。

把在**keil**下生成的“.hex”文件（见4.1.4小节）加载到电路图中的单片机内即可进行仿真了。加载步骤如下：“**Program File**”的对话框中，输入.hex目标代码文件，再在**Clock Frequency**栏中设置**12MHz**，则该虚拟系统以**12MHz**的时钟频率运行。此时，即可回到原理图界面进行仿真了。

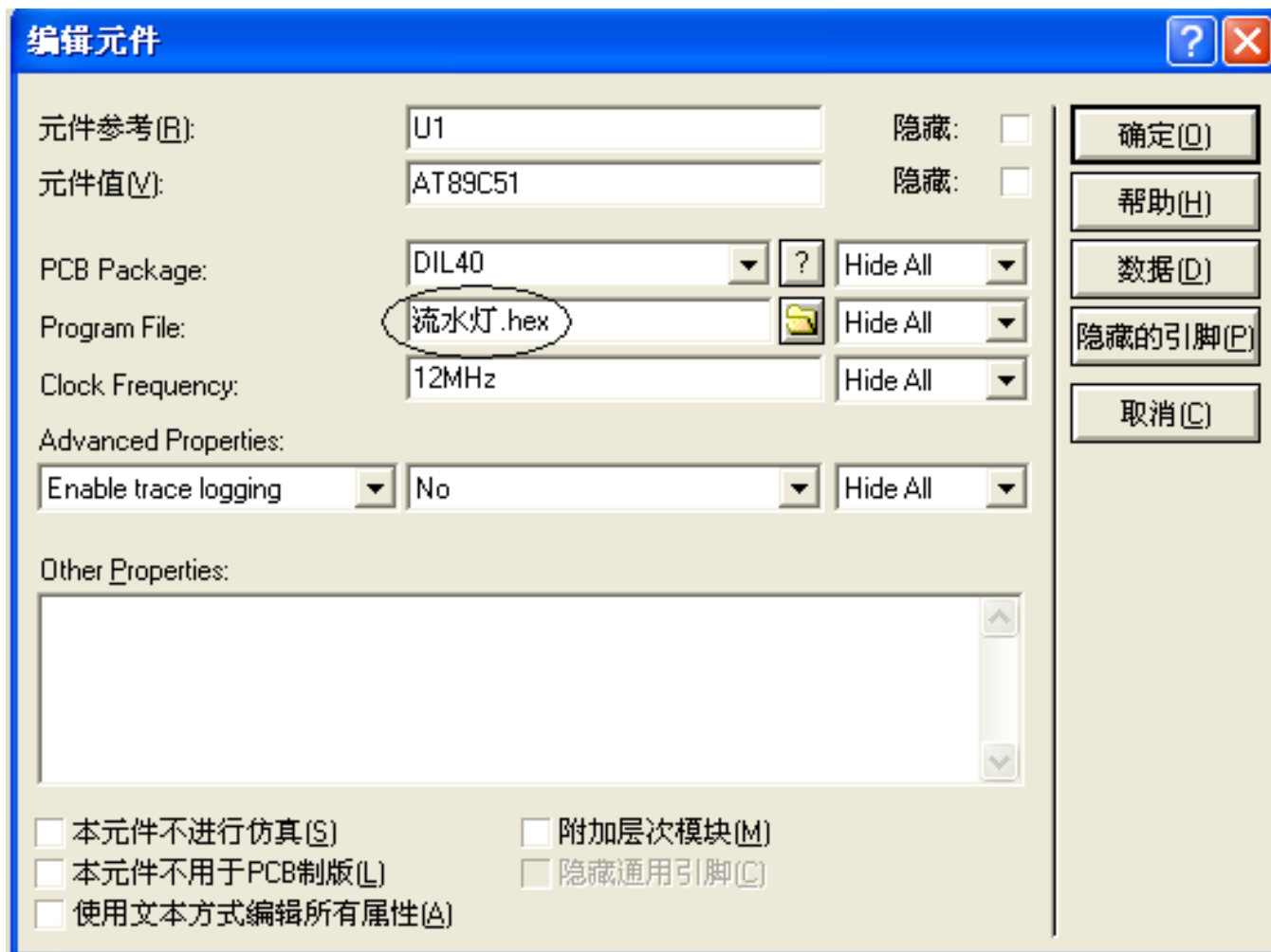


图4-49 加载目标代码文件



加载目标代码时需特别注意，因为运行时钟频率以单片机属性设置中的时钟频率（**Clock Frequency**）为准。

需要注意的是，在**Proteus**中绘制电路原理图时，**8051**单片最小系统所需的**时钟振荡电路、复位电路**，引脚与**+5V**电源的连接均可省略，**Proteus**已经默认，**不影响仿真效果**。

2. 仿真运行

完成上述所有操作后，只需要点击**Proteus ISIS**界面中的快捷命令按钮（**图4-21**左下角）运行程序即可。

这里再重温本章前面介绍的各种仿真运行命令按钮功能：



：运行程序。



：单步运行程序。



：暂停程序。



：停止运行程序。

4.2.5 Proteus与keil C51(keil μ Vision3)的联调

前面介绍如何在**Proteus**下完成原理图的设计文件（设计文件名后缀**.DSN**）后，再把在**keil C51**下编写**C51**程序，经过调试、编译最终生成，“**.hex**”文件后，并把“**.hex**”文件载入虚拟单片机中，然后进行软硬件联调，如果要修改程序，需再回到**keil C51**下修改、再经过调试、编译，重新生成“**.hex**”文件，重复上述过程，直至系统正常运行为止。但是对于较为复杂的程序，如果没有达到预期效果，这时可能需要**Proteus**与**keil C51**进行联合调试。

联调前需要安装**vudgi.exe**文件，该文件可到**Proteus**的官方网站下载。**vudgi.exe**文件安装后，需在**Proteus**与**keil C51**中进行相应设置。

设置时，首先打开**Proteus**需要联调的程序文件，但不要运行，然后选中“调试”菜单中的“使用远程调试监控”选项，使得**keil C51**能与**Proteus**进行通信。

完成上述设置后，在**keil C51**中打开程序项目文件，然后单击菜单“**Project/Options for Target**”选项（或单击工具栏上“**Options for Target**”快捷按钮），打开图4-50的项目对话框。

在**Debug**选项卡中选定右边的“**Use**”及选项“**Proteus VSM Simulator**”。如果**Proteus**与**Keil C51**安装在同一台计算机中，右边“**Setting**”中的**HOST**与**PORT**可保持默认值**127.0.0.1**与**8000**不变，见图4-51。

如果跨计算机调试则需要进行相应的修改。

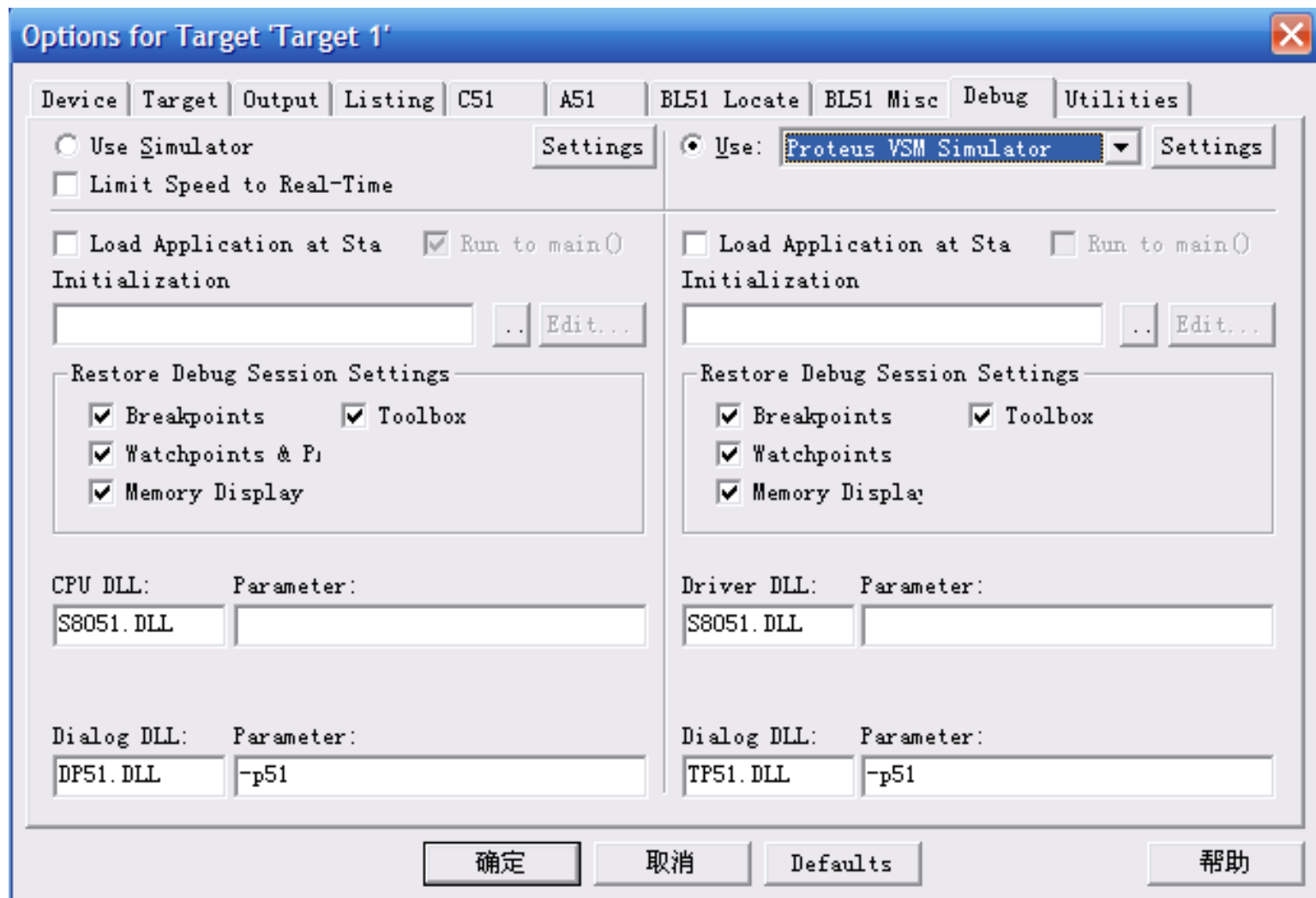


图4-50 项目对话选项框

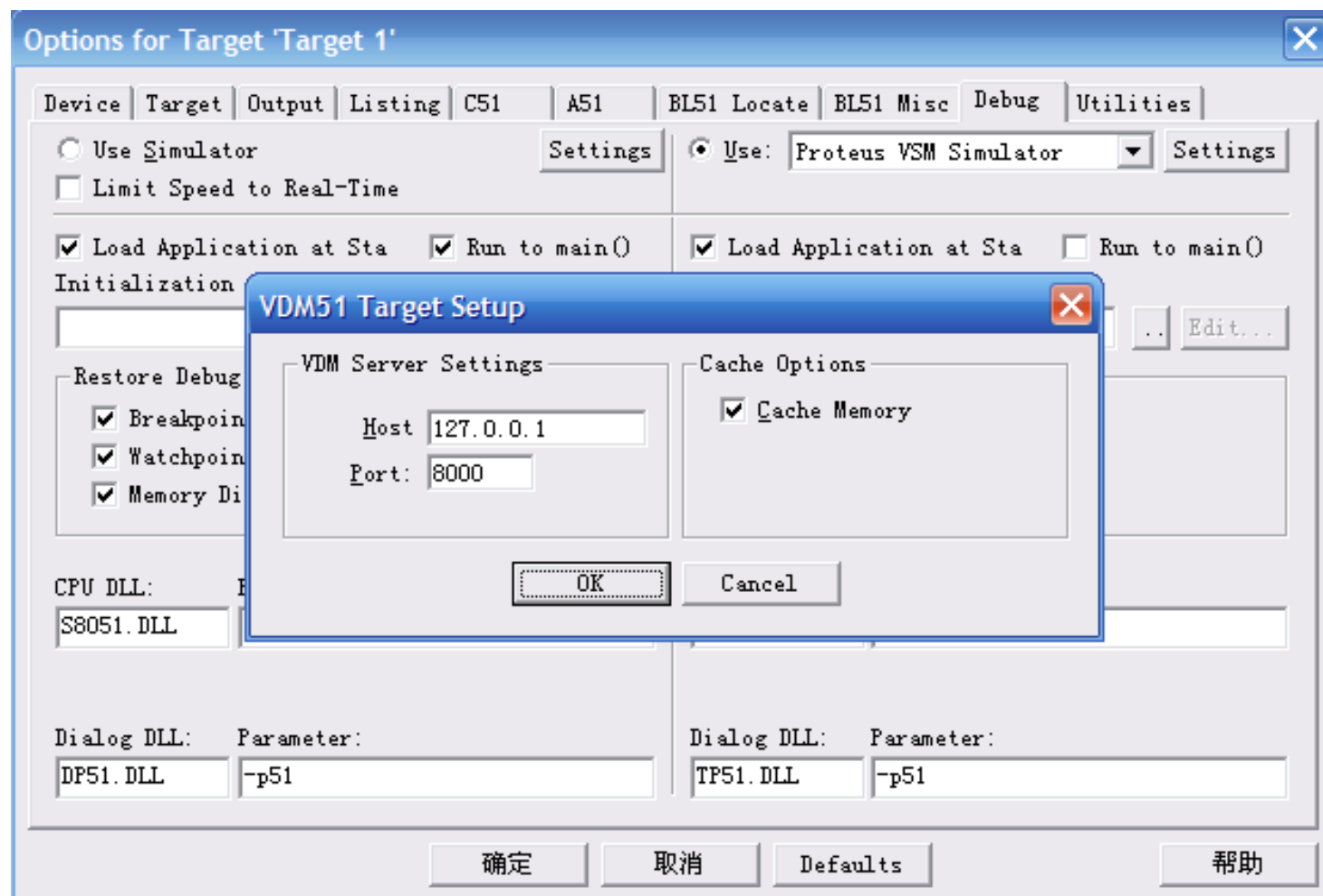


图4-51 项目对选项话框

完成上述设置后，在**keil C51**中全速运行程序时，**Proteus**中的单片机系统也会自动运行，出现的**联调界面**如**图4-52**所示。

左半部分为**keil C51**的调试界面，右半部分是**Proteus ISIS**的界面。如果希望观察运行过程中某些变量的值或者设备状态，需要在**keil C51**中恰当使用各种**Step In/ Step Over/ Step Out/ Run To Cursor Line**及**Breakpoint**进行跟踪，来观察右面的虚拟硬件系统运行的情况。

总之，需要把**keil C51**中的各种调试手段，如单步、跳出、运行到当前行、设置断点等恰当地配合来进行单片机系统运行的软硬件联调。

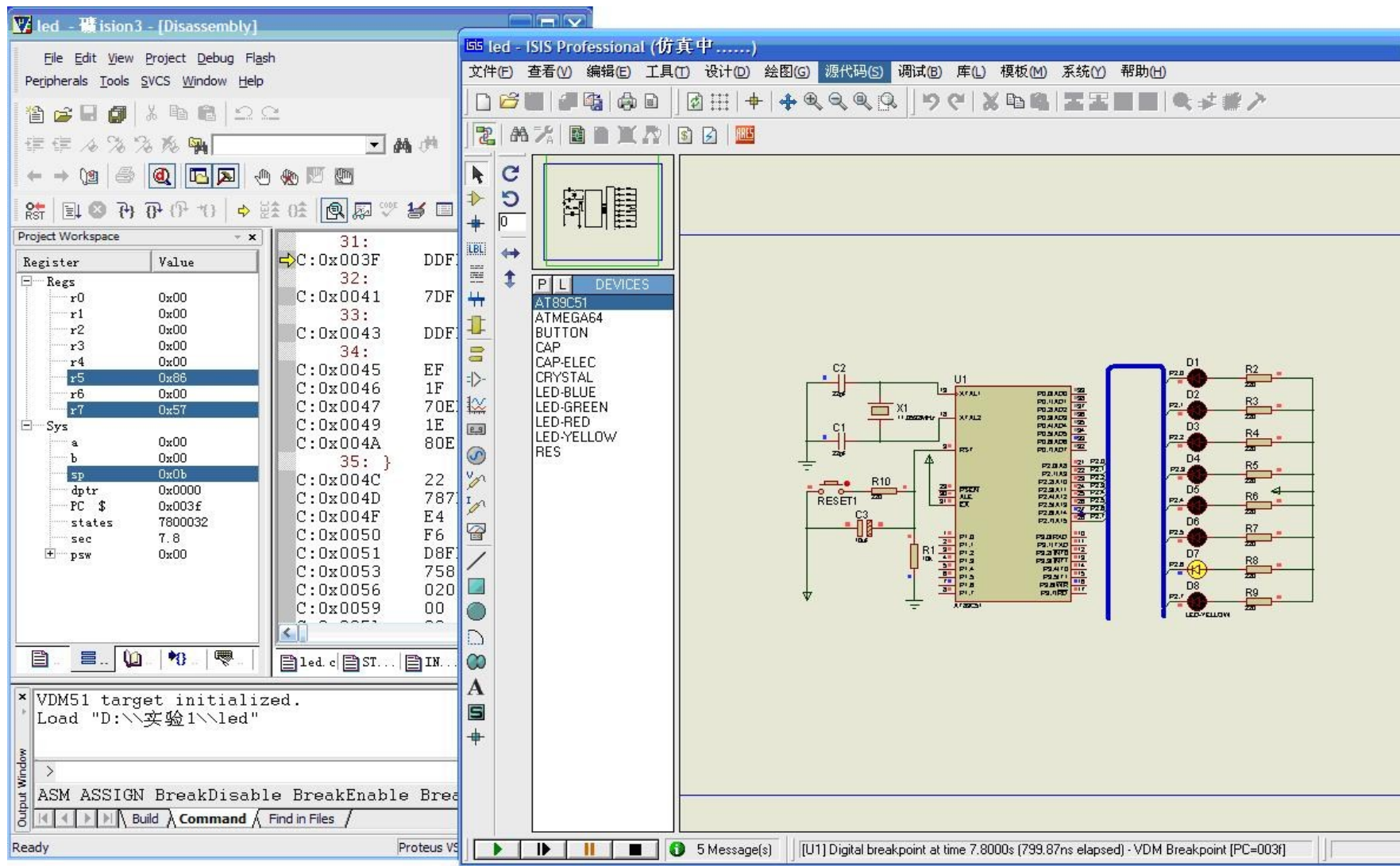


图4-52 联调界面

需说明的是，联调方式不支持需要调试的程序项目的中文名字，因此需将项目文件的中文文件名“流水灯.Uv2”改为英文文件名“led.Uv2”。

需要指出的是，**Lab center Electronics**公司最新推出的**Proteus8.x**版本，自带免费的编译器，目前已有**51、PIC、AVR**系列单片机的汇编软件。

如果是**8051**汇编语言编写的程序，直接使用自带的编译器就很方便，且**Proteus**不需要像**keil C51**那样还要新建项目，设置等。用**Proteus**直接调试很好用，不用上述的与**keil**联调，特别是调试小程序很简单。

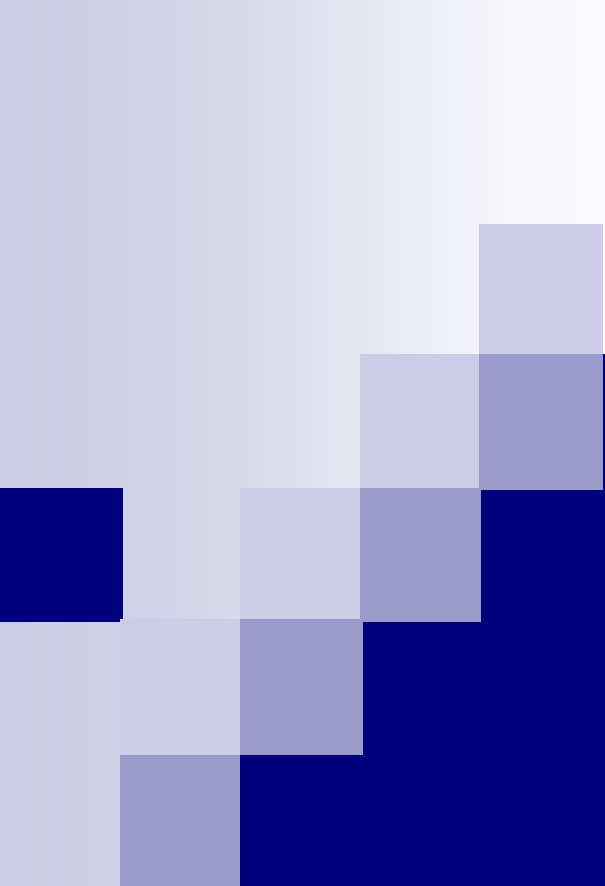
如采用**C51**语言编程，**Proteus8.x**版本可在**Proteus**环境下设定与外部的编译器的关联，设置与**keil C51**的连接（例如使用**Proteus8.2**版本，需在计算机中拷入**Keil μVision4**），这样可直接在**Proteus8.x**下进行源程序的



编辑和编译，此时系统自动生成名为**Debug.hex**的文件，并自动拷入到原理图的单片机中，点击“运行”按钮即可仿真运行，这相当于把图4-52的两个联调界面直接合为一个一体。

该版本不再需要用**Keil**编程后生成**.Hex**文件再手动拷入到原理图中，一切都是自动完成把**Debug.hex**拷入到原理图的单片机中，用户调试使用非常方便，节省很多时间。

这是 **Proteus8.x** 与 **Proteus7.x** 版本的很大区别。读者可尝试 **KeilμVision3** 分别与 **Proteus8.x** 或 **Proteus7.x** 环境下的联调的过程，体验 **Proteus8.x** 环境下的便捷之处。



第5章 单片机与开关、显示器件以及 键盘的接口设计



开关检测、键盘输入与显示是单片机应用系统的基本功能，本章介绍**AT89S52**单片机与常见**显示器件**：**发光二极管**、**LED数码管**、**LED点阵显示器**、**字符型LCD液晶显示器1602**和**点阵式液晶显示器LCD12864**的接口设计，以及与常见的**输入器件**：**开关**、**键盘**的接口设计。

5.1 单片机控制发光二极管显示

发光二极管常用来指示系统工作状态，制作节日彩灯、广告牌匾等。大部分发光二极管工作电流 $1\sim 5\text{mA}$ 之间，其内阻为 $20\sim 100\Omega$ 。电流越大，亮度也越高。

为保证发光二极管正常工作，同时减少功耗，限流电阻选择十分重要，若供电电压为 $+5\text{V}$ ，则限流电阻可选 $1\sim 3\text{k}\Omega$ 。

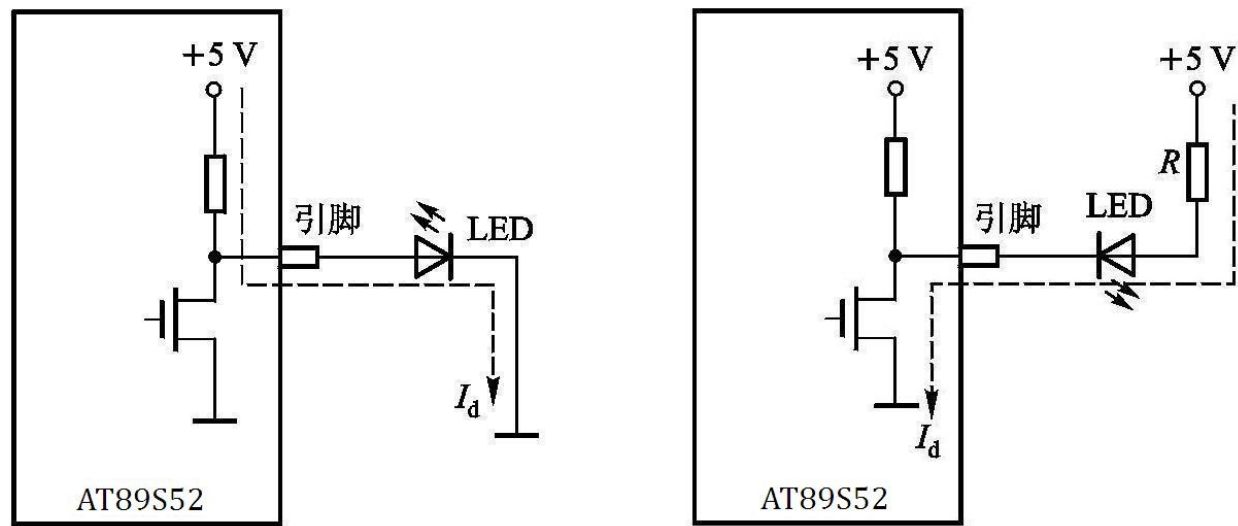
5.1.1 单片机与发光二极管的连接

第2章已介绍，P0口作通用I/O用，由于漏极开路，需外接上拉电阻。而P1~P3口内部有30k Ω 左右上拉电阻。

下面讨论P1~P3口如何与LED发光二极管驱动连接问题。

单片机并行端口P1~P3直接驱动发光二极管，电路见图5-1。

与P1、P2、P3口相比，P0口每位可驱动8个LSTTL输入，而P1~P3口每一位驱动能力，只有P0口一半。



(a) 不恰当的连接：高电平驱动

(b) 恰当的连接：低电平驱动

图5-1 发光二极管与单片机并行口的连接

当P0口某位为高电平时，可提供400 μ A的拉电流；当P0口某位为低电平（0.45V）时，可提供3.2mA的灌电流，而P1~P3口内有30k Ω 左右上拉电阻，如高电平输出，则从P1、P2和P3口输出的拉电流 I_d 仅几百 μ A，驱动能力较弱，亮度较差，见图5-1（a）。

如果端口引脚为低电平，能使灌电流 I_d 从单片机外部流入内部，则将大大增加流过的灌电流值，见图5-1（b）。AT89S52任一端口要想获得较大的驱动能力，要用低电平输出。

如果一定要高电平驱动，可在单片机与发光二极管间加驱动电路，如74LS04、74LS244等。

5.1.2 I/O端口的编程举例

单片机的I/O端口P0~P3是单片机与外设进行信息交换的桥梁，可通过读取I/O端口的状态来了解外设的状态，也可向I/O端口送出命令或数据来控制外设。

对单片机I/O端口进行编程控制时，需要对I/O端口的特殊功能寄存器进行声明，在C51的编译器中，这项声明包含在头文件reg51.h中，编程时，可通过预处理命令#include<reg51.h>，把这个头文件包含进去。下面通过一个例子介绍如何对I/O端口编程实现对发光二极管亮灭的控制。

【例5-1】 制作流水灯，原理电路见图5-2，8个发光二极管LED0～LED7经限流电阻分别接至P1口的P1.0～P1.7引脚上，阳极共同接高电平。编写程序来控制发光二极管由上至下的反复循环流水点亮，每次点亮一个发光二极管。

参考程序：

```
#include <reg51.h>
#include <intrins.h>           //包含移位函数_crol_( )的头文件
#define uchar unsigned char
#define uint unsigned int

void delay(uint i)             //延时函数
{
    uchar t;
    while (i--)
```



```

    {
        for(t=0;t<120;t++);
    }
}

```

```

void main( )

```

//主程序

```

{
    P1=0xfe;
    while (1)
    {
        delay( 500 );
        P1=_crol_(P1,1);
    }
}

```

//向P1口送出点亮数据

//500为延时参数，可根据实际需要调整

// 函数_crol_(P1,1)把P1中的数据循环左移1位

程序说明：

(1) while(1) 两种用法：

“while(1);”：while(1)后有分号，是使程序停留在这指令上；

“while(1) {……;}”：反复循环执行大括号内程序段，本例用法，即控制流水灯反复循环显示。

(2) C51函数库中的循环移位函数：循环移位函数包括：

- 循环左移函数 “_crol_”
- 循环右移函数 “_cror_”。

本例用循环左移 “_crol_(P1, 1)”，函数。括号第1个参数为循环左移对象，即对P1中的内容循环左移；第2个参数为左移位数，即左移1位。编程中一定要把含有移位函数的头文件intrinsic.h包含在内，例如第2行“#include <intrinsic.h>”。

在【例5-1】基础上，编写控制发光二极管反复循环点亮的流水灯。

【例5-2】电路仍采用图5-2，制作由上至下再由下至上反复循环点亮显示的流水灯，3种方法实现。

(1) 数组的字节操作实现

建立1个字符型数组，将控制8个LED显示的8位数据作为数组元素，依次送P1口。参考程序：

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
uchar tab[ ]={ 0xfe , 0xfd , 0xfb , 0xf7 , 0xef , 0xdf , 0xbf , 0x7f , 0x7f ,  
               0xbf , 0xdf , 0xef , 0xf7 , 0xfb , 0xfd , 0xfe };
```

```
/*前8个数据为左移点亮数据，后8个为右移点亮数据*/
```

```
void delay( )
```

```
{  
    uchar i,j;  
    for(i=0; i<255; i++)  
        for(j=0; j<255; j++);  
}
```

```
void main( )
```

//主函数

```
{  
    uchar i;  
    while (1)  
    {  
        for(i=0;i<16; i++)  
        {  
            P1=tab[i];  
            delay( );  
        }  
    }  
}
```

//向P1口送出点亮数据
//延时，即点亮一段时间

(2) 移位运算符实现

使用移位运算符“>>”、“<<”，把送P1口显示控制数据进行移位，从而实现发光二极管依次点亮。参考程序：

```
#include <reg51.h>
#define uchar unsigned char
void delay( )
{
    uchar i,j;
    for(i=0; i<255; i++)
        for(j=0; j<255; j++);
}

void main( )                                //主函数
{
    uchar i,temp;
    while (1)
```

{

temp=0x01;

//左移初值赋给temp

for(i=0; i<8; i++)

{

P1=~temp;

// temp中的数据取反后送P1口

delay();

// 延时

temp=temp<<1;

// temp 中数据左移一位

}

temp=0x80;

// 赋右移初值给temp

for(i=0; i<8; i++)

{

P1=~temp;

// temp中的数据取反后送P1口

delay();

// 延时

temp=temp>>1;

// temp 中数据右移一位

}

}

}

程序说明： 注意使用移位运算符“>>”、“<<”与使用循环左移函数“_crol_”和循环右移函数“_cror_”区别。左移移位运算“<<”是将高位丢弃，低位补0；右移移位运算、“>>”是将低位丢弃，高位补0。而循环左移函数“_crol_”是将移出的高位再补到低位，即循环移位；同理循环右移函数“_cror_”是将移出的低位再补到高位。

（3）用循环左、右移位函数实现

使用C51提供的库函数，即循环左移 n 位函数和循环右移 n 位函数，控制发光二极管点亮。参考程序：

```
#include <reg51.h>
```

```
#include <intrins.h>
```

```
//包含循环左、右移位函数的头文件
```

```
#define uchar unsigned char
```



```
void delay( )
```

```
{  
    uchar i,j;  
    for(i=0; i<255; i++)  
        for(j=0; j<255; j++);  
}
```

```
void main( )
```

```
{  
    uchar i,temp;  
    while (1)  
    {  
        temp=0xfe;  
        for(i=0; i<7; i++)
```

// 主函数

// 初值为11111110

```

{
    P1=temp;                // temp中的点亮数据送P1口，控制点亮显示
    delay( );               // 延时
    temp=_crol_( temp,1) ;  // temp 数据循环左移1位
}
for(i=0; i<7; i++)
{
    P1=temp;                // temp中的数据送P1口输出
    delay( );               // 延时
    temp=_cror_( temp,1) ;  //temp中数据循环右移1位
}
}

```

5.2 开关状态检测

下面介绍如何检测一个开关处于闭合状态还是打开状态，只需将被检测的开关一端接到I/O端口的引脚上，另一端接地，通过读入I/O端口的电平来判断开关是闭合状态还是处于打开状态。如果为低电平，则开关为闭合状态。

5.2.1 开关检测案例1

用I/O端口来进行开关状态检测，开关一端接到I/O端口引脚上，并通过上拉电阻接+5V上，开关另一端接地，当开关打开时，I/O引脚为高电平，当开关闭合时，I/O引脚为低电平。

【例5-3】 如图5-3，单片机的P1.4~P1.7接4个开关S0~S3，P1.0~P1.3接4个发光二极管LED0~LED3。

编程将P1.4~P1.7上的4个开关状态反映在P1.0~P1.3引脚控制的4个发光二极管上，开关闭合，对应发光二极管点亮。

例如P1.4引脚上开关S0状态，由P1.0脚上LED0显示，P1.6引脚上开关S2状态，由P1.2脚的LED2显示。

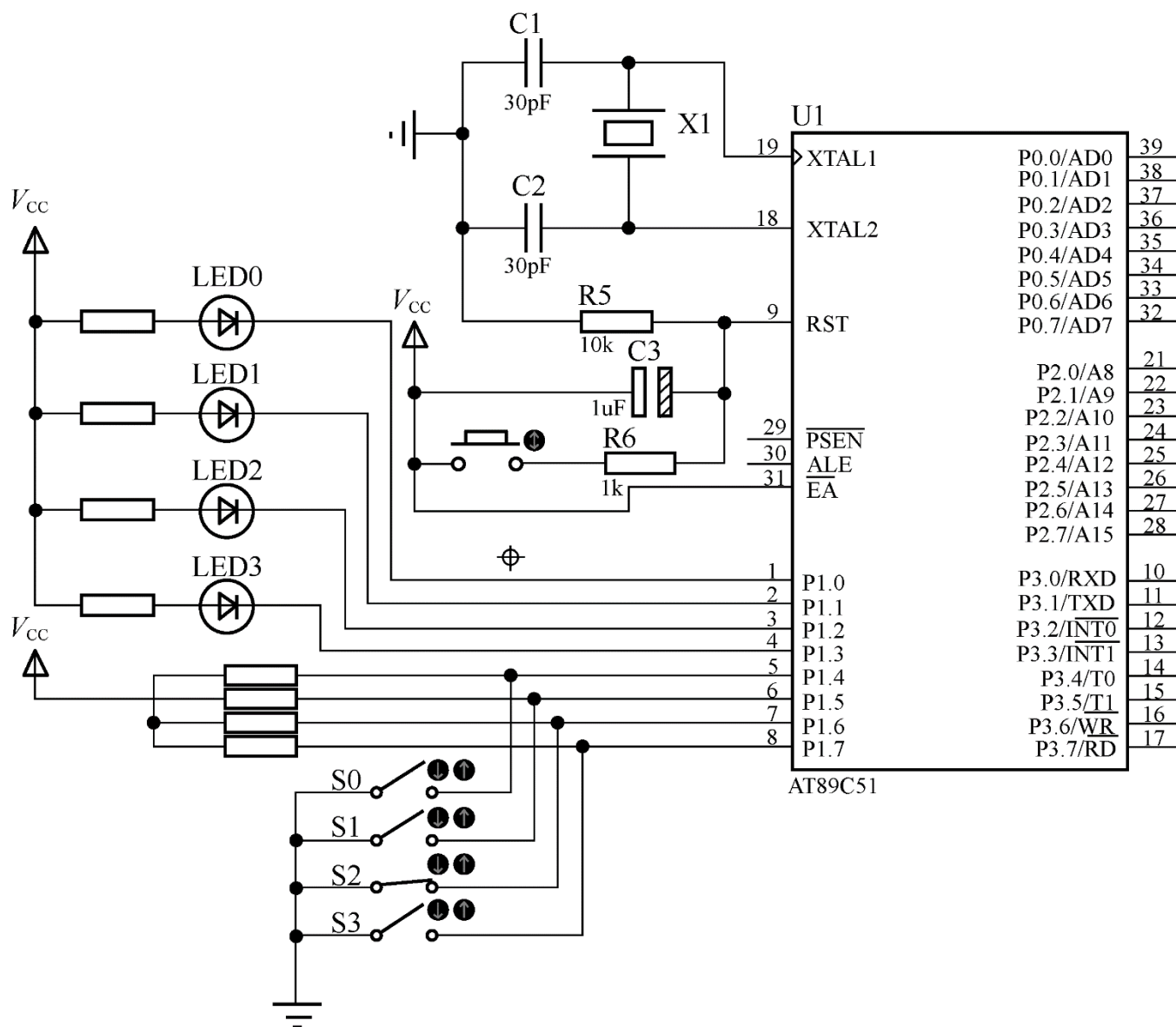


图5-3 开关、LED发光二极管与P1口的连接

参考程序如下：

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
void delay( ) //延时函数
```

```
{
```

```
    uchar i, j;
```

```
    for(i=0; i<255; i++)
```

```
        for(j=0; j<255; j++);
```

```
}
```

```
void main( ) //主函数
```

```
{
```

```
    while (1)
```

```
    {
```

```
        unsigned char temp; //定义临时变量temp
```

```
        P1=0xff; //P1口低4位置1，作为输入;高4位置1，发光二极管熄灭
```

```
        temp=P1&0xf0; //读P1口并屏蔽低4位，送入temp 中
```

```
        temp=temp>>4; //temp内容右移4位，P1口高4位移至低4位
```

```
P1=temp;                // temp中的数据送P1口输出
delay( );
}
```

5.2.2 开关检测案例2

【例5-4】 如图5-4，P1.0和P1.1引脚接有两只开关S0和S1，两引脚上的高低电平共4种组合，4种组合分别点亮P2.0～P2.3引脚控制的4只LED，即S0、S1均闭合，LED0亮，其余灭；S1闭合、S0打开，LED1亮，其余灭；S0闭合、S1打开，LED2亮，其余灭；S0、S1均打开，LED3亮，其余灭。编程实现此功能。

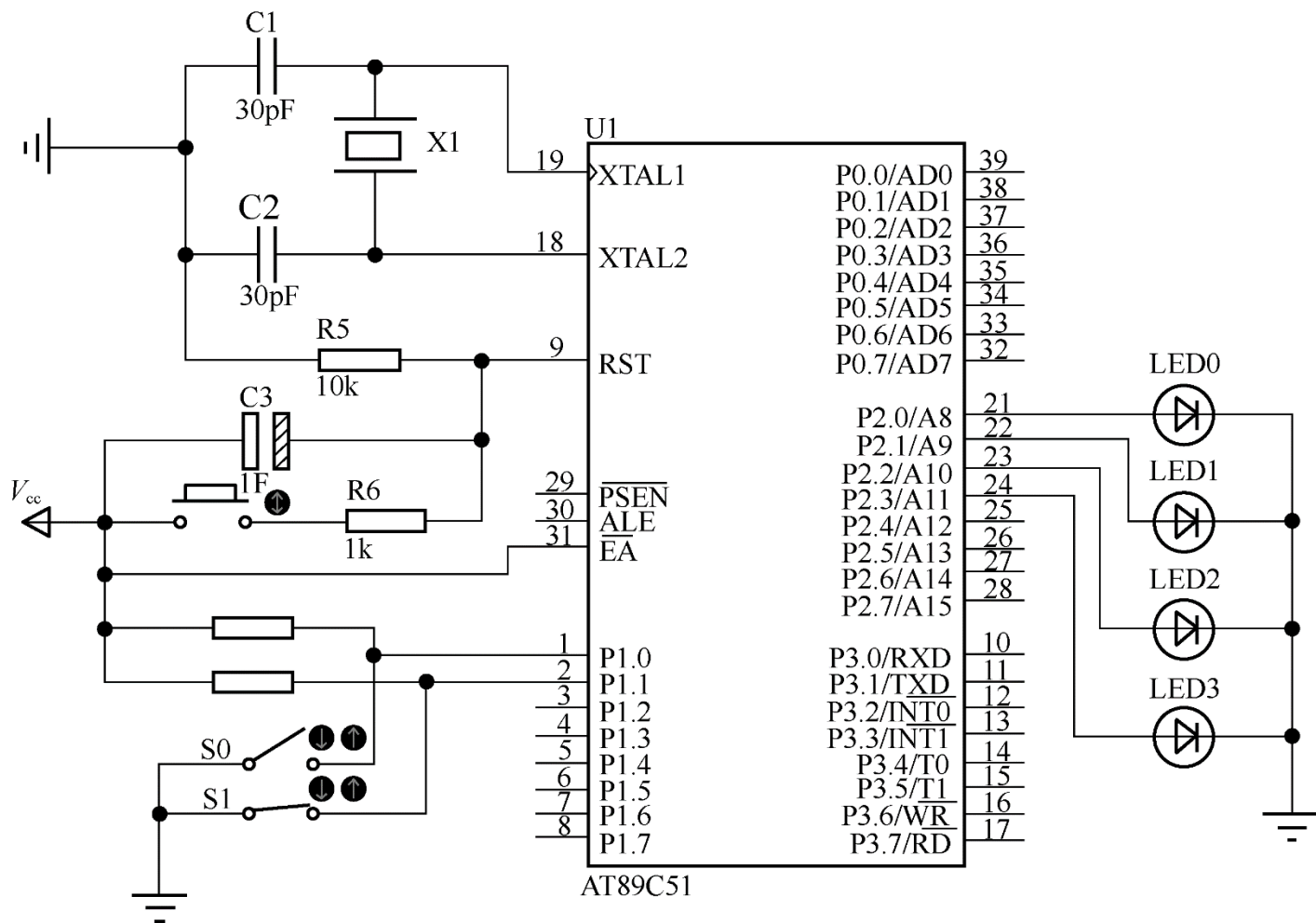


图5-4 开关检测指示器2接口电路与仿真

参考程序:

```
#include <reg51.h>
```

```
void main( )
```

```
{
```

```
    char state;
```

```
    do
```

```
    {
```

```
        P1=0xff;
```

```
        state=P1;
```

```
        state=state&0x03;
```

```
        switch (state)
```

```
        {
```

```
            case 0: P2=0x01; break;
```

```
            case 1: P2=0x02; break;
```

```
            case 2: P2=0x04; break;
```

```
            case 3: P2=0x08; break;
```

```
        }
```

```
    }while ( 1 );
```

```
}
```

```
// 包含头文件reg51.h
```

```
//主函数main()
```

```
// P1口为输入
```

```
// 读入P1口的状态，送入state
```

```
// 屏蔽P1口的高6位
```

```
// 判P1口低2位开关状态
```

```
// 点亮P2.0脚LED
```

```
// 点亮P2.1脚LED
```

```
// 点亮P2.2脚LED
```

```
// 点亮P2.3脚LED
```

程序段中用到循环结构控制语句do-while以及switch-case语句。

5.3 单片机控制LED数码管的显示

5.3.1 LED数码管显示原理

LED数码管：“8”字型，**7段**（不包括小数点）或**8段**（包括小数点），每段对应一个发光二极管，共阳极和共阴极两种，见图5-5。**共阳极**数码管的阳极连接在一起，接+5V；**共阴极**数码管阴极连在一起接地。

对于**共阴极**数码管，当某发光二极管阳极为高电平时，发光二极管点亮，相应段被显示。同样，**共阳极**数码管阳极连在一起，公共阳极接+5V，当某个发光二极管阴极接低电平时，该发光二极管被点亮，相应段被显示。

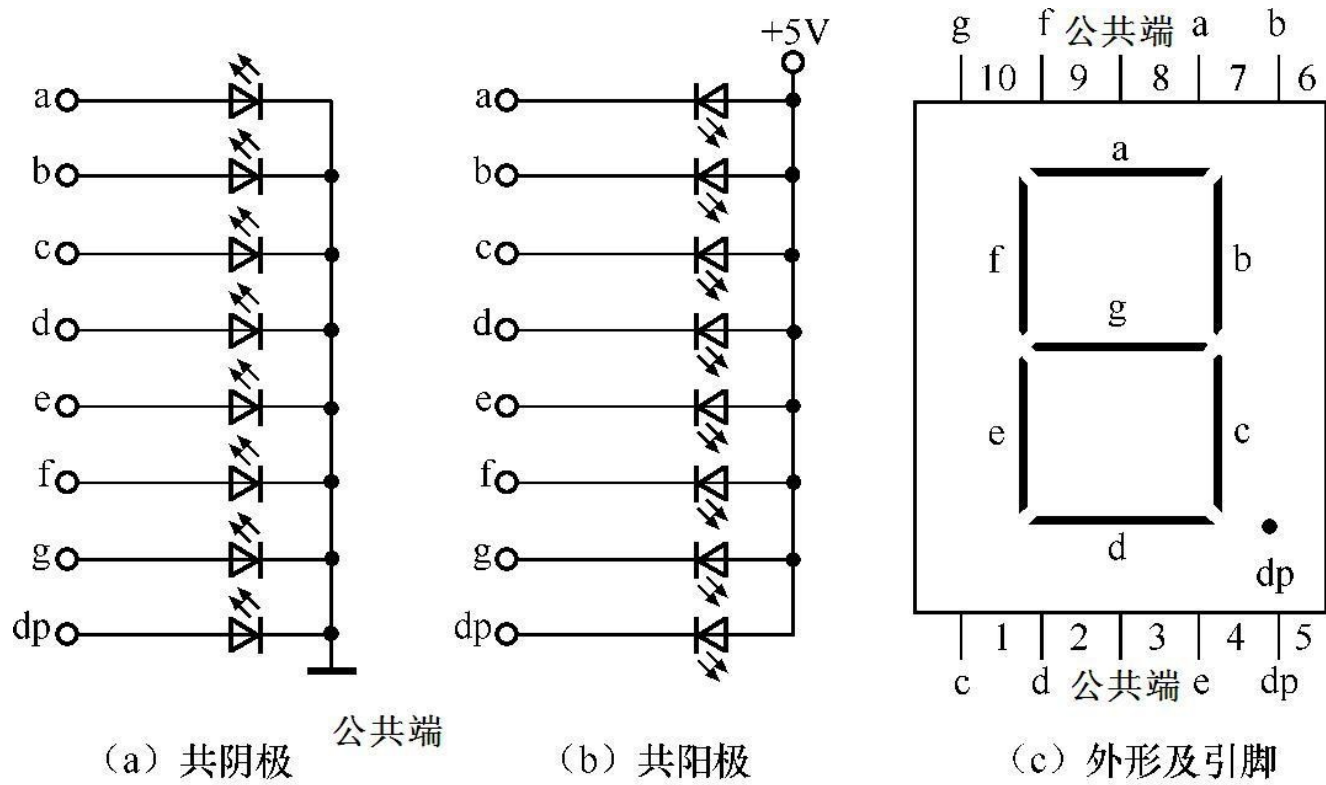


图5-5 8段LED数码管结构及外形

为使LED数码管显示不同字符，要把某些段点亮，就要为数码管各段提供一字节的二进制码，即**段码**。习惯上以“a”段对应段码字节的最低位。各字符**段码**见表5-1。

表5-1 LED数码管的段码

显示字符	共阴极段码	共阳极段码	显示字符	共阴极段码	共阳极段码
0	3FH	C0H	C	39H	C6H
1	06H	F9H	d	5EH	A1H
2	5BH	A4H	E	79H	86H
3	4FH	B0H	F	71H	8EH
4	66H	99H	P	73H	8CH
5	6DH	92H	U	3EH	C1H
5	7DH	82H	T	31H	CEH
7	07H	F8H	y	6EH	91H
8	7FH	80H	H	76H	89H
9	6FH	90H	L	38H	C7H
A	77H	88H	“灭”	00H	FFH
b	7CH	83H	...	...	...

如要在数码管显示某字符，只需将该字符的段码加到各段上即可。

例如某存储单元中的数为“02H”，想在共阳极数码管上显示“2”，需要把“2”的段码“A4H”加到数码管各段。将欲显示字符的段码作成一张表（数组），根据显示字符从表中查找到相应段码，然后把该字型码输出到数码管各个段上，同时数码管的公共端接+5V，此时在数码管上显示字符“2”。

下面介绍单片机如何控制LED数码管显示字符。

【例5-5】利用单片机控制一个8段LED数码管先循环显示单个偶数：0、2、4、6、8，再显示单个奇数：1、3、5、7、9，如此反复循环显示。

本例原理电路及仿真结果，见[图5-6](#)。

参考程序如下：

```
#include "reg51.h"
```

```
#include "intrins.h"
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
#define out P0
```

```
uchar code seg[]={0xc0,0xa4,0x99,0x82,0x80,0xf9,0xb0,0x92,0xf8,0x90,0x01};
```

```
    //共阳极段码表
```

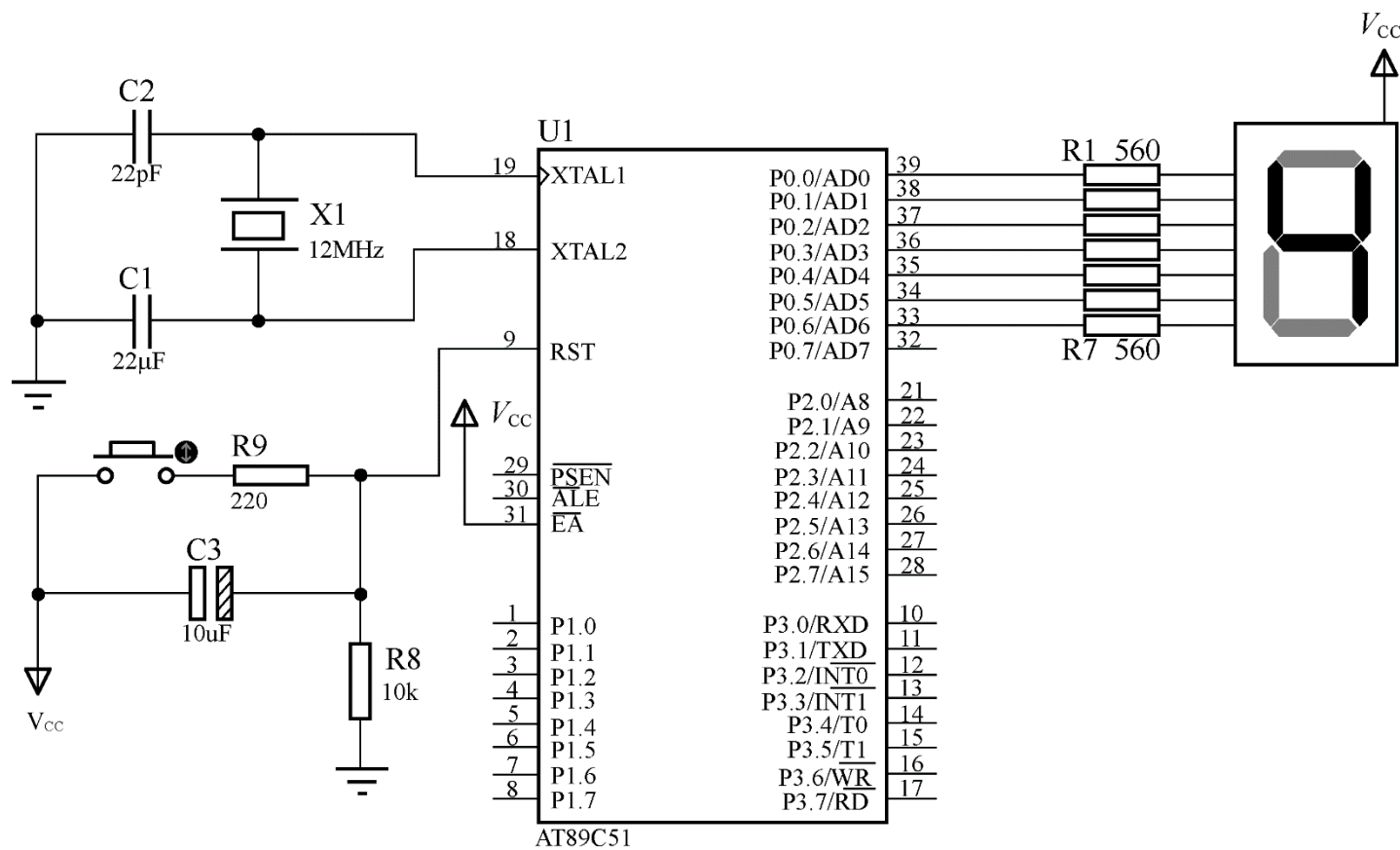


图5-6 控制数码管循环显示单个数字的电路及仿真



```
void delayms(uint);
```

```
void main(void)
```

```
{
```

```
uchar i;
```

```
while(1)
```

```
{
```

```
    out=seg[i];
```

```
    delayms(900);
```

```
    i++;
```

```
    if(seg[i]==0x01)i=0; // 如段码为0x01，表明一个循环显示已结束
```

```
}
```



```
}  
void delayms(uint j)                                // 延时函数  
{  
    uchar i;  
    for(;j>0;j--)  
    {  
        i=250;  
        while(--i);  
        i=249;  
        while(--i);  
    }  
}
```

说明：语句 “**if(seg[i]==0x01)i=0;**” 含义：如果欲送出的数组元素为 **0x01**（数字“9”段码**0x90**的下一个元素，即结束码），表明一个循环显示已结束，则**i=0**，则重新开始循环显示，从段码数组表的第一个元素

seg[0]，即段码**0xc0**（数字**0**）重新开始显示。

5.3.2 LED数码管的静态显示与动态显示

两种显示方式：静态显示和动态显示。

1. 静态显示方式

无论多少位**LED**数码管，都同时处于显示状态。

多位**LED**数码管工作于静态显示方式时，各位共阴极（或共阳极）连接在一起并接地（或接**+5V**）；每位数码管段码线（**a~dp**）分别与一个**8**位**I/O**口锁存器输出相连。如果送往各个**LED**数码管所显示字符的段码一经

确定，则相应I/O口锁存器锁存的段码输出将维持不变，直到送入下一个显示字符段码。静态显示方式**显示无闪烁，亮度较高**，软件控制较易。

图5-7为4位LED数码管静态显示电路，各数码管可独立显示，只要向控制各位I/O口锁存器送相应显示段码，该位就能保持相应的显示字符。

这样在同一时间，每一位显示的字符可各不相同。静态显示方式**占用I/O口端口线较多**。**图5-7**电路，**要占用4个8位I/O口（或锁存器）**。如数码管数目增多，则需增加I/O口数目。

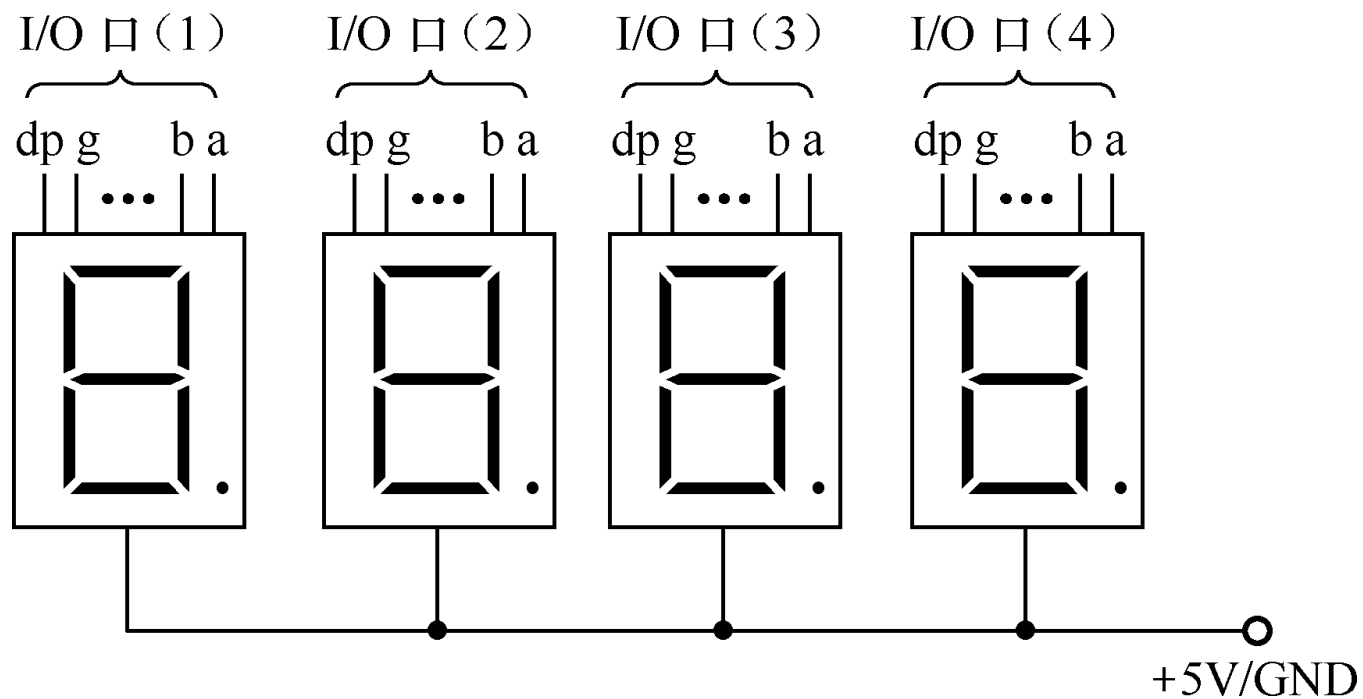


图5-7 4位LED静态显示的示意图

【例5-6】单片机控制2只数码管，静态显示2个数字“27”。原理电路见图5-8。

单片机用P0口与P1口，分别控制加到两个数码管DS0与DS1的段码，而共阳极数码管DS0与DS1的公共端（公共阳极端）直接接至+5V，因此数码管DS0与DS1始终处于导通状态。利用P0口与P1口带有的锁存功能，只需向单片机P0口与P1口分别写入相应的显示字符“2”和“7”的段码即可。

由于一个数码管就占用一个I/O端口。如果数码管数目增多，则需增加I/O口，但软件编程要简单的多。

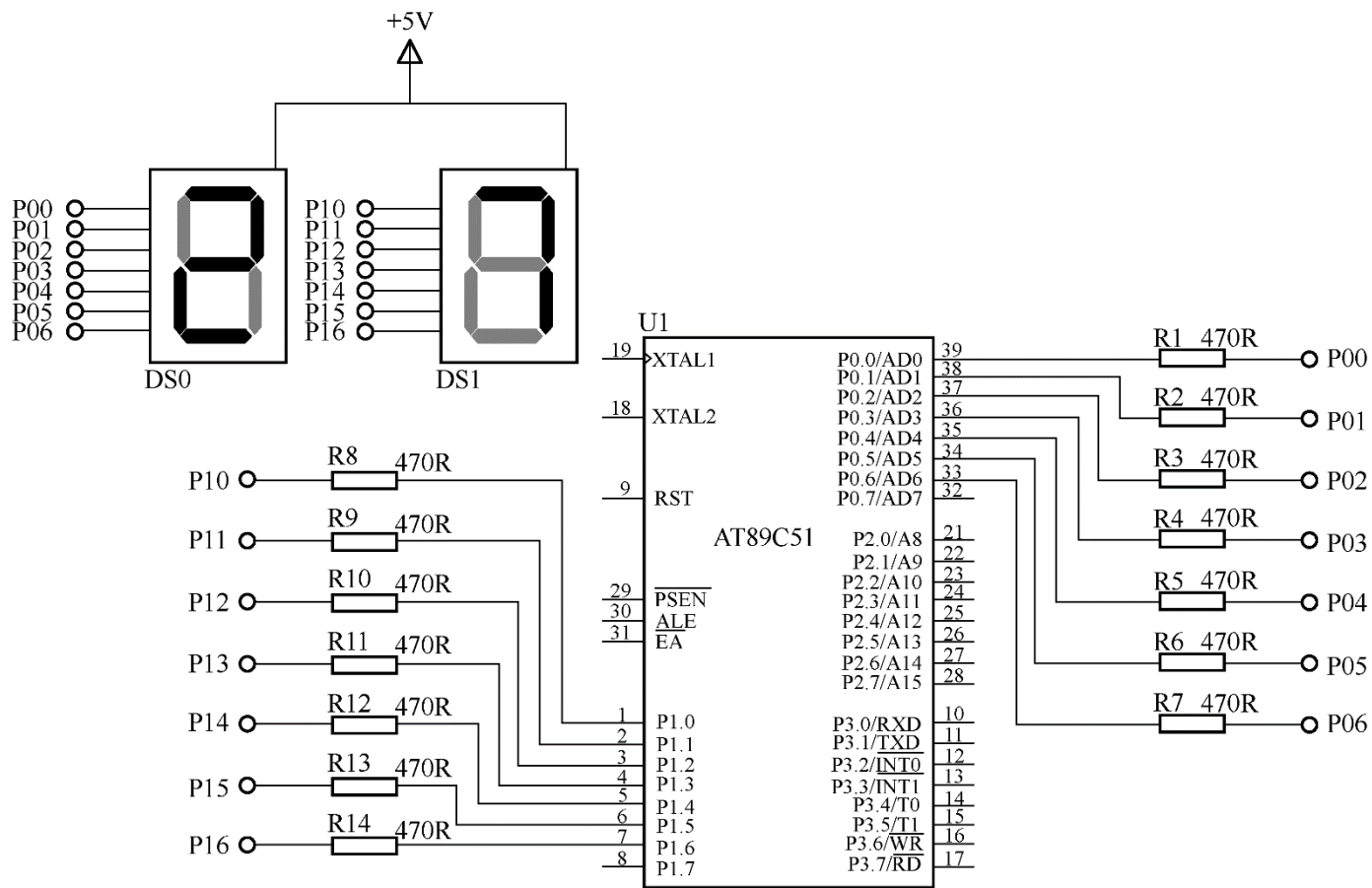


图5-8 2位数码管静态显示的原理电路与仿真

参考程序如下：

```
#include<reg51.h>           //包含8051单片机寄存器定义的头文件
void main(void)
{
    P0=0xa4;                 //将数字“2”的段码送P0口
    P1=0xf8;                 //将数字“7”的段码送P1口
    while(1)                 //无限循环
    ;
}
```

2. 动态显示方式

显示位数较多时，静态显示所占的I/O口多，这时常采用动态显示。为节省I/O口，通常将所有显示器段码线相应段并联在一起，由一个8位I/O口控制，各显示位公共端分别由另一单独I/O口线控制。

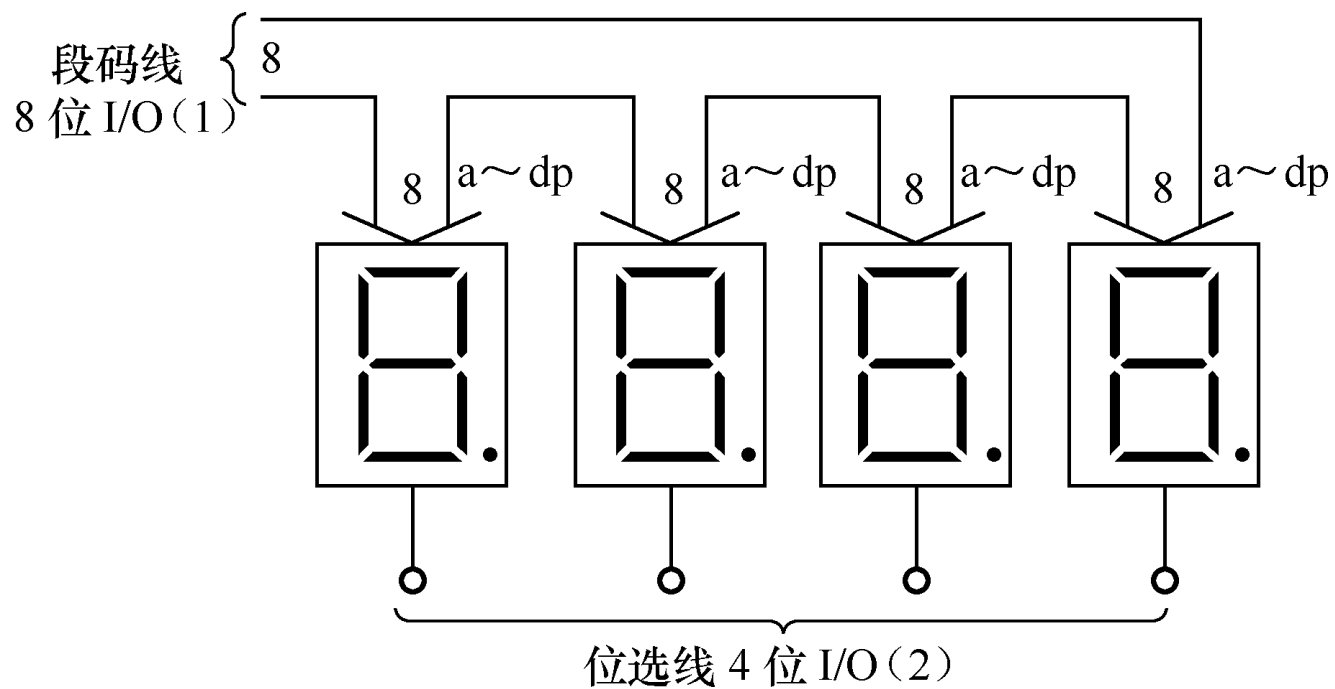


图5-9 4位LED数码管动态显示示意图

图5-9：4位8段LED动态显示器电路示意图。其中单片机发出的段码占用1个8位I/O（1）端口，而位选控制使用I/O（2）端口中4位口线。

动态显示就是单片机向段码线输出欲显示字符的段码。**每一时刻，只有1位位选线有效**，即选中某一位显示，其他各位位选线都无效。每隔一定时间逐位轮流点亮各数码管（扫描方式），由于数码管余辉和人眼的“视觉暂留”作用，只要控制好每位数码管显示时间和间隔，则可造成“多位同时亮”的假象，达到同时显示效果。

各位数码管轮流点亮的时间间隔（扫描间隔）应根据实际情况定。发光二极管从导通到发光有一定的延时，如果点亮时间太短，发光太弱，

人眼无法看清；时间太长，产生闪烁现象，且此时间越长，占用单片机时间也越多。另外，显示位数增多，也将占用单片机大量时间，因此**动态显示实质是以执行程序时间来换取I/O端口减少**。下面是动态显示实例。

【例5-7】 8只数码管，分别**滚动显示单个数字1~8**。程序运行后，单片机控制左边第1个数码管显示1，其他不显示，延时之后，控制左边第2个数码管显示1，其他不显示，直至第8个数码管显示8，其他不显示，反复循环上述过程。

动态显示电路见图5-10，P0口输出段码，P2口输出扫描的位控码，通过8个NPN晶体管的位驱动电路对8个数码管位控扫描。

由于数码管余辉和人眼“视觉暂留”作用，只要控制好每位数码管显示的时间和间隔，则可造成“多位同时亮”假象，达到同时显示效果。

本例使我们了解动态扫描显示实际过程。所以在对实际的动态扫描硬件电路调试时，需要仔细调节软件扫描的频率，以及每一位点亮的时间，以使“多位同时点亮”的效果达到最佳。

参考程序如下：

```
#include<reg51.h>
```

```
#include<intrins.h>
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
uchar code
```

```
dis_code[]={0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90,0x88,0xc0};
```

//共阳数码管段码表

```
void delay(uint t)
```

//延时函数

```
{
```

```
    uchar i;
```

```
    while(t-->0) for(i=0;i<200;i++);
```

```
}
```

```
void main()
```

```
{
```

```
    uchar i,j=0x80;
```

```
while(1)
{
    for(i=0;i<8;i++)
    {
        j=_crol_(j,1); //_crol_(j,1)为将对象j循环左移1位
        P0=dis_code[i];      //P0口输出段码
        P2=j;                //P2口输出位控码
        delay(180);          //延时，控制每位显示的时间
    }
}
}
```

5.4 单片机控制LED点阵显示器显示

LED点阵显示器应用非常广泛，在许多公共场合，如商场、银行、车

站、机场、医院随处可见。不仅能显示文字、图形，还能播放动画、图像、视频等信号。

LED点阵显示器分为**图文显示器**和**视频显示器**，有单色显示，还有彩色显示。下面仅介绍单片机**如何来控制单色LED点阵显示器**的显示。

5.4.1 LED点阵显示器结构与显示原理

由若干个发光二极管按矩阵方式排列而成。阵列点数可分为 **5×7** 、 **5×8** 、 **6×8** 、 **8×8 点阵**；按发光颜色可分为**单色**、**双色**、**三色**；按极性排列可分为**共阴极**和**共阳极**。

1. LED点阵结构

以 8×8 LED点阵显示器为例，外形见图5-11，内部结构见图5-12，由64个发光二极管组成，且每个发光二极管是处于行线（R0~R7）和列线（C0~C7）之间交叉点上。

2. LED点阵显示原理

显示的字符由一个个点亮的LED所构成。

由图5-12点亮点阵中一个发光二极管条件：对应行为高电平，对应列为低电平。如在很短时间内依次点亮很多个发光二极管，LED点阵就可显示一个稳定字符、数字或其他图形。控制LED点阵显示器显示，实质就是控

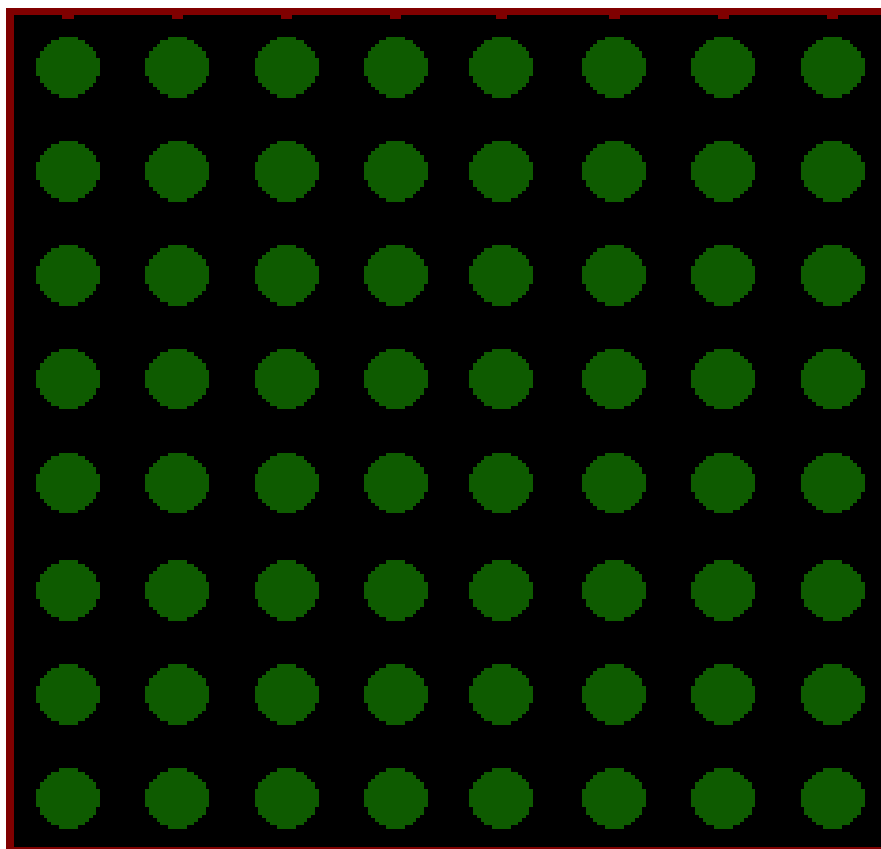


图5-11 8×8 LED点阵显示器外形

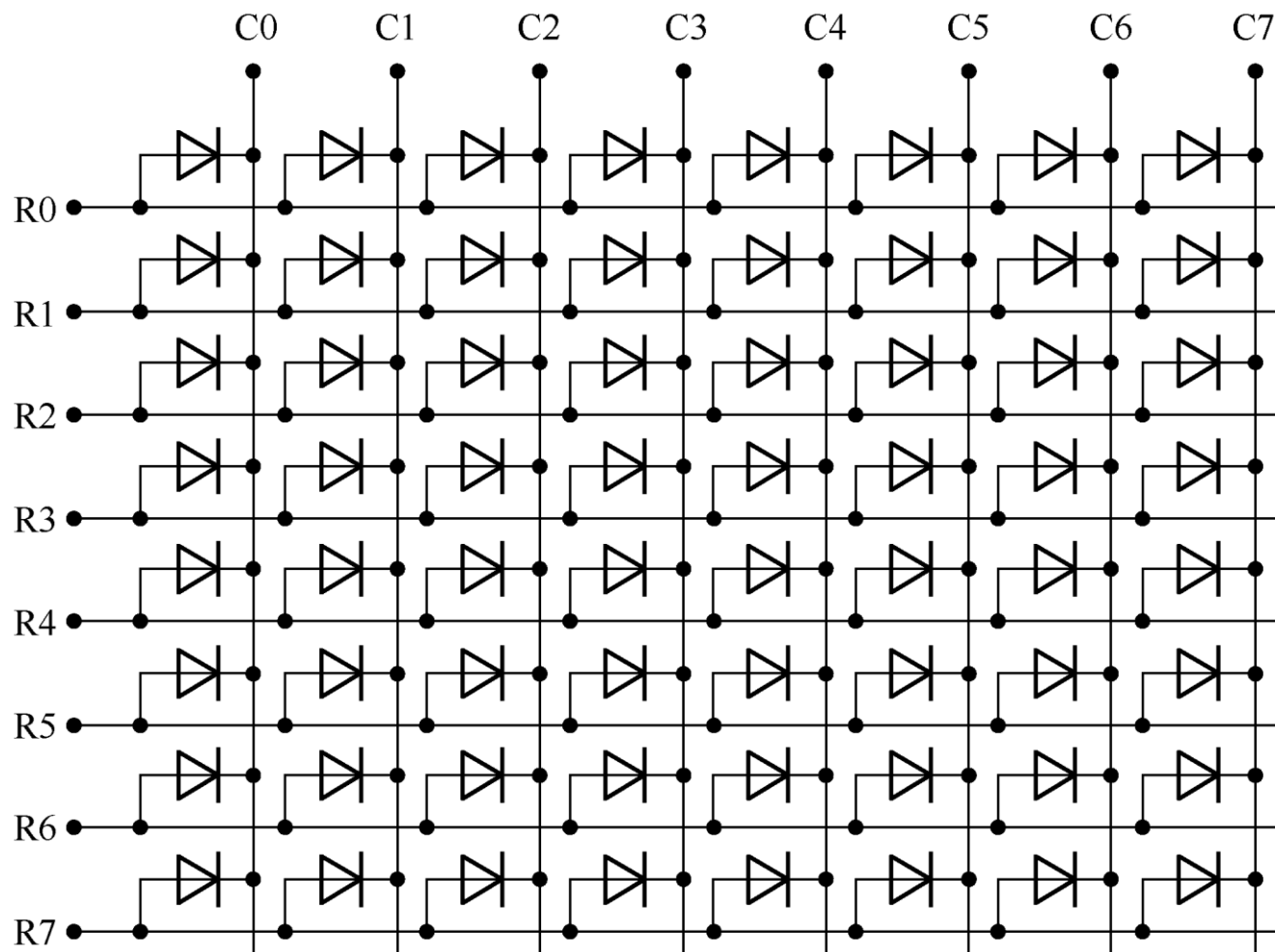


图5-12 8×8LED点阵显示器（共阴极）的结构

制加到行线和列线上编码，控制点亮某些发光二极管（点），从而显示出由不同发光点组成的各种字符。

16×16 LED点阵显示器的结构与**8×8LED**点阵显示模块内部结构及显示原理是类似的，只不过行和列均为**16**。**16×16**是由**4个8×8 LED**点阵组成，且每个发光二极管也是放置在行线和列线的交叉点上，当对应某一行置**0**电平，某一行置**1**电平时，该发光二极管点亮。

下面以显示字符“子”为例，见图**5-13**。

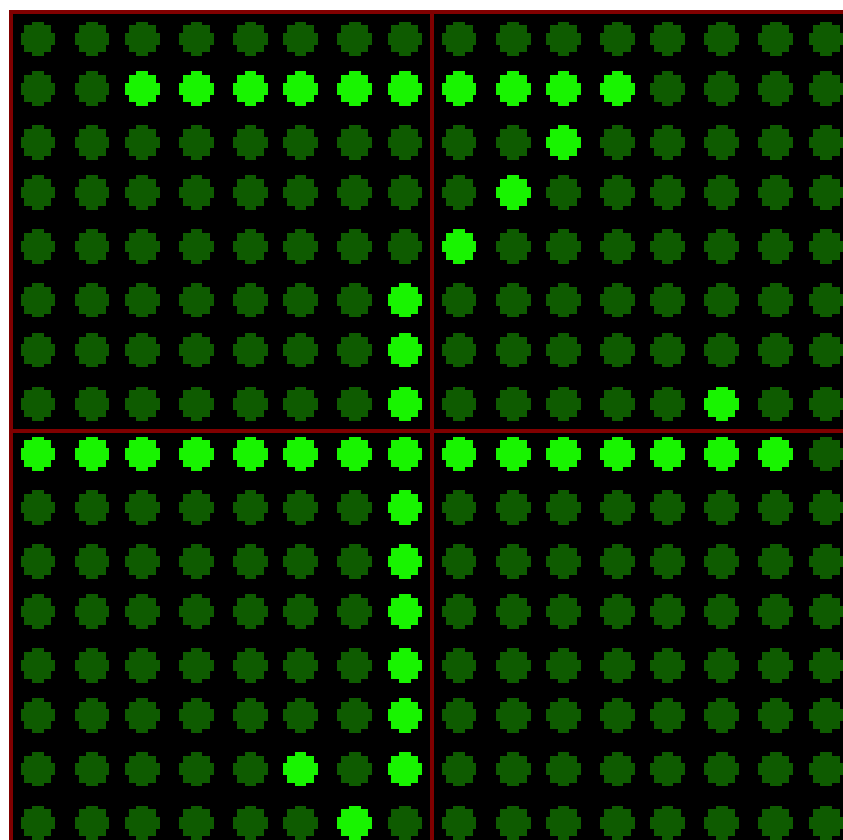


图5-13 16×16 LED点阵显示器显示字符“子”

显示过程如下：

先给LED点阵的**第1行送高电平**（行线高电平有效），同时给**所有列线送高电平**（列线低电平有效），从而第1行发光二极管全灭；

延时一段时间后，再**给第2行送高电平**，同时给所有列线送“**1100 0000 0000 1111**”，列线为0的发光二极管点亮，从而点亮10个发光二极管，显示出汉字“子”的第一横；

延时一段时间后，再给**第3行送高电平**，同时加到列线的编码为“**1111 1111 1101 1111**”，点亮1个发光二极管；

.....;

延时一段时间后，再**给第16行送高电平**，同时给列线送“**1111 1101 1111 1111**”，显示出汉字“子”的最下面的一行，点亮1个发光二极管。然后再重新循环上述操作，利用人眼视觉暂留效应，一个稳定字符“子”显示

出来，见图5-13。

5.4.2 控制16×16 LED点阵显示屏的案例

单片机控制16×16点阵显示屏显示字符案例。

【例5-8】如图5-14，利用单片机及 74LS154（4-16译码器）、74LS07、16×16 LED点阵显示屏来实现字符显示，编写程序，循环显示字符“电子技术”。

图中16×16 LED点阵显示屏16行行线R0~R15电平，由P1口低4位经4-16译码器74HC154的16条译码输出线L0~L15经驱动后的输出来控制。16列列线C0~C15的电平由P0口和P2口控制。剩下问题是如何确定显示字符的点阵编码，以及控制好每一屏逐行显示的扫描速度（刷新频率）。

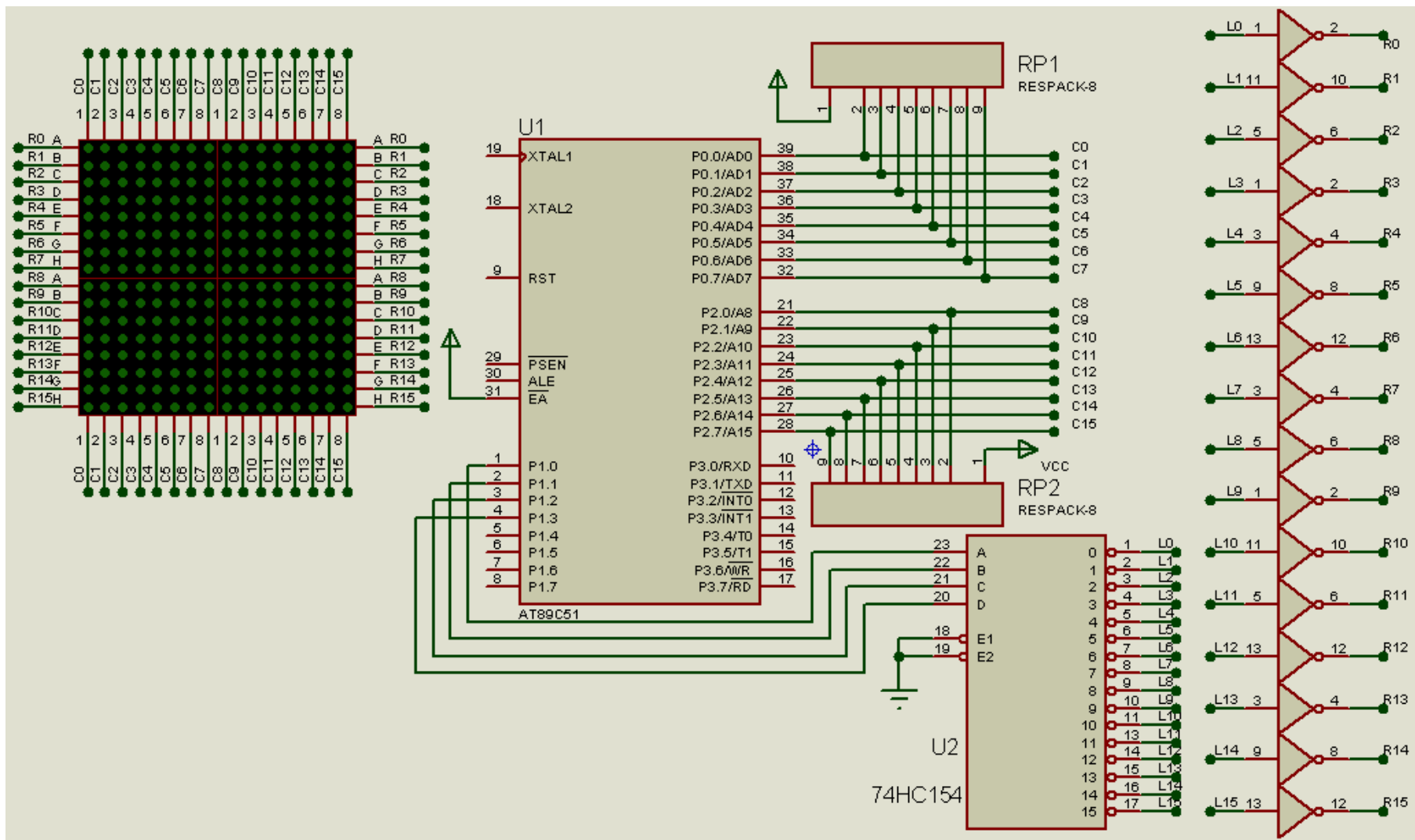


图5-14 控制16×16LED点阵显示器（共阴极）显示字符

参考程序如下:

```
#include<reg51.h>

#define uchar unsigned char
#define uint unsigned int
#define out0 P0
#define out2 P2
#define out1 P1
void delay(uint j)          //延时函数
{
    uchar i=250;
    for(;j>0;j--)
    {
        while(--i);
        i=100;
    }
}
```


uchar code string[]={

//汉字“电” 16×16点阵列码

0x7F,0xFF,0x7F,0xFF,0x7F,0xFF,0x03,0xE0,0x7B,0xEF,0x7B,0xEF,0x03,0xE0,0x7B,0xEF,0x7B,0xEF,0x7B,0xEF,0x03,0xE0,0x7B,0xEF,0x7F,0xBF,0x7F,0xBF,0xFF,0x00,0xFF,0xFF

//汉字“子” 16×16点阵列码

0xFF,0xFF,0x03,0xF0,0xFF,0xFB,0xFF,0xFD,0xFF,0xFE,0x7F,0xFF,0x7F,0xFF,0x7F,0xDF,0x00,0x80,0x7F,0xFF,0x7F,0xFF,0x7F,0xFF,0x7F,0xFF,0x7F,0xFF,0x5F,0xFF,0xBF,0xFF

//汉字“技” 16×16点阵列码

0xF7,0xFB,0xF7,0xFB,0xF7,0xFB,0x40,0x80,0xF7,0xFB,0xD7,0xFB,0x67,0xC0,0x73,0xEF,0xF4,0xEE,0xF7,0xF6,0xF7,0xF9,0xF7,0xF9,0xF7,0xF6,0x77,0x8F,0x95,0xDF,0xFB,0xFF

//汉字“术”的16×16点阵的列码

```
0x7F,0xFF,0x7F,0xFB,0x7F,0xF7,0x7F,0xFF,0x00,0x80,0x7F,0xFF,0x3F,0xFE,0
    x5F,0xFD,
0x5F,0xFB,0x6F,0xF7,0x77,0xE7,0x7B,0x8F,0x7C,0xDF,0x7F,0xFF,0x7F,0xFF,
    0xFF,0xFF,
};
```

void main()

{

uchar i,j,n;

while(1)

{

for(j=0;j<4;j++)

{

//共显示4个汉字

```
for(n=0;n<40;n++)
```

//每个汉字整屏扫描**40**次

```
{
```

```
    for(i=0;i<16;i++)
```

//逐行扫描**16**行

```
{
```

```
    out1=i%16;
```

//输出行码，

```
    out0=string[i*2+j*32];
```

//输出列码到**C0~C7**，逐行扫描

```
    out2=string[i*2+1+j*32];
```

//输出列码到**C8~C15**，逐行扫描

```
    delay(4);
```

//显示并延时一段时间

```
    out0=0xff;
```

//列线**C0~C7**为高电平，熄灭发光二极管

```
    out2=0xff;
```

//列线**C8~C15**为高电平，熄灭发光二极管

```
}
```

```
}
```

```
}
```

}

}

扫描显示时，单片机通过P1口低4位经4-16译码器74HC154的16条译码输出线L0~L15经驱动后的输出来控制，逐行为高电平，来进行扫描。由P0口与P2口控制列码的输出，从而显示出某行应点亮的发光二极管。

以显示汉字“子”为例，说明显示过程。由上面程序可看出，汉字“子”的前3行发光二级管的列码为

“0xFF,0xFF,0x03,0xF0,0xFF,0xFB,……”

第一行列码为：“0xff,0xff”，由P0口与P2口输出，无点亮的发光二极管。

第二行列码为：“0x03,0xf0”，通过P0口与P2口输出后，由图5-13看出，0x03加到列线C7~ C0的二进制编码为“0000 0011”，这里要注意加到8个发光二极管上的对应位置。

按照图5-12和图5-14连线关系，加到从左到右发光二极管应为C0~ C7的二进制编码为“1100 0000”，即最左边的2个发光二极管不亮，其余的6个发光二极管点亮。

同理，P2口输出的0xF0加到列线C15~ C8的二进制编码为“1111 0000”，即加到C8~ C15的二进制编码为“0000 1111”，所以第二行的最右边的4个发光二极管不亮，如图5-13所示。对应通过P0口与P2口输出加到第3行16个发光二极管的列码为“0xFF,0xFB,”，对应于从左到右的C0~ C15的二进制编码为“1111 1111 1101 1111”，从而第3行左边数第11个发光二极管被点亮，其余均熄灭，如图5-13所示。其余各行点亮的发光二极管，也是由16×16点阵的列码来决定。

5.5 单片机控制LCD 1602 液晶显示器的显示

液晶显示器（Liquid Crystal Display, LCD）具有省电、体积小、抗干扰能力强等优点，LCD显示器分为**字段型**、**字符型**和**点阵图形型**。

（1）**字段型**。以长条状组成字符显示，主要用于数字显示，也可用于显示西文字母或某些字符，广泛用于**电子表**、**计算器**、**数字仪表**中。

（2）**字符型**。专门用于显示字母、数字、符号等。一个**字符由 5×7 或 5×10 的点阵组成**，在单片机系统中已广泛使用。

（3）**点阵图形型**。广泛用于图形显示，如**笔记本电脑**、**彩色电视**和游戏机等。它是在平板上排列的多行列的矩阵式的晶格点，点**大小与多少**决定了显示的清晰度。

5.5.1 LCD 1602液晶显示模块简介

单片机系统中常用的字符型液晶显示模块。由于LCD显示面板较为脆弱，厂商已将LCD控制器、驱动器、RAM、ROM和液晶显示器用PCB连接到一起，称为**液晶显示模块（LCd Module, LCM）**，购买现成的即可。单片机只需向LCD显示模块**写入**相应**命令**和**数据**就可显示需要的内容。

1. 字符型液晶显示模块LCD 1602特性与引脚

字符型LCD模块常用的有16字×1行、**16字×2行**、20字×2行、20字×4行等模块，型号常用×××**1602**、×××**1604**、×××**2002**、×××**2004**来表示，其中×××为商标名称，**16**代表液晶显示器每行可显示16个字符，**02**表示显示2行。

LCD1602内有**字符库ROM(CGROM)**，能显示出**192个字符**（5×7点阵），如图5-15所示。

		高 4 位															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
低 4 位	0000	CG RAM (1)															
	0001	(2)															
	0010	(3)															
	0011	(4)															
	0100	(5)															
	0101	(6)															
	0110	(7)															
	0111	(8)															
	1000	(1)															
	1001	(2)															
	1010	(3)															
	1011	(4)															
	1100	(5)															
	1101	(6)															
	1110	(7)															
	1111	(8)															

图5-15 ROM字符库的内容

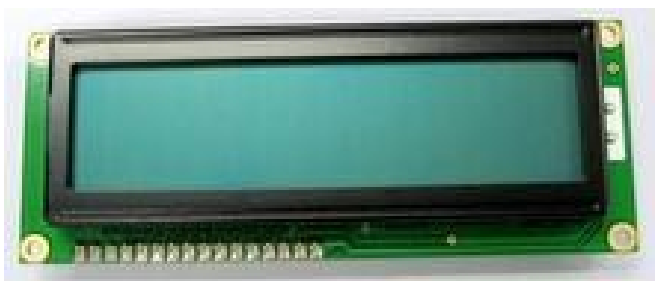
由字符库可看出显示器显示的数字和字母部分代码，恰是**ASCII码**表中编码。

单片机控制**LCD 1602**显示字符，只需将待显示字符的**ASCII码**写入显示**数据存储器（DDRAM）**，内部控制电路就可将字符在显示器上显示出来。

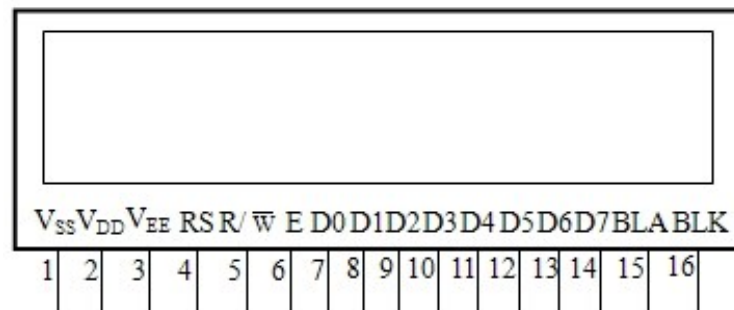
例如，显示字符“**A**”，单片机只需将字符“**A**”的**ASCII码41H**写入**DDRAM**，控制电路就会将对应的字符库**ROM（CGROM）**中的字符“**A**”的**点阵数据**找出来显示在**LCD**上。

模块内有**80字节数据显示RAM（DDRAM）**，除显示**192**个字符（**5×7**点阵）的**字符库ROM（CGROM）**外，还有**64字节**的**自定义字符RAM（CGRAM）**，用户可自行定义**8个5×7点阵字符**。

LCD 1602工作电压4.5~5.5V，典型5V，工作电流2mA。标准的14引脚（无背光）或16个引脚（有背光）的外形及引脚分布如图5-16所示。



(a) LCD 1602的外形



(b) LCD 1602的引脚

图5-16 LCD 1602外形及引脚

引脚包括8条数据线、3条控制线和3条电源线，见表5-2。通过单片机向模块写入命令和数据，就可对显示方式和显示内容做出选择。

表 5-2 LCD 1602 的引脚功能

引脚	引脚名称	引 脚 功 能
1	V_{SS}	电源地
2	V_{DD}	+5V 逻辑电源
3	V_{EE}	液晶显示偏压（调节显示对比度）
4	<u>RS</u>	寄存器选择（1—数据寄存器，0—命令/状态寄存器）
5	<u>R/ $\bar{W}$</u>	读/写操作选择（1—读，0—写）
6	<u>E</u>	使能信号
7~14	D0~D7	数据总线，与单片机的数据总线相连，三态
15	BLA	背光板电源，通常为+5V，串联 1 个电位器，调节背光亮度，如接地，此时无背光但不易发热
16	BLK	背光板电源地

2. LCD1602字符的显示及命令字

显示字符首先要解决待显示字符的ASCII码产生。用户只需在C51程序中写入欲显示的字符常量或字符串常量，C51程序在编译后会自动生成其标准的ASCII码，然后将生成的ASCII码送入显示用数据存储器DDRAM，内部控制电路就会自动将该ASCII码对应的字符在LCD1602显示出来。

让液晶显示器显示字符，首先对其进行初始化设置：对有、无光标、光标移动方向、光标是否闪烁及字符移动方向等进行设置，才能获得所需显示效果。

对LCD 1602的初始化、读、写、光标设置、显示数据的指针设置等，都是单片机向LCD 1602写入命令字来实现。命令字见表5-3。

表 5-3 LCD 1602 的命令字

编号	命令	RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0
1	清屏	0	0	0	0	0	0	0	0	0	1
2	光标返回	0	0	0	0	0	0	0	0	0	×
3	光标和显示模式设置	0	0	0	0	0	0	0	1	I/D	S
4	显示开/关及光标设置	0	0	0	0	0	0	1	D	C	B
5	光标或字符移位	0	0	0	0	0	1	S/C	R/L	×	×
6	功能设置	0	0	0	0	1	DL	N	F	×	×
7	CGRAM 地址设置	0	0	0	1	字符发生存储器地址					
8	DDRAM 地址设置	0	0	1	显示数据存储器地址						
9	读忙标志或地址	0	1	BF	计数器地址						
10	写数据	1	0	要写的数据							
11	读数据	1	1	读出的数据							

表5-3中11个命令功能说明如下：

命令1：清屏，光标返回地址00H位置（显示屏的左上方）。

命令2：光标返回到地址00H位置（显示屏的左上方）。

命令3：输入模式设置。

I/D—地址指针加1或减1选择位。

I/D=1，读或写一个字符后地址指针加1；

I/D=0，读或写一个字符后地址指针减1。

S—屏幕上所有字符移动方向是否有效的控制位。

S=1，当写入一字符时，整屏显示左移（I/D=1）或右移（I/D=0）；

S=0，整屏显示不移动。

命令4: 显示开/关及光标设置。

D—屏幕整体显示控制位，**D=0**关显示，**D=1**开显示。

C—光标有无控制位，**C=0**无光标，**C=1**有光标。

B—光标闪烁控制位，**B=0**不闪烁，**B=1**闪烁。

命令5: 光标或字符移位。

S/C—光标或字符移位选择控制位。**0:** 移动光标，**1:** 移动显示的字符。

R/L—移位方向选择控制位。**0:** 左移，**1:** 右移，

命令6：功能设置命令。

DL—传输数据的有效长度选择控制位。**1：** **8位**数据线接口；**0：** **4位**数据线接口。

N—显示器行数选择控制位。**0：** 单行显示，**1：** 两行显示。

F—字符显示的点阵控制位。**0：** 显示**5×7**点阵字符，**1：** 显示**5×10**点阵字符。

命令7： **CGRAM**地址设置。

命令8： **DDRAM地址设置**。LCD内部有一个数据地址指针，用户可通过它访问内部**全部80字节的数据显示RAM**。

命令格式： **80H+地址码**。其中，80H为命令码。

命令9：读忙标志或地址。

BF—忙标志。**1**:LCD忙，此时LCD不能接受命令或数据；**0**:表示LCD不忙。

命令10：写数据。

命令11：读数据。

例如，将显示设置为“**16×2显示，5×7点阵，8位数据接口**”，只需要向1602写入**光标和显示模式设置命令（命令6）**“00111000B”，即**38H**即可。

再如，要求液晶显示器开显示，显示光标且光标闪烁，那么根据显示开关及光标设置命令（命令4），只要令**D=1，C=1和B=1**，也就是写入命令“**00001111B**”，即**0FH**，就可实现所需的显示模式。

3. 字符显示位置的确定

LCD1602内部有**80字节**的**DDRAM**，与显示屏上字符显示位置一一对应，**图5-17**给出LCD1602显示RAM地址与字符显示位置的对应关系。

当向**DDRAM**的**00H~0FH**（第1行）、**40H~4FH**（第2行）地址的任一处写数据时，LCD立即显示出来，该区域也称为**可显示区域**。

而当写入**10H~27H**或**50H~67H**地址处时，字符不会显示出来，该区域也称为**隐藏区域**。如果要显示写入到隐藏区域的字符，需要通过**字符移位命令**（命令5）将它们**移入到可显示区域**方可正常显示。

需说明的是，在向**DDRAM**写入字符时，首先要设置**DDRAM**定位数据指针，此操作可通过**命令8**完成。

例如，要写字符到**DDRAM**的**40H**处，则**命令8**的格式为：

80H+40H=C0H，其中**80H**为命令代码，**40H**是要写入字符处的地址。

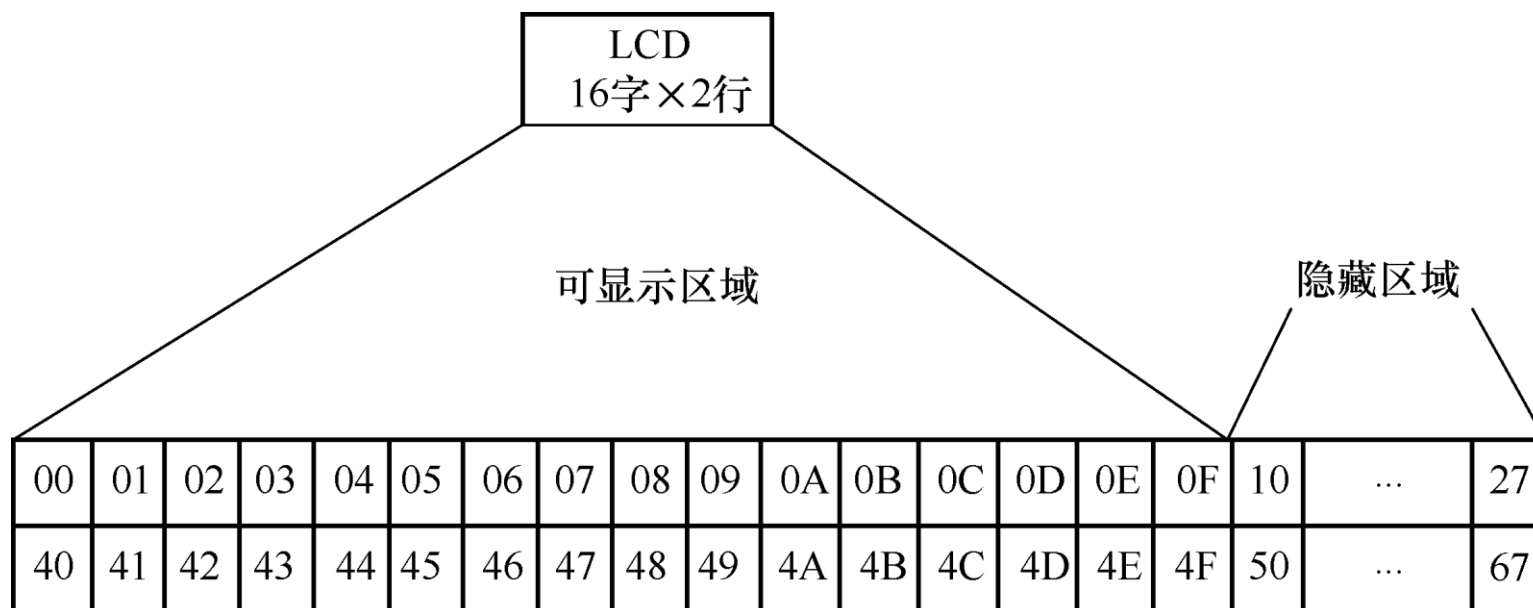


图5-17 LCD内部显示RAM的地址映射图

4. LCD1602基本操作

LCD1602的读写操作规定见表5-4。

表 5-4 LCD1602 的读写操作规定

	单片机发给 LCD 1602 的控制信号	LCD 1602 的输出
读状态	RS=0, R/ $\overline{W}$ =1, E=1	D0~D7=状态字
写命令	RS=0, R/ $\overline{W}$ =0, D0~D7=指令 E=正脉冲	无
读数据	RS=1, R/ $\overline{W}$ =1, E=1	D0~D7=数据
写数据	RS=1, R/ $\overline{W}$ =0, D0~D7=数据 E=正脉冲	无

LCD1602与AT89S52的接口电路见图5-18。

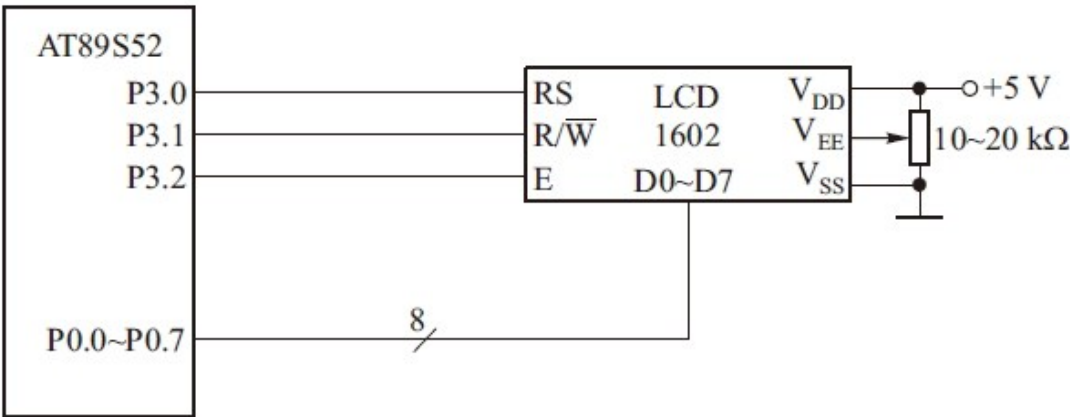


图5-18 单片机与LCD1602接口电路

由图5-18可看出，LCD1602的RS、R/W*和E这3个引脚分别接在P3.0、P3.1和P3.2引脚，只需通过对这3个引脚置“1”或清“0”，就可实现对LCD1602的读写操作。具体来说，显示一个字符的操作过程为“读状态→写命令→写数据→自动显示”。

(1) LCD 1602初始化

使用LCD 1602前，需对其显示模式进行初始化设置，初始化函数如下：

```
void LCD_initial(void)           //液晶显示器初始化函数
{
    write_command(0x38);          //写入命令0x38：两行显示，5×7点阵，8位数据
    _nop_();                      //空操作，给硬件反应时间
    write_command(0x0C);          //写入命令0x0C：开整体显示，光标关，无黑块
    _nop_();                      //空操作，给硬件反应时间
```

```
write_command(0x06);
```

//写入命令0x06: 光标右移

```
_nop();
```

//空操作, 给硬件反应时间

```
write_command(0x01);
```

//写入命令0x01: 清屏

```
delay(1);
```

```
}
```

(2) 检测忙标志

是对LCD1602的“忙”标志BF进行检测, 如果BF=1, 说明LCD处于忙状态, 不能对其写命令; 如果BF=0, 则可写入命令。

检测忙标志函数如下:

```
void check_busy(void)
```

```
//检查忙标志函数
```

```
{  
  uchar dt;  
  do  
  {  
    dt=0xff;           // dt为变量单元，初值为0xff  
    E=0;  
    RS=0;              //按照表5-4读写操作规定RS=0，E=1时才可读忙标志  
    RW=1;  
    E=1;  
    dt=out;            // out为P0口，P0口的状态送入dt中  
  }while(dt&0x80);     // 如果忙标志BF=1，继续循环检测，等待BF=0  
  E=0;                 // BF=0，LCD不忙，结束检测  
}
```

函数检测P0.7脚电平，即检测忙标志BF，如BF=1，说明LCD处于忙状态，不能执行写命令；BF=0，可执行写命令。

(3) 写命令

写命令函数如下：

```
void write_command(uchar com)    //写命令函数
{
    check_busy();
    E=0;                          //按规定RS和E同时为0时可以写入命令
    RS=0;
    RW=0;
    out=com;                      //将命令com写入P0口
    E=1;                          //按规定写命令时，E应为正脉冲，即正跳变，所以前面先置E=0
    _nop();                      //空操作1个机器周期，等待硬件反应
    E=0;                          // E由高电平变为低电平，LCD开始执行命令
    delay(1);                    //延时，等待硬件响应
}
```


(4) 写数据

将要显示字符的**ASCII**码写入**LCD**中的数据显示**RAM**（**DDRAM**），例如将数据“**dat**”，写入**LCD**模块。

写数据函数如下：

```
void write_data(uchar dat)    //写数据函数
{
    check_busy();              //检测忙标志BF=1则等待，若BF=0，则可对LCD操作
    E=0;                       //按规定写数据时，E应为正脉冲，所以先置E=0
    RS=1;                      //按规定RS=1和RW=0时可以写入数据
    RW=0;
    out=dat;                   //将数据dat从P0口输出，即写入LCD
    E=1;                       //E产生正跳变
```

```
_nop_();           //空操作，给硬件反应时间
E=0;              //E由高变低，写数据操作结束
delay(1);
}
```

(5) 自动显示

数据写入LCD后，自动读出字符库ROM(CGROM)中的字型点阵数据，并自动将字型点阵数据送到液晶显示屏上显示。该过程是自动完成的。

5.5.2 单片机控制字符型LCD 1602显示案例

【例5-9】用单片机驱动字符型液晶显示器LCD1602，使其显示两行文字：“Welcom”与“Harbin CHINA”。见图5-19。

在Proteus中，LCD1602的仿真模型采用LM016L。

1. LM016L引脚及特性

LM016L的原理符号及引脚见图5-20。与LCD1602引脚信号相同。引脚功能如下：

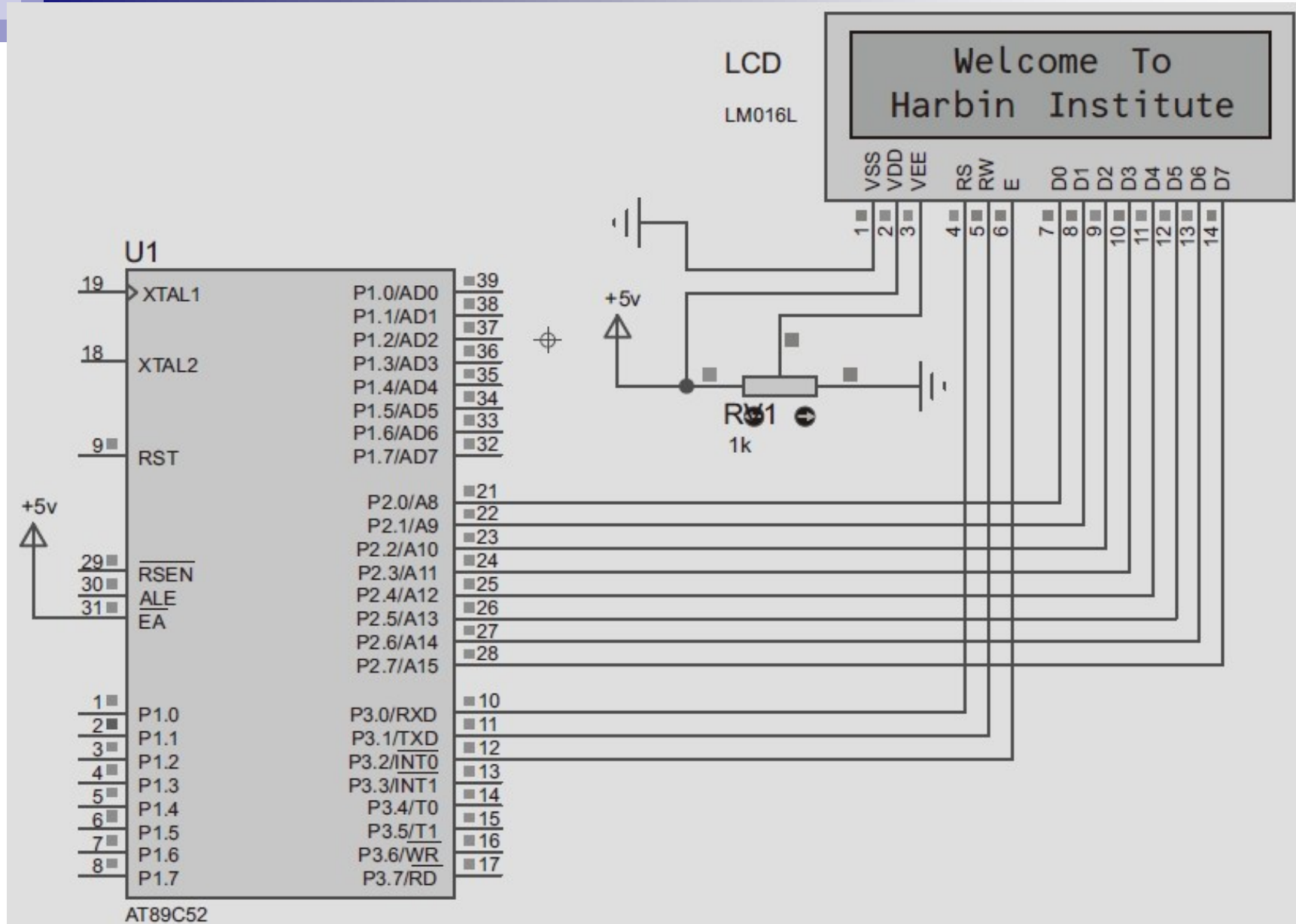
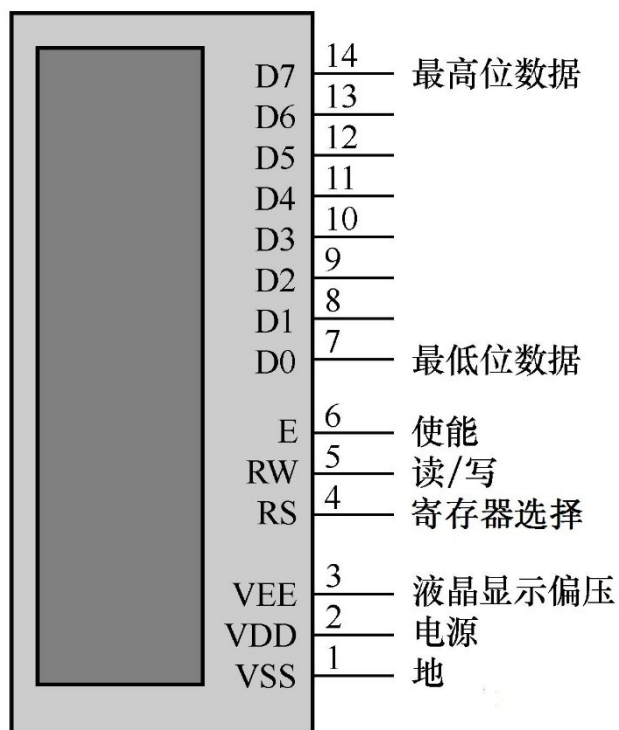


图5-19 单片机与字符型LCD接口电路与仿真

- (1) 数据线**D7~D0**;
- (2) 控制线 (3根: **RS、RW、E**) ;
- (3) 两根电源线 (**VDD、VEE**) 。
- (4) 地线**Vss**;

LM016L的**属性设置**见图**5-21**，具体如下：

- (1) 每行字符数为**16**，行数为**2**;
- (2) 时钟为**250kHz**;
- (3) 第**1**行字符的地址为**80H~8FH**;
- (4) 第**2**行字符的地址为**C0H~CFH**。



LM016L

图5-20 字符型液晶显示器LCD引脚

图5-21 字符型液晶显示器LM016L的属性设置

2. 原理电路设计

(1) 从Proteus库中选取元器件如下：

- **AT89C51**：单片机；
- **LM016L**：字符型显示器；
- **POT-LIN**：滑动变阻器；
- **RP1、RP2**：排电阻。

(2) 放置元器件、放置电源和地、连线、元器件属性设置、电气检测

所有操作都在ISIS中完成。

3. 程序设计

通过Keil C51建立工程，再建立源程序“*.c”文件，操作见第4章。在前面介绍的LCD1602基本操作函数的基础上，不难理解如下源程序。参考程序：

#include <reg51.h>	
#include <intrins.h>	//包含_nop_()空函数指令的头文件
#define uchar unsigned char	
#define uint unsigned int	
#define out P0	
sbit RS=P2^0;	//位变量
sbit RW=P2^1;	//位变量
sbit E=P2^2;	//位变量
_initial(void);	void lcd
void check_busy(void);	//LCD初始化函数
void write_command(uchar com);	//检查忙标志函数
void write_data(uchar dat);	//写命令函数
void string(uchar ad ,uchar *s);	//写数据函数
void lcd_test(void);	
void delay(uint);	//延时函数


```
void main(void)
```

```
//主函数
```

```
{
```

```
    lcd _initial( );
```

```
//调用对LCD初始化函数
```

```
    while(1)
```

```
    {
```

```
        string(0x85, "Welcome");
```

```
//显示的第1行字符串
```

```
        string(0xC2, "Harbin CHINA");
```

```
//显示的第2行字符串
```

```
        delay(100);
```

```
//延时
```

```
        write_command(0x01);
```

```
//写入清屏命令
```

```
        delay(100);
```

```
//延时
```

```
    }
```

```
}
```

```
void delay(uint j)
```

```
//1ms延时子程序
```

```
{
```

```
    uchar i=250;
```

```
    for (;j>0;j--)
```

```
{  
    while(--i);  
    i=249;  
    while(--i);  
    i=250;  
}  
}
```

```
void check_busy(void)
```

```
//检查忙标志函数
```

```
{  
    uchar dt;  
    do  
    {  
        dt=0xff;  
        E=0;  
        RS=0;  
        RW=1;
```

```
E=1;
dt=out;
}while(dt&0x80);
E=0;
}
```

```
void write_command(uchar com)
```

//写命令函数

```
{
    check_busy();
    E=0;
    RS=0;
    RW=0;
    out=com;
    E=1;
    _nop_();
    E=0;
    delay(1);
}
```

```
void write_data(uchar dat)
```

```
//写数据函数
```

```
{  
    check_busy();  
    E=0;  
    RS=1;  
    RW=0;  
    out=dat;  
    E=1;  
    _nop_();  
    E=0;  
    delay(1);  
}
```

```
void LCD_initial(void)
```

```
//液晶显示器初始化函数
```

```
{  
write_command(0x38); //写入命令0x38: 8位两行显示, 5×7点阵字符  
write_command(0x0C); //写入命令0x0C: 开整体显示, 光标关, 无黑块  
write_command(0x06); //写入命令0x06: 光标右移
```

```
write_command(0x01);
```

```
//写入命令0x01：清屏
```

```
delay(1);
```

```
}
```

```
void string(uchar ad, uchar *s)
```

```
//输出显示字符串的函数
```

```
{
```

```
write_command(ad);
```

```
while(*s>0)
```

```
{
```

```
write_data(*s++);
```

```
//输出字符串，且指针增1
```

```
delay(100);
```

```
}
```

```
}
```

最后通过按钮“**Build target**”编译源程序，生成目标代码“*.hex”文件。若编译失败，对程序修改调试直至编译成功。

4. Proteus仿真

(1) 加载目标代码文件

打开元器件单片机属性窗口，在“**Program File**”栏中添加上面编译好的目标代码文件“*.hex”；在“**Clock Frequency**”栏中输入晶振频率12MHz。

(2) 仿真

单击仿真按钮  启动仿真，见图5-19。

5.6 点阵式液晶显示器LCD12864的显示控制

下面介绍单片机控制点阵式液晶显示器LCD12864的应用编程。

目前比较流行的点阵式液晶显示屏LCD12864有两种：

一种是以KS0108为主控芯片，不带字库，显示的字符或图形是由不同的点阵组成，点阵的获得，可借助于取模软件；

另一种是以ST7920为主控芯片，带有ASCII码和中文的点阵字库。



图5-22为LCD12864点阵式液晶显示屏的外形及Proteus元件库中的元件模型AMPIRE 128×64（不带字库，可认为主控芯片为KS0108）与引脚。

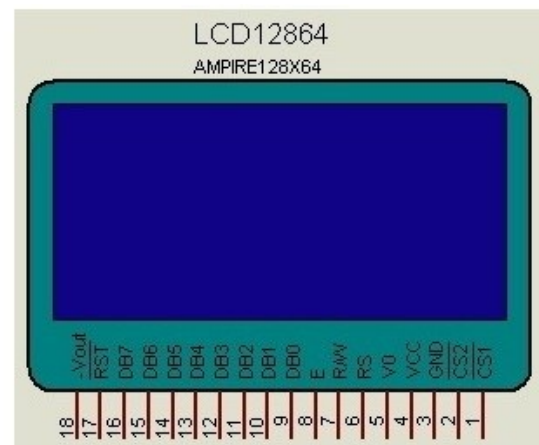
下面介绍KS0108为主控芯片的12864液晶显示屏的引脚及显示控制原理。

5.6.1 引脚及显示原理

LCD12864各引脚(以12864C为例，不同型号的引脚排列略有差别)功能见表5-5。



LCD12864液晶显示屏



Proteus元件库中的模型

图5-22 LCD12864的外形以及Proteus元件库中的模型与引脚

表5-5 KS0108为主控芯片的12864液晶显示屏的引脚

编号	符号	引脚功能	编号	符号	引脚功能
1	CS1	片选IC1信号	11	D2	数据线
2	CS2	片选IC2信号	12	D3	数据线
3	GND	电源地	13	D4	数据线
4	Vcc	电源正极 (+5v)	14	D5	数据线
5	V0	LCD驱动电压输入(对比度调节)	15	D6	数据线
6	RS	数据/命令选择 (H/L)	16	D7	数据线
7	R/W	读/写控制 (H/L)	17	RST	复位端 (H: 正常工作, L: 复位)
8	E	使能信号	18	VOUT	LCD驱动负压输出 (-5v)
9	D0	数据线	19	BLA	背光源正极
10	D1	数据线	20	BLK	背光源负极



KS0108控制的12864内部有**两个控制器**，分别控制**左半屏**和**右半屏**，显示原理图如**图5-23**所示。

左半屏和右半屏操作时写的地址其实是一样的，要由片选**CS1**和**CS2**来选择哪半个屏，如果两个都选通，则相当于两块 64×64 的液晶显示屏，而且显示的内容是相同的，取模方式是纵向8点，下为高位D7。列的范围是0~63，已在图中标出。行是不能按位来写的，而是写“页”，一个页相当于8个点，也就是8位，即一个字符，高位在下面。页的范围是0~7，共8页，8页 $\times$ 8个点正好64个点。

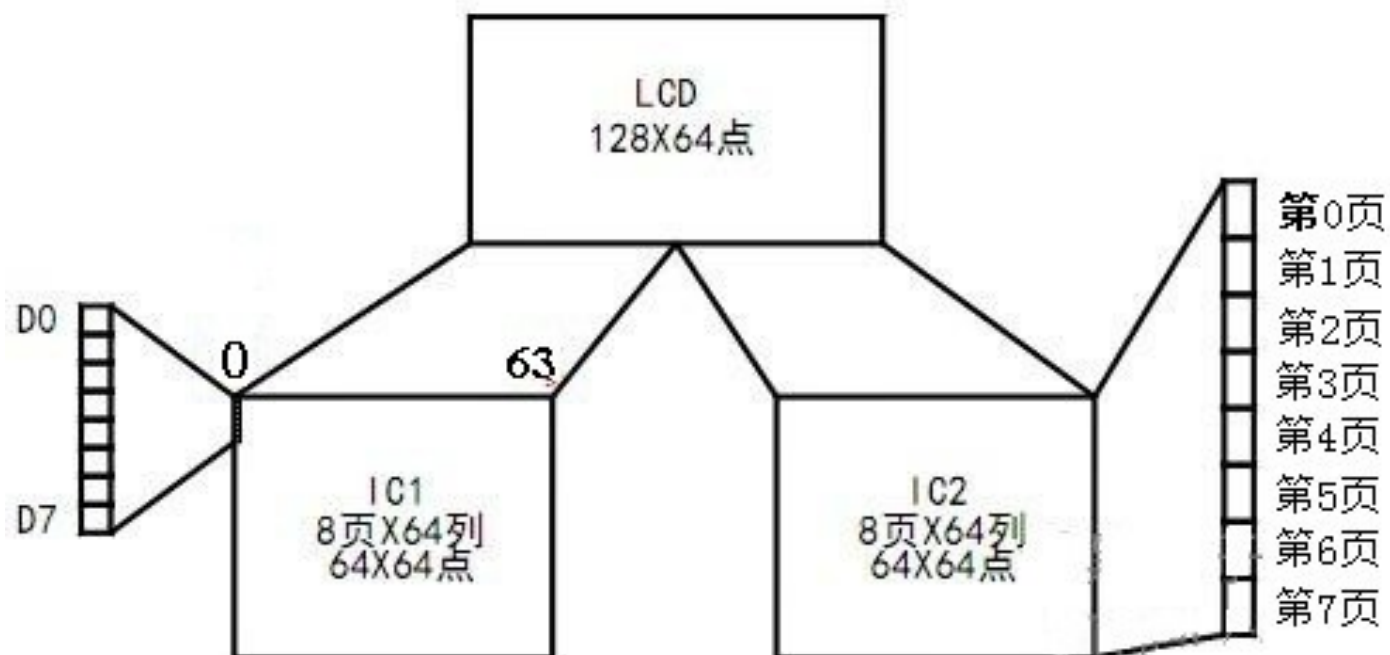


图5-23 LCD的128 × 64阵显示结构

图5-24是用取模软件截的一个“们”字，可以看出它是 16×16 大小，实际上占用了两个“页”，16个列，操作时先固定一个页，先写上面那页，假设为页n，从列0写到15，然后对页n+1，再从列0写到15，这样一个“们”字的点阵就出来了，下面是其点阵代码：

```
0x40,0x20,0xF8,0x07,0x00,0xF8,0x02,0x04,0x08,0x04,0x04,0x04,0x04,0
xFE,0x04,0x00,
0x00,0x00,0xFF,0x00,0x00,0xFF,0x00,0x00,0x00,0x00,0x00,0x40,0x80,0
x7F,0x00,0x00
```

可见 16×16 的字符占了32个字节（上面n页16个字节加n+1页16个），那么一幅满幅的图片就是 128×64 ，占用 $128 \times 8 = 1K$ 个字节，可见还是非常占空间的。

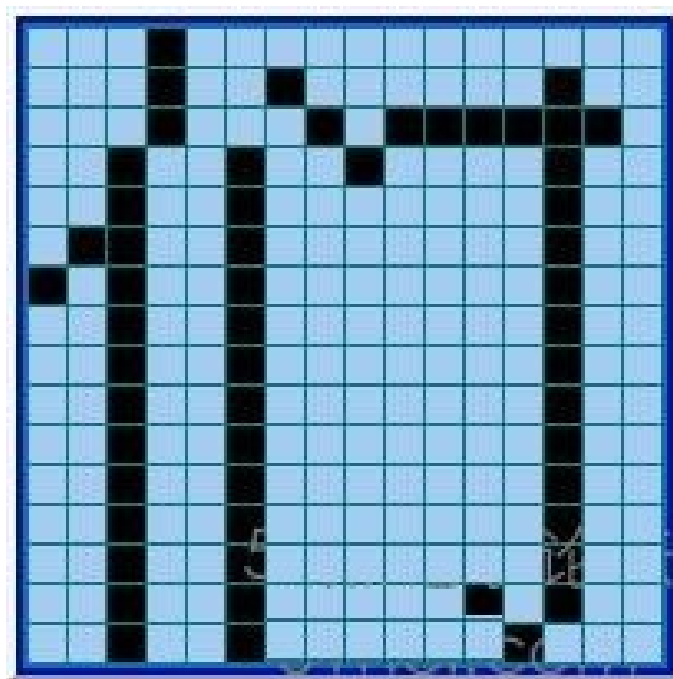


图5-24 “们”字的点阵

5.6.2 控制命令

LCD12864液晶显示屏的命令如表5-6所示。

表5-6 命令列表

[illegible]

图5-25是LCD12864命令写入的流程图。

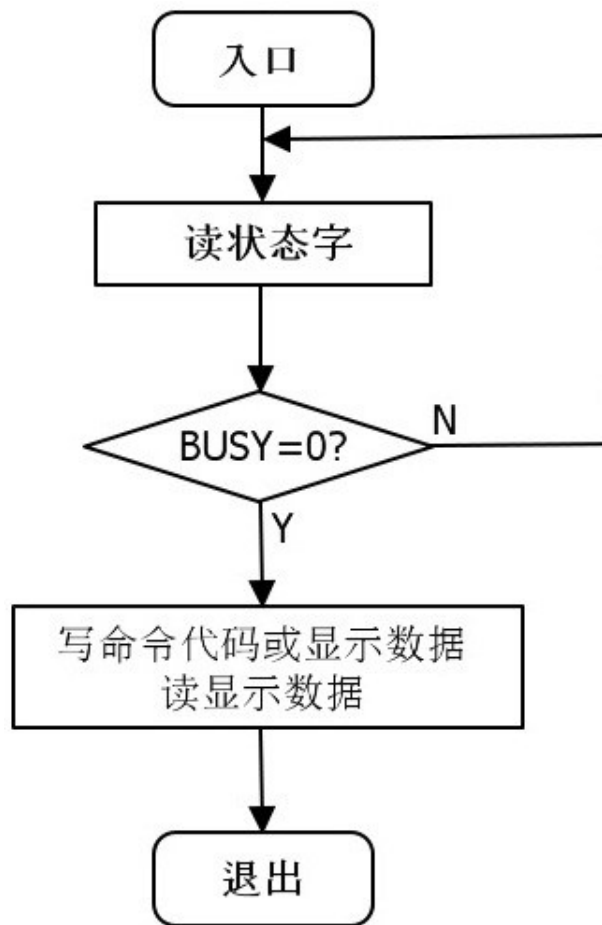


图5-25 LCD12864命令写入的流程图

1. 读状态字

在向LCD12864写入命令或显示数据前，应先读其状态字。状态字的格式如下。

BUSY	0	ON/OFF	RESET	0	0	0	0
------	---	--------	-------	---	---	---	---

各位的功能如下：

BUSY位 1：表示KS0108正在处理单片机发来的命令或数据，不能接受读状态字以外的任何操作；0：表示KS0108接口电路已处于准备好的状态，可接收单片机发来的命令或数据。

ON/OFF位 1：处于关显示状态；0：处于开显示状态。

RESET位

1: KS0108的RST引脚为低电平，KS0108处于复位状态。

0: KS0108的RST引脚为高电平，KS0108处于正常工作状态。

单片机在发送给LCD12864命令或显示数据时，一定要先检测BUSY位，只有当BUSY位为0时，单片机才能向LCD12864发送命令或显示数据。

2. 显示开关命令

命令格式如下。

0	0	1	1	1	1	1	D
---	---	---	---	---	---	---	---

D位为显示开关控制位。

D=1为显示设置，显示数据锁存器正常工作，显示屏上呈现所需的显示效果。此时状态字中的ON/OFF=0。

D=0为关显示设置，显示数据锁存器被置0，显示屏呈不显示状态，此时状态字中的ON/OFF=1。

3.显示起始行设置命令

命令格式如下。

1	1	L5	L4	L3	L2	L1	L0
---	---	----	----	----	----	----	----

该命令设置了显示起始行寄存器的内容。**KS0108**有64行显示的管理能力，该命令中的L5~L0为显示起始行，取值在00H~3FH(1~64)范围内，它规定了显示屏上最上面一行所对应的显示存储器的行地址。如果定时间隔地，等间距地修改（加1或减1）显示寄存器的内容，则显示屏则呈现显示内容向上或向下平滑滚动的显示效果。

4. 页面地址设置命令

命令格式如下。

1	0	1	1	1	P2	P1	P0
---	---	---	---	---	----	----	----

该命令设置了页面地址—X地址寄存器的内容。KS0108将显示存储器分成8页，命令代码中P2~P0就是要确定当前所要选择的页面地址，取值范围0H~7H，代表1~8页，该命令规定了读/写操作要在哪一个页面上进行。

5. 列地址设置命令

命令格式如下。

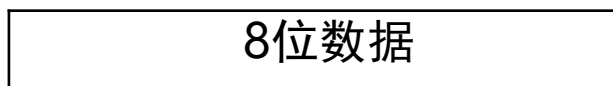
0	1	C5	C4	C3	C2	C1	C0
---	---	----	----	----	----	----	----

该命令设置了Y地址计数器的内容，C05~C = 0H~3FH（1~64）代表某一页面的某一单元的地址，随后的一次读/写数据将在这个单元中进行。Y地址计数器具有自动加1的功能，在每一次读/写数据后，它将自动加1，所以在连续进行读/写数据时，Y地址计数器不必每次都设置一次。

页面地址的设置和列地址的设置将显示存储器单元唯一地确定下来。

6. 写显示数据

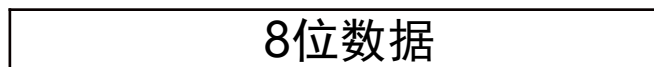
格式如下。



该操作将**8**位数据写入先前已确定的显示存储器的单元内，操作完成后列地址计数器自动加1。

7. 读显示数据

格式如下。



该操作将**KS0108**接口的输出寄存器的内容读出，然后列地址计数器自动加1。

5.6.3 单片机控制LCD12864显示的案例

【例5-10】 本例要求单片机控制点阵式液晶显示器LCD12864分两行显示“PROTEUS 电子设计与创新的最佳平台”，电路如图5-26所示。

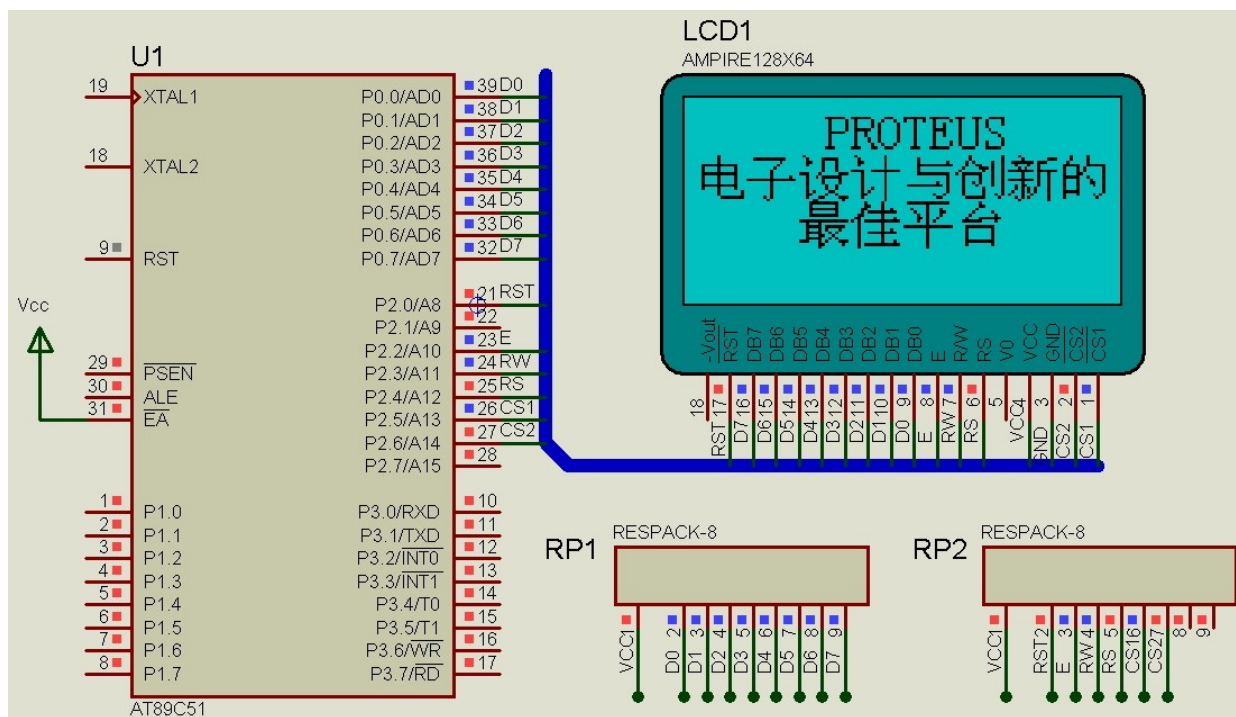


图5-26 单片机控制点阵式液晶显示器LCD12864的字符显示

参考程序如下。

```
#include <reg51.h>
```

```
#define uchar signed char
```

```
#define uint unsigned int
```

// 常量定义

```
#define lcdrow 0xc0
```

//设置起始行

```
#define lcdpage 0xb8
```

//设置起始页

```
#define lcdcolumn 0x40
```

//设置起始列

```
#define c_page_max 0x08
```

//页数最大值

```
#define c_column_max 0x40
```

//列数最大值

//端口定义

```
#define bus P0
```

```
sbit rst=P2^0;
```

```
sbit e=P2^2;
```

```
sbit rw=P2^3;
```




```
sbit rs=P2^4;  
sbit cs1=P2^5;  
sbit cs2=P2^6;
```

//函数声明

```
void delayms(uint);           //延时n ms  
void delayus10(void);         //延时10us  
void select(uchar);           //选择屏幕  
void send_cmd(uchar);         //写命令  
void send_data(uchar);        //写数据  
void clear_screen(void);      //清屏  
void initial(void);           //LCD初始化  
void display_zf(uchar,uchar,uchar,uchar); //显示字符  
void display_hz(uchar,uchar,uchar,uchar); //显示汉字  
void display(void);           //在LCD上显示  
//字符表  
//宋体12; 此字体下对应的点阵为：宽×高=8×16  
//取模方式：纵向取模下高位，从上到下，从左到右取模
```



```
uchar code table_zf[]={
```

```
// 文字: P
```

```
0x08,0xF8,0x08,0x08,0x08,0x08,0xF0,0x00,0x20,0x3F,0x21,0x01,0x01,0x01,0x00  
,0x00,
```

```
// 文字: R
```

```
0x08,0xF8,0x88,0x88,0x88,0x88,0x70,0x00,0x20,0x3F,0x20,0x00,0x03,0x0C,0x30  
,0x20,
```

```
// 文字: O
```

```
0xE0,0x10,0x08,0x08,0x08,0x10,0xE0,0x00,0x0F,0x10,0x20,0x20,0x20,0x10,0x0F  
,0x00,
```

```
// 文字: T
```

```
0x18,0x08,0x08,0xF8,0x08,0x08,0x18,0x00,0x00,0x00,0x20,0x3F,0x20,0x00,0x00,  
0x00,
```

```
// 文字: E
```

```
0x08,0xF8,0x88,0x88,0xE8,0x08,0x10,0x00,0x20,0x3F,0x20,0x20,0x23,0x20,0x18  
,0x00,
```

```
// 文字: U
```

```
0x08,0xF8,0x08,0x00,0x00,0x08,0xF8,0x08,0x00,0x1F,0x20,0x20,0x20,0x20,0x1F  
,0x00,
```

// 文字: S

0x00,0x70,0x88,0x08,0x08,0x08,0x38,0x00,0x00,0x38,0x20,0x21,0x21,0x22,0x1C,
0x00

};

// 汉字表

// 宋体12; 此字体下对应的点阵为: 宽×高=16×16

// 取模方式: 纵向取模下高位, 从上到下, 从左到右取模

uchar code table_hz[]={

// 文字: 电

0x00,0x00,0xF8,0x48,0x48,0x48,0x48,0xFF,0x48,0x48,0x48,0x48,0xF8,0x00,0x00,
0x00,

0x00,0x00,0x0F,0x04,0x04,0x04,0x04,0x3F,0x44,0x44,0x44,0x44,0x4F,0x40,0x70,
0x00,

// 文字: 子

0x00,0x00,0x02,0x02,0x02,0x02,0x02,0xE2,0x12,0x0A,0x06,0x02,0x00,0x80,0x00,
0x00,

0x01,0x01,0x01,0x01,0x01,0x41,0x81,0x7F,0x01,0x01,0x01,0x01,0x01,0x01,0x01,
0x00,

// 文字: 设

0x40,0x41,0xCE,0x04,0x00,0x80,0x40,0xBE,0x82,0x82,0x82,0xBE,0xC0,0x40,0x40,0x00,
0x00,0x00,0x7F,0x20,0x90,0x80,0x40,0x43,0x2C,0x10,0x10,0x2C,0x43,0xC0,0x40,0x00,

// 文字: 计

0x20,0x21,0x2E,0xE4,0x00,0x00,0x20,0x20,0x20,0x20,0xFF,0x20,0x20,0x20,0x20,0x00,
0x00,0x00,0x00,0x7F,0x20,0x10,0x08,0x00,0x00,0x00,0xFF,0x00,0x00,0x00,0x00,0x00,

// 文字: 与

0x00,0x00,0x00,0x00,0x7E,0x48,0x48,0x48,0x48,0x48,0x48,0x48,0x48,0xCC,0x08,0x00,
0x00,0x04,0x04,0x04,0x04,0x04,0x04,0x04,0x04,0x24,0x46,0x44,0x20,0x1F,0x00,0x00,

// 文字: 创

0x40,0x20,0xD0,0x4C,0x43,0x44,0x48,0xD8,0x30,0x10,0x00,0xFC,0x00,0x00,0xFF,0x00,
0x00,0x00,0x3F,0x40,0x40,0x42,0x44,0x43,0x78,0x00,0x00,0x07,0x20,0x40,0x3F,0x00,

// 文字: 新

0x20,0x24,0x2C,0x35,0xE6,0x34,0x2C,0x24,0x00,0xFC,0x24,0x24,0xE2,0x22,
0x22,0x00,

0x21,0x11,0x4D,0x81,0x7F,0x05,0x59,0x21,0x18,0x07,0x00,0x00,0xFF,0x00,0
x00,0x00,

// 文字: 的

0x00,0xF8,0x8C,0x8B,0x88,0xF8,0x40,0x30,0x8F,0x08,0x08,0x08,0x08,0xF8,
0x00,0x00,

0x00,0x7F,0x10,0x10,0x10,0x3F,0x00,0x00,0x00,0x03,0x26,0x40,0x20,0x1F,0
x00,0x00,

// 文字: 最

0x40,0x40,0xC0,0x5F,0x55,0x55,0xD5,0x55,0x55,0x55,0x55,0x5F,0x40,0x40,
0x40,0x00,

0x20,0x20,0x3F,0x15,0x15,0x15,0xFF,0x48,0x23,0x15,0x09,0x15,0x23,0x61,0
x20,0x00,

// 文字: 佳

0x40,0x20,0xF0,0x1C,0x47,0x4A,0x48,0x48,0x48,0xFF,0x48,0x48,0x4C,0x68,
0x40,0x00,



```
0x00,0x00,0xFF,0x00,0x40,0x44,0x44,0x44,0x44,0x7F,0x44,0x44,0x46,0x64,0x40
,0x00,
```

```
// 文字: 平
```

```
0x00,0x01,0x05,0x09,0x71,0x21,0x01,0xFF,0x01,0x41,0x21,0x1D,0x09,0x01,0x00
,0x00,0x01,0x01,0x01,0x01,0x01,0x01,0x01,0xFF,0x01,0x01,0x01,0x01,0x01,0x0
1,0x01,0x00,
```

```
// 文字: 台
```

```
0x00,0x00,0x40,0x60,0x50,0x48,0x44,0x63,0x22,0x20,0x20,0x28,0x70,0x20,0x00,
0x00,0x00,0x00,0x00,0x7F,0x21,0x21,0x21,0x21,0x21,0x21,0x21,0x7F,0x00,0x00
,0x00,0x00
```

```
};
```

```
void main()
```

```
{
```

```
    initial();
```

```
    display();
```

```
    clear_screen();
```

```
    display();
```

```
    while(1);
```

```
}
```

```
void delayus10(void)
```

//延时10μs函数

```
{  
    uchar i=5;  
    while(--i);  
}
```

```
void delayms(uint j)
```

//延时10ms函数

```
{  
    uchar i=250;  
    for(;j>0;j--)  
    {  
        while(--i); i=249;while(--i);i=250;  
    }  
}
```

//屏幕选择-cs=0选择双屏,cs=1选择左半屏,cs=2选择右半屏

```
void select(uchar cs)
```

```
{
```

```
if(cs==0)cs1=1,cs2=1;
    else if(cs==1)cs1=1,cs2=0;
    else cs1=0,cs2=1;
}
```

```
void send_cmd(uchar cmd)
```

//写命令函数

```
{
    rs=0;rw=0; bus=cmd;delayus10();e=1;e=0;
}
```

```
void send_data(uchar dat)
```

//写数据函数

```
{
    rs=1;rw=0; bus=dat;delayus10();e=1;e=0;
}
```

```
void clear_screen(void)
```

//清屏函数

```
{
    uchar c_page,c_column;
```




```
select(0);
    for(c_page=0;c_page<c_page_max;c_page++)
    {
        send_cmd(c_page+lcdpage);
        send_cmd(lcdcolumn);
        for(c_column=0;c_column<c_column_max;c_column++)
        {
            send_data(0X00);
        }
    }
}
```

```
void initial(void)
```

//LCD初始化函数

```
{
    select(0);
    rst=0;delayms(10);rst=1;
```



```
clear_screen();
send_cmd(lcdrow);
send_cmd(lcdcolumn);
send_cmd(lcdpage);
send_cmd(0x3f);
}
```

//写字符, c_page为当前页,c_column为当前列, num为字符数,
//offset为所取字符在显示缓冲区中的偏移单位

```
void display_zf(uchar c_page,uchar c_column,uchar num,uchar offset)
{
    uchar c1,c2,c3;
    for(c1=0;c1<num;c1++)
    {
        for(c2=0;c2<2;c2++)
```

```
{for(c3=0;c3<8;c3++)
    {
        send_cmd(lcdpage+c_page+c2);
        send_cmd(lcdcolumn+c_column+c1*8+c3);
        send_data(table_zf[(c1+offset)*16+c2*8+c3]);
    }
}
}
```

//写汉字， c_page为当前页,c_column为当前列， num为字符数，
//offset为所取汉字在显示缓冲区中的偏移单位

```
void display_hz(uchar c_page,uchar c_column,uchar num,uchar offset)
{
    uchar c1,c2,c3;
    for(c1=0;c1<num;c1++)
        {for(c2=0;c2<2;c2++)
            {
                for(c3=0;c3<16;c3++)
                {
                    send_cmd(lcdpage+c_page+c2);
                    send_cmd(lcdcolumn+c_column+c1*16+c3);

                    send_data(table_hz[(c1+offset)*32+c2*16+c3]);
                }
            }
        }
}
```



```
void display(void)
```

```
//在LCD上显示函数
```

```
{
```

```
    select(1);
```

```
    display_zf(0,40,3,0);
```

```
    display_hz(2,0,4,0);
```

```
    display_hz(4,32,2,8);
```

```
    select(2);
```

```
    display_zf(0,0,4,3);
```

```
    display_hz(2,0,4,4);
```

```
    display_hz(4,0,2,10);
```

```
}
```

5.7 键盘接口设计

键盘——向单片机输入数据、命令等功能，是人机对话的主要手段。

由若干按键按照一定规则组成。每一个按键实质上是一个按键开关，按构造可分为有触点开关按键和无触点按键。

有触点开关按键常见的有：触摸式键盘、薄膜键盘、导电橡胶、按键式键盘等，最常用按键式键盘。无触点开关按键有电容式按键、光电式按键和磁感应按键等。下面介绍按键式开关键盘工作原理、方式以及与键盘接口设计与软件编程。

5.7.1 键盘接口设计应解决的问题

1. 键盘的任务

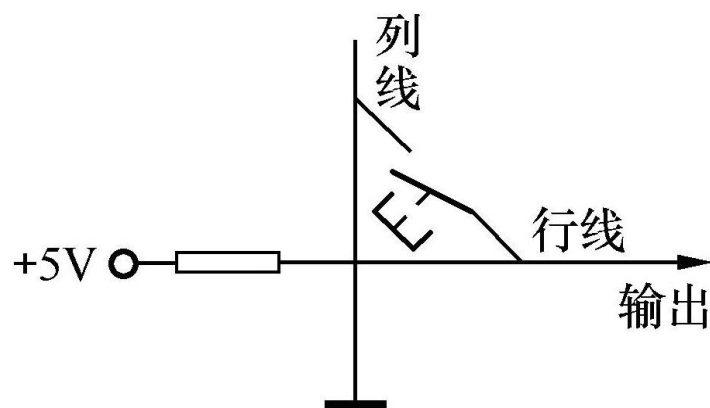
任务**3**项。

- (1) 判别是否有键按下？若有，进入第(2)步。
- (2) 识别哪一个键被按下，并求出相应的键值。
- (3) 根据键值，找到相应键值处理程序入口。

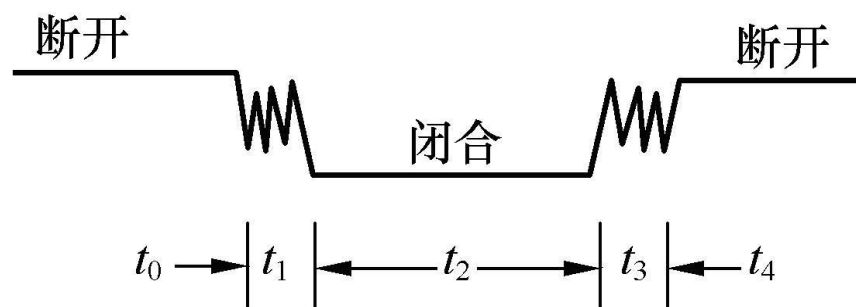
2. 键盘输入特点

键盘一个按键实质就是一个按钮开关。**图5-27 (a)**所示按键开关的两端分别连接在行线和列线上，列线接地，行线通过电阻接到+5V上。键盘开关机械触点的断开、闭合，其**行线电压输出波形如图5-27 (b) 所示**。

图5-27 (b) 所示的 t_1 和 t_3 分别为键的闭合和断开过程中的抖动期（呈现一串负脉冲），抖动时间长短与开关机械特性有关，一般为5~10ms， t_2 为稳定的闭合期，其时间由按键动作确定，一般为十分之几秒到几秒， t_0 、 t_4 为断开期。



(a) 按键开关



(b) 键闭合时行线输出电压波形

图5-27 键盘开关及其行线波形

3. 按键的识别

按键闭合与否，反应在行线输出电压上就是高电平或低电平，对行线电平高低状态检测，便可确认按键是否按下与松开。为了确保单片机对一次按键动作只确认一次按键有效，必须消除抖动期 t_1 和 t_3 的影响。

4. 如何消除按键的抖动

两种去抖动方法。一种是用软件延时来消除按键抖动，基本思想：在检测到有键按下时，该键所对应的行线为低电平，执行一段延时10ms的子程序后，确认该行线电平是否仍为低电平，如果仍为低电平，则确认该行确实有键按下。当按键松开时，行线的低电平变为高电平，执行一段延时10ms的子程序后，检测该行线为高电平，说明按键确实已经松开。

采取以上措施，可消除两个抖动期 t_1 和 t_3 的影响。另一种去除按键抖动的方法是采用专用的键盘/显示器接口芯片，这类芯片中都有自动去抖动的硬件电路。

键盘主要分为两类：非编码键盘和编码键盘。

非编码键盘是利用按键直接与单片机相连接而成，常用在按键数量较少的场合。该类键盘，系统功能比较简单，需要处理任务较少，成本低、电路设计简单。按下键号的信息通过软件来获取。

非编码键盘常见的有：独立式键盘和矩阵式键盘两种结构。

先介绍独立式键盘接口设计。

5.7.2 独立式键盘接口设计案例

独立式键盘**特点各键相互独立，每个按键各接一条I/O口线**，通过检测I/O输入线的电平状态，易判断哪个按键被按下。

图5-28为一独立式键盘，**8个按键k1~k8**分别接到单片机的**P1.0~ P1.7**引脚上，图中上拉电阻保证按键未按下时，保证对应I/O口线为稳定高电平。当某一按键按下时，对应I/O口线就变成低电平，与其他按键相连的I/O口线仍为高电平。

因此，**只需读入I/O口线状态，判别是否为低电平，就很容易识别出哪个键被按下**。可见独立式键盘优点是电路简单，各条检测线独立，识别按键号的软件编写简单。独立式键盘适于按键数目较少场合，如按键数目较多，要占用较多I/O口线。

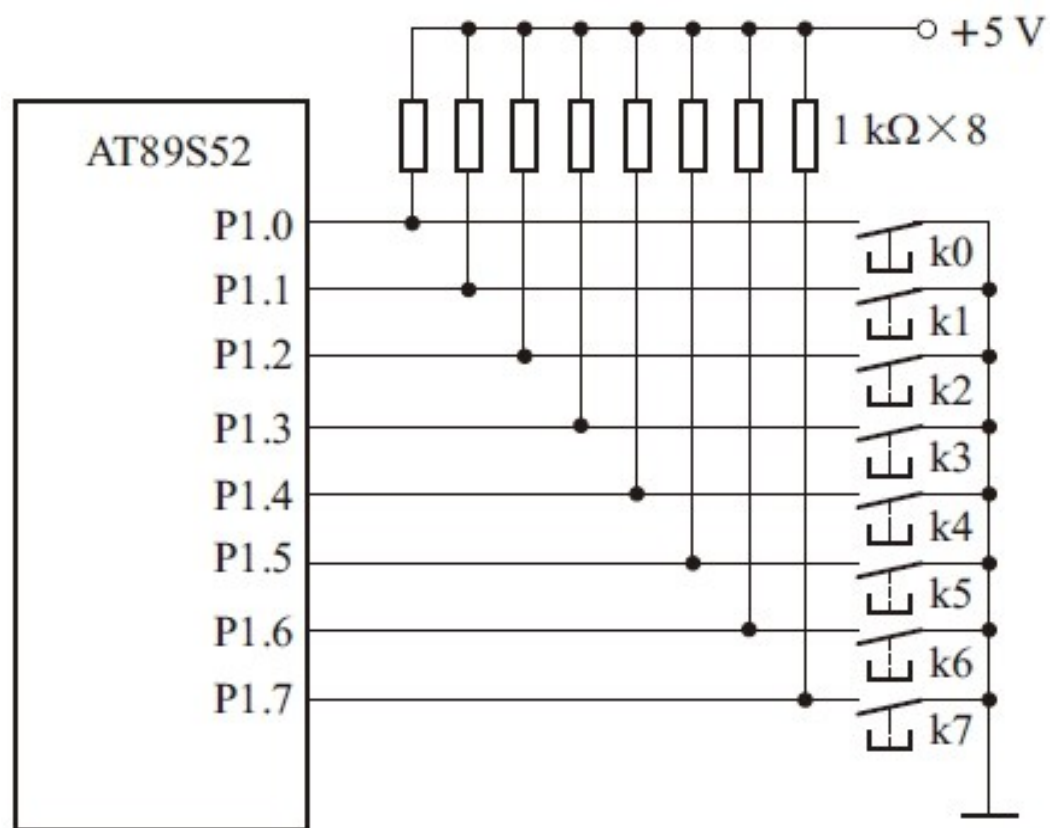


图5-28 独立式键盘的接口电路

1.独立式键盘的查询工作方式

【例5-11】对图5-28所示独立式键盘，用查询方式实现键盘扫描，根据按下不同按键，对其进行处理。扫描程序如下：

```
#include<reg51.h>
void key_scan(void)
{
    unsigned char keyval
do
{
    P1=0xff;           // P1口为输入
    keyval=P1;          //从P1口读入键盘状态
    keyval=~ keyval;    //键盘状态求反
```

switch(keyval)

{

case 1:;	//处理按下的k1键, “.....”为处理程序
break;	//跳出switch语句
case 2:;	//处理按下的k2键
break;	//跳出switch语句
case 4:;	//处理按下的k3键
break;	//跳出switch语句
case 8:;	//处理按下的k4键
break;	//跳出switch语句

```
case 16: .....;           //处理按下的k5键
    break;                 //跳出switch语句
case 32: .....;           //处理按下的k6键
    break;                 //跳出switch语句
case 64: .....;           //处理按下的k7键
    break;                 //跳出switch语句
case 128: .....;          //处理按下的k8键
    break;                 //跳出switch语句
default:
    break;                 //无按下键处理
}
}
while(1);
}
```

下面看用**Proteus**虚拟仿真独立式键盘实际案例。

【例5-12】单片机与**4个独立按键k1~k4**及**8个LED指示灯**的一个独立式键盘。4个按键接在**P1.0~P1.3**引脚，**P3**口接**8个LED**指示灯，控制**LED**指示灯亮与灭，原理电路见**图5-29**。

按下**k1**键，**P3**口**8个LED**正向（由上至下）流水点亮；

按下**k2**键，**P3**口**8个LED**反向（由下而上）流水点亮；

按下**k3**键，高、低**4个LED**交替点亮；

按下**k4**键，**P3**口**8个LED**闪烁点亮。

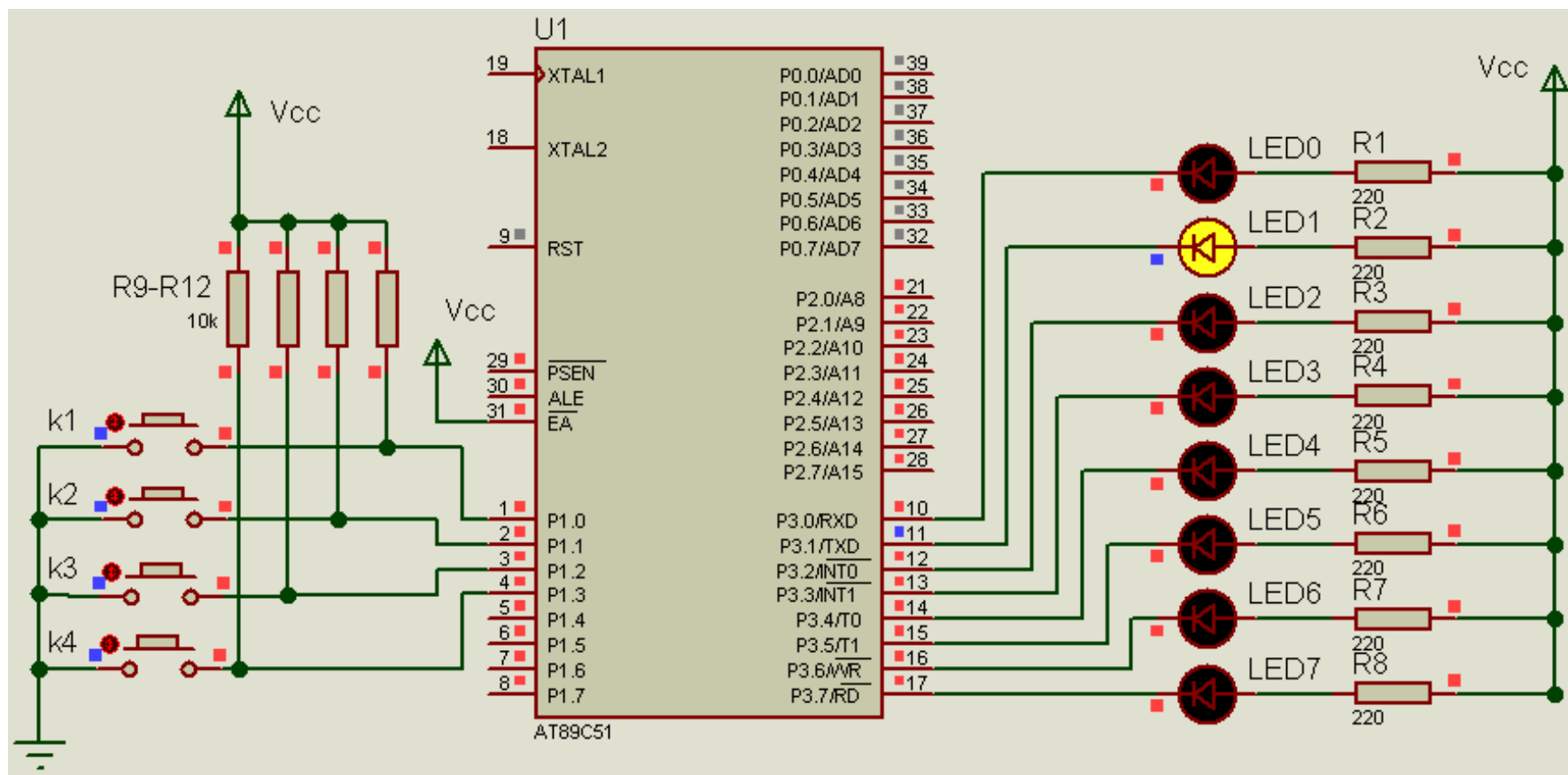


图5-29

虚拟仿真的独立式键盘的接口电路

由于本案例中的4个按键分别对应4个不同的点亮功能，且具有不同的按键值“keyval”，具体如下：

- 按下K1键时，keyval=1
- 按下K2键时，keyval=2
- 按下K3键时，keyval=3
- 按下K4键时，keyval=4

本独立式键盘工作原理如下：

(1) 首先判断是否有按键按下。将接有4个按键的P1口低4位（P1.0~P1.3）写入“1”，使P1口低4位为输入状态。然后读入低4位的电平，只要有一位不为“1”，则说明有键按下。读取方法：



```
P1=0xff;
```

```
if((P1&0x0f)!=0x0f);    //读P1口低4位按键值，按位“与”运算后结果非  
                        // 0x0f，表明低4位必有1位是“0”，说明有键按下
```

(2) 按键去抖动。当判别有键按下时，调用软件延时子程序，延时约**10ms**后再进行判别，若按键确实按下，则执行相应的按键功能，否则重新开始进行扫描。

(3) 获得键号。确认有键按下时，可采用扫描方法，来判哪个键按下，并获取键值。

首先Keil μ Vision3建立工程，再建立源程序“*.c”文件。

参考程序：

```
#include<reg51.h>
sbit S1=P1^0;           //将S1位定义为P1.0引脚
sbit S2=P1^1;           //将S2位定义为P1.1引脚
sbit S3=P1^2;           //将S3位定义为P1.2引脚
sbit S4=P1^3;           //将S4位定义为P1.3引脚
unsigned char keyval;    //定义键值储存变量单元

void main(void)         //主函数
{
    keyval=0;            //键值初始化为0
    while(1)
```



```
{
```

```
key_scan();
```

```
//调用键盘扫描函数
```

```
switch(keyval)
```

```
{
```

```
case 1:forward();
```

```
//键值为1，调用正向流水点亮函数
```

```
break;
```

```
case 2:backward();
```

```
//键值为2，调用反向流水点亮函数
```

```
break;
```

```
case 3:Alter();
```

```
//键值为3，调用高、低4位交替点亮函数
```

```
break;
```

```
case 4:blink ();
```

```
//键值为4，调用闪烁点亮函数
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
void key_scan(void)
```

//函数功能：键盘扫描

```
{
```

```
    P1=0xff;
```

```
    if((P1&0x0f)!=0x0f)
```

//检测到有键按下

```
{
```

```
    delay10ms();
```

//延时10ms再去检测

```
    if(S1==0)
```

//按键k1被按下

```
    keyval=1;
```

```
    if(S2==0)
```

//按键k2被按下

```
    keyval=2;
```

```
    if(S3==0)
```

//按键k3被按下

```
    keyval=3;
```

```
    if(S4==0)
```

//按键k4被按下

```
    keyval=4;
```

```
}
```

```
}
```

```
void forward(void)
```

```
{
```

```
    P3=0xfe;
```

```
    led_delay();
```

```
    P3=0xfd;
```

```
    led_delay();
```

```
    P3=0xfb;
```

```
    led_delay();
```

```
    P3=0xf7;
```

```
    led_delay();
```

```
    P3=0xef;
```

```
    led_delay();
```

```
    P3=0xdf;
```

```
    led_delay();
```

```
    P3=0xbf;
```

```
    led_delay();
```

```
    P3=0x7f;
```

```
//函数功能：正向流水点亮LED
```

```
//LED0亮
```

```
//LED1亮
```

```
//LED2亮
```

```
//LED3亮
```

```
//LED4亮
```

```
//LED5亮
```

```
//LED6亮
```

```
//LED7亮
```

```
led_delay();  
}
```

```
void backward(void)
```

```
//函数：反向流水点亮LED
```

```
{
```

```
    P3=0x7f;
```

```
//LED7亮
```

```
    led_delay();
```

```
    P3=0xbf;
```

```
//LED6亮
```

```
    led_delay();
```

```
    P3=0xdf;
```

```
//LED5亮
```

```
    led_delay();
```

```
    P3=0xef;
```

```
//LED4亮
```

```
    led_delay();
```

```
    P3=0xf7;
```

```
//LED3亮
```

```
    led_delay();
```

```
    P3=0xfb;
```

```
//LED2亮
```

```
    led_delay();
```

```
    P3=0xfd;
```

```
//LED1亮
```



```
led_delay();  
    P3=0xfe;                //LED0亮  
    led_delay();  
}
```

```
void Alter(void)            //函数：交替点亮高4位与低4位LED  
{  
    P3=0x0f;  
    led_delay();  
    P3=0xf0;  
    led_delay();  
}
```

```
void blink(void)           //函数：闪烁点亮LED  
{  
    P3=0xff;  
    led_delay();  
}
```

```
P3=0x00;  
    led_delay();  
}
```

```
void led_delay(void)  
{  
    unsigned char i, j;  
    for (i=0; i<220; i++)  
        for (j=0; j<220; j++)  
            ;  
}
```

//函数：延时

```
void delay10ms(void)  
{  
    unsigned char i, j;  
    for (i=0; i<100; i++)  
        for (j=0; j<100; j++)  
            ;  
}
```

//函数：软件消抖延时10ms

2. 独立式键盘的中断扫描方式

采用查询方式独立式键盘接口设计与程序设计，不管是否有按键按下，都必须去扫描键盘。为提高单片机扫描键盘的工作效率，可采用中断扫描方式，只有在键盘有键按下时，才进行扫描与处理。可见中断扫描方式的键盘实时性强，工作效率高。

【例5-13】设计一采用中断扫描方式独立式键盘，只有在键盘有按键按下时，才进行处理，接口电路见图5-30。当键盘中有键按下时，8输入与非门74LS30输出经过74LS04反相后向单片机外中断请求输入引脚INT0*发出低电平中断请求信号，单片机响应中断，进入外部中断的中断函数，在中断函数中，判断按键是否真按下。如确实按下，则把标志keyflag置1，并得到按下按键键值，然后从中断返回，根据键值跳向该键的处理程序。

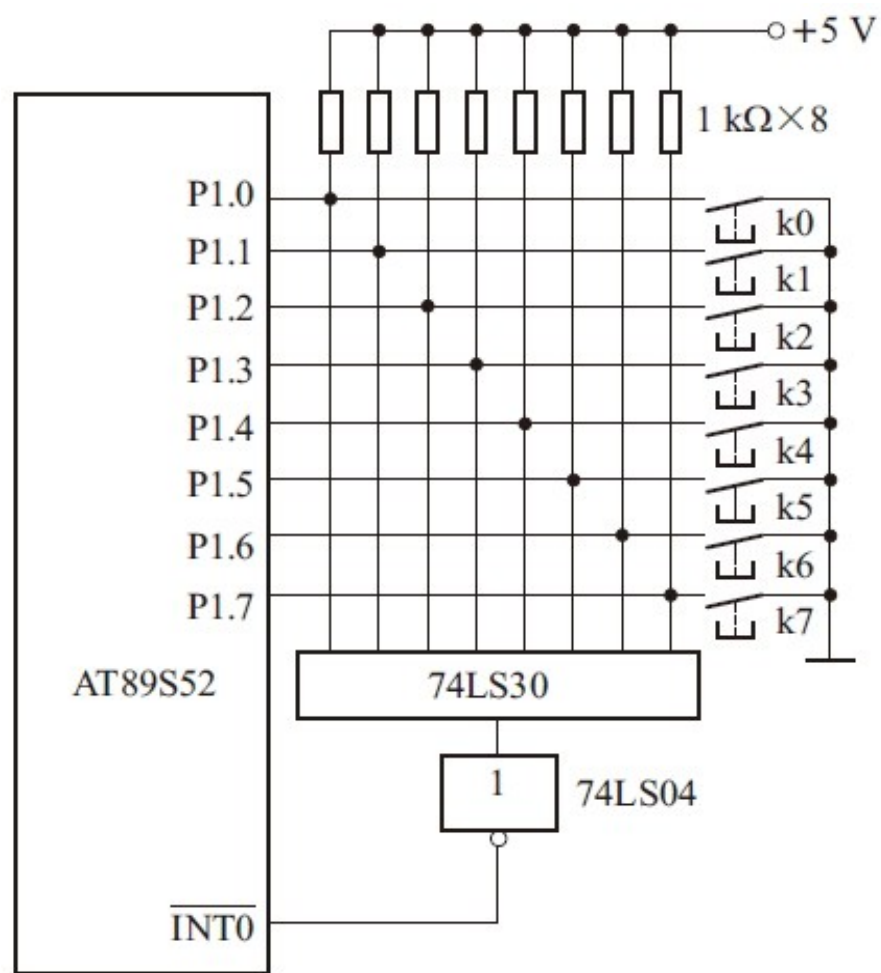


图5-30 中断扫描方式的独立式键盘的接口电路

参考程序如下：

```
#include<reg51.h>
#include<absacc.h>
#define uchar unsigned char
#define TRUE 1
#define FALSE 0
bit keyflag;           // keyflag为按键按下的标志位
uchar keyval;          // keyval为键值

void delay10ms(void);  //软件延时10ms函数

void main(void)
{
    IE=0x81;           //总中断允许EA=1，允许中断
    IP=0x01;           //设置外中断0为高优先级
    keyflag=0;         //设置按键按下标志为0
    do
    {
```

```
if (keyflag)
```

//如果按键按下标志keyflag =1, 则有键按下

```
{
```

```
    keyval=~keyval;
```

//键值取反

```
    switch (keyval)
```

//根据按下键的键值进行分支跳转

```
    {
```

```
        case 1: ...;
```

//处理0号键

```
            break;
```

```
        case 2: ...;
```

//处理1号键

```
            break;
```

```
        case 4: ...;
```

//处理2号键

```
            break;
```

```
        case 8: ...;
```

//处理3号键

```
            break;
```

```
        case 16: ...;
```

//处理4号键

```
            break;
```

```
        case 32: ...;
```

//处理5号键

```

        break;
        case 64: ...;           //处理6号键
        break;
        case 128: ...;         //处理7号键
        break;
        default;
        break;                 //无效按键，例如多个键同时按下
    }
    keyflag=0;                 //清按键按下标志
    }
} while(TRUE);
}

```

```

void int0( ) interrupt 0      //有键按下，则执行的中断函数
{
    uchar reread_key;        // reread_key为重读键值变量;
    IE=0x80;                 // 屏蔽中断
}

```

```
keyflag=0;           // 把按键按下标志keyflag清0
P1=0xff;             // 向P1口写1，设置P1口为输入
keyval=P1;           // 从P1口读入键盘的状态
delay10ms(void);     // 延时10ms
reread_key=P1;       // 再次从P1口读键盘状态，并存reread_key中
if (keyval==reread_key) // 比较两次读取的键值，如相同，说明键按下
{
    keyflag=1;        // 按键按下标志keyflag为1
}
IE=0x81;             // 重新允许中断
}
```


程序说明：程序中用到了**外部中断INT0***，当没有按键按下时，标志**keyflag=0**，程序一直执行“**do{ }while ()**”循环。当有键按下时，则**74LS04**输出端产生低电平，向单片机**INT0***脚发中断请求信号，单片机响应中断，执行中断函数。

如果确实按键按下，在中断函数中把**keyflag**置1，并得到键值。当执行完中断函数后，再进入“**do{ }while ()**”循环，此时由于“**if(keyflag)**”中的**keyflag=1**，则可根据键值“**keyval**”，采用“**switch(keyval)**”分支语句，进行按下按键的处理。

5.7.3 矩阵式键盘接口设计案例

矩阵式（也称行列式）键盘用于按键数目较多的场合，由行线和列线组成，按键位于行、列交叉点上，见图5-31，一个 4×4 的行、列结构可以构成一个16个按键的键盘，只需要一个8位的并行I/O口即可。

如果采用 8×8 的行、列结构，可以构成一个64按键的键盘，只需要两个并行I/O口即可。

在按键数目较多场合，矩阵式键盘要比独立式键盘节省较多I/O口线。

在图5-31中行线通过上拉电阻接+5V，当无键按下时，行线为高电平，而有键按下时，则对应的行线和列线短接，行线电平状态将由与此行线相连的列线电平决定。

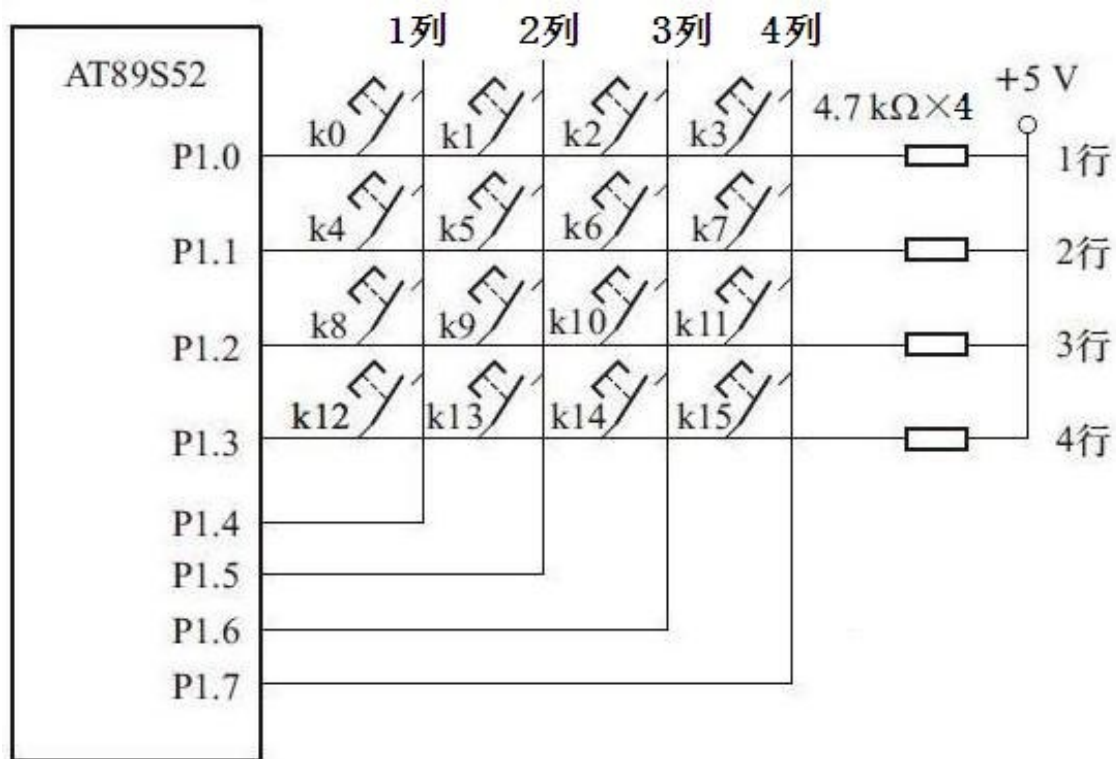


图5-31 矩阵式（行列式）键盘的接口电路

键盘矩阵中**无按键按下**时，行线处于高电平状态；当**有按键按下**时，行线电平状态将由与此行线相连的列线的电平决定。列线的电平如果为低，则行线电平为低；列线的电平如果为高，则行线的电平也为高，这一点是识别矩阵式键盘是否有按键按下的关键所在。由于矩阵式键盘中行、列线为多键共用，各按键均影响该键所在行和列的电平，因此各按键彼此将相互发生影响，所以必须将行、列线信号配合，才能确定闭合键的位置。下面讨论矩阵式键盘按键的扫描法的工作原理。

识别矩阵键盘有无键被按下，可分两步进行：第**1**步，首先识别键盘有无键按下；第**2**步，如有键被按下，识别出具体的键号。

下面以**图5-31**所示的**K3**键被按下为例，说明扫描法识别此键的过程。

第1步，识别键盘有无键按下。首先把所有列线均置为0电平，然后检查各行线电平是否都为高电平，如果不全为高电平，说明有键按下，否则说明无键被按下。例如，当K3键按下时，第1行线电平为低电平，但还不能确定是K3被按下，因为如果同一行的K2、K1或K0之一被按下，行线也为低电平。所以，只能得出第1行有键被按下的结论。

第2步，识别出哪个按键被按下。采用逐行扫描法，在某一时刻只让1条列线处于低电平，其余所有列线处于高电平。当第1列为低电平，其余各列为高电平时，因为是K3被按下，所以第1行的行线仍处于高电平状态；而当第2列为低电平，其余各列为高电平时，同样也会发现第1行的行线仍处于高电平状态；直到让第4列为低电平，其余各列为高电平时，此时第1行的行线电平变为低电平，据此，可判断第1行第4列交叉点处的按键，即K3被按下。

与独立式键盘类似，常见的矩阵式键盘扫描的工作方式也分为**查询方式**和**中断方式**。

1. 矩阵式键盘的查询方式扫描

下面介绍矩阵式键盘的查询方式的设计。

【例5-14】单片机的P1.7~P1.4接4×4矩阵键盘的行线，P1.3~P1.0接矩阵键盘的列线，键盘各按键的编号如图5-32所示，使用数码管来显示4×4矩阵键盘中按下键的键号。数码管的显示由P2口控制，当矩阵键盘的某一键按下时，在数码管上显示对应的键号。例如，1号键按下时，数码管显示“1”；E键按下时，数码管显示“E”等等。

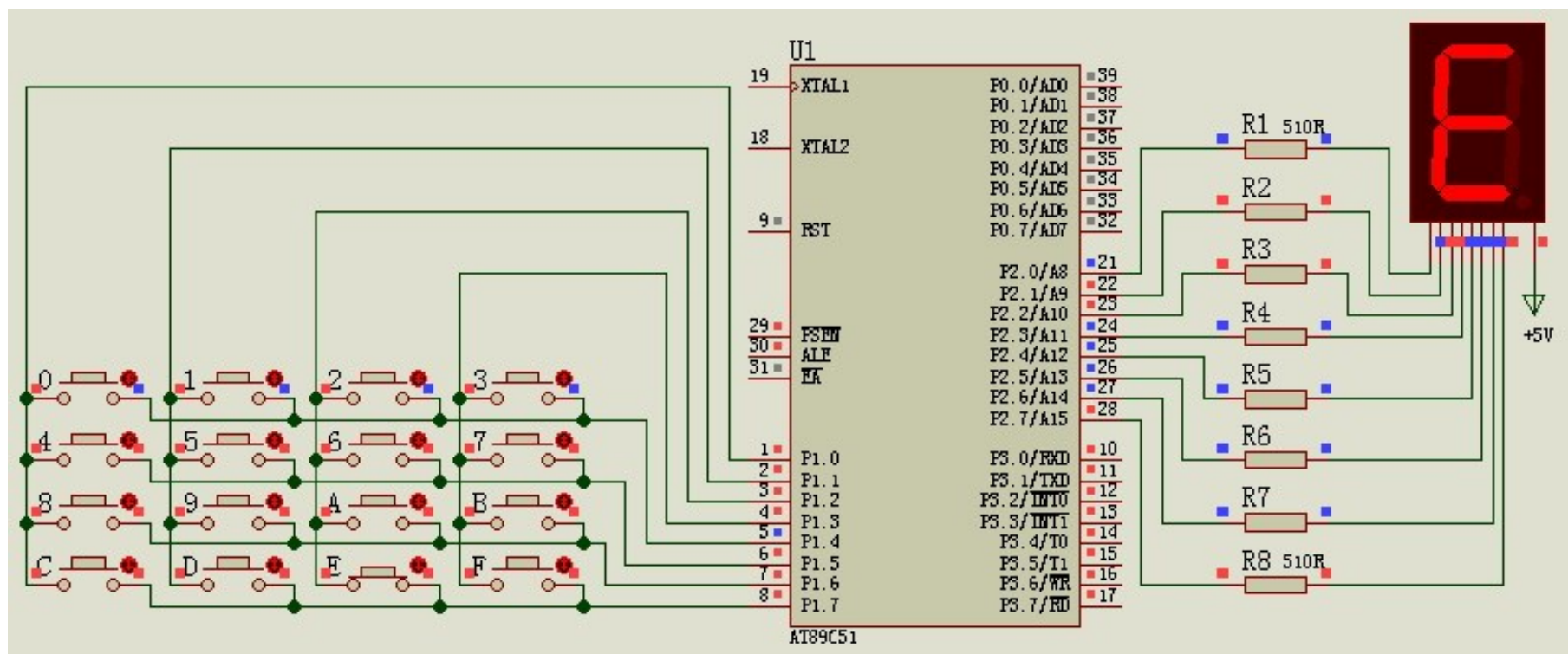


图5-32 数码管显示4×4矩阵键盘键号的原理电路

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
sbit L1=P1^0;
```

// 定义键盘的4列线

```
sbit L2=P1^1;
```

```
sbit L3=P1^2;
```

```
sbit L4=P1^3;
```

```
uchar dis[16]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,
```

```
0x80,0x90,0x88,0x83,0xc6,0xa1,0x86, 0x8e};
```

//共阳极数码管字符0~F

// 对应的段码

```
unsigned int time;
```

```
delay(time)
```

//延时函数

```
{
```

```
    unsigned int i;
```

```
    for(j=0;j<time;j++)
```



```
{  
}  
}
```

main()

//主函数

```
{  
    uchar temp;  
    uchar k,i;  
  
    while(1)  
    {  
        P1=0xef; //行扫描初值, P1.4=0, P1.5~ P1.7=1  
        for(i=0;i<=3;i++) //按行扫描, i为行变量, 一共4行  
        {  
            if(L1==0) P2=dis[i*4+0]; //判第1列是否有键按下,若有,键号可  
                                     //能为0,4,8,C, 键号的段码送显示  
            if(L2==0) P2=dis[i*4+1]; //判第2列是否有键按下,若有,键号可能为  
                                     //1,5,9,d, 键号的段码送显示  
        }  
    }  
}
```

```
if(L3==0) P2=dis[i*4+2];
```

```
P2=dis[i*4+3];
```

```
delay(500);
```

```
temp=P1;
```

```
temp=temp|0x0f;
```

```
temp=temp<<1;
```

```
temp=temp|0x0f;
```

```
P1=temp;
```

```
}
```

```
}
```

```
}
```

//判第3列是否有键按下,若有,键号可能为

//2,6,A,E, 键号的段码送显示if(L4==0)

//判第4列是否有键按下,若有,键号可

//3,7,b,F, 键号的段码送显示

//延时

//读入P1口的状态

//使P1.3~P1.0为输入

// P1.7~P1.4左移1位, 准备下一行扫描

//移位后, 置P1.3~P1.0为1, 保证其仍为输入

//行扫描值送P1口, 为下一行扫描做准备

程序说明：本例的关键是如何获取键号。具体采用了逐行扫描，先驱动行**P1.4=0**，然后依次读入各列的状态，**P1.4**脚对应的行变量**i=0**，**P1.5**脚对应的行变量**i=1**，**P1.6**脚对应的行变量**i=2**，**P1.7**脚对应的行变量**i=3**。

假设**4号键**按下，此时**4号键**所在的**P1.5**脚对应的行变量**i=1**，又**L2=0**（**P1.5=0**），执行语句“**if (L2==0) P2=dis [i*4+1]**”后，**i*4+1=5**，从而查找到字型码数组**dis[]**中显示“**4**”的段码“**0x99**”，把段码“**0x99**”送**P2**口，从而驱动数码管显示“**4**”。

2. 矩阵式键盘的中断方式扫描

在查询扫描方式中，不管键盘上有无按键按下，程序总要扫描键盘，而在实际应用中，键盘并不经常工作，因此单片机经常处于空扫描状态，工作效率较低。为提高工作效率，可采用中断扫描方式，即当键盘上有键闭合时产生中断请求，单片机响应中断后，转去执行中断服务程序，判断闭合键的键号，并作相应的处理。

【例5-15】图5-33是一种常用的中断扫描式的矩阵键盘接口电路。键盘的列线与P1口的P1.4~P1.7相连，是扫描输入线，键盘的行线与P1口的P1.0~P1.3相连，是扫描输出线，图中的与门四输入端与行线相连，与门的输出产生按键中断请求信号。

工作过程如下：程序首先把所有列线置为低电平，然后检测各行线的状态，若所有行线均为高电平，说明键盘中无键按下。当有键按下时，相应行线为低电平，与门输出也为低电平，向单片机申请中断，单片机响应中断，转去执行键盘扫描子程序。键盘扫描原理与查询扫描相同。

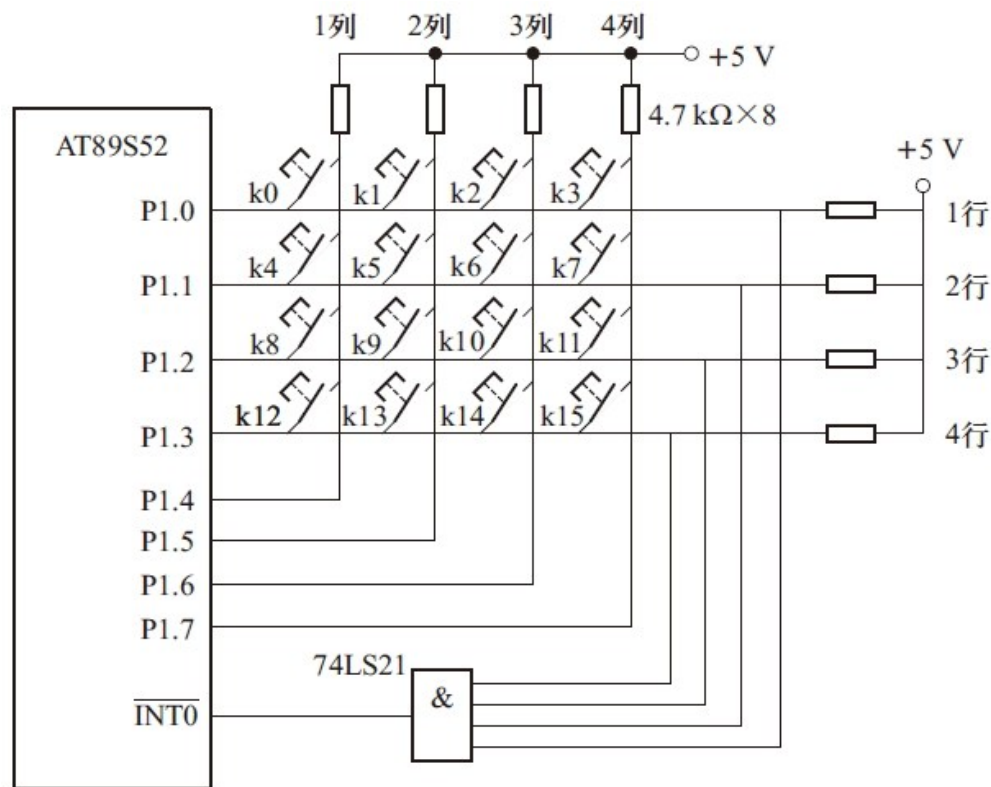


图5-33 中断方式的4×4矩阵键盘的接口电路

参考程序如下：

```
#include<reg52.h>
```

```
void main(void)
```

```
{
```

```
    IT0=1;
```

```
    IP=0x01;
```

```
    P1=0x0f;
```

```
    EA=1;
```

```
    EX0=1;
```

```
    do{ }while(1);
```

```
}
```

```
void int0( ) interrupt 0 using 1
```

```
{
```

```
    .....
```

```
}
```

```
//设置外中断0跳沿触发  
//设置外中断0高优先级  
//设置P1.0~P1.3为输入,  
//置P1.4~P1.7输出为0
```

```
//总中断允许  
//允许外中断0中断  
//等待中断
```

```
//有键按下，则执行INT0*中断函数
```

```
//键盘扫描程序
```

5.7.4 键盘扫描工作方式的选择

单片机系统运行在忙于其他各项工作任务时，如何来兼顾键盘的输入，这取决于键盘扫描的工作方式。键盘扫描工作方式选取的原则是，既要保证及时响应按键操作，又不要过多占用单片机执行其他任务的工作时间。通常，键盘的扫描工作方式有3种：查询扫描、定时扫描和中断扫描。

1. 查询扫描

查询扫描方式是利用单片机空闲时，调用键盘扫描子程序，反复扫描键盘，来响应键盘的输入请求，但是如果单片机的查询频率过高，虽能及时响应键盘的输入，但也会影响其他任务的进行。如果查询的频率过低，有可能出现键盘输入的漏判现象。所以要根据单片机系统的繁忙程度和键盘的操作频率，来调整键盘扫描的频率。

2. 定时扫描

单片机对键盘的扫描也可每隔一定的时间对键盘扫描一次，即定时扫描。这种方式中，通常利用单片机内的定时器产生的定时中断，进入中断子程序后对键盘进行扫描，在有键按下时识别出按下的键，并执行相应键的处理程序。由于每次按键的时间一般不会小于100ms，所以为了不漏判有效的按键，定时中断的周期一般应小于100ms。

3. 中断扫描方式

为进一步提高单片机扫描键盘的工作效率，可采用中断扫描方式，即键盘只有在有按键按下时，才会向单片机发出中断请求信号。单片机响应中断，执行键盘扫描中断服务子程序，识别出按下的按键，并跳向该按键的处理程序。如果无键按下，单片机将不理睬键盘。该方式的优点是，只有按键按下时，才进行处理，所以其实时性强，工作效率高。

5.7.5 单片机与专用键盘/显示器芯片HD7279的接口设计

单片机通过专用可编程键盘/显示器接口芯片与键盘/显示器连接，直接得到闭合键键号（编码键盘），还可省去编写键盘/显示器动态扫描程序以及键盘去抖动程序的繁琐工作。

1. 各种专用的键盘/显示器接口芯片简介

常见的专用键盘/显示器芯片有：**HD7279**、**ZLG7289A**（周立功公司），**CH451**（南京沁恒公司）等。

专用键盘/显示器接口芯片**HD7279**与单片机间采用串行连接，功能强，具有一定的抗干扰能力，可控制与驱动8位LED数码管以及实现8×8的键盘管理。由于其外围电路简单，价格低廉，目前在键盘/显示器接口的设计中得到较为广泛的应用。

2. HD7279A简介

HD7279A能同时驱动8个共阴极LED数码管（或64个独立的LED发光二极管）和8×8的编码键盘。对LED数码管采用的是动态自动扫描的循环显示方式。

HD7279A为**28**引脚双列直插（**DIP**）式封装，单一**+5V**供电，其引脚见图**5-34**，引脚功能如表**5-7**所列。

DIG0~DIG7为位驱动输出端，可分别连接**8**只**LED**数码管的共阴极；段驱动输出端**SA~SG**分别连接至**LED**数码管的**a~g**段的阳极，而**DP**引脚连至小数点**dp**的阳极。**DIG0~DIG7**、**DP**和**SA~SG**还分别是**8×8**矩阵键盘的列线和行线，完成对键盘的译码和键号识别。**8×8**矩阵键盘中按下键的键号可用读键盘命令读出，键号的范围是**00H~3FH**。

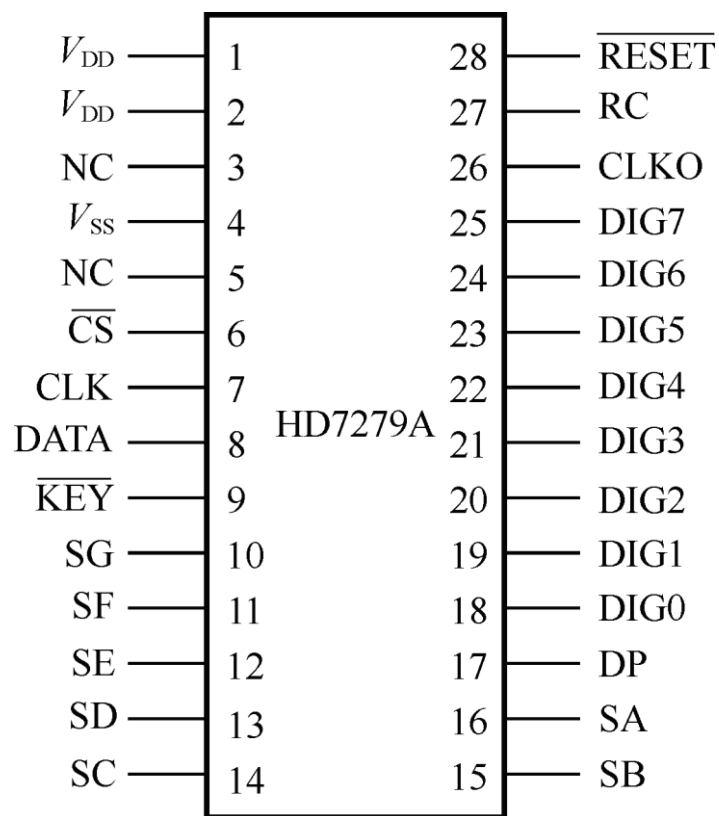


图5-33 HD7279A的引脚

HD7279A芯片与单片机连接仅需4条信号线：CS*、DATA、CLK和

KEY*, 各引脚功能如下：

表5-7 HD7279A的引脚功能

引 脚	名 称	说 明
1, 2	V_{DD}	正电源 (+5V)
3, 5	NC	悬空
4	V_{SS}	地
5		片选信号
7	CLK	同步时钟输入端
8	DATA	串行数据写入/读出端
9		按键信号输出端
10~15	SG~SA	数码管的g~a段驱动输出
17	DP	小数点驱动输出端
18~25	DIG0~DIG7	数码管的位驱动输出
25	CLKO	振荡信号输出端
27	RC	RC振荡器连接端
28		复位端

- **CS***: 当单片机访问**HD7279A**芯片（写入命令、显示数据、位地址、段地址或读出键值等）时，应将置低电平。
- **DATA**: 串行数据输入/输出端，当单片机向**HD7279A**发送数据时，**DATA**为输入端；当单片机从**HD7279A**读键值时，**DATA**为输出端。
- **CLK**: 数据串行传送的同步时钟输入端，时钟的上升沿将数据写入**HD7279A**中或从**HD7279A**中读出数据。
- **KEY***: 按键信号输出端，无键按下为高电平，有键按下为低电平，且一直保持到该键释放为止。
- **RESET***: 复位端，通常该端接+5V。若对可靠性要求较高，则可外接复位电路，或直接由单片机控制。
- **RC**: 该脚外接振荡元件，其典型值为 $R=1.5k\Omega$ ， $C=15pF$ 。
- **NC**: 悬空。

3. AT89S52单片机与HD7279A接口设计

图5-35为单片机通过HD7279A控制8个数码管及64键矩阵键盘的接口电路。晶振频率为12MHz。上电后，HD7279A约经过15~18ms时间才进入工作状态。

单片机通过P1.3脚检测KEY*脚电平，来判断键盘矩阵中是否有按键按下。HD7279A采用动态循环扫描方式，如普通数码管亮度不够，可采用高亮度或超高亮度数码管。

图5-35所示的电路中，HD7279的3、5引脚悬空。

有关HD7279的程序设计，请读者参阅有关资料。

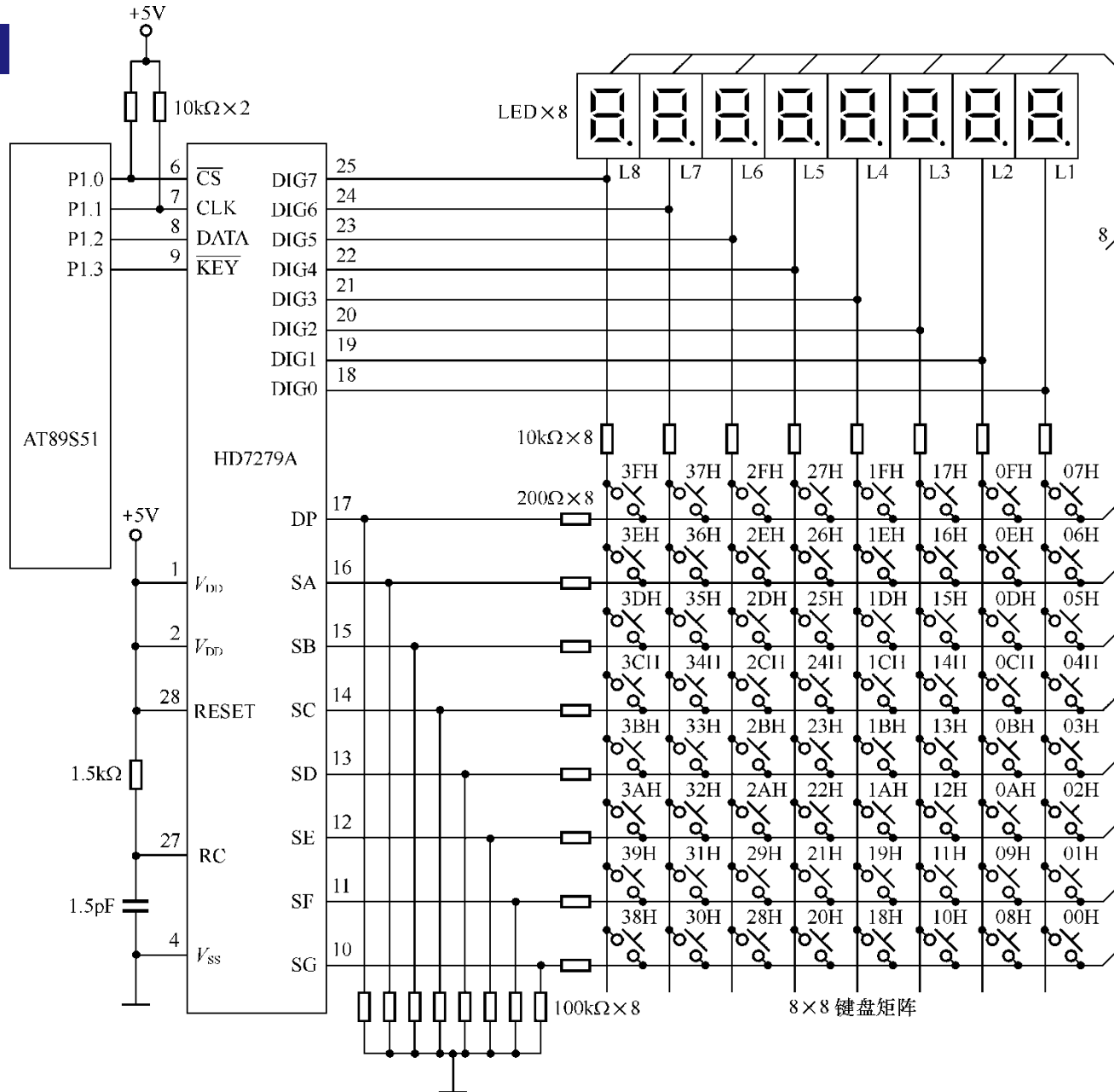


图5-35 AT89S52单片机与HD7279A的接口电路



第6章 AT89S52单片机的中断系统



【内容概要】 本章介绍**AT89S52**单片机片内功能部件中断系统的硬件结构和工作原理。**AT89S52**的中断系统能够实时地响应片内功能部件和外围设备发出的中断请求并进入中断服务子程序进行处理。通过本章学习，读者应重点掌握与中断系统有关的特殊功能寄存器及中断系统的应用特性，应能熟练地进行中断系统的初始化编程以及中断函数的设计。

在计算机（单片机）系统中，都具有**对外界随机发生的事件**的**作出及时响应并处实时处理**的功能，这是靠中断技术来实现的。

6.1 单片机中断技术概述

中断技术主要用于实时监测与控制，要求单片机能及时地响应中断请求源提出的服务请求，并快速响应与及时处理。

图6-1显示单片机对外设中断服务请求整个中断响应和处理过程。

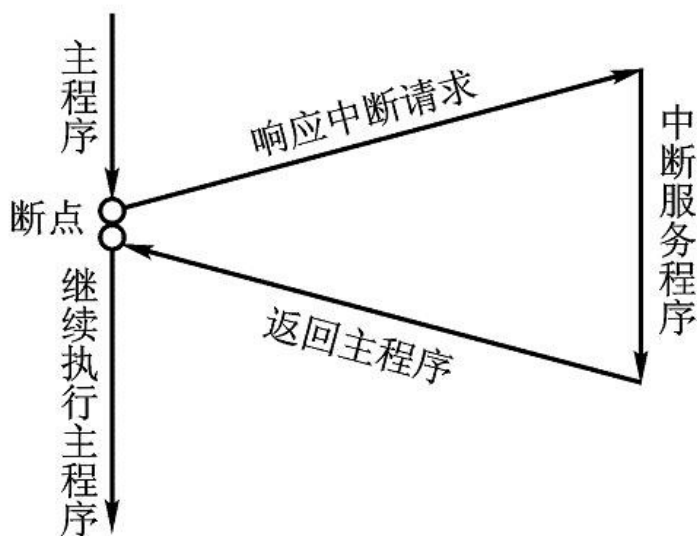


图6-1 中断响应和处理过程

当某个中断请求源发出中断请求时，并且中断请求被允许的话，单片机暂时中止当前正在执行的主程序，转到中断处理程序处理中断服务请求。中断服务请求处理完后，再回到原来被中止的程序之处（断点），继续执行被中断的主程序。

如没有单片机中断系统，大量时间可能会浪费在查询是否有服务请求的定时查询操作上，即不论是否有服务请求，都必须去查询。采用中断技术完全消除查询方式的等待，大大提高单片机工作效率和实时性。

6.2 AT89S52中断系统结构

中断系统结构见图6-2。中断系统有6个中断请求源（简称中断源），2个中断优先级，可实现2级中断服务程序嵌套。每一中断源可用软件独立控制为允许中断或关闭中断状态；每一个中断源的优先级均可用软件设置。

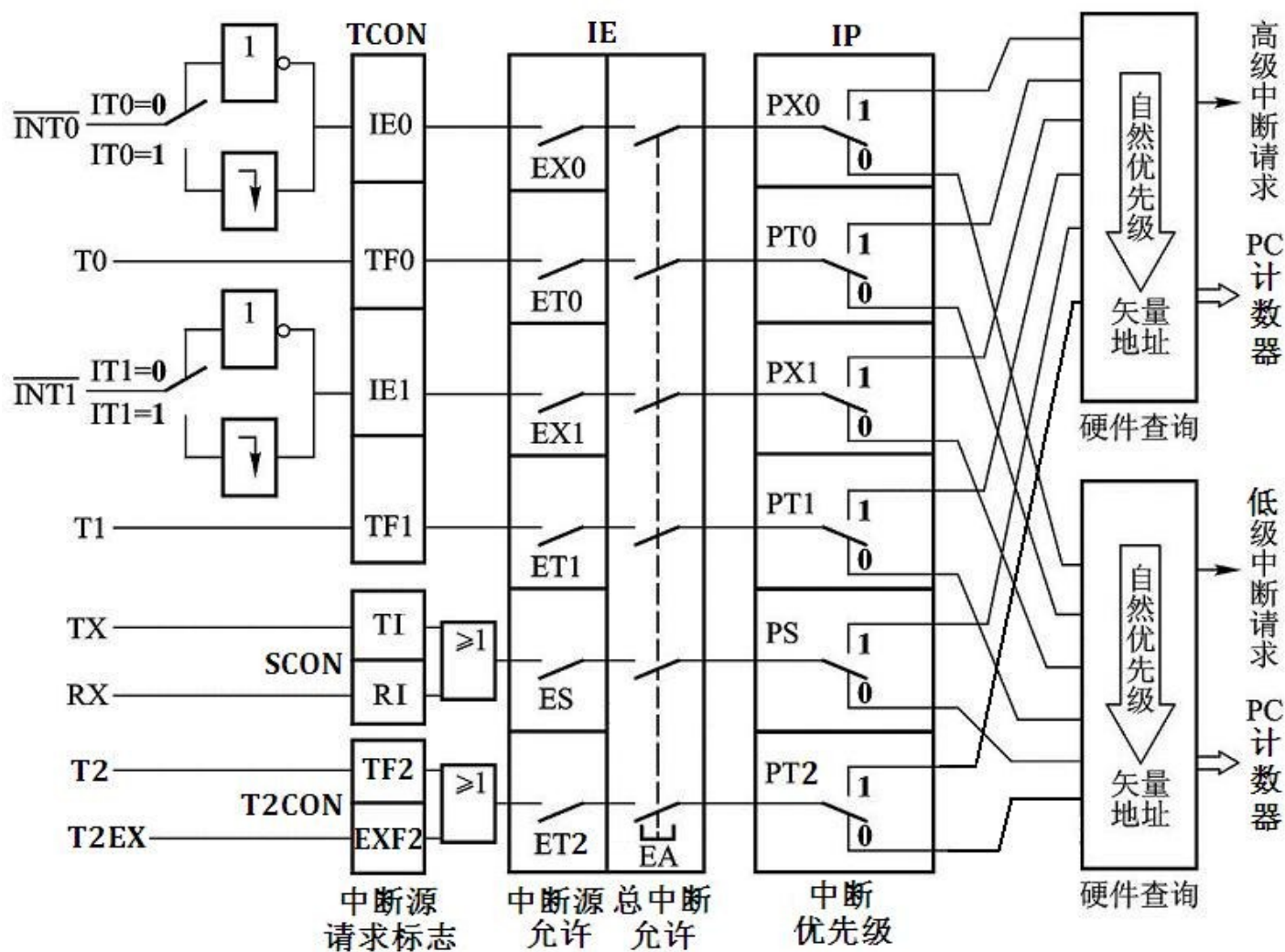


图6-2 AT89S52的中断系统结构

6.2.1 中断请求源

由图6-2，中断系统共有6个中断请求源，它们是：

- (1) INT0*—外部中断请求0，外部中断请求信号（低电平或负跳变有效）由INT0*引脚输入，中断请求标志为IE0。
- (2) INT1*—外部中断请求1，外部中断请求信号（低电平或负跳变有效）由INT1*引脚输入，中断请求标志为IE1。
- (3) 定时器/计数器T0计数溢出的中断请求，标志为TF0。
- (4) 定时器/计数器T1计数溢出的中断请求，标志为TF1。
- (5) 串行口中断请求，标志为发送中断TI或接收中断RI。
- (6) 定时器/计数器T2的中断请求源，含有计数溢出（TF2）和“捕捉”（EXF2）两种中断请求标志，经或门共用一个中断向量。两种中断触发是由T2的两种不同工作方式决定的。

6.2.2 中断请求标志寄存器

6个中断请求源的中断请求标志分别由特殊功能寄存器TCON和SCON相应位锁存（见图6-2）。

1. TCON寄存器

为定时器/计数器的控制寄存器，字节地址为88H，可位寻址。既包括定时器/计数器T0、T1溢出中断请求标志位TF0和TF1，也包括两个外部中断请求的标志位IE1与IE0，还包括两个外部中断请求源的中断触发方式选择位。TCON格式见图6-3。

	D7	D6	D5	D4	D3	D2	D1	D0	
TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	88H
位地址	8FH	—	8DH	—	8BH	8AH	89H	88H	

图6-3 特殊功能寄存器**TCON**的格式

TCON寄存器中与中断系统有关各标志位功能如下：

(1) **TF1**—定时器/计数器**T1**的溢出中断请求标志位。

当启动T1计数后，T1从初值开始加1计数，当最高位产生溢出时，硬件置TF1为“1”，向CPU申请中断，响应TF1中断时，TF1标志硬件自动清“0”，TF1也可由软件清“0”。

(2) **TF0**—定时器/计数器**T0**溢出中断请求标志位，与TF1类似。

(3) **IE1**—外部中断请求**1**中断请求标志位。

(4) **IE0**—外部中断请求**0**中断请求标志位，与IE1类似。

(5) IT1—选择外中断请求1为跳沿触发还是电平触发方式。

0—电平触发方式，加到INT0*脚上的外中断请求输入信号为低电平有效，并把IE1置“1”。转向中断服务程序时，则由硬件自动把IE1清“0”。

1—跳沿触发方式，加到INT1*脚上的外中断请求输入信号从高到低的负跳变有效，并把IE1置“1”。转向中断服务程序时，则由硬件自动把IE1清“0”。

(6) IT0—选择外中断请求0为跳沿触发方式还是电平触发方式，与IT1类似。

当AT89S52复位后，TCON被清“0”，6个中断源的中断请求标志均为0。

TR1（D6位）、TR0（D4位）这2位与中断系统无关，仅与定时器/计数器T1和T0有关，将在第7章定时器/计数器中介绍。

2. SCON寄存器

串行口控制寄存器，字节地址为98H，可位寻址。SCON的低二位锁存串口的发送中断和接收中断的中断请求标志TI和RI，格式见图6-4。

	D7	D6	D5	D4	D3	D2	D1	D0	
SCON	—	—	—	—	—	—	TI	RI	98H
位地址	—	—	—	—	—	—	99H	98H	

图6-4 SCON中的中断请求标志位

SCON标志位功能:

(1) **TI**—串口发送中断请求标志位。CPU将1字节的数据写入串口的发送缓冲器SBUF时，就启动一帧串行数据的发送，每发送完一帧串行数据后，硬件使**TI自动置“1”**。CPU响应串口发送中断时，并不清除TI中断请求标志，TI标志**必须在中断服务程序中用指令对其清“0”**。

(2) **RI**—串行口接收中断请求标志位。在串口接收完一个串行数据帧，硬件自动使RI中断请求标志置“1”。CPU在响应串口接收中断时，RI标志并不清“0”，须**在中断服务程序中用指令对RI清“0”**。

3. 定时器2的控制寄存器T2CON

特殊功能寄存器T2CON的字节地址为C8H，可位寻址，位地址为C8H～CFH。格式见图6-5。

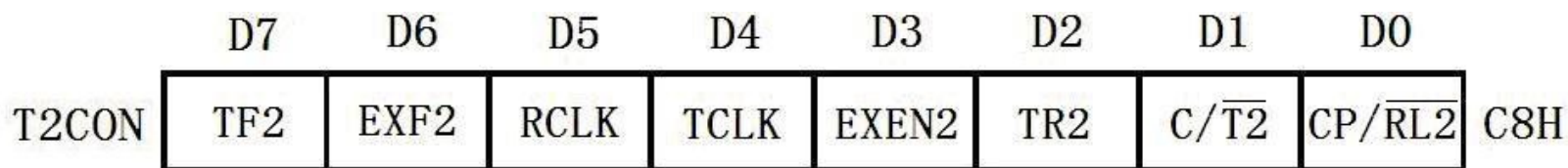


图6-5 T2CON格式

T2CON中的最高两位为定时器/计数器T2的中断请求标志位TF2和EXF2

(1) TF2 (D7)：当T2的计数器 (TL2、TH2) 计数计满溢出回0时，由内部硬件置位TF2 (寄存器T2CON.7)，向CPU发出中断请求。但是当RCLK位或TCLK位为1时将不予置位。本标志位必须由软件清0。

(2) EXF2 (D6)：当由引脚**T2EX (P1.1脚)**上的负跳变引起“捕捉”或“重新装载”且**EXEN2**位为1，则置位**EXF2**标志位（寄存器**T2CON.6**），向**CPU**发出中断请求。

上述两种中断请求，在满足中断响应条件时，**CPU**都将响应其中断请求，转向同一个中断向量地址进行中断处理。因此，必须在**T2**的中断服务程序中对**TF2**和**EXF2**两个中断请求标志位进行查询，然后正确转入对应的中断处理程序。

中断结束后，中断请求标志位**TF2**或**EXF2**必须由软件清0。

6.3 中断允许与中断优先级的控制

实现中断允许控制和中断优先级控制分别由中断允许寄存器IE和中断优先级寄存器IP实现。下面介绍两个特殊功能寄存器。

6.3.1 中断允许寄存器IE

各中断源开放或屏蔽，是由片内中断允许寄存器IE控制。IE字节地址为A8H，可进行位寻址，格式见图6-6。

	D7	D6	D5	D4	D3	D2	D1	D0	
IE	EA	—	ET2	ES	ET1	EX1	ET0	EX0	A8H
位地址	AFH	—	ADH	ACH	ABH	AAH	A9H	A8H	

图6-6 中断允许寄存器IE的格式

IE对中断开放和关闭实现两级控制。两级控制就是有一个总的中断开关控制位EA（IE. 7位），当EA=0，所有中断请求被屏蔽，CPU对任何中断请求都不接受；当EA=1时，CPU开中断，但5个中断源的中断请求是否允许，还要由IE中的低6位所对应的6个中断请求允许控制位的状态来决定（见图6-6）。

IE中各位的功能如下：

(1) EA—中断允许总开关控制位。

EA=0，所有的中断请求被屏蔽。

EA=1，所有的中断请求被开放。

(2) ET2—定时器/计数器T2的中断允许位。

ET2=0，禁止T2中断。

ET2=1，允许T2中断。

(3) ES—串行口中断允许位。

ES=0，禁止串行口中断。

ES=1，允许串行口中断。

(4) ET1—定时器/计数器T1溢出中断允许位。

ET1=0, 禁止T1溢出中断。

ET1=1, 允许T1溢出中断。

(5) EX1—外部中断1的中断允许位。

EX1=0, 禁止外部中断1中断。

EX1=1, 允许外部中断1中断。

(6) ET0—定时器/计数器T0的溢出中断允许位。

ET0=0, 禁止T0中断。

ET0=1, 允许T0中断。

(7) EX0——外部中断0中断允许位。

EX0=0, 禁止外部中断0中断。

EX0=1, 允许外部中断0中断。

AT89S52复位以后，**IE**被清“0”，所有的中断请求被禁止。**IE**中与各个中断源相应的位可用指令置“1”或清“0”，即可允许或禁止各中断源的中断申请。若使某一个中断源被允许中断，除了**IE**相应的位被置“1”外，还必须使**EA**位置“1”，即**EA**位置“1”为中断请求的必要条件。

改变**IE**的内容，可向寄存器**IE**写入相应的内容。

【例6-1】单片机复位后，若允许两个外中断源中断，并禁止其他中断源的中断请求，请编写设置**IE**的命令语句。

(1) 采用对**IE**寄存器和**TCON**寄存器进行位操作。

{

.....

EA=1;

//总中断允许

EX0=1;

//允许外部中断0中断

```

EX1=1;           //允许外部中断1中断
IT0=1;           //设置外部中断0为跳沿触发
IT1=1;           //设置外部中断1为跳沿触发
.....
}

```

由于单片机复位后，**IE**寄存器各个位均为**0**，所以只需要把允许中断的中断源设置为**1**即可。

(2) 采用对**IE**寄存器和**TCON**寄存器进行字节操作。

```

{
.....
IE=0x85;          //允许两个外中断中断
TCON=0x05;        //允许两个外中断均为跳沿触发
.....
}

```

6.3.2 中断优先级寄存器IP

中断请求源有两个中断优先级，每一个中断请求源可由软件设置为高优先级中断或低优先级中断，也可实现**两级中断嵌套**。

所谓两级中断嵌套，就是AT89S52正在执行低优先级中断的服务程序时，可被高优先级中断请求所中断，待高优先级中断处理完毕后，再返回低优先级中断服务程序。

两级中断嵌套见**图6-7**。

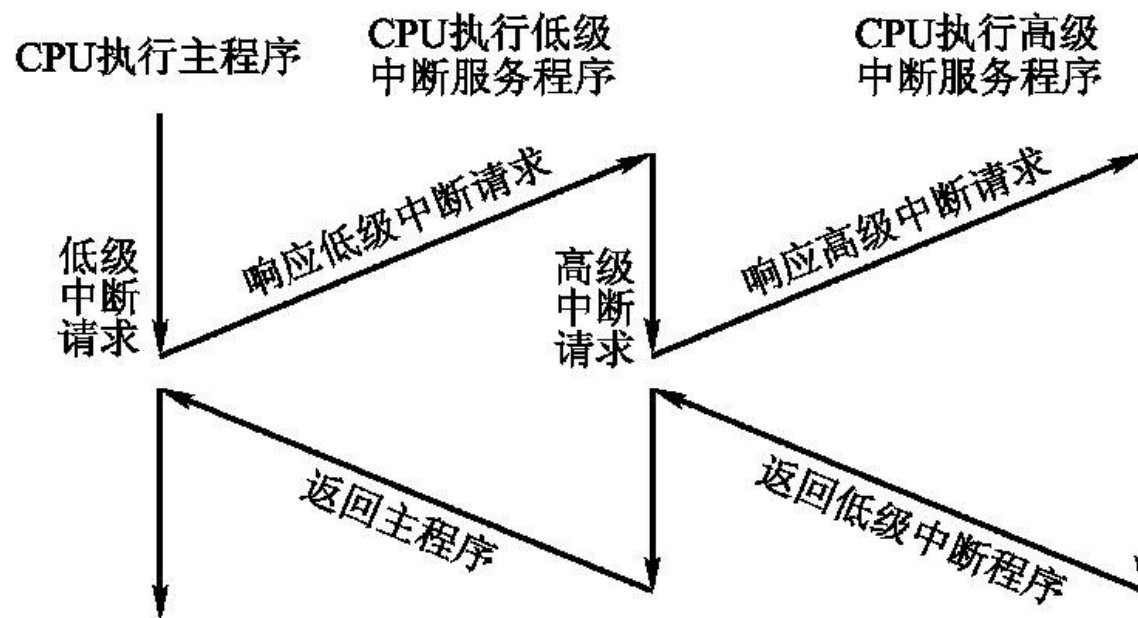


图6-7 两级中断嵌套过程

各中断源的中断优先级关系，可归纳为下面**两条基本规则**：

(1) 低优先级可被高优先级中断，高优先级不能被低优先级中断。

(2) 任何一种中断（不管是高级还是低级）一旦得到响应，不会再被它的同级中断源所中断。如果某一中断源被设置为高优先级中断，在执行该中断源的中断服务程序时，则不能被任何其他的中断源的中断请求所中断。

AT89S52片内有一个中断优先级寄存器IP，字节地址为**B8H**，可位寻址。只要用程序改变其内容，即可进行各中断源中断优先级设置，IP寄存器格式见**图6-8**。

	D7	D6	D5	D4	D3	D2	D1	D0	
IP	—	—	PT2	PS	PT1	PX1	PT0	PX0	B8H
位地址	—	—	BDH	BCH	BBH	BAH	B9H	B8H	

图6-8 IP寄存器的格式

中断优先级寄存器IP各位含义：

(1) **PT2**—T2中断优先级控制位

1—T2中断为高优先级；**0**—T2中断为低优先级

(2) **PS**—串行口中断优先级控制位，**1**—高级；**0**—低级。

(3) **PT1**—T1中断优先级控制位，**1**—高级；**0**—低级。

(4) **PX1**—外部中断1中断优先级控制位，**1**—高级；**0**—低级。

(5) **PT0**—T0中断优先级控制位，**1**—高级；**0**—低级。

(6) **PX0**—外部中断0中断优先级控制位，**1**—高级；**0**—低级。

中断优先级控制寄存器IP各位都可由程序置“1”和清“0”，用位操作指令或字节操作指令可更新IP的内容，改变各中断源的中断优先级。

AT89S52复位后，各中断源均为**低优先级中断**。

下面介绍AT89S52的中断优先级结构。

中断系统有**两个不可寻址**的“**优先级激活触发器**”，其中一个指示某高优先级中断正在执行，所有后来中断均被阻止；另一个触发器指示某低优先级中断正在执行，所有同级中断都被阻止，但不阻断高优先级的中断请求。

在同时收到几个同优先级的中断请求时，哪一个中断请求能优先得到响应，取决于**内部查询顺序**。这相当于在同一个优先级还存在另一辅助优先级结构，其**查询顺序**见表6-1。

由表6-1，各中断源在相同优先级的条件下，外部中断0的中断优先权最高，T2溢出中断或EXF2中断的中断优先权最低。

表 6-1 同一优先级中断的查询次序

中断源	中断级别
外部中断 0	最高
T0 溢出中断	
外部中断 1	
T1 溢出中断	
串行口中断	
T2 中断	
	最低

【例6-2】 设置IP寄存器，使AT89S52的两个外中断请求为高优先级，其他中断源的中断请求为低优先级。

```
{  
    .....  
    IP=0x05;  
    .....  
}
```

6.4 响应中断请求的条件

一个中断源中断请求被响应，须满足以下必要条件：

- (1) 总中断允许开关接通，即IE寄存器中的中断总允许位EA=1。
- (2) 该中断源发出中断请求，即该中断源对应的中断请求标志为“1”。
- (3) 该中断源的中断允许位=1，即该中断被允许。
- (4) 无同级或更高级中断正在被服务。

中断响应就是CPU对中断源提出的中断请求的接受，当查询到有效的中断请求时，满足上述条件时，紧接着就进行中断响应。

中断响应过程：

首先由硬件自动生成一条长调用指令“LCALL addr16”。即程序存储区中相应的中断入口地址。例如，对于外部中断1的响应，硬件自动生成的长调用指令为：

LCALL 0013H

生成LCALL指令后，紧接着就由CPU执行该指令。首先将程序计数器PC内容压入堆栈以保护断点，再将中断入口地址装入PC，使程序转向响应中断请求的中断入口地址。各中断源服务程序入口地址是固定的，见表6-2。

其中两个中断入口间只相隔8字节，一般情况下难以安放一个完整的中断服务程序。因此，通常总是在中断入口地址处放置一条无条件转移指令，使程序执行转向在其他地址存放的中断服务程序入口。

表 6-2 中断入口地址表

中断源	中断入口地址
外部中断 0	0003H
定时器/计数器 T0	000BH
外部中断 1	0013H
定时器/计数器 T1	001BH
串行口中断	0023H
定时器/计数器 T2 (T2+EXF2)	002BH

中断响应是有条件的，并不是查询到的所有中断请求都能被立即响应，当遇到下列3种情况之一时，中断响应被封锁：

（1）CPU正在处理同级或更高优先级的中断。因为当一个中断被响应时，要把对应的中断优先级状态触发器置“1”（该触发器指出CPU所处理的中断优先级别），从而封锁了低级中断请求和同级中断请求。

（2）所查询的机器周期不是当前正在执行指令的最后一个机器周期。设定这个限制的目的在于只有在当前指令执行完毕后，才能进行中断响应，以确保当前指令执行的完整性。

（3）正在执行的指令是RETI或是访问IE或IP的指令。因为按中断系统的规定，在执行完这些指令后，需再执行完一条指令，才响应新的中断请求。

如存在上述3种情况之一，CPU将丢弃中断查询结果，不能对中断进行响应。

6.5 外部中断的响应时间

在使用外部中断时，有时需考虑从外部中断请求有效（外部中断请求标志置“1”）到转向中断入口地址所需要的响应时间，即外部中断响应的实时性问题。下面就来讨论这个问题。

外中断最短响应时间为3个机器周期。其中中断请求标志位查询占1个机器周期，而这个机器周期恰好处于指令的最后一个机器周期。在这个机器周期结束后，中断即被响应，CPU接着执行1条硬件子程序调用指令LCALL以转到相应的中断服务程序入口，这需要2个机器周期。

外部中断响应最长时间为8个机器周期。这种情况发生在CPU进行中断标志查询时，刚好才开始执行RETI或访问IE或IP的指令，则需把当前指令执行



完再继续执行一条指令后，才能响应中断。

执行上述的`RETI`或访问`IE`或`IP`的指令，最长需要2个机器周期。而接着再执行1条指令，我们按最长的指令（乘法指令`MUL`和除法指令`DIV`）来算，也只有4个机器周期。再加上硬件子程序调用指令`LCALL`的执行，需要2个机器周期，所以，外部中断响应的最长时间为8个机器周期。

如已在处理同级或更高级中断，外部中断请求响应时间取决于正在执行的中断服务程序的处理时间，此情况下，响应时间无法计算。

这样，在单一中断系统里，AT89S52 对外部中断请求响应时间总是在3~8个机器周期之间。

6.6 外部中断的触发方式选择

外部中断有两种触发方式：**电平触发**方式和**跳沿触发**方式。

6.6.1 电平触发方式

若外中断定义为电平触发方式，外部中断申请触发器状态随着 CPU 在每个机器周期采样到的外部中断输入引脚电平变化而变化，这能提高CPU对外部中断请求的响应速度。当外部中断源被设定为电平触发方式时，在中断服务程序返回之前，外部中断请求输入必须无效（即外部中断请求输入已由低电平变为高电平），否则CPU返回主程序后会再次响应中断。

所以电平触发适合于外部中断以低电平输入且中断服务程序能清除外部中断请求源（即外部中断输入电平又变为高电平）的情况。如何清除电平触发的外部中断请求源的电平信号，本章后将介绍。

6.6.2 跳沿触发方式

外部中断若定义为跳沿触发方式，外部中断申请触发器能锁存外部中断输入线上的负跳变。即便是CPU暂时不能响应，中断请求标志也不会丢失。在这种方式下，如果相继连续两次采样，一个机器周期采样到外部中断输入为高，下一机器周期采样为低，则中断申请触发器置“1”，直到CPU响应此中断时，该标志才清“0”，这样就不会丢失中断，但输入的负脉冲宽度至少要保持1个机器周期（若晶振频率为6MHz，则为2s），才能被CPU采样到。

外部中断的跳沿触发方式适合于以负脉冲形式输入的外部中断请求。

6.7 中断请求的撤销

某中断请求被响应后，就存在着一个中断请求撤销问题。下面按中断请求源的类型分别说明中断请求的撤销方法。

1. 定时器/计数器T1、T0中断请求的撤销

定时器/计数器T1、T0中断的中断请求被响应后，硬件会自动把中断请求标志位（TF0或TF1）清“0”，因此定时器/计数器T1、T0中断请求是自动撤销的。

2. 外部中断请求的撤销

（1）跳沿方式外部中断请求的撤销

中断请求撤销两项内容：中断标志位清“0”和外中断信号的撤销。其



中，中断标志位（IE0或IE1）清“0”是在中断响应后由硬件自动完成的。而外中断请求信号的撤销，由于跳沿信号过后也就消失了，所以跳沿方式的外部中断请求也是自动撤销的。

（2）电平方式外中断请求撤销

中断请求标志自动撤销，但中断请求信号低电平可能继续存在，在以后的机器周期采样时，又会把已清“0”的IE0或IE1标志位重新置“1”。要彻底解决电平方式外部中断请求撤销，除标志位清“0”之外，还需在中断响应后把中断请求信号输入引脚从低电平强制改变为高电平。为此，可增加图6-9所示电路。

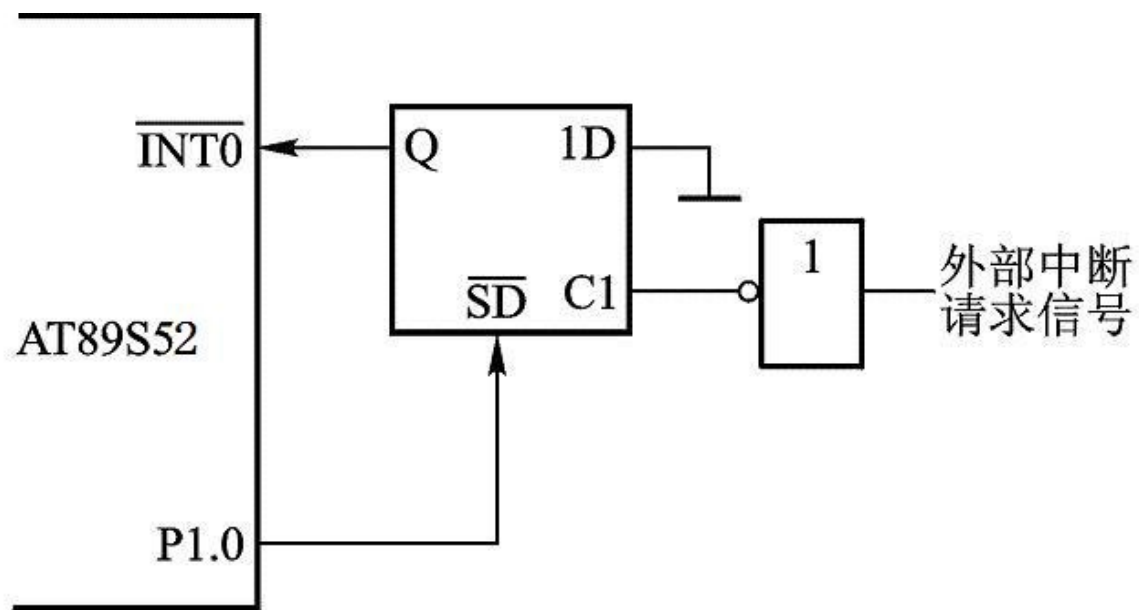


图6-9 电平方式的外部中断请求的撤销电路

由图6-9，D触发器锁存外来的中断请求低电平，并通过其输出端Q接到（INT0*或INT1*）。所以，增加的D触发器不影响中断请求。中断响应后，为撤销中断请求，可利用D触发器直接置“1”SD端实现，即把SD端接AT89S52的P1.0。因此，只要P1.0端输出一个负脉冲就可以使D触发器置“1”，从而就撤销低电平的中断请求信号。负脉冲可在中断服务程序中先P1.0置1，再让P1.0为0，再把P1.0置1。

3. 串行口中断请求的撤销

只有标志位清“0”的问题。串行口中断标志位是TI和RI，但对这两个中断标志CPU不自动清“0”。

因为响应串口中断后，CPU无法知道是接收中断还是发送中断，还需测试这两个中断标志位来判定，然后才清除。所以串口中断请求撤销只能使用软件在中断服务程序中把串行口中断标志位TI、RI清0。

4. 定时器/计数器T2中断请求的撤销

定时器/计数器T2的中断请求包括两种：TF2和EXF2。

上述两种中断请求，在满足中断响应条件时，CPU都将响应其中断请求，转向同一个中断入口地址。

因此，必须在T2的中断函数中对TF2和EXF2两个中断请求标志位进行查询，然后正确转入对应的中断处理程序。中断结束后，中断请求标志位TF2或EXF2必须由软件清0。所以定时器/计数器T2中断请求的撤销只能使用软件的方法，在中断函数返回前完成。

6.8 中断函数

为直接使用C51编写中断服务程序，C51中定义了中断函数。

由于C51编译器在编译时对声明为中断服务程序的函数自动添加相应现场保护、阻断其他中断、返回时自动恢复现场等处理的程序段，因而在编写中断函数时可不必考虑这些问题，减小编写中断服务程序烦琐程度。

中断服务函数的一般形式为：

函数类型 函数名（形式参数表） interrupt n using m

关键字interrupt后面的 n是中断号，对于AT89S52单片机，n的取值为0～5，编译器从 $8 \times n + 3$ 处产生中断向量。AT89S51中断源对应的中断号和中断向量见表6-3。

AT89S52内部RAM中可使用4个工作寄存器区，每个工作寄存器区包含8个工作寄存器（R0～R7）。关键字using后面的m用来选择4个工作寄存器区。using是一选项，如不选，中断函数中的所有工作寄存器内容将被保存到堆栈中。

表 6-3 AT89S52 单片机的中断号和中断向量

中断号 n	中 断 源	中断向量 ($8 \times n + 3$)
0	外部中断 0	0003H
1	定时器 T0	000BH
2	外部中断 1	0013H
3	定时器 T1	001BH
4	串行口	0023H
5	定时器 T2	0028H
其他值	保留	$8 \times n + 3$

关键字using对函数目标代码的影响如下：

在中断函数的入口处将当前工作寄存器区内容保护到堆栈中，函数返回前将被保护的寄存器区内容从堆栈中恢复。使用using在函数中确定一个工作寄存器区须十分小心，要保证任何工作寄存器区的切换都只在指定的控制区域中发生，否则将产生不正确的函数结果。

例如，外中断1（INT1*）中断服务函数如下：

```
void int1( ) interrupt 2 using 0    //中断号n=2，选择0区工作寄存器区
```

中断调用与标准C的函数调用是不一样的，当中断事件发生后，对应的中断函数被自动调用，即没有参数，也没有返回值，会带来如下影响。

(1) 编译器会为中断函数自动生成中断向量。

(2) 退出中断函数时，所有保存在堆栈中的工作寄存器及特殊功能寄存器被恢复。

(3) 在必要时特殊功能寄存器Acc、B、DPH、DPL以及PSW的内容被保存到堆栈中。

编写中断程序，应遵循以下规则：

(1) 中断函数没有返回值，如果定义一个返回值，将会得到不正确结果。**建议将中断函数定义为void类型**，明确说明无返回值。

(2) 中断函数不能进行参数传递，如果中断函数中包含任何参数声明都将导致编译出错。



(3) 任何情况下都不能直接调用中断函数，否则会产生编译错误。因为中断函数的返回是由汇编语言指令RETI完成的。RETI指令会影响AT89S52硬件中断系统内的不可寻址的中断优先级寄存器的状态。如没有实际中断请求情况下，直接调用中断函数，也就不会执行RETI指令，其操作结果有可能产生一个致命错误。

(4) 如在中断函数中再调用其他函数，则被调用的函数所用的寄存器区必须与中断函数使用的寄存器区不同。

6.9 中断系统应用设计案例

本节通过几个例程介绍有关中断应用程序的编写。

6.9.1 单一外中断的应用

【例6-3】在单片机P1口上接有8只LED。在外部中断0输入引脚（P3.2）接一只按钮开关K1。要求将外部中断0设置为电平触发。程序启动时，P1口上的8只LED全亮。

每按一次按钮开关K1，使引脚接地，产生一个低电平触发的外中断请求，在中断服务程序中，让低4位的LED与高4位的LED交替闪烁5次。然后从中断返回，控制8只LED再次全亮。原理电路及仿真结果见[图6-10](#)。

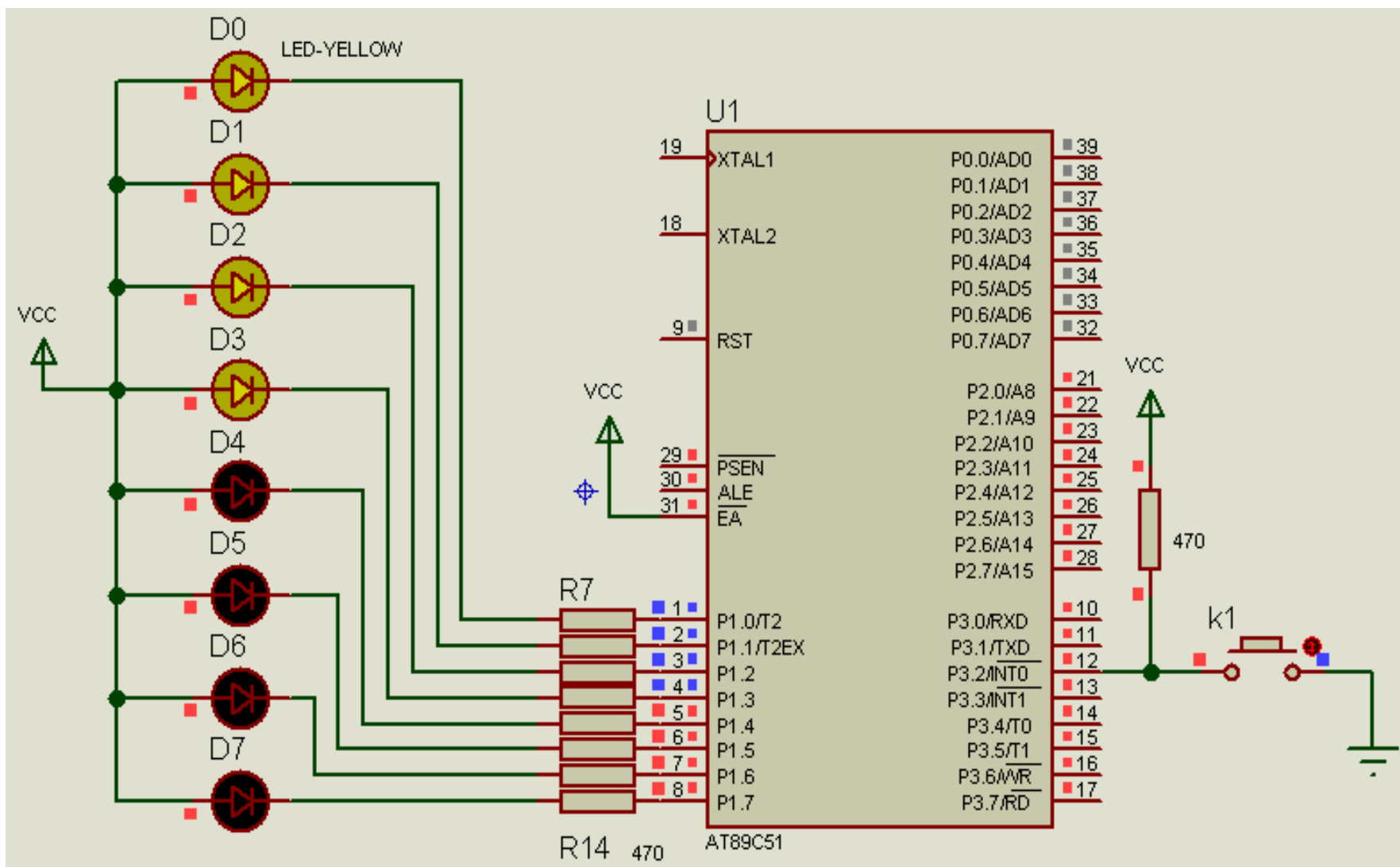


图6-10 利用中断控制8只LED交替闪烁1次的电路

参考程序如下：

```
#include <reg51.h>
#define uchar unsigned char

void Delay(unsigned int i) //延时函数Delay( ), i形式参数, 不能赋初值
{
    unsigned int j;
    for(;i > 0;i--)
        for(j=0;j<333;j++) //晶振为12MHz, j选择与晶振频率有关
            {;} //空函数
}

void main( ) //主函数
{
    EA=1; //总中断允许
    EX0=1; //允许外部中断0中断
    IT0=1; //选择外部中断0为跳沿触发方式
    while(1) //循环
```

```

        { P1=0;}                                // P1口的8只LED全亮
    }

void int0( )  interrupt 0  using 0                //外中断0的中断服务函数
{
    uchar  m;
    EX0=0;                                        //禁止外部中断0中断
    for (m=0;m<5;m++)                            //交替闪烁5次
    {
        P1=0x0f;                                //低4位LED灭，高4位LED亮
        Delay(400) ;                             //延时
        P1=0xf0;                                //高4位LED灭，低4位LED亮
        Delay(400) ;                             //延时
        EX0=1;                                    //中断返回前，打开外部中断0中断
    }
}

```



本例程包含两部分，一部分是主程序段，完成中断系统初始化，并把8个LED全部点亮。另一部分是中断函数部分，控制4个LED交替闪烁1次，然后从中断返回。

6.9.2 两个外中断的应用

当需要多个中断源时，只需增加相应的中断服务函数即可。**例6-4**是处理两个外中断请求的例子。

【例6-4】如**图6-11**所示，在单片机P1口上接有8只LED。在外部中断0输入引脚（P3.2）接有一只按钮开关K1。在外部中断1输入引脚（P3.3）接有一只按钮开关K2。

要求K1和K2都未按下时，P1口的**8只LED呈流水灯显示**，仅K1（P3.2）按下再松开时，**上下各4只LED交替闪烁10次**，然后再回到流水灯显示。如果按下再松开K2（P3.3）时，P1口的8只LED全部闪烁10次，然后再回到流水灯显示。设置两个外中断的优先级相同。

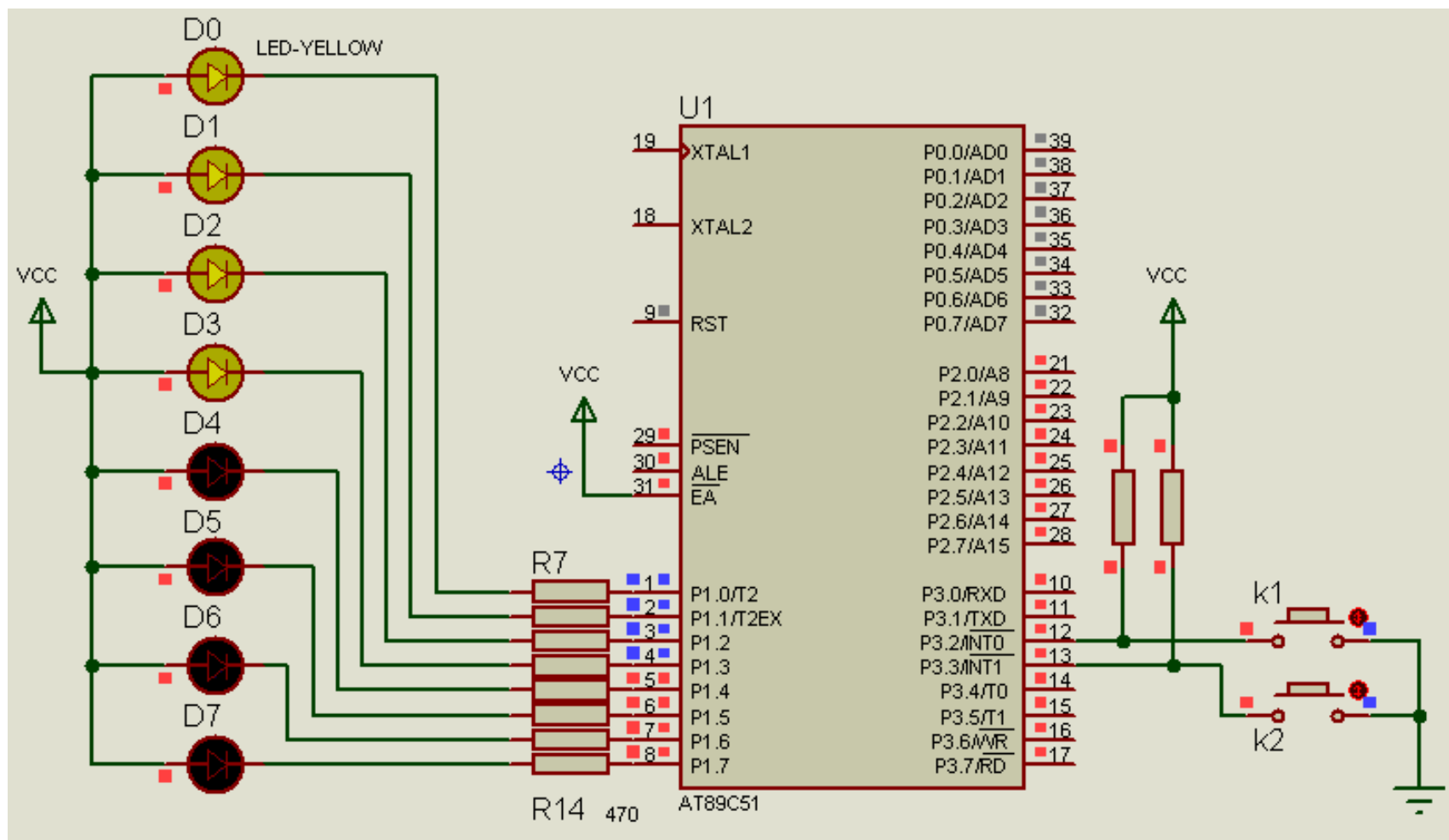


图6-11 两个外中断控制8只LED显示的电路

参考程序如下：

```
#include <reg51.h>
#define uchar unsigned char

void Delay(unsigned int i) //延时函数Delay( ), i为形式参数, 不能赋初值
{
    uchar j;
    for(; i>0; i--)
        for(j=0; j<125; j++)
            ; //空函数
}
```

```
void main( )
```

```
//主函数
```

```
{
```

```
    uchar display[9]={0xff, 0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f};
```

```
        //流水灯显示数据数组
```

```
    unsigned int a;
```

```
    for(;;)
```

```
{
```

```
    EA=1;
```

```
        //总中断允许
```

```
    EX0=1;
```

```
        //允许外部中断0中断
```

```
    EX1=1;
```

```
        //允许外部中断1中断
```

```
    IT0=1;
```

```
        //选择外部中断0为跳沿触发方式
```

```
    IT1=1;
```

```
        //选择外部中断1为跳沿触发方式
```



```
IP=0;
```

```
//两个外部中断均为低优先级
```

```
for (a=0;a<9;a++)
```

```
{
```

```
    Delay(500);
```

```
//延时
```

```
    P1=display[a];
```

```
//将已定义的流水灯显示数据送到P1口
```

```
}
```

```
}
```

```
}
```



```
void int0_isr(void)  interrupt 0  using 1//外中断0的中断服务函数
{
    uchar  n;
    for (n=0;n<10;n++)                //高、低4位显示10次
    {
        P1=0x0f;                      //低4位LED灭，高4位LED亮
        Delay(500);                   //延时
        P1=0xf0;                      //高4位LED灭，低4位LED亮
        Delay(500);                   //延时
    }
}
```

```
void int1_isr (void)  interrupt 2  using 2    //外中断1中断服务函数
{
    uchar  m;
    for (m=0;m<10;m++)                        //闪烁显示10次
    {
        P1=0xff;                               //全灭
        Delay (500) ;                          //延时
        P1=0;                                   //全亮
        Delay (500) ;                          //延时
    }
}
```

6.9.3 中断嵌套的应用

中断嵌套只发生正执行一个低优先级中断，此时又有一高优先级中断产生，就会去执行高优先级中断服务程序。高优先级中断服务程序完成后，再继续执行低优先级中断程序。

【例6-5】电路见图6-11，设计一中断嵌套程序：要求K1和K2都未按下时，P1口8只LED呈流水灯显示，当按一下K1时，产生一个低优先级外中断0请求（跳沿触发），进入外中断0中断服务程序，上下4只LED交替闪烁。此时按一下K2时，产生一个高优先级的外中断1请求（跳沿触发），进入外中断1中断服务程序，使8只LED全部闪烁。当显示5次后，再从外中断1返回继续执行外中断0中断服务程序，即P1口控制8只LED，上、下4只LED交替闪烁。设置外中断0为低优先级，外中断1为高优先级。

参考程序如下：

```
#include <reg51.h>
#define uchar unsigned char

void Delay(unsigned int i)                //延时函数Delay( )
{
    unsigned int j;
    for(;i > 0;i--)
        for(j=0;j<125;j++)
            {;}                            //空函数
}

void main( )                              //主函数
{
    uchar display[9]={0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f};
                                                //流水灯显示数据组
    uchar a;
```



```
for(;;)
```

```
{
```

```
    EA=1;           //总中断允许
    EX0=1;          //允许外部中断0中断
    EX1=1;          //允许外部中断1中断
    IT0=1;          //选择外部中断0为跳沿触发方式
    IT1=1;          //选择外部中断1为跳沿触发方式
    PX0=0;          //外部中断0为低优先级
    PX1=1;          //外部中断1为高优先级
```

```
    for(a=0;a<9;a++)
```

```
    {
```

```
        Delay(500);    //延时
        P1=display[a]; //流水灯显示数据送到P1口驱动LED显示
```

```
    }
```

```
}  
}
```

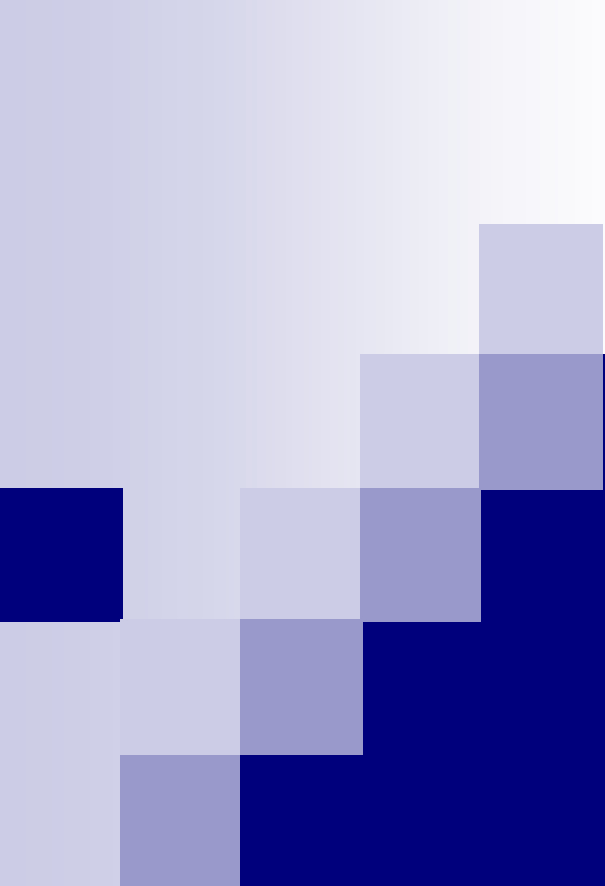
```
void int0_isr(void) interrupt 0 using 0 //外中断0中断函数  
{  
    for(;;)  
    {  
        P1=0x0f;           //低4位LED灭，高4位LED亮  
        Delay(400);        //延时  
        P1=0xf0;           //高4位LED灭，低4位LED亮  
        Delay(400);        //延时  
    }  
}
```

```

void int1_isr (void)  interrupt 2  using 1      //外中断1中断函数
{
    uchar m;
    for (m=0;m<5;m++)                          //8位LED全亮全灭5次
    {
        P1=0;                                  //8位LED全亮
        Delay(500);                             //延时
        P1=0xff;                                //8位LED全灭
        Delay(500);                             //延时
    }
}

```

本例如设置**外中断1**为**低优先级**，**外中断0**为**高优先级**，仍然先按下再松开K1，后按下再松开K2或者设置两个外中断源的中断优先级为同级，均不会发生中断嵌套。



第7章 定时器/计数器的工 作原理及应用



AT89S52单片机片内集成有**3个定时器/计数器T0、T1和T2**，可以满足计数或定时的需要。

本章首先介绍与**AT89S51**兼容的**T0、T1**定时器/计数器的结构及工作原理。而**AT89S52**增加的定时器/计数器**T2**的功能较强，内部结构及工作原理比**T0、T1**稍显复杂，因此将其与**T0、T1**分开，在本章后面单独介绍。

7.1 定时器/计数器T0与T1的结构

AT89S51定时器/计数器结构见图 7-1，定时器/计数器T0由特殊功能寄存器TH0、TL0构成，T1由特殊功能寄存器TH1、TL1构成。

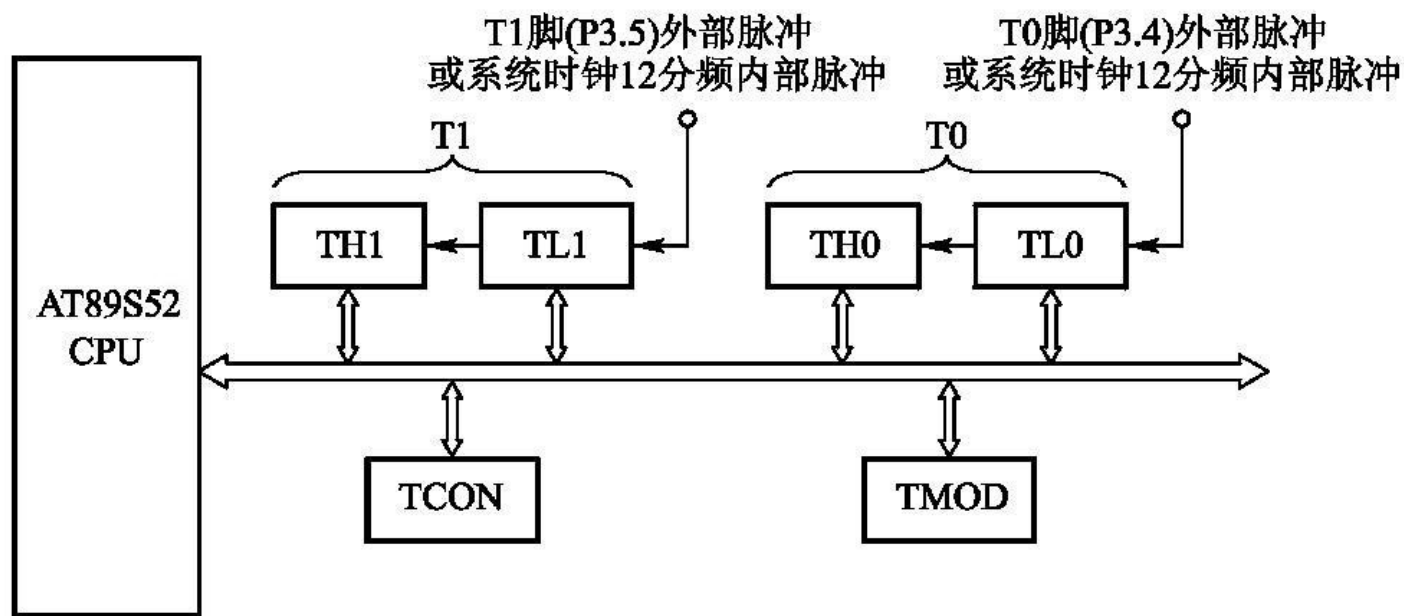


图7-1 定时器/计数器结构框图

两个定时器/计数器都具有**定时器**和**计数器2种工作模式**（定时器模式和计数器模式），**4种工作方式**（方式0、方式1、方式2和方式3）。定时器/计数器属于**增1**计数器。

图7-1中的特殊功能寄存器**TMOD**用于选择定时器/计数器**T0**、**T1**的工作模式和工作方式。特殊功能寄存器**TCON**用于控制**T0**、**T1**的启动和停止计数，同时包含了**T0**、**T1**的状态。**T0**、**T1**不论是工作在定时器模式还是计数器模式，实质都是对脉冲信号进行计数，只不过是计数信号的来源不同。

计数器工作模式是对加在**T0**（**P3.4**）和**T1**（**P3.5**）两个引脚上的外部脉冲进行计数（见后面的**图7-4**）；而**定时器工作模式**是对系统的时钟振荡器信号经片内**12**分频后的内部脉冲信号计数。



由于时钟频率是固定值，12分频后的脉冲信号周期也为固定值，所以可根据对内部脉冲信号的计数值计算出定时时间。

计数器的起始计数都是从计数器的初值开始。**AT89S52**单片机复位时计数器的初值为**0**，也可用指令给计数器装入一个新的初值，从新的初值开始计数。定时器/计数器属于**增1计数器**。

7.1.1 工作方式控制寄存器TMOD

TMOD用于选择定时器/计数器的工作模式和工作方式，**字节地址**为89H，不能位寻址，格式见**图7-2**。

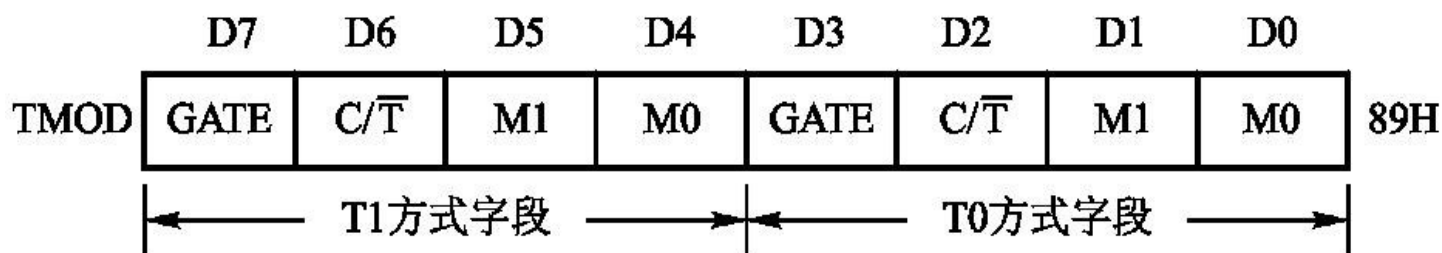


图7-2 寄存器TMOD格式

8位分两组，高4位控制T1，低4位控制T0。

TMOD各位说明如下。

(1) GATE—门控位

GATE=0，定时器是否计数，由控制位TR_x (x = 0, 1) 来控制。

GATE=1，定时器是否计数，由外中断引脚INTx* 上的电平与运行控制位TRx共同控制。

(2) M1、M0—工作方式选择位

M1、M0 4种编码，对应于**4种**工作方式的选择，见**表7-1**。

表7-1 M1、M0工作方式选择

M1	M0	工 作 方 式
0	0	方式0，为13位定时器/计数器
0	1	方式1，为16位定时器/计数器
1	0	方式2，8位的常数自动重新装入的定时器/计数器
1	1	方式3，仅适用于T0，此时T0分成两个8位计数器，T1停止计数

(3) C/T* —计数器模式和定时器模式选择位

C/T*=0，定时器模式，对系统时钟12分频后的脉冲进行计数。

C/T*=1，计数器模式，计数器对外部输入引脚T0（P3.4）或T1（P3.5）的外部脉冲（负跳变）计数。

7.1.2 定时器/计数器控制寄存器TCON

TCON字节地址**88H**，位地址为**88H~8FH**。格式见图7-3。

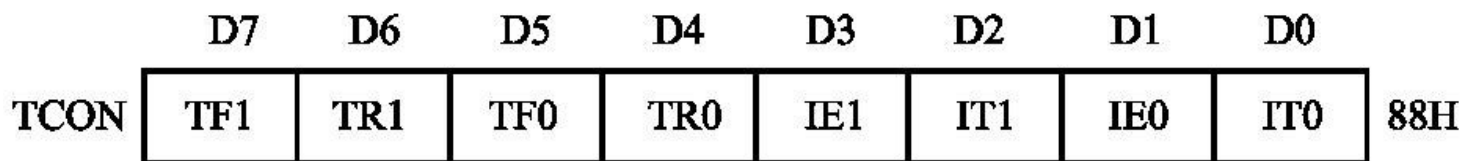


图7-3 TCON格式

第6章已介绍与外中断有关的低4位。这里仅介绍高4位功能。

(1) TF1、TF0—计数溢出标志位

当计数器计数溢出时，该位置“1”。使用查询方式时，此位可供CPU查询，但应注意查询后，用软件及时将该位清“0”。使用中断方式时，作为中断请求标志位，进入中断服务程序后由硬件自动清“0”。

(2) TR1、TR0—计数运行控制位

TR1位（或TR0）=1，启动计数器计数的必要条件。

TR1位（或TR0）=0，停止计数器计数。

该位可由软件置“1”或清“0”。

7.2 定时器/计数器的4种工作方式

分别介绍如下。

7.2.1 方式0

当M1、M0=00，设置为方式0，定时器/计数器等效逻辑结构见图7-4（以T1为例，TMOD.5、TMOD.4 = 00）。

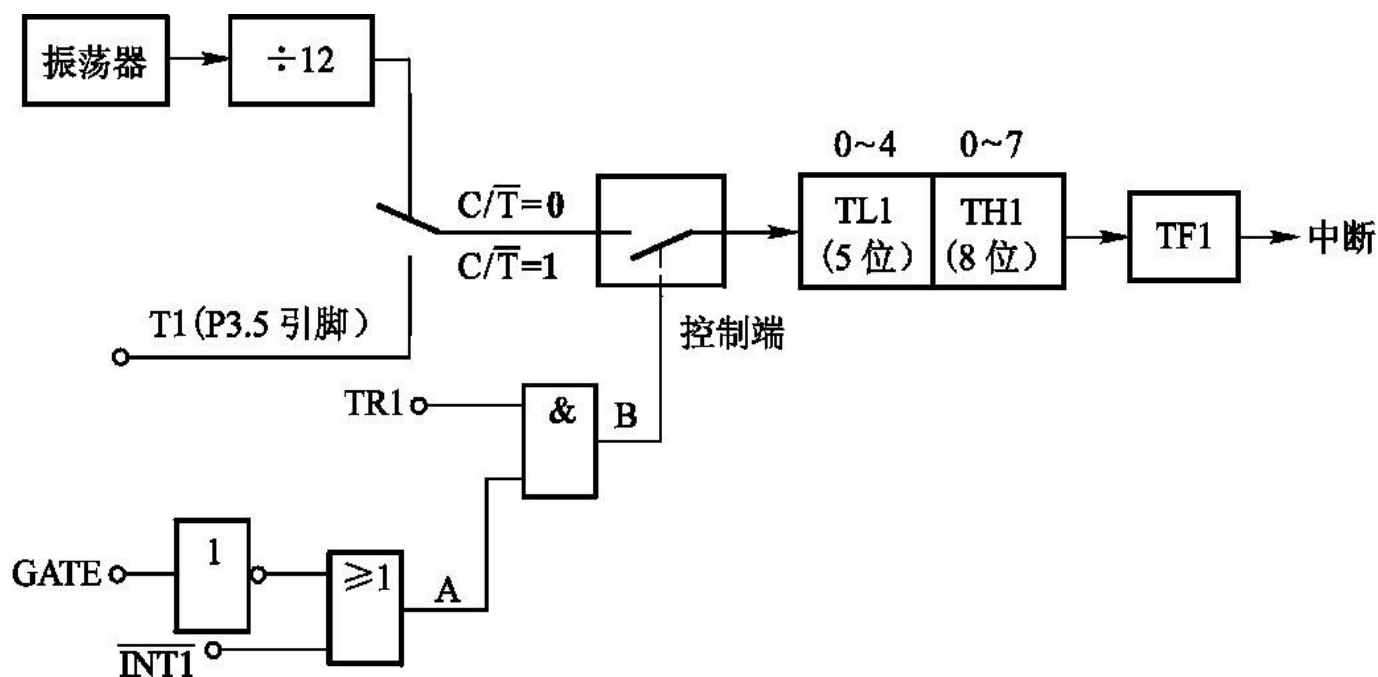


图7-4 定时器/计数器方式0的逻辑结构框图

方式0为13位计数器，由TLx（ $x = 0, 1$ ）的低5位和THx的高8位构成。TLx低5位溢出则向THx进位，THx计数溢出则把TCON中的溢出标志位TFx置“1”。

图7-2中，C/T*位控制电子开关决定2种工作模式。

(1) $C/T^*=0$ ，电子开关打在上面，T1（或T0）为定时器工作模式，系统时钟12分频后的脉冲作为计数信号。

(2) $C/T^*=1$ ，电子开关打在下面，T1（或T0）为计数器工作模式，对P3.5（或P3.4）引脚上的外部输入脉冲计数，当引脚上发生负跳变时，计数器加1。

GATE位状态决定定时器/计数器运行控制取决于TRx一个条件，还是取决于TRx和INTx*引脚状态两个条件。

(1) **GATE=0**时, A点(见图7-4)电位恒为1, B点电位仅取决于TR_x状态。TR_x=1, B点为高电平, 控制端控制电子开关闭合, 允许T1(或T0)对脉冲计数。TR_x=0, B点为低电平, 电子开关断开, 禁止T1(或T0)计数。

(2) **GATE=1**时, B点电位由INT_x^{*} ($x = 0, 1$)的电平和TR_x的状态两个条件来确定。当TR_x=1, 且INT_x^{*}=1时, B点才为1, 电子开关闭合, 允许T1(或T0)计数。故这种情况下计数器是否计数是由TR_x和INT_x^{*}两个条件来共同控制的。

7.2.2 方式1

当**M1、M0=01**时, 工作于方式1, 等效电路逻辑结构见图7-5。

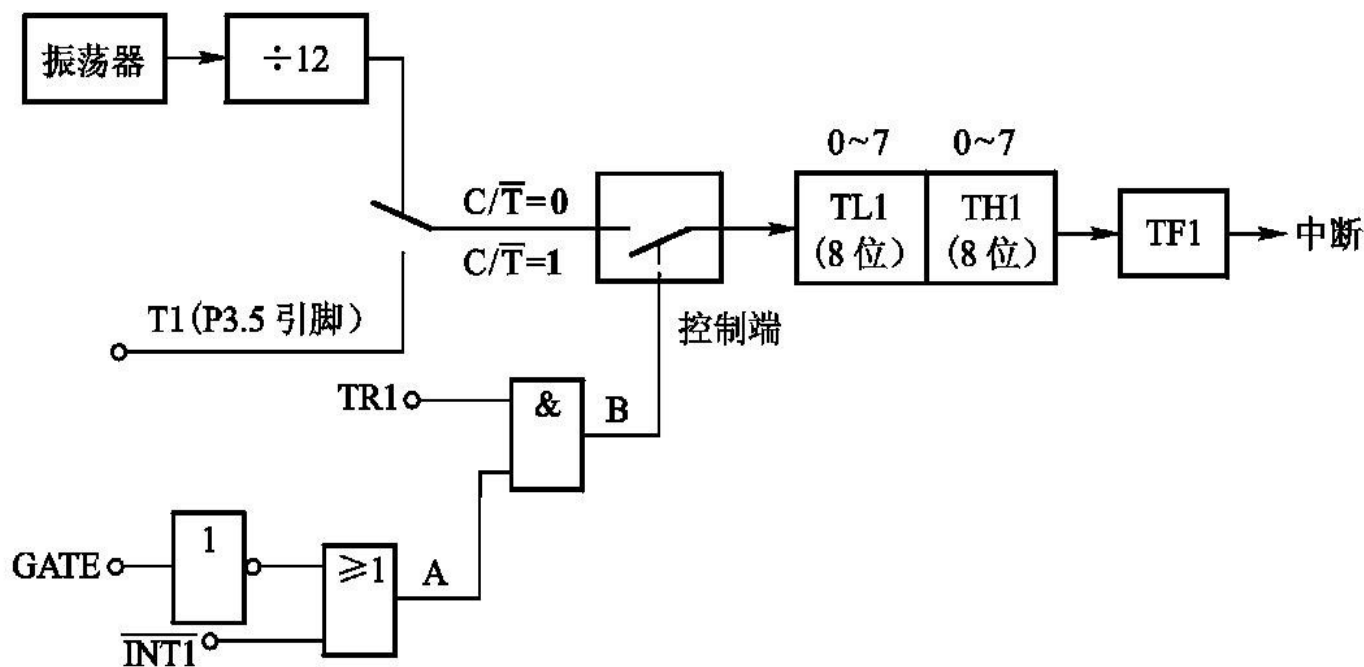


图7-5 方式1的逻辑结构框图

方式1和方式0差别仅仅在于计数器的位数不同，方式1为16位计数器，由THx高8位和TLx低8位构成（ $x = 0, 1$ ），方式0则为13位计数器，有关控制状态位含义（GATE、C/T*、TFx、TRx）与方式0相同。

7.2.3 方式2

方式0和方式1最大特点是计数溢出后，计数器为全0。因此在循环定时或循环计数应用时就存在用指令反复装入计数初值的问题，这会影响定时精度，方式2就是为解决此问题而设置的。

当M1、M0=10时，工作方式2，等效逻辑结构见图7-6（以T1为例，x=1）。

工作方式2为自动恢复初值（初值自动装入）的8位定时器/计数器，TLx（x=0，1）作为常数缓冲器，当TLx计数溢出时，在溢出标志TFx置“1”的同时，还自动将THx中的初值送至TLx，使TLx从初值开始重新计数。定时器/计数器方式2工作过程见图7-7。

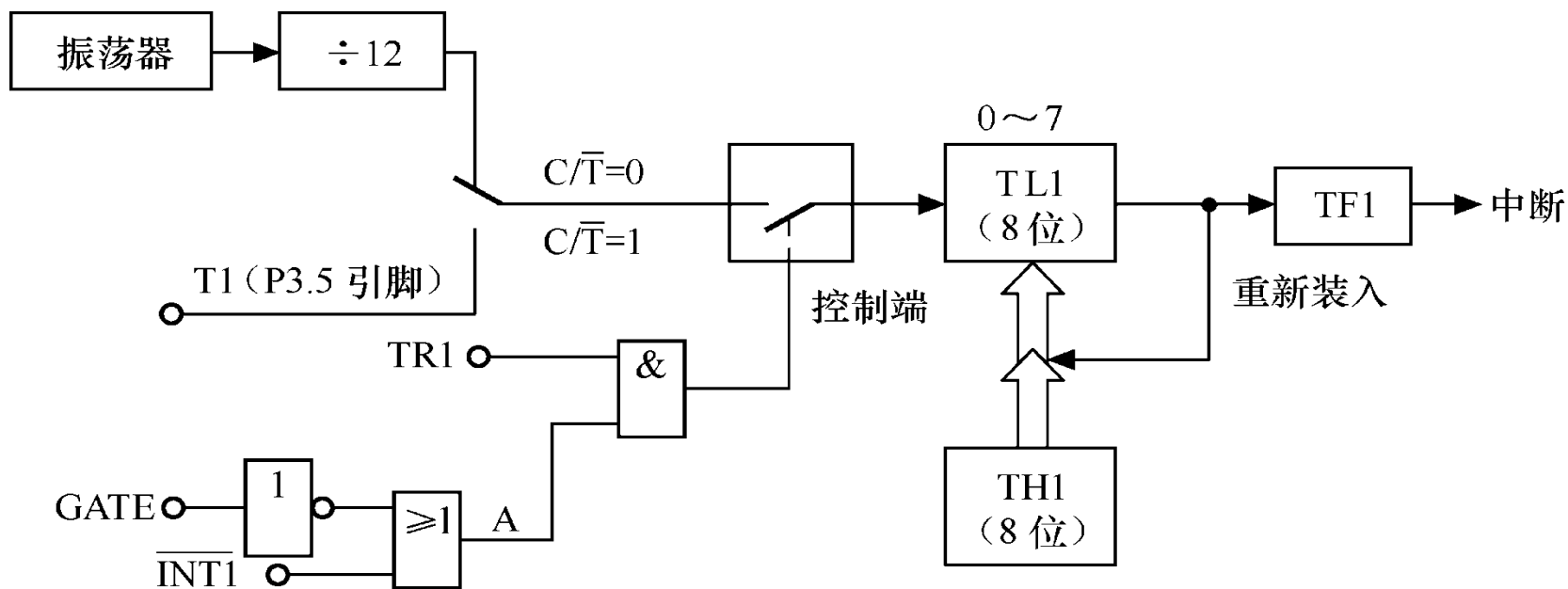


图7-6 方式2逻辑结构框图

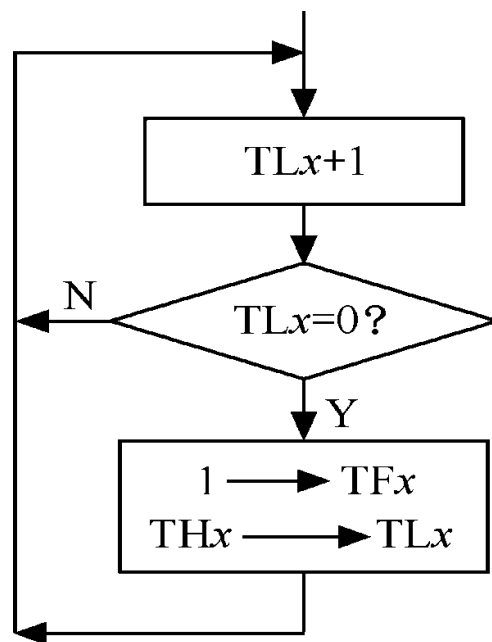


图7-7 方式2工作过程

方式2可省去用户软件中重装初值的指令执行时间，简化定时初值的计算方法，可相当精确地定时。

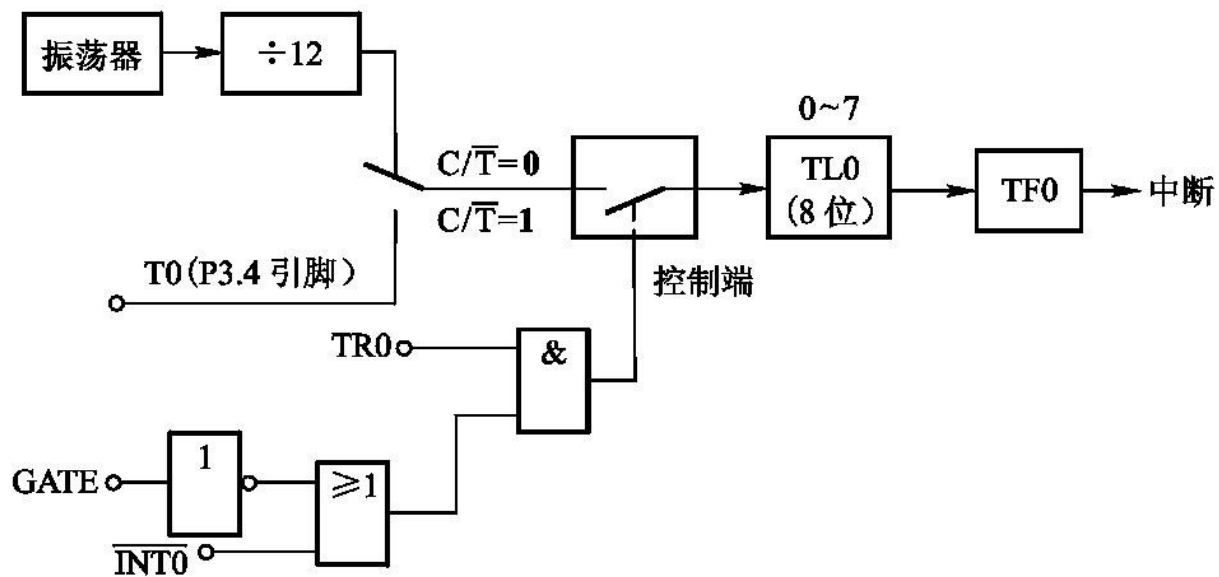
7.2.4 方式3

方式3是为增加一个附加的8位定时器/计数器而设置的，从而使AT89S52具有3个定时器/计数器。方式3只适用于T0，T1不能工作在方式3。T1方式3时相当于TR1 = 0，停止计数（此时T1可作为串口波特率产生器）。

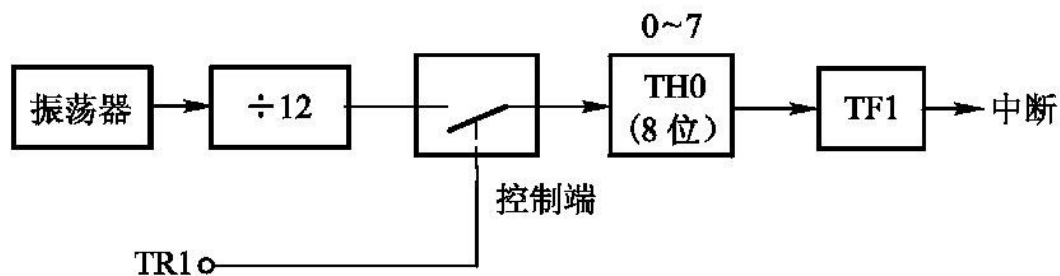
1. 工作方式3下的T0

当TMOD的低2位为11时，T0被选为方式3，各引脚与T0的逻辑关系见图7-8。

T0分为两个独立的8位计数器TL0和TH0，TL0使用T0的状态控制位C/T*、GATE、TR0，而TH0被固定为一个8位定时器（不能作为外部计数模式），并使用定时器T1的状态控制位TR1，同时占用定时器T1的中断请求源TF1。



(a) TL0作为8位定时器/计数器



(b) TH0作为8位定时器

图7-8 T0方式3的逻辑结构框图

2. T0工作在方式3时T1的各种工作方式

一般情况下，当T1用作串口波特率发生器时，T0才工作在方式3。T0方式3时，T1可为方式0、1、2，作为串口波特率发生器，或不需要中断的场合。

(1) T1工作在方式0

T1的控制字中**M1、M0 = 00**时，T1工作在方式0，工作示意图如图7-9所示。

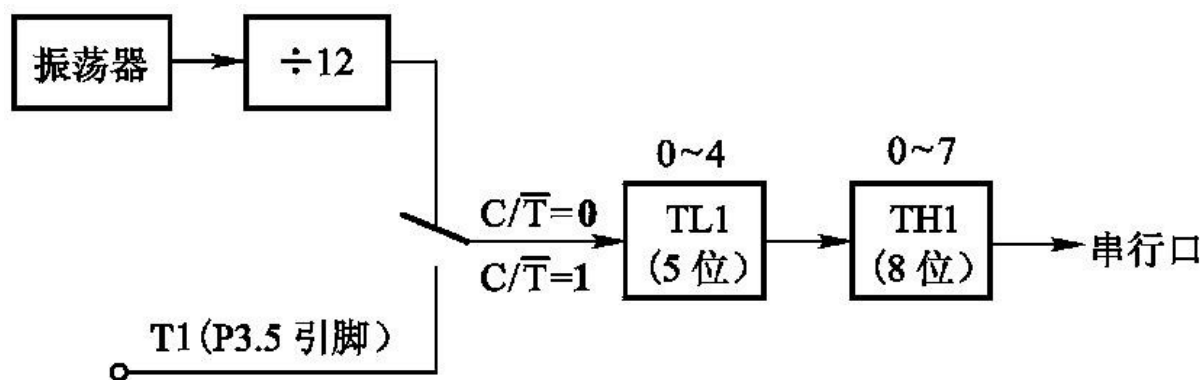


图7-9 T0方式3时T1为方式0工作示意图

(2) T1工作在方式1

当T1的控制字中**M1、M0 = 01**时，T1工作在方式1，工作示意图见图7-10。

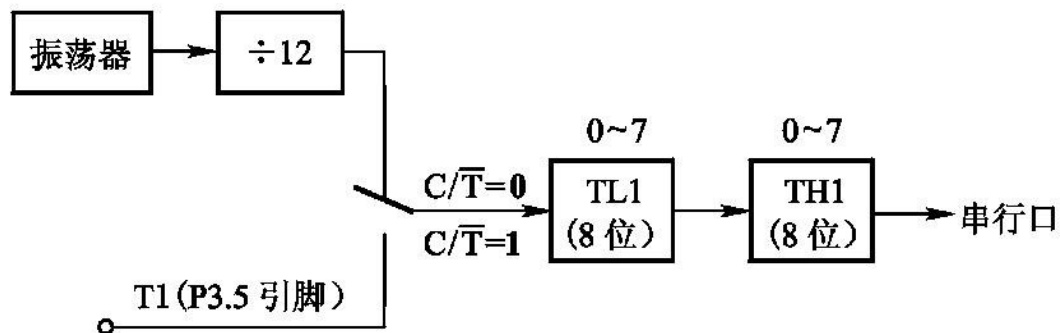


图7-10 T0方式3时T1为方式1工作示意图

(3) T1工作在方式2

当T1控制字中**M1、M0 = 10**时，T1为方式2，工作示意如**图7-11**所示。

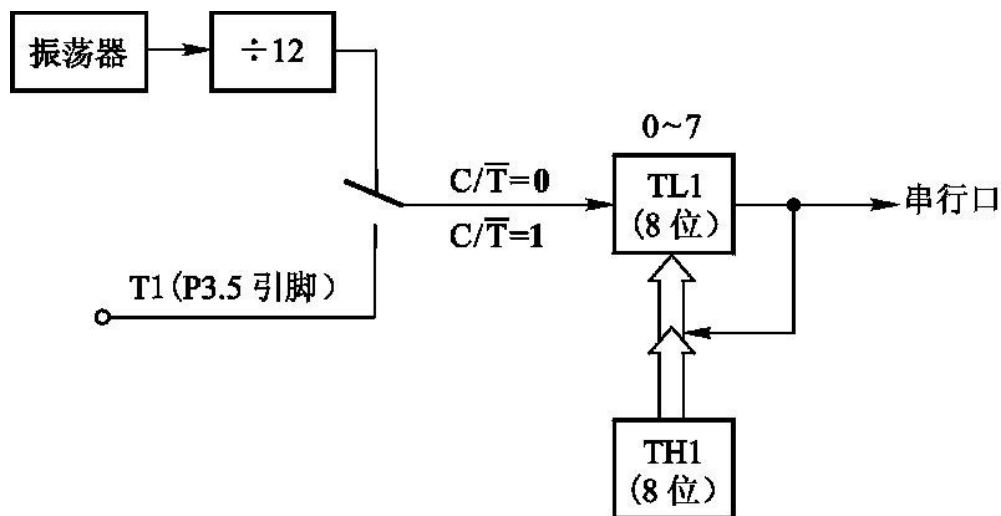


图7-11 T0方式3时T1为方式2工作示意图

(4) T1设置在方式3

T0方式3时，再把T1也设置成方式3，此时T1停止计数。

7.3 计数器模式对外部输入的计数信号的要求

计数器模式时，计数脉冲来自外部输入引脚T0或T1。当输入信号产生负跳变时，计数值增1。每个机器周期S5P2期间，都对外部输入引脚T0或T1进行采样。如在第1个机器周期中采得值为1，而在下一个机器周期中采得的值为0，则在紧跟着的再下一个机器周期S3P1期间，计数器加1。

由于确认一次负跳变要花2个机器周期，即24个振荡周期，因此外部输入的计数脉冲的最高频率为系统振荡器频率1/24。

如选用6MHz晶体，允许输入脉冲频率最高为250kHz。如选用12MHz频率晶体，则可输入最高频率500kHz外部脉冲。对外输入信号占空比没有限制，但为确保某一给定电平在变化前能被采样1次，则该电平至少保持1个机器周期。故对外部输入信号要求见图7-12，图中 T_{cy} 为机器周期。

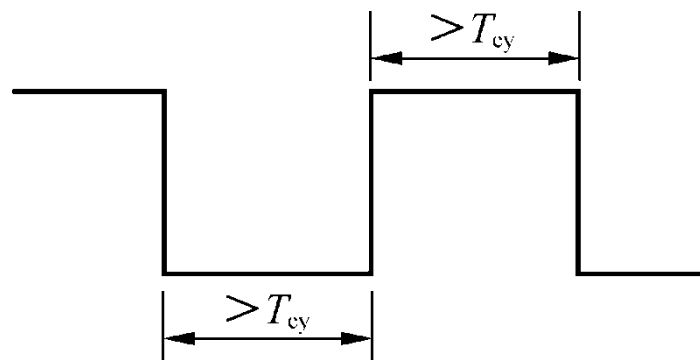


图7-12 对外部计数输入信号的要求

7.4 定时器/计数器T1、T0的编程和应用

4种工作方式中，方式0与方式1基本相同，只是计数位数不同。方式0为13位，方式1为16位。由于方式0是为兼容MCS-48而设，计数初值计算复杂，所以在实际应用中，一般不用方式0，常采用方式1。

7.4.1 P1口控制8只LED每0.5s闪亮一次

【例7-1】在AT89S51的P1口上接有8只LED，原理电路见图7-13。采用T0方式1的定时中断方式，使P1口外接的8只LED每0.5s闪亮一次。

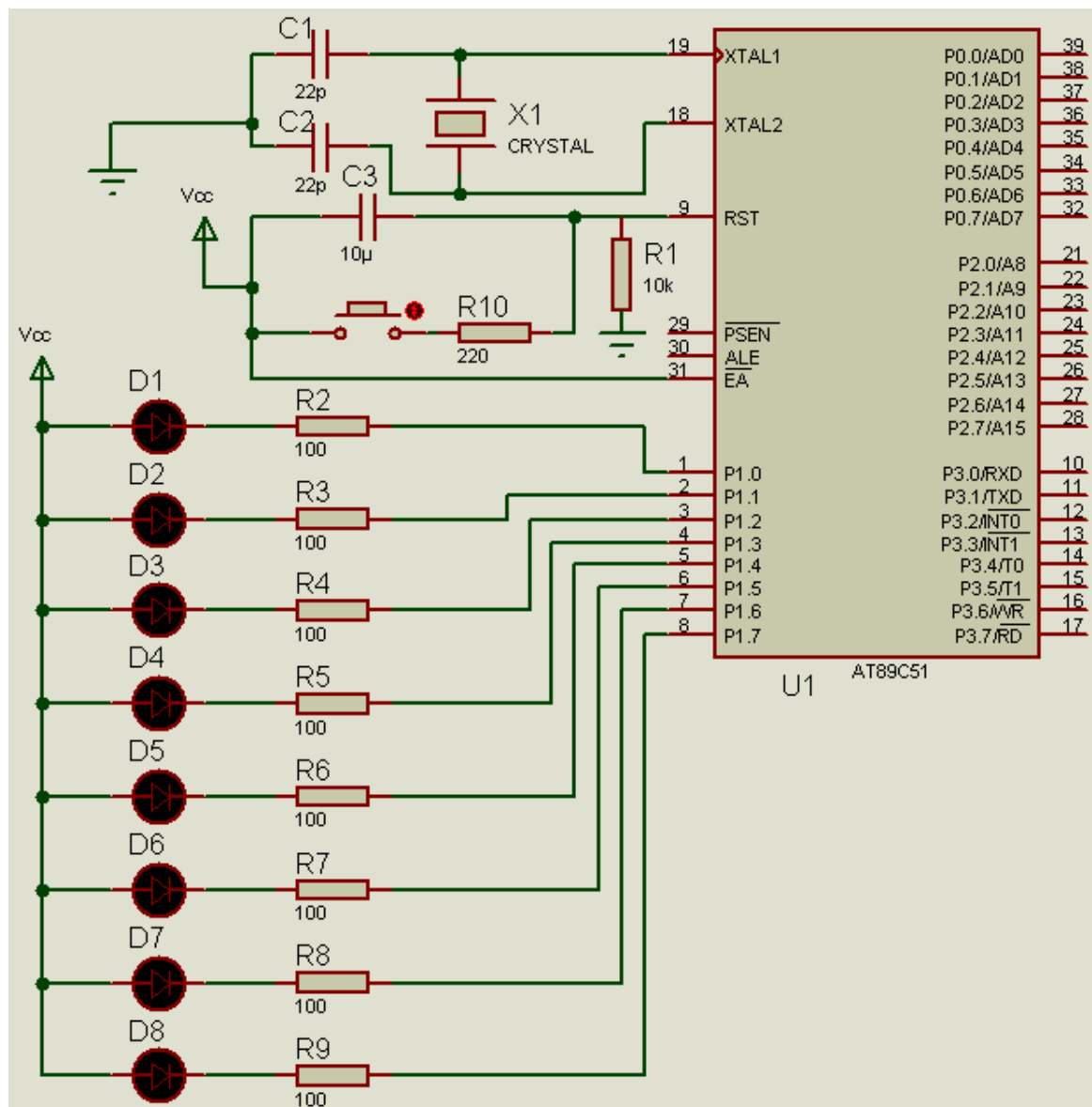


图7-13 方式1定时中断控制LED闪亮

(1) 设置TMOD寄存器

T0工作在方式1，应使TMOD寄存器的M1、M0=01；应设置C/T*=0，为定时器模式；对T0的运行控制仅由TR0来控制，应使相应的GATE位为0。定时器T1不使用，各相关位均设为0。所以，TMOD寄存器应初始化为0x01。

(2) 计算定时器T0的计数初值

设定时间5ms（即5 000 μ s），设T0计数初值为X，假设晶振的频率为11.059 2MHz，则定时时间为：

定时时间 $= (2^{16} - X) \times 12 / \text{晶振频率}$

则 $5\ 000 = (2^{16} - X) \times 12 / 11.059\ 2$

得 $X = 60\ 928$

转换成十六进制： $0xee00$ ，其中 $0xee$ 装入TH0， $0x00$ 装入TL0。

(3) 设置IE寄存器

本例由于采用定时器T0中断，因此需将IE寄存器中的EA、ET0位置1。

(4) 启动和停止定时器T0

将定时器控制寄存器TCON中的TR0=1，则启动定时器T0；TR0=0，则停止定时器T0定时。

参考程序：

```
#include<reg51.h>
```

```
char i=100;
```

```
void main ()
```

```
{
```

```
    TMOD=0x01;
```

```
    //定时器T0为方式1
```

```
    TH0=0xee;
```

```
    //设置定时器初值
```

```
    TL0=0x00;
```

P1=0x00;	//P1口8个LED点亮
EA=1;	//总中断开
ET0=1;	//开T0中断
TR0=1;	//启动T0
while(1);	//循环等待
{	
;	
}	
}	

void timer0() interrupt 1	//T0中断程序
{	
TH0=0xee;	//重新赋初值
TL0=0x00;	

```

i--;                                //循环次数减1
if(i<=0)
{
    P1=~P1;                          //P1口按位取反
    i=100;                          //重置循环次数
}
}

```

7.4.2 计数器的应用

【例7-2】如**图7-14**，T1采用计数模式，方式1中断，计数输入引脚T1（P3.5）上外接按钮开关，作为计数信号输入。**按4次按钮开关后，P1口的8只LED闪烁不停。**

（1）设置TMOD寄存器

T1工作在方式1，应使TMOD的M1、M0=01；设置C/T*=1，为计数器模式；对T0运行控制仅由TR0来控制，应使GATE0=0。定时器T0不使用，各相关位均设为0。所以，TMOD寄存器应初始化为0x50。

(2) 计算定时器T1的计数初值

由于每按1次按钮开关，计数1次，按4次后，P1口的8只LED闪烁不停。因此计数器初值为 $65\ 536 - 4 = 65\ 532$ ，将其转换成十六进制后为0xfffc，所以，TH0=0xff，TL0=0xfc。

(3) 设置IE寄存器

本例由于采用T1中断，因此需将IE寄存器的EA、ET1位置1。

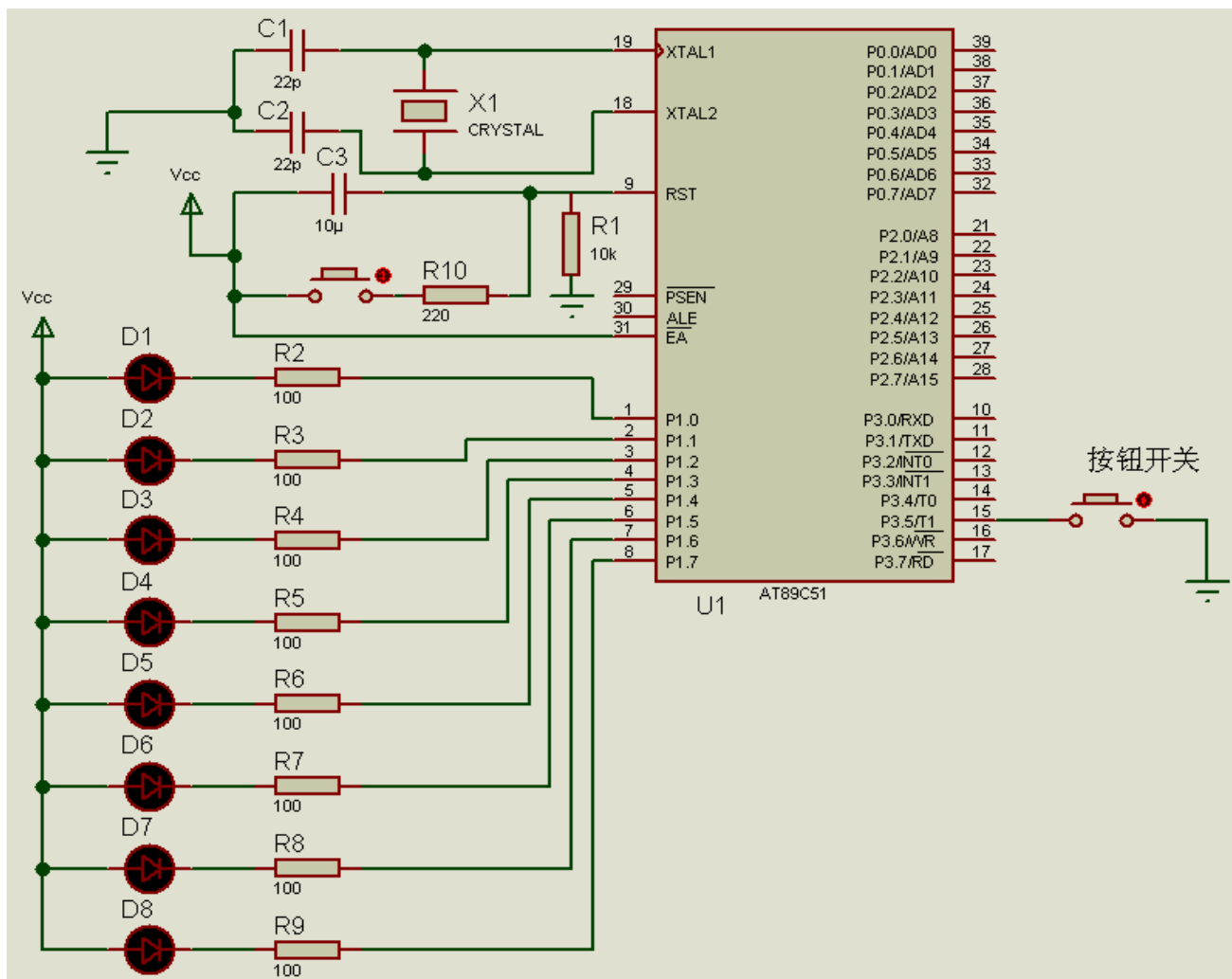


图7-14 由外部计数输入信号控制LED的闪烁

(4) 启动和停止定时器T1

将寄存器TCON中TR1=1，则启动T1计数；TR1=0，则停止T1计数。

参考程序如下：

```
#include <reg51.h>

void Delay(unsigned int i)    //延时函数Delay( ), i是形式参数，不能赋初值
{
    unsigned int j;
    for(;i>0;i--)             //变量i由实际参数传入一个值
                                //因此i不能赋初值

    for(j=0;j<125;j++)
    {;}                        //空函数
}

void main( )                  //主函数
{
    TMOD=0x50;                //设置定时器T1为方式1计数
```


TH1=0xff;	//向TH1写入初值的高8位
TL1=0xfc;	//向TL1写入初值的低8位
EA=1;	//总中断允许
ET1=1;	//定时器T1中断允许
TR1=1;	//启动定时器T1
while(1) ;	//无穷循环，等待计数中断
}	
 void T1_int(void) interrupt 3	 //T1中断函数
{	
for(;;)	//无限循环
{	
P1=0xff;	//8位LED全灭
Delay(500) ;	//延时500ms
P1=0;	//8位LED全亮
Delay(500);	//延时500ms
}	
}	

7.4.3 控制P1.0产生周期为2ms的方波

【例7-3】假设系统时钟为12MHz，设计电路并编写程序实现从P1.0引脚上输出一个周期为2ms的方波，见图7-15。

要在P1.0上产生周期为2ms的方波，定时器应产生1ms的定时中断，定时时间到则在中断服务程序中对P1.0求反。使用定时器T0，方式1定时中断，GATE不起作用。

本例的原理电路见图7-16。其中在P1.0引脚接有虚拟示波器，用来观察产生的周期2ms的方波。

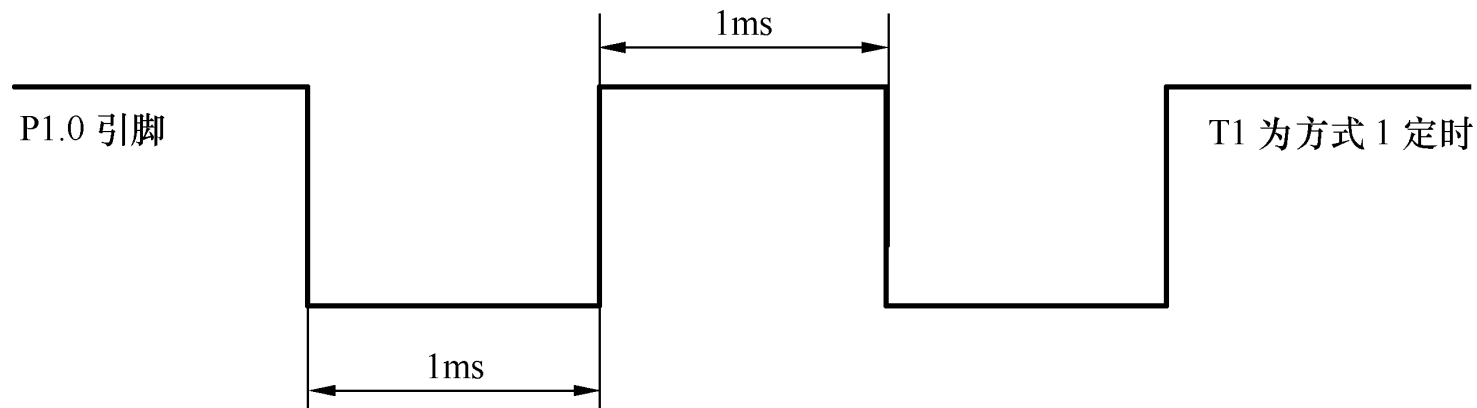


图7-15 定时器控制**P1.0**输出一个周期**2ms**方波

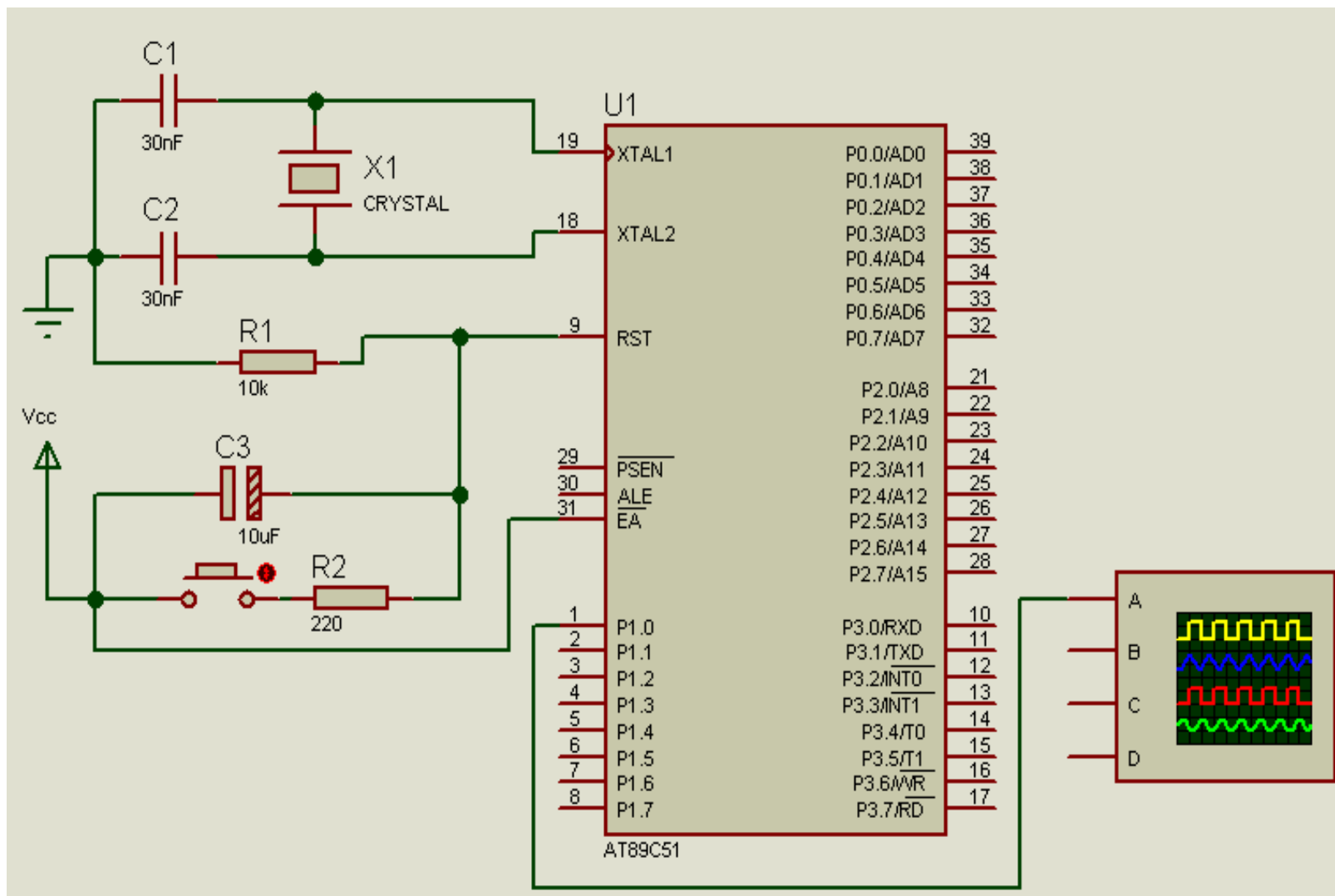


图7-16 定时器控制P1.0输出周期2ms的方波的原理电路

下面来计算T0初值 X ：

设T0的初值为 X ，有

$$(2^{16}-X) \times 1 \times 10^{-6} = 1 \times 10^{-3}$$

即 $65\,536 - X = 1\,000$

得 $X=64\,536$ ，化为16进制数就是0xfc18。将高8位0xfc装入TH0，低8位0x18装入TL0。

参考程序如下：

```
#include <reg51.h>
```

```
//头文件reg51.h
```

```
sbit P1_0=P1^0;
```

```
//定义特殊功能寄存器P1的位变量P1_0
```

```
void main(void)
```

```
//主程序
```

```
{
```

```
    TMOD=0x01;
```

```
//设置T0为方式1
```

```

TR0=1;                //接通T0
while(1)              //无限循环
{
    TH0=0xfc;          //置T0高8位初值
    TL0=0x18;          //置T0低8位初值
    do {} while(!TF0); //TF0为0原地循环，为1则T0溢出，往下执行
    P1_0=!P1_0;         // P1.0状态求反
    TF0=0;             //TF0标志清零
}
}

```

仿真时，右键单击虚拟数字示波器，出现下拉菜单，点击“Digital oscilloscope”选项，就会在数字示波器上显示P1.0引脚输出周期为2ms方波，如图7-17所示。

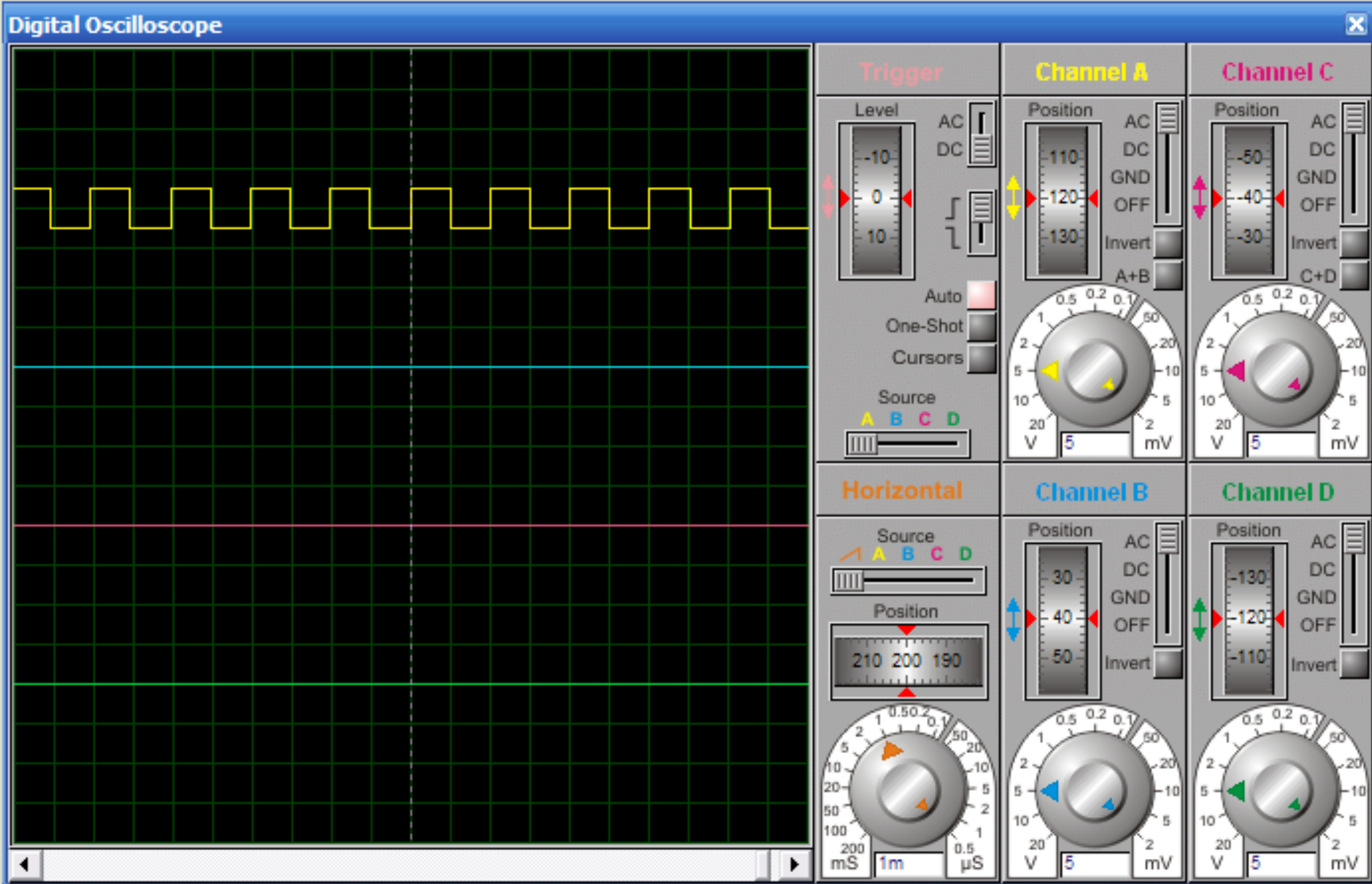


图7-17 虚拟数字示波器显示的2ms的方波波形

7.4.4 利用T1控制发出1kHz的音频信号

【例7-4】利用T1的中断控制P1.7引脚输出频率为1kHz方波音频信号，驱动蜂鸣器发声。系统时钟为12MHz。方波音频信号周期1ms，因此T1的定时中断时间为0.5 ms，进入中断服务程序后，对P1.7求反。电路见图7-18。

先计算T1初值，系统时钟为12MHz，则机器周期为1μs。1kHz音频信号周期为1ms，要定时计数的脉冲数为a。则T1初值：

$$TH1 = (65536 - a) / 256;$$

$$TL1 = (65536 - a) \% 256$$

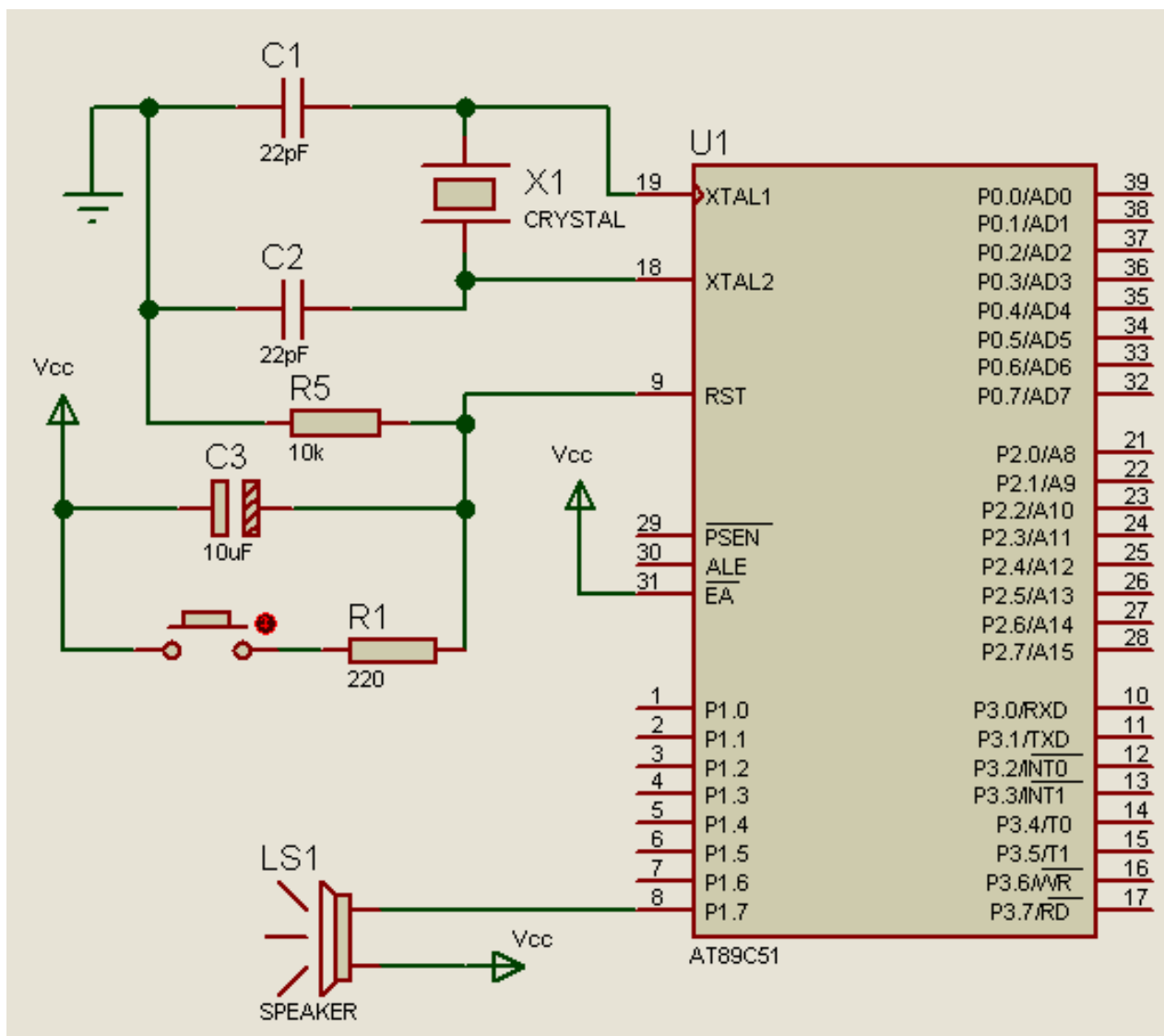


图7-18 控制蜂鸣器发出1kHz的音频信号

参考程序如下：

```
#include<reg51.h>                                     //包含头文件
sbit sound=P1^7;                                       //将sound位定义为P1.7脚
#define f1(a) (65536-a)/256                            //定义装入定时器高8位时间常数
#define f2(a) (65536-a)%256                          //定义装入定时器低8位时间常数
unsigned int i=500;
unsigned int j=0;

void main(void)
{
    EA=1;                                              //开总中断.
    ET1=1;                                            //允许定时器T1中断
    TMOD=0x10;                                       //TMOD=0001 000B, 使用T1的方式1定时
    TH1=f1(i);                                       //给T1高8位赋初值.
    TL1=f2(i);                                       //给T1低8位赋初值.
    TR1=1;                                           //启动T1
    while(1)
```

```

{
    i=460;
    while (j<2000) ;
    j=0;
    i=360;
    while (j<2000) ;
    j=0;
}
}

```

//循环等待

```
void T1(void) interrupt 3 using 0
```

//定时器T1中断函数

```
{
```

```
    TR1= 0;
```

//关闭T1

```
    sound=~sound;
```

//P1. 7输出求反

```
    TH1=f1(i);
```

//T1的高8位重新赋初值.

```
    TL1=f2(i);
```

//T1的低8位重新赋初值.

```
j++;  
TR1=1;           //启动定时器T1  
}
```

7.4.5 LED数码管秒表的制作

【例7-5】用2位数码管显示计时时间，最小计时单位为“百毫秒”，计时范围0.1~9.9s。当第1次按一下计时功能键时，秒表开始计时并显示；第2次按一下计时功能键时，停止计时，将计时的时间值送到数码管显示；如果计时到9.9s，将重新开始从0计时；第3次按一下计时功能键，秒表清0。再次按一下计时功能键，则重复上述计时过程。

本秒表应用定时器模式，计时范围0.1~9.9s。此外还涉及如何编写控制LED数码管显示的程序。

原理电路见图7-19。

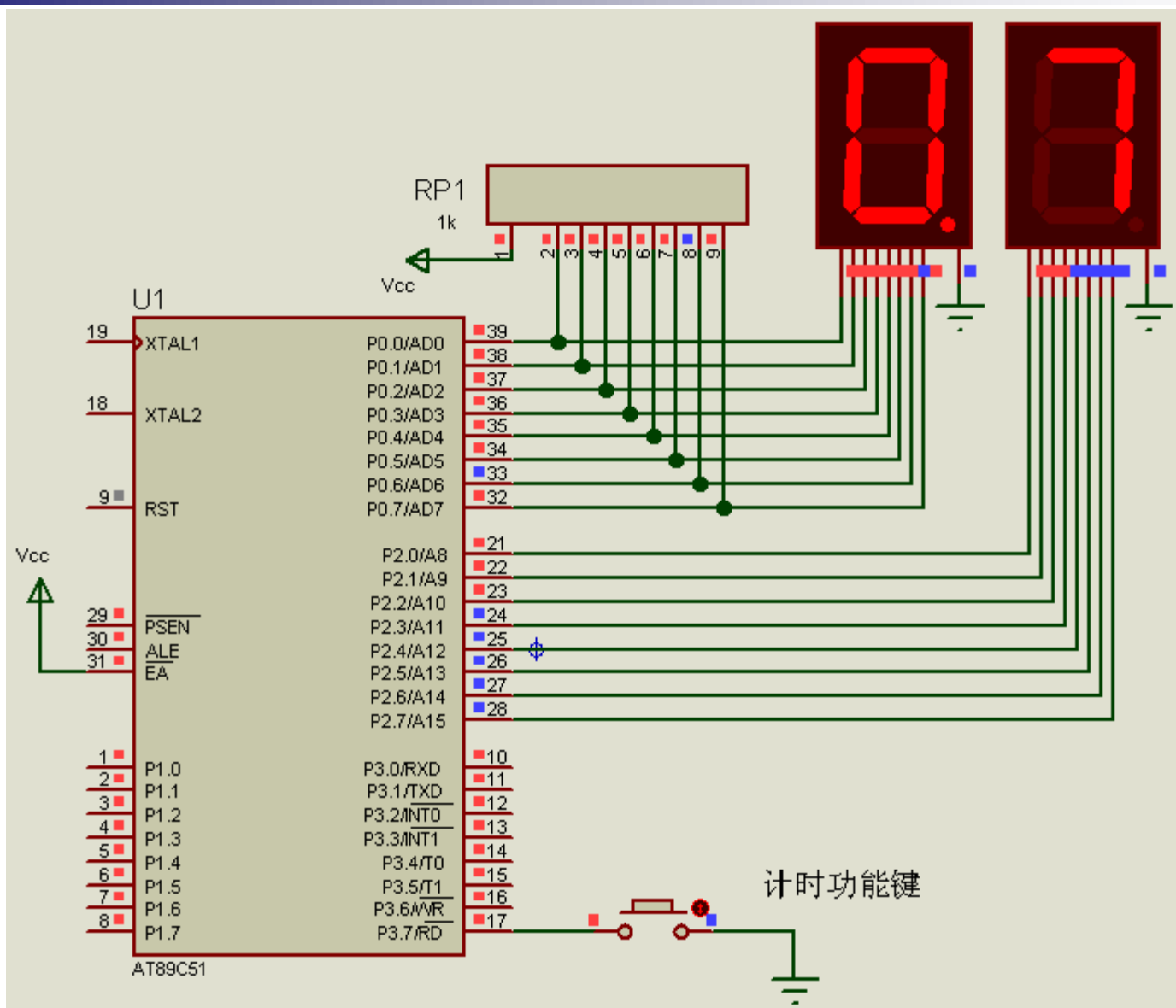


图7-19 LED数码管显示的秒表原理电路及仿真

参考程序如下：

```
#include<reg51.h>                //头文件

unsigned char code discode1[]=
    {0xbf, 0x86, 0xdb, 0xcf, 0xe6, 0xed, 0xfd, 0x87, 0xff, 0xef};
    //数码管显示0~9的段码表, 带小数点

unsigned char code
discode2[]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
    //数码管显示0~9的段码表, 不带小数点

unsigned char timer=0;            //timer记录中断次数
unsigned char second;            //second储存秒
unsigned char key=0;              //key记录按键次数

main()                            //主函数
{
    TMOD=0x01;                    //定时器T0方式1定时
    ET0=1;                        //允许定时器T0中断
```

```
EA=1; //总中断允许
second=0; //设初始值
P0=discode1[second/10]; //显示秒位0
P2=discode2[second%10]; //显示0.1s位0
while(1) //循环
{
    if((P3&0x80)==0x00) //当按键被按下时
    {
        key++; //按键次数加1
        switch(key) //根据按键次数分三种情况
        {
            case 1: //第一次按下为启动秒表计时
                TH0=0xee; //向TH0写入初值的高8位
```

```

    TL0=0x00;           //向TL0写入初值的低8位，定时5ms
    TR0=1;              //启动定时器T0
    break;
    case 2:              //按下两次暂定秒表
    TR0=0;              //关闭定时器T0
    break;
    case 3:              //按下3次秒表清0
    key=0;              //按键次数清
    second=0;           //秒表清0
    P0=discode1[second/10]; //显示秒位0
    P2=discode2[second%10]; //显示0.1s位0
    break;
}
while((P3&0x80)==0x00); //如果按键时间过长在此循环
}

```



```
}  
}
```

```
void int_T0() interrupt 1 using 0           //定时器T0中断函数  
{  
    TR0=0;                                //停止计时，执行以下操作（会带来计时误差）  
    TH0=0xee;                             //向TH0写入初值的高8位  
    TL0=0x00;                             //向TL0写入初值的低8位，定时5ms  
    timer++;                              //记录中断次数  
    if (timer==20)                        //中断20次，共计时20*5ms=100ms=0.1s  
    {  
        timer=0;                          //中断次数清0  
        second++;                          //加0.1s  
        P0=discode1[second/10];           //根据计时，即时显示秒位  
        P2=discode2[second%10];          //根据计时，即时显示0.1s位  
    }  
}
```

```
if(second==99)           //当计时到9.9s时
{
    TR0=0;                //停止计时
    second=0;             //秒数清0
    key=2;                //按键数置2，当再次按下按键时，
                          //key++,即key=3，秒表清0复原
}
else                      //计时不到9.9s时
{
    TR0=1;                //启动定时器继续计时
}
}
```

7.4.6 测量脉冲宽度—门控位GATEx的应用

介绍门控位GATE应用。以定时器T1为例，利用**GATE测量INT1*脚上正脉冲宽度**。

【例7-6】门控位GATE1可使T1启动计数受INT1*控制，当GATE1=1,TR1=1时,只有INT1*引脚输入高电平时,T1才被允许计数。利用该功能，可测量INT1*脚正脉冲宽度，方法见**图7-20**。

原理电路见**图7-21**，图中省略复位电路和时钟电路。利用门控位GATE1来**测量INT1*脚上正脉冲宽度**,并在6位数码管上以机器周期数显示。对被测量脉冲信号宽度,要求能通过旋转信号源旋钮可调。

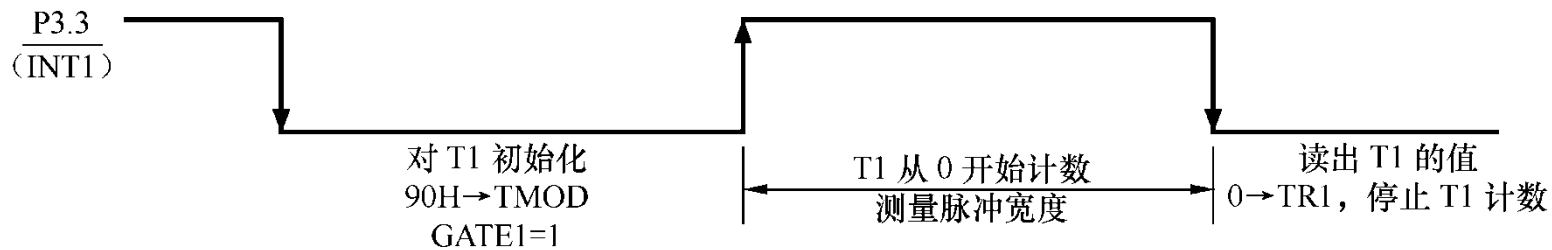


图7-20 利用**GATE**位测量正脉冲的宽度

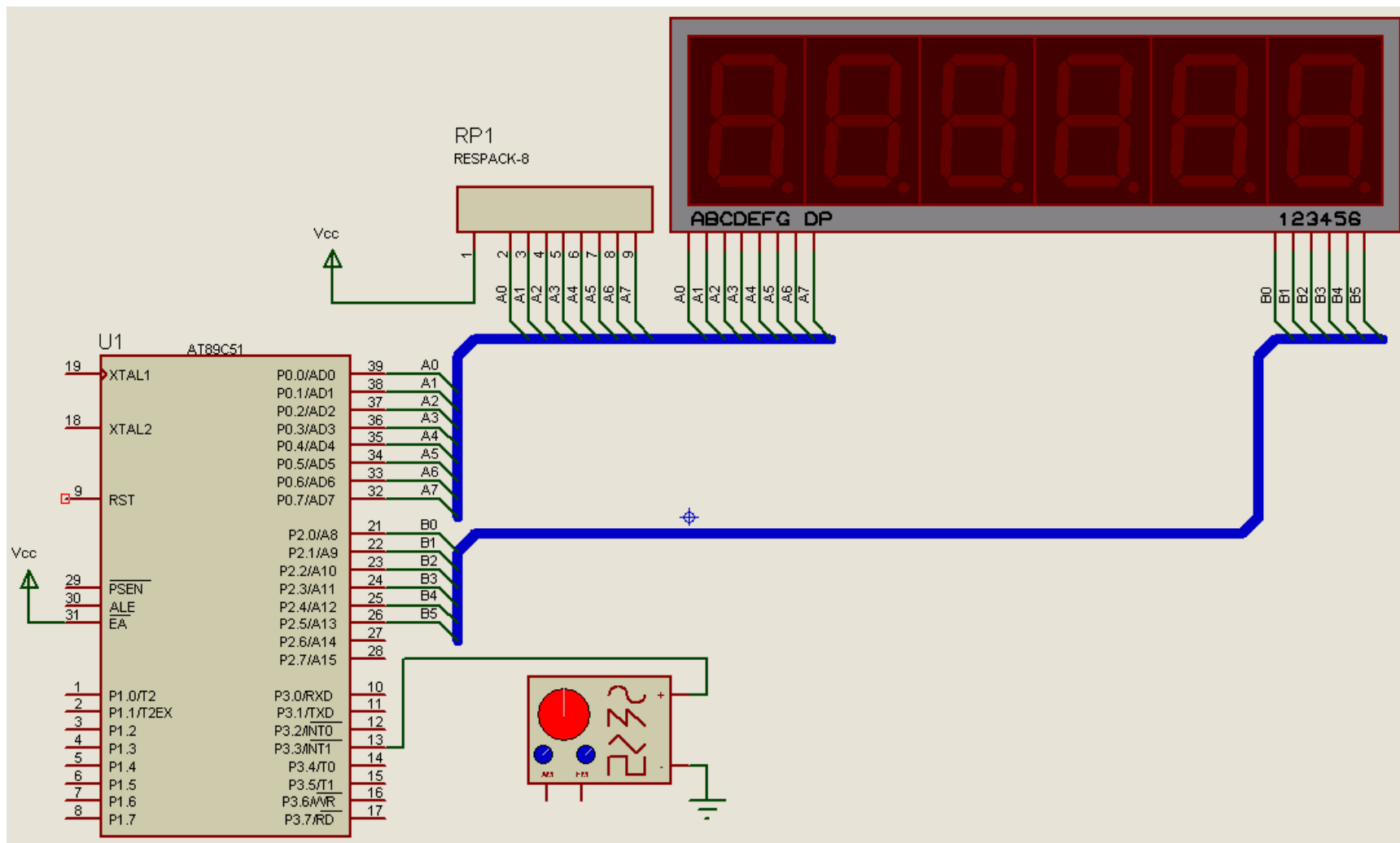


图7-21 利用GATE位测量 INT1*引脚上正脉冲的宽度的原理电路

参考程序如下：

```
#include<reg51.h>
```

```
#define uint unsigned int
```

```
#define uchar unsigned char
```

```
sbit P3_3=P3^3;
```

//位变量定义

```
uchar count_high;
```

//定义计数变量，用来读取TH0

```
uchar count_low;
```

//定义计数变量，用来读取TL0

```
uint num;
```

```
uchar shiwan, wan, qian, bai, shi, ge;
```

```
uchar flag;
```

```
uchar code table[]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
```

//共阴极数码管段码表

```
void delay(uint z)
```

//延时函数

```
{  
    uint x,y;  
    for(x=z;x>0;x--)  
        for(y=110;y>0;y--);  
}
```

```
void display(uint a,uint b,uint c,uint d,uint e,uint f) //数码管显示函数
```

```
{  
    P2=0xfe;  
    P0=table[f];  
    delay(2);  
    P2=0xfd;  
    P0=table[e];  
    delay(2);  
    P2=0xfb;  
    P0=table[d];  
}
```



```
    delay(2);  
    P2=0xf7;  
    P0=table[c];  
    delay(2);  
    P2=0xef;  
    P0=table[b];  
    delay(2);  
    P2=0xdf;  
    P0=table[a];  
    delay(2);  
}
```

```
void read_count()
```

```
//读取计数寄存器的内容
```



```
{  
    do  
    {  
        count_high=TH1;           //读高字节  
        count_low=TL1;           //读低字节  
    }while (count_high!=TH1);  
    num=count_high*256+count_low;  
    //可将两字节的机器周期数进行显示处理  
}
```

```
void main( )
```

```
{  
    while(1)  
    {  
        flag=0;  
        TMOD=0x90;           //设置定时器T1为方式1定时
```

```
TH1=0;           //向定时器T1写入计数初值
TL1=0;
while(P3_3==1);  //等待INT1*变低
TR1=1;           //如果INT1*为低，启动T1（未真正开始计数）
while(P3_3==0);  //等待INT1*变高，变高后T1真正开始计数
while(P3_3==1);  //等待INT1*变低，变低后T1停止计数
TR1=0;
read_count();     //读计数寄存器内容的函数
shiwanshi=num/100000;
wan=num%100000/10000;
qian=num%10000/1000;
bai=num%1000/100;
shi=num%100/10;
```

```
ge=num%10;
```

```
while(flag!=100)
```

//减小刷新频率

```
{
```

```
flag++;
```

```
display(ge,shi,bai,qian,wan,shiwan);
```

```
}
```

```
}
```

```
}
```

执行上述程序仿真，把INT1*引脚上出现的正脉冲宽度显示在LED数码管显示器上。晶振频率为12MHz，如果默认信号源输出频率为1kHz的方波，则数码管显示为500。

注意：在仿真时，偶尔显示501是因为信号源的问题，若将信号源换成频率固定的激励源则不会出现此问题。

7.4.7 LCD时钟的设计

【例7-7】使用定时器实现一个LCD显示时钟。采用LCD 1602。LCD时钟的原理电路见图7-22。

最小计时单位是秒，如何获得1s的定时？可将T0定时时间定为50ms，采用中断方式进行溢出次数累计，满20次，则秒计数变量second加1；若秒计满60，则分计数变量minute加1，同时将秒计数变量second清0；若分钟计满60，则小时计数变量hour加1；若小时计数变量满24，则将小时计数变量hour清0。

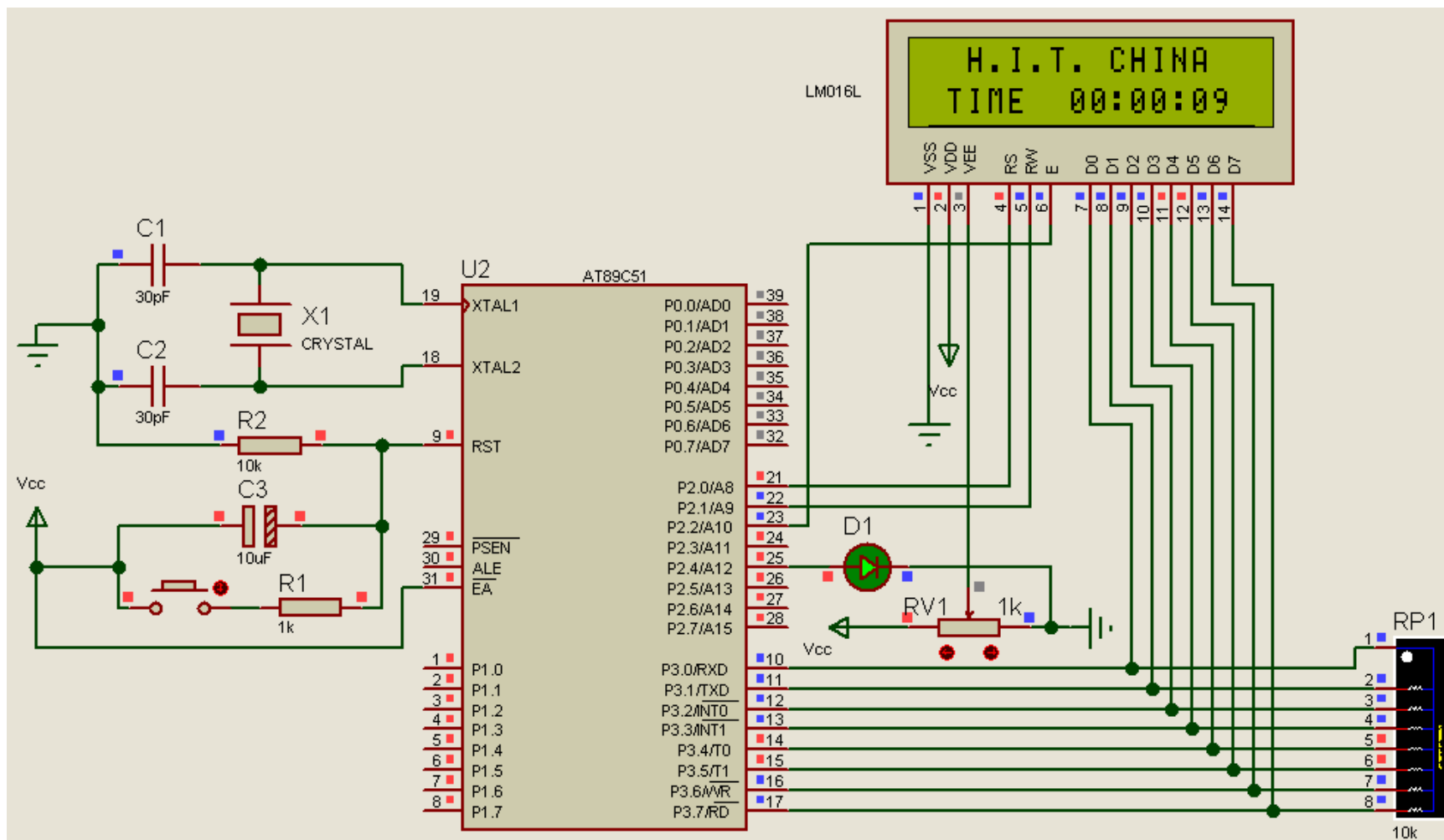


图7-22 LCD时钟的原理电路

先将定时器以及各计数变量设定完毕，然后调用时间显示子程序。秒计时由T0中断服务子程序来实现。

参考程序如下：

```
#include<reg51.h>
#include<lcd1602.h>
#define uchar unsigned char
#define uint unsigned int
uchar int_time;           //定义中断次数计数变量
uchar second;            //秒计数变量
```

```
uchar minute;  
uchar hour;  
uchar code date[]=" H.I.T. CHINA ";  
uchar code time[]=" TIME 23:59:55 ";  
uchar second=55,minute=59,hour=23;
```

```
//分钟计数变量  
//小时计数变量  
//LCD第1行显示的内容  
//LCD第2行显示的内容
```

```
void clock_init()
```

```
{  
    uchar i,j;  
    for(i=0;i<16;i++)  
    {  
        write_data(date[i]);  
    }
```

```
write_com(0x80+0x40);
```

```
for(j=0;j<16;j++)
{
    write_data(time[j]);
}
}
```

```
void clock_write( uint s, uint m, uint h)
{
    write_sfm(0x47,h);
    write_sfm(0x4a,m);
    write_sfm(0x4d,s);
}
```

```
void main()
{
    init1602();                //LCD初始化
    clock_init();              //时钟初始化
}
```



```
TMOD=0x01;           //设置定时器T0为方式1定时
EA=1;                 // 总中断开
ET0=1;               // 允许T0中断
TH0=(65536-46483)/256; //给T0装初值
TL0=(65536-46483)%256;
TR0=1;
int_time=0;           //中断次数、秒、分、时单元清0
second=55;
minute=59;
hour=23;
while(1)
{
    clock_write(second ,minute, hour);
}
}
```

```
void T0_interserve(void) interrupt 1 using 1
```

```
{
```

```
    int_time++;
```

```
    if(int_time==20)
```

```
    {
```

```
        int_time=0;
```

```
        second++;
```

```
    }
```

```
    if(second==60)
```

```
    {
```

```
        second=0;
```

```
        minute ++;
```

```
    }
```

//T0中断服务子程序

//中断次数加1

//若中断次数计满20次

//中断次数变量清0

//秒计数变量加 1

//若计满60s

//秒计数变量清0

//分计数变量加 1

```
if(minute==60)                //若计满60分
{
    minute=0;                //分计数变量清0
    hour ++;                 //小时计数变量加1
}
if(hour==24)
{
    hour=0;                  //小时计数计满24，将小时计数变量清0
}
TH0=(65536-46083)/256;       //定时器T0重新赋值
TL0=(65536-46083)%256;
}
```

执行上述程序仿真运行，就会在LCD上显示实时时间。

7.5 定时器/计数器T2的结构与工作方式

AT89S52与AT89S51单片机相比，新增加了一个16位定时器/计数器T2。
与T2相关的特殊功能寄存器共有2个：T2CON和T2MOD。

7.5.1 T2的特殊功能寄存器T2CON和T2MOD

1. 特殊功能寄存器T2CON

T2有3种工作方式：自动重装载（递增或递减计数）、捕捉和波特率发生器，由特殊功能寄存器中的控制寄存器T2CON 中的相关位来进行选择。

T2CON的字节地址为C8H，可位寻址，位地址为C8H～CFH，格式见图7-23。

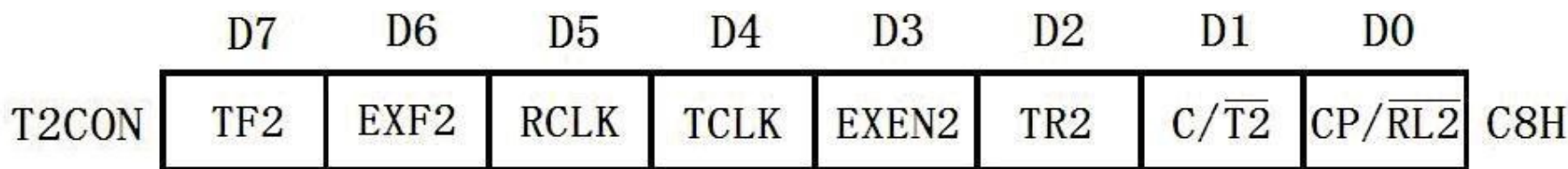


图7-23 TCON格式

T2CON寄存器各位的定义如下：

TF2 (D7)： T2计数溢出中断请求标志位。当T2计数溢出时，由内部硬件置位TF2，向CPU发出中断请求。但是当RCLK位或TCLK位为1时将不予置位。本标志位必须由软件清0。

EXF2 (D6)： T2外部中断请求标志位。当由引脚T2EX上的负跳变引起“捕捉”或“自动重装载”且EXEN2位为1，则置位EXF2标志位，并向CPU发出中断请求。该标志位必须由软件清0。

RCLK (D5) : 串行口接收时钟标志位。当**RCLK**位为**1**时，串行通信端使用**T2**的溢出信号作为串行通信方式**1**和方式**3**的接收时钟；当**RCLK**位为**0**时，使用**T1**的溢出信号作为串行通信方式**1**和方式**3**的接收时钟。

TCLK (D4) : 串行发送时钟标志位。当**TCLK**位为**1**时，串行通信端使用**T2**的溢出信号作为串行通信方式**1**和方式**3**的发送时钟；当**TCLK**位为**0**时，串行通信端使用**T1**的溢出信号作为串行通信方式**1**和方式**3**的发送时钟；

EXEN2 (D3) : **T2**外部采样允许标志位。当**EXEN2**位=**1**时，如果**T2**不是正工作在串行口的时钟，则在**T2EX**引脚 (**P1.1**) 上的负跳变将触发“捕捉”或“自动重装载”操作；当**EXEN2**位=**0**时，在**T2EX**引脚 (**P1.1**) 上的负跳变对**T2**不起作用。

TR2 (D2) : T2启动/停止控制位。当软件置位TR2时, 即TR2=1, 则启动T2开始计数, 当软件清TR2位时, 即TR2=0, 则T2停止计数。

C/T2* (D1) : T2的计数或定时方式选择位, 当设置C/=1时, 为对外部事件计数方式; C/=0时, 为定时方式。

CP/RL2 (D0) : T2捕捉/自动重载选择位。当设置CP/RL2=1时, 如果EXEN2为1, 则在T2EX引脚 (P1.1) 上的负跳变将触发“捕捉”操作; 当设置CP/RL2=0时, 如果EXEN2为1, 则T2计数溢出或T2EX引脚上的负跳变都将引起自动重载操作; 当RCLK位为1或TCLK位为1, CP/RL2标志位不起作用。T2计数溢出时, 将迫使T2进行自动重载操作。

由软件编程对**T2CON** 中的相关位进行设置来选择**T2** 的**3种工作方式**：**16位自动重载**（递增或递减计数）、**捕捉**和**波特率发生器**，如**表7-2**所示。

表 7-2 T2 的工作方式设置

RCLK+TCLK	CP/RL2	TR2	工作模式
0	0	1	16 位自动重载
0	1	1	16 位捕捉
1	×	1	波特率发生器
×	×	0	停止工作并关闭

2. 特殊功能寄存器T2MOD

与T2相关的另一个特殊功能寄存器为T2MOD。T2MOD寄存器的格式见图7-24。

T2MOD寄存器各位的定义如下：

T2OE (D1)： T2输出的启动位。

DCEN (D0)： 置位为1时允许T2增1/减1计数,并由T2EX引脚 (P1.1) 上的逻辑电平决定是增1还是减1计数。

—：保留位。

当单片机复位时，DCEN为0，默认T2为增1计数方式；当把DCEN置1时，将由T2EX引脚 (P1.1) 上的逻辑电平决定T2是增1还是减1计数。

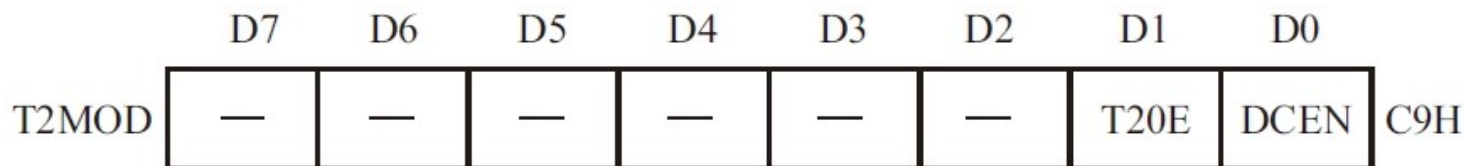


图7-24 T2MOD的格式

7.5.2 T2的16位自动重载方式

T2的16位自动重载工作方式见图7-25。

图中RCAP2L为陷阱寄存器低字节，字节地址为CAH；RCAP2H为陷阱寄存器高字节，字节地址为CBH。T2引脚为P1.0，T2EX引脚为P1.1，因此当使用T2时，P1.0和P1.1就不能作I/O口用了。另外有两个中断请求，通过一个“或”门输出。因此当单片机响应中断后，在中断服务程序中应该用软件识别是哪一个中断请求，分别进行处理，该中断请求标志位必须用软件清“0”。

(1) 当设置T2MOD寄存器的DCEN 位为0（或上电复位为0）时，T2为增1型自动重新装载方式，此时根据T2CON寄存器中的EXEN2位的状态，可选择两种操作方式：

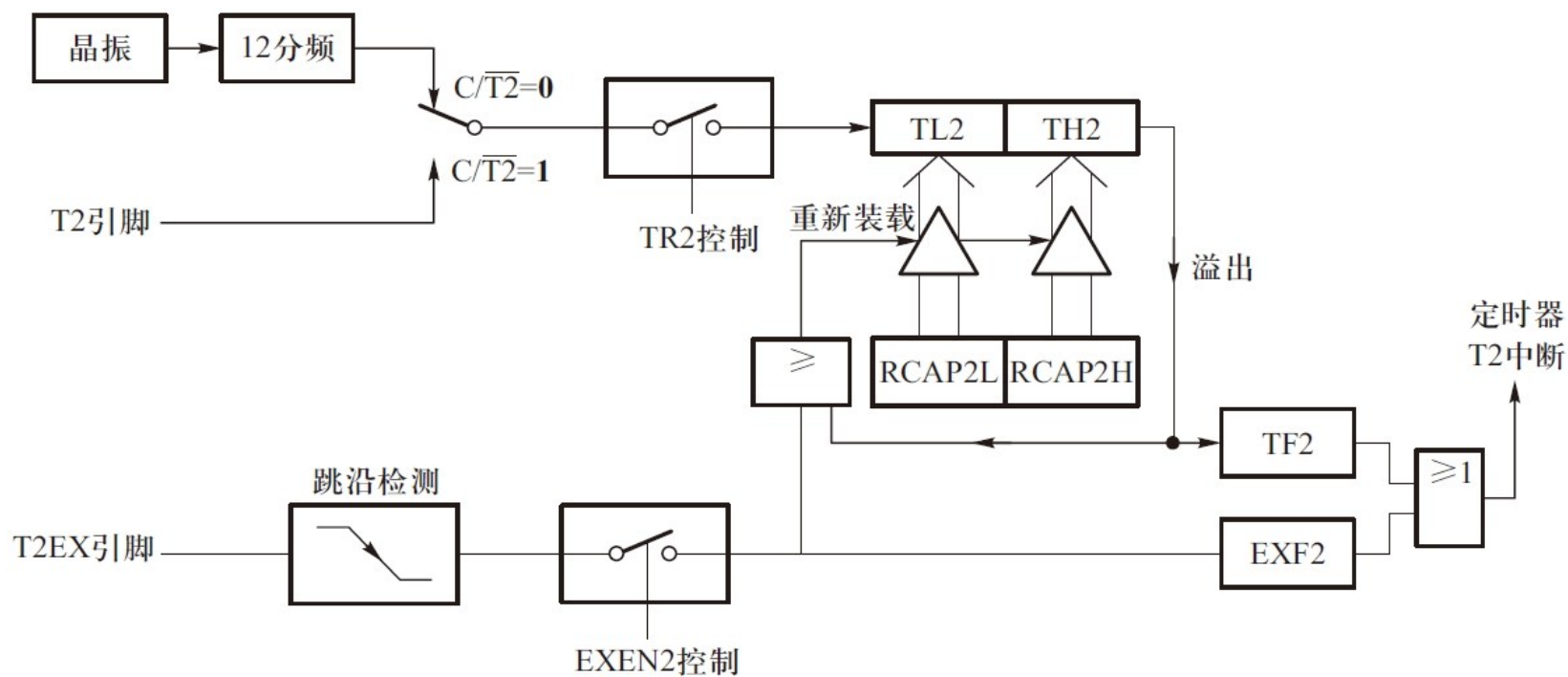


图7-25 T2的自动重载方式的工作示意图

① 当**EXEN2**标志位清0，**T2**计满溢出回0，一方面使中断请求标志位**TF2**置1，同时又将陷阱寄存器**RCAP2L**、**RCAP2H**中预置的16位计数初值自动重装入计数器**TL2**、**TH2**中，自动进行下一轮的计数操作，其功能与**T0**、**T1**的方式2（自动装载）相同，只是本计数方式为16位，计数范围大。**RCAP2L**、**RCAP2H**寄存器的计数初值由软件预置。

② 当设置**EXEN2**标志位为1，**T2**仍具有上述①的功能，并增加了新的特性。当外部输入引脚**T2EX**（**P1.1**）产生负跳变时，能触发三态门将**RCAP2L**、**RCAP2H**陷阱寄存器中的计数初值自动装载到**TH2**和**TL2**中，重新开始计数，并置位**EXF2**为1，发出中断请求。

(2) 当**T2MOD**寄存器的**DCEN**位置为1时，可以使**T2**既可以增1计数，也可实现减1计数，增1还是减1取决于**T2EX**引脚上的逻辑电平。图7-26为**T2**增1/减1计数方式的结构示意图。

由图7-26见，当设置DCEN 位为1时，可以使T2具有增1/减1计数功能。

当T2EX (P1.1) 引脚为“1”时，T2执行增1计数功能。当不断加1计满溢出回0时，一方面置位TF2为1，发出中断请求，另一方面，溢出信号触发三态门，将存放在陷阱寄存器RCAP2L、RCAP2H中的计数初值自动装载到TL2和TH2计数器中继续进行加1计数。

当T2EX (P1.1) 引脚为“0”时，T2执行减1计数功能。当TL2和TH2计数器中的值等于陷阱寄存器RCAP2L、RCAP2H中的值时，产生向下溢出，一方面置位TF2为1，发出中断请求，另一方面，下溢信号触发三态门，将0FFFFH装入TL2和TH2计数器中，继续进行减1计数。

中断请求标志位**TF2**和**EXF2**位必须用软件清0。

【例7-8】利用**T2**实现1秒定时并控制**P1.0**引脚上的**LED** 1秒闪灭1次，晶振频率为**12MHz**。

编程思想：将**T2**设置为**1/16**秒的定时，定时中断**16**次，即为**1**秒，**1**秒时间到后，把**P1.0**的状态求反。

定时初值x计算：每秒中断**16**次，则每次溢出为 $1000000/16=62500$ 个机器周期。

因此： $65536-x=62500$ ，初值 $x=3036=0BDCH$ 。

#include "reg52.h"	//头文件
#define uint unsigned int	
#define uchar unsigned char	
sbit P1_0=P1^0;	//定义 P1.0 位变量
 void T2_INT(void) interrupt 5	 //T2 中断函数
{	
static uint i=0;	// 静态变量 i 为 T2 的溢出次数
TF2=0;	// 用软件清溢出中断标志 TF2
i++;	// 溢出次数增 1
if(i==16)	// 如果 T2 溢出 16 次则 1 秒定时到
{	
i=0;	// T2 溢出次数清 0
P1_0=~P1_0;	// P1.0 引脚的电平取反
}	
}	


```

void main (void)
{
    CP_RL2=0;EXEN2=0;           // 设置 T2 为 16 位自动重装载定时器工作方式
    P1_0=1;                     // P1.0 引脚 LED 熄灭
    TH2=RCAP2H=0x0B;           // 给 T2 赋预装载初值，溢出时间为 1/16 秒
    TL2=RCAP2L=0xDC;
    ET2=1;                      // T2 中断允许
    EA=1;                       // 总中断允许
    TR2=1;                      // 启动 T2
    while(1);                   // 循环等待 T2 的 1/16 秒的溢出中断
}

```

程序说明：在中断函数中用到了静态变量“**static uint i**”。静态变量的特点是语句执行后，其占用的存储单元不释放，在下一次执行该语句时，该变量仍为上一次的值，它只需赋一次初值。也就是说，只有在第一次进中断时“**uint i=0**”，才对*i*赋值，以后再进入中断时，不会再对*i*赋值。

7.5.3 T2的捕捉方式

捕捉方式就是及时“捕捉”住输入信号发生的跳变及有关信息。常用于精确测量输入信号的变化如脉宽等等。捕捉方式的工作示意结构见图7-27。

根据T2CON寄存器中EXEN2位的不同设置，“捕捉”方式有两种选择。

(1) 当EXEN2位=0时，T2是一个16位的定时器/计数器。当设置C/T2*位为1时，选择外部计数方式，即对T2引脚（P1.0）上的负跳变信号进行计数。计数器计满溢出时置1中断请求标志TF2，发出中断请求信号。CPU响应中断进入该中断服务程序后，必须用软件将标志位TF2清0。其他操作均与T0和T1的工作方式1相同。

(2) 当**EXEN2**位=1时，**T2**除上述功能外，还可增加“捕捉”功能。当外部**T2EX**引脚（**P1.1**）上的信号发生负跳变，将选通三态门控制端（见图7-27“捕捉”处），把计数器**TH2**和**TL2**中的当前计数值分别“捕捉”进**RCAP2L**和**RCAP2H**中，同时**T2EX**引脚（**P1.1**）上的信号负跳变将置位**T2CON**的**EXF2**标志位，向**CPU**请求中断。

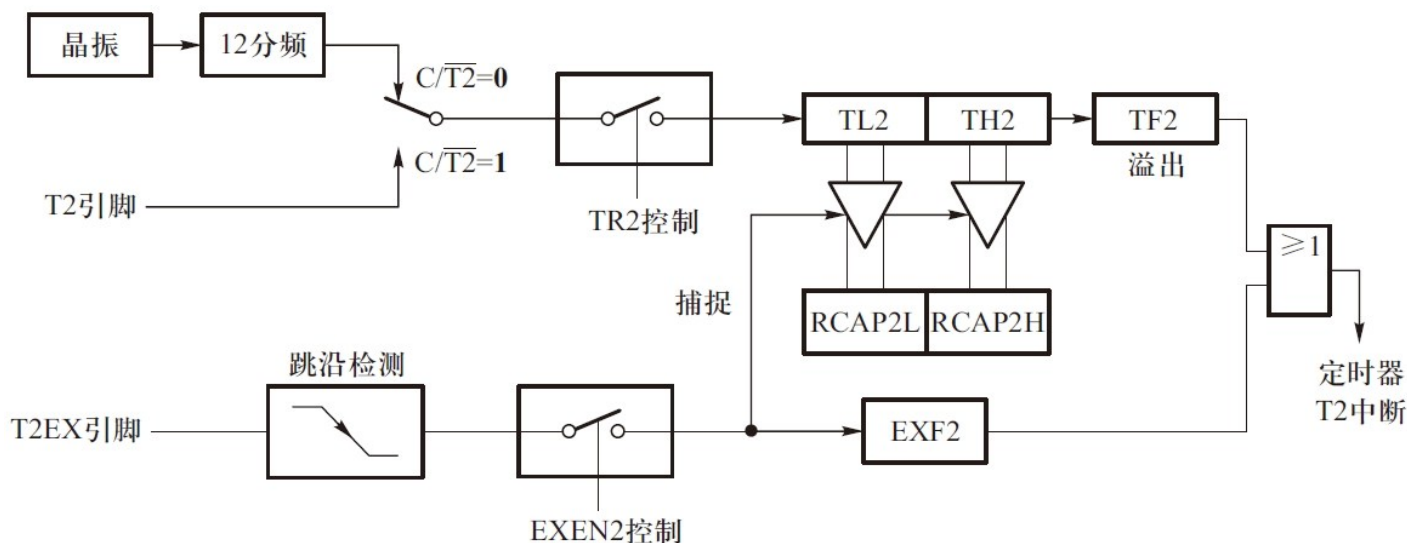


图7-27 T2的捕捉方式结构示意图

7.5.4 T2的波特率发生器方式及可编程时钟输出

T2可工作于波特率发生器方式，还可作为可编程时钟输出。

1. 波特率发生器方式

T2具有专用的“波特率发生器”（波特率发生器就是控制串行口接收/发送数字信号的时钟发生器）的工作方式。通过软件置位T2CON寄存器中的RCLK和/或TCLK，可将T2设置为波特率发生器。需要注意的是，如果T2用于波特率发生器和T1用于别的功能，则这个接收/发送波特率可能是不同的。

当置位RCLK和/或TCLK，T2进入波特率发生器模式，如图7-28所示。由图7-28，当设置T2CON寄存器中的C/为0，设置RCLK和/或TCLK为1时，输出16分频的接收/发送波特率。

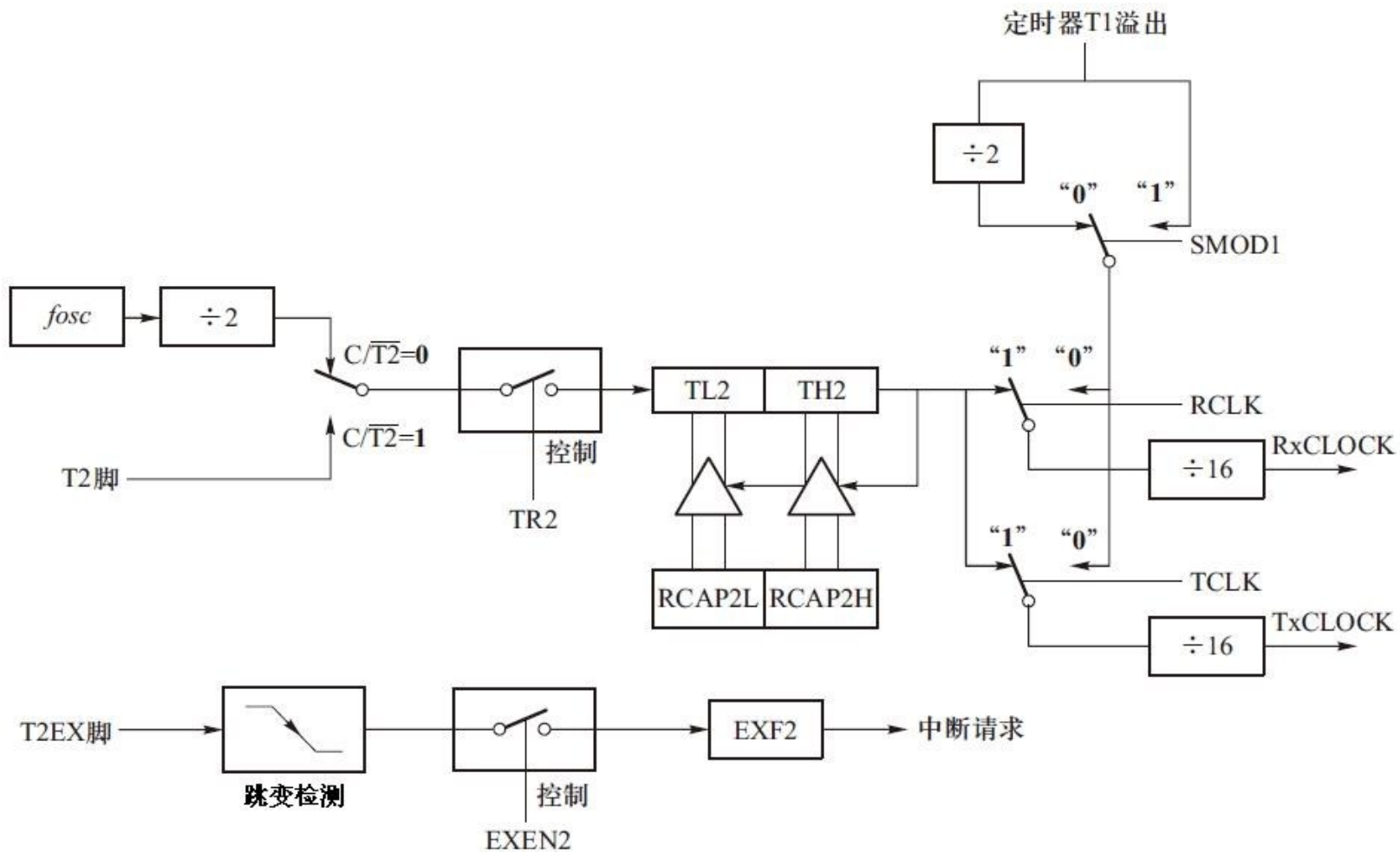


图7-28 T2作为串行通信波特率发生器示意图



另外通过对**T2EX**引脚（**P1.1**）跳变信号的检测，并置位**EXF2**中断请求标志位，向**CPU**请求中断。需要注意的是，图7-28中的主振频率**fosc**是经过2分频，而不是12分频。

T2工作在波特率发生器方式，属于**16**位自动重载的定时模式。串行通信方式**1**和方式**3**（见第8章的介绍）的波特率计算公式为：

串行通信方式1和方式3的波特率 = 定时器T2的溢出率/16

T2的波特率发生器可选择定时模式或计数模式，一般都选择定时模式。注意，在选择定时器使用时，是主振频率**fosc**经12分频为一个机器周期作为

加1计数信号，而作为波特率发生器使用时是以每个时钟状态S（2分频主振频率）作为加1计数信号。

因此串行通信方式1和方式3的波特率计算公式为：

$$\text{方式1和方式3的波特率(b/s)} = f_{\text{osc}} / \{32 \times [65536 - (\text{RCAP2H RCAP2L})]\} \quad (7-1)$$

式（7-1）中“RCAP2H RCAP2L”为T2的初值。例如“RCAP2H RCAP2L”初值为FFFFH，则 $65536 - 65535 = 1$ ，则式（7-1）的波特率 $= (f_{\text{osc}} / 32) \text{b/s}$ 。

设主振频率 $f_{\text{osc}} = 12\text{MHz}$ ，则上述波特率 $= 375\text{kb/s}$ 。

从式（7-1）可见，采用T2用作波特率发生器，其波特率设置范围极广。

从图7-28可见，当T2用作波特率发生器时，具有以下特点：

- ① 必须设置**T2CON**寄存器中的**RCLK**和/或**TCLK**为1（有效）；
- ② 计数器溢出再装载,但不会置位**TF2**向**CPU**请求中断；
- ③ 如果**T2EX**引脚上发生负跳变将置位**EXF2**为1，向**CPU**请求中断处理，但不会将陷阱寄存器“**RCAP2H RCAP2L**”中预置的计数初值装入**TH2**和**TL2**中。因此，可将**T2EX**引脚用作额外的输入引脚或外部中断源；
- ④ 采用定时模式作波特率发生器时，是对**fosc**经2分频（时钟状态**S**）作为计数单位，而不是**fosc**经12分频的机器周期信号。
- ⑤ 波特率设置范围广，精确度高。

另外要注意的是，**T2**在波特率工作模式下作为定时器模式时（**TR2**为1），不能对**TH2**、**TL2**进行读写。这时的**T2**是以每个时钟状态（**S**）进行加1计数，这时进行读写可能出错。对陷阱寄存器**RCAP2**可以读，但不能写，因为写**RCAP2**可能会覆盖重装的数据并使装入出错。处理**T2**或**RCAP2**寄存器前不能关闭**T2**（即清0 **TR2**位）。

2. 可编程时钟信号输出

T2可通过软件编程在**P1.0**引脚输出时钟信号。**P1.0**除用作通用I/O引脚外还有两个功能可供选用：用于**T2**的外部计数输入和频率为**61Hz~4MHz**的时钟信号输出。**图7-29**为时钟输出和外部事件计数方式示意图。

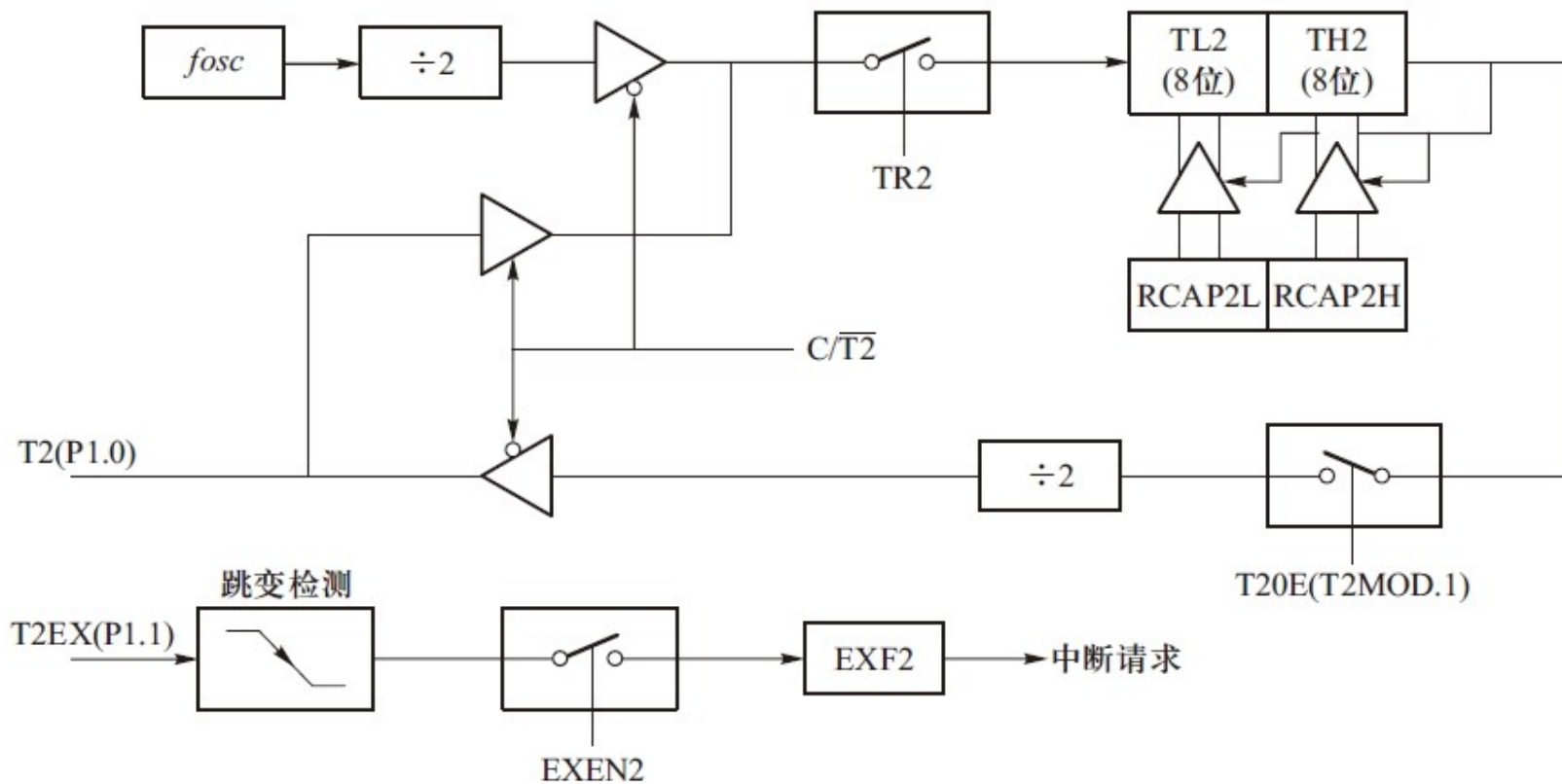


图7-29 T2时钟输出和外部事件计数方式示意图

通过软件对**T2CON.1**位**C**/复位为0，对**T2MOD.1**位**T2OE**置1就可将**T2**选
定为时钟信号发生器，而**T2CON.2**位**TR2**控制时钟信号输出开始或结束
（**TR2**为启动/停止控制位）。由主振频率**fosc**和**T2**定时、自动重载方式的
计数初值决定时钟信号的输出频率,其设置公式如下：

$$\text{时钟信号输出频率} = (12 \times 10^6) / [4 \times (65536 - (\text{RCAP2H} \text{ RCAP2L}))] \quad (7-2)$$

从式（7-2）可见，在主振频率（**fosc**）设定后，时钟信号输出频率就取
决于计数初值。

在时钟输出模式下，计数器溢出回0不会产生中断请求。这种功能相当于
T2用作波特率发生器，同时又可用作时钟发生器。但必须注意，无论如何波
特率发生器和时钟发生器都不能单独确定各自不同的频率。原因是两者都用
同一个陷阱寄存器**RCAP2H**、**RCAP2L**，不可能出现两个计数初值。

第8章

AT89S52单片机的串行口



内容概要

本章介绍**AT89S52**单片机片内全双工通用异步收发（**UART**）串行口的基本结构与工作原理以及相关的特殊功能寄存器，串行口的**4**种工作方式。

本章还介绍如何利用单片机串行口实现多机串行通信，单片机串行通信的各种应用编程以及单片机与**PC**机的串行通信。

此外，从实用角度对目前单片机串行通信广泛使用的各种常见的标准串行通信接口**RS232**、**RS422**以及**RS485**也作以简要介绍。

8.1 串行通信基础

随着单片机的广泛应用与计算机网络技术的普及，单片机与个人计算机或单片机与单片机之间的通信使用较多。

8.1.1 并行通信与串行通信

单片机的数据通信有并行通信与串行通信两种方式。

1. 并行通信

单片机的并行通信通常使用多条数据线将数据字节的各个位同时传送，每一位数据都需要一条传输线，此外还需要一条或几条控制信号线。并行通信的示意图见图8-1。

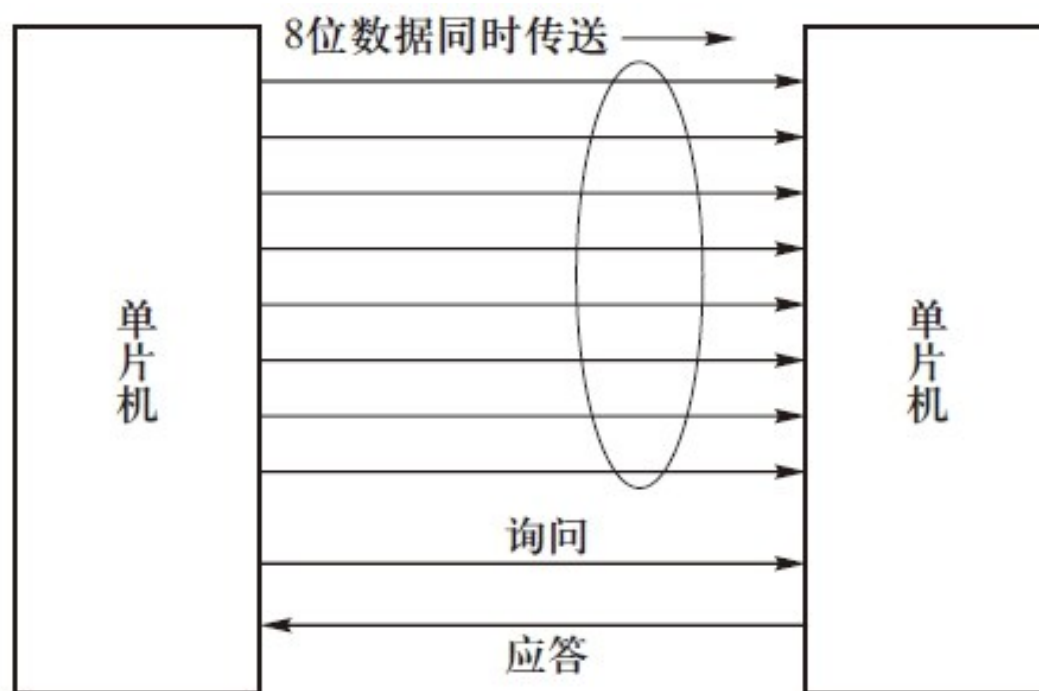


图8-1 并行通信示意图

并行通信相对传输速度快。但由于传输线较多，长距离传送时成本高，因此这种方式适合于短距离的数据传输。

2. 串行通信

单片机串行通信是将数据字节分成一位一位的形式在一条传输线上逐个传送。一次只能传送一位，对于一个字节的数据，至少要分8位才能传送完毕。如图8-2所示。

串行通信在发送时，要把并行数据变成串行数据发送到线路上去，接收时要把串行数据再变成并行数据。

串行通信传输线少，长距离传送时成本低，且可以利用电话网等现成设备，因此在单片机应用系统中，串行通信的使用非常普遍。



图8-2 串行通信的示意图

8.1.2 同步通信与异步通信

串行通信又有两种方式：异步通信与同步通信。

同步串行通信是采用一个**同步时钟**，通过一条同步时钟线，加到收发双方，使收、发双方达到完全同步，此时，传输数据的位之间的距离均为“位间隔”的整数倍，同时传送的字符间不留间隙，既保持位同步关系。同步通信及数据格式见**图8-3**。

异步串行通信是指收、发双方**使用各自的时钟**控制数据的发送和接收，这样可省去连接收、发双方的一条同步时钟信号线，使得异步串行通信连接更加简单且容易实现。为使收发双方协调，要求收、发双方的时钟尽可能一致。

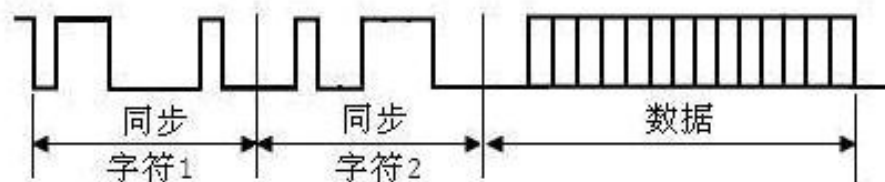
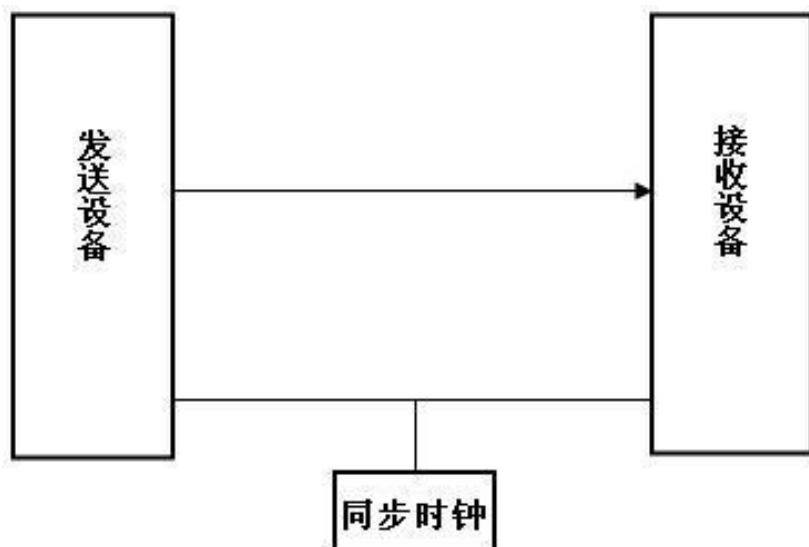


图8-3 同步通信及数据格式

图8-4给出了异步串行通信的示意图以及数据帧格式。异步串行通信是以数据帧为单位进行数据传输，各数据帧之间的间隔是任意的，但每个数据帧中的各位是以固定的时间传送的。

异步串行通信不要求收、发双方时钟严格一致，实现容易，成本低，但是每个数据帧要附加起始位、停止位有时还要再加上校验位。

同步串行通信相比异步串行通信，同步串行通信数据传输的效率较高，但是额外增加了一条同步时钟线。

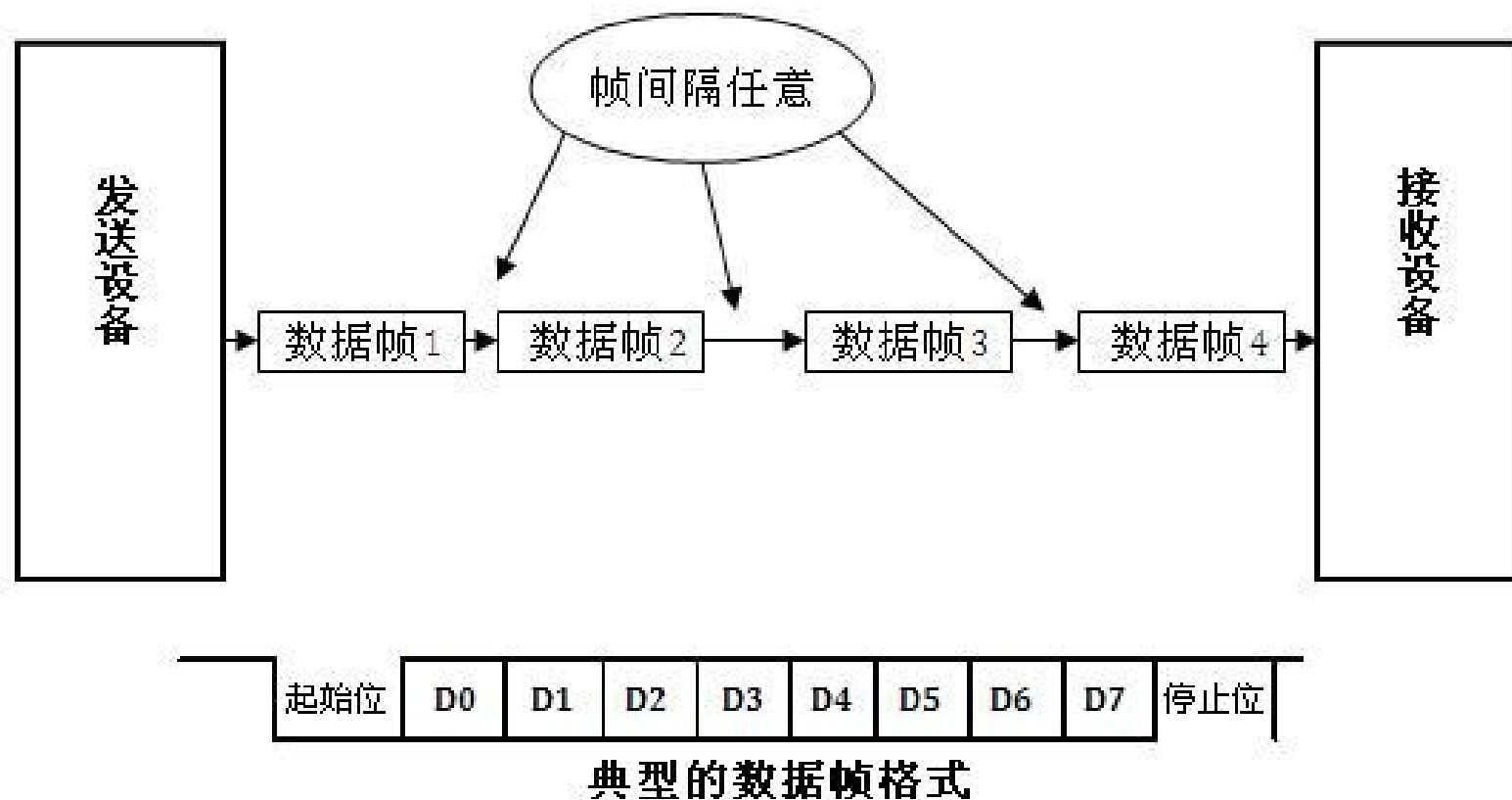


图8-4 异步串行通信

8.1.3 串行通信的传输模式

串行通信按照按照数据传输的方向及时间关系可分为单工、半双工和全双工。

1.单工

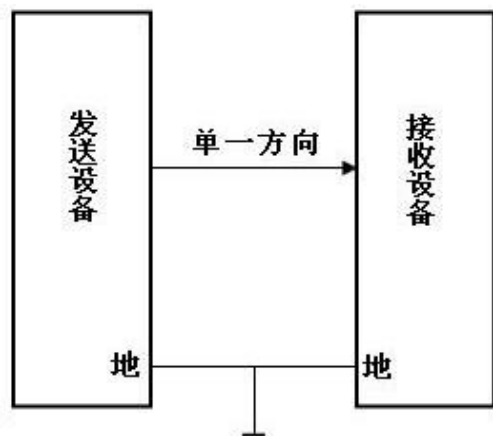
数据传输仅能按一个固定方向传输，不能反向传输，如图8-5（a）。

2.半双工

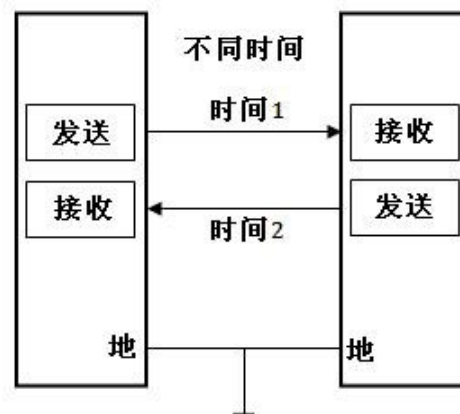
数据传输可双向传输，但不能同时传输，如图8-5（b）。

3.全双工

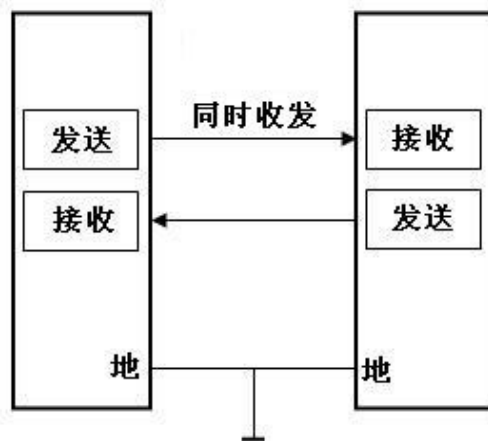
数据传输可同时进行双向传输，如图8-5（c）所示。



(a) 单工



(b) 半双工



(c) 全双工

图8-5 单工、半双工和全双工的数据传输模式

8.1.4 串行通信的错误校验

在串行通信中，往往要对数据传送的正确与否进行校验。校验时保证传输数据准确无误的关键。常用的有奇偶校验与循环冗余码校验等方法。

1. 奇偶校验

串行发送数据时，数据位尾随1位奇偶校验位（1或0）。

当约定为奇校验时，数据中“1”的个数与校验位“1”的个数之和应为奇数；

当约定为偶校验时，数据中“1”的个数与校验位“1”的个数之和应为偶数。数据发送方与接收方应一致。在接收数据帧时，对“1”的个数进行校验，若发现不一致，则说明数据传输过程中出现了差错，则通知发送端重发。

2. 代码和校验

代码和校验是发送方将所发数据块求和或各字节异或，产生一个字节的校验字符（校验和）附加到数据块末尾。接收方接收数据时同时对数据块（除校验字节）求和或各字节异或，将所得结果与发送方的“校验和”进行比较，如果相符，则无差错，否则即认为在传输过程中出现了差错。

3. 循环冗余码校验

循环冗余码校验纠错能力强，容易实现。校验是通过某种数学运算实现有效信息与校验位之间的循环校验，常用于对磁盘信息的传输、存储区的完整性校验等。是目前应用最广的检错码编码方式之一，广泛用于同步通信中。

8.2 串行口的结构

结构见图8-6。

有两个物理上独立的接收、发送缓冲器**SBUF**（属于特殊功能寄存器），可同时发送、接收数据。

发送缓冲器只能写入不能读出

接收缓冲器只能读出不能写入

两个缓冲器**共用一个特殊功能寄存器字节地址（99H）**。

控制寄存器共有两个：特殊功能寄存器**SCON**和**PCON**。

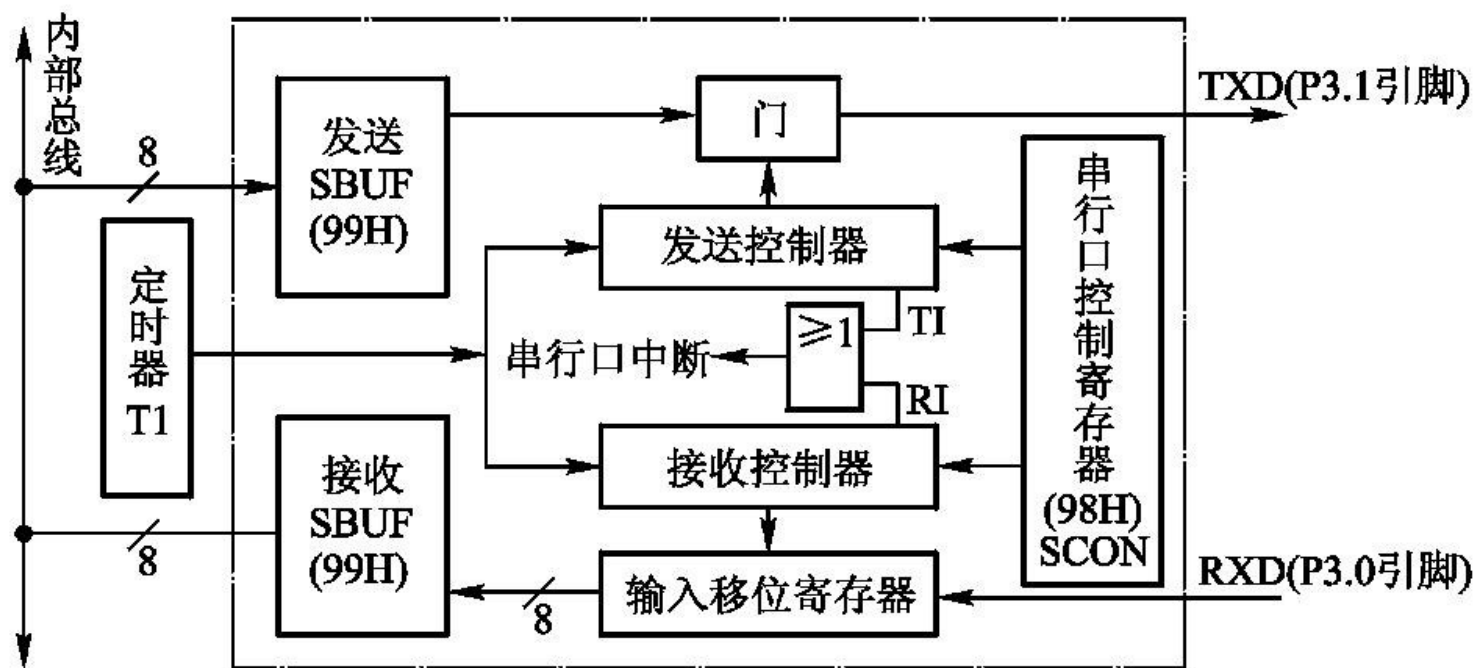


图8-6 串行口的内部结构图

8.2.1 串行口控制寄存器SCON

字节地址**98H**，可位寻址，**位地址**为98H~9FH。格式如**图8-7**。

		D7	D6	D5	D4	D3	D2	D1	D0		
SCON		SM0	SM1	SM2	REN	TB8	RB8	TI	RI	98H	
	位地址	9FH	9EH	9DH	9CH	9BH	9AH	99H	98H		

图8-7 串行口控制寄存器SCON的格式

介绍SCON中各位的功能。

(1) SM0、SM1——串行口4种工作方式选择位

SM0、SM1所对应的**4种工作方式**见**表8-1**。

表8-1 串行口的 4 种工作方式

SM0	SM1	方 式	功 能 说 明
0	0	0	同步移位寄存器方式（用于扩展 I/O 口）
0	1	1	8 位异步收发，波特率可变（由定时器控制）
1	0	2	9 位异步收发，波特率为 $f_{osc}/64$ 或 $f_{osc}/32$
1	1	3	9 位异步收发，波特率可变（由定时器控制）

（2）SM2——多机通信控制位

多机通信是在方式2和方式3下进行。当串口以方式2或方式3接收时，如果SM2=1，则只有当接收到的第9位数据（RB8）为“1”时，才使RI置“1”，产生中断请求，并将接收到的前8位数据送入SBUF。

当接收到的第9位数据（RB8）为“0”时，则将接收到的前8位数据丢弃。

当**SM2= 0**时，则**不论第9位数据是1还是0**，都将前8位数据送入SBUF中，并使**RI置1**，产生中断请求。

在**方式1**时，如果**SM2=1**，则只有收到有效的停止位时才会激活RI。

在**方式0**时，**SM2**必须为0。

(3) REN——允许串行接收位。

由软件置“1”或清“0”。

REN=1，**允许**串行口接收数据。

REN=0，**禁止**串行口接收数据。

(4) TB8——发送的第9位数据

方式2和方式3，TB8是要发送的第9位数据，其值由软件置“1”或清“0”。在双机串行通信时，一般作为奇偶校验位使用；在多机串行通信中用来表示主机发送的是地址帧还是数据帧，TB8=1为地址帧，TB8=0为数据帧。

(5) RB8——接收的第9位数据

方式2和方式3，RB8存放接收到的第9位数据。在方式1，如SM2=0，RB8是接收到的停止位。在方式0，不使用RB8。

(6) TI——发送中断标志位

方式0，串行发送的第8位数据结束时TI由硬件置“1”，在其他方式中，串行口发送停止位的开始时置TI为“1”。

TI = 1，表示**一帧数据发送结束**。TI的状态可供软件查询，也可申请中断。CPU响应中断后，在中断服务程序中向SBUF写入要发送的下一帧数据。**TI必须由软件清“0”**。

(7) RI—接收中断标志位

方式0时，**接收完第8位数据**时，RI由硬件置“1”。在其他工作方式中，串行接收到停止位时，该位置“1”。**RI = 1**，表示一帧数据接收完毕，并申请中断，要求CPU从接收SBUF取走数据。该位的状态也可供软件查询。**RI必须由软件清“0”**。

SCON的**所有位**都可进行位操作清“0”或置“1”。

8.2.2 特殊功能寄存器PCON

字节地址为87H，不能位寻址。格式如图8-8所示。

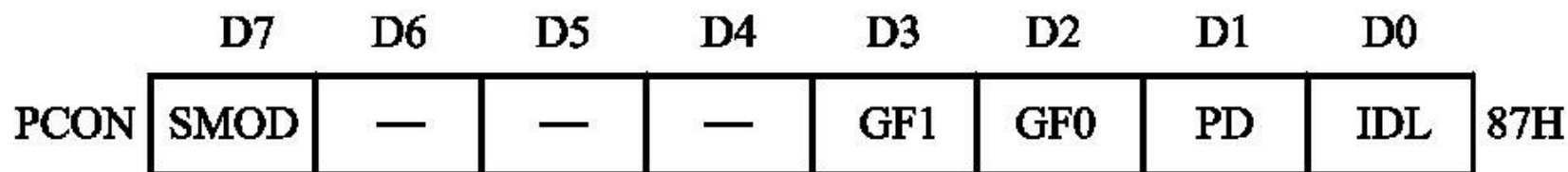


图8-8 特殊功能寄存器PCON的格式

下面介绍PCON中各位功能。仅最高位**SMOD**与串口有关，其他各位的功能已在第2章的节电工作方式一节中作过介绍。

SMOD：波特率选择位。

例如，方式1的波特率计算公式为

$$\text{方式1波特率} = \frac{2^{\text{SMOD}}}{32} \times \text{定时器T1的溢出率}$$

当**SMOD** = 1时，要比**SMOD** = 0时的波特率加倍，所以也称**SMOD**位为**波特率倍增位**。

8.3 串行口的4种工作方式

4种工作方式由SCON中SM0、SM1位定义，编码见表6-1。

8.3.1 方式0

方式0为同步移位寄存器输入/输出方式。该方式并不用于两个单片机之间的异步串行通信，而是用于串行口外接移位寄存器，扩展并行I/O口。

8位数据为一帧，无起始位和停止位，先发送或接收最低位。波特率固定，为 $f_{\text{osc}}/12$ 。帧格式如图8-9所示。

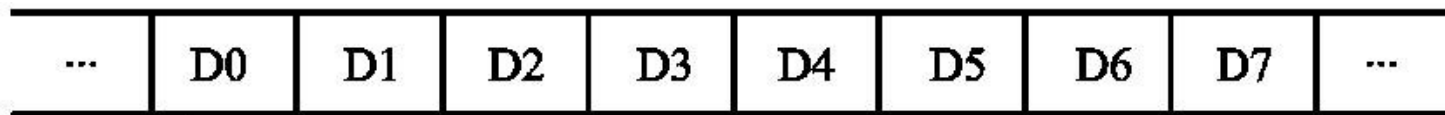


图8-9 方式0的帧格式

1. 方式0输出

(1) 方式0输出的工作原理

当CPU执行一条将数据写入发送缓冲器SBUF的指令时，产生一个正脉冲，串行口开始把SBUF中的8位数据以 $f_{osc}/12$ 的固定波特率从RXD引脚串行输出，低位在先，TXD引脚输出同步移位脉冲，发送完8位数据，中断标志位TI置“1”。发送时序如图8-10所示。

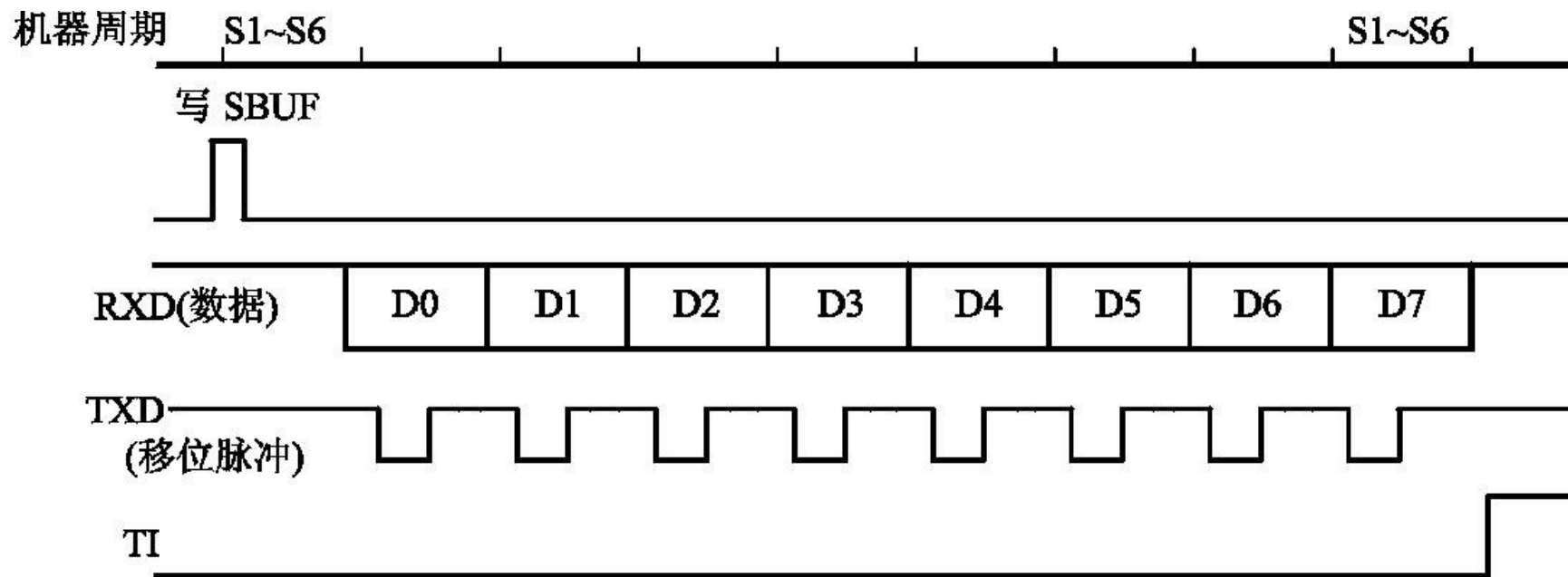


图8-10 方式0发送时序

(2) 方式0输出的应用案例

方式0输出的典型应用是串行口**外接串行输入/并行输出的同步移位寄存器74LS164**，实现并行输出端口的扩展。

图8-11所示为串行口工作在方式0，通过**74LS164**的输出来控制**8**个**外接LED**发光二极管亮灭的接口电路。当串行口被设置在方式0输出时，串行数据由**RXD**端（**P3.0**）送出，移位脉冲由**TXD**端（**P3.1**）送出。在移位脉冲的作用下，串行口发送缓冲器的数据逐位地从**RXD**端串行地移入**74LS164**中。

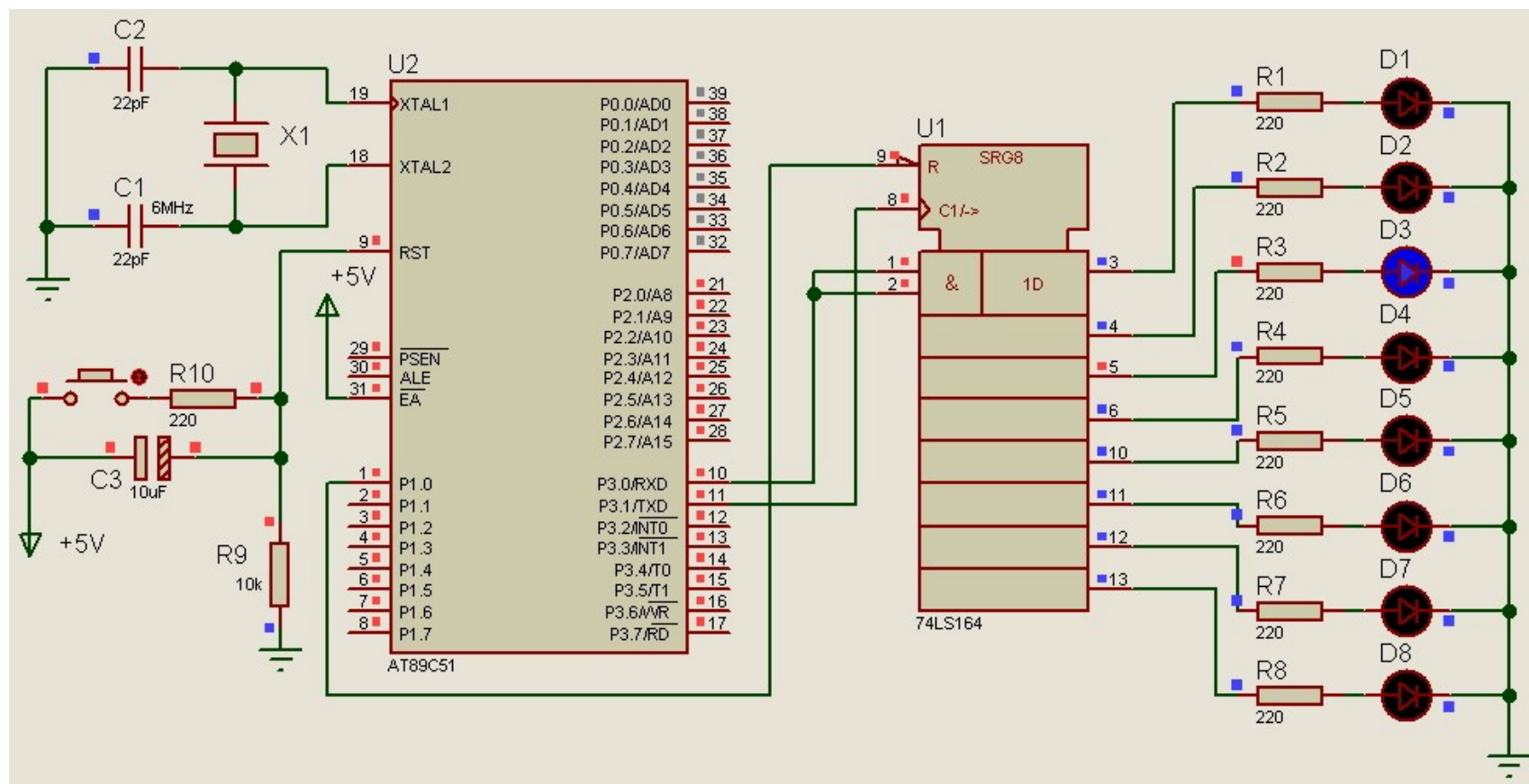


图8-11 方式0输出外接8个LED发光二极管接口电路

【例8-1】如图8-11所示，编写程序控制8个发光二极管流水点亮。

图中74LS164的8脚（CLK端）为同步脉冲输入端，9脚为控制端，由单片机的P1.0控制，当9脚为0时，允许串行数据由RXD端（P3.0）向74LS164的串行数据输入端（1脚和2脚）输入，此时74LS164的8位并行输出端关闭；当9脚为1时，串行数据输入端（1脚和2脚）关闭，但是允许74LS164中的8位数据并行输出。

当串行口将8位串行数据发送完毕后，申请中断，在中断服务程序中，单片机向串行口输出下一个8位数据。

采用中断方式的参考程序如下：


```
#include <reg51.h>
```

```
#include <stdio.h>
```

//包含移位函数的头文件

```
sbit P1_0=0x90;
```

```
unsigned char nSendByte;
```

```
void delay(unsigned int i)
```

//延时函数

```
{
```

```
    unsigned char j;
```

```
    for(;i>0;i--)
```

//变量i由实际参数传入一个值，因此i不能赋初值

```
        for(j=0;j<125;j++)
```

```
        ;
```

```
}
```

```
main( )
```

//主函数

```
{
```

```
    SCON=0x00;
```

// 设置串行口为方式0

```
    EA=1;
```

// 全局中断允许

```

ES=1;                // 允许串行口中断
nSendByte=1;         // 点亮数据初始值为0000 0001送入
nSendByte
SBUF=nSendByte;      // CPU向SBUF写入点亮数据，启动串行发送
P1_0=0;              // P1.0=0控制串行口向74LS164串行发送数据
while(1)
{;}
}

```

```

void Serial_Port( ) interrupt 4 using 0           //串行口中断函数
{
    if(TI)                                         // 判TI的值，如果TI=1，1个字节串行发完
    {
        P1_0=1;                                  // P1_0=1，允许74LS164并行输出，流水点亮二极管
    }
}

```

```

SBUF=nSendByte;                                // 向SBUF写入数据，启动串行发送
delay(500);                                       // 延时，点亮二极管持续一段时间
P1_0=0;                                           // P1_0=0，允许向74LS164串行写入，关闭并行输出
nSendByte=nSendByte<<1;                          // 点亮二极管的数据左移1位
if(nSendByte==0) nSendByte=1;
                                                    // 判点亮数据是否左移8次？是，重新向nSendByte
                                                    // 写入点亮数据0000 0001

SBUF=nSendByte;                                // 向74LS164再次串行发送点亮数据
}
TI=0;
RI=0;
}

```

程序说明如下：

(1) 程序的第4行定义了全局变量**nSendByte**，以便在中断函数中能访问该变量。全局变量**nSendByte**用于存放从串行口发出的点亮数据，在程序中使用左移1位操作符“<<”对变量**nSendByte**进行移位，使得从串口发出的数据为**0x01**、**0x02**、**0x04**、**0x08**、**0x10**、**0x20**、**0x40**、**0x80**，从而流水点亮各个发光二极管。

(2) 程序中if语句的作用是当**nSendByte**左移1位8次由**0x80**变为**0x00**后，需对变量**nSendByte**重新赋值为**0x01**。

(3) 主程序中的**SBUF=nSendByte**语句必不可少，如果没有该语句，主程序就不会启动从串行口发送数据，也就不会产生随后的发送完成中断。

(4) 语句“**while (1) {;}**”用于实现反复循环。

2. 方式0输入

(1) 方式0输入的工作原理

方式0输入，REN为串行口允许接收控制位，0—禁止接收；1—允许接收。

当向SCON寄存器写入控制字（设置为方式0，并使REN位置1，同时RI = 0）时，产生一个正脉冲，串行口开始接收数据。

引脚RXD为数据输入端，TXD为移位脉冲信号输出端，接收器以 $f_{osc}/12$ 的固定波特率采样RXD引脚的数据信息，当接收完8位数据时，中断标志RI置1，表示一帧数据接收完毕，可进行下一帧数据的接收，时序如图8-12所示。

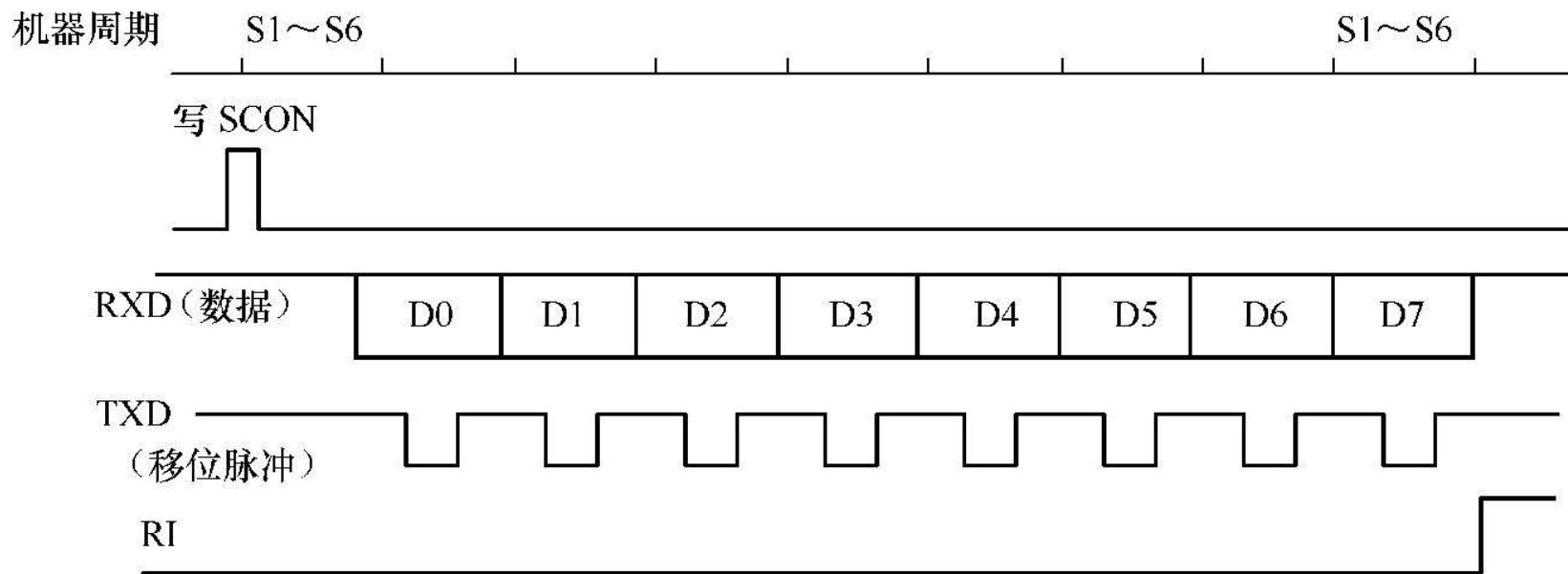


图8-12 方式0接收时序

(2) 方式0输入的应用举例

【例8-2】图8-13所示为串行口外接一片8位并行输入、串行输出的同步移位寄存器74LS165，从而实现了扩展一个8位并行输入口，可将接在74LS165的8个开关S0~S7的状态通过串行口的方式0输入方式读入到单片机内。

74LS165的SH/LD*端（1脚）为控制端，由单片机的P1.1脚控制。若SH/LD*=0，则允许74LS165可以并行输入数据，且串行输出端关闭；当SH/LD*=1，则并行输入关断，可向单片机串行发送数据。

当P1.0连接的开关k合上时，可对反映开关S0~S7的状态数字量并行读入，见图8-13。单片机采用中断方式来对S0~S7状态进行读取，并由单片机的P2口控制对应的开关按下的二极管发光点亮。

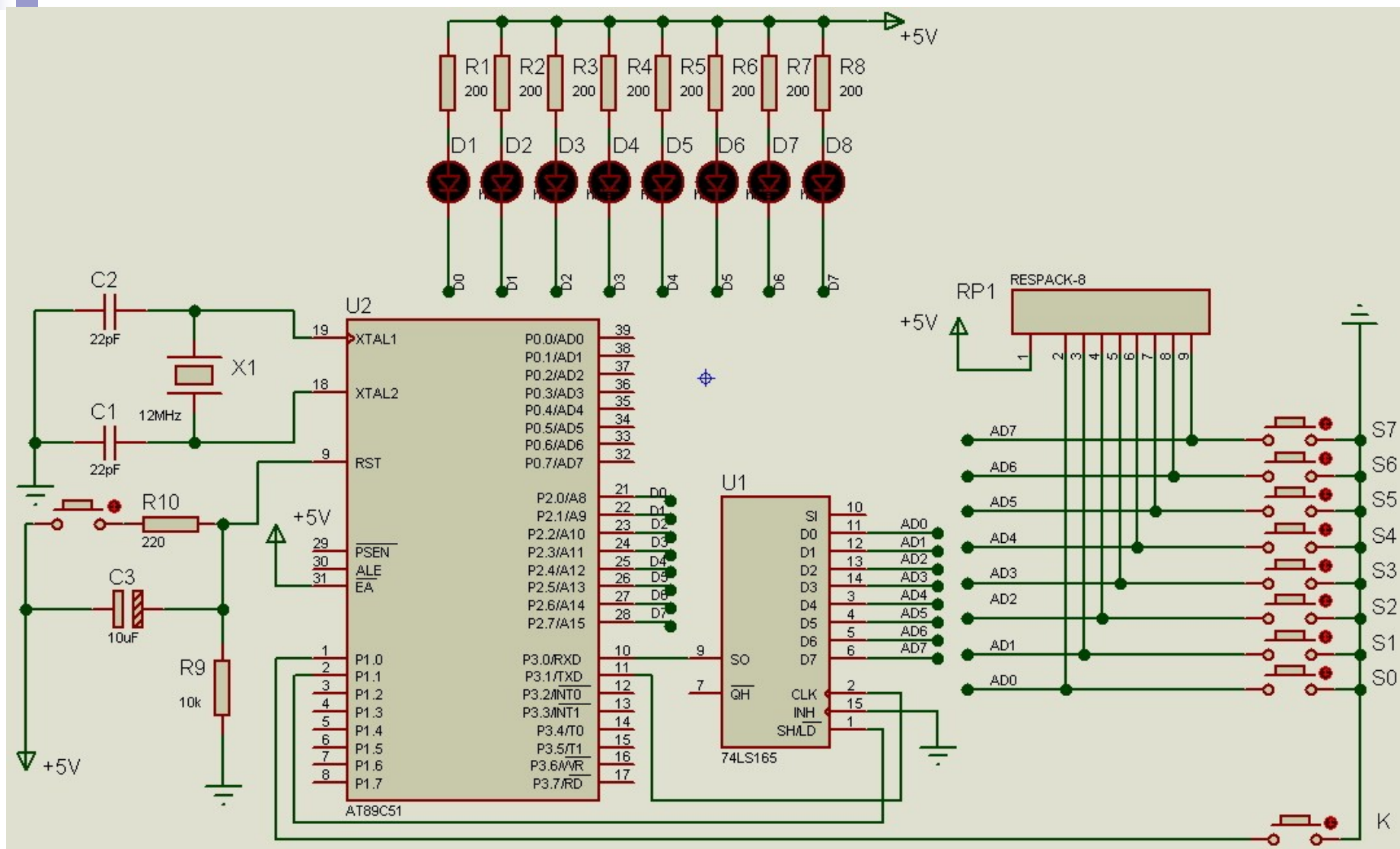


图8-13 串口方式0外接并行输入、串行输出的同步移位寄存器

参考程序如下：

```
#include <reg51.h>
```

```
#include "intrins.h"
```

```
#include<stdio.h>
```

```
sbit P1_0=0x90;
```

```
sbit P1_1=0x91;
```

```
unsigned char nRxByte;
```

```
void delay(unsigned int i)
```

```
{
```

```
    unsigned char j;
```

```
    for(;i>0;i--)
```

```
        for(j=0;j<125;j++);
```

```
}
```

//延时函数

//变量i由实际参数传入一个值



```
main()
```

```
{  
    SCON=0x10;           // 向SCON写入控制字，初始化为方式0  
    ES=1;                // 允许串行口中断  
    EA=1;                // 允许全局中断  
    for(;;);  
}
```

```
void Serial_Port() interrupt 4 using 0    // 串行口中断函数
```

```
{  
    if(P1_0==0)          // 如果P1_0=0表示开关k按下，可以读开关S0~S7的状态  
    {  
        P1_1=0;          // P1_1=0并行读入开关S0~S7的状态  
        delay(1);  
        P1_1=1;          // P1_1=1将开关S0~S7状态数据，串行读入到串行口中  
        RI=0;            // 接收中断标志RI清0  
    }
```

```
nRxByte=SBUF; // 接收的开关状态数据从SBUF读入到nRxByte 单元中
P2=nRxByte;    // 开关S0~S7的状态数据送P2口，驱动发光二极管发光
}
}
```

程序说明

当P1.0为0，即开关k按下，表示允许并行读入开关S0~S7的状态数字量，通过P1.1把SH/LD* =0，则并行读入开关S0~S7的状态。再让P1.1=1，即SH/LD* =1，74LS165将刚才读入的S0~S7状态数据通过QH端（RXD脚）串行读入到单片机的SBUF中，在中断函数中把SBUF中的数据读到nRxByte单元，并送到P2口控制驱动8个发光二极管点亮。

8.3.2 方式1

方式1为双机串行通信方式，如图8-14所示。

当SM0、SM1=01时，串行口设为方式1的双机串行通信。TXD脚和RXD脚分别用于发送和接收数据。

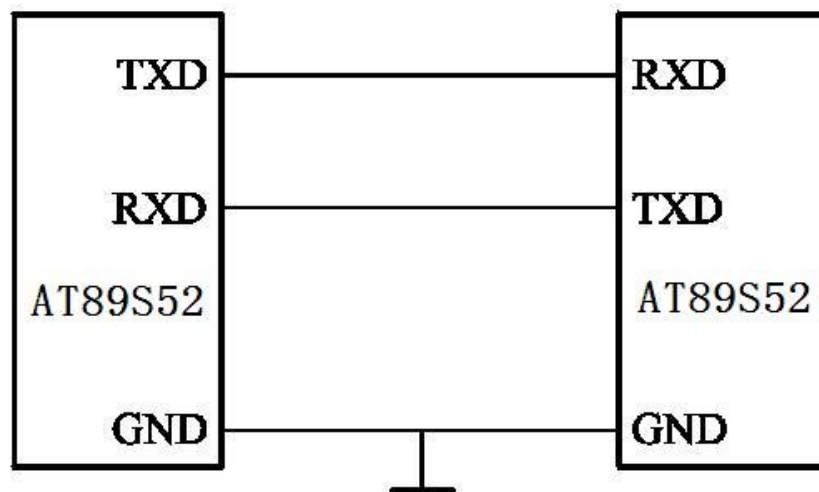


图8-14 方式1双机串行通信的连接电路

方式1一帧为10位，1个起始位（0），8个数据位，1个停止位（1），先发送或接收最低位。帧格式如图8-15所示。

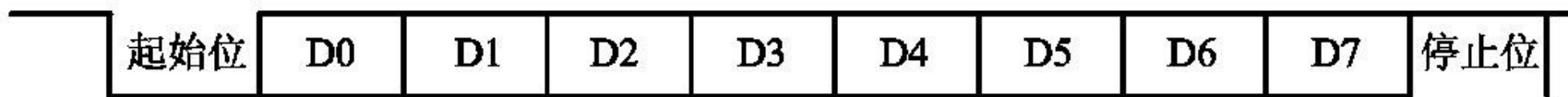


图8-15 方式1的帧格式

方式1为波特率可变的8位异步通信接口。波特率由下式确定：

$$\text{方式1波特率} = \frac{2^{\text{SMOD}}}{32} \times \text{定时器T1的溢出率}$$

式中，SMOD为PCON寄存器的最高位的值（0或1）。

1. 方式1发送

串行口以方式1输出时，数据位由TXD端输出，发送一帧信息为10位，1位起始位0，8位数据位（先低位）和1位停止位1，当CPU执行写数据到发送缓冲器SBUF的命令后，就启动发送。方式1发送时序如图8-16所示。

图8-16中发送时钟为TX，TX时钟的频率就是发送的波特率。发送开始时，内部逻辑将起始位向TXD引脚（P3.1）输出，此后每经过1个TX时钟周期，便产生1个移位脉冲，并由TXD引脚输出1个数据位。8位数据位全部发送完毕后，中断标志位TI置“1”。

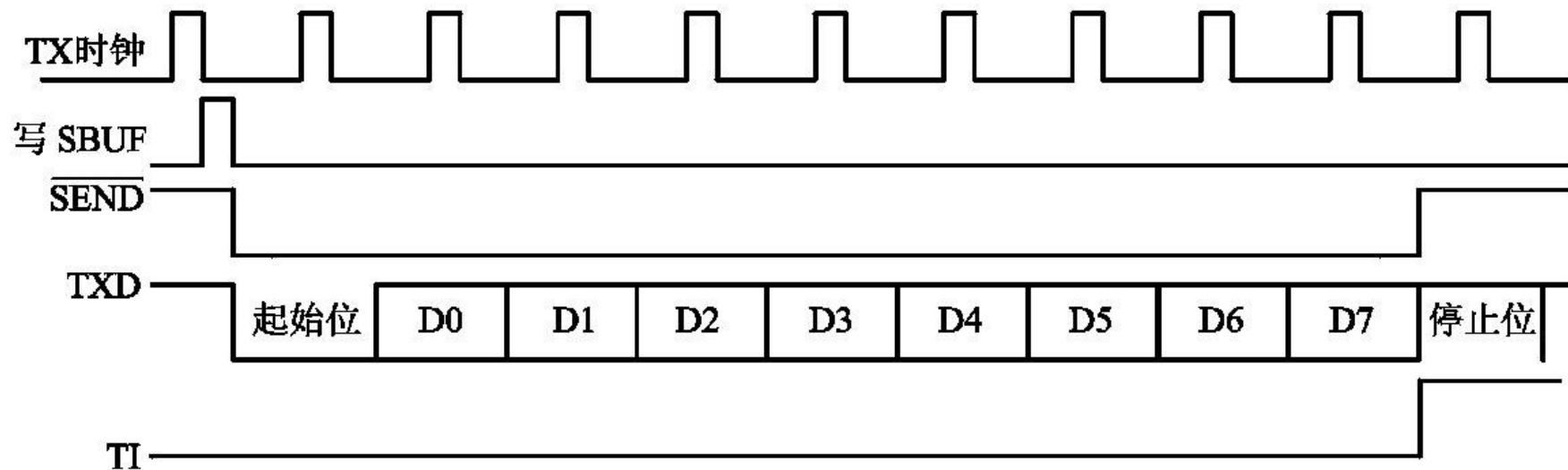


图8-16 方式1发送时序

2. 方式1接收

方式1接收时（REN = 1），数据从RXD（P3.0）引脚输入。当检测到起始位的负跳变，则开始接收。

接收时序见图8-17。

接收时，定时控制信号有两种，一种是接收移位时钟（RX时钟），它的频率和传送的波特率相同，另一种是位检测器采样脉冲，频率是RX时钟的16倍。以波特率的16倍速率采样RXD脚状态。当采样到RXD端从1到0的负跳变时就启动检测器，接收的值是3次连续采样（第7、8、9个脉冲时采样）取两次相同的值，以确认起始位（负跳变）的开始，较好地消除干扰引起的影响，以保证可靠无误地开始接收数据。

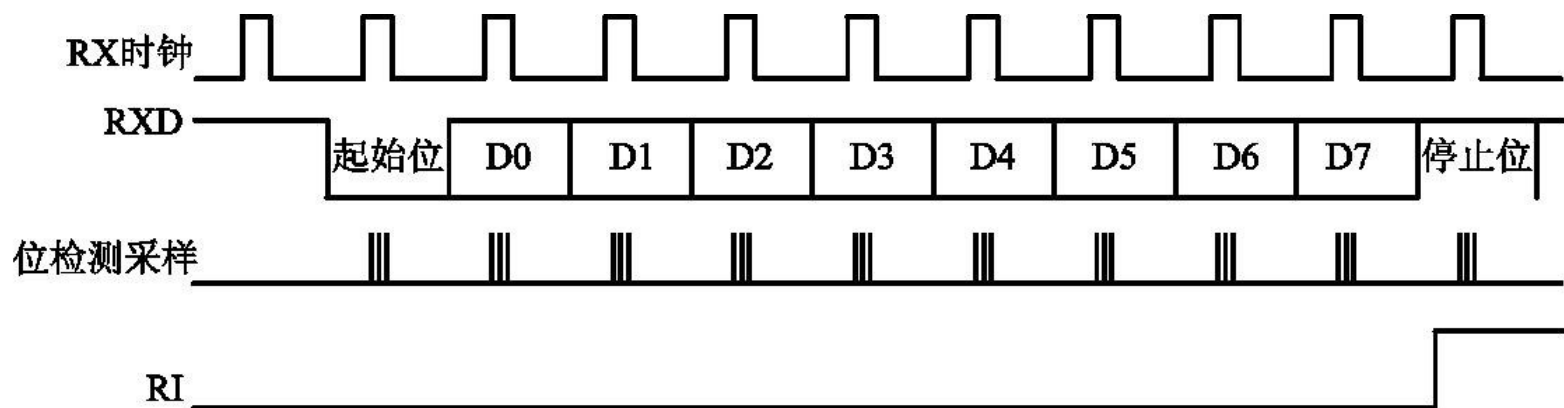


图8-17 方式1接收时序

当确认**起始位有效**时，**开始接收**一帧信息。每一位数据，也都进行**3次连续采样**（第7、8、9个脉冲采样），接收的值是**3次采样中至少两次相同的值**。当一帧数据**接收完毕后**，**同时满足以下两个条件**，接收才有效。

(1) **RI=0**，即上一帧数据接收完成时，**RI=1**发出的中断请求已被响应，SBUF中的数据已被取走，说明“接收SBUF”已空。

(2) **SM2=0**或收到的**停止位=1**（方式1时，停止位已进入RB8），则将接收到的数据装入SBUF和RB8（装入的是停止位），且中断标志RI置“1”。

若**不同时满足两个条件**，收的数据不能装入SBUF，该帧数据将丢弃。

8.3.3 方式2

方式2和方式3，为9位异步通信接口。每帧数据为11位，1位起始位0，8位数据位（先低位），1位可编程为1或0的第9位数据和1位停止位。方式2、方式3帧格式如图8-18所示。

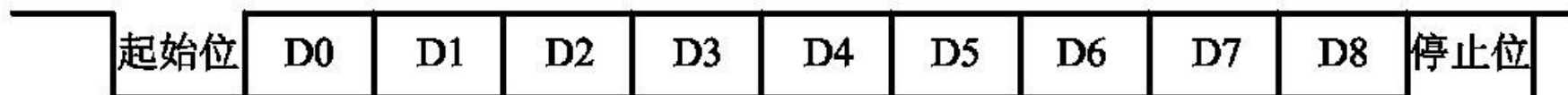


图8-18 方式2、方式3的帧格式

$$\text{方式2波特率} = \frac{2^{\text{SMOD}}}{64} \times f_{\text{osc}}$$

1. 方式2发送

发送前，先根据通信协议由软件设置**TB8**（如奇偶校验位或多机通信的地址/数据标志位），然后将要发送的数据写入SBUF，即启动发送。**TB8**自动装入第9位数据位，逐一发送。发送完毕，使TI位置“1”。

发送时序如图8-19所示。

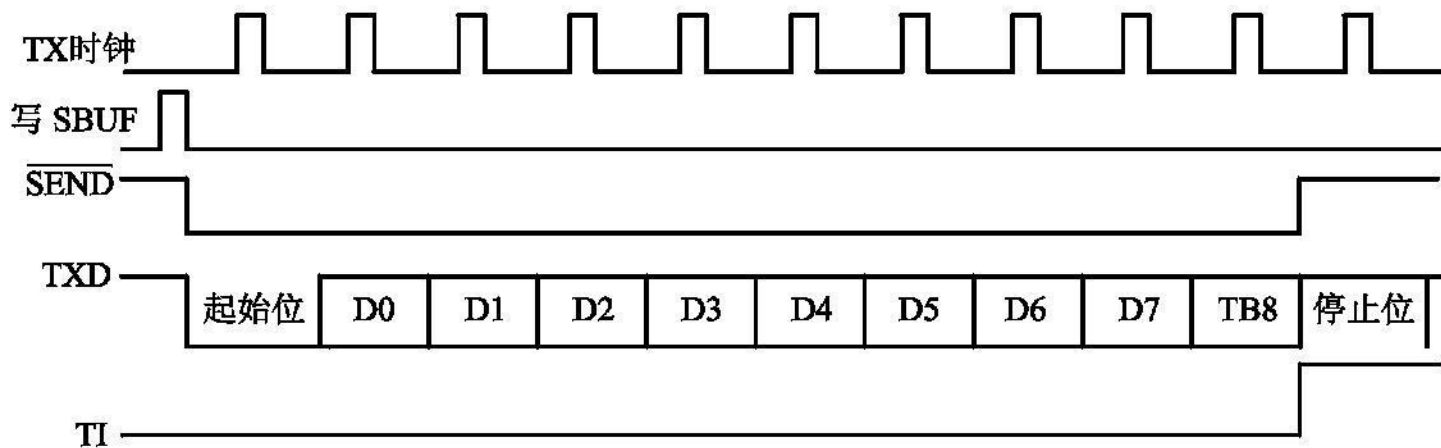


图8-19 方式2和方式3发送时序

2. 方式2接收

SM0、SM1=10，且REN=1时，以方式2接收数据。数据由RXD端输入，接收11位信息。当位检测逻辑采样到RXD的负跳变，判断起始位有效，便开始接收一帧信息。在接收完第9位数据后，需满足以下两个条件，才能将接收到的数据送入SBUF（接收缓冲器）。

(1) RI=0，意味着接收缓冲器为空。

(2) SM2=0 或 RB8=1。

当满足上述两个条件时，收到的数据送SBUF（接收缓冲器），第9位数据送入RB8，且RI置“1”。若不满足这两个条件，接收的信息将被丢弃。

串行口方式2和方式3接收时序如图8-20所示。

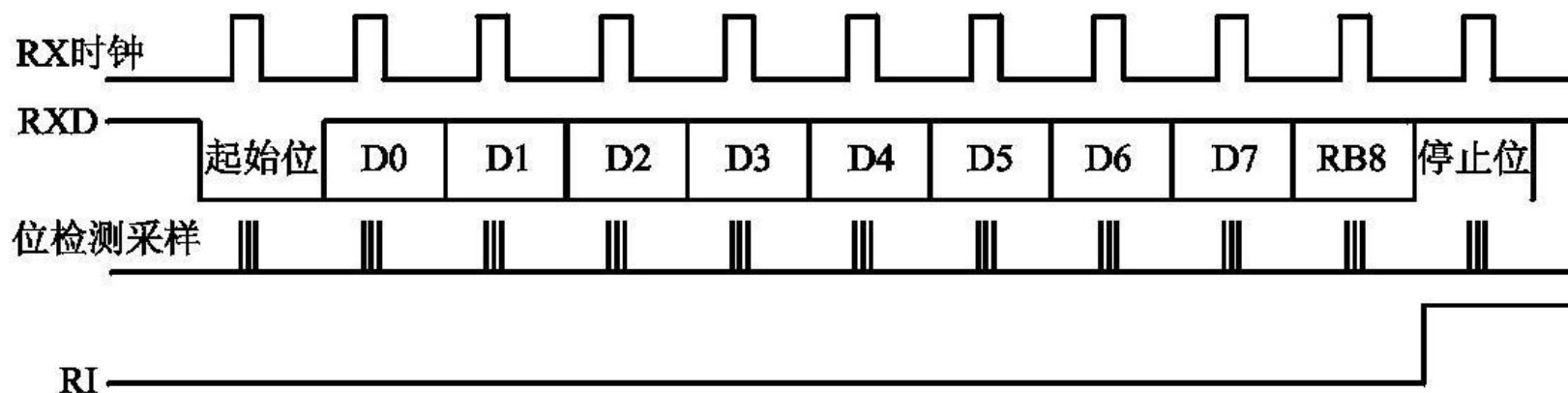


图8-20 方式2和方式3接收时序

8.3.4 方式3

当SM0、SM1两位为11时，串行口被定义为方式3。方式3为波特率可变的9位异步通信方式。方式3发送和接收时序如图8-19和图8-20所示。

方式3的波特率由下式确定：

$$\text{方式3波特率} = \frac{2^{\text{SMOD}}}{32} \times \text{定时器T1的溢出率}$$

8.4 多机通信

多个单片机可利用串行口进行多机通信，常采用图8-21的主从式结构。系统有1个主机（单片机或其他有串行接口的微机）和多个单片机组成的从机系统。主机RXD与所有从机的TXD端相连，TXD与所有从机的RXD端相连。从机地址分别为01H、02H和03H。

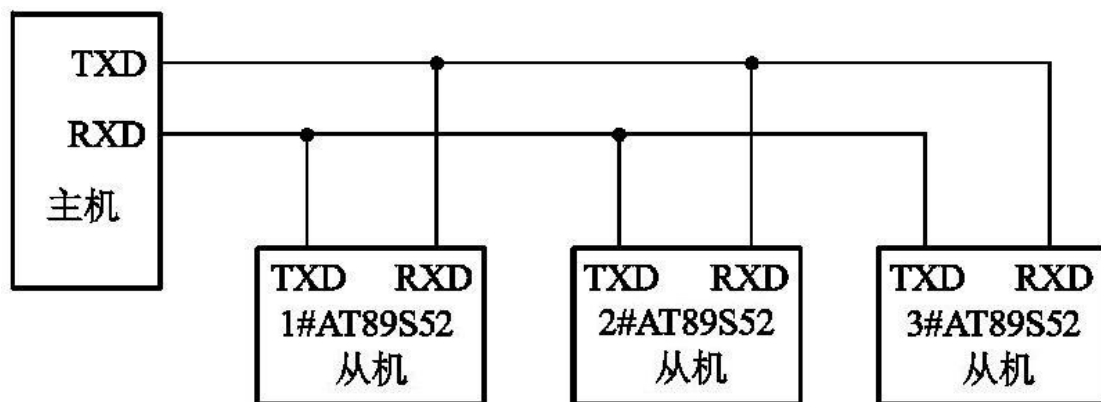


图8-21 多机通信系统示意图

主从式是指多机系统中，只有一个主机，其余全是从机。**主机**发送的信息可以被**所有从机接收**，任何一个从机发送的信息，只能由**主机**接收。从机和从机之间不能进行直接通信，**只能经主机**才能实现。

多机通信的工作原理：

要保证**主机与所选择的从机**通信，须保证**串口有识别功能**。SCON中的**SM2位**就是为满足这一条件设置的**多机通信控制位**。其工作原理是在串行口以方式2（或方式3）接收时，**若SM2=1**，则表示进行多机通信，可能以下**两种情况**：

(1) 从机接收到的主机发来的第9位数据RB8=1时，前8位数据才装入SBUF，并置中断标志RI = 1，向CPU发出中断请求。

在中断服务程序中，从机把接收到的SBUF中的数据存入数据缓冲区中。

(2) 如果从机接收到的第9位数据RB8=0时，则不产生中断标志RI=1，不引起中断，从机不接收主机发来的数据。

若SM2 = 0，则接收的第9位数据不论是0还是1，从机都将产生RI = 1中断标志，接收到的数据装入SBUF中。

应用这一特性，可实现AT89S52单片机的多机通信。

多机通信的工作过程：

(1) 各从机初始化程序允许从机的串行口中断，将串行口编程为方式2或方式3接收，即9位异步通信方式，且SM2和REN位置“1”，使从机处于多机通信且只接收地址帧的状态。

(2) 在主机和某个从机通信之前，先将从机地址（即准备接收数据的从机）发送给各个从机，接着才传送数据（或命令），主机发出的地址帧信息的第9位为1，数据（或命令）帧的第9位为0。

当主机向各从机发送地址帧时，各从机的串行口接收到的第9位信息RB8为1，且由于各从机的SM2=1，则RI置“1”，各从机响应中断，在中

断服务子程序中，判断主机送来的地址是否和本机地址相符合，若为本机地址，则该从机SM2位清“0”，准备接收主机的数据或命令；若地址不相符，则保持SM2=1。

(3) 接着主机发送数据（或命令）帧，数据帧的第9位为0。此时各从机接收到的RB8=0。

只有与前面地址相符合的从机（即SM2位已清“0”的从机）才能激活中断标志位RI，从而进入中断服务程序，接收主机发来的数据（或命令）；与主机发来的地址不相符的从机，由于SM2保持为1，又RB8=0，因此不能激活中断标志RI，就不能接受主机发来的数据帧。从而保证主机与从机间通信的

正确性。此时主机与建立联系的从机已经设置为单机通信模式，即在整个通信中，通信的双方都要保持发送数据的第9位（即TB8位）为0，防止其他的从机误接收数据。

（4）结束数据通信并为下一次的多机通信做好准备。在多机系统，**每个从机都被赋予唯一的地址。**例如，**图8-21**三个从机的地址可设为：01H、02H、03H。还要**预留1~2个“广播地址”**，它是所有从机共有的地址，例如将“广播地址”设为00H。

当主机与从机的数据通信结束后，一定要将从机再设置为多机通信模式，以便进行下一次的多机通信。

这时要求与主机正在进行数据传输的从机必须随时注意，一旦接收的数据第9位（RB8）为“1”，说明主机传送的不再是数据，而是地址，这个地址就有可能是“广播地址”。

当收到“广播地址”后，便将从机的通信模式再设置成多机模式，为下一次的多机通信做好准备。

8.5 波特率的制定方法

串行通信，收、发双方发送或接收的波特率必须一致。4种工作方式。

方式0和方式2的波特率是固定的；

方式1和方式3的波特率是可变的，由T1溢出率确定。

8.5.1 波特率的定义

波特率的定义：串行口每秒发送（或接收）的位数。设发送一位所需要的时间为 T ，则波特率为 $1/T$ 。

例如，串行口发送一位所需要的时间为 $1\mu\text{s}$ ，则波特率为 1Mbit/s 。

定时器的不同工作方式，得到的波特率的范围不一样，这是由T1在不同工作模式下计数位数的不同所决定。

8.5.2 定时器T1产生波特率的计算 与串行口的工作方式有关。

(1) 方式0时，波特率固定为时钟频率 f_{osc} 的1/12，不受SMOD位值的影响。
若 $f_{osc} = 12 \text{ MHz}$ ，波特率为1Mbit/s。

(2) 方式2时，波特率仅与SMOD位的值有关。

$$\text{方式 2 波特率} = \frac{2^{\text{SMOD}}}{64} \times f_{osc}$$

若 $f_{osc} = 12 \text{ MHz}$ ： SMOD=0，波特率 = 187.5 kbit/s； SMOD=1，波特率为375 kbit/s。

(3) 方式1或方式3定时，常用T1作为波特率发生器，其关系式为

$$\text{波特率} = \frac{2^{\text{SMOD}}}{32} \times \text{定时器 T1 的溢出率} \quad (8-1)$$

由式 (8-1) 见, T1溢出率和SMOD的值共同决定波特率。

在实际设定波特率时, T1常设置为方式2定时 (自动装初值), 即TL1作为8位计数器, TH1存放备用初值。这种方式操作方便, 也避免因软件重装初值带来的定时误差。

设定定时器T1方式2的初值为X, 则有

$$\text{定时器T1的溢出率} = \text{计数速率} / (256 - x) = (f_{\text{osc}} / 12) / (256 - x) \quad (8-2)$$

将式（8-2）代入式（8-1），则有

$$\text{波特率} = (2^{\text{SMOD}}/32) \times (f_{\text{osc}}/12) / (256-x) \quad (8-3)$$

由式（8-3）可见，波特率随 f_{osc} 、SMOD和初值 x 而变化。

实际使用时，经常根据已知波特率和时钟频率 f_{osc} 来计算T1的初值 x 。为避免繁杂的初值计算，常用的波特率和初值 x 间的关系常列成表8-2的形式，以供查用。

表8-2 用定时器T1产生的常用波特率

波特率	f_{osc}	SMOD位	方式	初值X
62.5kbit/s	12 MHz	1	2	FFH
19.2kbit/s	11.0592 MHz	1	2	FDH
9.6kbit/s	11.0592 MHz	0	2	FDH
4.8kbit/s	11.0592 MHz	0	2	FAH
2.4kbit/s	11.0592 MHz	0	2	F4H
1.2kbit/s	11.0592 MHz	0	2	E8H

对表8-2有两点需要注意：

(1) 在使用的时钟振荡频率 f_{osc} 为12MHz或6MHz时，将初值X和 f_{osc} 带入式(6-3)中计算出的波特率有一定误差。消除误差可采用时钟频率11.0592MHz。

(2) 如果选用很低的波特率，如波特率选为55，可将定时器T1设置为方式1定时。但在这种情况下，T1溢出时，需在中断服务程序中重新装入初值。中断响应时间和执行指令时间会使波特率产生一定的误差，可用改变初值的方法加以调整。

【例8-3】 若时钟频率为**11.0592MHz**，选用**T1的方式2**定时作为波特率发生器，**波特率为2 400bit/s**，求初值。

设T1为**方式2**定时，选**SMOD=0**。

将已知条件带入**式（8-3）**中

$$\text{波特率} = \frac{2^{\text{SMOD}}}{32} \times \frac{f_{\text{osc}}}{12(256 - X)} = 2400$$

从中解得 **X = 244 = F4H**。

只要把**F4H**装入**TH1**和**TL1**，则T1产生的波特率为**2 400bit/s**。该结果也可直接从**表8-2**中查到。

这里时钟振荡频率选为**11.0592MHz**，就可使**初值为整数**，从而产生精确的波特率。

8.6 串行口应用设计案例

单片机串行通信接口设计时，需考虑如下问题。

- (1) 确定串行通信双方的数传速率和通信距离。
- (2) 由串行通信的数传速率和通信距离确定采用的串行通信接口标准。
- (3) 注意串行通信的通信线选择，一般选用双绞线较好，并根据传输的距离选择纤芯的直径。如空间干扰较多，还要选择带有屏蔽层的双绞线。

下面首先介绍有关串行通信中为提高数传速率、通信距离以及抗干扰性能经常采用的各种串行通信接口标准。

8.6.1 串行通信标准接口RS232、RS422与RS485简介

AT89S52串口输入、输出均为TTL电平。这种以TTL电平来串行传输数据，抗干扰性差，传输距离短，传输速率低。为提高串行通信可靠性，增大串行通信距离和提高传输速率，在实际设计中都采用标准串行接口，如RS-232、RS-422A、RS-485等。

根据双机通信距离和抗干扰性要求，可选择TTL电平传输，或选择RS-232C、RS-422A、RS-485串行接口进行串行数据传输。

1. TTL电平通信接口

如两个AT89S52相距在1.5m之内，串行口可直接相连，接口电路如图8-14。甲机RXD与乙机TXD端相连，乙机RXD与甲机TXD端相连，从而直接用TTL电平传输方法来实现双机通信。

2. RS-232C双机通信接口

如双机通信距离在1.5~15m时，可用RS-232C标准接口实现点对点的双机通信，接口电路见图8-22。

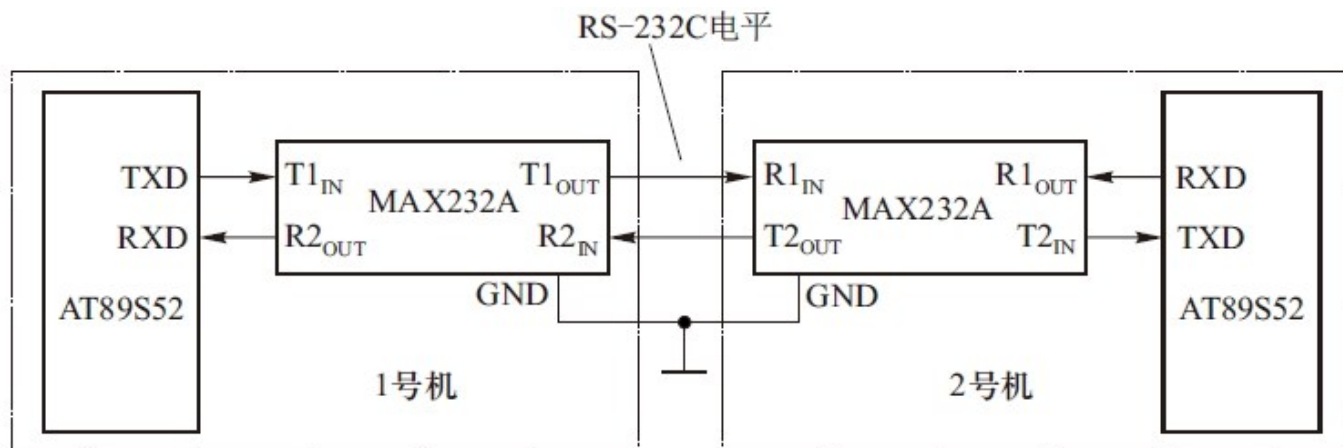


图8-22 RS-232C双机通信接口电路

图8-22芯片MAX232A是美国MAXIM公司生产的RS-232C全双工发送器/接收器电路芯片。

RS-232C标准规定**电缆长度限定在 ≤ 15 米**，最高数传速率为**20kbit/s**。足以覆盖个人计算机使用的**50~9600 bit/s**范围。传送的数字量采用负逻辑，且与地对称。其中：

逻辑“1”： $-3 \sim -15\text{V}$ ；

逻辑“0”： $+3 \sim +15\text{V}$ 。

由于单片机的引脚为**TTL**电平，与**RS-232C**标准的电平互不兼容，所以单片机使用**RS-232C**标准串行通信时，必须进行**TTL**电平与**RS-232C**标准电平之间的转换。

RS-232C电平与**TTL**电平的之间转换，常采用美国**MAXIM**公司的**MAX232A**，它是全双工发送器/接收器接口电路芯片，可实现**TTL**电平到**RS-232C**电平、**RS-232C**电平到**TTL**电平的转换。**MAX232A**的引脚见

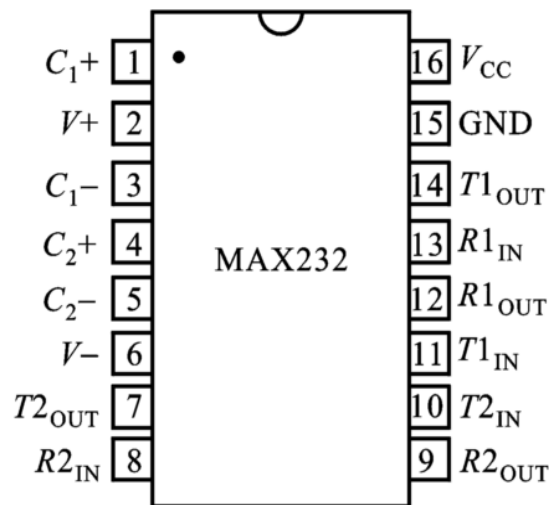


图8-23 MAX232A的引脚

图8-23，内部结构及外部元件如**图8-24**所示。由于芯片内部有自升压的电平倍增电路，将+5V转换成-10V~+10V，满足RS-232C标准对逻辑“1”和逻辑“0”的电平要求。工作时仅需单一的+5V电源。其片内有2个发送器，2个接收器，有TTL信号输入/RS-232C输出的功能，也有RS-232C输入/TTL输出的功能。

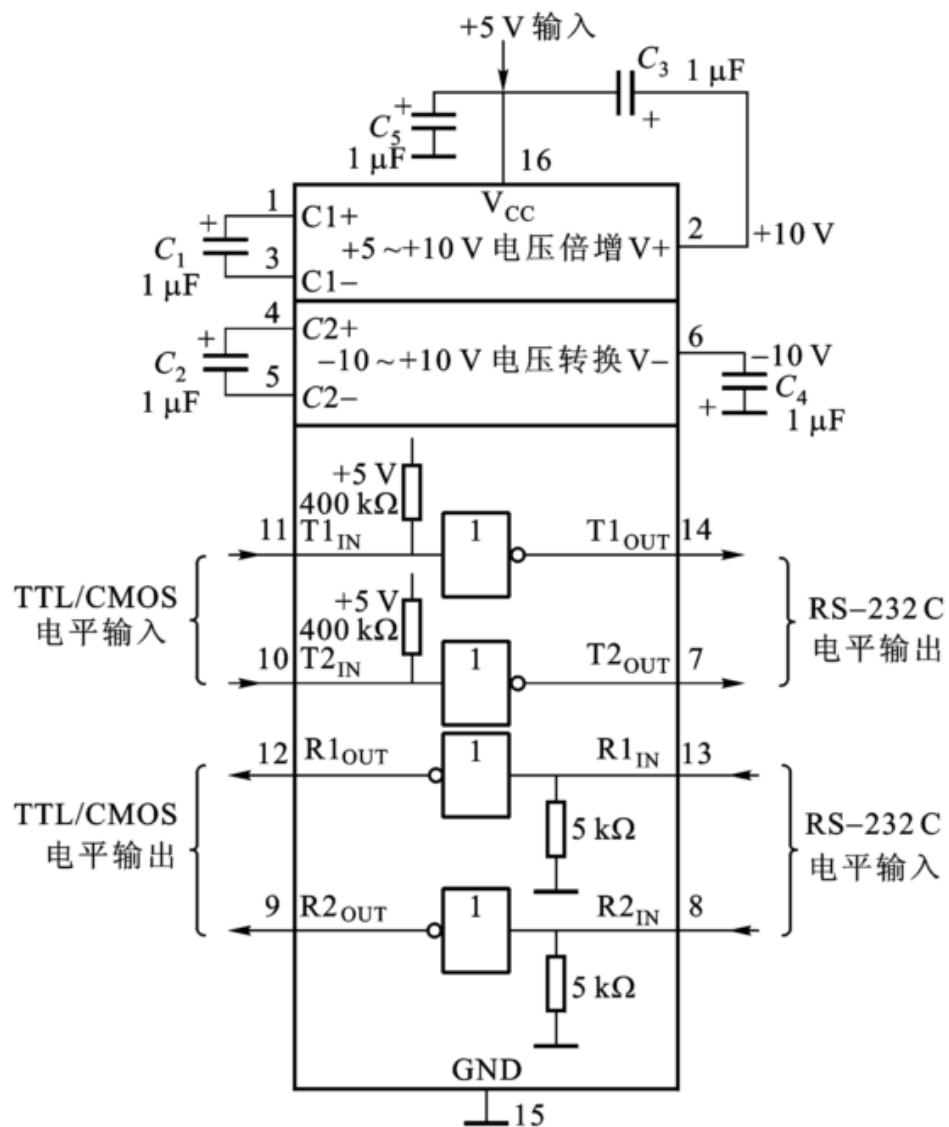


图8-24 MAX232的内部结构及外部元件

3. RS-422A双机通信接口

RS-232C虽应用广泛，但推出较早，有明显缺点：传输速率低、通信距离短、接口处信号易产生串扰等。于是国际上又推出了RS-422A标准。

RS-422A与RS-232C主要区别是，收发双方信号地不再共地，RS-422A采用了平衡驱动和差分接收的方法。每个方向用于数据传输的是两条平衡导线，这相当于两个单端驱动器。输入同一个信号时，其中一个驱动器输出永远是另一个驱动器反相信号。于是两条线上传输的信号电平，当一个表示逻辑“1”时，另一条一定为逻辑“0”。



若传输过程中，信号中混入了干扰和噪声（以共模形式出现），由于差分接收器的作用，就能识别有用信号并正确接收传输信息，使干扰和噪声相互抵消。

因此，RS-422A能在长距离、高速率下传输数据。最大传输率为10Mbit/s，此速率下，电缆允许长度为12m，如采用较低速率，最大传输距离可达1219m。

为增加通信距离，可在通信线路上采用光电隔离，利用RS-422A标准进行双机通信的接口电路见图8-25。

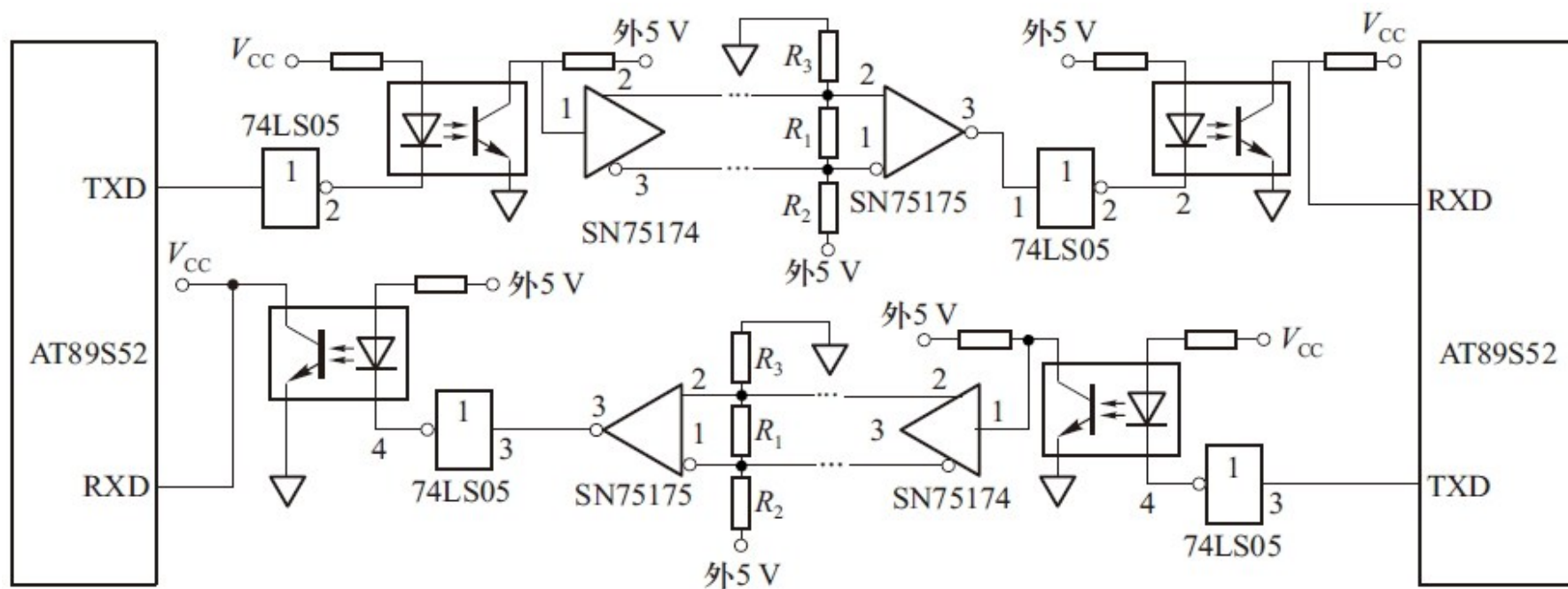


图8-25 RS-422A双机通信接口电路

图8-25中，每通道接收端都接有3个电阻R1、R2和R3，其中R1为传输线的匹配电阻，取值范围在50~1k，其他两个电阻是为了解决第1个数据误码而设置的匹配电阻。为起到隔离、抗干扰作用，图8-25中必须使用两组独立的电源。

图8-25所示的SN75174、SN75175是TTL电平到RS-422A电平与RS-422A电平到TTL电平的电平转换芯片。

4. RS-485双机通信接口

RS-422A双机通信需四芯传输线，长距离通信不经济。在工业现场，常采用双绞线传输的RS-485串行通信接口，很易实现多机通信。RS-485是RS-422A变型，与RS-422A区别是：RS-422A为全双工，采用两对平衡差分信号线；

而RS-485为**半双工**，采用一对平衡差分信号线。RS-485与**多站互连**是十分方便的，很易实现**1对N**的多机通信。

RS-485标准允许最多并联32台驱动器和32台接收器。**图8-26**为RS-485双机通信接口。RS-485与RS-422A一样，**最大传输距离约1219m**，**最大传输速率**为**10Mbit/s**。**通信线路**要采用**平衡双绞线**。平衡双绞线长度与传输速率成反比，在100kbit/s速率以下，才可能使用规定的最长电缆。只有在很短距离下才能获得最大传输速率。一般**100m长双绞线**最大传输速率仅为**1Mbit/s**。

图8-26中，RS-485以双向、半双工方式实现双机通信。在AT89S51系统发送或接收数据前，应先将**SN75176**的发送门或接收门打开，当**P1.0=1**时，发送

门打开，接收门关闭；当P1.0=0时，接收门打开，发送门关闭。

图8-26的SN75176片内集成一个差分驱动器和一个差分接收器，且兼有TTL电平到RS-485电平、RS-485电平到TTL电平的转换功能。此外常用的RS-485接口芯片还有MAX485。

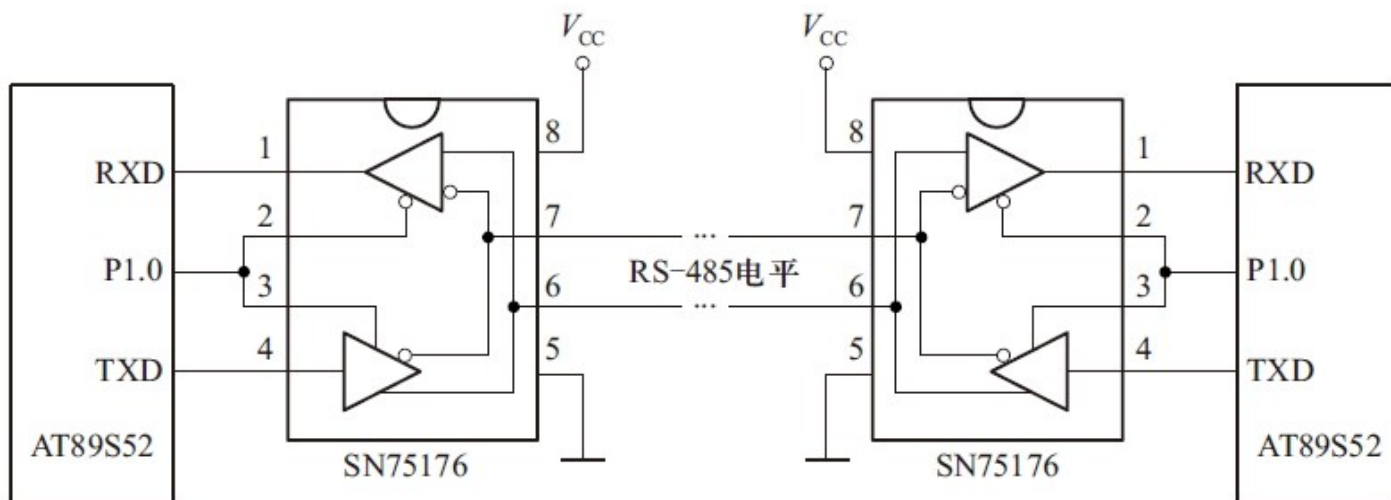


图8-26 RS-485双机通信接口电路

8.6.2 方式1的应用设计

【例8-4】图8-27为单片机甲、乙双机串行通信，双机RXD和TXD相互交叉相连，甲机P1口接8个开关，乙机P1口接8个发光二极管。甲机设置为只能发送不能接收的单工方式。要求甲机读入P1口的8个开关的状态后，通过串行口发送到乙机，乙机将接收到的甲机的8个开关的状态数据送入P1口，由P1口的8个发光二极管来显示8个开关的状态。双方晶振均采用11.0592MHz。

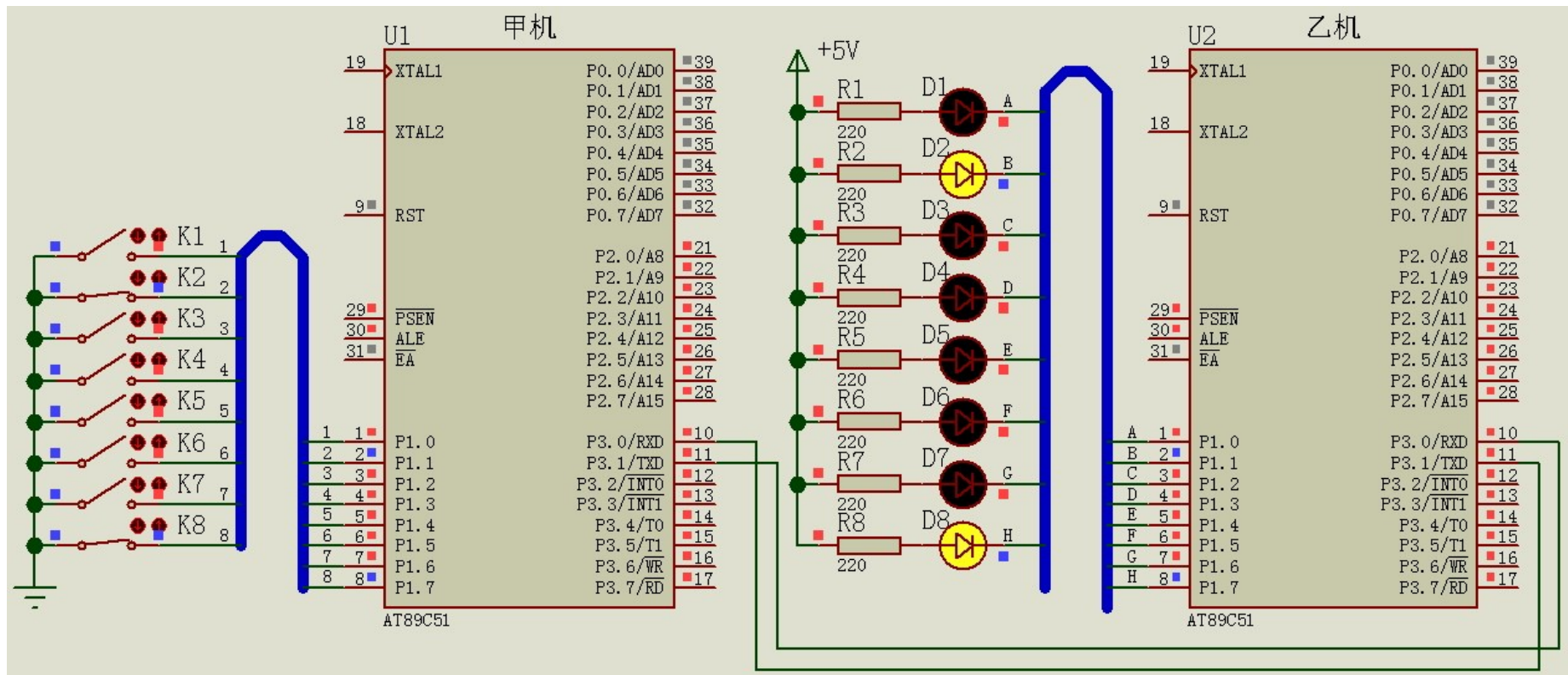


图8-27 单片机方式1双机通信的连接

参考程序如下：

//甲机串行发送

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
void main()
```

```
{    uchar temp=0;
```

```
    TMOD=0x20;
```

//设置定时器T1为方式2

```
    TH1=0xfd;
```

//波特率9600

```
    TL1=0xfd;
```

```
    SCON=0x40;
```

//串口初始化方式1发送，不接收

```
    PCON=0x00;
```

// SMOD=0

```
    TR1=1;
```

//启动T1

```
    P1=0xff;
```

//设置P1口为输入

```
    while(1)
```

```
    {
```

```
        temp=P1;
```

//读入P1口开关的状态

```
        SBUF=temp;
```

//状态数据送串行口发送

```
while(TI==0);  
TI=0;
```

```
//如果TI=0, 未发送完, 循环等待  
//已发送完, 把TI清0
```

```
}
```

//乙机串行接收

```
#include <reg51.h>  
#define uchar unsigned char  
#define uint unsigned int  
void main( )  
{
```

```
    uchar temp=0;
```

```
    TMOD=0x20;
```

```
    TH1=0xfd;
```

```
    TL1=0xfd;
```

```
    SCON=0x50;
```

```
    PCON=0x00;
```

```
    TR1=1;
```

```
    while(1)
```

```
{
```

```
    while(RI==0);
```

```
    RI=0;
```

```
    temp=SBUF;
```

```
//设置定时器T1为方式2
```

```
//波特率9600
```

```
//设置串口为方式1接收, REN=1
```

```
//SMOD=0
```

```
//启动T1
```

```
// 若RI为0, 未接收到数据
```

```
// 接收到数据, 则把RI清0
```

```
// 读取数据存入temp中
```

```
P1=temp;
```

```
//接收的数据送P1口控制8个LED的亮灭
```

```
}
```

```
}
```

【例8-5】如图8-28，甲乙两机以方式1进行串行通信，双方晶振频率均为11.059 2MHz，波特率为2400bit/s。甲机TXD脚、RXD脚分别与乙机RXD、TXD脚相连。为观察串行口传输的数据，电路中添加了两个虚拟终端来分别显示串口发出的数据。添加虚拟终端，只需单击图4-21左侧工具箱中的虚拟仪器图标，在预览窗口中显示的各种虚拟仪器选项，点击“VIRTUAL TERMINAL”项，并放置在原理图编辑窗口，然后把虚拟终端的“RXD”端与单片机的“TXD”端相连即可。

当串行通信开始时，甲机首先发送数据AAH，乙机收到后应答BBH，表示同意接收。甲机收到BBH后，即可发送数据。如果乙机发现数据出错，就向甲机发送FFH，甲机收到FFH后，重新发送数据给乙机。

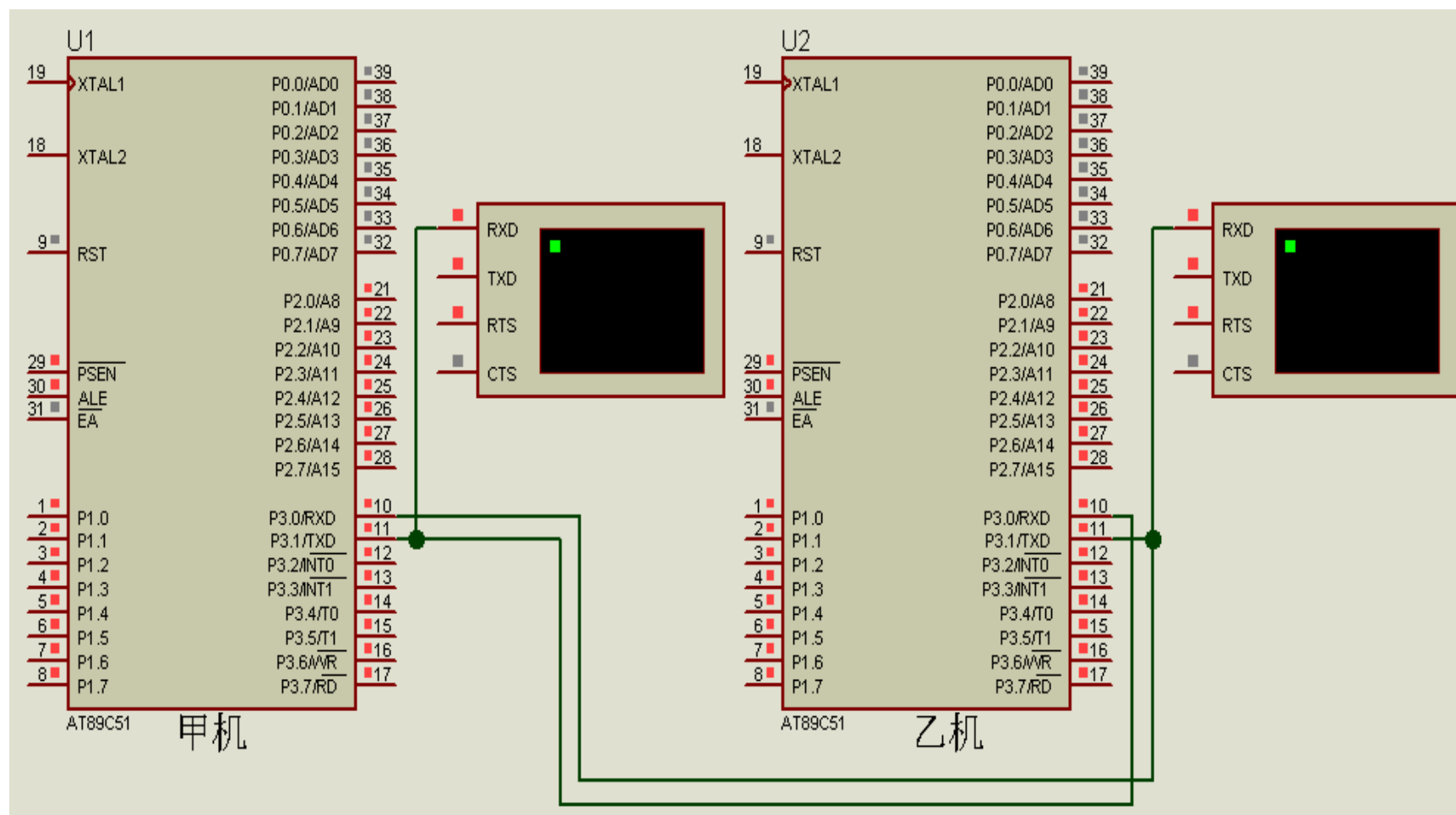


图8-28 方式1双机通信的连接

串行通信时，如观察单片机仿真运行时串行口发送出的数据，只需用鼠标右键点击虚拟终端，会出现选择菜单，点击最下方“Virtual Terminal”项，会弹出窗口，窗口显示了单片机串口“TXD”端发出的一个个数据字节，如图8-29所示。

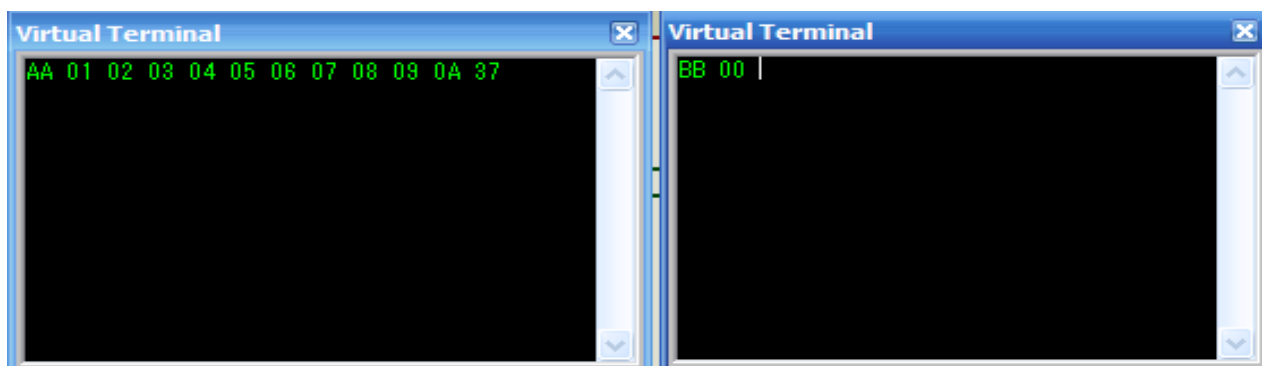


图8-29 通过串口观察两个单片机串行口发出的数据

设发送字节块长度为10字节，数据缓冲区为buf，数据发送完毕要立即发送校验和，进行数据发送准确性验证。乙机接收到的数据存储在数据缓冲区buf，收到一个数据块后，再接收甲机发来的校验和，并将其与乙机求得的校验和比较：若相等，说明接收正确，乙机回答00H；若不等，说明接收不正确，乙机回答FFH；请求甲机重新发送。

选择定时器**T1**为**方式2**定时，波特率不倍增，即SMOD=0。查表8-2，可得写入T1的初值应为F4H。

以下为双机通信程序，该程序可在甲乙两机中运行，不同的是在程序运行之前，要人为设置TR。若选择TR=0，表示该机为发送方；若TR=1，表示该机是接收方。程序根据TR设置，利用发送函数send()和接收函数receive()分别实现发送和接收功能。

参考程序如下：

//甲机串口通信程序

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
#define TR // 接收、发送的区别值，TR=0，为发送
```

```
uchar buf[10] = {0x01, 0x02, 0x03, 0x04, 0x05,  
                 0x06, 0x07, 0x08, 0x09, 0x0a}; //发送的10个数据
```

```
uchar sum;
```

```
void main(void)
```

//甲机主程序

```
{
```

```
    init ( );
```

//调用甲机串口初始化

```
    if (TR==0)
```

// TR=0，为发送

```
    {send( );}
```

//调用发送函数

```
    if (TR==1)
```

// TR=1，为接收

```
    {receive( );}
```

//调用接收函数

```
}
```

```
void delay(unsigned int i)
```

```
//延时函数
```

```
{  
    unsigned char j;  
    for(;i>0;i--)  
        for(j=0;j<125;j++)  
            ;  
}
```

```
void init(void)
```

```
//甲机串口初始化函数
```

```
{  
    TMOD=0x20;           //T1方式2定时  
    TH1=0xf4;            //波特率2400  
    TL1=0xf4;  
    PCON=0x00;           //SMOD=0  
    SCON=0x50;           //串行口方式1, REN=1允许接收  
    TR1=1;               //启动T1  
}
```

```
void send(void )
```

```
{
```

```
    uchar i
```

```
    do{
```

```
        delay(1000);
```

```
        SBUF=0xaa;
```

```
        while(TI==0);
```

```
        TI=0;
```

```
        while(RI==0);
```

```
        RI=0;
```

```
    } while(SBUF!=0xbb);
```

```
//甲机发送函数
```

```
//发送联络信号
```

```
//等待数据发送完毕
```

```
//发送完毕，清TI
```

```
//等待乙机应答
```

```
//乙机应答完毕，甲机RI清0
```

```
//乙机未准备好，继续联络
```

```
do {  
    sum=0; //校验和变量清0  
    for(i=0; i<10; i++)  
    {  
        delay(1000);  
        SBUF= buf[i]; //向乙机发数据  
        sum+= buf[i]; //求校验和  
        while(TI==0);  
        TI=0; //甲机发送数据完毕，清TI  
    }  
  
    delay(1000);  
    SBUF=sum; //发送校验和  
    while(TI==0); TI=0;  
    while(RI==0); RI=0;  
} while(SBUF!=0x00); //出错，重新发送  
while(1);  
}
```

```
void receive(void )
```

```
{
```

```
    uchar i;
```

```
    RI=0;
```

```
    while (RI==0); RI=0;
```

```
    while (SBUF!=0xaa);
```

```
    SBUF=0xBB;
```

```
    while (TI==0);
```

```
    TI=0;
```

```
    sum=0;
```

```
    for(i=0; i<10; i++)
```

```
    {
```

```
        while (RI==0);    RI=0;
```

```
        buf[i]= SBUF;
```

```
        sum+=buf[i];
```

```
    }
```

```
//甲机接收函数
```

```
//判乙机是否发出请求
```

```
//发送应答信号BBH
```

```
//等待发送结束
```

```
//清校验和
```

```
//接收数据
```

```
//接收一个数据
```

```
//求校验和
```

```
while (RI==0) ;  
RI=0;  
if (SBUF==sum)  
{  
    SBUF=0x00;  
}  
else  
{  
    SBUF=0xFF;  
    while (TI==0) ;TI=0;  
}  
}
```

//接收乙机的校验和
//比较校验和

//校验和相等，则发00H

//出错发FFH，重新接收

```
#include <reg51.h>                                //乙机串行通信程序
#define uchar unsigned char
#define TR 1                                       //接收、发送的区别值，TR=1，为接收
uchar idata buf[10] = {0x01, 0x02, 0x03, 0x04, 0x05,
                      0x06, 0x07, 0x08, 0x09, 0x0a};
uchar sum;                                       // 校验和

void delay(unsigned int i)
{
    unsigned char j;
    for(;i>0;i--)
        for(j=0;j<125;j++)
            ;
}
```


void init(void) //乙机串口初始化函数

```
{  
    TMOD=0x20;           //T1方式2定时  
    TH1=0xf4;            //波特率2400  
    TL1=0xf4;  
    PCON=0x00;           //SMOD=0  
    SCON=0x50;           //串行口方式1, REN=1允许接收  
    TR1=1;               //启动T1  
}
```

void main(void) //乙机主程序

```
{  
    init ( );  
    if(TR==0)           // TR=0, 为发送  
        {send( );}      //调用发送函数  
    else  
        {receive( );}   //调用接收函数  
}
```

```
void send(void )
```

```
//乙机发送函数
```

```
{
```

```
    uchar i;
```

```
    do{
```

```
        SBUF=0xAA;
```

```
//发送联络信号
```

```
        while(TI==0);
```

```
//等待数据发送完毕
```

```
        TI=0;
```

```
        while(RI==0);
```

```
//等待乙机应答
```

```
        RI=0;
```

```
    } while(SBUF!=0xbb);
```

```
//乙机未准备好，继续联络(按位取异或)
```

```
do{
```

```
    sum=0;
```

```
//校验和变量清0
```

```
    for(i=0; i<10; i++)
```

```
    {
```

```
        SBUF = buf[i];
```

```
        sum+= buf[i];
```

```
//求校验和
```

```
        while(TI==0);
```

```
        TI=0;
```

```
    }
```

```
    SBUF=sum;
```

```
    while(TI==0); TI=0;
```

```
    while(RI==0); RI=0;
```

```
    }while (SBUF!=0);
```

```
//出错，重新发送
```

```
}
```

```
void receive(void )
```

```
//乙机接收函数
```

```
{
```

```
    uchar i;
```

```
    RI=0;
```

```
    while(RI==0); RI=0;
```

```
    while(SBUF!=0xaa)
```

```
    {
```

```
        SBUF=0xff;
```

```
        while(TI!=1);
```

```
        TI=0;
```

```
        delay(1000);
```

```
    }
```

```
    SBUF=0xBB;
```

```
    while (TI==0);
```

```
    TI=0;
```

```
    sum=0;
```

```
//判甲机是否发出请求
```

```
//发送应答信号0xBB
```

```
//等待发送结束
```

```

for(i=0; i<10; i++)
{
    while(RI==0);RI=0;           //接收校验和
    buf[i]= SBUF;                //接收一个数据
    sum+=buf[i];                 //求校验和
}
while(RI==0);
RI=0;                           //接收甲机的校验和
if(SBUF==sum)                    //比较校验和
{
    SBUF=0x00;                  //校验和相等，则发00H
}
else
{
    SBUF=0xFF;                  //出错发FFH，重新接收
    while(TI==0);    TI=0;
}
}

```

8.6.3 方式2和方式3的应用设计

方式2与方式1相比**两点不同之处**:

(1) 方式2接收/发送11位信息，第0位为起始位，第1~8位为数据位，第9位是程控位，由用户设置的TB8位决定，第10位是停止位1，这是方式2与方式1的一个不同点。

(2) 方式2的波特率变化范围比方式1小，**方式2的波特率=振荡器频率/n**。

当**SMOD=0**时， $n=64$ 。

当**SMOD=1**时， $n=32$ 。

而**方式2和方式3相比，除了波特率的差别外**，其它都相同，所以下面介绍的方式3应用编程，也适用于方式2。

【例8-6】如图8-30，甲、乙两单片机进行 方式3（或方式2）串行通信。甲机把控制8个流水灯点亮的的数据发送给乙机并点亮其P1口的8个LED。方式3比方式1多了1个可编程位TB8，该位一般作奇偶校验位。乙机接收到的8位二进制数据有可能出错，需进行奇偶校验，其方法是将乙机的RB8和PSW的奇偶校验位P进行比较，如果相同，接收数据；否则拒绝接收。

本例使用了一个虚拟终端来观察甲机串口发出的数据。

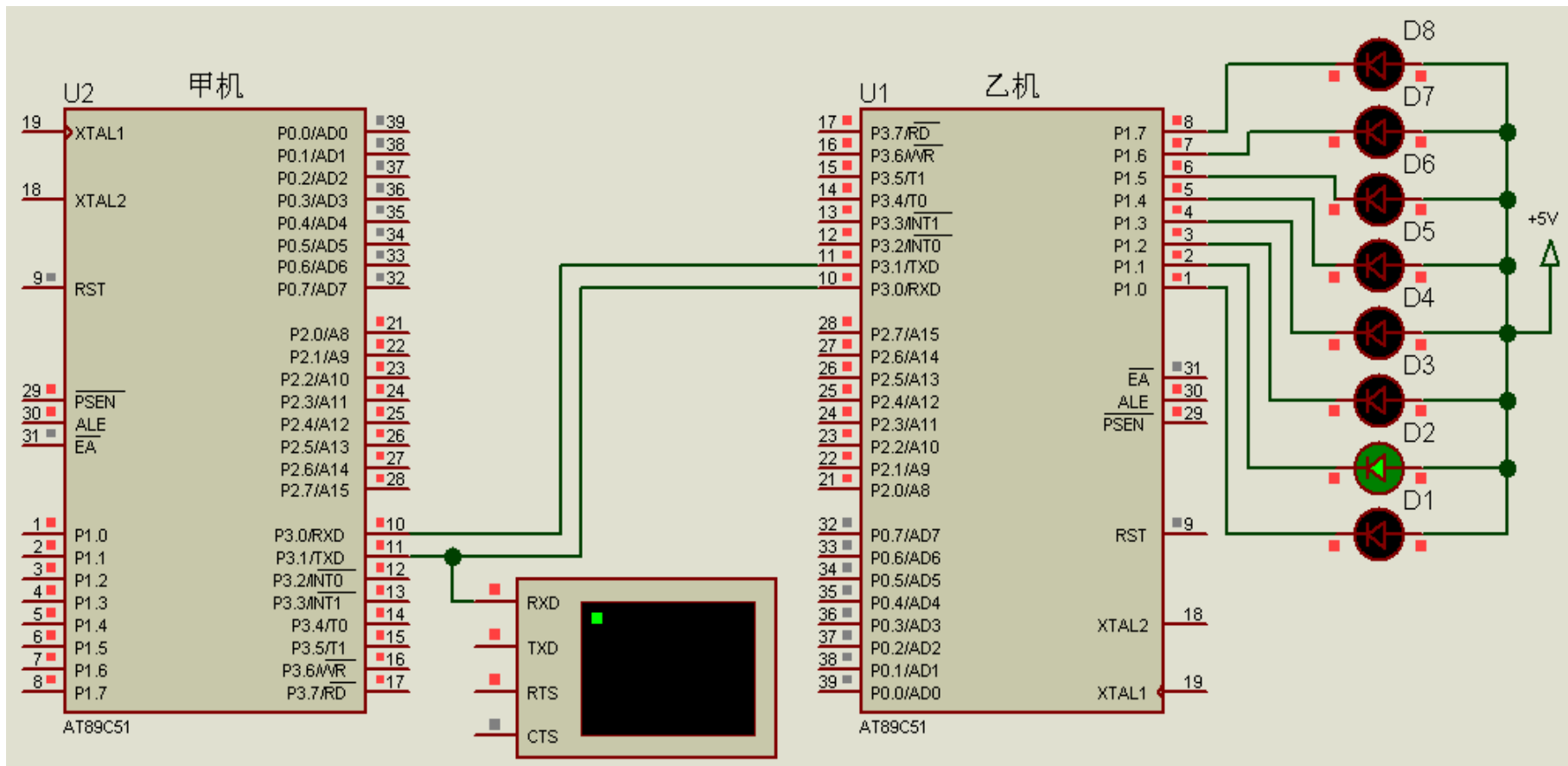


图8-30 甲乙两个单片机进行方式3（或方式2）串行通信

参考程序如下。

//甲机发送程序

```
#include <reg51.h>
```

```
sbit P=PSW^0; //P位为PSW寄存器的第0位，即奇偶校验位
```

```
unsigned char Tab[8]= {0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf,  
                      0xbf, 0x7f}; //控制流水灯显示数据数组，为全局变量
```

```
void main(void)
```

```
//主函数
```

```
{
```

```
    unsigned char i;
```

```
    TMOD=0x20;
```

```
//设置定时器T1为方式2
```

```
    SCON=0xc0;
```

```
//设置串口为方式3
```

```
    PCON=0x00;
```

```
//SMOD=0
```

TH1=0xfd;	//给T1赋初值，波特率设置为9600
TL1=0xfd;	
TR1=1;	//启动定时器T1
while(1)	
{	
for(i=0;i<8;i++)	
{	
Send(Tab[i]);	
delay();	//大约200ms发送一次数据
}	
}	
}	
 void Send(unsigned char dat)	// 发送1字节数据的函数
{	
TB8=P;	// 将偶校验位作为第9位数据发送
SBUF=dat;	
while(TI==0);	//检测TI，TI=0，未发送完
;	// 空操作
TI=0;	// 1字节发送完，TI清0
}	

```
void delay (void)
```

```
// 延时约200ms的函数
```

```
{  
    unsigned char m,n;  
    for(m=0;m<250;m++)  
        for(n=0;n<250;n++);  
}
```

//乙机接收程序

```
#include <reg51.h>
```

```
sbit P=PSW^0;
```

```
// P位为PSW 寄存器的第0位，即奇偶校验位
```

```
void main(void)
```

```
//主函数
```

```
{  
    TMOD=0x20;  
    SCON=0xd0;  
    PCON=0x00;  
    TH1=0xfd;  
    TL1=0xfd;  
    TR1=1;  
    REN=1;
```

```
//设置定时器T1为方式2
```

```
//设置串口为方式3，允许接收REN=1
```

```
// SMOD=0
```

```
//给定时器T1赋初值，波特率为9600
```

```
//接通定时器T1
```

```
//允许接收
```

```
while(1)
{
P1= Receive( );           //将接收到的数据送P1口显示
}
}
```

```
unsigned char Receive(void)    //接收1字节数据的函数
{
unsigned char dat;
while(RI==0);                 //检测RI, RI=0, 未接收完, 则循环等待
;
RI=0;                         //已接收一帧数据, 将RI清0
ACC=SBUF;                     //将接收缓冲器的数据存于ACC
if(RB8==P)                    //只有偶校验成功才能往下执行, 接收数据
{
    dat=ACC;                  //将接收缓冲器的数据存于dat
    return dat;              //将接收的数据返回
}
}
```

8.6.4 多机通信的应用设计

下面通过一个具体案例，介绍如何实现单片机的多机通信。

【例8-7】实现主单片机分别与3个从单片机串行通信，原理电路见图8-31。用户通过分别按下开关k1、k2或k3来选择主机与对应1#、2#或3#从机串行通信，当黄色LED点亮，表示主机与相应的从机连接成功；该从机的8个绿色LED闪亮，表示主机与从机在进行串行数据通信。如果断开k1、k2或 k3，则主机与相应从机的串行通信中断。

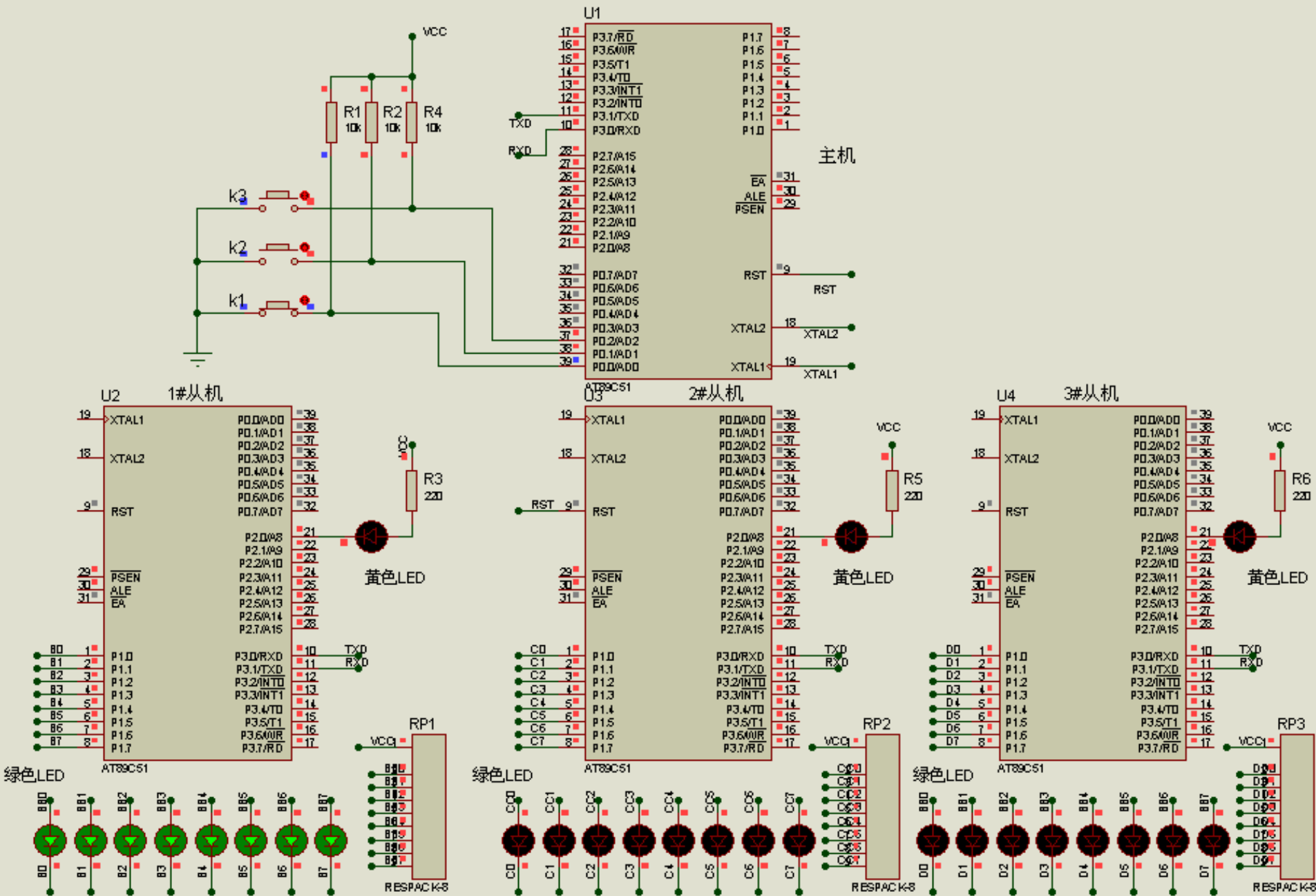


图8-31 主机与3从机的多机通信的原理电路与仿真

本例实现主、从机串行通信，各从机程序都相同，只是地址不同。串行通信约定如下。

- (1) 3台从机的地址为01H~03H。
- (2) 主机发出的0xff为控制命令，使所有从机都处于SM2=1的状态。
- (3) 其余的控制命令：00H—接收命令，01H—发送命令。这两条命令是以数据帧形式发送的。
- (4) 从机的状态字如图8-32所示。

	D7	D6	D5	D4	D3	D2	D1	D0
状态字	ERR	0	0	0	0	0	TRDY	RRDY

图8-32 从机状态字格式约定

其中：

ERR (D7位) =1, 表示收到非法命令。

TRDY (D1位) =1, 表示发送准备完毕。

RRDY (D0位) =1, 表示接收准备完毕。

串行通信时，主机采用查询方式，从机采用中断方式。主机串口设为方式3，允许接收，并置TB8为1。因只有1个主机，所以主机SCON控制寄存器中的SM2不要置1，故控制字为11011000，即0xd8。

参考程序如下：

//主机程序

```
#include <reg51.h>
#include <math.h>
sbit switch1=P0^0;
sbit switch2=P0^1;
sbit switch3=P0^2;

//定义k1与P0.0连接
//定义k2与P0.1连接
//定义k3与P0.2连接

void main()
{
    EA=1;
    TMOD=0x20;
    TL1=0xfd;
    TH1=0xfd;
    PCON=0x00;
    SCON=0xd0;
    TR1=1;
    ES=1;
    SBUF=0xff;

    //主函数

    //总中断允许
    //设置T1定时方式2 自动装载初值
    //波特率设为9600

    //SMOD=0, 不倍增
    //SM2设为0, TB8设为0
    //启动定时器T1
    //允许串口中断
    //串口发送0xff
```

```

while(TI==0);           //判是否发送完毕
TI=0;                   //发送完毕，TI清0
while(1)
{
    delay_ms(100);
    if(switch1==0)      //判是否k1按下，k1按下往下执行
    {
        TB8=1;          //第9位数据为1，送TB8，准备发地址帧
        SBUF=0x01;      //串口发1#从机的地址0x01以及TB8=1
        while(TI==0);   //判是否发送完毕
        TI=0;           //发送完毕，TI清0
        TB8=0;          //发送的第9位数据为0，送TB8，准备发数据帧
        SBUF=0x00;      //串口发送0x00以及TB8=0
        while(TI==0);   //判是否发送完毕
        TI=0;           //发送完毕，TI清0
    }
}

```

```
if (switch2==0)           //判断是否k2按下，k2按下往下执行
{
    TB8=1;                //发送的第9位数据为1，发地址帧
    SBUF=0x02;             //串口发2#从机的地址0x02
    while (TI==0);         //判断是否发送完毕
    TI=0;                  //发送完毕，TI清0
    TB8=0;                //准备发数据帧
    SBUF=0x00;             //发数据帧0x00及TB8=0
    while (TI==0);         //判断是否发送完毕
    TI=0;                  //发送完毕，TI清0
}
```

```
if (switch3==0)
```

```
{
```

```
    TB8=1;
```

```
//准备发地址帧
```

```
    SBUF=0x03;
```

```
//发3#从机地址
```

```
    while (TI==0);
```

```
//判是否发送完毕
```

```
    TI=0;
```

```
//发送完毕，TI清0
```

```
    TB8=0;
```

```
//准备发数据帧
```

```
    SBUF=0x00;
```

```
//发数据帧0x00及TB8=0
```

```
    while (TI==0);
```

```
//判是否发送完毕
```

```
    TI=0;
```

```
//发送完毕，TI清0
```

```
}
```

```
}
```

```
}
```

```
void delay_ms(unsigned int i)    //函数功能： 延时
```

```
{
    unsigned char j;
    for(;i>0;i--)
        for(j=0;j<125;j++)
            ;
}
```

//从机1串行通信程序

```
#include <reg51.h>
```

```
#include <math.h>
```

```
sbit led=P2^0;    //定义P2.0连接的黄色LED
```

```
bit rrdy=0;    //接收准备标志位rrdy=0，表示未做好接收准备
```

```
bit trdy=0;    //发送准备标志位trdy=0，表示未做好发送准备
```

```
bit err=0;    //err=1，表示接收到的命令为非法命令
```

```
void main()
```

```
{
```

```
    EA=1;
```

```
    TMOD=0x20;
```

```
    TL1=0xfd;
```

```
    TH1=0xfd;
```

```
    PCON=0x00;
```

```
    SCON=0xd0;
```

```
    TR1=1;
```

```
    P1=0xff;
```

```
    ES=1;
```

```
    while (RI==0);
```

```
    if (SBUF==0xff) err=0; //如果接收到的数据为0xff, err=0, 表示正确
```

```
    else err=1; //err=1, 表示接收出错
```

```
    RI=0; //接收中断标志清0
```

```
    SM2=1; //多机通讯控制位, SM2置1
```

```
    while(1);}
```

```
//从机1主函数
```

```
//总中断打开
```

```
//T1方式2自动装载, 用于设置波特率
```

```
//波特率设为9600
```

```
//SMOD=0
```

```
//SM2设为0, TB8设为0
```

```
//启动定时器T1
```

```
//向P1写入全1, 8个绿色LED全灭
```

```
//允许串口中断
```

```
//接收控制指令0xff
```

```
//err=0, 表示正确
```

```
//err=1, 表示接收出错
```

```
//接收中断标志清0
```

```
//多机通讯控制位, SM2置1
```

```

void int1() interrupt 4    //T1中断函数
{
    if(RI)                //如果RI=1
    {
        if(RB8)           //如果RB8=1，表示接收的为地址帧
        {
            RB8=0;
            if(SBUF==0x01)  //如接收的数据为地址帧0x01，是本从机的地址
            {
                SM2=0;      //则SM2清0，准备接收数据帧
                led=0;       //点亮本从机黄色发光二极管
            }
        }
    }
    else                  //如果接收的不是本从机的地址
    {
        rrdy=1;           //准备好接收标志置1
        P1=SBUF;          //串口接收的数据送P1
    }
}

```

```
SM2=1;           // SM2仍为1
led=1;           // 熄灭本从机黄色发光二极管
}
RI=0;
}
delay_ms(50);
P1=0xff;         //熄灭本从机8个绿色发光二极管
}
```

```
void delay_ms(unsigned int i) //函数功能： 延时
{
    unsigned char j;
    for(;i>0;i--)
        for(j=0;j<125;j++)
            ;
}
```


//从机2串行通信程序

```
#include <reg51.h>
#include <math.h>
sbit led=P2^0;
bit rrdy=0;
bit trdy=0;
bit err=0;
void delay_ms(unsigned int i)
{unsigned char j;
for(;i>0;i--)
for(j=0;j<125;j++;)}
```

```
void main()
```

```
//从机2主程序
```

```
{
```

```
EA=1;
```

```
//总中断打开
```

```
TMOD=0x20;//定时器1 工作方式2 自动装载 用于串口设置波特率
```

```
TL1=0xfd;
```

```
//SM2设为1, TB8设为0
```

```
TR1=1;
```

```
//定时器1 打开
```

```
P1=0xff;
```

```
ES=1;
```

```
//允许串口中断
```

```
while(RI==0);
```

```
//接收控制指令 0xff
```

```
if(SBUF==0xff) err=0;
```

```
else err=1;
```

```
RI=0;
```

```
SM2=1;
```

```
while(1);
```

```
}
```

```
void int1() interrupt 4    //函数功能：串口中断函数
```

```
{  
    if(RI)  
    {if(RB8)  
        {RB8=0;  
        if(SBUF==0x02)  
            {SM2=0;  
            led=0;  
            }  
        }  
        else  
        {    rrdy=1;P1=SBUF;  
            SM2=1;led=1;  
        }  
        RI=0;  
    }  
    delay_ms(50);  
    P1=0xff;  
}
```

//从机3串行通信程序

```
#include <reg51.h>
```

```
#include <math.h>
```

```
sbit led=P2^0;
```

```
bit rrdy=0;
```

```
bit trdy=0;
```

```
bit err=0;
```

```
void delay_ms(unsigned int i)
```

//函数功能：延时

```
{
```

```
    unsigned char j;
```

```
    for(;i>0;i--)
```

```
        for(j=0;j<125;j++);
```

```
}
```

```
void main()
```

```
//从机3主程序
```

```
{
```

```
EA=1;
```

```
//总中断打开
```

```
TMOD=0x20;
```

```
//T1方式2,用于串口设置波特率
```

```
TL1=0xfd;TH1=0xfd;
```

```
//波特率设为9600
```

```
PCON=0x00;
```

```
//波特率不倍增 0x80 为倍增
```

```
SCON=0xf0;
```

```
//SM2设为1, TB8设为0
```

```
TR1=1;
```

```
//接通T1
```

```
P1=0xff;ES=1;
```

```
while(RI==0);
```

```
//接收控制指令 0xff
```

```
if(SBUF==0xff) err=0;
```

```
else err=1;
```

```
RI=0;SM2=1;
```

```
while(1);
```

```
}
```

`void int1() interrupt 4` `//函数功能： 串行口中断函数`

```
{  
    if(RI)  
    {if(RB8)  
        {   RB8=0;  
            if(SBUF==0x03)  
            {  
                SM2=0;led=0;}  
            }  
            else  
            {  
                rrdy=1;P1=SBUF;  
                SM2=1;led=1;    }  
                RI=0;    }  
                delay_ms(50);  
                P1=0xff;}
```

8.6.5 单片机与PC机串行通信的设计

工业现场测控系统中，常用单片机进行监测点的数据采集，然后单片机通过串口与PC通信，把采集的数据串行传送到PC机上，再在PC机上进行数据处理。

PC机配置都是RS-232标准串口，为9针“D”型插座，输入/输出为RS-232电平。“D”型9针插头引脚定义见图8-33。

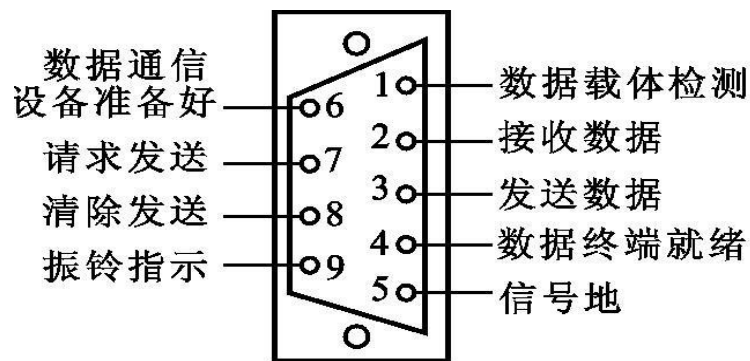


图8-33 “D”型9针插头引脚定义

表8-3为RS-232C “D”型9针插头引脚定义。由于两者电平不匹配，因此必须把单片机输出的TTL电平转换为RS-232电平。单片机与PC机接口方案如图8-34所示。图中电平转换芯片为MAX232，接口连接只用了3条线，即RS-232插座中的2脚、3脚与5脚。

表8-3 PC的RS-232C接口信号

引脚号	功能	符号	方向
1	数据载体检测	DCD	输入
2	接收数据	TXD	输出
3	发送数据	RXD	输入
4	数据终端就绪	DTR	输出
5	信号地	GND	
6	数据通信设备准备好	DSR	输入
7	请求发送	RTS	输出
8	清除发送	CTS	输入
9	振铃指示	RI	输入

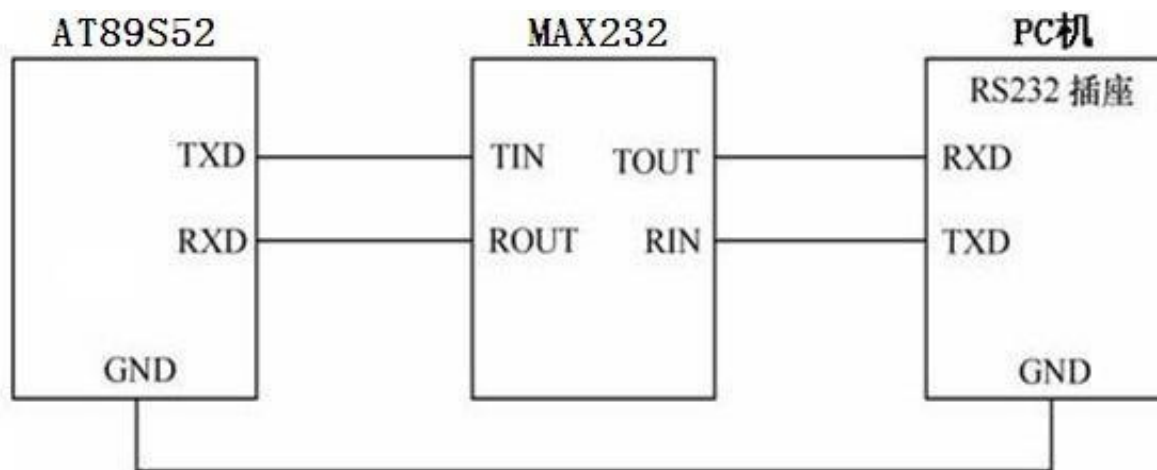


图8-34 单片机与PC机的RS-232串行通信接口

1. 单片机向PC机发送数据

【例8-8】单片机向计算机发送数据的Proteus仿真电路如图8-35。要求单片机通过串行口的TXD脚向计算机串行发送8个字节数据。本例使用两个串口虚拟终端，观察串行口线上出现的串行传输数据。

允许弹出的两个虚拟终端窗口，见图8-36。VT1窗口显示的数据表示单片机串口发给PC机数据，VT2显示的数据表示由PC机经RS232串口模型COMPIM接收到的数据，由于使用了串口模型COMPIM，从而省去PC机模型，解决了单片机与PC机串行通信的虚拟仿真问题。

实际上单片机向计算机和单片机向单片机发送数据的方法是完全一样。

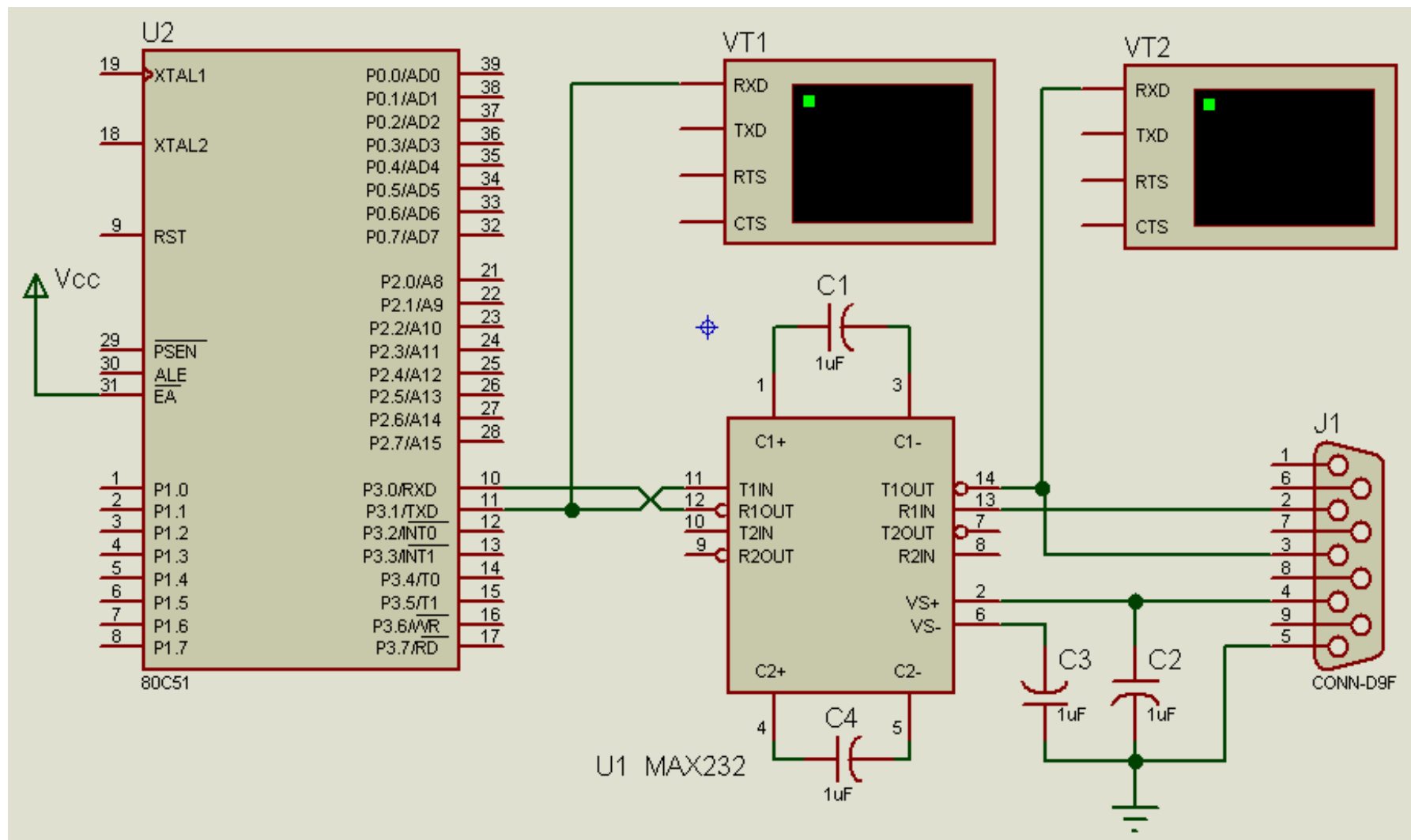


图8-35 单片机向计算机发送数据的proteus仿真电路

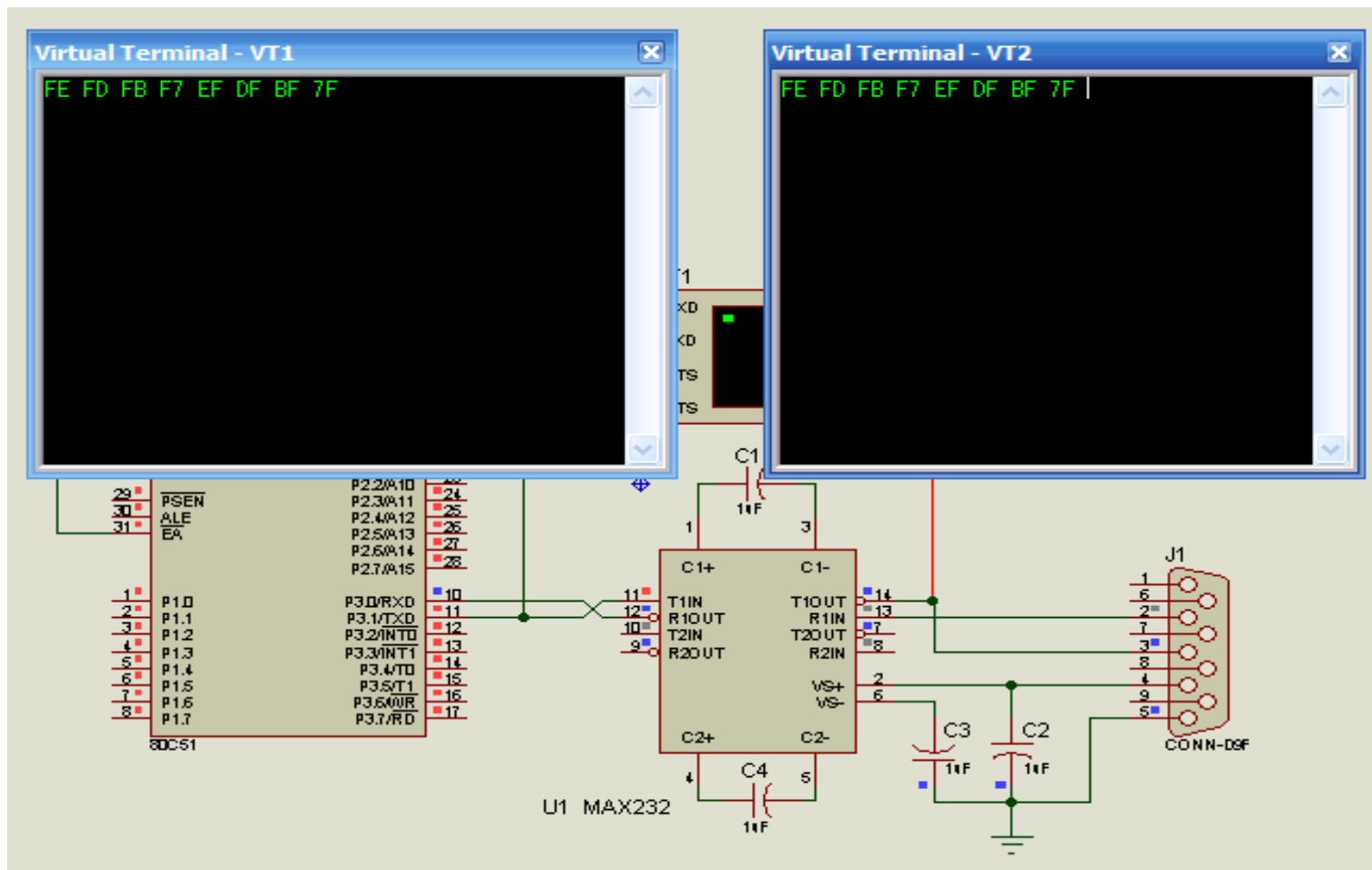


图8-36 从两个虚拟终端窗口观察到的串行通信数据

参考程序:

```
#include <reg51.h>
code Tab[ ]={ 0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f };
void send(unsigned char dat )
{
    SBUF=dat;                //待发送数据写入发送缓冲寄存器
    while(TI==0);           //串口未发完, 等待
    ;                        //空操作
    TI=0;                   //1字节发送完, 软件将TI标志清0
}

void delay(void )           //延时约200ms函数
{
    unsigned char m,n;
    for(m=0;m<250;m++)
    for(n=0;n<250;n++);}
```

```
void main(void)
```

```
//主函数
```

```
{
```

```
    unsigned char i;
```

```
    TMOD=0x20;
```

```
//设置T1为定时器方式2
```

```
    SCON=0x40;PCON=0x00;
```

```
//串行口方式1，TB8=1
```

```
    TH1=0xfd;TL1=0xfd;
```

```
//波特率9600
```

```
    TR1=1;
```

```
//启动T1
```

```
    while(1)
```

```
//循环
```

```
    {
```

```
        for(i=0;i<8;i++)
```

```
//发送8次流水灯控制码
```

```
        {
```

```
            send(Tab[i]);
```

```
//发送数据
```

```
            delay( );
```

```
//每隔200ms发送一次数据
```

```
        }while(1);
```

```
    }
```

```
}
```

2. 单片机接收PC机发送的数据

【例8-9】 单片机接收计算机发送的串行数据，并把接收到的数据送P1口8位LED显示。原理电路见图8-37。本例采用串行口来模拟PC机的串行口。

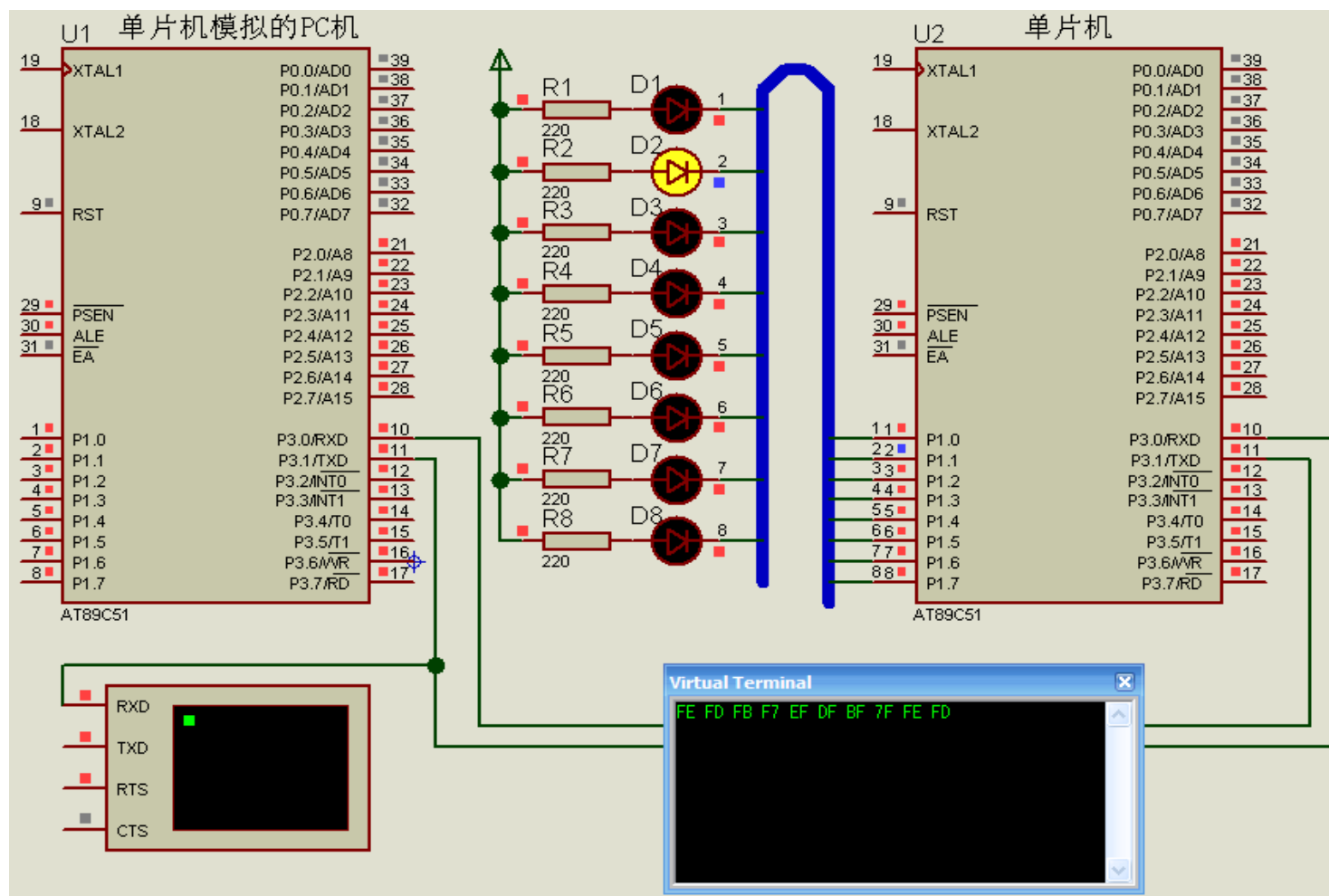


图8-37 单片机接收PC机发送的串行数据的原理电路

参考程序:

//PC机发送程序(用单片机串口模拟PC机串口发送数据)

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
uchar tab[]={0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f};
```

```
void delay(unsigned int i)
```

```
{
```

```
    unsigned char j;
```

```
    for(;i>0;i--)
```

```
        for(j=0;j<125;j++);
```

```
}
```



```
void main()
```

```
{    uchar i;                //设置定时器T1为方式2
    TH1=0xfd;TL1=0xfd;      //波特率9600
    SCON=0x40;              //方式1只发，不收
    PCON=0x00;              //串口初始化为方式0
    TR1=1;                  //启动T1
    while(1)
    {
        for(i=0;i<8;i++)
        {SBUF=tab[i];        //数据送串行口发送
          while(TI==0);      //如果TI=0，未发送完，循环等待
          TI=0;               //已发送完，再把TI清0
          delay(1000);}
    }
}
```

//单片机接收程序

```
#include <reg51.h>
#define uchar unsigned char
#define uint unsigned int
void main( )
{
    uchar temp=0;
    TMOD=0x20;           //设置T1为方式2
    TH1=0xfd;TL1=0xfd;   //波特率9600
    SCON=0x50;           //设置串口为方式1接收, REN=1
    PCON=0x00;           // SMOD=0
    TR1=1;               // 启动T1
    while(1)
    {
        while(RI==0);    //若RI为0, 未接收到数据
        RI=0;            //接收到数据, 则把RI清0
        temp=SBUF;        //读取数据存入temp中
        P1=temp ;         //接收的数据送P1口控制8个LED的亮与灭
    }
}
```

8.6.6 PC机与单片机或与多个单片机的串行通信

一台PC与若干台AT89S52可构成小型分布式测控系统，见图8-38。

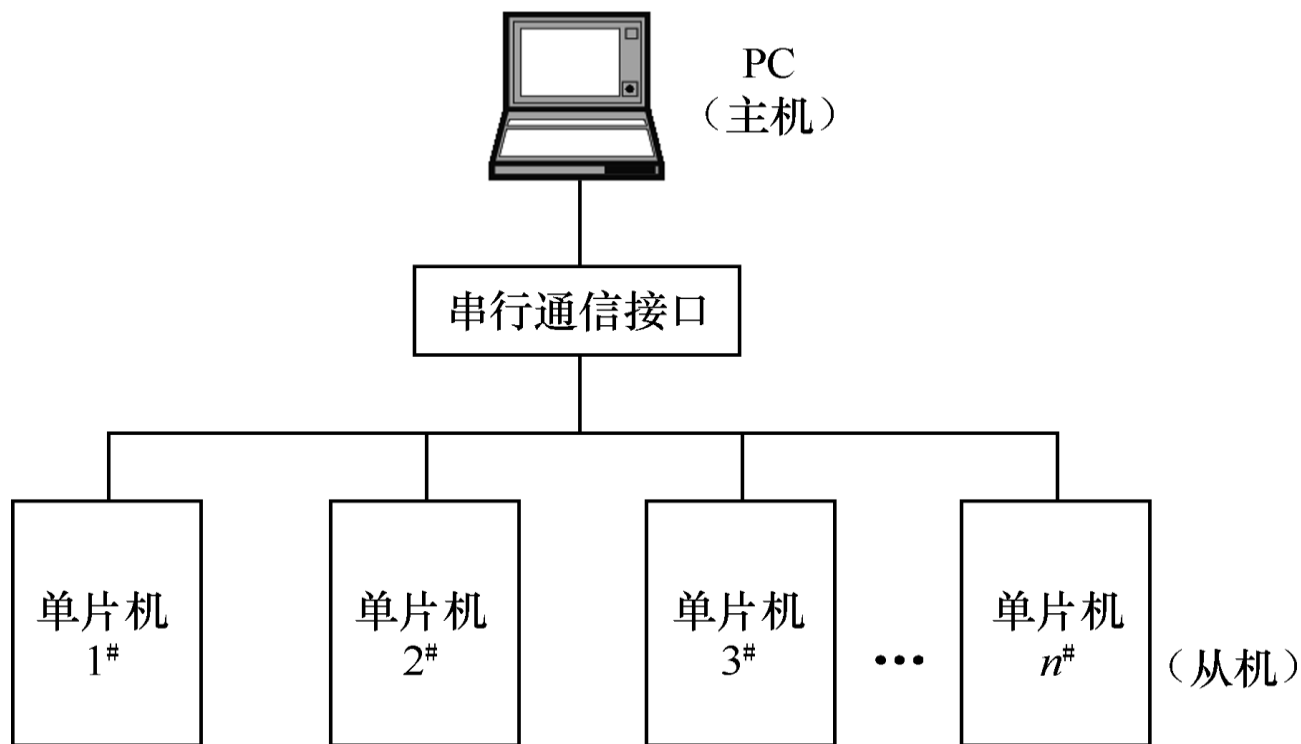


图8-38 PC机与多台单片机构成小型的分布式测控系统



图8-38所示系统在许多实时工业控制和数据采集集中，充分发挥了单片机功能强、抗干扰性好、面向控制等优点，同时又利用PC机弥补单片机在数据处理和人机对话等方面不足。

应用系统一般以PC为主机，定时扫描以AT89S52为核心的前沿单片机，以便采集数据或发送控制信息。在系统中，以AT89S52为核心的智能测量和控制仪表（从机）既能独立完成数据处理和控制任务，又可将数据传送给PC（主机）。PC将数据进行处理或显示或打印，同时将各种控制命令传送给各个子机，以实现集中管理和最优控制。

显然，要组成这样一个分布式测控系统，首先要解决PC机与单片机间的串行通信接口问题。

下面以RS-485串行多机通信为例，说明PC机与数台AT89S51单片机进行多机通信的接口电路设计方案。

PC机配有RS-232C串行标准接口，可通过转换电路转换成RS-485串行接口，AT89S52单片机本身具有一个全双工的串行口，该串口加驱动电路后就可实现RS-485串行通信。PC机与数台AT89S52单片机进行多机通信的RS-485串行通信接口电路见图8-39。

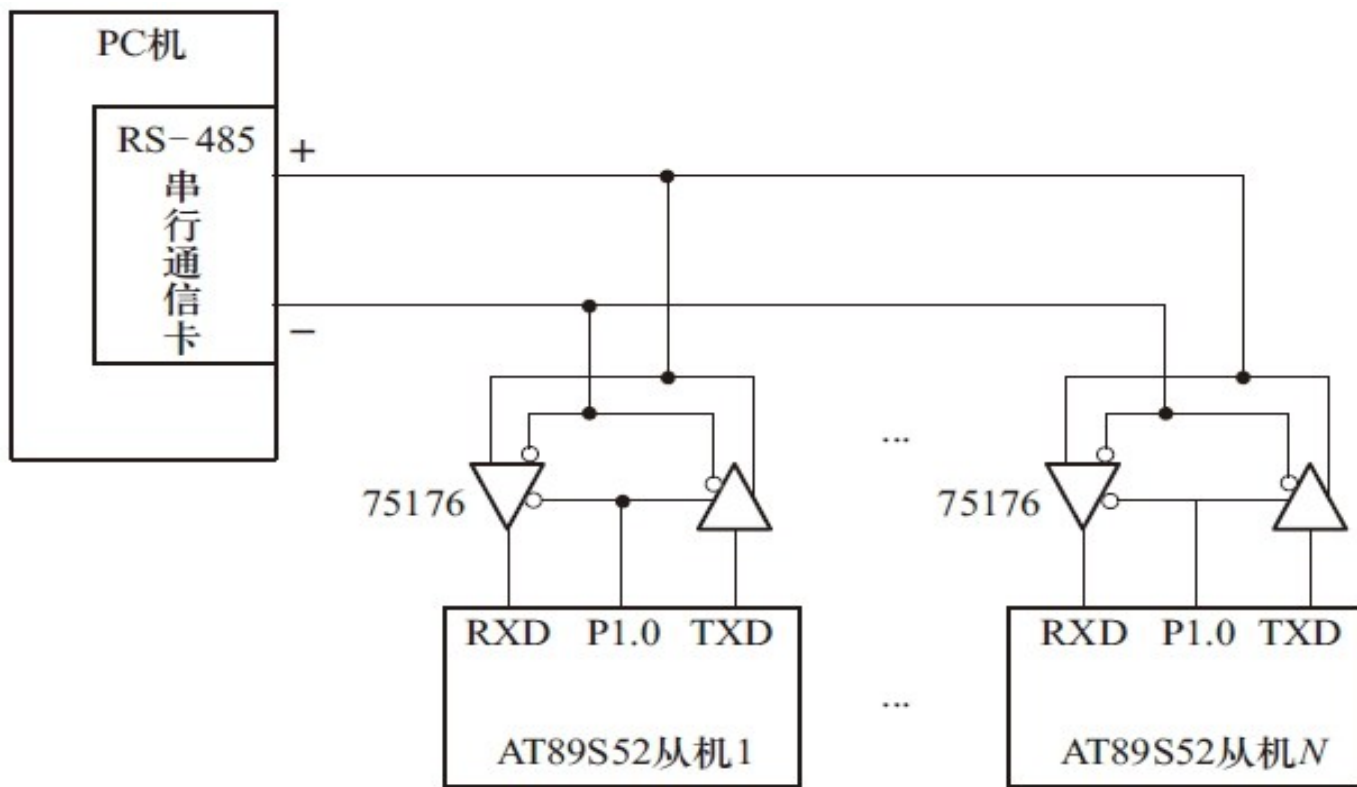
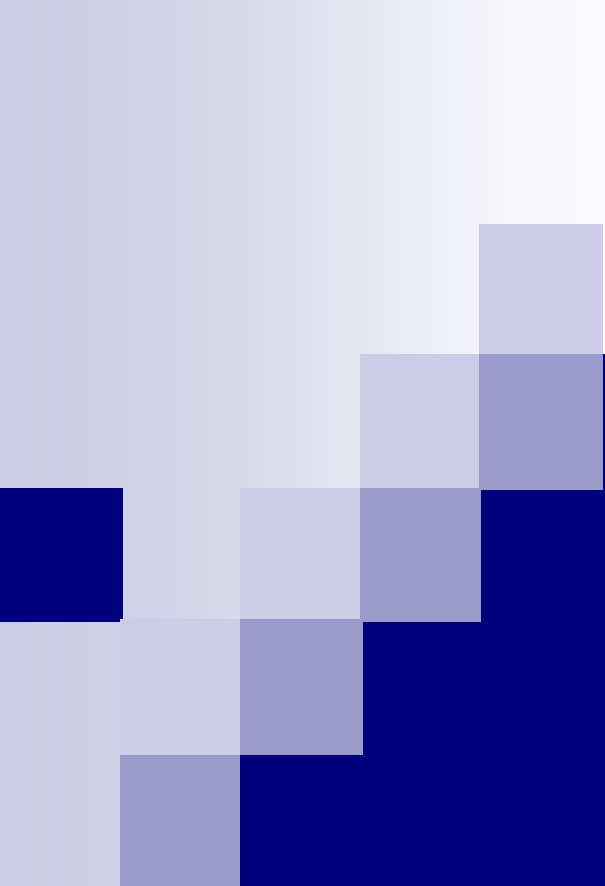


图8-39 PC机与AT89S52串行通信接口电路



在图8-39中，AT89S52单片机串口通过75176芯片驱动后就可转换成RS-485标准接口，根据RS-485接口特性，从机数量不多于32个。PC机与AT89S52间通信采用主从方式，PC为主机，各单片机为从机，由PC机来确定与哪个单片机进行通信。

有关PC机与多单片机的串行通信软件编程，可供参考的资料较多，读者可查阅相关的参考文献。



第9章 单片机系统的并行扩展



虽然**AT89S52**单片机片内集成**8K程序存储器**、**256个单元**的数据存储器以及**4个8位并行I/O口**，但在许多情况下，片内存储器与**I/O资源**以及外围器件还不能满足需要，为此需要对单片机进行存储器、**I/O**以及外围器件的扩展。

本章介绍单片机系统的并行扩展，下一章介绍串行扩展。

9.1 系统并行扩展技术

9.1.1 系统并行扩展结构

单片机系统并行扩展结构见图9-1。

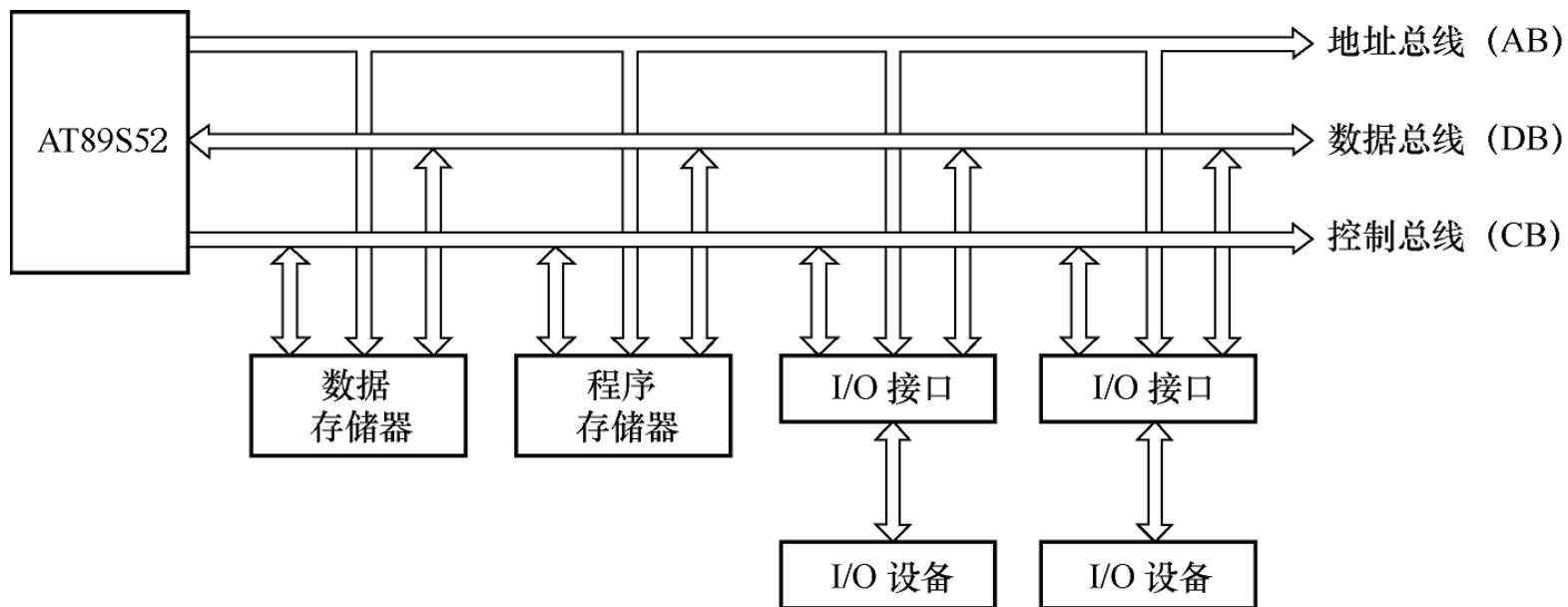


图9-1 AT89S52单片机的系统扩展结构

由图9-1可看出，系统并行扩展主要包括数据存储器扩展、程序存储器扩展和I/O接口的扩展。由于目前AT89S5x系列单片机片内都集成了不同容量的串行下载可编程的Flash存储器与一定数量的RAM，如表9-1所示，如果片内存储器资源能够满足系统设计需求，扩展存储器的工作可以省去。

AT89S52单片机采用程序存储器空间和数据存储器空间截然分开的哈佛结构，因此形成了两个并行的外部存储器空间。在AT89S52系统中，I/O端口与数据存储器采用统一编址方式，即I/O接口芯片的每一个端口寄存器就相当于一个RAM存储单元。

由于AT89S52单片机采用并行总线结构，扩展的各种外围接口器件只要符合总线规范，就可方便地接入系统。并行扩展是通过系统总线把AT89S52单片机与各扩展器件连接起来。因此，要并行扩展首先要构造

系统总线。

表9-1 AT89S5x系列单片机片内的存储器资源

型 号	片内闪烁存储器容量	片内RAM存储器容量
AT89S51	4 k B	128 B
AT89S52	8 k B	256 B
AT89S53	12k B	256 B
AT89S54	16k B	256 B
AT89S55	20k B	256 B

系统总线按功能通常分为3组，见**图9-1**。

(1) 地址总线 (**Address Bus, AB**) : 传送**单片机单向发出的地址信号**, 以便进行存储单元和**I/O**接口芯片中的寄存器单元选择。

(2) 数据总线 (**Data Bus, DB**) : 用于单片机与外部存储器之间或与**I/O**接口之间双向传送数据。

(3) 控制总线 (**Control Bus, CB**) : 是单片机发出的各种控制信号线。

下面介绍如何构造系统的**三总线**。

1. P0口作为低8位地址/数据总线

AT89S52受引脚数目限制，**P0**口既用作**低8位地址总线**，又用作**数据总线**（分时复用），因此需增加1个8位地址锁存器。**AT89S52**单片机对外部扩展的存储器单元或**I/O**接口寄存器进行访问时，先发出低8位地址送地址锁存器锁存，锁存器输出作为系统的低8位地址（**A7~A0**）。随后，**P0**口又作为数据总线口（**D7~D0**），见图**9-2**。

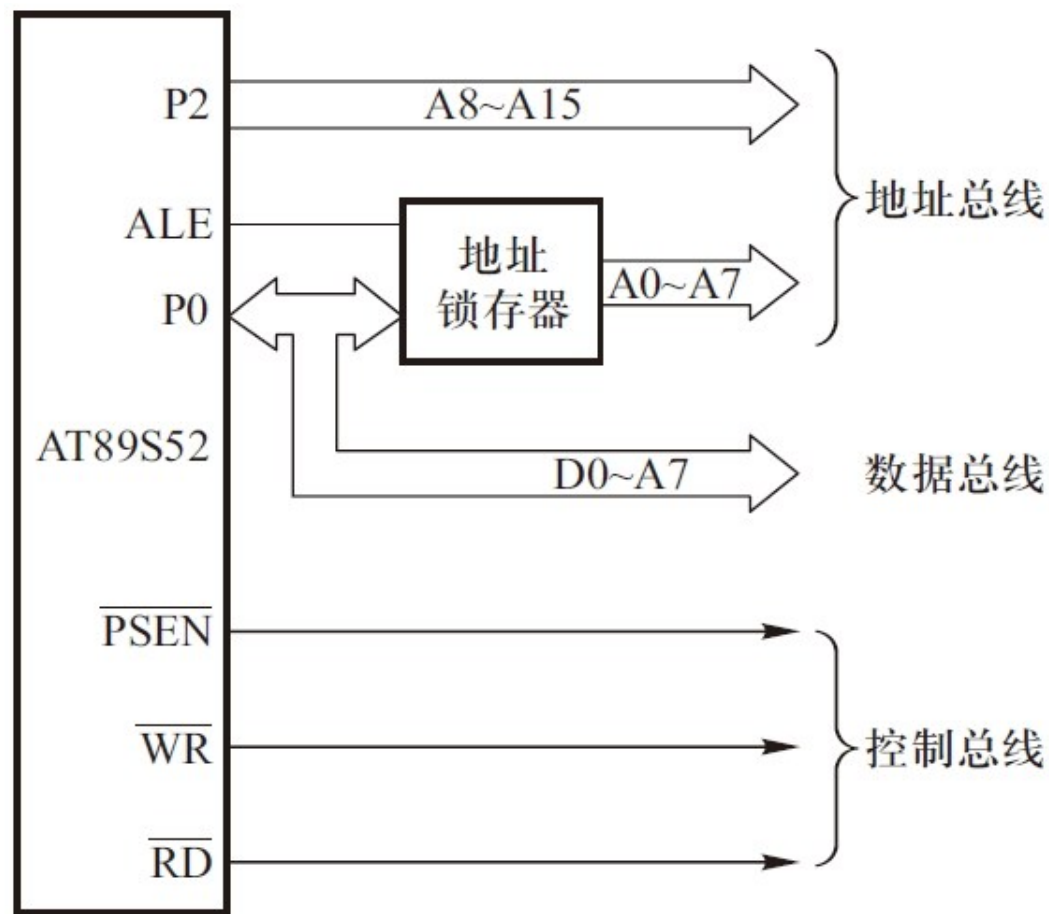


图9-2 AT89S52单片机扩展的片外三总线

2. P2口的口线作为高位地址

P2口的全部8位口线用作系统高8位地址线，再加上地址锁存器输出提供的低8位地址，便形成了系统的16位地址总线（见**图9-2**），从而使单片机系统的寻址范围达到64KB（ 2^{16} ）。

3. 控制信号线

除了地址线和数据线外，还要有控制总线。这些信号有的就是单片机引脚的第一功能信号，有的则是P3口第二功能信号。其中包括：

(1) **PSEN***信号作为外部扩展的程序存储器的读选通控制信号；

(2) **RD***和**WR***信号作为外部扩展的数据存储器和I/O接口寄存器的读、写选通控制信号；

(3) **ALE**信号作为P0口发出的低8位地址的锁存控制信号。

(4) **EA***信号为片内、片外程序存储器访问允许控制端。

由上看出，尽管AT89S52有4个并行I/O口，共32条口线，但由于系统扩展的需要，真正给用户作为通用I/O使用的，就剩下P1口和P3口的部分口线了。

9.1.2 地址空间分配

如何把片外两个64KB地址空间分配给各个存储器与I/O接口芯片，使一个存储单元只对应一个地址，避免单片机对一个地址单元访问时，发生数据冲突。这就是存储器地址空间的分配问题。

AT89S52发出的地址信号用于选择某存储器单元，在外扩多片存储器芯片中，要完成这种功能，必须进行**两种选择**：一是必须选中该存储器芯片，这称为**“片选”**，只有被“选中”的存储器芯片才能被单片机访问，未被选中的芯片不能被访问；二是在“片选”的基础上还同时“选中”芯片的某一单元对其进行读写，这称为**“单元选择”**。每个扩展的芯片都有“片选”引脚，同时每个芯片也都有多条地址引脚，以便对其进行单元选择。

需要注意的是，“片选”和“单元选择”都是单片机通过地址线一次发出的地址信号来完成选择的。

常用的存储器地址空间分配方法有两种：线性选择法（简称线选法）和地址译码法（简称译码法），下面分别介绍。

1. 线选法

直接利用单片机某一高位地址线作为存储器芯片（或I/O接口芯片）的“片选”控制信号。为此，只需用某一高位地址线与存储器芯片的“片选”端直接连接即可。

优点：电路简单，不需另外增加地址译码器硬件电路，体积小，成本低。

缺点：可寻址芯片数目受限制。另外，地址空间不连续，存储单元地址不唯一，这会给程序设计带来一些不便。线选法适用于外扩芯片数目不多的单片机系统的系统扩展。

2. 译码法

使用译码器对AT89S52单片机的高位地址进行译码，将译码器的译码输出作为存储器芯片的片选信号。本法能有效地利用存储器空间，适于多芯片的存储器扩展。

常见有74LS138（3-8译码器）、74LS139（双2-4译码器）与74LS154（4-16译码器）。下面介绍74LS138和74LS139译码器芯片。

（1）74LS138是3-8译码器，有3个数据输入端，经译码后产生8种状态，其引脚见图9-3，真值表见表9-2。

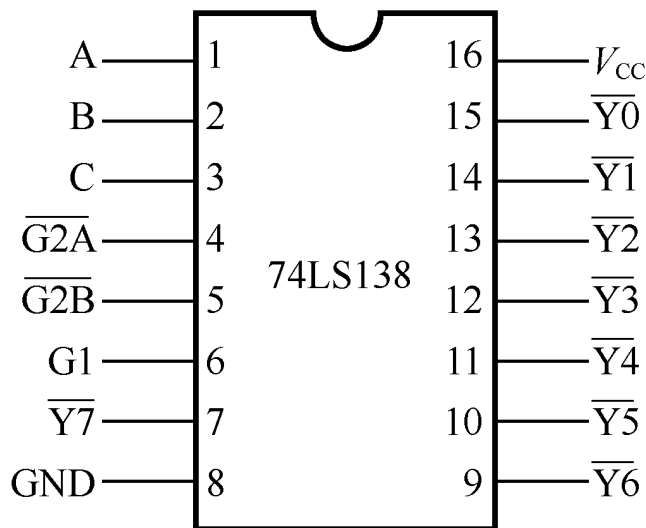


图9-3 74LS138引脚图

由表9-2可见，当译码器输入为某一固定编码时，其8个输出引脚Y0*~Y7*中仅有1个引脚输出为低，其余全为高。而输出低电平的引脚恰好作为片选信号。

表9-2 74LS138 真值表

输 入 端						输 出 端							
G1	$\overline{G2A}$	$\overline{G2B}$	C	B	A	$\overline{Y7}$	$\overline{Y6}$	$\overline{Y5}$	$\overline{Y4}$	$\overline{Y3}$	$\overline{Y2}$	$\overline{Y1}$	$\overline{Y0}$
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1
其他状态			×	×	×	1	1	1	1	1	1	1	1

注：1 表示高电平，0 表示低电平，×表示任意。

(2) 74LS139 双2-4译码器。两个译码器完全独立，分别有各自的数据输入端、译码状态输出端以及数据输入允许端，其引脚如图9-4所示，其中1组的真值表见表9-3。

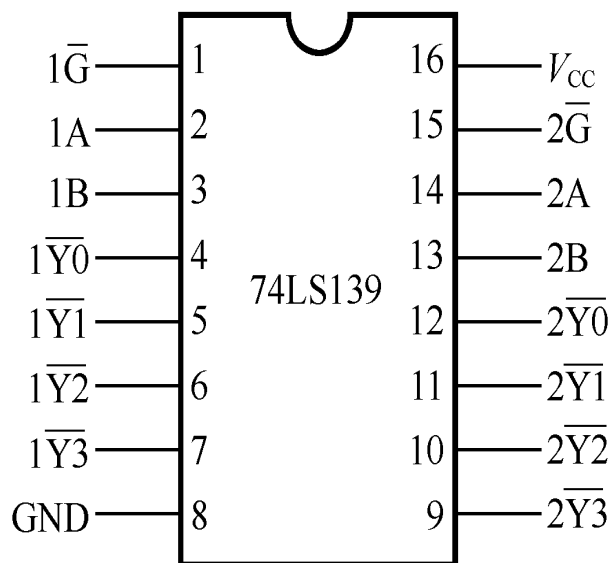


图9-4 74LS139引脚图

表9-3 74LS139真值表

输 入 端			输 出 端			
允许	选择					
$\bar{G}$	B	A	$\bar{Y}_3$	$\bar{Y}_2$	$\bar{Y}_1$	$\bar{Y}_0$
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1
1	×	×	1	1	1	1

注：1 表示高电平，0 表示低电平，×表示任意。

以74LS138为例，介绍如何进行空间地址分配。**例如**，要扩8片8KB的RAM 6264，如何通过74LS138把64KB空间分配给各个芯片？

由74LS138真值表可知，把**G1**接到+5V，该**G_{2A}***、**G_{2B}*** 接地，**P2. 7**、**P2. 6**、**P2. 5**（高3位地址线）分别接到74LS138的**C**、**B**、**A**端，由于对高3位地址译码，这样译码器有8个输出**Y0*~Y7***，分别接到8片6264的“**片选**”端，实现**8选1片选**。

而**低13位地址**（**P2. 4~P2. 0**，**P0. 7~P0. 0**）完成对选中的6264芯片中的各个存储单元的“**单元选择**”。这样就把64KB存储器空间分成8个8KB空间了。64KB地址空间的分配见**图9-5**。

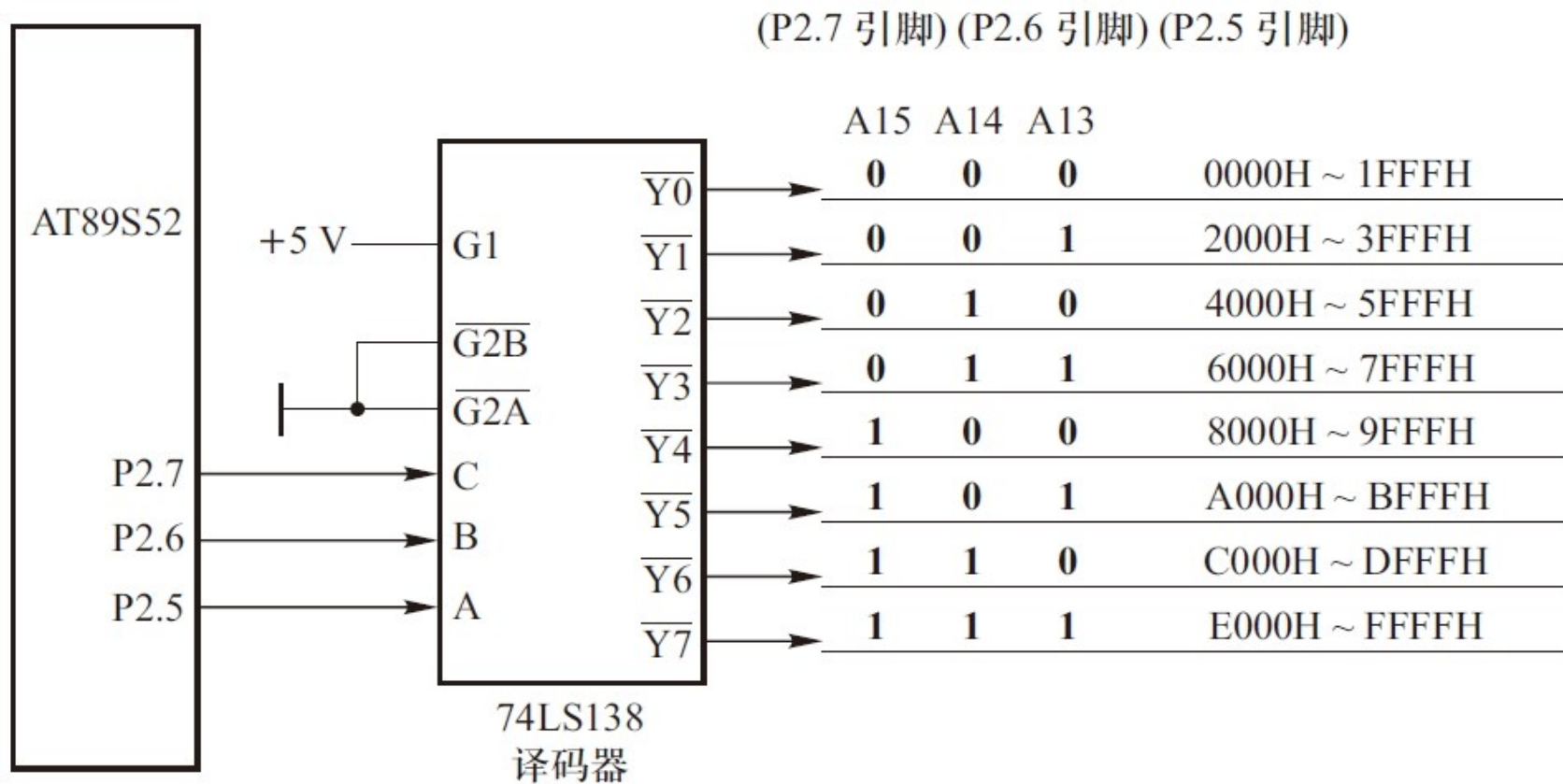


图9-5 64KB空间划分成8个8KB空间



当**AT89S52**单片机发出**16**位地址码时，每次只能选中一片芯片以及该芯片的唯一存储单元。

采用译码器划分的地址空间块相等，如将地址空间块划分为不等的块，可用**FPGA**实现非线性译码逻辑来代替译码器。

9.1.3 外部地址锁存器

AT89S52单片机受引脚数的限制，**P0**口兼用数据线和低**8**位地址，为了将它们分离出来，需要在**单片机外部增加地址锁存器**。目前，常用的地址锁存器芯片有**74LS373**、**74LS573**等。

1. 锁存器74LS373

带有三态门的8D锁存器，其引脚见图9-6，内部结构如图9-7所示。

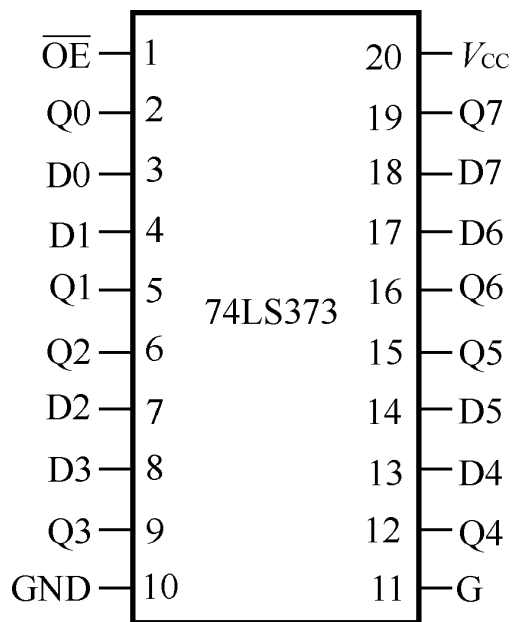


图9-6 锁存器74LS373的引脚

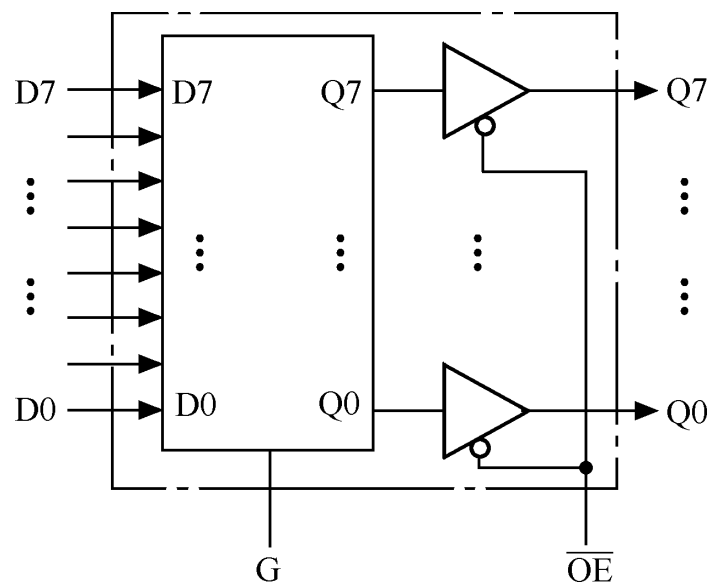


图9-7 74LS373的内部结构

74LS373引脚说明:

- D7~D0—8位数据输入线。
- Q7~Q0—8位数据输出线。
- G—数据输入锁存选通信号。当加到该引脚的信号为高电平时，外部数据选通到内部锁存器，负跳变时，数据锁存到锁存器中。
- OE*—数据输出允许信号，低电平有效。当该信号为低电平时，三态门打开，锁存器中数据输出到数据输出线。当该信号为高电平时，输出线为高阻态。

74LS373锁存器功能见表9-4。

表9-4 74LS373功能表

$\overline{OE}$	G	D	Q	
0	1	1	1	
0	1	0	0	
0	0	×	不变	
1	×	×	高阻态	

AT89S52单片机与74LS373锁存器的连接如图9-8所示。

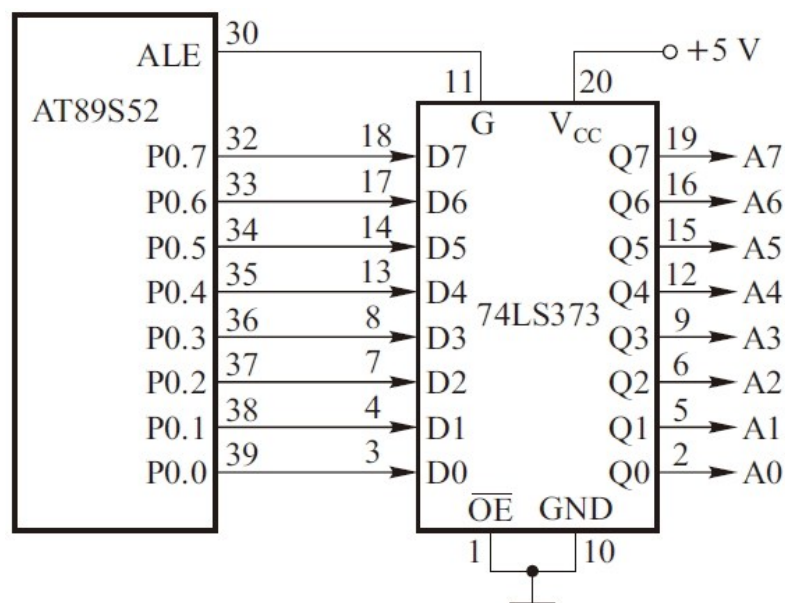


图9-8 AT89S51单片机P0口与74LS373的连接

2. 锁存器74LS573

带三态门的8D锁存器，功能及内部结构与74LS373完全一样，只是其引脚排列与74LS373不同，图9-9为74LS573引脚图。

由图9-9，与74LS373相比，74LS573输入D端和输出的Q端依次排列在芯片两侧，为绘制印制电路板提供较大方便。

74LS573引脚说明如下。

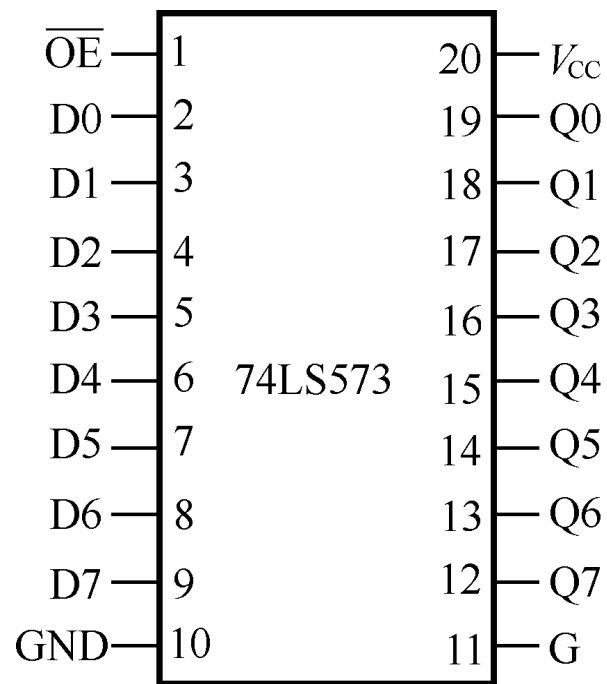


图9-9 锁存器74LS573的引脚

- D7~D0 —8位数据输入线。

- Q7~Q0 —8位数据输出线。

- G —数据输入锁存选通，该引脚与74LS373的G端功能相同。

- OE* —数据输出允许，低电平有效。当低电平时，三态门打开，锁存器中数据输出到数据输出线。当该信号为高电平时，输出线为高阻态。

9.2 外部数据存储器RAM的并行扩展

AT89S52片内有**256B RAM**，如不能满足需要，须扩展外部数据存储器。在单片机系统中，外扩的数据存储器都采用静态数据存储器**SRAM**。

9.2.1 常用的静态RAM（SRAM）芯片

单片机系统中常用**RAM**典型芯片有**6116（2KB）**、**6264（8KB）**、**62128（16KB）**、**62256（32KB）**。

都是单一**+5V**电源供电，双列直插，**6116**为**24引脚**，**6264**、**62128**、**62256**为**28引脚**。**RAM**芯片引脚见图**9-10**。

各引脚功能如下。

- **A0~A14** —地址输入线。
- **D0~D7** —双向三态数据线。
- **CE*** —片选信号输入线，低电平有效。对于**6264**芯片，当**24**脚（**CS**）为高电平且为低电平时才选中该片。
- **OE*** —读选通信号输入线，低电平有效。
- **WE*** —写允许信号输入线，低电平有效。
- **VCC** —工作电源**+5V**。
- **GND** —地

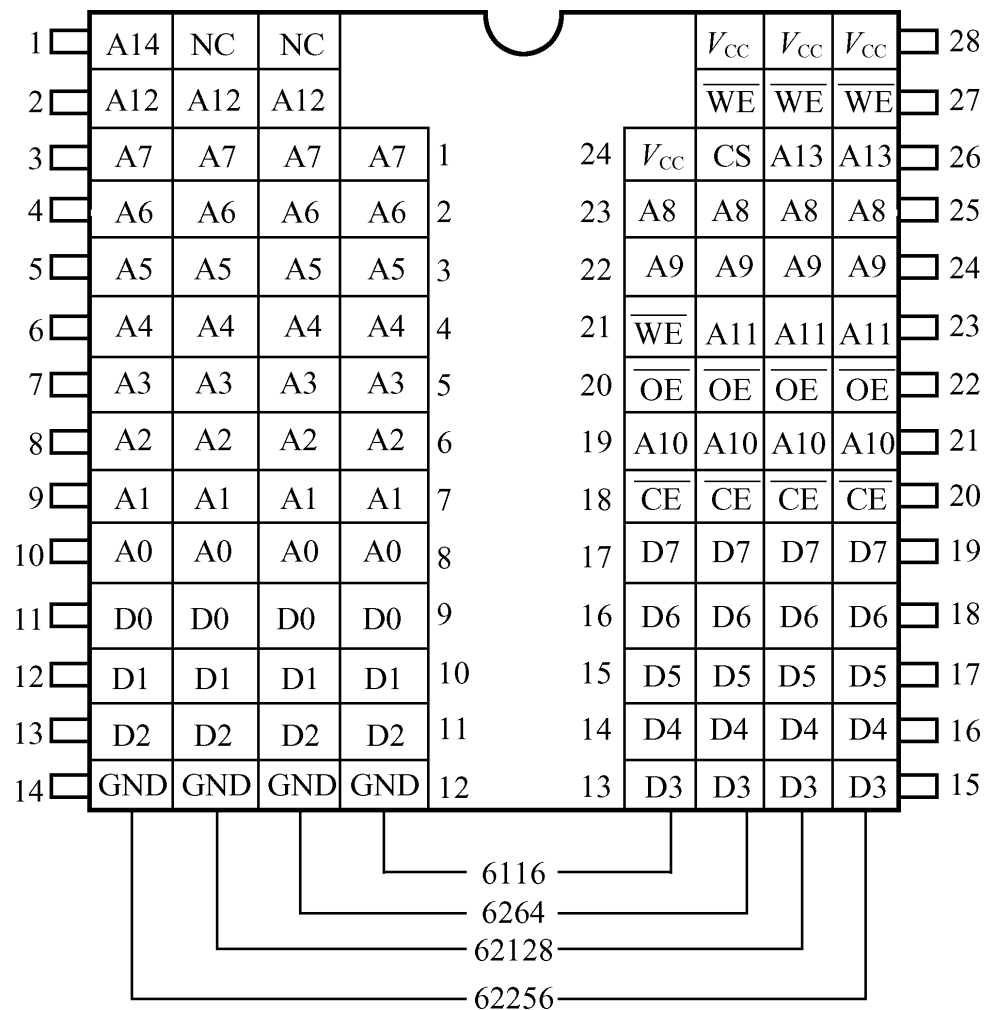


图9-10 常用RAM引脚

RAM存储器有**读出、写入、维持**3种工作方式，见表9-5。

表 9-5 6116、6264、62256 芯片 3 种工作方式的控制

信号 工作方式	$\overline{CE}$	$\overline{OE}$	$\overline{WE}$	D0~D7
读出	0	0	1	数据输出
写入	0	1	0	数据输入
维持*	1	×	×	高阻态

*对于 CMOS 的静态 RAM， $\overline{OE}$ 为高电平，电路处于低功耗状态。此时， V_{cc} 电压可降至 3V 左右，内部所存储信息也不会丢失。

9.2.2 读写片外RAM的操作时序

下面介绍AT89S52单片机对片外RAM的读和写的两种操作时序。

1. 读片外RAM的时序

AT89S52单片机若外扩一片RAM，应将其WR*引脚与RAM芯片的WE*引脚连接，RD*引脚与芯片OE*引脚连接。ALE信号的作用是作为锁存P0口发出的低8位地址的锁存器控制信号。

AT89S52单片机读片外RAM的时序如图9-11所示。

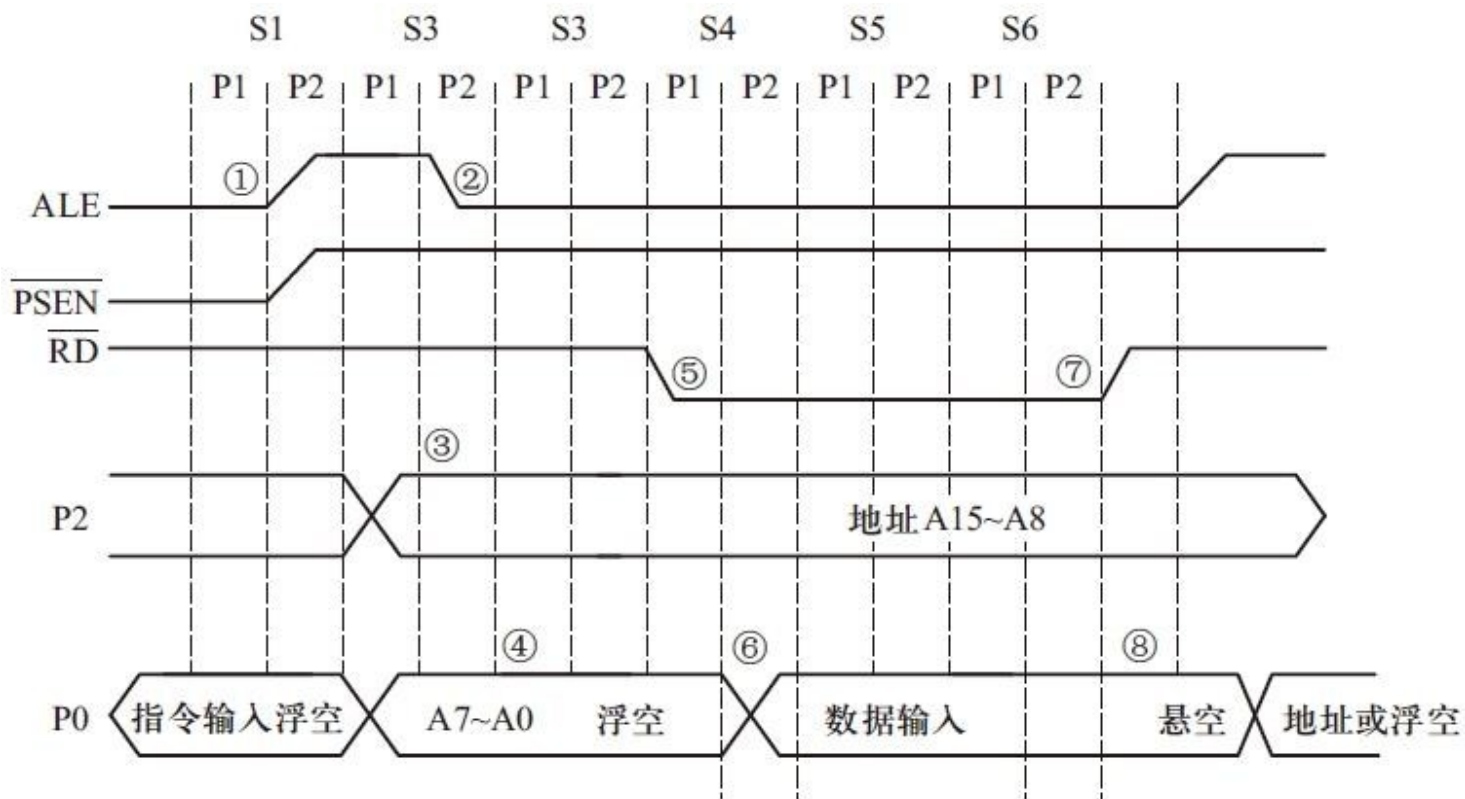


图9-11 AT89S52单片机读片外RAM的操作时序图



在第一个机器周期的**S1**状态，**ALE**信号由低变高（见①处），读**RAM**周期开始。在**S2**状态，**CPU**把低8位地址送到**P0**口总线上，把高8位地址送上**P2**口。**ALE**的下降沿（见②处），把低8位地址信息锁存到外部锁存器**74LS373**内。而高8位地址信息一直锁存在**P2**口锁存器中（见③处）。

在**S3**状态，**P0**口总线变成高阻悬浮状态④。在**S4**状态，执行读指令后使**RD***信号变为有效（见⑤处），**RD***信号使被寻址的片外**RAM**过片刻后把数据送上**P0**口总线（见⑥处），当**RD***回到高电平后（见⑦处），**P0**总线变为悬浮状态（见⑧处）。至此，读片外**RAM**周期结束。

2. 写片外RAM操作时序

当AT89S52单片机执行向片外RAM写指令后，单片机的信号为低电平有效，此信号使RAM的端被选通。

写片外RAM的操作时序如图9-12所示。开始的过程与读过程类似，但写的过程是单片机主动把数据送上P0口总线，故在时序上，单片机先向P0口总线上送完8位地址后，在S3状态就将数据送到P0口总线（见③处）。此间，P0总线上不会出现高阻悬浮现象。

在S4状态，写控制信号WR*有效（见④处），选通片外RAM，稍过片刻，P0口上数据就写到RAM内了，然后写控制信号变为无效（见⑤处）。

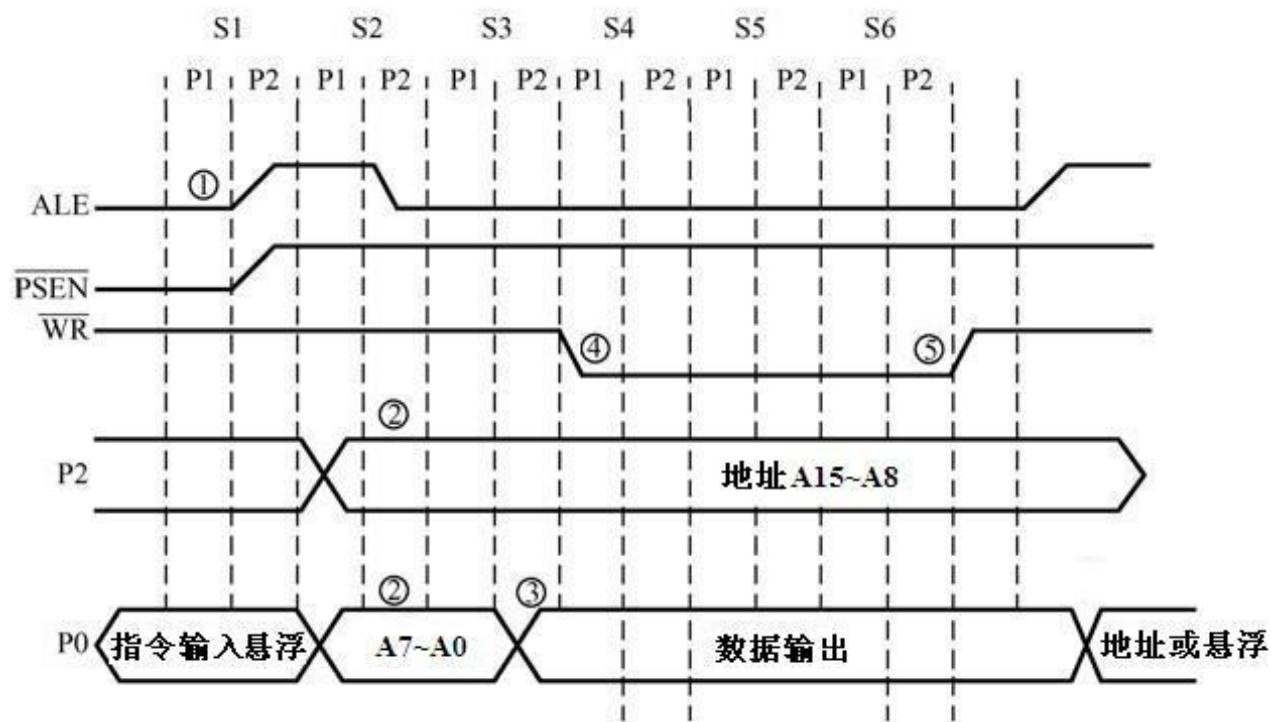


图9-12 AT89S52单片机写片外RAM的操作时序图

9.2.3 并行扩展数据存储器的设计

访问外扩展的数据存储器，要由**P2**口提供高**8**位地址，**P0**口提供低**8**位地址和**8**位双向数据总线。

AT89S52单片机对片外**RAM**的读和写由单片机的**RD*** (**P3.7**) 和**WR*** (**P3.6**) 信号控制，片选端由地址译码器的译码输出控制。因此，进行接口设计时，主要解决地址分配、数据线和控制信号线的连接。如果读/写速度要求较高，还要考虑单片机与**RAM**的读/写速度匹配问题。

图9-13所示为用线选法扩展外部数据存储器的电路。数据存储器选用**6264**，该芯片地址线为**A0~A12**，故**AT89S52**单片机剩余地址线为**3**条。用线选法可扩展**3**片**6264**，**3**片**6264**的存储器空间如**表9-6**所示。

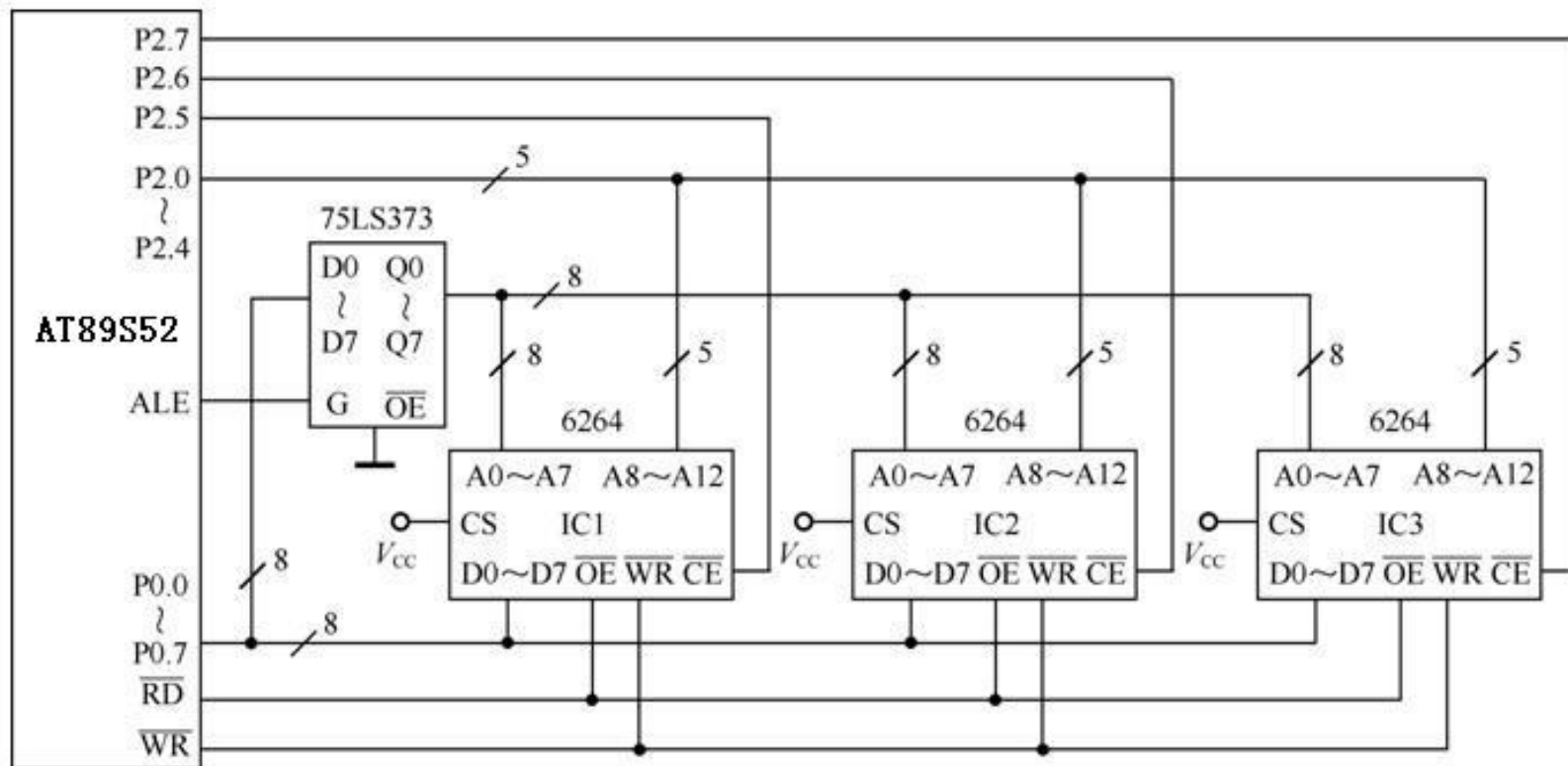


图9-13 线选法扩展外部数据存储器电路图

表 9-6 3 片 6264 芯片对应的存储空间表

P2.7	P2.6	P2.5	选中芯片	地址范围	存储容量
1	1	0	IC1	C000H~DFFFH	8KB
1	0	1	IC2	A000H~BFFFH	8KB
0	1	1	IC3	6000H~7FFFH	8KB

用译码法扩展外部数据存储器的接口电路如图9-14所示。图中数据存储器选用**62128**，该芯片地址线为**A0~A13**，这样，**AT89S51**剩余地址线为两条，采用**2-4**译码器可扩展**4**片**62128**。各**62128**芯片的地址范围如表9-7所示。

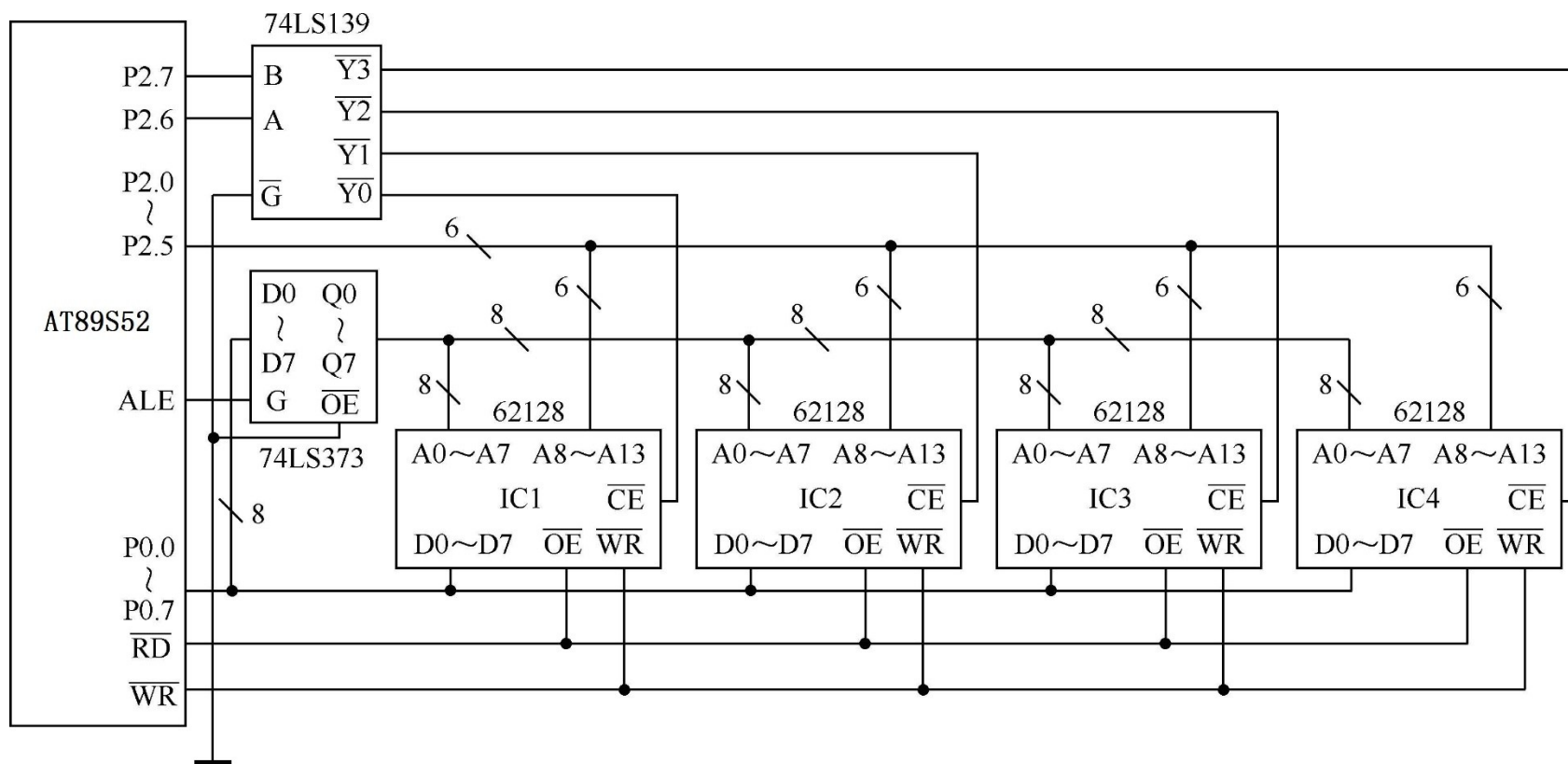


图9-14 译码法扩展外部数据存储器电路图

表9-7 各62128芯片的地址空间分配

2-4 译码器输入 P2.7 P2.6		2-4 译码器 有效输出	选中芯片	地址范围	存储容量
0	0	$\overline{Y0}$	IC1	0000H~3FFFH	16KB
0	1	$\overline{Y1}$	IC2	4000H~7FFFH	16KB
1	0	$\overline{Y2}$	IC3	8000H~BFFFH	16KB
1	1	$\overline{Y3}$	IC4	C000H~FFFFH	16KB

【例9-1】 编写程序将片外数据存储器中的0x5000~0x50FF的256个单元全部清“0”。参考程序：

```
xdata unsigned char databuf[256] _at_ 0x5000;
```

```
void main(void)
```

```
{
```

```
    unsigned char i;
```

```
    for(i=0;i<256;i++)
```

```
    {
```

```
        databuf[i]=0
```

```
    }
```

```
}
```


9.2.4 单片机外扩数据存储器RAM6264的案例设计

单片机外部扩展1片外部数据存储器RAM6264。原理电路如图9-15所示。

单片机先向RAM6264的0x0000地址写入64字节的数据0x01~0x40，写入的数据同时送到P1口通过8个LED显示出来。然后再将这些数据反向复制到0x0080地址开始处，复制操作时，数据也通过P1口的8个LED显示出来。

上述两个操作执行完成后，发光二极管D1被点亮。表示数据第1次的写入起始地址0x0000的64个字节，以及将这64个字节数据反向复制到起始

地址**0x0080**的读写已经完成。

如要查看**RAM6264**中的内容，可在**D1**点亮后，单击“暂停”按钮，然后单击调试（**Debug**）菜单，在下拉菜单中选择“**Memory Contents**”，即可看到图9-16所示的窗口中显示的**RAM6264**中的数据。

可看到单元地址**0x0000~0x003f**中的内容为**0x01~0x40**。而从起始地址**0x0080**开始的**64**个单元中的数据为**0x40~0x01**，可见完成了反向复制。

Memory Contents - U3																	ASCII
0000	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	
0010	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	
0020	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	30	!"#\$%&'()*+,-./0
0030	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F	40	123456789:;<=>?@
0040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0080	40	3F	3E	3D	3C	3B	3A	39	38	37	36	35	34	33	32	31	@?>=<;:987654321
0090	30	2F	2E	2D	2C	2B	2A	29	28	27	26	25	24	23	22	21	0/.,-;+*)('&%\$#"!
00A0	20	1F	1E	1D	1C	1B	1A	19	18	17	16	15	14	13	12	11	
00B0	10	0F	0E	0D	0C	0B	0A	09	08	07	06	05	04	03	02	01	
00C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

图9-16 RAM6264第1次写入的数据与反向复制的数据

参考程序如下：

//先向6264中写入0x00~0x40，然后再将其反向复制到起始地址0x0080的64个单元

```
#include <reg51.h>
```

```
#include <absacc.h>
```

//定义地址所需的头文件

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
sbit LED=P2^7;
```

```
void Delay(uint t)
```

//延时函数

```
{
```

```
    uint i,j,k;
```

```
    for(i=2;i>0;i--)
```

```
        for(j=46;j>0;j--)
```

```
            for(k=t;k>0;k--);
```

```
}
```



```
void main()
```

```
{
```

```
    uint i;
```

```
    uchar temp;
```

```
    LED=1;
```

```
    for(i=0;i<64;i++)           //向6264的0x0000地址开始写入0x01~0x40
```

```
    {
```

```
        XBYTE[i]=i+1;
```

```
        temp=XBYTE[i];
```

```
        P1=~temp           //向P1口送显示数据，控制外部LED的亮灭
```

```
        Delay(200);
```

```
    }
```

```
    for(i=0;i<64;i++) //将6264中的0x40~0x01反向复制到起始地址0x0080处
```

```
    {
```

```
        XBYTE[i+0x0080]=XBYTE[63-i];
```

```
        temp=XBYTE[i+0x0080];           //反向读取6264数据
```

```
        P1=~temp;           //向P1口送显示数据，控制外部的LED的亮灭
```

```
        Delay(200);  
    }  
    LED=0;           //点亮发光二极管D1，表示数据反向复制完成  
    while(1);  
}
```

程序说明：主程序中共有两个**for**循环，第1个**for**循环，完成将数据**0x01~0x40**写入起始地址**0x0000**的**64**个字节，第2个**for**循环，完成将这**64**个字节数据**0x40~0x01**反向复制到起始地址**0x0080**开始的**64**个单元中。

9.3 片内Flash存储器的编程

程序存储器具有非易失性，在电源关断后，存储器仍能保存程序。

AT89S5x系列单片机片内分别集成有不同容量的**Flash ROM**，来作为片内程序存储器使用，具体见表9-1。在片内**Flash ROM**满足要求的情况下，外部程序存储器的扩展工作即可省去。

本节只讨论如何把已调试完毕的程序代码写入到**AT89S52**的片内**Flash**存储器中，即对**Flash**存储器的编程问题。

AT89S52单片机片内**8KB Flash**存储器的基本特性如下：

- (1) 可循环写入/擦除1 000次；
- (2) 存储器数据保存时间为10年；
- (3) 具有3级加密保护。

单片机芯片出厂时，**Flash**存储器处于全部空白状态（各单元均为**FFH**），可直接进行编程。若**Flash**存储器不全为空白状态（即单元中有不是**FFH**），应该首先将芯片擦除（即各个单元均为**FFH**）后，才可向其写入调试通过的程序代码。

AT89S52片内的**Flash**存储器有3个可编程的加密位，定义了3个加密级别，用户只要对3个加密位：**LB1**、**LB2**、**LB3**进行编程即可实现3个不同级别的加密，但还是可以解密。现在还有一种非恢复性加密（**OTP**加密）

方法，就是将**AT89S52**的第**31**脚（**EA***脚）烧断或某些数据线烧断，经过上述处理后的芯片仍然正常工作，但不再具有读取、擦除、重复烧写等功能。这是一种较强的加密手段。国内某些厂家生产的编程器直接具有此功能。

目前对片内**Flash**存储器的编程有两种常用方法：**一种**是**PC**机通过**ISP**下载线进行在线编程；**另一种**是使用通用编程器编程。

9.3.1 使用下载线的**ISP**编程

AT89S5x系列单片机支持对片内**Flash**存储器在线编程（**ISP**），即**PC**机直接通过**USB**口下载线向单片机片内**Flash**存储器写入程序代码。编程完毕的片内**Flash**存储器也可用**ISP**方式擦除或再编程。

ISP下载线可自行制作，也可在电子市场购买。

ISP下载线与单片机一端的连接端口通常采用**ATMEL**公司提供的接口标准，即**10**引脚的**IDC**端口。**图9-17**为**IDC**端口以及**ISP**下载线。

采用**ISP**下载程序时，用户目标板上必须装有上述**IDC**端口，端口中的信号线必须与目标板上**AT89S52**的对应引脚连接。注意，图中的**8脚P1.4（SS*）**端只是对**AT89LP**系列单片机有效，对**AT89S5x**系列单片机无效，不用连接即可。

使用**ISP**下载编程时，运行所带的驱动软件，按照使用说明书进行操作即可。

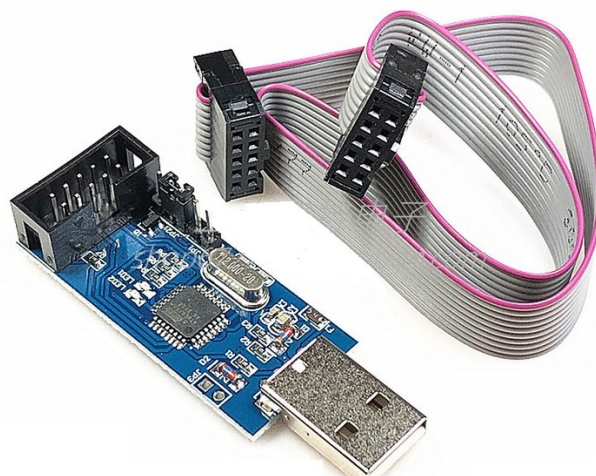
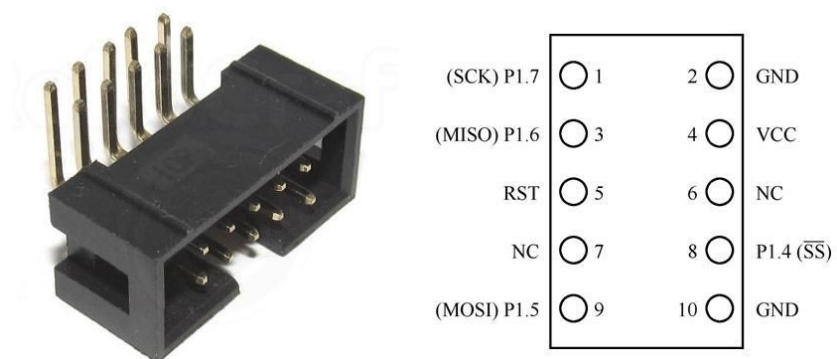


图9-17 IDC端口定义以及ISP下载线

9.3.2 使用通用编程器的程序写入

通用编程器一般通过**USB**口与**PC**机相连，并配有相应的驱动软件。在编程器与**PC**机连接后，在**PC**机上运行驱动软件，首先选择所要编程的单片机型号，再调入调试完毕的程序代码文件，执行写入命令，编程器就将调试通过的程序代码烧录到单片机片内的**Flash**存储器中。开发者只需在电子市场购买一台通用编程器即可完成上述工作。

编程器通过**USB**口与**PC**机通讯，可进行芯片型号自动判别，编程过程中的擦除、烧写、校验等各种操作。编程器供电部分由**USB**端口的**5V**电源提供，省去笨重的外接电源。



编程器的驱动软件界面友好，菜单、工具栏、快捷键齐全，具有编程、读取、校验、空检查、擦除、**Flash**存储器加密等功能。

就单片机的发展方向而言，目前已经趋向于**ISP**程序下载方式，一方面由于原有不支持**ISP**下载芯片逐渐被淘汰（部分已经停产，例如**AT89C51**），另一方面**ISP**使用起来十分方便，不需要编程器就可实现程序的下载，所以**ISP**下载方式已经逐步成为主流。

另外有些厂家的单片机机型不支持**ISP**下载方式，所以有时还是要用到编程器进行程序下载的。目前，电子市场上的编程器型号较多，只要根据自己的需求进行选择即可，这里不再赘述

9.4 E²PROM的并行扩展

在以单片机为核心的智能仪器仪表、工业监控等应用系统中，对某些状态参数数据，不仅要求能够在线修改保存，而且断电后能保持。断电数据的保护可采用电可擦除写入的存储器**E²PROM**，其突出优点是能够在线擦除和改写。

E²PROM与**Flash**存储器都可在线擦除与改写，它们之间的区别在于**Flash**存储器结构简单，同样的存储容量占芯片面积较小，成本自然比**E²PROM**低，且大数据量下的操作速度更快，但缺点是擦除改写都是按扇区进行的，操作过程麻烦，特别是在小数据量反复改写时。所以单片机中**Flash**存储器的结构更适合作为不需频繁改写的程序存储器。而传统结构的**E²PROM**，操作简单，可字节写入，非常适合用作运行过程中

频繁改写某些非易失的小数据量的存储器。

E²PROM有并行和串行之分，并行**E²PROM**的速度比串行的快，容量大。例如并行的**E²PROM 2864A**的容量为 $8k \times 8$ 位。而串行**I²C**接口的**E²PROM**与单片机的接口简单，连线少，比较流行的**ATMEL**公司的串行**E²PROM**芯片**AT24C02/AT24C08/AT24C16**。串行**E²PROM**的扩展将在下一章介绍。本节只介绍**AT89S52**单片机扩展并行**E²PROM**芯片**2864**的设计。

9.4.1 并行**E²PROM**芯片简介

常见的并行**E²PROM**芯片有**2816/2816A**，**2817/2817A**，**2864A**等。这些芯片的引脚如**图9-18**所示。

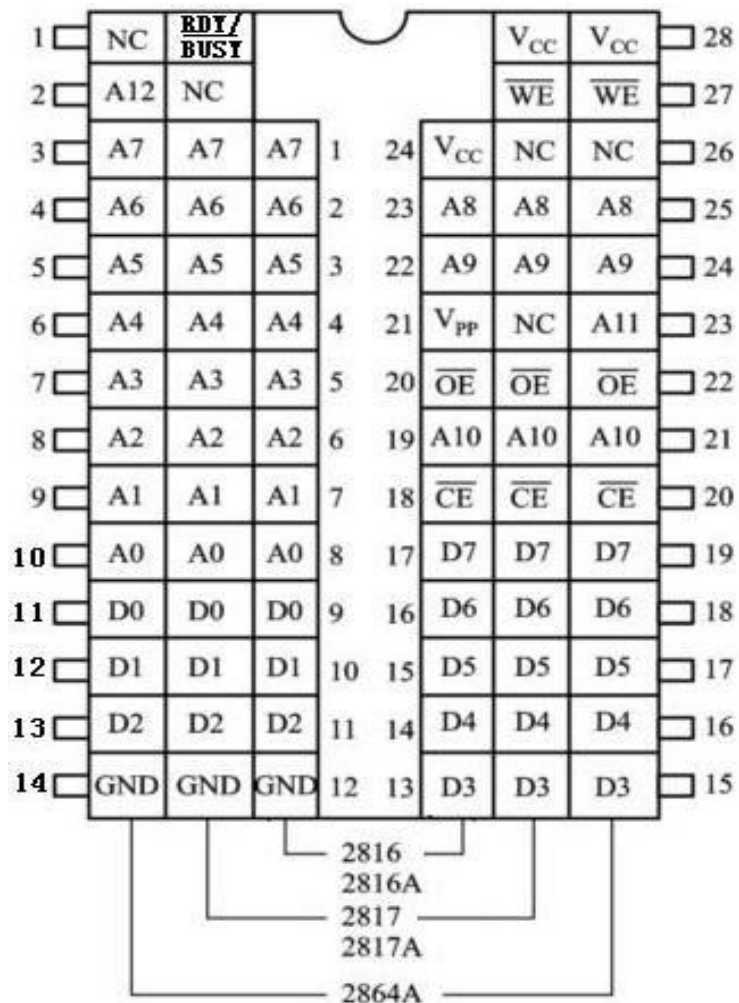


图9-18 常见的并行E2PROM引脚图

9.4.2 AT89S52单片机扩展E2PROM AT2864的设计

2864A与AT89S52单片机的接口电路如图9-19所示。2864A的存储容量为8K字节，与同容量的静态RAM 6264的引脚是兼容的，2864A的片选端由高位地址线P2.7（A15）来控制。

单片机对2864A的读写非常方便，在单一+5V电压下写入新数据即覆盖了旧的数据，类似于对RAM的读写操作，2864A典型的读出数据时间为200~350ns，但是字节编程写入时间为10ms~15ms，要比对RAM写入时间长许多。

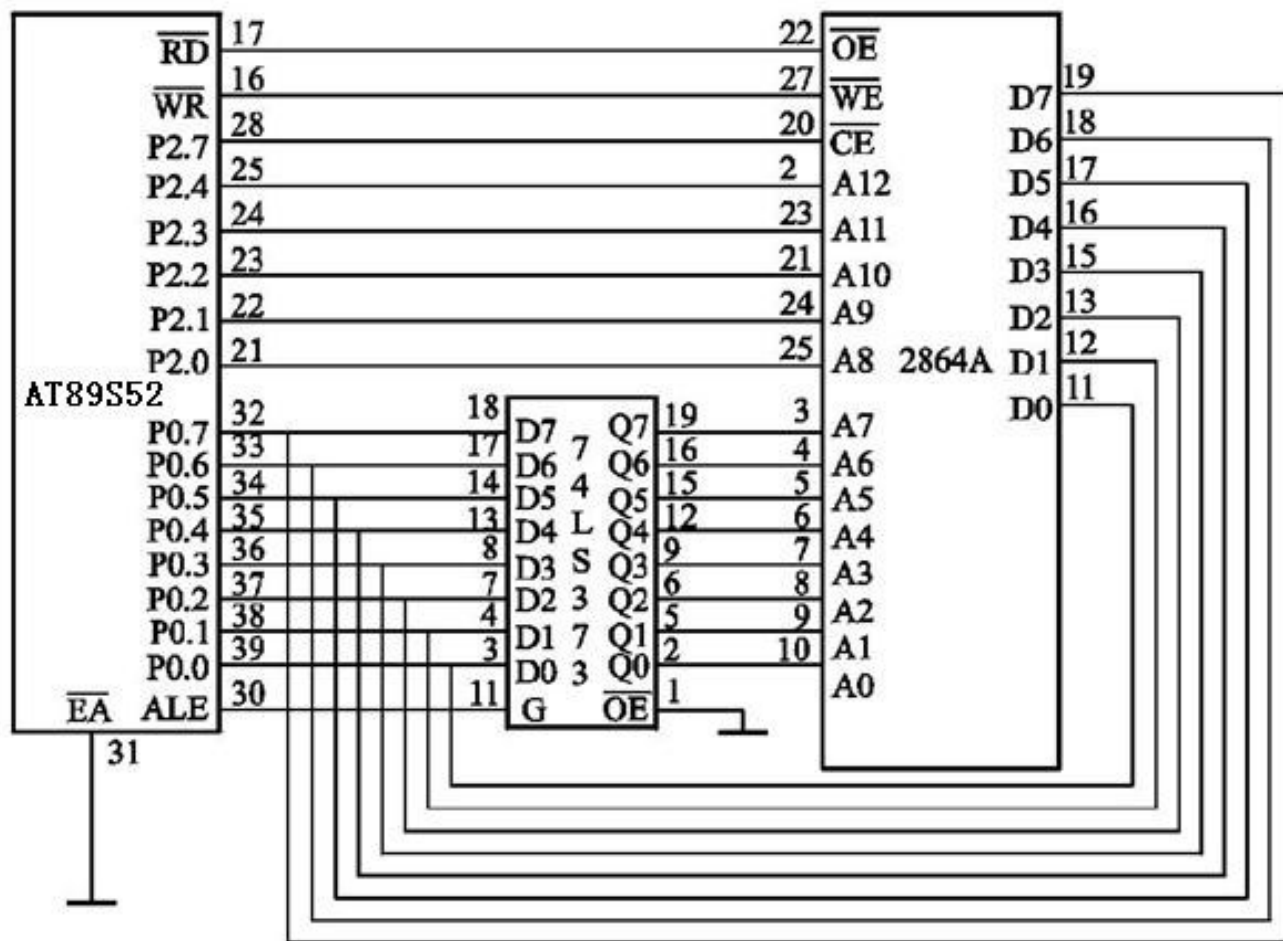


图9-19 2864A与AT89S52单片机的接口电路

9.5 AT89S52扩展并行I/O芯片82C55的设计

AT89S52本身有4个通用的并行I/O口P0~P3，但真正用作通用I/O口线的只有P1口和P3口某些位线。当AT89S52本身4个并行I/O口不够用时，需进行外部I/O接口扩展。本节介绍AT89S52单片机扩展可编程并行I/O接口芯片82C55的设计。此外还介绍使用廉价的74LSTTL芯片扩展并行I/O接口以及使用AT89S52串行口来扩展并行I/O口的设计。

9.5.1 I/O接口扩展概述

由本章前面介绍可知，系统扩展除扩展存储器外，还包括扩展I/O接口。

1. I/O接口的基本功能要求

I/O接口作为单片机与外设交换信息的桥梁，应满足如下功能要求。

(1) 实现和不同外设的速度匹配

多数外设速度慢，无法和 μs 级单片机比。单片机只有在确认外设已为数据传送做好准备前提下才进行数据传送。要知道外设是否准备好，就需I/O接口电路与外设间传送状态信息，以实现单片机与外设间的速度匹配。

(2) 输出数据锁存

与外设相比，单片机工作速度快，送出数据在总线上保留时间十分短暂，无法满足慢速外设的数据接收。所以在扩展的I/O接口电路中应有输出数据锁存器，以保证单片机输出数据能为慢速接收设备接收。

(3) 输入数据三态缓冲

外设向单片机输入数据时，要经过数据总线，但数据总线上可能“挂”有多个数据源。为使传送数据时不发生冲突，只允许当前时刻正在接收数据的I/O接口使用数据总线，其余I/O接口应处于隔离状态，为此要求I/O接口电路能为输入数据提供三态输入缓冲功能。

2. I/O端口的编址

介绍I/O端口编址之前，首先弄清I/O接口（Interface）和I/O端口（Port）的概念。I/O接口是单片机与外设间连接电路总称。I/O端口（简称I/O口）是指I/O接口电路中具有单元地址的寄存器或缓冲器。

一个I/O接口芯片可以有多个I/O端口，传送数据端口称数据口，传送命令端口称命令口，传送状态端口称状态口。当然，并不是所有外设都需这3种齐全的I/O端口。

每个I/O接口中端口都要有地址，以便AT89S52进行端口访问，来和外设交换信息。常用I/O端口编址两种方式，独立编址方式和统一编址方式。

(1) 独立编址

独立编址方式就是I/O端口地址空间和存储器地址空间分开编址。优点是两个地址空间相互独立，界限分明。但需要设置一套专门的读写I/O端口的指令和控制信号。

(2) 统一编址

把I/O端口与数据存储器单元同等对待，即接口芯片中一个端口就相当于一个RAM单元。AT89S52使用的就是I/O端口和外部数据存储器RAM统一编址方式，因此AT89S52外部数据存储器空间也包括I/O端口在内。统一编址方式优点是**不需要专门的I/O指令**，直接使用访问数据存储器指令进行I/O读写操作，简单、方便。但需把外部数据存储器所占的单元地址与I/O端口所占地址划分清楚，避免发生数据冲突。

3. I/O数据的传送方式

为了实现和不同外设速度匹配，I/O接口须根据不同外设选择恰当I/O数据传送方式。I/O数据传送方式有：同步传送、异步传送和中断传送。

(1) 同步传送

又称无条件传送。当外设速度和单片机速度相比拟时，常采用本方式，最典型的同步传送就是单片机和外部数据存储器间的数据传送。

(2) 异步传送

实质就是查询传送。单片机通过查询外设“准备好”后，再进行数据传送。**优点**是通用性好，硬件连线和查询程序十分简单，但由于程序在运行中经常查询外设是否“准备好”，因此**工作效率不高**。

(3) 中断传送

为提高单片机对外设的工作效率，常采用中断传送方式，即利用单片机本身的中断功能和I/O接口芯片的中断功能来实现数据传送。单片机只有在



外设准备好后，才中断主程序执行，从而执行与外设进行数据传送的中断服务子程序。中断服务完成后又返回主程序断点处继续执行。中断方式可大大提高单片机的**工作效率**。

常用的可编程通用并行I/O接口芯片为**82C55**（3个8位I/O口）。它可以与**AT89S52**单片机直接连接，接口逻辑十分简单。下面介绍**AT89S52**扩展**82C55**的设计。

9.5.2 并行I/O芯片82C55简介

本小节首先简要介绍可编程并行I/O接口芯片82C55的应用特性，然后介绍AT89S51单片机与82C55的接口电路设计以及软件设计。

1. 82C55引脚与内部结构

82C55是Intel公司生产的可编程并行I/O接口芯片，它具有3个8位并行I/O口，3种工作方式，可编程，因而使用灵活方便，可作为单片机与多种外设连接时的中间接口电路。82C55引脚及内部结构见图9-20和图9-21。

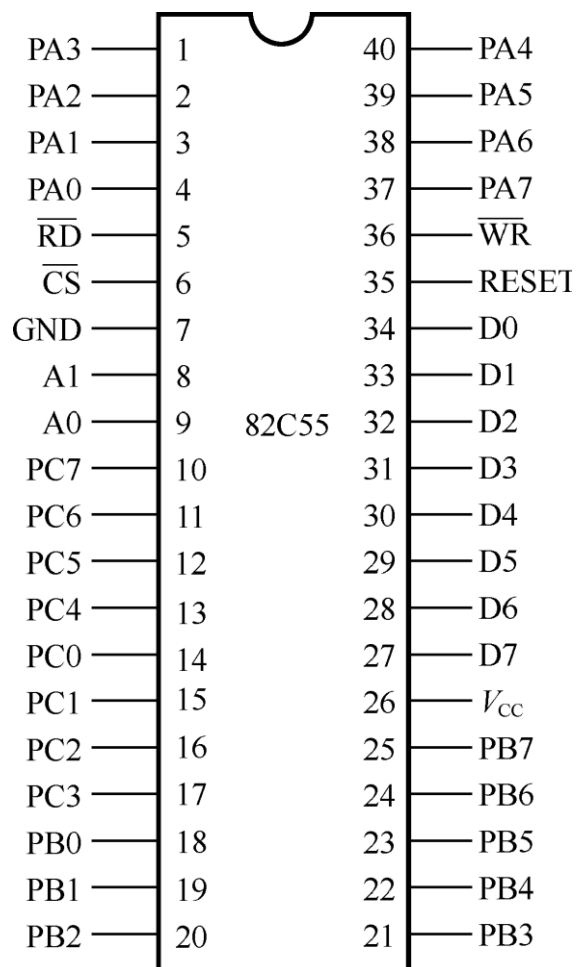


图9-20 82C55的引脚图

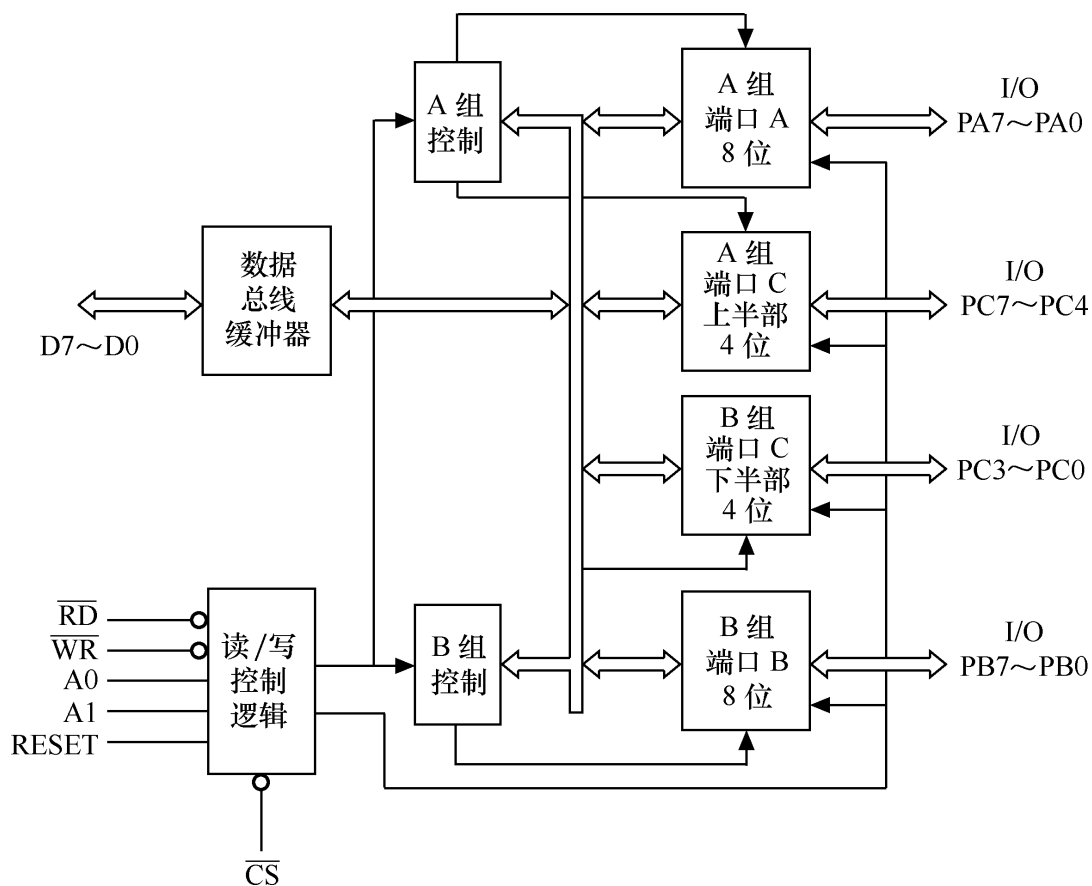


图9-21 82C55的内部结构

(1) 引脚说明

双列直插封装， 40只引脚， 功能如下。

- **D7~D0**—三态双向数据线， 与单片机的**P0**口连接， 用来与单片机之间传送数据信息。
- **CS***—片选信号线， 低电平有效， 表示本芯片被选中。
- **RD***—读信号线， 低有效， 用来读出**82C55**端口数据的控制信号。
- **WR***—写信号线， 低有效， 用来向**82C55**写入端口数据的控制信号。
- **Vcc** — +5V电源。
- **PA7~PA0**— 端口**A**输入/输出线。
- **PB7~PB0**— 端口**B**输入/输出线。
- **PC7~PC0**— 端口**C**输入/输出线。

- **A1、A0**—地址线，选择82C55内部4个端口。
- **RESET**—复位引脚，高有效。

(2) 内部结构

82C55内部结构见图9-21。左侧引脚与单片机连接，右侧引脚与外设连接。各部件功能如下。

① 端口PA、PB、PC

3个8位并行口PA、PB和PC，它们都可选为输入/输出工作模式，但功能和结构上有些差异。

- **PA口**：1个8位数据输出锁存器和缓冲器；1个8位数据输入锁存器。
- **PB口**：1个8位数据输出锁存器和缓冲器；1个8位数据输入缓冲器。
- **PC口**：1个8位的输出锁存器；1个8位数据输入缓冲器。

通常**PA口**、**PB口**作为输入/输出口，**PC口**既可作为输入/输出口，也可在软件控制下，分为两个4位端口，作为端口**PA**、**PB**选通方式操作时的状态控制信号。

② A组和B组控制电路

这是两组根据**AT89S51**写入的“命令字”控制**82C55**工作方式的控制电路。**A**组控制**PA**口和**PC**口的上半部（**PC7~PC4**）；**B**组控制**PB**口和**PC**口的下半部（**PC3~PC0**），并可使用“命令字”来对端口**PC**的每一位实现按位置“1”或清“0”。

③ 数据总线缓冲器

数据总线缓冲器是一个三态双向**8**位缓冲器，作为**82C55**与系统总线之间的接口，用来传送数据、指令、控制命令以及外部状态信息。

④ 读/写控制逻辑电路

读/写控制逻辑电路接收**AT89S52**单片机发来的控制信号**RD***、**WR***、**RESET**、地址信号**A1**、**A0**。**A1**、**A0**共4种组合**00**、**01**、**10**、**11**，分别选择**PA**、**PB**、**PC**及控制寄存器的端口地址。根据控制信号不同组合，端口数据被**AT89S52**读出，或者将**AT89S51**送来的数据写入端口。

各端口工作状态与地址信号**A1**、**A0**及控制信号关系见表**9-8**。

表9-8 82C55端口工作状态选择

A1	A0	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	工 作 状 态
0	0	0	1	0	PA 口数据→数据总线 (读端口 A)
0	1	0	1	0	PB 口数据→数据总线 (读端口 B)
1	0	0	1	0	PC 口数据→数据总线 (读端口 C)
0	0	1	0	0	总线数据→PA 口 (写端口 A)
0	1	1	0	0	总线数据→PB 口 (写端口 B)
1	0	1	0	0	总线数据→PC 口 (写端口 C)
1	1	1	0	0	总线数据→控制寄存器 (写控制字)
×	×	×	×	1	数据总线为三态
1	1	0	1	0	非法状态
×	×	1	1	0	数据总线为三态

2. 工作方式选择控制字及端口PC置位/复位控制字

AT89S52可向82C55控制寄存器写入**两种不同控制字**：工作方式选择控制字及端口PC置位/复位控制字。首先来介绍工作方式选择控制字。

(1) 工作方式选择控制字

82C55有3种工作方式：

- **方式0**——基本输入/输出；
- **方式1**——应答输入/输出；
- **方式2**——双向传送（仅PA口有此工作方式）。

3种工作方式由写入控制寄存器的方式控制字决定。方式控制字格式见**图9-22**。最高位**D7=1**，为本控制字标志，以便与端口PC口置位/复位控制字

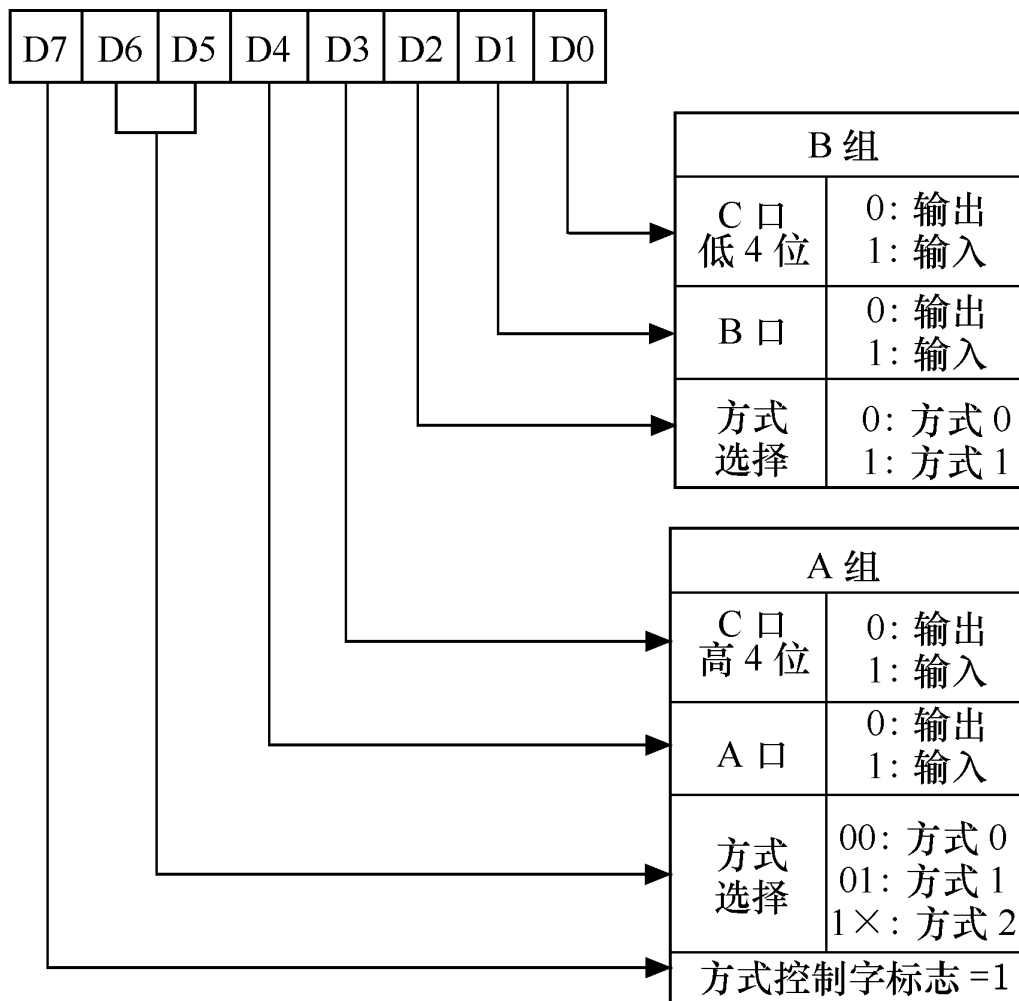


图9-22 82C55的方式控制字格式

相区别（端口**PC**置位/复位控制字最高位**D7 = 0**）。

3个端口中**PC**口被分为两个部分，上半部分随**PA**口称为A组，下半部分随**PB**口称为B组。其中**PA**口可工作于**方式0、1和2**，而**PB**口只能工作在**方式0和1**。

【例9-3】 AT89S52单片机向82C55控制寄存器（端口地址为0xff7f）写入工作方式控制字0x95，根据图9-22，可将82C55编程设置为：**PA**口方式0输入，**PB**口方式1输出，**PC**口的上半部分（**PC7~PC4**）输出，**PC**口的下半部分（**PC3~PC0**）输入。

参考程序：

```
#include <absacc.h>
#define COM8255 XBYTE[0xff7f] //0xff7f为82C55的控制寄存器地址
#define uchar unsigned char
...
void init8255(void)
{
    COM8255=0x95;           //方式选择控制字写入82C55控制寄存器
    ... ..
}
```

(2) PC口按位置位/复位控制字

为82C55的另一个控制字，即PC口8位中任一位，可用一个写入82C55控制口的置位/复位控制字对PC口按位置“1”或清“0”。该功能

主要用于位控。**PC**口按位置位/复位控制字见图9-23。

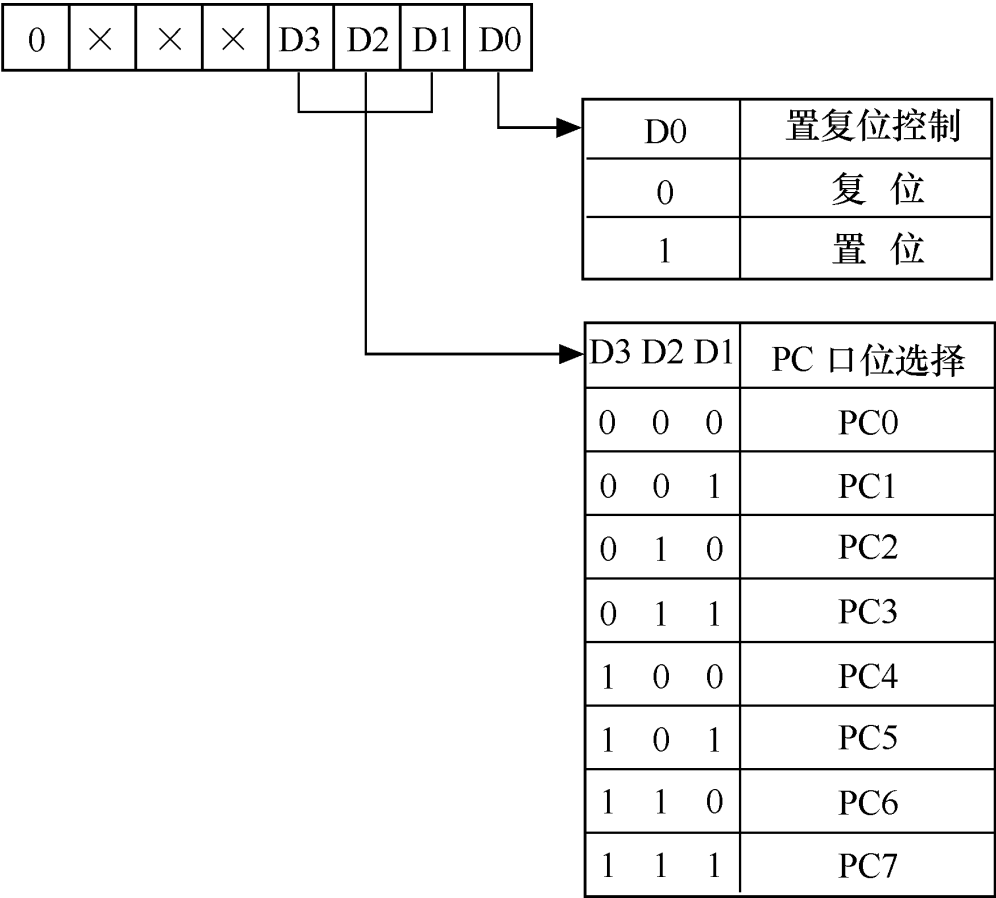


图9-23 PC口按位置位/复位控制字格式

【例9-3】 AT89S52单片机向82C55的控制字寄存器写入工作方式控制字07H，则PC3置1；08H写入控制口，则PC4清0。假设82C55的端口寄存器的地址为0xff7f。

参考程序段如下：

```
#include <absacc.h>
```

```
#define COM8255 XBYTE[0xff7f]           //0xff7f为82C55控制端口地址
```

```
.....
```

```
void init8255(void)
```

```
{
```

```
    COM8255=0x07;           //置位/复位控制字写入控制端口，PC3=1
```

```
    COM8255=0x08;           //置位/复位控制字写入控制端口，PC4=0
```

```
.....
```

```
}
```


9.5.3 82C55的3种工作方式

1. 方式0

基本输入/输出方式。方式0下，单片机可对82C55进行I/O数据无条件传送。外设I/O数据可在82C55各端口得到锁存和缓冲，因此，方式0为**基本输入/输出方式**。

方式0下，3个端口都可以由软件设置为输入或输出，不需要应答联络信号。**方式0基本功能**如下：

- (1) 具有两个8位端口（PA、PB）和两个4位端口（PC的上半部分和下半部分）；
- (2) 任何端口都可设定为输入或输出，各端口输入、输出共有16种组合。

82C55的PA口、PB口和PC口均可设定为方式0，并可根据需要，向控制寄存器写入工作方式控制字（见图9-22），来规定各端口为输入或输出方式。

【例9-5】假设82C55的控制字寄存器端口地址为0xff7f，则令PA口和PC口的高4位工作在方式0输出，PB口和PC口的低4位工作于方式0输入，初始化程序如下：

```
uchar xdata COM8255_at_0xff7f      //0xff7f为82C55控制寄存器地址
...
void init8255(void)
{
    COM8255=0x83;                    //工作方式选择控制字写入控制寄存器
    ...
}
```

2. 方式1

应答联络的输入/输出工作方式。**PA**口和**PB**口皆可独立设置该工作方式。方式1下，**82C55**的**PA**口和**PB**口通常用于**I/O**数据传送，**PC**口用作**PA**口和**PB**口的应答联络信号线，以实现采用中断方式来传送**I/O**数据。

PC口中某些线作为应答联络线已事先规定，其各位分配见**图9-24**和**图9-26**，图中，标有**I/O**各位仍作基本输入/输出，不作应答联络用。

下面介绍方式1输入/输出时应答联络信号与工作原理。

(1) 方式1输入

方式1输入各应答联络信号见图9-24。其中**STB***与**IBF**为一对应答联络信号。各应答联络信号功能如下。

- **STB*** —输入外设发给**82C55**的选通输入信号。
- **IBF** —输入缓冲器满，应答信号。**82C55**通知外设已收到外设发来的数据。
- **INTR**—**82C55**向单片机发出的中断请求信号。
- **INTEA**—**PA**口是否允许中断的控制信号，由**PC4**的置/复位控制。
- **INTEB** —**PB**口是否允许中断的控制信号，由**PC2**置/复位来控制。

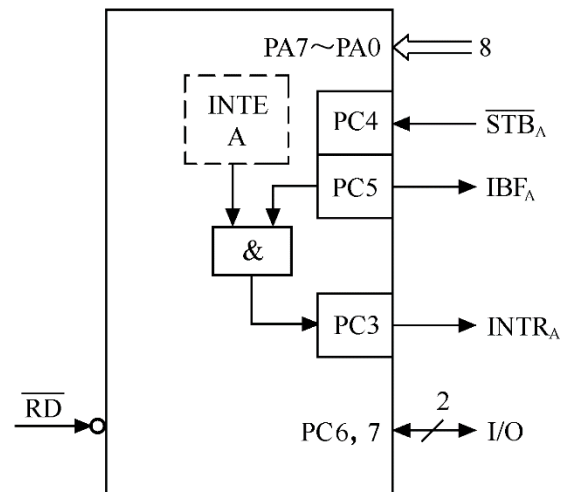
A 组方式控制字

D7	D6	D5	D4	D3	D2	D1	D0
1	0	1	1	I/O	×	×	×

A 口方式 1

A 口输出
PC6, 7
1: 输入
0: 输出

方式 1 输入 (A 口)



B 组方式控制字

D7	D6	D5	D4	D3	D2	D1	D0
1	×	×	×	×	1	1	×

B 口输入

B 口方式 1

方式 1 输入 (B 口)

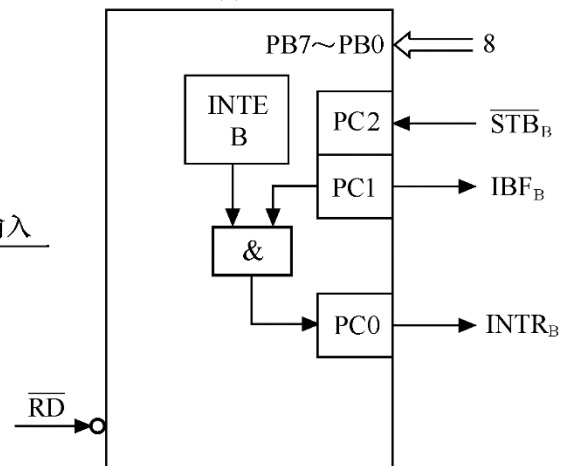


图9-24 方式1输入应答联络信号

方式1输入见**图9-25**。以**PA口方式1**输入为例，介绍工作过程。

① 当外设向**82C55**输入一个数据并送到**PA7~PA0**上时，外设自动在选通输入线**STBA***上向**82C55**发送一个低电平选通信号。

② **82C55**收到选通信号**STBA***后，首先把**PA7~PA0**上输入的数据存入**PA口**的输入数据缓冲/锁存器，然后使输出应答线**IBFA**变为高电平，以通知输入外设，**82C55**的**PA口**已收到它送来的输入数据。

③ **82C55**检测到**STBA***由低电平变为高电平，**IBFA (PC5)**为“1”状态和中断允许**INTEA (PC4) = 1**时，使**INTRA (PC3)**变为高电平，向单片机发出中断请求**INTRA**。**INTEA**状态可由**PC4**置/复位控制字来控制。

④ 单片机响应中断后，进入中断服务子程序读取**PA**口的外设发来的输入数据。当输入数据被单片机读走后，**82C55**撤销**INTRA**上的中断请求，并使**IBFA**变为低电平，以通知输入外设可以传送下一个输入数据。

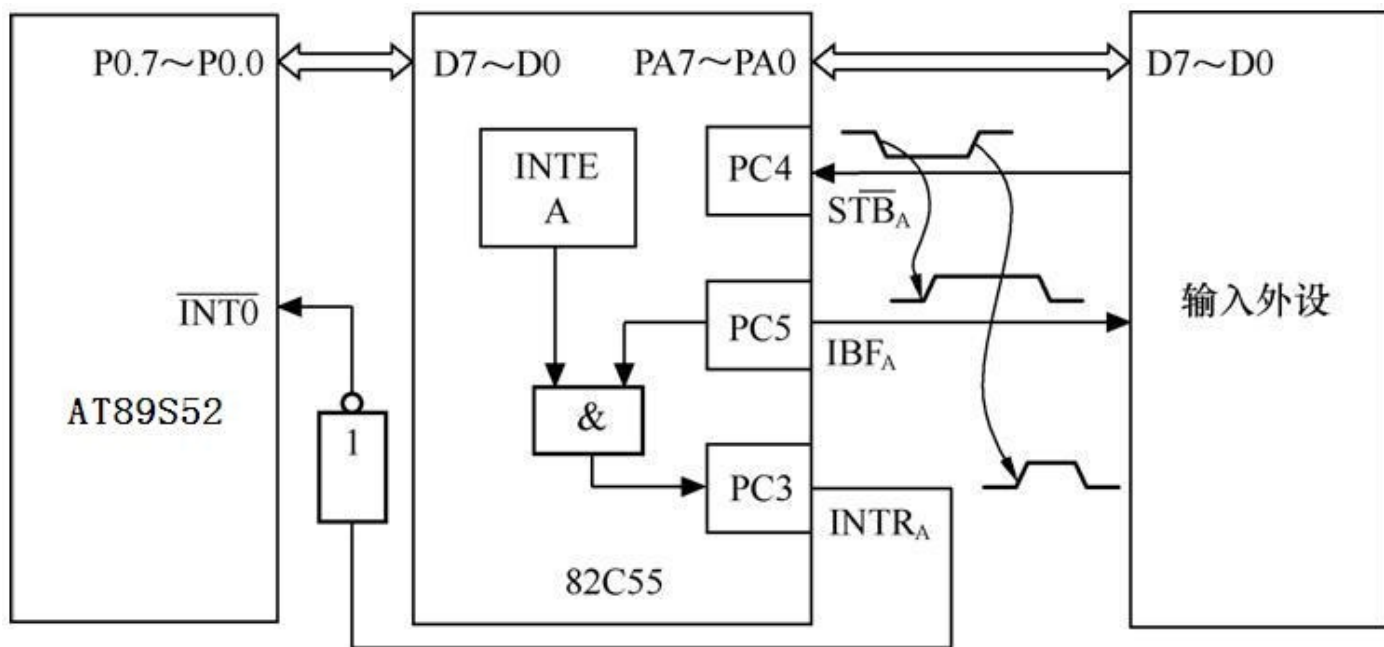


图9-25 PA口方式1输入工作过程示意图

(2) 方式1输出

方式1输出时的应答联络信号如图9-26所示。

OBF*与 **ACK***构成了一对应答联络信号，功能如下。

- **OBF*** —端口输出缓冲器满信号，是82C55发给外设的联络信号，表示单片机已经把数据输出到82C55的指定端口，外设可将数据取走。
- **ACK*** —外设的应答信号。表示外设已把82C55端口的数据取走。
- **INTR** —中断请求信号。表示该数据已被外设取走，向单片机发出中断请求，如果单片机响应该中断，则在中断服务子程序中向82C55端口输出下一个数据。
- **INTE_A** —PA口是否允许中断的控制信号，由PC6置/复位来控制。
- **INTE_B** —PB口是否允许中断的控制信号，由PC2置/复位来控制。

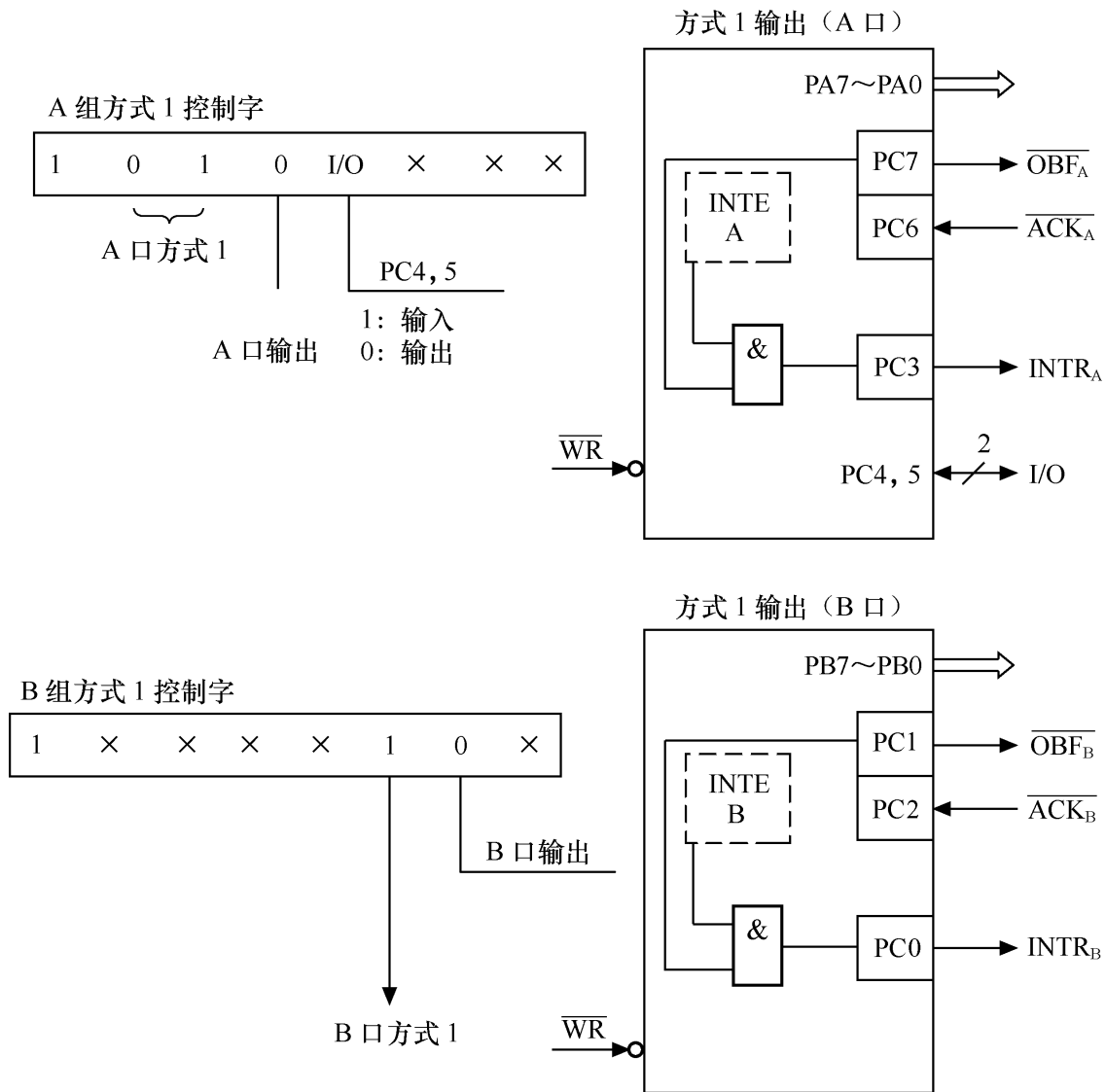


图9-26 方式1输出应答联络信号

方式1输出工作示意图图9-27。下面以PB口方式1输出为例，介绍方式1输出的工作过程。

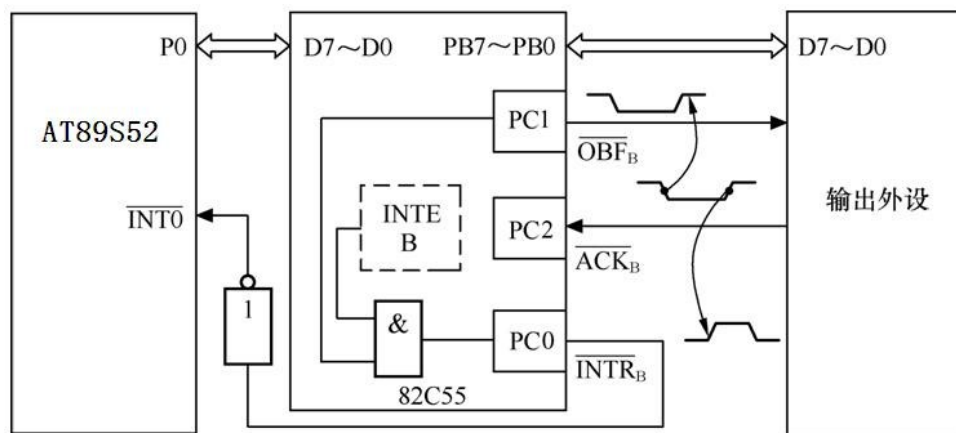


图9-27 PB口方式1输出工作过程示意图

① 单片机可通过传送指令把输出数据送到B口的输出数据锁存器，82C55收到数据后便令输出缓冲器满引脚 $\overline{OBF}_B^*$ (PC1) 变为低电平，以通知输出外设单片机输出的数据已在PB口的PB7~PB0上。

② 输出外设收到 $\overline{OBF}_B^*$ 上低电平后，先从PB7~PB0上取走输出数据，然后使 $\overline{ACK}_B^*$ 变为低电平，通知82C55输出外设已收到82C55输出给外设的数据。

③ 82C55从应答输入线**ACKB***收到低电平后就对**OBFB***和中断允许控制位**INTEB**状态进行检测，若它们皆为高电平，则**INTRB**变为高电平而向单片机请求中断。

④ 单片机响应**INTRB**上中断请求后，在中断服务程序中把下一个输出数据送到**PB**口的输出数据锁存器。重复上述过程，完成数据输出。

【例9-6】 设置**PA**口为应答方式输入，**PB**口为应答方式输出。假设82C55的端口寄存器的地址为**0xff7f**，参考程序段如下：

```
uchar xdata COM8255 _at_ 0xff7f      //0xff7f为82C55控制寄存器地址
...
void init8_255(void)
{
    COM8255=0xb4;                      //方式控制字写入控制寄存器
    .....
}
```

3. 方式2

只有**PA**口才能设定为方式2，实质上是方式1输入和方式1输出组合。方式2特别适用于像键盘、显示器一类外设，有时需要把键盘上输入的编码信号通过**PA**口送给单片机，有时又需把单片机发出的数据通过**PA**口送给显示器显示。

图9-28所示为方式2下工作示意图。在方式2下，**PA7~PA0**为双向I/O总线。当作为输入端口使用时，**PA7~PA0**受**STB***和**IBF_A**控制，其工作过程和方式1输入相同；当作为输出端口使用时，**PA7~PA0**受**OBF_A**、**ACK_A***控制，其工作过程和方式1输出时相同。

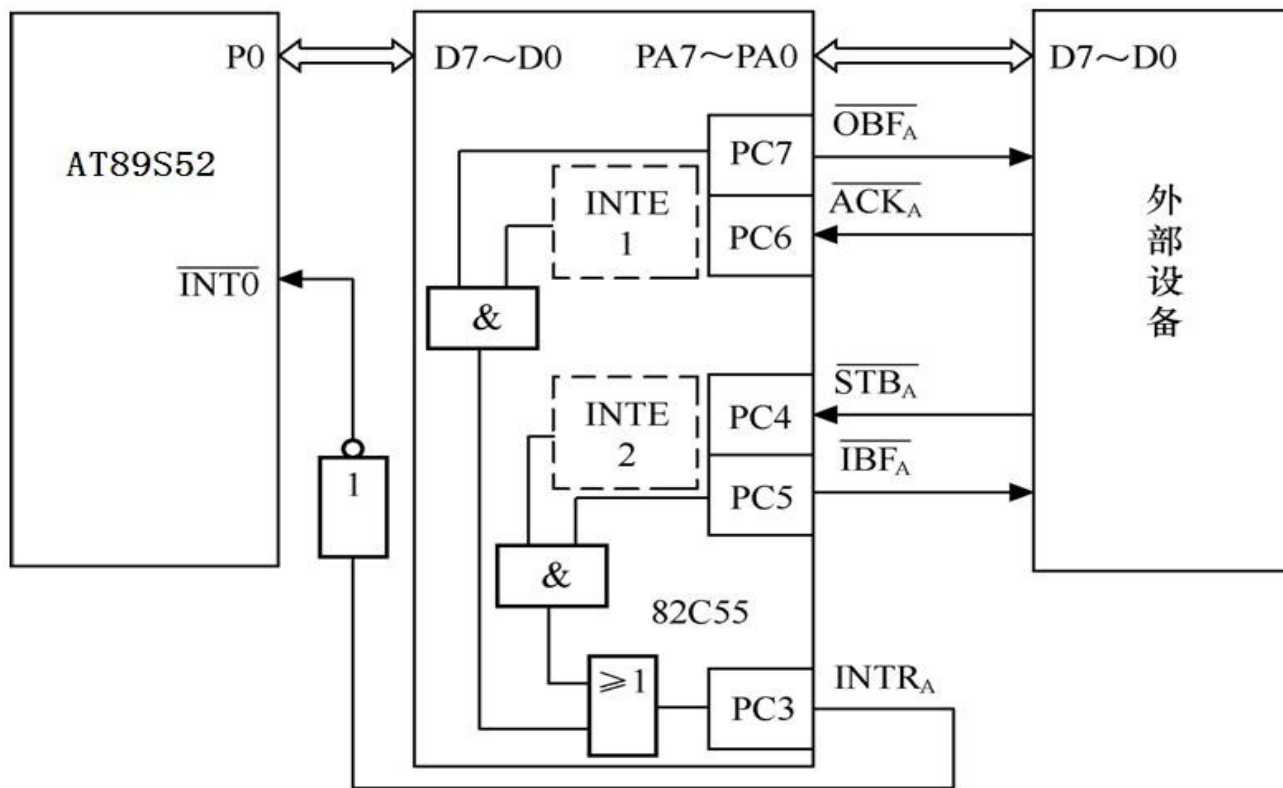


图9-28 PA口方式2工作示意图

9.5.4 AT89S52单片机与82C55的接口设计

1. 硬件接口电路

图9-29为AT89S51扩展一片82C55的电路图。图中，74LS373是地址锁存器，P0.1、P0.0经74LS373与82C55的地址线A1、A0连接；

P0.7经74LS373与片选端 **CS***相连，其他地址线悬空；82C55的控制线 **RD***、**WR***直接与AT89S52单片机的**RD***和**WR***端相连；单片机数据总线P0.0~P0.7与82C55的数据线D0~D7连接。

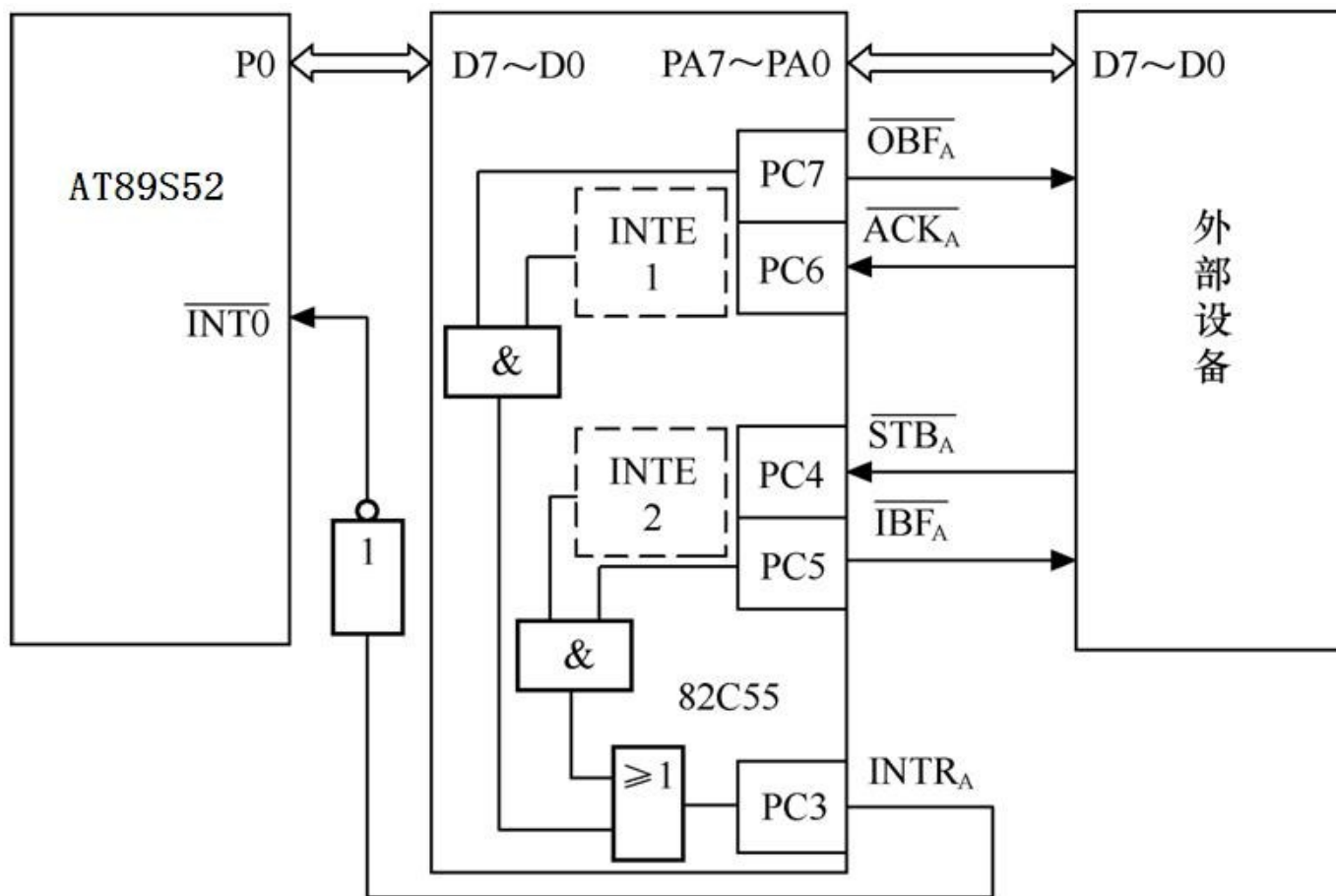


图9-29 AT89S52单片机扩展一片82C55的接口电路

2. 确定82C55端口地址

图9-29中82C55只有3条线与AT89S52单片机的地址线相接，片选端CS*与P0.7相连，端口地址选择端A1、A0分别与P0.1和P0.0连接，其他地址线未用。显然，只要保证P0.7为低电平时，即可选中82C55；若P0.1、P0.0再为“00”，则选中82C55的PA口，同理P0.1、P0.0为“01”、“10”、“11”分别选中PB口、PC口及控制口。

若端口地址用十六位表示，其他未用端全为“1”，则82C55的PA、PB、PC及控制口地址分别为0xff7c、0xff7dh、0xff7eh以及0xff7fh。

3. 软件编程

在实际应用设计中，须根据外设类型选择**82C55**操作方式，并在初始化程序中把相应控制字写入控制口。下面介绍对**82C55**的编程。

【例9-7】根据**图9-29**，要求**82C55**的**PC**口工作在方式0，并从**PC5**脚输出连续的方波信号，频率为**500Hz**，参考程序：

```
#include <reg51.h>
#include <absacc.h>
#define PA8255 XBYTE[0xff7c]           //0xff7c为82C55PA端口地址
#define PB8255 XBYTE[0xff7d]           //0xff7d为82C55PB端口地址
#define PC8255 XBYTE[0xff7e]           //0xff7e为82C55PC端口地址
#define COM8255 XBYTE[0xff7f]          //0xff7f为82C55控制端口地址
#define uchar unsigned char
```

```
extern void delay_1000us ( );
```

```
void init8255(void)
```

```
{  
    COM8255=0x85;    //工作方式控制字写入控制寄存器  
}
```

```
void main(void)
```

```
{  
    init8255(void)  
    for(;;)  
    {  
        COM8255=0x0b;    //PC5脚为高电平  
        delay_1000us ( );    //高电平持续1000s  
        COM8255=0x0a;    //PC5脚为低电平  
        delay_1000us ( );    //低电平持续1000s  
    }  
}
```

9.6 利用74LS TTL电路扩展并行I/O口

在单片机应用系统中，有些场合可采用TTL电路、CMOS电路锁存器或三态门电路构成各种简单输入/输出口。通常这种I/O都是通过P0口扩展。由于P0口只能分时复用，故构成输出口时，接口芯片应具有锁存功能；构成输入口时，要求接口芯片应能三态缓冲或锁存选通。

图9-30为一利用74LS244和74LS373芯片，扩展了简单的I/O口的电路。74LS244和74LS373的工作受单片机的P2.7、RD*、WR* 3条控制线控制。74LS244是缓冲驱动器，作为扩展的输入口，8个输入端分别分别接8个开关S7~S0。

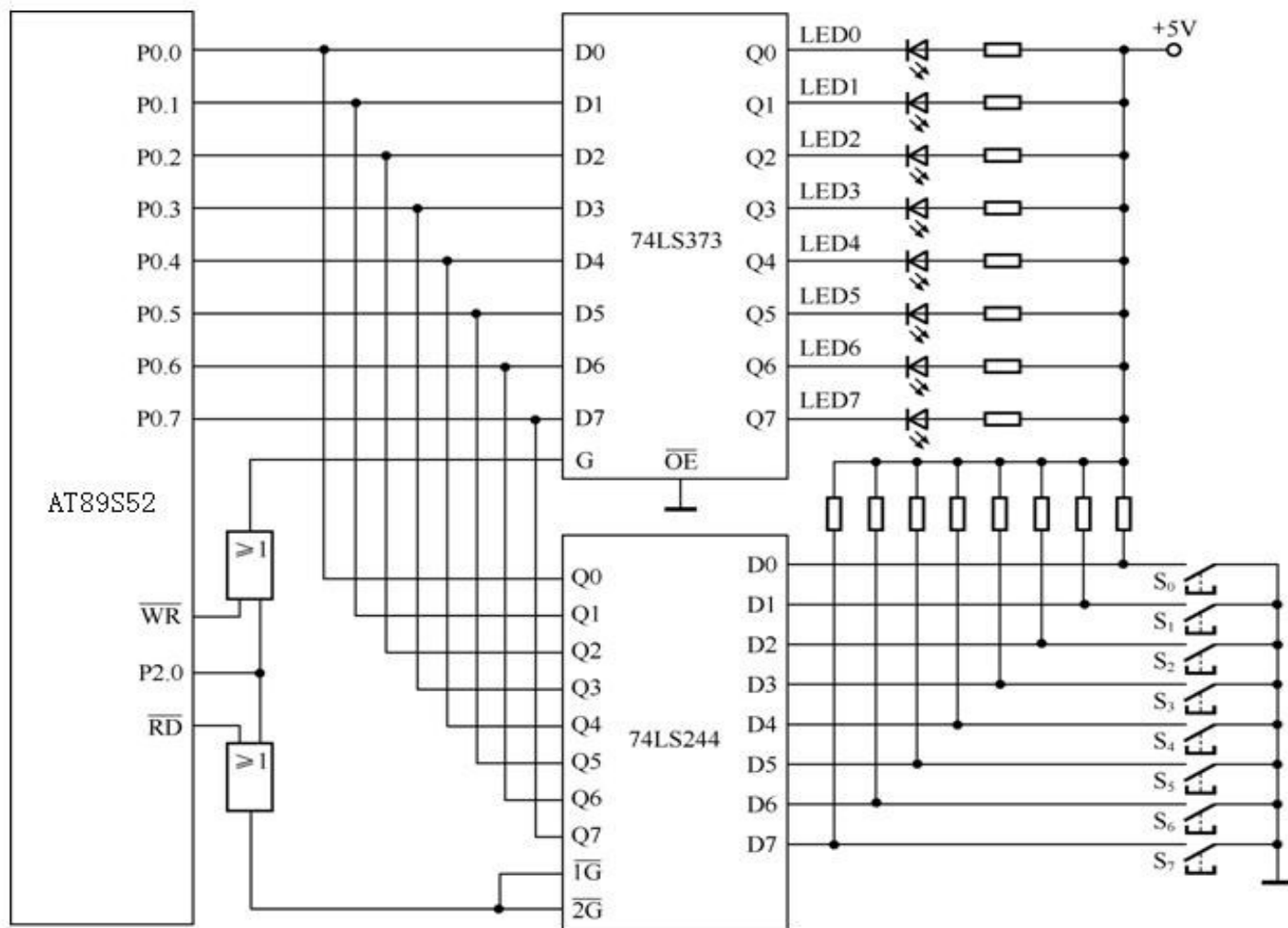


图9-30 利用74LSTTL电路扩展I/O举例

74LS373是**8D**锁存器，作为扩展的输出口，输出端接**8**个发光二极管**LED7~LED0**。当某输入口线的开关按下时，该输入口线为低电平，读入单片机后，其相应位为“**0**”，然后再将口线的状态经**74LS373**输出，某位低电平时二极管发光，从而显示出按下的开关的位置。

由图**9-30**可确定扩展的**74LS244**和**74LS373**芯片具有相同端口地址：**0xfeff**，只不过读入时，**P2.0**和**RD***有效，选中**74LS244**；输出时**P2.0**和**WR***有效选中**74LS373**。

【例9-8】 电路如**图9-30**所示，编程把开关**S7~S0**的状态通过**74LS373**输出端的**8**个发光二极管显示出来。参考程序如下：

```
#include <absacc.h>

#define uchar unsigned char

... ..

uchar i

i=XBYTE[0xfeff]

XBYTE[0xfeff]=i

...
```

由程序可以看出，对于所扩展接口的输入/输出如同对外部**RAM**读/写数据一样方便。**图9-30**仅扩展**1**片输出芯片和**1**片输入芯片，如不够用，可根据需要扩展多片**74LS244**、**74LS373**之类的芯片，但需要在端口地址上对各芯片加以区分。

9.7 用AT89S51单片机的串行口扩展并行输入/输出口

AT89S52串行口方式0用于并行I/O扩展。在方式0时，串行口为同步移位寄存器工作方式，其波特率是固定的，为 $f_{osc}/12$ （ f_{osc} 为系统的振荡器频率）。数据由RXD端（P3.0）输入，同步移位时钟由TXD端（P3.1）输出。发送、接收的数据是8位，低位在先。

9.7.1 用74LS165扩展并行输入口

图9-31为串口外扩两片74LS165扩展两个8位并行输入口的接口电路。

74LS165是8位并行输入串行输出的寄存器。当74LS165的S/L*端由高到低跳变时，并行输入端的数据被置入寄存器；当S/L*=1，且时钟禁止端（第15脚）为低电平时，允许TXD（P3.1）移位时钟输入，这时在时钟脉冲作用下，数据由右向左方向移动。

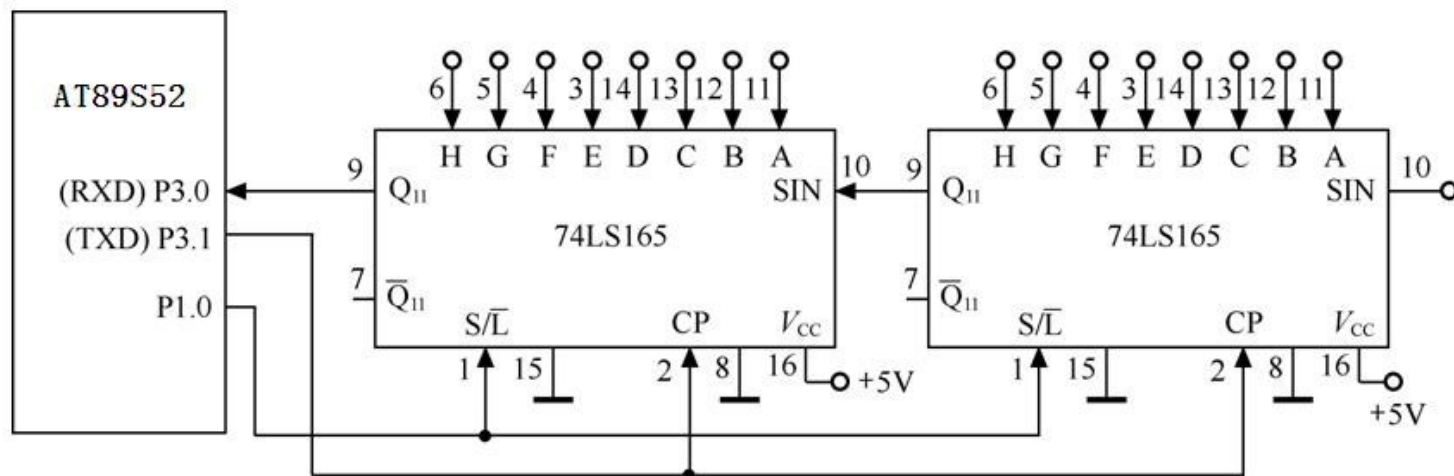


图9-31 利用74LS165扩展并行输入口

图9-31中，**TXD (P3.1)** 作为移位脉冲输出与所有**74LS165**的移位脉冲输入端**CP**相连；**RXD (P3.0)** 作为串行数据输入端与**74LS165**的串行输出端**QH**相连；**P1.0**与**S/L***相连，用来控制**74LS165**的串行移位或并行

输入；74LS165的时钟禁止端（第15脚）接地，表示允许时钟输入。当扩展多个8位输入口时，相邻两芯片首尾（QH与SIN）相连。

【例9-9】下面程序是从16位扩展口读入4组数据（每组2B），并存入到内部RAM缓冲区。参考程序如下：

```
#include <reg51.h>
typedef unsigned char BYTE;
BYTE rx_data[8];
sbit test_flag;           //定义读入字节的奇偶标志
sbit P1_0=P1^0;          //定义工作状态控制端
```

BYTE receive(void) //读入数据函数

```
{  
    BYTE temp;  
    while(RI==0); RI=0; temp=SBUF;  
    return temp;  
}
```

void main(void) //主程序

```
{  
    BYTE i;  
    test_flag=1; //奇偶标志初始值为1，表示读的是奇数字节  
    for(i=0; i<8; i++) //循环读入8字节数据  
    {  
        P1_0=0; //并行置入2字节数据  
        P1_0=1; //允许串行移位读入  
        SCON=0x10; //设置串行口方式0  
        rx_data[i]= receive(); //接收1字节数据  
        test_flag=~test_flag; //改写读入字节奇偶性，以决定是否重新  
        并行置入  
    }  
}
```

```
        if(test_flag==0)
        {
            SCON=0x10;          //设置串行口方式0
            rx_data[i]= receive( );    //接收1字节数据
            test_flag=~test_flag;      //改写读入字节奇偶性，以决定是否重新并
            行置入
        }
    }
}
```

上面程序串行接收过程采用查询等待的控制方式，如必要，也可改用中断方式。从理论上讲，按图9-31扩展的输入口几乎是无限，但扩展越多，口操作速度也就越慢。

9.7.2 用74LS164扩展并行输出口

单片机串行口的方式0外接一片**74LS164**扩展一个**8**位并行输出口的**Proteus**虚拟仿真设计案例见【例8-1】。

下面介绍串行口外接两片**74LS164**扩展两个**8**位并行输出口的接口电路。接口电路见图9-32所示。

当单片机串行口工作在方式0发送状态时，串行数据由**P3.0 (RXD)**送出，移位时钟由**P3.1 (TXD)**送出。在移位时钟的作用下，串行口发送缓冲器的数据一位一位地从**P3.0**移入**74LS164**中。需要指出的是，由于**74LS164**无并行输出控制端，因而在串行输入过程中，其输出端的状态会不断变化，故在某些应用场合，在**74LS164**的输出端应加接输出三态门控

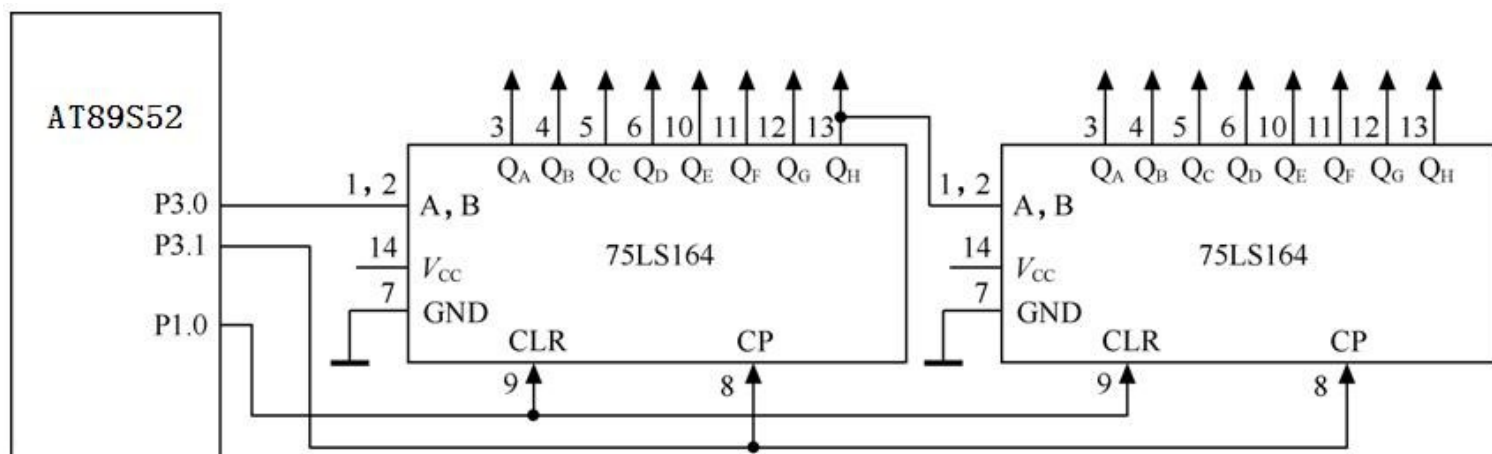


图9-32 利用74LS164扩展并行输出口

制，以便保证串行输入结束后再输出数据。

【例9-10】 由图9-32，将内部RAM缓冲区的8字节经串口由74LS164并行输出，

参考程序：

```
#include <reg51.h>
typedef unsigned char BYTE;
BYTE data[8]={0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08 }
```

```
void main(void)
```

```
//主函数
```

```
{
```

```
    SCON=0x00;
```

```
//设置串行口方式0
```

```
for(i=0; i<=8; i++)
```

```
{
```

```
    SBUF= data[j]
```

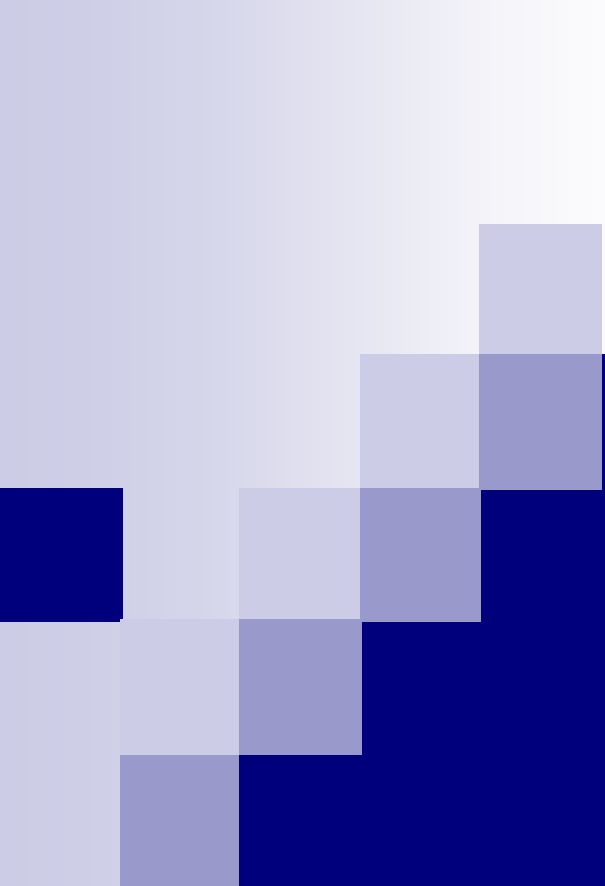
```
    while(TI==0);TI=0;
```

```
}
```

```
while(1);
```

```
}
```

//输出8字节数据



第11章 AT89S52单片机与DAC、 ADC的接口



单片机处理完毕的数字量，有时根据控制要求需要转换为模拟信号输出。**数字量转换成模拟量的器件**称为DAC（D/A转换器）。

而对温度、压力、流量、速度等非电物理量，经传感器先转换成连续变化的模拟电信号（电压或电流），然后再将模拟电信号转换成数字量后才能在单片机中进行处理。**实现模拟量转换成数字量的器件**称为ADC（A/D转换器）。

本章从应用的角度，介绍典型的ADC、DAC芯片与AT89S51单片机的硬件接口设计以及接口驱动程序设计，此外还介绍了单片机与ADC构成的数据采集系统的几种软件滤波方法。

11.1 单片机扩展DAC概述

单片机只能输出数字量，但是对于某些控制场合，常常需要输出模拟量，例如直流电动机的转速控制。下面介绍单片机如何扩展DAC。

目前集成化的DAC芯片种类繁多，设计者只需要合理选用芯片，了解它们的性能、引脚外特性以及与单片机的接口设计方法即可。由于现在部分单片机的芯片中集成了DAC，位数一般在10位左右，且转换速度也很快，所以单片的DAC开始向高的位数和高转换速度上转变。而低端的并行8位DAC，开始面临被淘汰的危险，但是在实验室或涉及某些工业控制方面的应用，低端8位DAC以其优异的性价比还是具有较大的应用空间。

1. D/A转换器简介

购买和使用D/A转换器时，要注意有关D/A转换器选择的几个问题。

(1) D/A转换器的输出形式

D/A转换器有两种输出形式：**电压输出**和**电流输出**。电流输出的D/A转换器在输出端加一个运算放大器构成的I-V转换电路，即可转换为电压输出。

(2) D/A转换器与单片机的接口形式

单片机与D/A转换器的连接，早期多采用8位的并行传输的接口，现在除了并行接口外，带有串行口的D/A转换器品种也不断增多，目前较为流行的多采用SPI串行接口。在选择单片D/A转换器时，要根据系统结构考虑单片机与D/A转换器的接口形式。

2. 主要技术指标

D/A转换器的指标很多，设计者最关心的几个指标如下。

(1) 分辨率

分辨率指单片机输入给D/A转换器的单位数字量的变化，所引起的模拟量输出的变化，通常定义为输出满刻度值与 2^n 之比（ n 为D/A转换器的二进制位数），习惯上用输入数字量的位数表示。显然，二进制位数越多，分辨率越高，即D/A转换器输出对输入数字量变化的敏感程度越高。

例如，8位的D/A转换器，若满量程输出为10V，根据分辨率定义，则分辨率为 $10V/2^n$ ，分辨率为 $10V/256 = 39.1\text{mV}$ ，即输入的二进制数最低位数字量的变化可引起输出的模拟电压变化39.1mV，该值占满量程的0.391%，

常用符号1LSB表示。

同理：

10位D/A转换

$$1 \text{ LSB} = 9.77\text{mV} = 0.1\% \text{满量程}$$

12位D/A转换

$$1 \text{ LSB} = 2.44\text{mV} = 0.024\% \text{满量程}$$

16位D/A转换

$$1 \text{ LSB} = 0.076\text{mV} = 0.00076\% \text{满量程}$$

(2) 建立时间

建立时间是用于描述D/A转换器转换时间长短的参数。其值为从输入数字量到输出达到终值误差 $\pm(1/2)\text{LSB}$ （最低有效位）时所需的时间。电流输出的转换时间较短，而电压输出的转换器，由于要加上完成I-V转换的时间，因此建立时间要长一些。快速D/A转换器的建立时间可控制在 $1\mu\text{s}$ 以下。

(3) 转换精度

理想情况下，转换精度与分辨率基本一致，位数越多精度越高。但由于电源电压、基准电压、电阻、制造工艺等各种因素存在误差。严格地讲，转换精度与分辨率并不完全一致。两个相同位数的不同的DAC，只要位数相同，分辨率则相同，但转换精度会有所不同。

例如，某种型号的8位DAC精度为 $\pm 0.19\%$ ，而另一种型号的8位DAC精度为 $\pm 0.05\%$ 。

11.2 单片机扩展并行8位DAC0832的设计

11.2.1 DAC 0832简介

美国国家半导体公司的DAC0832芯片具有**两级输入数据寄存器**的8位DAC，能直接与AT89S52连接，特性如下。

- (1) 分辨率为8位。
- (2) 电流输出，建立时间为 $1\mu\text{s}$ 。
- (3) 可双缓冲输入、单缓冲输入或直通输入。
- (4) 单一电源供电（+5V~+15V），低功耗，20mW。

2. DAC0832的引脚及逻辑结构

引脚见**图11-1**。

内部结构见**图11-2**。

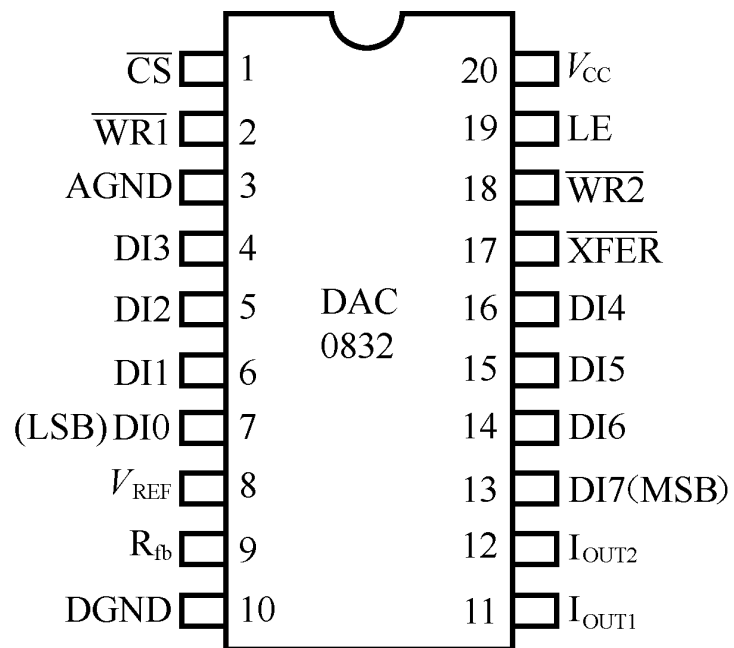


图11-1 DAC0832引脚

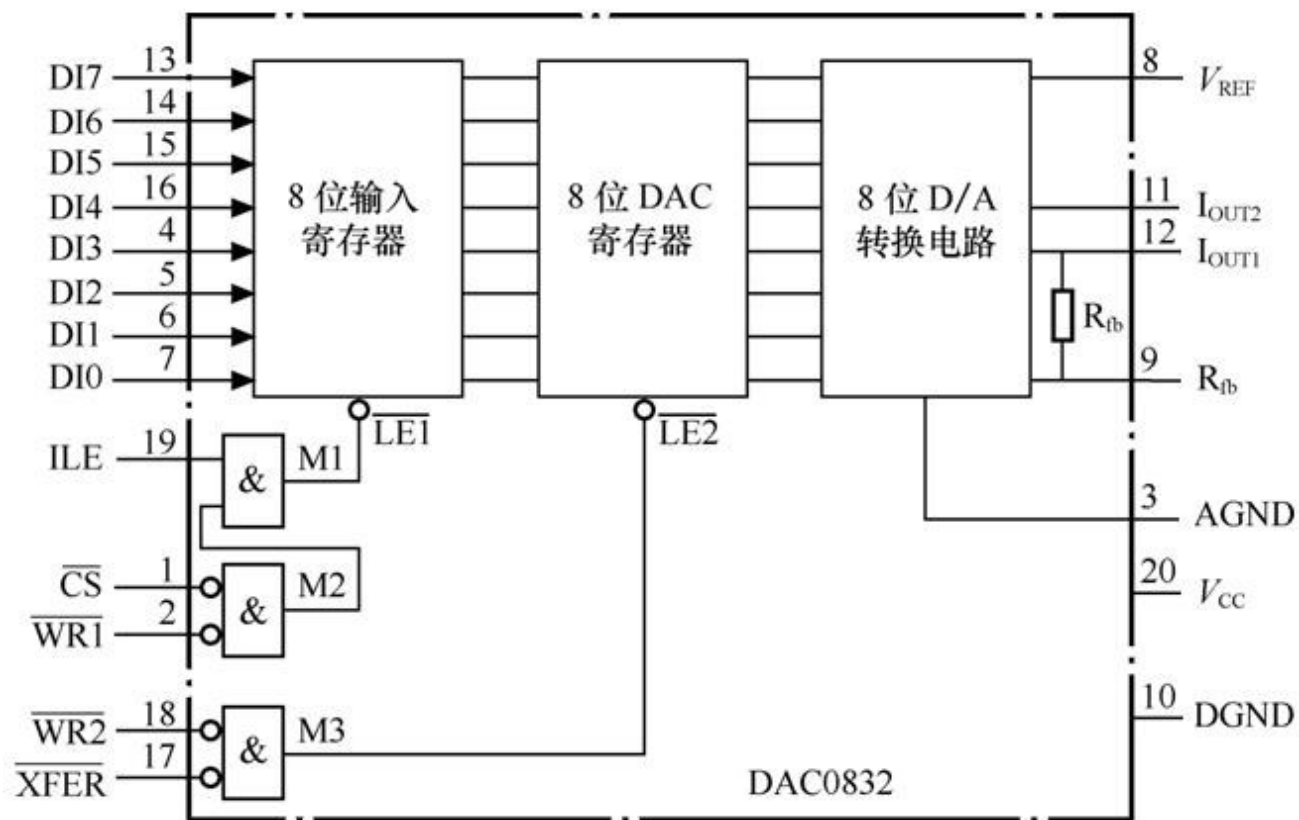


图11-2 DAC0832逻辑结构

由图11-2可见，片内共两级寄存器。

第一级为“8位输入寄存器”，用于存放单片机送来的数字量，使得该数字量得到缓冲和锁存，由LE1*（即M1=1时）加以控制；

“8位DAC寄存器”是第二级8位输入寄存器，用于存放待转换的数字量，由LE2*控制（即M3=1时），这两级8位寄存器，构成两级输入数字量缓存。“8位D/A转换电路”受“8位DAC寄存器”输出数字量控制，输出和数字量成正比的模拟电流。

各引脚的功能如下。

● **DI7~DI0:** 8位数字量输入端，接收发来的数字量。

● **ILE、CS*、WR1*:** 当ILE=1，CS*=0，WR1*=0时，即M1=1，第一级8位输入寄存器被选中。待转换的数字信号被锁存到第一级8位输入寄存器中。

● **XFER*、WR2*:** 当XFER*=0，WR2*=0时，第一级8位输入寄存器中待转换数字进入第二级8位DAC寄存器中。

● **I_{OUT1}:** D/A转换电流输出1端，输入数字量全为“1”时，I_{OUT1}最大，输入数字量全为“0”时，I_{OUT1}最小。

- I_{OUT2} : D/A转换电流输出2端, $I_{OUT2} + I_{OUT1} = \text{常数}$ 。

- R_{fb} : I-V转换时的外部反馈信号输入端, 内部已有反馈电阻 R_{fb} , 根据需要也可外接反馈电阻。

- V_{REF} : 参考电压输入端。

- V_{CC} : 电源输入端, 在+5V~+15V范围内。

- DGND: 数字地。

- AGND: 模拟地, 最好与基准电压共地

11.2.2 案例设计: 单片机扩展DAC0832的程控电压源

单片机控制DAC0832可实现数字调压, 单片机只要送给DAC0832不同数字量, 即可实现不同模拟电压输出。

DAC0832输出可用单缓冲方式或双缓冲方式。

单缓冲方式是DAC0832片内的两级数据寄存器的有一个处于直通方式，另一处于受单片机控制的锁存方式。实际应用中，如只有一路模拟量输出，或虽是多路模拟量输出，但并不要求多路输出同步情况下，就可采用单缓冲方式。

单片机控制DAC0832实现数字调压的单缓冲方式接口电路见图11-3。由于**XFER*=0**、**WR2*=0**，所以第二级“8位DAC寄存器”处于直通。第一级“8位输入寄存器”为单片机控制的锁存方式，3个锁存控制端的**ILE**直接接到有效的高电平，另两个控制端**CS***、**WR1*** 分别由单片机**P2.0**和**P2.1**控制

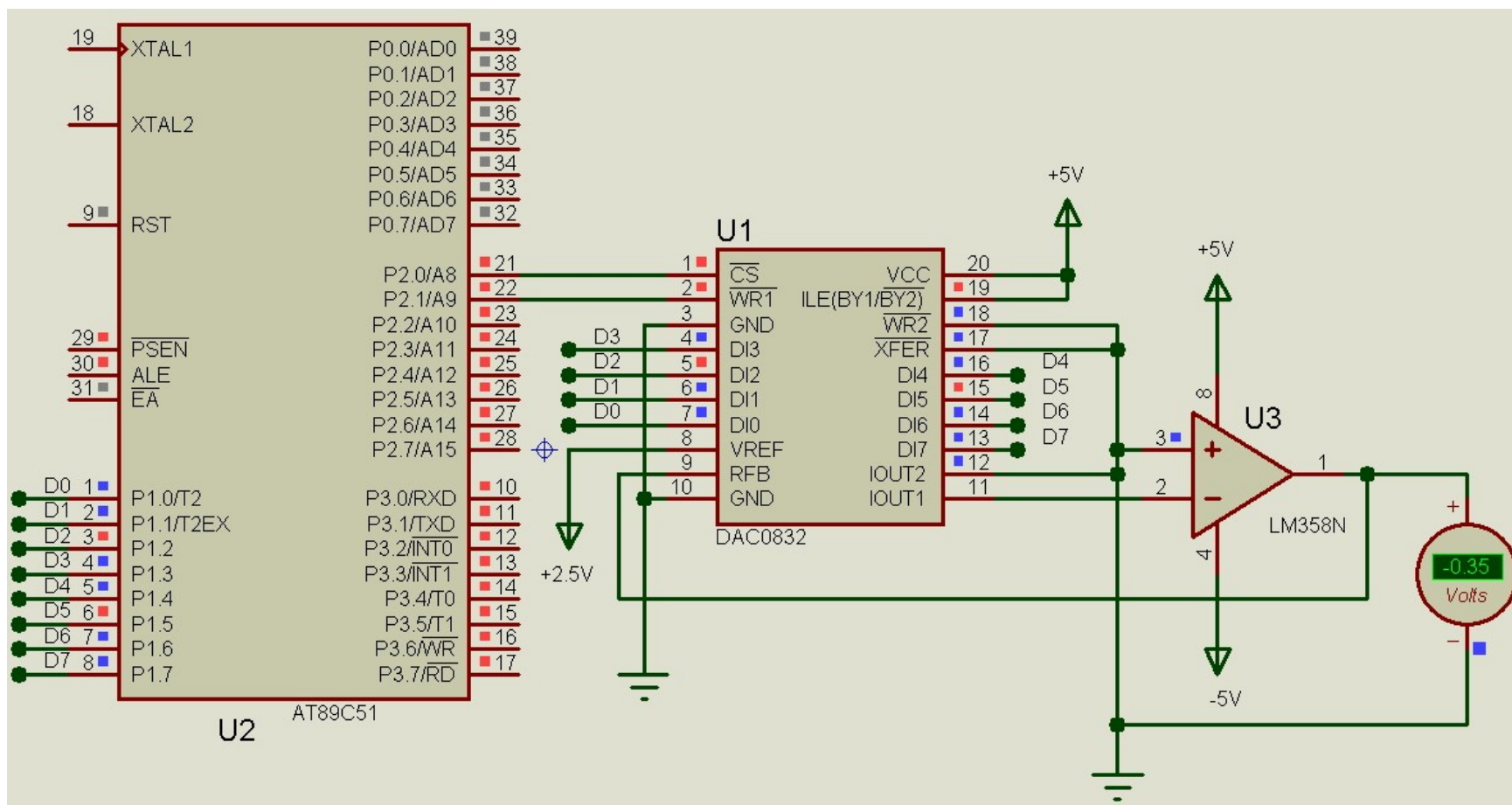


图11-3 单缓冲方式的单片机与DAC0832的接口原理电路

DAC 0832输出电压 V_o 与输入数字量 B 关系为:

$$V_o = -(B * V_{REF}) / 256$$

由上式见，输出模拟电压 V_o 与输入数字量 B 以及基准电压 V_{REF} 成正比，且 B 为0时， V_o 也为0， B 为255时， V_o 为最大的绝对值输出，且不会大于 V_{REF} 。下面介绍单缓冲方式下单片机扩展DAC0832程控电压源的设计。

【例11-1】单片机与DAC0832单缓冲方式接口见图11-3，单片机P2.0脚控制DAC0832的CS*脚，P2.1控制WR1*端。当P2.0脚为低时，如果同时WR1*有效，单片机就会把数字量通过P1口送入DAC0832的DI7~DI0端，并转换输出。用虚拟直流电压表测量经运放LM358N的I/V转换后的电压值，并观察输出电压变化。

仿真运行后，可看到虚拟直流电压表测量输出电压在-2.5V~0V（参考电压为2.5V）范围内不断线性变化。如参考电压为5V，则输出电压在-5V~0V范围内变化。如果虚拟直流电压表太小，看不清楚电压显示值，可用鼠标滚轮放大直流电压表。

单片机送给DAC0832不同的数字量，就可得到不同的输出电平，从而使单片机控制DAC0832成为一个程控电压源。

参考程序如下:

```
#include "reg51.h"
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
#define out P1
```

```
sbit DAC_cs=P2^0;
```

```
sbit DAC_wr=P2^1;
```

```
void main(void)
```

```
//主函数
```

```
{
```

```
    uchar temp,i=255;
```

```
    while(1)
```

```
    {
```

```
        {
```

```
            out=temp;
```

```

DAC_cs=0;           //单片机控制脚为低
DAC_wr=0;           //WR1脚为低，向DAC写入转换的数字量
DAC_cs=1;
DAC_wr=1;
temp++;
while(--i);         //i先减1，然后再使用i的值
while(--i);
while(--i);
    }
}
}

```

单片机送给DAC0832不同的数字量，就可得到不同的输出电平，从而使单片机控制的DAC0832成为一个程控电压源。

11.2.3 案例设计2：波形发生器的制作

单片机如把不同**波形数据**发送给DAC0832，就可产生各种不同波形信号。下面介绍单片机控制DAC0832产生各种函数波形案例。

【例11-2】单片机控制DAC0832产生**正弦波、方波、三角波、梯形波和三角波**。

原理电路见**图11-4**。单片机P1.0~ P1.4接有5个按键，当按键按下时，分别对应产生正弦波、方波、三角波、梯形波和三角波。

单片机控制DAC0832产生各种波形，实质就是单片机把波形的采样点数据送至DAC0832，经D/A转换后输出模拟信号。改变送出的函数波形采样点后的延时时间，就可改变函数波形的频率。产生各种波形原理如下。

(1) 正弦波产生原理

单片机把正弦波的256个采样点的数据送给DAC0832。正弦波采样数据可用软件编程或MATLAB等工具计算。

(2) 方波产生原理

采用定时器定时中断，时间常数决定方波高、低电平持续时间。

(3) 三角波产生原理

单片机把初始数字量0送DAC0832后，不断增1，增至0xff后，然后再把送给DAC0832的数字量不断减1，减至0后，再重复上述过程。

(4) 锯齿波产生原理

单片机把初始数据0送DAC0832后，数据不断增1，增至0xff后，再增1则溢出清“0”，模拟输出又为0，然后再重复上述过程，如此循环，则输出锯齿波。

(5) 梯形波产生原理

输入给DAC0832数字量从0开始，逐次加1。当输入数字量为0xff时，

延时一段时间，形成梯形波平顶，然后波形数据再逐次减1，如此循环，则输出梯形波。

参考程序如下：

```
#include<reg51.h>
sbit wr=P3^6;
sbit rd=P3^2;
sbit key0=P1^0;           //定义P1.0脚的按键为正弦波键key0
sbit key1=P1^1;           //定义P1.1脚的按键为方波键key1
sbit key2=P1^2;           //定义P1.2脚的按键为三角波键key2
sbit key3=P1^3;           //定义P1.3脚的按键为梯形波键key3
sbit key4=P1^4;           //定义P1.3脚的按键为锯齿波键key4
unsigned char flag;        //flag为1、2、3、4、5时对应正弦波、方波、
                           //三角波、梯形波、锯齿波
```

```
unsigned char const code //以下为正弦波采样点数组256个数据
```

[illegible]

unsigned char keyscan()

//键盘扫描函数

```
{  
    unsigned char keyscan_num,temp;  
    P1=0xff;                // P1口输入  
    temp=P1;                //从P1口读入键值，存入temp中  
    if(~(temp&0xff))        //判是否有键按下，即键值不为0xff，则有键按下  
    {  
        if(key0==0)        //产生正弦波的按键按下， P1.0=0  
        {  
            keyscan_num=1;  //得到的键值为1，表示产生正弦波  
        }  
        else if(key1==0)    //产生方波的按键按下， P1.1=0  
        {  
            keyscan_num=2;  //得到键值为2，表示产生方波  
        }  
    }  
}
```

```
else if(key2==0)                //产生三角波的按键按下， P1.2=0
{
    keyscan_num=3;              //得到的键值为3， 表示产生三角波
}
else if(key3==0)                //产生梯形波的按键按下， P1.3=0
{
    keyscan_num=4;              //得到的键值为4， 表示产生梯形波
}
else if(key4==0)                //产生锯齿波的按键按下， P1.3=0
{
    keyscan_num=5;              //得到的键值为5， 表示产生锯齿波
}
else
{
    keyscan_num=0;              //没有按键按下， 键值为0
}
return keyscan_num;             //得到的键值返回
}
```

```
void init_DA0832()
```

```
//DAC0832初始化函数
```

```
{  
    rd=0;  
    wr=0;  
}
```

```
void SIN( )
```

```
//正弦波函数
```

```
{  
    unsigned int i;  
    do  
    {  
        P2=SIN_code[i];           //由P2口输出给DAC0832正弦波数据  
        i=i+1;                     //数组数据指针增1  
    }while(i<256);                //判是否已输出完256个波形数据，未完继续输出  
}
```

void Square()

{

EA=1;

ET0=1;

TMOD=1;

TH0=0xff;

TL0=0x83;

TR0=1;

}

void Triangle ()

{

P2=0x00;

do

{

P2=P2+1;

//方波函数

//总中断允许

//允许T0中断

//T0工作在方式1

//给T0高8位装入时间常数

//给T0低8位装入时间常数

//启动T0

//三角波函数

//三角波函数初始值为0

//三角波上升沿

```
}while(P2<0xff);
```

//判是否已经输出为0xff

```
P2=0xff;
```

```
do
```

```
{
```

```
    P2=P2-1;
```

//三角波下降沿

```
}while(P2>0x00);
```

//判是否已经输出为0

```
P2=0x00;
```

```
}
```

```
void Sawtooth ( )
```

//锯齿波函数

```
{
```

```
    P2=0x00;
```

```
do
```

```
{  
    P2=P2+1; //产生锯齿波的上升沿  
}while(P2<0xff);  
//判上升沿是否已经结束  
}
```

```
void Trapezoidal ( ) //梯形波函数  
{  
    unsigned char i;  
    P2=0x00;  
    do
```

```
{  
    P2=P2+1;                //产生梯形波的上升沿  
    }while(P2<0xff);  
    P2=0xff;                //产生梯形波的平顶  
    for(i=255;i>0;i--)      //梯形波的平顶延时  
    {  
        P2=0xff;            //产生梯形波的下沿  
    }  
do  
{  
    P2=P2-1;                //产生梯形波的下沿  
    }while(P2>0x00);        //判梯形波的下沿是否结束  
    P2=0x00;  
}
```



```
void main()
```

```
// 主函数
```

```
{
```

```
    init_DA0832();
```

```
// DA0832的初始化函数
```

```
    do
```

```
    {
```

```
        flag=keyscan();
```

```
//将键盘扫描函数得到的键值赋给flag
```

```
    }while(!flag);
```

```
    while(1)
```

```
    {
```

```
        switch(flag)
```

```
        {
```

```
        case 1:
```

```
        do{
```

```
            f lag=keyscan();
```

```
            SIN( );
```

```
            }while(flag==1);
```

```
        break;
```


case 2:

Square ();

do{

flag=keyscan();

}while(flag==2);

TR0=0;

break;

case 3:

do{

flag=keyscan();

Triangle ();

}while(flag==3);

break;

case 4:

```
do{  
    flag=keyscan();  
    Trapezoidal ();  
}while(flag==4);  
break;
```

case 5:

```
do{  
    flag=keyscan();  
    Sawtooth ();  
}while(flag==5);  
break;
```

default:

```
flag=keyscan();  
break;  
}
```

```
}
```

```
}
```

```
void timer0(void) interrupt 1           //定时器T0的中断函数
{
    P2=~P2;                             //方波的输出电平求反
    TH0=0xff;                           //重装定时时间常数
    TL0=0x83;
    TR0=1;                              //启动定时器T0
}
```

本案例在仿真运行时，可看到弹出的虚拟示波器，从虚拟示波器屏幕上可观察到由按键选择的函数波形输出。

如在仿真时关闭该虚拟示波器后，再启用虚拟示波器观察波形，可点击鼠标右键，现下拉菜单，点击“**oscilloscope**”后，仿真界面又会出现虚拟示波器屏幕。

11.3 AT89S51扩展10位串行DAC-TLC5615

11.3.1 串行DACTLC5615简介

美国TI公司产品，串行接口DAC，电压输出型，最大输出电压是基准电压值两倍。带上电复位功能，即上电时把DAC寄存器复位至全零。单片机只需用3根串行总线就可完成10位数据的串行输入，易于和工业标准的微处理器或单片机接口，非常适于电池供电的测试仪表、移动电话，也适用于数字失调与增益调整以及工业控制场合。

TLC5615的引脚见图11-5。

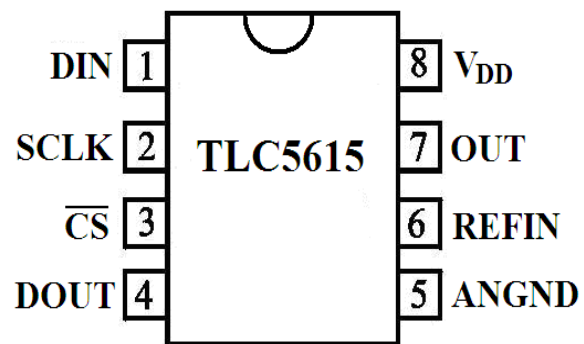


图11-5 TLC5615引脚

8只引脚功能如下：

- **DIN**: 串行数据输入端；
- **SCLK**: 串行时钟输入端；
- **CS***: 片选端，低电平有效；

- **DOUT**: 用于级联时的串行数据输出端;
- **AGND**: 模拟地;
- **REF_{IN}**: 基准电压输入端, $2V \sim (VDD - 2)$;
- **OUT**: DAC 模拟电压输出端;
- **VDD**: 正电源端, $4.5 \sim 5.5V$, 通常取 $5V$ 。

TLC5615 内部功能框图见[图11-6](#)。

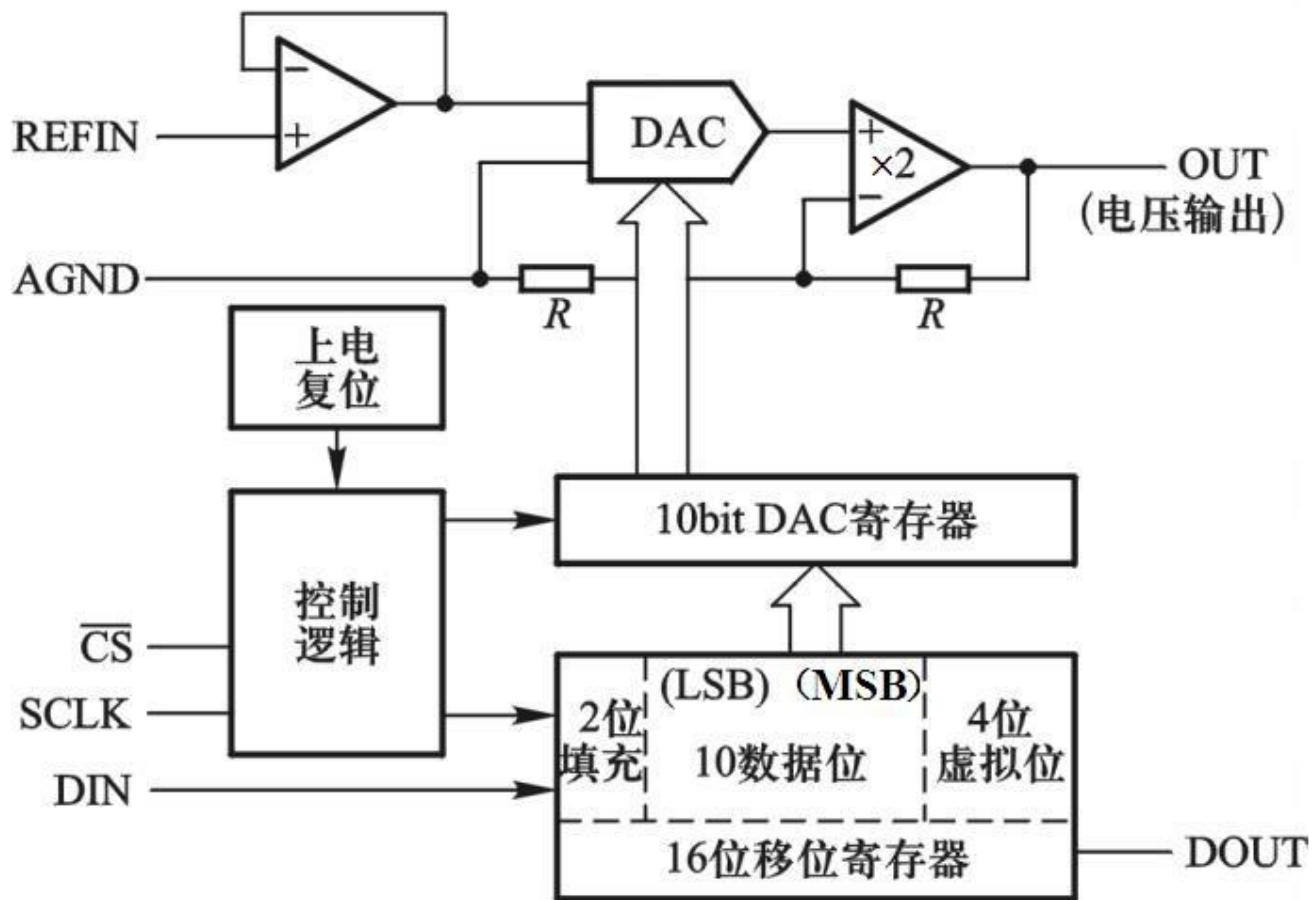


图11-6 TLC5615 内部功能框图

主要由以下几部分组成:

- 10 位 DAC 电路;
- 一个 16 位移位寄存器, 接收串行移入的二进制数, 且有一个级联的数据输出端 D_{OUT} ;
- 并行输入输出的 10 bit DAC 寄存器, 为 10 位 DAC 电路提供待转换的二进制数据;
- 电压跟随器为参考电压端 REF_{IN} 提供高输入阻抗, 大约 $10M\Omega$;
- $\times 2$ 电路提供最大值为 2 倍于 REF_{IN} 输出;
- 上电复位电路和控制逻辑电路。

两种工作方式:

(1) 第1种工作方式: 12 位数据序列。从图11-6看出, 16 位移位寄存器分为高4位的虚拟位、低2位的填充位以及10位有效数据位。在 TLC5615 工作时, 只需要向 16 位移位寄存器先后输入10位有效位和低2位的任意填充位。

(2) 第2种工作方式: 级联方式, 即 16 位数据列, 可将本片的 D_{OUT} 接到下一片的 D_{IN} , 需向 16 位移位寄存器先后输入高 4 位虚拟位、10 位有效位和低 2 位填充位, 由于增加了高 4 位虚拟位, 所以需要 16 个时钟脉冲。

当TLC5615片选脚CS*为低时，串行输入数据才能被移入16位移位寄存器。

当CS*为低时，在每一个 SCLK 时钟的上升沿将 DIN 的一位数据移入 16 位移寄存器。注意，二进制最高有效位被导前移入。接着，CS*的上升沿将16位移位寄存器的10位有效数据锁存于10位 DAC 寄存器，供DAC电路进行转换。

11.3.2 案例设计：单片机扩展串行DAC-TLC5615的设计

【例11-3】单片机控制串行DAC-TLC5615进行D/A转换，原理电路及仿真见图11-7。调节可变电位计RV1的值，使TLC5615的输出电压可在0～5V 内调节，从虚拟直流电压表可观察到DAC转换输出的电压值。

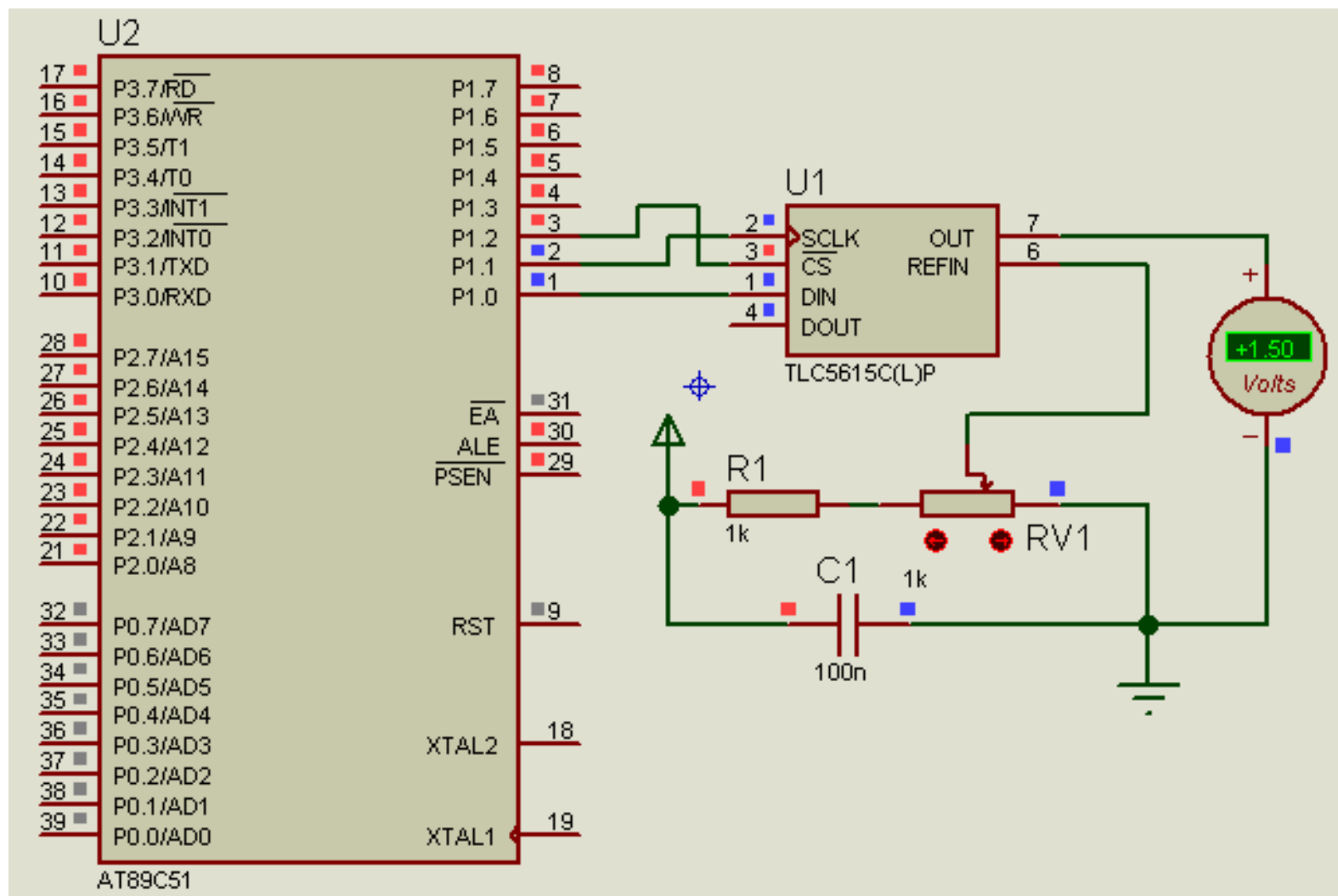


图11-7 单片机与DAC-TLC5615的接口电路

参考程序如下：

```
#include<reg51.h>
```

```
#include<intrins.h>
```

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
sbit SCL=P1^1;
```

```
sbit CS=P1^2;
```

```
sbit DIN=P1^0;
```

```
uchar bdata dat_in_h;
```

```
uchar bdata dat_in_l;
```

```
sbit h_7 = dat_in_h^7;
```

```
sbit l_7 = dat_in_l^7;
```

```
void delayms(uint j)
```

```
{
```

```
    uchar i=250;
```

```
    for(;j>0;j--)
```

```

{
    while(--i);
    i=249;
    while(--i);
    i=250;
}
}

```

void Write_12Bits(void)

```

{
    uchar i;
    SCL=0;
    CS=0;
    for(i=0;i<2;i++)
    {
        if(h_7)
        {

```

//一次向TLC5615中写入12bit数据函数

//置零SCL，为写bit做准备；

//片选端 =0；

//循环2次，发送高两位；

//高位先发；

```
DIN = 1;           //将数据送出;
SCL = 1;           //提升时钟，写操作在时钟上升沿触发
SCL = 0;           //结束该位传送，为下次写作准备;
}

else

{

    DIN= 0;
    SCL = 1;
    CL = 0;

}

dat_in_h <<= 1;
}
```

```
for(i=0;i<8;i++)                //循环八次，发送低八位；
{
    if(l_7)
    {
        DIN = 1;                //将数据送出；
        SCL = 1;                //提升时钟，写操作在时钟上升沿触发
        SCL = 0;                //结束该位传送，为下次写作准备
    }
    else
    {
        DIN= 0;
        SCL = 1;
        SCL = 0;}
    dat_in_l <<= 1;}
for(i=0;i<2;i++)                //循环2次，发送两个填充位
```

```
{  
    DIN= 0;SCL = 1;SCL = 0;  
}  CS = 1; SCL = 0;}
```

```
void TLC5615_Start(uint dat_in)
```

```
//启动DAC转换函数
```

```
{  
    dat_in %= 1024;  
    dat_in_h=dat_in/256;  
    dat_in_l=dat_in%256;  
    dat_in_h <=< 6;  
    Write_12Bits();  
}
```

```
void main()
```

```
//主函数
```

```
{  
    while(1)  
    {TLC5615_Start(0xffff); delayms(1);}  
}
```


11.4 单片机扩展ADC概述

A/D转换器（ADC）把模拟量转换成数字量，单片机才能进行数据处理，这就是通常所说的数据采集系统。

1. A/D转换器简介

目前单片ADC芯片较多，对设计者来说，只需合理的选择芯片即可。现在部分的单片机片内也集成了A/D转换器，位数为8位、10位或12位，且转换速度也很快，但是在片内A/D转换器不能满足需要的情况下，还是需要外扩。

因此，作为外部扩展A/D转换器的基本方法，读者还是应当掌握。

尽管A/D转换器种类很多，但目前广泛应用于单片机应用系统中的主要有逐次比较型转换器和双积分型转换器，此外 Σ - Δ 式转换器也逐渐得到重视和应用。

逐次比较型ADC，在精度、速度和价格都适中，是最常用的A/D转换器。**双积分型ADC**，具有精度高、抗干扰性好、价格低廉等优点，与逐次比较型A/D转换器相比，转换速度较慢，近年来在单片机应用领域中已得到广泛应用。

Σ - Δ 式ADC具有积分式与逐次比较型ADC的优点。它对工业现场串模干扰具有较强抑制能力，不亚于双积分ADC，它比双积分ADC有较高转换速度。与逐次比较型ADC相比，有较高信噪比，分辨率高，线性度好。由于上述优点， Σ - Δ 式ADC得到了重视，已有多种 Σ - Δ 式A/D芯片可供用户选用。

A/D转换器按照输出数字量的有效位数分为4位、8位、10位、12位、14位、16位并行输出以及BCD码输出的位、位、位等多种。目前，除了并行的A/D转换器外，带有同步SPI串行接口的A/D转换器的使用也逐渐增多。串行接口的A/D转换器具有占用单片机的端口线少、使用方便、接口简单等优点，已经得到广泛的使用。较为典型的串行A/D转换器为美国TI公司的TLC549（8位）、TLC1549（10位）以及TLC1543（10位）和TLC2543（12位）等。

A/D转换器按照转换速度可大致分为超高速（转换时间 $\leq 1\text{ns}$ ）、高速（转换时间 $\leq 1\mu\text{s}$ ）、中速（转换时间 $\leq 1\text{ms}$ ）、低速（转换时间 $\leq 1\text{s}$ ）等几种不同转换速度的芯片。

2. A/D转换器主要技术指标

(1) 转换时间或转换速率。转换时间是指A/D转换器完成一次转换所需要的时间。转换时间的倒数为转换速率。

(2) 分辨率。分辨率是衡量A/D转换器能够分辨出输入模拟量最小变化程度的技术指标。分辨率取决于A/D转换器的位数，习惯上用输出的二进制位数或BCD码位数表示。

例如，A/D转换器AD1674的满量程输入电压为5V，可输出12位二进制数，即用 2^{12} 个数进行量化，其分辨率为1LSB，也即 $5V/2^{12}=1.22mV$ ，其分辨率为12位，或能分辨出输入电压1.22mV的变化。

又如，**双积分型**输出BCD码的A/D转换器MC14433，其满量程输入电压为2V，其输出最大的十进制数为1999，分辨率为三位半，如果换算成二进制位数表示，其分辨率约为11位，因为1999最接近于 $2^{11}=2048$ 。

量化过程引起的误差称为**量化误差**。量化误差是由于有限位数字量对模拟量进行量化而引起的误差。**量化误差**理论上规定为一个单位分辨率的 $\pm 1/2\text{LSB}$ ，提高位数既可以提高分辨率，又能减少量化误差。

(3) 转换精度。定义为一个实际A/D转换器与一理想A/D转换器在量化值上的差值，可用绝对误差或相对误差表示。

这里需要注意的问题是，两片具有相同位数的A/D转换器，它们的转换精度未必相同。

11.5 单片机并行扩展8位A/D转换器ADC0809

1. ADC0809引脚及功能

逐次比较型8路模拟输入、8位数字量输出的A/D转换器，引脚见图11-8。

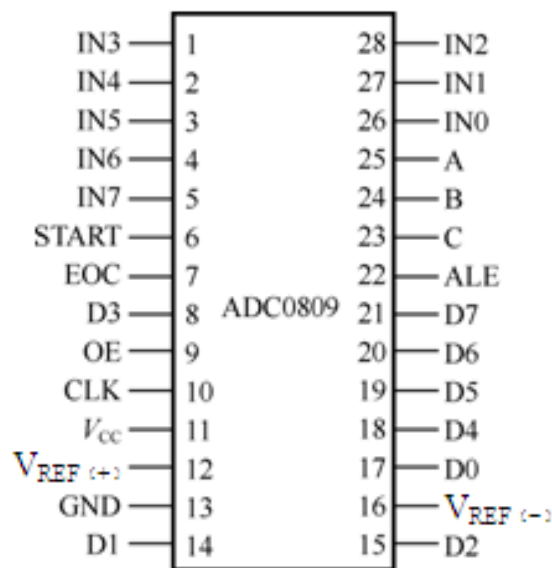


图11-8 ADC0809引脚

共28个引脚，双列直插式封装，引脚功能如下：

- **IN0~IN7**：8路模拟信号输入。
- **D0~D7**：转换完毕的8位数字量输出端。
- **C、B、A与ALE**：C、B、A端控制8路模拟输入通道的切换，分别与单片机的3条地址线相连。C、B、A = 000~111对应IN0~IN7通道地址。各路模拟输入通道之间切换由改变加到C、B、A上的编码来实现。ALE为ADC0809接收C、B、A编码时的锁存控制信号。
- **OE、START、CLK**：**OE**为转换结果输出允许端；**START**为启动信号输入端；**CLK**为时钟信号输入端，ADC0809的CLK信号须外加。
- **EOC**：转换结束输出信号。当A/D转换开始时，该引脚为低电平，当A/D转换结束时，该引脚为高电平。
- **VR (+)、VR (-)**：基准电压输入端。

2. ADC0809的内部结构

结构见图11-9。采用**逐次比较**方法完成A/D转换，由单一+5V电源供电。片内带有锁存功能的8路选1模拟开关，由加到C、B、A引脚编码决定所选通道。

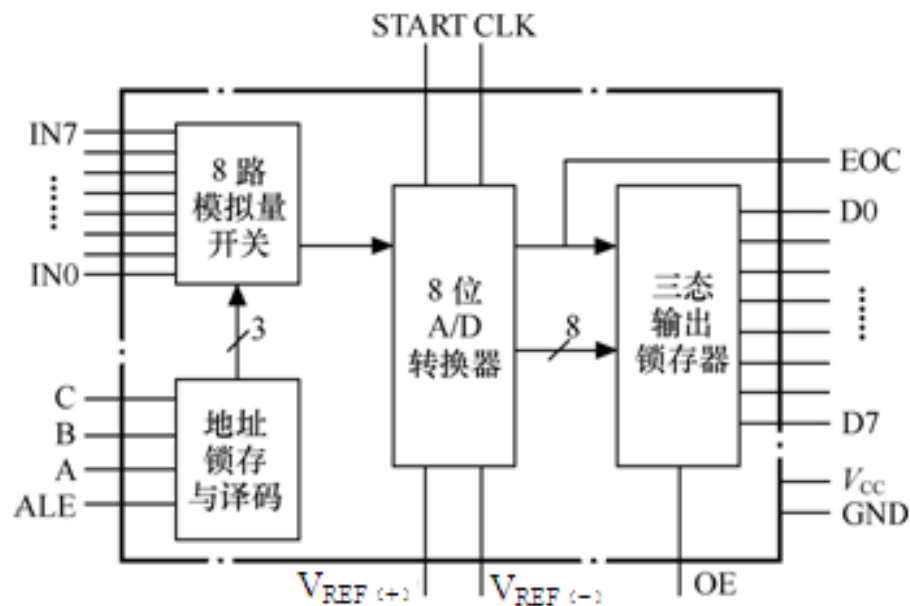


图11-9 ADC0809结构框图

ADC0809完成一次转换需 $100\mu\text{s}$ （此时加在CLK引脚的时钟频率为640MHz，即转换时间与加在CLK引脚的时钟频率有关），它具有输出TTL三态锁存缓冲器，可直接连到单片机的数据总线上。通过适当的外接电路，ADC0809可对 $0\sim 5\text{V}$ 的模拟信号进行转换。

3. 输入模拟电压与输出数字量的关系

ADC0809输入模拟电压与转换输出结果数字量关系如下：

$$V_{\text{IN}} = \frac{[V_{\text{REF}(+)} - V_{\text{REF}(-)}]}{256} \bullet N + V_{\text{REF}(-)}$$

其中： V_{IN} 处于（ $VREF(+)-VREF(-)$ ）之间，N为十进制数。通常情况下 $VREF(+)$ 接+5V， $VREF(-)$ 接地，即模拟输入电压范围为0~+5V，对应的数字量输出为0x00~0xff。

4. ADC0809的转换工作原理

讨论接口设计前，先了解单片机如何控制ADC开始转换，如何得知转换结束以及如何读入转换结果的问题。

单片机控制0809进行A/D转换过程如下：首先由加到C、B、A上的编码决定选择ADC0809的某一路模拟输入通道，同时产生高电平加到0809的START引脚，开始对选中通道转换。



当转换结束时，ADC0809发出**转换结束EOC**（高电平）信号。当单片机读取转换结果时，需控制OE端为高电平，把转换完毕的数字量读入到单片机内。

单片机读取A/D转换结果可采用**查询方式**和**中断方式**。

查询方式是**检测EOC脚**是否变为高电平，如为高电平则说明转换结束，然后单片机读入转换结果。

中断方式是单片机启动ADC转换后，单片机执行其他程序。ADC0809转换结束后EOC变为高电平，EOC通过反相器向单片机发出中断请求，单片机响应中断，进入中断服务程序，在中断服务程序中读入转换完毕的数字量。很明显，中断方式效率高。

11.5.1 案例设计：单片机控制ADC0809进行A/D转换

【例11-4】采用查询方式控制ADC0809（Proteus元件库中没有ADC0809，可用库中ADC0808替代，与ADC0809性能完全相同，用法一样，只是在**非调整误差**方面**有所不同**(ADC0808为 $\pm 1/2\text{LSB}$ ，而ADC0809为 $\pm 1\text{LSB}$)进行A/D转换，原理电路见图11-10。

输入给ADC0809模拟电压可通过调节电位器RV1来实现，ADC0808将输入的模拟电压转换成二进制数字，并通过P1口输出，控制发光二极管亮与灭，来显示转换结果的二进制数字量。

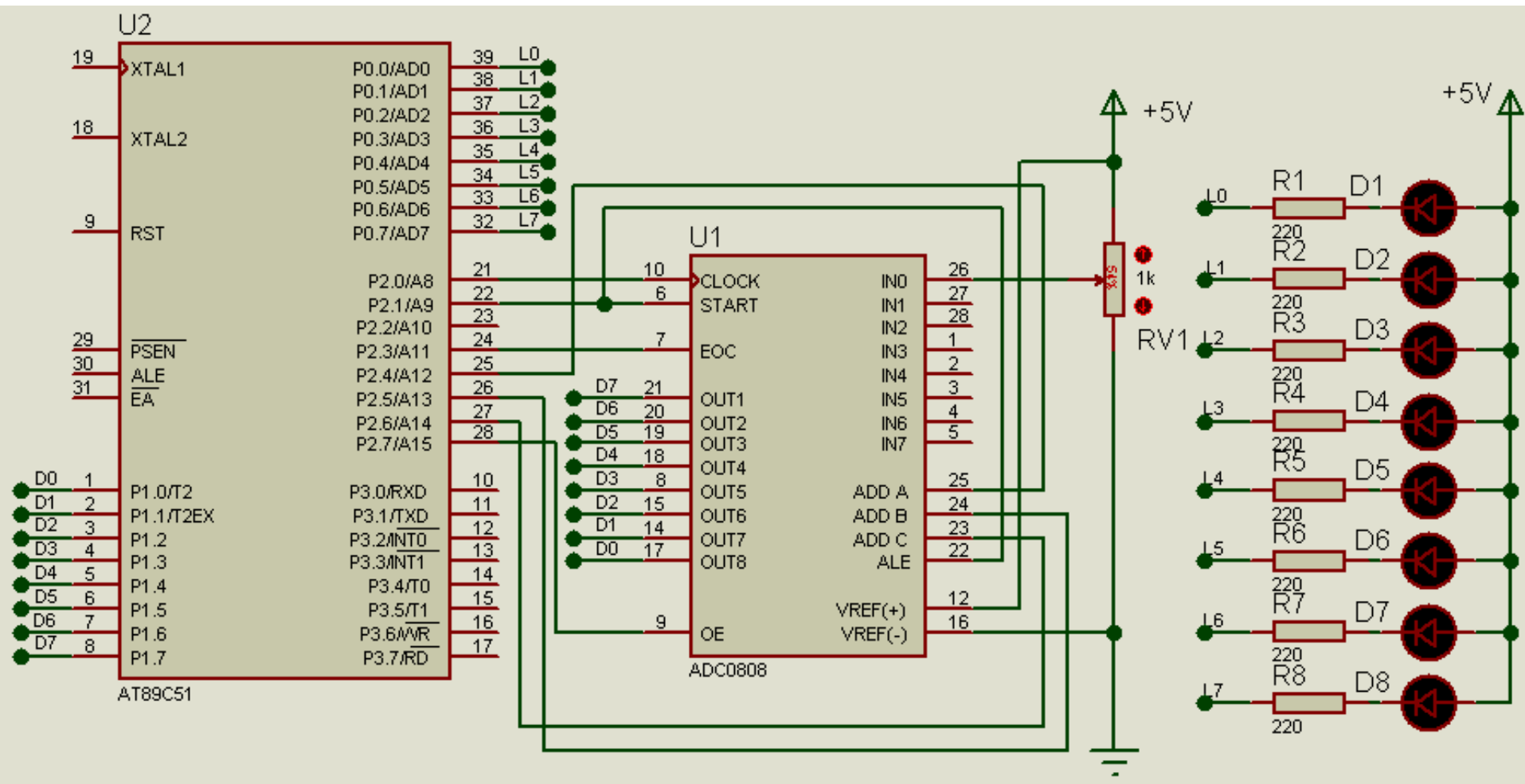


图11-10 单片机控制ADC0809进行转换

ADC0808转换一次约需100 μ s，采用查询方式，即使用**P2.3**来查询EOC脚电平，**判断A/D转换是否结束**。如果EOC脚为高，说明转换结束，单片机从P1口读入转换二进制的结果，然后把结果从P0口输出给8个发光二极管，发光二极管被点亮的位，对应转换结果“0”。

```
#include "reg51.h"

#define uchar unsigned char
#define uint unsigned int

#define LED P0
#define out P1

sbit start=P2^1;
sbit OE=P2^7;
sbit EOC=P2^3;
```

```
sbit CLOCK=P2^0;
```

```
sbit add_a=P2^4;
```

```
sbit add_b=P2^5;
```

```
sbit add_c=P2^6;
```

```
void main(void)
```

```
{
```

```
    uchar temp;
```

```
    add_a=0;add_b=0;add_c=0;
```

```
//选择ADC0809的通道0
```

```
    while(1)
```

```
    {
```

```
        start=0;
```

```
        start=1;
```

```
        start=0;
```

```
//启动转换
```

```
        while(1)
```

```
        {
```

```
            clock=!clock;if(EOC==1)break;} //等待转换结束
```

```
    OE=1;                //允许输出
    temp=out;             //暂存转换结果
    OE=0;                //关闭输出
    LED=temp;             //采样结果通过P0口输出到LED
}
}
}
```

A/D转换时须加基准电压，单独用高精度稳压电源，其电压变化要小于1LSB，这是保证转换精度的基本条件。否则当被转换的输入电压不变，而基准电压的变化大于1LSB，也会引起A/D转换器输出的数字量变化。

如用中断方式读取结果。可将EOC引脚与单片机P2.3脚断开，EOC引脚接反相器（例如74LS04）的输入，反相器输出接单片机的外部中断请求输入端（INTx* 脚），转换结束时，向单片机发出中断请求信号。可将本例接口电路及程序修改，采用中断方式来读取A/D转换结果。

11.5.2 案例：2路输入的数字电压表的设计

【例11-5】 设计一个单片机采用查询方式对2路模拟电压（0~5V）交替采集的数字电压表。原理电路与仿真见[图11-11](#)。

2路0~5V被测电压加到ADC0809**IN0**和**IN1**通道，进行A/D转换，两路输入电压的大小可通过**手动调节RV1**和**RV2**来实现。

本例将1.25V和2.5V作为两路输入的报警值，当通道**IN0**和**IN1**的电压分别超过1.25V和2.50V时，对应二进制数值分别为0x40和0x80。当A/D转换结果超过该数值时，将驱动发光二极管**D2**闪烁与蜂鸣器发声，表示超限。测得的电压交替显示在**LED**数码管上，同时也显示在两个虚拟电压表图标上，通过鼠标滚轮来放大虚拟电压表图标，可清楚地看到输入电压测量结果。

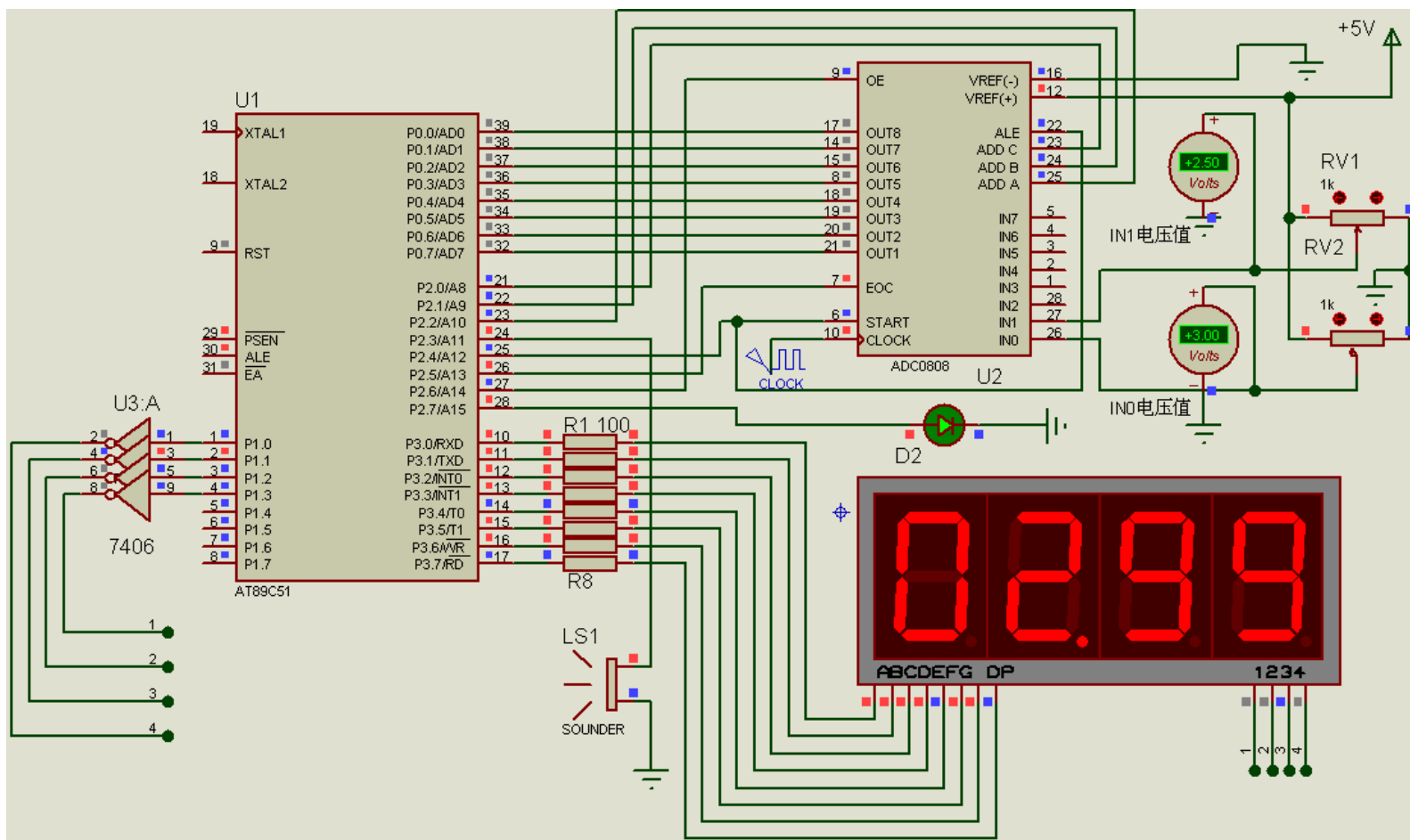


图11-11 查询方式的数字电压表电路原理图与仿真

ADC0809采用的基准电压为+5V。转换所得结果二进制数字addata所代表的电压的绝对值为 $(\text{addata} \div 256) \times 5\text{V}$ ，而若将其显示到小数点后两位，不考虑小数点存在（将其乘以100），其计算数值为：

$$(\text{addata} \times 100 \div 256) \times 5\text{V} \approx \text{addata} \times 1.96 \text{ V}$$

控制小数点显示在左边第二位数码管上，即为实际测量电压。

参考程序如下:

```
#include<reg51.h>

unsigned char
a[16]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x
79,0x71,},b[4],c=0x01;

sbit START=P2^4;
sbit OE=P2^6;
sbit EOC=P2^5;
sbit add_a=P2^2;
sbit add_b=P2^1;
sbit add_c=P2^0;
sbit led=P2^7;
sbit buzzer=P2^3;
```

```
void Delay1ms(unsigned int count)           //延时函数
```

```
{
    unsigned int i,j;
    for(i=0;i<count;i++)
        for(j=0;j<120;j++);
}
```

void show()

//显示函数

```
{  
    unsigned int r;  
    for(r=0;r<4;r++)  
    {  
        P1=(c<<r);  
        P3=b[r];  
        if(r==2)                //显示小数点  
        P3=P3|0x80;  
        Delay1ms(1);  
    }  
}
```

void main(void)

```
{  
    unsigned int addata=0,i;  
    while(1)
```

{

add_a=0; add_b=0;add_c=0;

//采集**第一路**信号

START=1;

//根据时序**启动**ADC0808

START=0;

while(EOC==0)

{

OE=1;

}

addata=P0;

if(addata>=0x40)

//大于1.25V时，则使用LED和蜂鸣器报警

{

for(i=0;i<=100;i++)

{

led=~led;

buzzer=~buzzer;

}

```
led=1;           //控制发光二极管VD2闪烁，发出光报警信号
buzzer=1;        //控制蜂鸣器发声，发出声音报警信号
}
else             //否则取消报警
{
    led=0;        //控制发光二极管VD2灭
    buzzer=0;     //控制蜂鸣器不发声
}
addata=addata*1.96; //将采得的二进制数转换成可读的电压值
OE=0;
b[0] =a[addata%10]; //显示到数码管上
b[1] =a[addata/10%10];
b[2] =a[addata/100%10];
b[3] =a[addata/1000];
```



```
for(i=0;i<=200;i++)
```

```
{
```

```
    show();
```

```
}
```

```
add_a=1;
```

```
//采集第二路信号
```

```
add_b=0;
```

```
add_c=0;
```

```
START=1;
```

```
// 启动ADC0808开始转换
```

```
    START=0;
```

```
while(EOC==0)
```

```
{
```

```
    OE=1;
```

```
}
```

```
addata=P0;
```



```
if(addata>=0x80)           //当大于2.5V时，则使用LED和蜂鸣器报警
{
    for(i=0;i<=100;i++)
    {
        led=~led;
        buzzer=~buzzer;
    }
    led=1;
    buzzer=1;
}
else                        //否则取消报警
{
    led=0;
    buzzer=0;
}
addata=addata*1.96;        //将采得的二进制数转换成可读的电压
```



```
OE=0;
```

```
b[0] =a[addata%10];
```

```
//显示到数码管上
```

```
b[1] =a[addata/10%10];
```

```
b[2] =a[addata/100%10];
```

```
b[3] =a[addata/1000];
```

```
for(i=0;i<=200;i++)
```

```
{
```

```
    show();
```

```
}
```

```
}
```

```
}
```

11.6 单片机扩展串行8位A/D转换器TLC549

串行A/D转换器与单片机连接具有占用I/O口线少的优点，使用较多，大有取代并行A/D转换器的趋势。下面介绍单片机扩展**串行8位** A/D转换器 TLC549的应用设计。

11.6.1 TLC549的特性及工作原理

TLC549是美国TI公司推出的价廉、高性能的带有SPI接口的8位 A/D转换器，其转换速度小于 $17\mu\text{s}$ ，最大转换速率为40kHz，内部系统时钟的典型值为4MHz，电源为3~6V。能方便与各种单片机通过SPI串行接口连接。

1. TLC549的引脚及功能

引脚见图11-12。

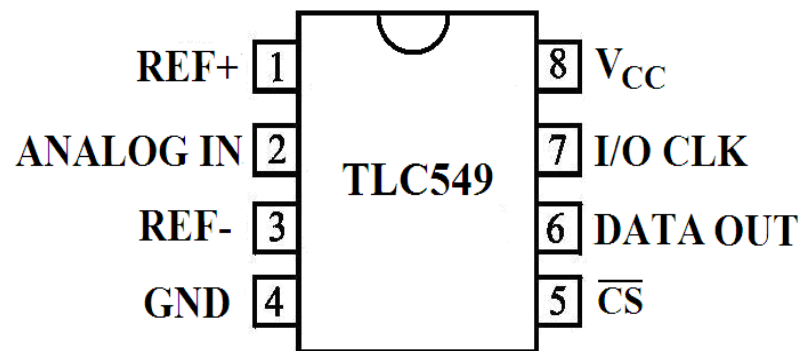


图11-12 TLC549的引脚

引脚功能如下：

- **CS***：片选端。
- **DATAOUT**：转换结果数据TTL电平串行输出端，输出时高位在前。
- **ANALOGIN**：模拟信号输入端， $0 \leq \text{ANALOGIN} \leq V_{CC}$ ，当
 $\text{ANALOGIN} \geq \text{REF+}$ 电压时，转换结果为全“1” (0xff)，
 $\text{ANALOGIN} \leq \text{REF-}$ 电压时，转换结果为全“0” (0x00)。
- **I/O CLOCK**：外接输入/输出时钟输入端，同于同步芯片的输入输出操作，无需与芯片内部系统时钟同步。
- **REF+**：正基准电压输入 $2.5V \leq \text{REF+} \leq V_{CC} + 0.1V$ 。
- **REF-**：负基准电压输入端， $-0.1V \leq \text{REF-} \leq 2.5V$ 。且： $(\text{REF+}) - (\text{REF-}) \geq 1V$ 。

- V_{CC} : 电源 $3V \leq V_{CC} \leq 6V$ 。
- GND: 地。

2. TLC549 工作时序

工作时序见图11-13。从图可知：

- (1) 串行数据中高位A7先输出，最后输出低位A0；
- (2) 在每一次I/O COLCK高电平期间DATA OUT 线上数据产生有效输出，每出现一次I/O COLCK，DATA OUT线就输出1位数据。一个周期出现8次I/O COLCK 信号并对应8位数据输出；

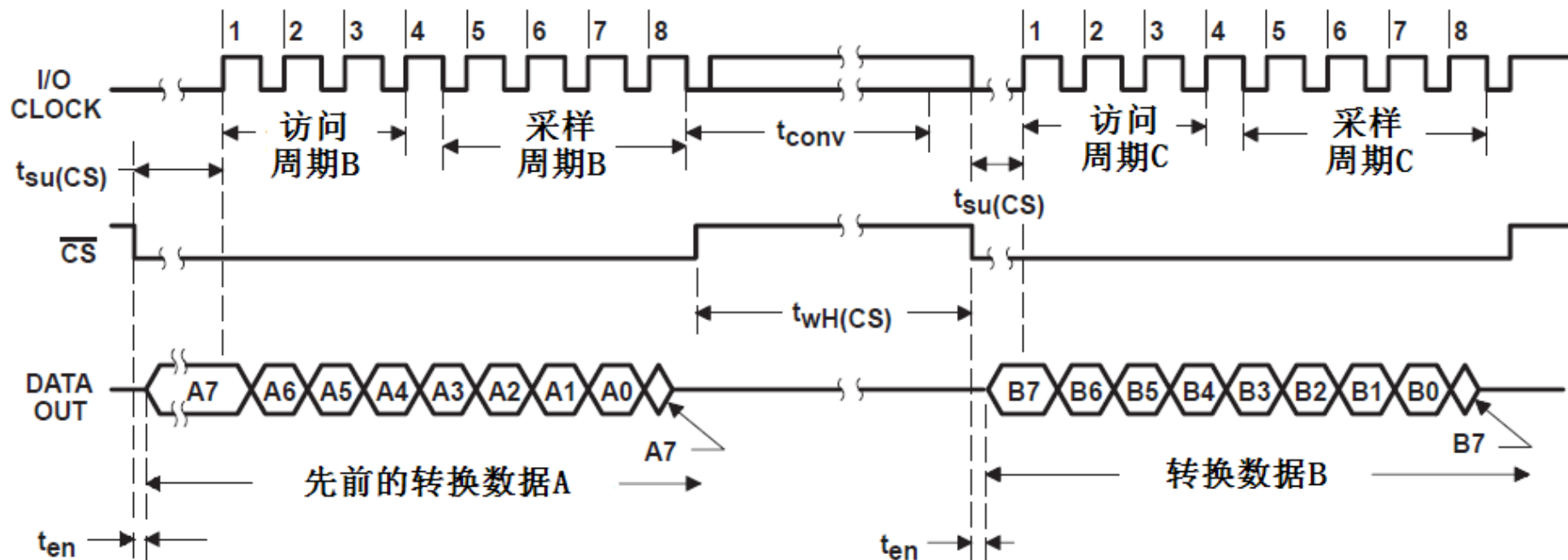


图11-13 TLC549的工作时序

(3) 在CS*变为低电平后，最高有效位 (A7) 自动置于DATA OUT 总线。其余7位 (A6~A0) 在前7个I/O CLOCK 下降沿由时钟同步输出。

B7~B0 以同样方式跟在其后；

(4) t_{su} 在片选信号CS*变低后，I/O COLCK 开始正跳变的最小时间间隔 $1.4\mu s$ ；

(5) t_{en} 是从CS*变低到DATA OUT 线上输出数据最小时间 ($1.2\mu s$) ；

(6) 只要I/O COLCK 变高就可读取DATA OUT 线上数据；

(7) 只有在CS*端为低时，TLC549 才工作；

(8) TLC549的A/D转换电路没有启动控制端，只要读取前一次数据后马上就可以开始新的A/D 转换。转换完成后就进入保持状态。TLC549 每次转换所需时间是 $17\mu s$ ，它开始于 变为低电平后I/OCLOCK 的第8个下降沿，没有转换完成标志信号。

当CS*变为低电平后，TLC549被选中，同时前次转换结果的最高有效位MSB（A7）自DATA OUT端输出，接着要求从I/O CLOCK端输入8个外部时钟信号，前7个I/O CLOCK信号的作用，是配合TLC549输出前次转换结果的A6~A0位，并为本次转换做准备：在第4个I/O CLOCK信号由高至低的跳变之后，片内采样/保持电路对输入模拟量采样开始，第8个I/O CLOCK信号的下降沿使片内采样/保持电路进入保持状态并启动A/D开始转换。转换时间为36个时钟周期，最大为17μs。直到A/D转换完成前的这段时间内，TLC549的控制逻辑要求：或者CS*保持高电平，或者I/O CLOCK时钟端保持36个系统时钟周期低电平。

由此可见，在TLC549的I/O CLOCK端输入8个外部时钟信号期间需要完成以下工作：读入前次A/D转换结果；对本次转换的输入模拟信号采样并保持；启动本次A/D转换开始。

11.6.2 案例设计：单片机扩展TLC549的设计

【例11-6】单片机控制串行8位A/D转换器**TLC549**进行A/D转换，原理电路与仿真结果见**图11-14**。

由电位计RV1提供给TLC549模拟量输入，通过调节RV1上的“+”、“-”端，改变输入电压值。编写程序将模拟电压量转换成二进制数字量，本例用**P0口**输出控制**8个发光二极管的亮与灭**显示转换结果的**二进制码**，也可通过**LED数码管**将转换完毕的数字量以**16进制数**形式显示出来。

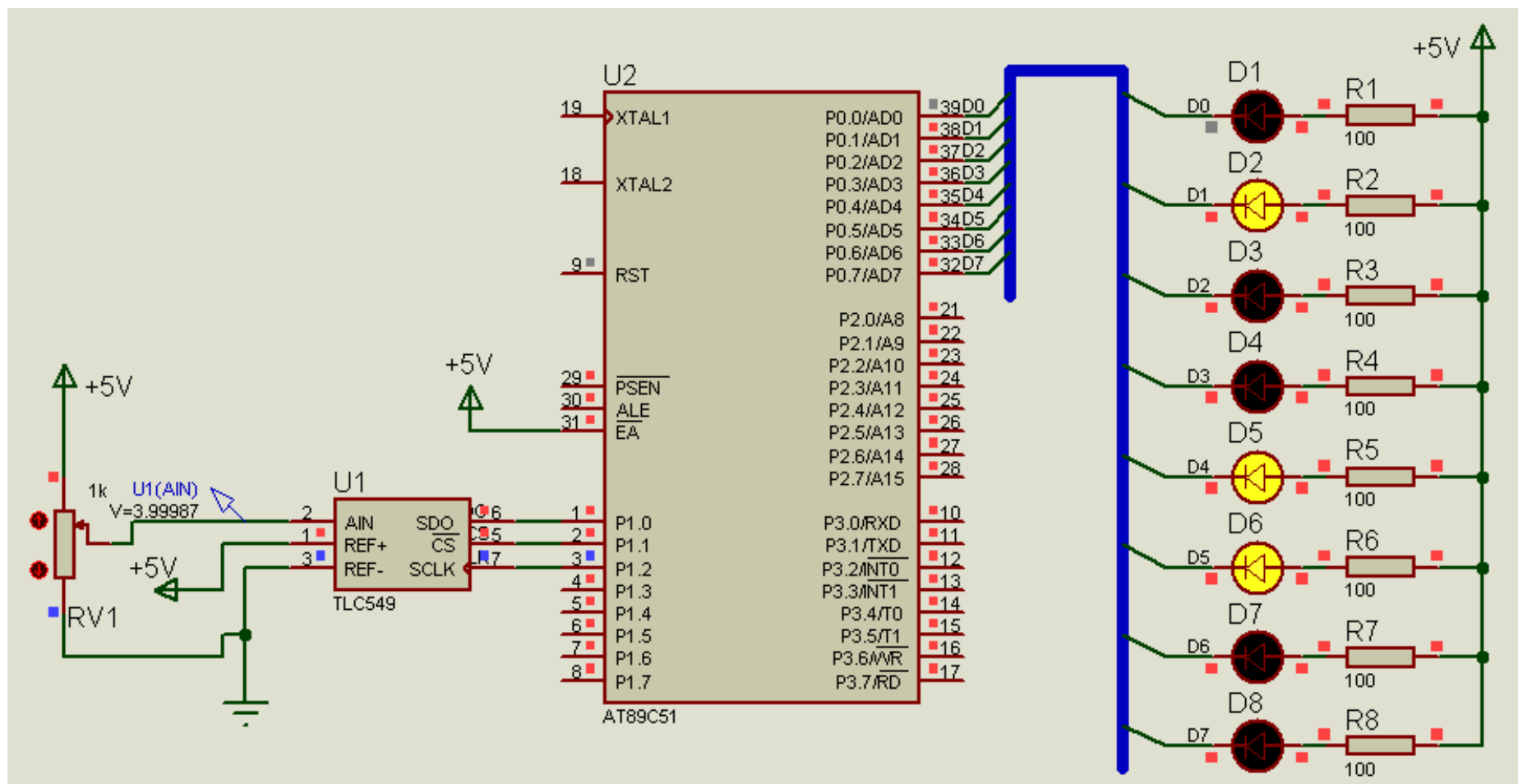


图11-14 单片机与TLC549接口的原理电路

参考程序如下：

```
#include<reg51.h>
```

```
#include<intrins.h>
```

//包含_nop_()函数头文件

```
#define uchar unsigned char
```

```
#define uint unsigned int
```

```
#define led P0
```

```
sbit sdo=P1^0;      //定义P1.0与TLC549的SD0脚（即5脚DATA OUT）连接
```

```
sbit cs=P1^1;      //定义P1.1与TLC549的 脚连接
```

```
sbit sclk=P1^2;    //定义P1.2与TLC549的SCLK脚（即7脚I/O CLOCK）连接
```

```
void delayms(uint j)      //延时函数
```

```
{
```

```
    uchar i=250;
```

```
    for(;j>0;j--)
```

```
    {
```

```
        while(--i);
```

```
        i=249;
        while(--i);
        i=250;
    }
}
```

```
void delay18us(void)                //延时18μs函数
{
    _nop_();_nop_();_nop_();_nop_();_nop_();_nop_();_nop_();_nop_();_nop_
    ();
    _nop_();_nop_();_nop_();_nop_();_nop_();_nop_();_nop_();
    nop_();_nop_();
}
```

```
uchar convert(void)
{
    uchar i, temp;
    cs=0;
    delay18us();
```

```
for(i=0;i<8;i++)
{
    if(sdo==1) temp=temp|0x01;
    if(i<7) temp=temp<<1;
    sclk=1;
    _nop_(); _nop_(); _nop_(); _nop_();
    sclk=0;
    _nop_(); _nop_();
}
cs=1;
return(temp);
}
```

```
void main()
{
    uchar result;
    led=0;
    cs=1;
```

```
sclk=0;
sdo=1;
while(1)
{
    result=convert();
    led=result;           //转换结果从P0口输出驱动LED
    delayms(1000);
}
}
```

由于TLC549 转换时间应大于 $17\mu\text{s}$ ，本例采用延时操作方案，延时时间大约 $18\mu\text{s}$ ，每次读取转换数据时间大于 $17\mu\text{s}$ 即可。

11.7 AT89S51扩展12位串行ADC-TLC2543的设计

TLC2543是美国TI公司的12位串行SPI接口的A/D转换器，转换时间为 $10\mu\text{s}$ 。下面首先介绍串行A/D转换器TLC2543的特性及工作原理。

11.7.1 TLC2543的特性及工作原理

TLC2543片内有1个14路模拟开关，用来选择11路模拟输入以及3路内部测试电压中的1路进行采样。为了保证测量结果的准确性，该器件具有3路内置自测试方式，可分别测试“REF+”高基准电压值，“REF-”低基准电压值和“REF+/2”值，该器件的模拟量输入范围为REF+~REF-，一般模拟量的变化范围为0~+5V，所以此时REF+脚接+5V，REF-脚接地。

由于TLC2543与单片机接口简单，且价格适中，分辨率较高，因此在智能仪器仪表中有着较为广泛应用。

1. TLC2543的引脚

TLC2543的引脚见图11-15，各引脚功能如下。

- **AIN0~AIN10**：11路模拟量输入端。
- **CS***：片选端。
- **DATAINPUT**：串行数据输入端。由4位的串行地址输入来选择模拟量输入通道。

- **DATA OUT:** A/D转换结果的三态串行输出端。为高时处于高阻抗状态，为低时处于转换结果输出状态。
- **EOC:** 转换结束端。
- **I/O CLOCK:** I/O时钟端。
- **REF+:** 正基准电压端。基准电压的正端（通常为V_{cc}）被加到REF+，最大的输入电压范围为加在本引脚与REF-引脚的电压差。
- **REF-:** 负基准电压端。基准电压低端（通常为地）加此端。
- **V_{cc}:** 电源。
- **GND:** 地。

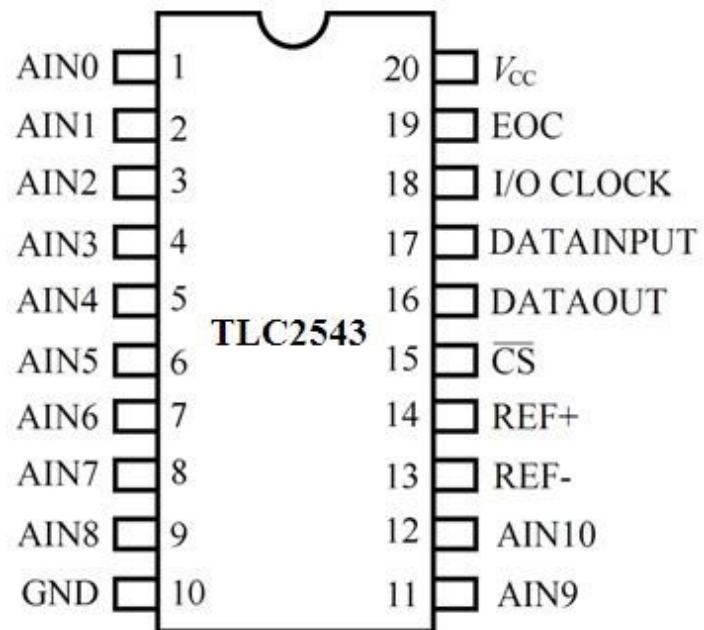


图11-15 TLC2543引脚

2. TLC2543工作过程

工作过程分为两个周期：**I/O周期**和**实际转换周期**。

(1) I/O周期

I/O周期由外部提供的I/O CLOCK定义，延续8、12或16个时钟周期，取决于选定的输出数据长度。器件进入I/O周期后同时进行**两种操作**。

① TLC2543的工作时序如**图11-16**所示。在I/OCLOCK的前8个脉冲的上升沿，以MSB前导方式从DATAINPUT端输入8位数据到输入寄存器。其中前4位为模拟通道地址，控制14通道模拟多路器从11个模拟输入和3个内部自测电压中，选通1路到采样保持器，该电路从第4个I/OCLOCK

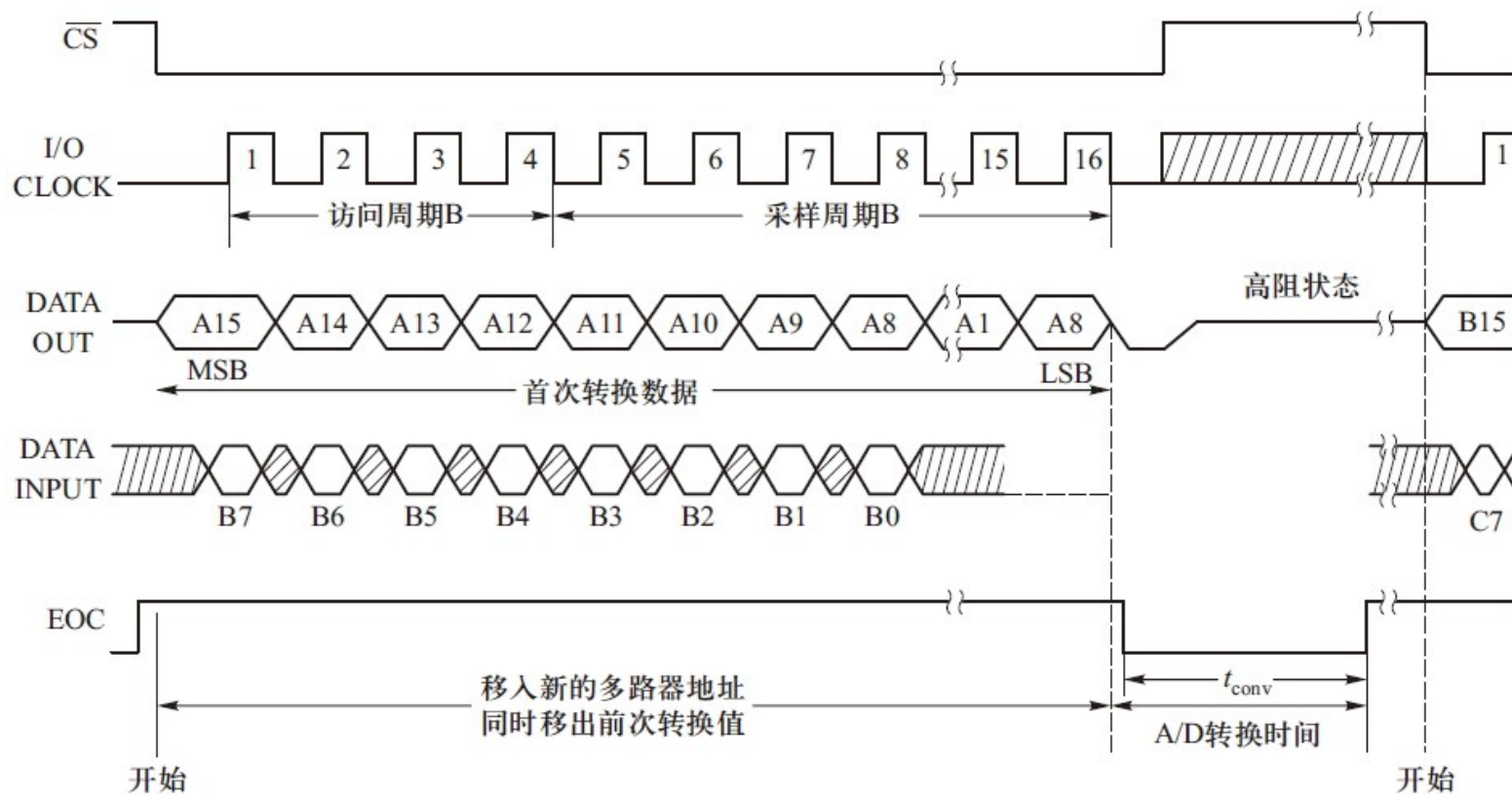


图11-16 TLC2543的工作时序

脉冲下降沿开始，对所选的信号进行采样，直到最后一个I/O CLOCK脉冲下降沿。

I/O脉冲时钟个数与输出数据长度（位数）有关，输出数据的长度由输入数据的D3、D2可选择为8位、12位或16位。当工作于12位或16位时，在前8个脉冲之后，DATAINPUT无效。

② 在DATA OUT端串行输出8位、12位或16位数据。当保持为低时，第1个数据出现在EOC的上升沿，若转换由控制，则第1个输出数据发生在 的下降沿。这个数据是前1次转换的结果，在第1个输出数据位之后的每个后续位均由后续的I/OCLOCK脉冲下降沿输出。

(2) 转换周期

在I/O周期最后一I/OCLOCK脉冲下降沿后，EOC变低，采样值保持不变，转换周期开始，片内转换器对采样值进行逐次逼近式A/D转换，其工作由与I/OCLOCK同步的内部时钟控制。

转换结束后EOC变高，转换结果锁存在输出数据寄存器中，待下一I/O周期输出。I/O周期和转换周期交替进行，从而可减少外部的数字噪声对转换精度影响。

3. TLC2543命令字

每次转换都必须向TLC2543写入命令字，以便确定被转换信号来自哪个通道，转换结果用多少位输出，输出的顺序是高位在前还是低位在前，输出结果是有符号数还是无符号数。命令字写入顺序是高位在前。命令字格式如下：

通道地址选择 (D7~D4)	数据的长度 (D3~D2)	数据的顺序 (D1)	数据的极性 (D0)
----------------	---------------	------------	------------

(1) 通道地址选择位

选择输入通道。0000~1010分别是11路模拟量AIN0~AIN10的地址；地址1011、1100和1101所选择的自测试电压分别是 $((VREF+) - (VREF-)) / 2$ 、VREF-、VREF+。1110是掉电地址，选掉电后，TLC2543处于休眠状态，此时电流小于20 μ A。

(2) 数据长度 (D3~D2) 位用来选择转换的结果用多少位输出。D3D2为
x 0: 12位输出; D3D2为01: 8位输出; D3D2为11: 16位输出。

(3) 数据的顺序位 (D1) 用来选择数据输出的顺序。D1=0, 高位在前;
D1=1, 低位在前。

(4) 数据的极性位 (D0) 用来选择数据的极性。D0=0, 数据是无符号数
; D0=1, 数据是有符号数。

11.7.2 案例设计：单片机扩展TLC2543的设计

介绍单片机与TLC2543接口设计及软件编程。

【例11-7】单片机与TLC2543接口电路见图11-17，编写程序对AIN2模拟通道进行数据采集，结果在数码管上显示，输入电压调节通过调节RV1来实现。

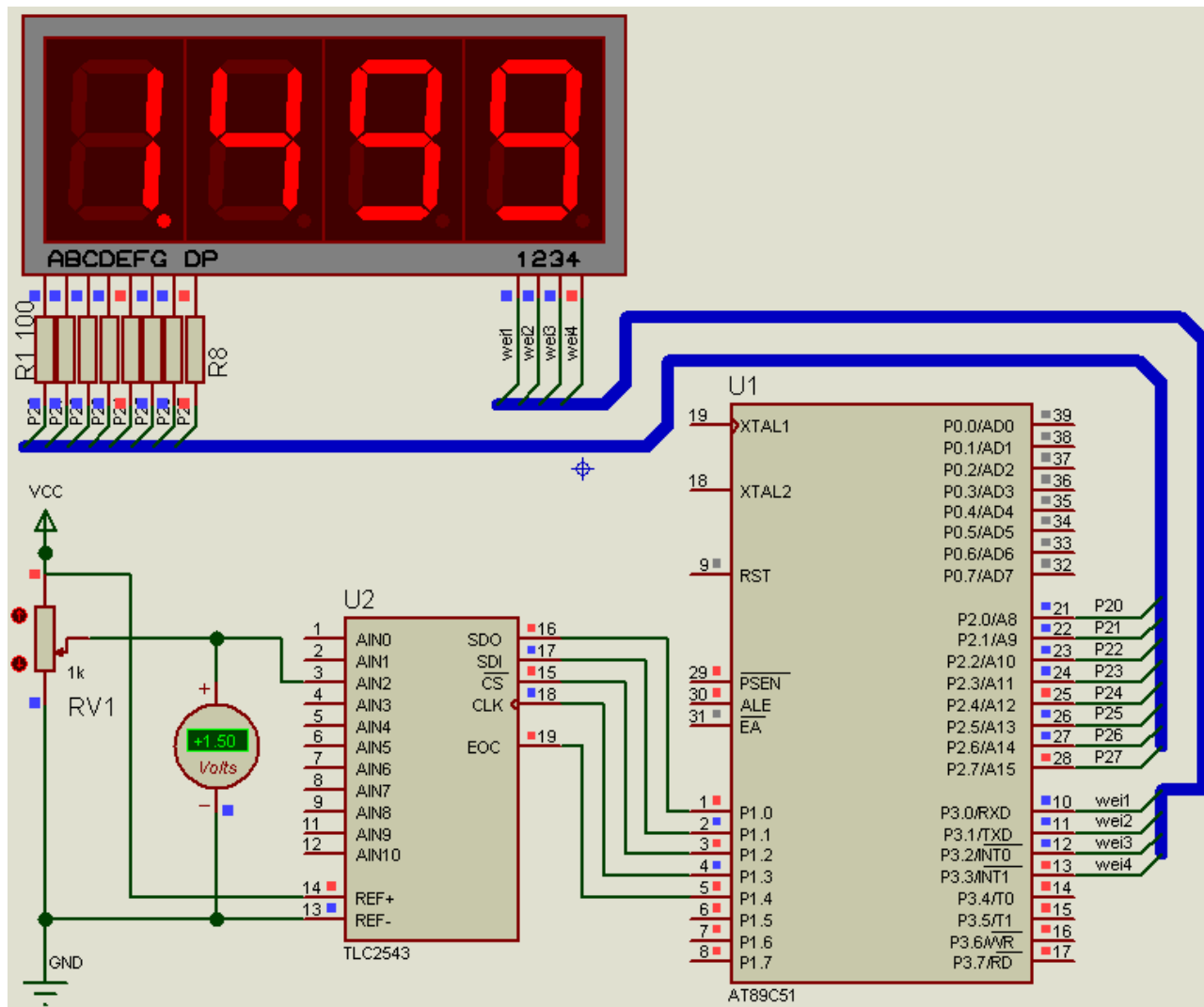


图10-17 单片机与TLC2543的接口电路

TLC2543与单片机接口采用**SPI串行接口**（SPI：串行外设接口，已在第10章介绍），由于AT89S52不带SPI接口，须采用软件与I/O口线相结合，模拟SPI接口时序。TLC2543三个控制输入端分别为I/O CLOCK（18脚，输入/输出时钟）、DATAINPUT（17脚，4位串行地址输入端）以及CS*（15脚，片选），分别由单片机P1.3、P1.1和P1.2控制。转换结果（16脚）由单片机P1.0脚串行接收，AT89S52将命令字通过P1.1引脚串行写入到TLC2543的输入寄存器中。

片内**14通道选择开关**可选择**11个模拟输入**中的任一路或**3个内部自测电压**中的一个并且自动完成采样保持。

转换结束后“**EOC**”变高，**转换结果**由三态输出端“**DATA OUT**”输出。

采集的数据为12位无符号数，采用高位在前的输出数据。写入TLC2543的命令字为0xa0。

由TLC2543的工作时序，命令字写入和转换结果输出是同时进行的，即在读出转换结果的同时也写入下一次的命令字，采集11个数据要进行12次转换。第1次写入的命令字是有实际意义的操作，但是第1次读出的转换结果是无意义的操作，应丢弃；而第11次写入的命令字是无意义的操作，而读出的转换结果是有意义的操作

参考程序如下:

```
#include <reg51.h>
#include <intrins.h>           //包含_nop_()函数的头文件
#define uchar unsigned char
#define unit unsigned int
unsigned char code table[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};
unit ADresult[11];           //11个通道的转换结果单元

sbit DATOUT=P1^0;             //定义P1.0与DATA OUT相连
sbit DATIN=P1^1;              //定义P1.1与DATA INPUT相连
sbit CS=P1^2;                 //定义P1.2与 端相连
```

```
sbit IOCLK=P1^3;
```

//定义P1.3与I/O CLOCK相连

```
sbit EOC=P1^4;
```

//定义P1.4与EOC引脚相连

```
sbit wei1=P3^0;
```

```
sbit wei2=P3^1;
```

```
sbit wei3=P3^2;
```

```
sbit wei4=P3^3;
```

```
void delay_ms(unit i)
```

```
{
```

```
    int j;
```

```
    for(; i>0; i--)
```

```
        for(j=0; j<123; j++);
```

```
}
```

```
unit getdata(uchar channel)
```

```
// getdata()为获取转换结果函数， channel  
//为通道号
```

```
{
```

```
    uchar i,temp;
```

```
    unit read_ad_data=0;
```

```
// 分别存放采集的数据，先清0
```

```
    channel=channel<<4;
```

```
// 结果为12位数据格式，高位导前，单极性
```

```
// ××××0000
```

```
    IOCLK=0;
```

```
    CS=0;
```

```
// 下跳沿，并保持低电平
```

```
    temp=channel;
```

```
// 输入要转换的通道
```

```
    for(i=0;i<12;i++)
```

```
    {
```

```
        if(DATOUT) read_ad_data=read_ad_data|0x01;//读入转换结果
```

```
        DATIN=(bit)(temp&0x80);
```

```
//写入方式/通道命令字
```

```
        IOCLK=1;
```

```
//IOCLK上跳沿
```

```
        _nop_();_nop_();_nop_();
```

```
//空操作延时
```

```
        IOCLK=0;
```

```
//IOCLK下跳沿
```



```
_nop_0);_nop_0);_nop_0);
```

```
temp=temp<<1;
```

//左移1位，准备发送方式通道控制字下一位

```
read_ad_data<=<=1;
```

//转换结果左移1位

```
}
```

```
CS=1;
```

// CS上跳沿

```
read_ad_data>>=1;
```

// 抵消第12次左移，得到12位转换结果

```
return(read_ad_data);
```

```
}
```

```
void dispaly(void)
```

// 显示函数

```
{
```

```
    uchar qian,bai,shi,ge;
```

// 定义千、百、十、个位

```
    unit value;
```

```
    value=ADresult[2]*1.221; /*5000/4095
```

```
    qian=value%10000/1000;
```

```
    bai=value%1000/100;
```

```
    shi=value%100/10;
```

```
    ge=value%10;
```



```
wei1=1;
```

```
P2=table[qian]-128;
```

```
delay_ms(1);
```

```
wei1=0;
```

```
wei2=1;
```

```
P2=table[bai];
```

```
delay_ms(1);
```

```
wei2=0;
```

```
wei3=1;
```

```
P2=table[shi];
```

```
delay_ms(1);
```

```
wei3=0;
```

```
wei4=1;
```

```
P2=table[ge];
```

```
delay_ms(1);
```

```
wei4=0;
```

```
}
```



```
main(void)
```

```
{  
    ADresult[2]=getdata(2);           //启动2通道转换，第1次转换结果无意义  
    while(1)  
    {  
        _nop_(); _nop_(); _nop_();  
        ADresult[2]=getdata(2);       //读取本次转换结果，同时启动下次转换  
        while(!EOC);                 //判是否转换完毕，未转换完则循环等待  
  
        dispaly();  
    }  
}
```

由本例见，AT89S52单片机与TLC2543接口电路十分简单，只需用软件控制4条I/O引脚，按规定时序对TLC2543进行访问即可。

11.8 数据采集的软件滤波

对于实时数据采集系统，为了消除传感器通道中的干扰信号，可对A/D转换后的所采集的数据进行软件滤波抑制，消除叠加在输入信号上噪声的影响，剔除虚假信号,求取真值。

由于软件设计灵活，节省硬件资源，所以软件滤波方法已得到较为广泛的使用。下面介绍几种常见的软件滤波的方法。

11.8.1 算术平均滤波

算术平均滤波法就是对一点数据连续取 n 个值进行采样，然后求算术平均。这种方法一般适用于具有随机干扰的信号的滤波。这种信号的

特点是有一个平均值，信号在某一数值范围附近上下波动。

这种滤波法，当 n 值较大时，信号的平滑度高，但灵敏度低；当 n 值较小时，平滑度低，但灵敏度高。

应视具体情况选取 n 值，既要节约时间，又要滤波效果好。对于一般流量测量，通常取经验值 $n=12$ ；若为压力测量，则取经验值 $n=4$ 。一般情况下，经验值 n 取3~5次平均即可。

【例11-8】求 N 点采样值的算术平均滤波。



```
#define N 12
```

// **N值**可根据具体实际情况进行选取

```
char filter( )
```

```
{
```

```
    char ADCRESULT;
```

//ADCRESULT中存的是采集的数据

```
    int sum=0;
```

```
    for(count=0;count<N;count++)
```

```
    {
```

```
        sum+=ADCRESULT;
```

// 求**N点**采样值的累加和

```
    }
```

```
    return(char) (sum/N);
```

// 求**N点**采样值累加和的平均值并返回

```
}
```

11.8.2 滑动平均滤波

上面介绍的算术平均滤波法，每计算一次数据需要测量 n 次。对于测量速度较慢或要求数据计算速度较快的实时控制系统来说，该方法无效。下面介绍一种只需测量一次，就能得到当前算术平均值的方法——滑动平均滤波法。

滑动平均滤波法是把 n 个采样值看成一个队列，队列的长度为 n ，每进行一次采样，就把最新的采样值放入队尾，而扔掉原来队首的一个采样值，这样在队列中始终有 n 个“最新”采样值。对队列中的 n 个采样值进行平均，就可以得到新的滤波值。

滑动平均滤波法对周期性干扰有良好的抑制作用，平滑度高，灵敏度低；但对偶然出现的脉冲性干扰的抑制作用差，不易消除由此引起的采样值的偏差。

因此它不适用于脉冲干扰比较严重的场合。通常，观察不同 n 值下滑动平均的输出响应，据此选取 n 值，以便既少占有时间，又能达到最好的滤波效果，其工程经验值参考如下：

参数	温度	压力	流量	液面
n 值	1~4	4	12	4~12

【例11-9】 N点滑动平均滤波。

```
#define N 12                                     // N的值可根据具体实际情况进行选取
char value_buf[N];
char i=0;
char ADCRESULT;

char filter()
{
    char count;
    int sum=0;
    value_buf[i++]=ADCRESULT; // ADCRESULT中存放的是A/D转换结果
    if(i==N)i=0;
    for(count=0;count<N;count++)
        sum=value_buf[count];
    return(char) (sum/N);
}
```

11.8.3 中位值滤波

就是对某一被测参数接连采样 n 次（一般 n 取奇数），然后把 n 次采样值从小到大排列（升序法），或从大到小排列（降序法），取中间值为本次采样值。

中位值滤波能有效地克服因偶然因素引起的波动干扰。对温度、液位等变化缓慢的被测参数采用此方法能收到良好的滤波效果。但对于流量、速度等快速变化的参数一般不宜采用中位值滤波法。

最常用的数据排序算法是冒泡法。冒泡法是相邻数互换的排序方法，因其过程类似水中气泡上浮，故称冒泡法。排序时，从前向后进行相邻两个数的比较，如果数据的大小次序与要求的顺序不符时，就将两个数互换；否则，顺序符合要求就不互换。以升序排序法为例，通过相邻数互换，115

使小数向前移，大数向后移。如此从前向后进行一次次相邻数互换（冒泡），就会把这批数据的最大数排到最后，次大数排在倒数第二的位置，从而实现一批数据由小到大的排列。

对于 n 个数，理论上应进行 $(n-1)$ 次冒泡才能完成排序，但实际上有时不到 $(n-1)$ 次就已完成排序。

【例11-10】采用冒泡法排序的中位值滤波。

```
#define N 11
```

```
char filter()
```

```
{
```

```
    char ADCRESULT;
```

```
    char value_buf[N];
```

```
char count,i,j,temp;
for(count=0;count<N;count++)
{
    value_buf[count]=ADCRESULT; // 将A/D转换结果赋给value_buf[count]
    delay ( ) ;                  // 延时
}
for(j=0;j<N-1;j++)              // (N-1) 次冒泡
{
    for(i=0;i<N-j;i++)
    {
        if(value_buf>value_buf[i+1]) // 采用升序进行冒泡
        {
            temp=value_buf;
            value_buf=value_buf[i+1];
            value_buf[i+1]=temp;
        }
    }
}
```

```
}  
    return value_buf[(N-1)/2];           // 求得中间值  
}
```

11.8.4 防脉冲干扰滤波

前面介绍的算术平均与滑动平均滤波法，在脉冲干扰比较严重的场合，则干扰将会“平均”到结果中去，故上述两种平均值法不易消除由于脉冲干扰而引起的误差。这时可采用去极值平均值滤波法。

思想：连续采样 n 次后累加求和，同时找出其中的最大值与最小值，再从累加和中减去最大值和最小值，按 $n-2$ 个采样值求平均，即可得到有效采样值。这种方法类似于体育比赛中的去掉最高、最低分，再求平均分的评分办法。

为使平均滤波算法简单， $n-2$ 应为2、4、6、8或16，故 n 常取4、6、8、10或18。

具体做法有两种：对于快变参数，先连续采样 n 次，然后再处理，但要在RAM中开辟 n 个数据的暂存区；对于慢变参数，可一边采样，一边处理，而不必在RAM中开辟数据暂存区。实践中，为了加快测量速度，一般 n 不能太大，常取为4。它具有计算方便速度快，存储量小等优点。

【例11-11】防脉冲干扰的数字滤波，在连续4次数据采集后，去掉最大值和最小值，然后计算中间两个数据的平均值。



```
#define N 4
```

```
char filter( )
```

```
{
```

```
    char ADCRESULT;
```

```
    char count,i,j;
```

```
    char value_buf[N];
```

```
    int sum=0;
```

```
    for(count=0;count<N;count++)
```

```
    {
```

```
        value_buf[count]=ADCRESULT;        // 将转换结果赋给
```

```
value_buf[count]
```

```
        delay ( ) ;
```

```
    }
```

```
    for(j=0;j<N-1;j++)
```

```
    {
```

```
        for(i=0;i<N-j;i++)
```

```
        {
```

```
            if(value_buf> value_buf[i+1])
```

```
temp=value_buf;  
value_buf[i+1]=temp
```

```
}
```

```
}
```

```
}
```

```
for(count=1;count<N-1;count++)
```

```
sum+= value[count];
```

```
// 求N-2点采样值的累加和
```

```
return(char) (sum/ N-2);
```

```
// 求N-2点采样值的平均值
```

```
}
```

《END》